Quantifying the advantages of applying quantum approximate algorithms to portfolio optimisation

Haomu Yuan,[1,*] Christopher K. Long,[1] Hugo V. Lepage,[1] and Crispin H. W. Barnes[1]

[1] *Cavendish Laboratory, Department of Physics, University of Cambridge, Cambridge, CB3 0US, United Kingdom*

(Dated: October 6, 2026)

We present a quantum algorithm for portfolio optimisation. Specifically, We present san end-to-end quantum approximate optimisation algorithm to solve the discrete global minimum variance portfolio model. This model finds a portfolio of risky assets with the lowest possible risk contingent on the number of traded assets being discrete. We provide a complete pipeline for this model and analyse its viability for noisy intermediate-scale quantum computers. We design initial states, a cost operator, and ansätze within a binary encoding. Further, we perform numerical simulations to analyse several optimisation routines, including layerwise optimisation, utilising constrained optimisation by linear approximation and dual annealing. Finally, we consider the impacts of thermal relaxation and stochastic measurement noise. We find dual annealing with a layerwise optimisation routine provides the most robust performance. We observe that realistic thermal relaxation noise levels preclude quantum advantage. However, stochastic measurement noise will dominate when hardware sufficiently improves. Within this regime, we numerically demonstrate a favourable scaling in the number of shots required to obtain the global minimum—an indication of quantum advantage in portfolio optimisation.

Keywords: QAOA, portfolio optimisation, quantum finance, DGMVP

arXiv:2410.16265v2 [quant-ph] 5 Oct 2026

I. INTRODUCTION

Variational quantum algorithms promise to be powerful tools for approximate optimisation of both classical and quantum problems [1–3]. The quantum approximate optimisation algorithm (QAOA) is one of the most well-known quantum approximate optimisers. It was first introduced to solve binary optimisation problems, MaxCut and MaxSAT, and was then applied to problems in chemistry, quantum field theory, and portfolio optimisation [4–9]. Compared with other applications, quantum portfolio optimisation is arguably the most successful because it relates directly to real-world hedge funds and investment management. Other quantum algorithms have been developed tackling portfolio optimisation with simple models [10–13]. However, many still require validation in the presence of quantum noise and stochastic noise using experiment or simulation. The QAOA has developed applications on more sophisticated portfolio optimisation models with a substantial body of simulations and benchmarks [14–17]. Experiments on current quantum devices suggest that QAOA is capable of reaching a regime where it could challenge the best classical algorithms [18].

After Markowitz introduced the mean-variance portfolio model, portfolio optimisation became an important research field in financial management [19, 20]. However, any departure of the asset return distribution from normality can significantly affect the mean-variance approximation to the optimal portfolio allocation [21]. The complexity and interdependence of global economies make it challenging to predict economic stability, driving investors to pay more attention to portfolio risk. The global minimum variance portfolio (GMVP) is a known risk-focus portfolio that yielded superior results in many empirical studies [22–25]. However, trading in a practical scenario is usually quantified by discrete numbers of assets because financial products such as stocks, options and futures are not usually infinitely divisible. The discrete GMVP (DGMVP) is the discretisation of the GMVP, in which the number of traded assets is discrete. The DGMVP is a special case of integer quadratic programming (IQP). As IQP is well known to be NP-hard [26], a more efficient method to estimate the DGMVP is of great interest to investors and quantitative analysts.

A mathematical summarisation of the DGMVP model can be written as

$$\underset{\boldsymbol{w}}{\operatorname{argmin}}\ f(\boldsymbol{w}) = \boldsymbol{w}^\intercal \boldsymbol{\Sigma} \boldsymbol{w} \tag{1}$$

$$\text{s.t.}\quad \boldsymbol{w}^\intercal \mathbf{1}_n = 1, \tag{2a}$$

$$w_i \in [0, 1], \tag{2b}$$

$$w_i / a \in \mathbb{Z},\ i = 1, 2, \ldots, n \tag{2c}$$

* Haomu Yuan hy374@cam.ac.uk

where $\boldsymbol{w} = (w_1, w_2, \ldots, w_n)^\intercal$ is an n-dimensional vector of weights for assets $1, \ldots, n$, such as stocks, futures, and options, $\mathbf{1}_n$ is an n-dimensional ones vector, and a is the unit trading lot for assets. The $n \times n$ covariance matrix $\boldsymbol{\Sigma}$ and its elements σ_{ij} quantify the risk between each pair of assets. The first constraint Eq. (2a) is the budget constraint, the second constraint Eq. (2b) ensures no short investments, and the third constraint Eq. (2c) is the discretisation. In this article, we will present, to the best of our knowledge, the first end-to-end DGMVP solver based on the QAOA. We design a variational QAOA ansatz and quantify the use of quantum resources. The simulation analysis gives insight into solving other portfolio models and constrained mixed-IQP with quantum approximate algorithms in the future.

The rest of this article is structured as follows. In Section II, we review the original QAOA and general QAOA methods. In Section III, we outline a set of QAOA ansatz designs and compare their performance in solving the DGMVP model. In Section IV, we compare different optimisers and optimisation methods to get the most efficient optimisation strategy. In Section V, we analyse the scaling of the quantum algorithm performance as a function of the asset number and the coarseness of the discretisation and compare it with classical scaling. In Section VI, we add quantum decoherence noise and apply a post-selection method to improve the approximation. Finally, we conclude in Section VII.

II. REVIEW OF QAOA

Farhi *et al.* first introduced the quantum approximate optimisation algorithm (the original QAOA) to solve the combinatorial graph problem MaxCut and showed potential advantages [4]. The original QAOA was then generalised by the Quantum Alternating Operator Ansatz (the general QAOA) by redefining a family of cost and mixing operators [5]. To avoid ambiguity, we use the QAOA to represent the general QAOA in the rest of this article, which typically consists of:

1. **Initial state preparation.** We start with a pre-processing step to adjust the zero register to a desired state such as

$$|\,\mathrm{Init}\rangle = \hat{U}_I |0\rangle^{\otimes n}, \tag{3}$$

where $\hat{U}_I$ is a pre-processing operator acting on the $|0\rangle^{\otimes n}$ state. For example, in the MaxCut problem [4], a Hadamard transformation is used to prepare the initial state:

$$\hat{U}_I = \hat{\mathrm{H}}^{\otimes n}, \quad |\,\mathrm{Init}\rangle = \hat{\mathrm{H}}^{\otimes n}|0\rangle^{\otimes n}, \tag{4}$$

where $\hat{\mathrm{H}}$ is a Hadamard gate. A good initial state will increase the performance of quantum optimisation. Other initial states include the Hartree-Fock [27, 28], Dicke [29], and warm-started states [6, 30]. The warm-started preparation method indicates a strong advantage and beats the GW approximation in solving the MaxCut problem in a low-depth circuit [30].

2. **Cost operator design.** The cost operator $\hat{U}(\hat{C}, \gamma)$ is defined such that

$$\hat{U}(\hat{C}, \gamma) = e^{-i\gamma\hat{C}}, \tag{5}$$

where γ is the phase parameter and $\hat{C}$ is the cost Hamiltonian. The parameter γ is usually restricted by a range. For example, $\gamma \in [0, 2\pi]$ in the MaxCut problem. The cost Hamiltonian $\hat{C}$ encodes a cost function f, such as the one in Eq. (1), as a series of Pauli-Z strings such that

$$\hat{C}|z\rangle = f(z)|z\rangle, \tag{6}$$

where $|z\rangle$ is a computational basis state. The cost operator is sometimes named a phase operator because it only generates a phase if acting on a state from the Pauli-Z eigenbasis.

3. **Mixing operator design.** A mixing operator is another parameterised operator alternating with the cost operator. In the MaxCut problem, the authors use a mixing operator $\hat{U}(\hat{B}, \beta)$ with $\hat{B}$ defined by Pauli-X operators:

$$\hat{B} = \sum_{j=1}^{n} \hat{X}_j, \quad \hat{U}(\hat{B}, \beta) = e^{-i\beta\hat{B}} = \prod_{j=1}^{n} e^{-i\beta\hat{X}_j}, \tag{7}$$

where $\beta \in [0, \pi]$, n is the the number of qubits, and $\hat{X}_j$ is the Pauli-X operator acting on the jth qubit. The design of mixing operators is becoming a notable research field. A variety of mixing operators have been developed for a range of problems utilising sophisticated structures and providing other functionalities [5, 31, 32].

4. **Parameterised alternating ansatz construction.** A parameterised alternating circuit consists of cost operators and mixing operators applied in an alternating fashion to the initial state as shown in Fig. 1. The output state will be

$$|\boldsymbol{\gamma}, \boldsymbol{\beta}\rangle := \hat{U}(\hat{B}, \beta_p)\hat{U}(\hat{C}, \gamma_p)...\hat{U}(\hat{B}, \beta_1)\hat{U}(\hat{C}, \gamma_1)|\,\text{Init}\rangle, \tag{8}$$

where p denotes the number of layers (one mixing operator and one cost operator) of the QAOA, $\boldsymbol{\gamma} = (\gamma_1, \gamma_2, ..., \gamma_p)$, and $\boldsymbol{\beta} = (\beta_1, \beta_2, ..., \beta_p)$.

5. **Parameter optimisation.** The parameter array $(\boldsymbol{\gamma}, \boldsymbol{\beta})$ is optimised by an objective function. We utilise the expectation value of the cost Hamiltonian as the objective function, but other common choices are Conditional Value-at-Risk, or variance [33, 34]. If we consider a more realistic situation each measurement of the quantum circuit yields a computational basis state $|\boldsymbol{w}\rangle$ representing a classical value $\boldsymbol{w}$. The corresponding cost function value is $f(\boldsymbol{w})$, and each measurement constitutes a single function access. After k measurements, we can estimate the expectation value of cost Hamiltonian $\hat{C}$ as

$$\langle \hat{C} \rangle_k := \langle \boldsymbol{\gamma}, \boldsymbol{\beta} | \hat{C} | \boldsymbol{\gamma}, \boldsymbol{\beta} \rangle_k = \frac{1}{k} \sum_{i=1}^{k} f(\boldsymbol{w}_i). \tag{9}$$

where such an estimation uses k function accesses. Classical optimisers are used to optimise the cost and mixing operators' parameters [9, 35–38].

III. QUANTUM OPTIMISATION METHODS

In this section, we first binary-encode the weight vector $\boldsymbol{w}$. Then, this section presents the three main elements of the QAOA within the encoding: initial states, a cost operator, and a constrained mixing operator.

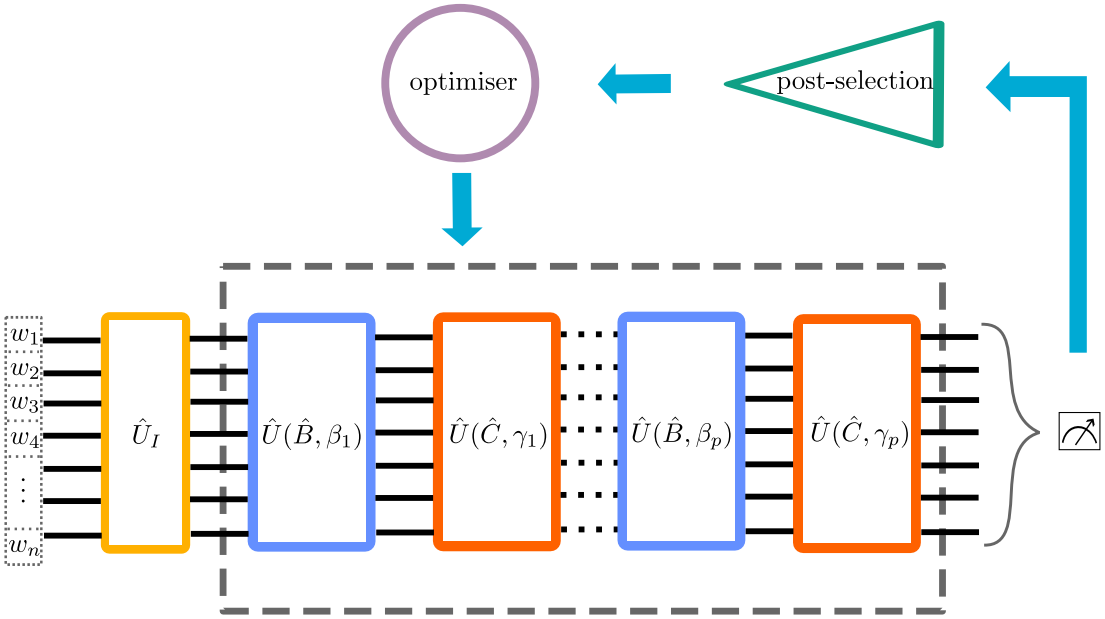


Figure 1. A parameterised QAOA ansatz for predicting the optimal DGMVP solution. The variables $w_1, w_2, \ldots, w_n$ in the dotted squares are the binary-encoded weight variables. The unitary $\hat{U}_I$ prepares the initial state. The grey dashed box includes the mixing $[\hat{U}(\hat{B}, \beta])]$ and cost $[\hat{U}(\hat{C}, \gamma))]$ operators. A tunable parameter controls each operator. After measurements, a post-selection process will be applied to filter out unconstrained results and feed them to the classical optimiser.

A. Binary block encoding method

In previous quantum portfolio optimisation algorithms [6, 11, 14], the authors encode at most three trading positions (long, short, and on-hold) with two binary variables by treating the first variable as a long position and the second as a short position. However, the DGMVP and most portfolio strategies usually require an encoding with more positions encoded as multiples of the unit trading lot a. A quasi-binary encoding method provided in [17] uses a large enough factor $1/a$ to amplify all weights variables to integers. This article uses a binary encoding method with a fixed length for all assets.

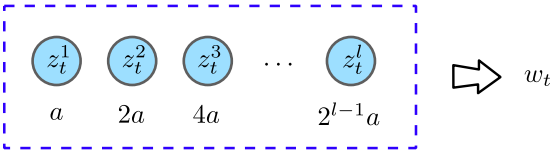


Figure 2. A diagram of binary block encoding for the tth asset investment weight w_t. The circles represent binary variables $z_t^1, z_t^2, \ldots, z_t^l$ from measuring qubits on the Pauli-Z basis. The weight of variables $a, 2a, \ldots, 2^{l-1}a$ are shown at the bottom of the corresponding variables circles. Each w_t combines an l length block of binary variables from the measurement of qubits.

The variable w_t can be encoded with a block of l binary variables (see Fig. 2),

$$w_t = \sum_{k=1}^{l} b^k z_t^k, \tag{10}$$

where $z_t^k \in \{0, 1\}$ is a binary variable corresponding to the measurement of a qubit state in Pauli-Z basis, and b^k is

$$b^k = 2^{k-1}a, \tag{11}$$

where a is the binary precision and can be interpreted as the unit trading lot:

$$a = \frac{1}{\sum_{i=0}^{l-1} 2^i} = \frac{1}{2^l - 1}. \tag{12}$$

Then, the total number of shares is $1/a$. As block sizes increase, the binary encoding method will use exponentially fewer qubits than the one-hot encoding method [39].

B. Initial state preparations

A good design of an initial state for target applications can avoid initialising far away from the global optimum and reduce the convergence time in quantum optimisation [6, 30]. This article provides four initial state preparation methods for the DGMVP model: max-biased state, ranked warm-started state, approximated equal-weighted state, and random-weighted state. Simulations in Appendix C show max-biased state and ranked warm-started state outperform other methods. Thus, we focus on these two methods in the body and summarise the rest of the initial states in the Appendix C.

The max-biased state $|\boldsymbol{w}\rangle_{mb}$*:*—This state represents an all-in-one investment without diversification, as

$$w_i = \begin{cases} 1, i = t' \\ 0, i \neq t'. \end{cases} \tag{13}$$

The ranked warm-started state $|\boldsymbol{w}\rangle_{ws}$*:*—This state represents an approximation to the optimal DGMVP solution from the round-off of the GMVP model's continuous solution $\tilde{\boldsymbol{w}}$. The GMVP can be solved using a semidefinite programming solver. The values in $\tilde{\boldsymbol{w}}$ can be divided by the binary precision a to get sets of reminders $\tilde{\boldsymbol{r}}$ and divisors $\tilde{\boldsymbol{z}}$:

$$\tilde{z}_i = \left\lfloor \frac{\tilde{w}_i}{a} \right\rfloor, \quad \tilde{r}_i = \frac{\tilde{w}_i}{a} - \left\lfloor \frac{\tilde{w}_i}{a} \right\rfloor, \quad \forall \tilde{w}_i \in \tilde{\boldsymbol{w}}. \tag{14}$$

Further, we define the remaining budget $u \coloneqq 1/a - \sum_{\tilde{z}_i \in \tilde{\boldsymbol{z}}} \tilde{z}_i$. First, we order the assets from the largest value $\tilde{r}_i$ to the smallest. Then, we assign the remaining budget one-by-one to the first u assets. A ranked warm-started state $|\boldsymbol{w}\rangle_{ws}$ of the DGMVP model prepares the asset weights as

$$w_i = \begin{cases} a\tilde{z}_i + a, i \leq u \\ a\tilde{z}_i, i > u \end{cases} . \tag{15}$$

Finding continuous solution $\tilde{\boldsymbol{w}}$ corresponds to solving a convex quadratic program which can be achieved in polynomial time in n [40]. The sorting can be achieved in $\mathcal{O}(n \log n)$ by the quick sort algorithm [41].

C. The design of cost Hamiltonian and cost operator for the DGMVP model

This section introduces the construction of the cost Hamiltonian $\hat{C}$ for the DGMVP model, the corresponding cost operator, and the quantum circuit. Using the binary block encoding from Section III A, the DGMVP problem in Eq. (1) can be rewritten as

$$\begin{array}{ll} \underset{\boldsymbol{x}}{\text{argmin}} & f = a^2 \sum_{i,j} \sigma_{ij} x_i x_j \\ \text{s.t.} & \sum_i a x_i = 1, \\ & 0 \leq x_i \leq D,\ x_i \in \mathbb{Z} \end{array} \tag{16}$$

where $\boldsymbol{x}$ is a n dimensional vector with elements $x_i = w_i/a = \sum_{k=1}^{l} b^k z_i^k$ (see in Section III A) and $D = 1/a$. Substituting for the w_i, we find

$$f = 2a^2 \sum_{i<j} \sum_{k_1,k_2} \sigma_{ij} b^{k_1} z_i^{k_1} b^{k_2} z_j^{k_2} + a^2 \sum_i \sum_{k_1,k_2} \sigma_{ii} b^{k_1} z_i^{k_1} b^{k_2} z_i^{k_2}. \tag{17}$$

Using the relations

$$\frac{\hat{\mathbb{1}} - \hat{Z}}{2}|0\rangle = 0|0\rangle, \frac{\hat{\mathbb{1}} - \hat{Z}}{2}|1\rangle = 1|1\rangle, \tag{18}$$

where $\hat{Z}$ is a Pauli-Z operator and $\hat{\mathbb{1}}$ is the identity operator, and Eq. (5), the Hamiltonian $\hat{C}$ can be written as

$$\begin{aligned} \hat{C} = & \frac{a^2}{2} \sum_{i<j} \sum_{k_1,k_2} \sigma_{ij} b^{k_1} b^{k_2} \left(\hat{Z}_i^{k_1} \hat{Z}_j^{k_2} - \hat{Z}_i^{k_1} - \hat{Z}_j^{k_2} + \hat{\mathbb{1}} \right) \\ & + \frac{a^2}{4} \sum_i \sum_{k_1,k_2} \sigma_{ii} b^{k_1} b^{k_2} \left(\hat{Z}_i^{k_1} \hat{Z}_i^{k_2} - \hat{Z}_i^{k_1} - \hat{Z}_i^{k_2} + \hat{\mathbb{1}} \right), \end{aligned} \tag{19}$$

where $\hat{Z}_i^k$ is the Pauli-Z operator on the kth qubit of the ith asset. Since every weight operator use the same number of binary variables for the encoding, then

$$\frac{a^2}{4} \sum_i \sum_{k_1,k_2} \sigma_{ii} b^{k_1} b^{k_2} \hat{Z}_i^{k_2} = \frac{a^2}{4} \sum_i \sum_{k_1,k_2} \sigma_{ii} b^{k_1} b^{k_2} \hat{Z}_i^{k_1} \tag{20}$$

and

$$\begin{aligned} & \frac{a^2}{2} \sum_{i<j} \sum_{k_1,k_2} \sigma_{ij} b^{k_1} b^{k_2} \hat{Z}_i^{k_1} + \frac{a^2}{2} \sum_{i<j} \sum_{k_1,k_2} \sigma_{ij} b^{k_1} b^{k_2} \hat{Z}_j^{k_2} + \frac{a^2}{2} \sum_i \sum_{k_1,k_2} \sigma_{ii} b^{k_1} b^{k_2} \hat{Z}_i^{k_2} \\ = & \frac{a^2}{2} \sum_{i<j} \sum_{k_1,k_2} \sigma_{ij} b^{k_1} b^{k_2} \hat{Z}_i^{k_1} + \frac{a^2}{2} \sum_{j<i} \sum_{k_1,k_2} \sigma_{ij} b^{k_1} b^{k_2} \hat{Z}_i^{k_1} + \frac{a^2}{2} \sum_i \sum_{k_1,k_2} \sigma_{ii} b^{k_1} b^{k_2} \hat{Z}_i^{k_1} \\ = & \frac{a^2}{2} \sum_{i,j} \sum_{k_1,k_2} \sigma_{ij} b^{k_1} b^{k_2} \hat{Z}_i^{k_1} = \frac{a^2}{2} \sum_{i,j} \sum_{k_2} b^{k_2} \sum_{k_1} \sigma_{ij} b^{k_1} \hat{Z}_i^{k_1} = \frac{a^2}{2} \sum_{i,j} \frac{1}{a} \sum_{k_1} \sigma_{ij} b^{k_1} \hat{Z}_i^{k_1} \\ = & \frac{a}{2} \sum_i \sum_k \tilde{\sigma}_i b^k \hat{Z}_i^k, \end{aligned} \tag{21}$$

where the second equality comes from the symmetry $\sigma_{ij} = \sigma_{ji}$, and $\tilde{\sigma}_i = \sum_j \sigma_{ij}$. By substituting the Eqs. (20) and (21) into Eq. (19), the DGMVP Hamiltonian $\hat{C}$ can be expressed as

$$\hat{C} = \frac{a^2}{2} \sum_{i<j} \sum_{k_1,k_2} \sigma_{ij} b^{k_1} b^{k_2} \hat{Z}_i^{k_1} \hat{Z}_j^{k_2} + \frac{a^2}{4} \sum_i \sum_{k_1 \neq k_2} \sigma_{ii} b^{k_1} b^{k_2} \hat{Z}_i^{k_1} \hat{Z}_i^{k_2} - \frac{a}{2} \sum_i \sum_k \tilde{\sigma}_i b^k \hat{Z}_i^k + c, \tag{22}$$

where c is a constant scalar, the $k_1 \neq k_2$ condition in the second term is derived from $\hat{Z}_i^{k_1} \hat{Z}_i^{k_2} = \hat{\mathbb{1}}$ when $k_1 = k_2$. We can drop the constant offset c because it only yields a global phase. The cost operator of the DGMVP Hamiltonian $\hat{C}$ is

$$\begin{aligned} \hat{U}(\hat{C}, \gamma) &= e^{-i\gamma \hat{C}} \\ &= \prod_{i<j} \prod_{k_1,k_2} e^{-\frac{i\gamma a^2}{2} \sigma_{ij} b^{k_1} b^{k_2} \hat{Z}_i^{k_1} \hat{Z}_j^{k_2}} \prod_i \prod_{k_1 \neq k_2} e^{-\frac{i\gamma a^2}{4} \sigma_{ii} b^{k_1} b^{k_2} \hat{Z}_i^{k_1} \hat{Z}_i^{k_2}} \prod_i \prod_k e^{\frac{i\gamma a}{2} \tilde{\sigma}_i b^k \hat{Z}_i^k}, \end{aligned} \tag{23}$$

where $\gamma \in [0, 2\pi]$. We can separate the products of exponentials because Pauli-Z strings commute with each other. Eq. (23) contains both singular and coupling Pauli-Z strings—the corresponding digital quantum circuit is shown in Fig. 3(a) and (b), respectively. As the covariance matrix Σ is dense, coupling Pauli-Z strings in $\hat{U}(\hat{C}, \gamma)$ will contain interactions between all pairs of qubits. Thus, a dense quantum circuit can be constructed with $\mathcal{O}(N)$ depth, where N is the number of qubits [42, 43].

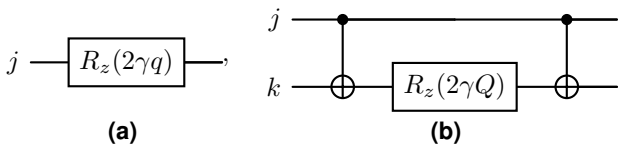


Figure 3. (a) Cost operator circuit design of $e^{-i\gamma q \hat{Z}_j}$. (b) Cost operator circuit design of $e^{-i\gamma Q \hat{Z}_j \hat{Z}_k}$.

D. Constrained mixing operator based on qubit excitations

In this section, we design a constrained mixing operator based on the qubit excitation operators to conserve the budget constraint in the QAOA optimisation. We define two types of constrained mixing operators in the current quantum portfolio solvers: *Hard-constraint mixing operators* and *soft-constraint mixing operators*, for problems with hard and soft constraints, respectively [5]. Hard-constraint mixing operators, unlike soft-constraint mixing operators, obey the problem constraints. In Appendix B, we show that the gap between the constrained and unconstrained feasible region size scales exponentially. This gap may imply that soft-constraint mixing operators can induce significant measurement overheads to optimisation. Hadfield *et al.* designed the *parity mixer*, a hard-constraint mixing operator for preserving Hamming weights within the one-hot encoding [5]. Hodson *et al.* alternated the application of the parity mixer between the odd and even spin subsets—representing short and long positions, respectively [14]. They showed that the parity mixer outperforms the standard QAOA soft-constraint mixing operator in portfolio rebalancing. However, the one-hot encoding will require an exponential number of qubits for the same precision as the binary block encoding [39]—see Section III A for details. Chen *et al.* extended the design of the parity mixer as the ring mixer by adding XYY mixers to solve a quasi-binary encoded problem [17]. Our algorithm includes a hard-constraint mixing operator design constructed using two- and three-qubit excitation operators and the induced quantum bridge phenomenon introduced in Appendices D and E.

In principle, within the binary encoding of the DGMVP model, a combination of two binary arithmetic operations can reach all constrained $\boldsymbol{w}$. The first operation, *exchange*, swaps excited qubits representing the variables with the same binary coefficient, such as z_t^k and $z_{t'}^k$, where z_t^k and $z_{t'}^k$ are the kth qubits in the t and t' asset blocks of the binary block encoding. This conserves the total number of excited qubits. The exchange operation includes two transformations:

$$\begin{cases} \underset{z_t^k \;\; z_{t'}^k}{(1,\; 0)} \mapsto \underset{z_t^k \;\; z_{t'}^k}{(0,\; 1)} \\ \underset{z_t^k \;\; z_{t'}^k}{(0,\; 1)} \mapsto \underset{z_t^k \;\; z_{t'}^k}{(1,\; 0)} \end{cases}. \tag{24}$$

The other operation is the *carry-borrow* operation and includes transformations as follows:

$$\begin{cases} (\underset{z_t^{k+1}}{0}, \underset{z_{t'}^{k}}{1}, \underset{z_{t''}^{k}}{1}) \mapsto (\underset{z_t^{k+1}}{1}, \underset{z_{t'}^{k}}{0}, \underset{z_{t''}^{k}}{0}) & \textit{carry} \\ (\underset{z_t^{k+1}}{1}, \underset{z_{t'}^{k}}{0}, \underset{z_{t''}^{k}}{0}) \mapsto (\underset{z_t^{k+1}}{0}, \underset{z_{t'}^{k}}{1}, \underset{z_{t''}^{k}}{1}) & \textit{borrow} \end{cases}, \tag{25}$$

where assets label $t' \neq t''$. The carry in the quantum setup corresponds to exciting a qubit at binary position $k+1$ and de-exciting two other qubits at binary position k, and the borrowing process is the reverse direction.

We implement these transformations with Givens rotations. The corresponding rotation for the exchange operation Eq. (24) is generated by a two-qubit excitation operator [44]:

$$\hat{S}_{tt'}^{k} \coloneqq i\left(\hat{Q}_t^{k\dagger}\hat{Q}_{t'}^{k} - \hat{Q}_t^{k}\hat{Q}_{t'}^{k\dagger}\right), \tag{26}$$

where

$$\hat{Q}_t^{k\dagger} = \frac{1}{2}(\hat{X}_t^k - i\hat{Y}_t^k), \quad \hat{Q}_t^{k} = \frac{1}{2}(\hat{X}_t^k + i\hat{Y}_t^k), \tag{27}$$

and $\hat{X}_t^k$ and $\hat{Y}_t^k$ are the Pauli-X and Pauli-Y operators acting on kth qubit of the tth asset. Then, Eq. (26) can be written as the Pauli operators:

$$\hat{S}_{tt'}^{k} = -\frac{1}{2}(\hat{X}_t^k\hat{Y}_{t'}^k - \hat{Y}_t^k\hat{X}_{t'}^k). \tag{28}$$

Similarly, the Givens rotation for the carry-barrow operation in Eq. (25) is generated by the three-qubit excitation operator

$$\begin{aligned} \hat{P}_{tt't''}^{k} &\coloneqq i\left(\hat{Q}_t^{k+1\dagger}\hat{Q}_{t'}^{k}\hat{Q}_{t''}^{k} - \hat{Q}_t^{k+1}\hat{Q}_{t'}^{k\dagger}\hat{Q}_{t''}^{k\dagger}\right) \\ &= -\frac{1}{4}(\hat{X}_t^{k+1}\hat{X}_{t'}^k\hat{Y}_{t''}^k + \hat{X}_t^{k+1}\hat{Y}_{t'}^k\hat{X}_{t''}^k - \hat{Y}_t^{k+1}\hat{X}_{t'}^k\hat{X}_{t''}^k + \hat{Y}_t^{k+1}\hat{Y}_{t'}^k\hat{Y}_{t''}^k), \end{aligned} \tag{29}$$

where assets label $t' \neq t''$.

Using these qubit excitation operators we can express the Givens rotations as:

$$\begin{aligned} e^{-i\beta\hat{S}_{tt'}^{k}} &= \cos^2\frac{\beta}{2}\hat{\mathbb{1}} + \sin^2\frac{\beta}{2}\hat{Z}_t^k\hat{Z}_{t'}^k - i\sin\beta\hat{S}_{tt'}^{k} \\ e^{-i\beta\hat{P}_{tt't''}^{k}} &= \frac{1}{4}\left(3+\cos(\beta)\right)\hat{\mathbb{1}} + \frac{1}{2}\sin^2\left(\frac{\beta}{2}\right)\left(\hat{Z}_t^{k+1}\hat{Z}_{t'}^k + \hat{Z}_t^{k+1}\hat{Z}_{t''}^k - \hat{Z}_{t'}^k\hat{Z}_{t''}^k\right) - i\sin(\beta)\hat{P}_{tt't''}^{k}, \end{aligned} \tag{30}$$

where $\beta \in [0, \pi]$ and the corresponding quantum circuit design is shown in Fig. 4. Thus, $e^{-i\beta\hat{S}_{tt'}^{k}}$ has support on $\hat{\mathbb{1}}$, $\hat{Z}_t^k\hat{Z}_{t'}^k$, and the two-qubit excitation operators $\hat{S}_{tt'}^{k}$. Additionally, $e^{-i\beta\hat{P}_{tt't''}^{k}}$ has support on $\hat{\mathbb{1}}$, $\hat{Z}_t^{k+1}\hat{Z}_{t'}^k$, $\hat{Z}_t^{k+1}\hat{Z}_{t''}^k$, $\hat{Z}_{t'}^k\hat{Z}_{t''}^k$, and the three-qubit excitation operator $\hat{P}_{tt't''}^{k}$. The $\hat{\mathbb{1}}$ leaves the state invariant. The Pauli-Z strings generate phases between computational basis states. The qubit excitation operators, $\hat{S}_{tt'}^{k}$ and $\hat{P}_{tt't''}^{k}$, perform the binary arithmetic operations in Eqs. (24) and (25), respectively. The support on the qubit excitation operators changes the occupation of computational basis states while still obeying the constraints in Eqs. (2a) to (2c). The parameter β controls the degree of superposition, performing optimisation improves the superposition to a better approximation. In Fig. 5 (a) and (b), we present two graphs that represent the support of the two- and three-qubit excitations, respectively.

Additionally, we could add a phase rotation to the Givens rotations and still satisfy the constrains. For example, the parity mixer [5]

$$\begin{aligned} \hat{T}_{tt'}^{k} &\coloneqq \frac{1}{2}(\hat{X}_t^k\hat{X}_{t'}^k + \hat{Y}_t^k\hat{Y}_{t'}^k) \\ &= \hat{Q}_t^{k\dagger}\hat{Q}_{t'}^{k} + \hat{Q}_t^{k}\hat{Q}_{t'}^{k\dagger}, \end{aligned} \tag{31}$$

and XYY mixer [17]

$$\begin{aligned} \hat{O}_{tt't''}^{k} &\coloneqq \frac{1}{4}(\hat{X}_t^{k+1}\hat{X}_{t'}^k\hat{X}_{t''}^k + \hat{Y}_t^{k+1}\hat{X}_{t'}^k\hat{Y}_{t''}^k - \hat{X}_t^{k+1}\hat{Y}_{t'}^k\hat{Y}_{t''}^k + \hat{Y}_t^{k+1}\hat{Y}_{t'}^k\hat{X}_{t''}^k) \\ &= \hat{Q}_t^{k+1\dagger}\hat{Q}_{t'}^k\hat{Q}_{t''}^k + \hat{Q}_t^{k+1}\hat{Q}_{t'}^{k\dagger}\hat{Q}_{t''}^{k\dagger}. \end{aligned} \tag{32}$$

However, for simplicity we employ the Givens rotations and only generate phase in our ansatz with the cost operator.

Furthermore, if we compose the two- and three-qubit excitation operators as follows:

$$e^{-i\beta\hat{S}^{k'}_{tt'}}e^{-i\beta\hat{P}^{k'}_{ttt'}}e^{-i\beta\hat{S}^{k'}_{tt'}}, \tag{33}$$

it will also include the support of qubit excitation operators of $e^{-i\beta\hat{P}^{k'}_{tt't}}$ as shown from Fig. 5 (c) to (d) due to a phenomenon we denote as the *quantum bridge* outlined in Appendix E. Thus, by stacking qubit excitations, the composition

$$\begin{aligned}&\left(\bigotimes_{i=k',k''} e^{-i\beta\hat{S}^{i}_{tt'}}\right)\left(e^{-i\beta\hat{P}^{k'}_{ttt'}}e^{-i\beta P^{k''}_{ttt'}}\right)\left(\bigotimes_{i=k',k''} e^{-i\beta\hat{S}^{i}_{tt'}}\right)\\ &=\left(e^{-i\beta\hat{S}^{k'}_{tt'}}e^{-i\beta\hat{P}^{k'}_{ttt'}}e^{-i\beta\hat{S}^{k'}_{tt'}}\right)\left(e^{-i\beta\hat{S}^{k''}_{tt'}}e^{-i\beta\hat{P}^{k''}_{ttt'}}e^{-i\beta\hat{S}^{k''}_{tt'}}\right),\end{aligned} \tag{34}$$

has support on all two- and three-qubit excitation operators between qubits sub-systems $z^{k'}_t$, $z^{k'}_{t'}$, $z^{k''}_t$, and $z^{k''}_{t'}$—Eq. (34) is derived in Appendix F.

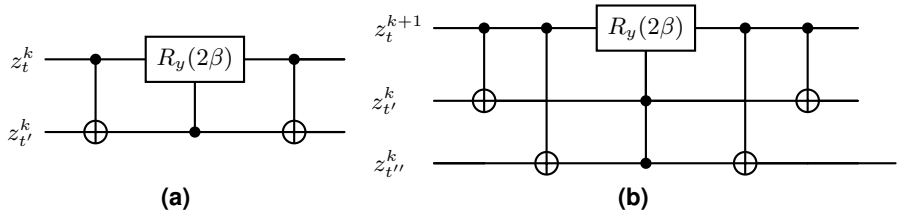


Figure 4. Quantum circuit design for (a) $e^{-i\beta\hat{S}^{k}_{tt'}}$ and (b) $e^{-i\beta\hat{P}^{k}_{tt't''}}$.

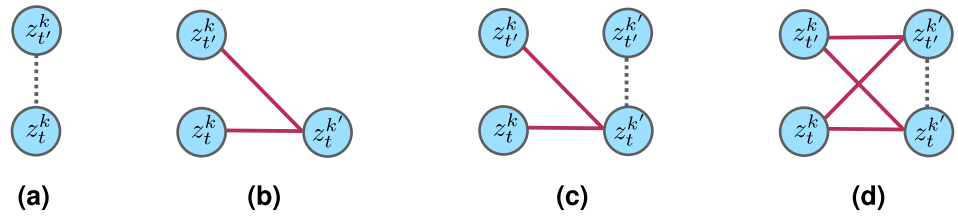


Figure 5. Graphs representing the composition of qubit excitations. The nodes represent the qubits acted upon non-trivially. The edges represent support of the resultant unitary on the generators of two- (dash) and three-qubit excitations (solid). (a) The unitary $e^{-i\beta\hat{S}^{k}_{tt'}}$ generated by the two-qubit excitation. (b) The unitary $e^{-i\beta\hat{P}^{k}_{ttt'}}$ generated by the three-qubit excitation. (c) The union of the support of the unitaries $e^{-i\beta\hat{S}^{k'}_{tt'}}$, $e^{-i\beta\hat{P}^{k'}_{ttt'}}$, and $e^{-i\beta\hat{S}^{k'}_{tt'}}$ generated by two- and three-qubit excitations. (d) The composition $e^{-i\beta\hat{S}^{k'}_{tt'}}e^{-i\beta\hat{P}^{k'}_{ttt'}}e^{-i\beta\hat{S}^{k'}_{tt'}}$ also includes support on $\hat{P}^{k'}_{t't't}$ due to the quantum bridge—see Appendix E.

By extending this stacked structure consisting of four layers, we can construct a circuit of constant depth with support on all the generators of two- and three-qubit excitations that correspond to valid binary arithmetic operations between assets t and t':

$$\hat{H}_{tt'}(\beta) = \tilde{S}_{tt'}(\beta)\tilde{P}^{e}_{tt'}(\beta)\tilde{P}^{o}_{tt'}(\beta)\tilde{S}_{tt'}(\beta), \tag{35}$$

where

$$\begin{aligned}\tilde{S}_{tt'}(\beta) &= \bigotimes_{k=1}^{l} e^{-i\beta\hat{S}^{k}_{tt'}},\\ \tilde{P}^{e}_{tt'}(\beta) &= \bigotimes_{k_1\in\mathcal{K}_1} e^{-i\beta\hat{P}^{k_1}_{ttt'}},\ \mathcal{K}_1 = \{(2c_1) \bmod l,\ c_1 = 1,2,\ldots,\lfloor l/2\rfloor\},\\ \tilde{P}^{o}_{tt'}(\beta) &= \bigotimes_{k_2\in\mathcal{K}_2} e^{-i\beta\hat{P}^{k_1}_{ttt'}},\ \mathcal{K}_2 = \{(2c_2-1) \bmod l,\ c_2 = 1,2,\ldots,\lceil l/2\rceil\},\end{aligned} \tag{36}$$

see Appendix F for details. The graph of the support of $\hat{H}_{tt'}(\beta)$ on the generators of two- and three-qubit excitations is as shown in Fig. 6.

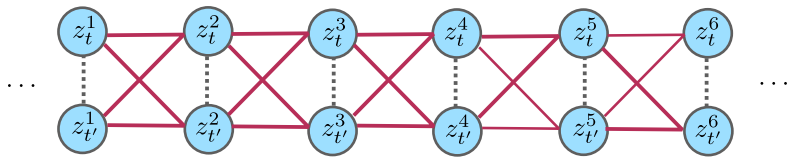


Figure 6. A graph representing the composition $\hat{H}_{tt'}(\beta)$ of qubit excitations. The nodes represent the qubits acted upon non-trivially. The edges represent the support of the resultant unitary on the generators of two- (dash) and three-qubit excitations (solid).

We can then define a hard-constraint mixing operator as

$$\hat{M}_L(\beta) = \prod_{d=1}^{L} \prod_{tt'} \hat{H}_{tt'}(\beta), \quad |t-t'| \bmod n = d, \ 1 \leq L \leq \lfloor n/2 \rfloor, \tag{37}$$

where L represents the maximum distance between assets. Assuming all-to-all qubit connectivity the depth complexity is $\mathcal{O}(L)$. If we set $L = 1$, a nearest-neighbour hard-constraint mixing operator is

$$\hat{M}_1(\beta) = \prod_{tt'} \hat{H}_{tt'}(\beta), \quad |t-t'| \bmod n = 1. \tag{38}$$

When choosing $L = \lfloor n/2 \rfloor$, the mixing operator will contain qubit excitations between all pairs of assets. In the following sections, unless stated, we use $\hat{M}_1(\beta)$ as the mixing operator due to its better performance in the optimisation (see Appendix G) and the minimal use of quantum gates. Further, by repeatedly applying $\hat{M}_L(\beta)$ interleaved with cost operators in the QAOA ansatz, we generate *long-exchange*, *-carry* and *-borrow* operations between assets through transporting excitation from neighbour to neighbour.

Finally, in Appendix H, we perform preliminary simulations to compare our mixing operator to the a mixer with the same structure but with parity and XYY mixers substituted for the two- and three-qubit excitation operators, respectively. We find our qubit excitation mixing operator performs best in portfolio optimisation. Thus, we utilise our qubit excitation mixing operator throughout this article.

IV. PARAMETERISED CIRCUIT OPTIMISATION

This section benchmarks two classical parameter optimisers, dual annealing (DA) [45] and constrained optimisation by linear approximation (COBYLA) [46], and two layerwise optimisation methods, *frozen-layerwise* and *unfrozen-layerwise* for our QAOA ansatz.

Optimisation of our ansatz is governed primarily by two hyperparameters which we will consider in turn below: The number of shots N_m used to calculate expectation values and the maximum number of expectation values that can be estimated by the optimiser $\mathcal{J}$.

As with most QAOA applications, our simulations use the estimation of an expectation value $\langle \hat{C} \rangle_{N_m}$ using N_m (optimisation) shots as the objective function. We observe in Fig. 7 that insufficient shots will lead to a sizeable stochastic noise and an unstable optimisation. We use $N_M \geq N_m$ (post-optimisation) shots to obtain a more precise estimation of the expectation value of a post-optimised quantum circuit. A further discussion about the effect of different N_m on optimisation is given in Section IV A.

From our simulations, we also find classical optimisers sometimes fail to identify convergence (see left column of Fig. 7) due to the poor loss landscape of the quantum circuit and stochastic measurement noise. Thus, employing an $\mathcal{J}$ is important to limit the overuse of resources. Finally, we denote the total number of objective functions assessed during the optimisation as N_f—which is determined by N_m and $\mathcal{J}$.

To evaluate the efficacy of a QAOA after optimisation, a common metric is the mean value approximation ratio:

$$\alpha_{\text{mean}} = \frac{\langle \hat{C} \rangle_{N_M} - f_{\min}}{f_{\max} - f_{\min}}, \tag{39}$$

where $\langle \hat{C} \rangle_{N_M}$ is defined in Eq. (9), $f_{\min}$ and $f_{\max}$ are the global minimal and maximal values of the cost function [Eq. (1)], respectively. α_{mean} is normalised within the range $[0, 1]$. Unlike the approximation ratio in the maxcut

problem, our approximation ratio is better when it's lower. However, the portfolio strategy with the global optimum that a solver can find is more useful for the DGMVP model, so we also introduce the minimal value approximation ratio:

$$\alpha_{\min} = \frac{\min_{i\in[1,N_M]}\{\langle \boldsymbol{w}_i|\hat{C}|\boldsymbol{w}_i\rangle\} - f_{\min}}{f_{\max} - f_{\min}}, \tag{40}$$

where the $\boldsymbol{w}_i$ is the ith measured weight vector.

A. Classical optimisers

COBYLA is a derivative-free local optimisation method to approximate constrained optimisation problems with linear programming problems iteratively. Benchmarks demonstrate that COBYLA can converge to lower energies faster than other local optimisers when optimising variational quantum algorithms in the absence of noise [47, 48]. While COBLYA is a good local optimiser, we wanted to include a global optimiser. DA is a stochastic global optimisation method that couples a local search with simulated annealing. We did preliminary simulations and found DA to perform well from a range of optimisers. A more thorough comparison with other optimisers is left for the future work.

In Fig. 7, we use the COBYLA and DA to optimise one ansatz parameter for different numbers of measurement shots N_m during the optimisation, fixing the DGMVP instance and keeping other ansatz parameters the same. The simulation demonstrates the difficulty of classical optimisers when optimising the loss landscape in the presence of stochastic noise [49]. This is particularly evident for COBYLA (see left-hand column of Fig. 7): as the number of optimisation shots N_m decreases, COBYLA starts to terminate the optimisation at a stochastically generated position as opposed to a local minimum. Although a sufficiently large N_m can be helpful to the optimisation, it will also lead to excessive cost function accesses, diminishing the advantage of quantum algorithms. Fortunately, we observe a better result for DA (see right-hand column of Fig. 7): a stable performance in finding a point near the global minimum as N_m is reduced with a constant number of expectation value estimations. Notably, even when decreasing the optimisation shots drastically, as shown in the bottom-right panel of Fig. 7, DA still stops at a region close to the optimal parameter. However, DA will fail to identify convergence at the end of optimisation due to stochastic noise, and an $\mathcal{J}$ should be set to avoid the over-optimisation cost.

Furthermore, a set of simulations in Appendix I shows the advantages of using DA over COBYLA to optimise multilayered ansätze in the presence of stochastic measurement errors. In Appendix J, we also consider DA for a range of $\mathcal{J}$ and N_m and determine a reasonable choice as $\mathcal{J} = 2000$ and $N_m = 16$, which we employ in Section V.

B. Layerwise optimisation

Layerwise optimisation is becoming a popular optimisation method for the QAOA and indicates faster convergence in many simulations [50–53]. There are many variant versions of the layerwise optimisation methods, and they can be categorised as *frozen-layerwise* and *unfrozen-layerwise* for our ansatz. For a fixed $p > 1$, both methods start with optimising the $p' = 1$ circuit with random initial parameters and store the optimised parameters as $\boldsymbol{\gamma}', \boldsymbol{\beta}'$. Then, we add subsequent layers by setting the previous layers' parameters as $\boldsymbol{\gamma}', \boldsymbol{\beta}'$ and initialising the new layer's parameters as zero (an identity layer) to avoid detrimenting the prior optimisation. On the one hand, in frozen-layerwise optimisation, we fix parameters from before the new layer is added and only optimise the parameters in the new layer. On the other hand, in unfrozen-layerwise optimisation, we optimise all of the layers' parameters together. For both methods, a well-optimised p layer quantum circuit can then be iteratively constructed by adding a new layer and optimising. When using DA as the layerwise optimiser, we limit the number of expectation value estimations used during optimisation after adding each new layer to $\mathcal{J}'$. Thus, after adding a total of p layers, the maximum number of expectation value estimations is $\mathcal{J} = \mathcal{J}'p$. Additionally, we use the *fixed-p ansatz* for benchmarking, referring to optimisation of all parameters in an ansatz at fixed p layers.

In Fig. 8, we perform a comparison of the unfrozen-layerwise, the frozen-layerwise, and the fixed-p ansatz optimisation methods with COBYLA and DA to a fixed DGMVP instance with a fixed number of measurement shots $N_m = 1024$ during the optimisation. The simulation shows that unfrozen-layerwise optimisation, in general, outperforms frozen-layerwise in both α_{mean} and $\alpha_{\min}$. The value of α_{mean} after optimisation is roughly constant after $p = 2$ for both layerwise methods with COBYLA and DA. In the COBYLA simulations [see Fig. 8(a)], we observe a lower α_{mean} for both layerwise methods compared to the fixed-p ansatz approach. Furthermore, for the DA simulations, the unfrozen-layerwise optimisation in Fig. 8(c) uses the same number of expectation value estimations in total but

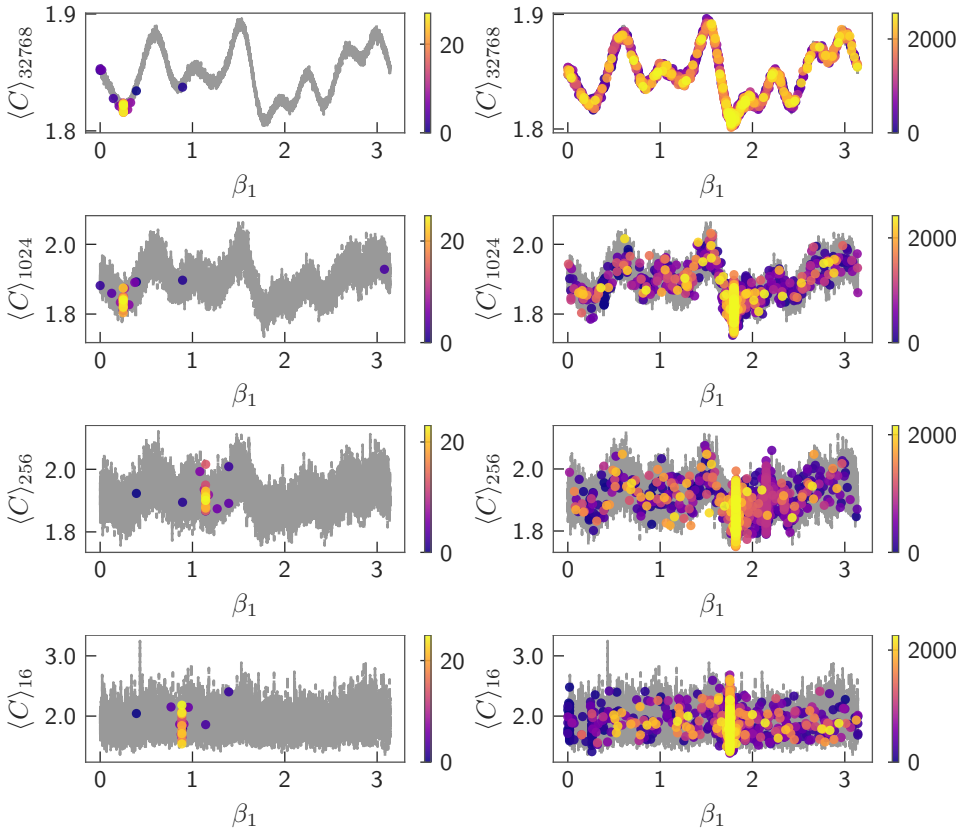


Figure 7. The performance of COBYLA (left column) and DA (right column) when optimising β_1 with other parameters fixed. These simulations use $n = 4$ assets, an asset encoding binary block length of $l = 4$, a QAOA circuit with p=10 layers and $\hat{M}_2(\beta)$ as the mixing operator, and $|\boldsymbol{w}\rangle_{mb}$ as the initial state. The loss landscape based on sampling from the cost function of β_1 with stochastic noise is plotted as grey lines. For DA, we use $\mathcal{J} = 2000$ as the maximal number of expectation value estimations. The level of stochastic noise increases down the page. That is the number of shots utilised for sampling is N_m=2^{15}, 2^{10}, 2^8 and 2^4 from the top row to the bottom. The overlayed coloured points indicate the values of the cost function sampled by the optimiser at each step during the optimisation. The colour bar on the right indicates the optimisation steps.

achieves a better result than the fixed-p ansatz approach for circuits with larger p. However, α_{mean} is used only during the optimisation. Our algorithm aims to find the bit string with minimum cost. Thus, the quality of the output of our algorithm is the minimum cost measured. Therefore, α_{min} better reflects the performance of our algorithm. Despite the plateau in α_{mean}, we find the median of α_{min} is pinned at zero, in Fig. 8(b) and (d), for all methods but frozen-layerwise with DA at almost all p—at least 50% of the time the true global minimum is found. Thus, we plot the 80% percentile to indicate the tail of the distribution. When using COBYLA, we observe that the fixed-p ansatz approach gives the tightest distribution for the larger p circuits—the output of the algorithm improves with the number of layers. Further, we find that the unfrozen-layerwise approach using DA returns the global minimum with a probability greater than 80% for $p \geq 3$ and outperforms all other methods presented. Compared to optimising the p layer circuit directly as in the fixed-p ansatz approach, the unfrozen-layerwise optimisation also uses a shorter circuit until adding the last layer which reduces quantum resource usage. The layerwise methods may also be more noise-robust by minimising the noise induced by finite coherence times during the early stages of optimisation.

V. SCALING RESULTS

To investigate the performance of our algorithm in solving the DGMVP model, we conduct stochastic simulations of models randomly selected from the financial market with varying asset numbers n and binary block lengths l. As we target the practical advantages of solving Eq. (1), we are interested in the probability of our algorithm generating portfolio with low variances. Thus, we provide a comprehensive analysis of distribution generated by the post-optimisation quantum circuit. However, the stochastic noise in estimating the final expectation value $\langle \hat{C} \rangle_{N_M}$ of the post-optimisation ansatz will increase with problem size if we fix the number of shots N_M. Thus, to gain a precise

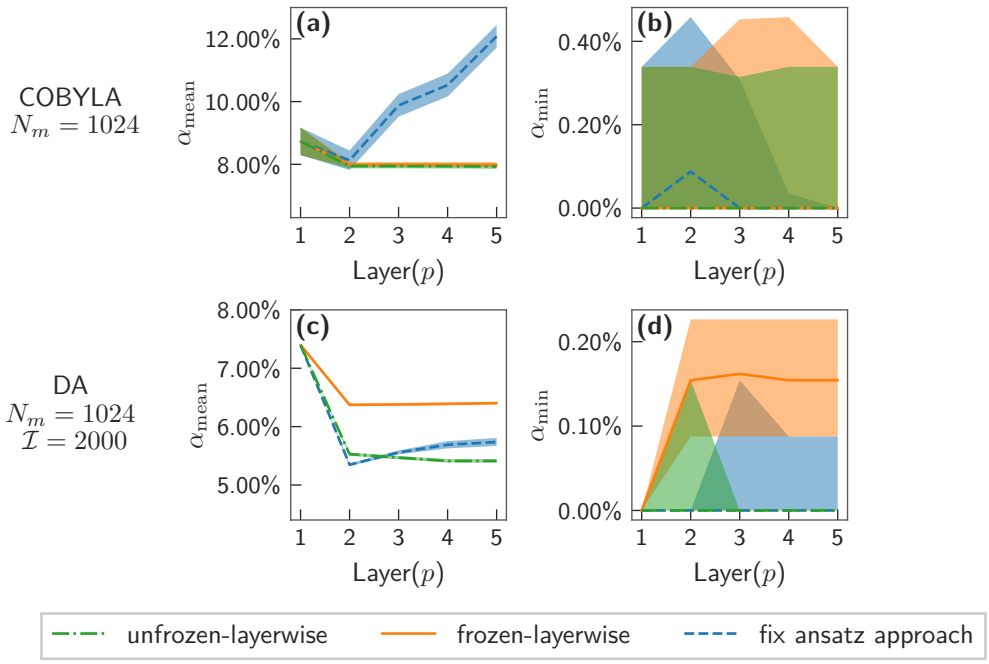


Figure 8. A comparison of optimisation methods: (a) and (b) use COBYLA as the optimiser with $N_m = 1024$; (c) and (d) use DA as the optimiser with $\mathcal{J} = 2000$ and $N_m = 1024$. The lines in (a) and (c) are the mean of α_{mean} over stochastic simulations with 100 randomly generated initial ansatz parameters for $p = 1$. The lines in (b) and (d) are the median of α_{min}. Shaded regions take percentiles 80% of data. These simulations use the mixing operator $\hat{M}_1(\beta)$ and the initial state $|\boldsymbol{w}\rangle_{mb}$ with $n = l = 4$. The final post-optimisation expectation value measurement shots of $N_M = 2^{16} = 65536$.

description of the optimised ansatz, we rebuild the parameterised quantum circuit using a statevector representation and calculate the exact expectation value $\langle \hat{C} \rangle$. This allows us to benchmark the scaling of our quantum algorithm's performance with a range of metrics independent of N_M: the approximation ratios within the lower k percentile $\alpha^{k\%}_{\text{mean}}$ ($\alpha^{100\%}_{\text{mean}} = \alpha_{\text{mean}}$); the minimal value approximation ratio α_{min}; the probability of measuring the minimal value P_{min}; the probability of measuring global minimum P_{gm}. The results are presented in Figs. 9 to 11—for simulation details see Appendix K.

In Figs. 9 to 11, we compare two portfolios with the portfolio generated by our quantum algorithm. We use the maxbias and the warm-started portfolio as both classical strategies and our initial states (see Section III B). For the warm-started portfolio, we post-select out the instances that are not the exact global minimum solutions in order to compare the level of improvement—see Table I for the percentage of instances retained.

	$n=3$ $l=3$	$n=4$ $l=3$	$n=5$ $l=3$	$n=6$ $l=3$	$n=4$ $l=2$	$n=4$ $l=4$	$n=4$ $l=5$
Retention percentage	8.99%	17.48%	24.54%	30.20%	14.27%	14.85%	14.46%

Table I. The percentage of instances retained during post-selection of the warm-started portfolio on random instances. We retain instances for which the warm-started portfolio is not the optimal DGMVP solution. n is the number of assets, and l is the asset encoding binary block length.

For these two portfolios, we observe that the warm-started classical benchmark portfolio outputs better results by orders of magnitude compared to the maxbias classical benchmark portfolio. For our quantum algorithm, we use DA as the optimiser and unfrozen-layerwise optimisation method with $\hat{M}_1(\beta)$. As the computational cost of optimising for the hyperparameters, N_m and $\mathcal{J}$, for all n and l configurations is high, we fix $N_m = 16$ and $\mathcal{J} = 2000$ as motivated by Appendix J. We justify fixing these values for all n and l as follows: Yanakiev *et al.* [54] demonstrates in appendix D that the number of shots required to estimate the approximation ratio to a fixed precision is independent of n and l. Further, we assume the number of optimisation steps to reach a given accuracy depends only on the number of parameters and hence the number of layers. Thus, we choose to fix the total number of shots used for optimisation for each layer.

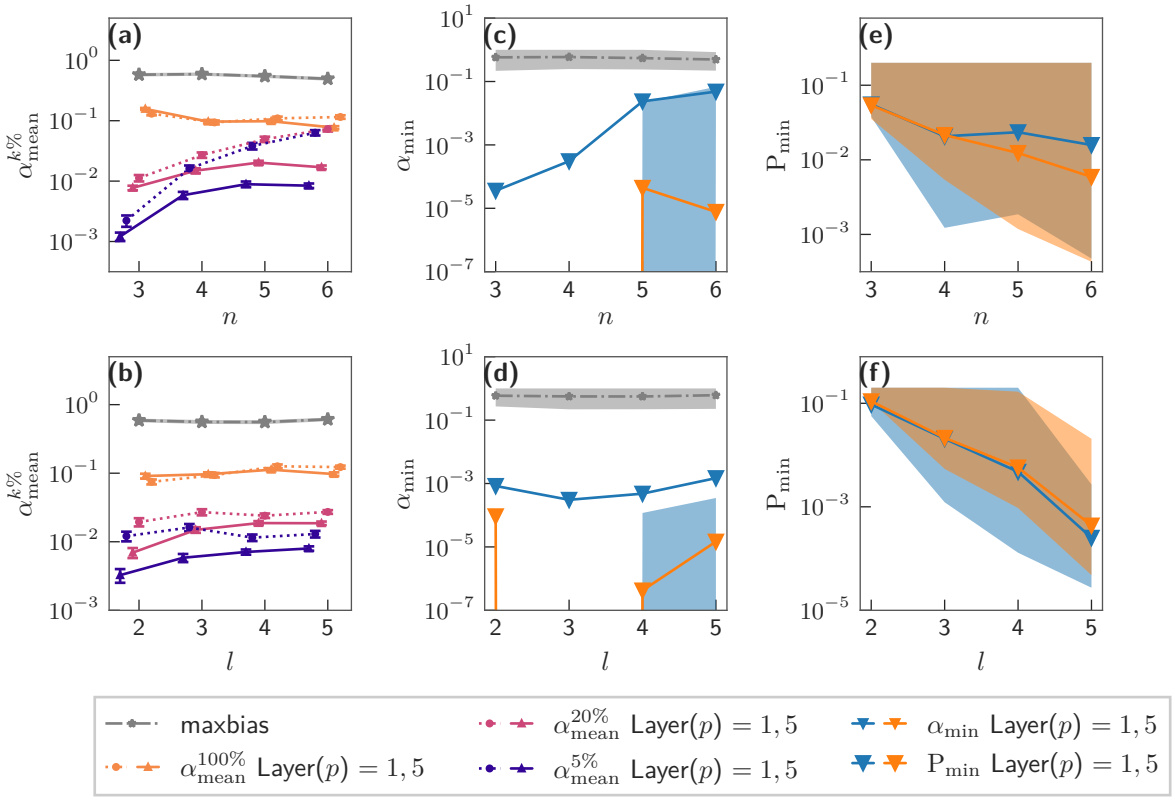


Figure 9. The $\alpha^{k\%}_{\text{mean}}$ (a) and (b), $\alpha_{\min}$ (c) and (d) and $\mathrm{P}_{\min}$ (e) and (f) of the stochastic optimisation over 100 randomly generated DGMVP instances as a function of n with $l = 3$ (the top three panels) and l with $n = 4$ (the bottom three panels). The simulations use the initial state $|\boldsymbol{w}\rangle_{mb}$ and the mixing operator $\hat{M}_1(\beta)$, and the initial ansatz parameters are randomly generated. The grey lines are the approximation ratio of maxbias classical benchmark portfolios (also considered initial states). The coloured lines are the average data of $\alpha^{k\%}_{\text{mean}}$, $\alpha_{\min}$ and $\mathrm{P}_{\min}$ of the post-optimised ansatz. The shade regions and error bars in (a) and (b) represent the standard error of the data. The shaded regions in (c) and (d) are 25%-75% percentiles of the data, and the shaded regions in (e) and (f) are 50%-100% percentiles of the data. The missing data points in (c) and (d) for $\alpha_{\min}$ correspond to values equal to 0. The classical optimiser is DA with $\mathcal{J} = 2000$ and $N_m = 16$, and the optimisation method is unfrozen-layerwise optimisation.

In Fig. 9(a) and (b), we observe our quantum algorithm achieves a significantly lower $\alpha^{k\%}_{\text{mean}}$ for all tested k and $\alpha_{\min}$ compared to the maxbias classical benchmark portfolio as a function of n and l. Additionally, we find $\alpha^{k\%}_{\text{mean}}$ and $\alpha_{\min}$ improve with increasing p in the maxbias-initialised simulations. In particular, for the small l and n configurations, the post-optimised quantum circuit can measure the optimal DGMVP solution with a relatively high probability—see Fig. 9(c) and (d). We observe $\alpha^{k\%}_{\text{mean}}$ appears to saturate [see Fig. 9(a) and (b)] and $\mathrm{P}_{\min}$ decreases [see Fig. 9(e) and (f)] as a function of n or l. This could be because we use fixed values for N_m and $\mathcal{J}$ for all the n and l configurations.

When using the warm-started initial state, we find $\alpha^{100\%}_{\text{mean}}$ is worse than the approximation ratio of the warm-started classical benchmark portfolio itself. This is potentially due to stochastic noise at finite N_m and $\mathcal{J}$ during optimisation—see Fig. 10(a) and (b). We find that, while the optimisation increases the mean, the distribution skews towards the global minimum: The $\alpha^{5\%}_{\text{mean}}$ and $\alpha^{20\%}_{\text{mean}}$ curves lie close to the warm-started classical benchmark approximation ratio. This is likely due to an exacerbation of the stochastic noise from additional states being mixed into the statevector. Additionally, increasing p increases $\alpha^{k\%}_{\text{mean}}$. However, it is $\alpha_{\min}$ that characterises the performance of our algorithm. The post-optimisation $\alpha_{\min}$ for the warm-started initial state [see Fig. 10(c) and (d)] is smaller than the warm-started classical benchmark portfolio. Also, while the mean of $\alpha_{\min}$ over 100 instances does not change with p, the distribution of $\alpha_{\min}$ skews towards the global minimum. Further, the corresponding $\mathrm{P}_{\min}$ [see Fig. 10(e) and (f)] is almost constant as a function of n and l with fixed N_m and $\mathcal{J}$ throughout the optimisation.

A global minimum is more important for investment because it represents the lowest-risk portfolio. Thus, in Fig. 11, we plot the inverse of the probability of measuring the global minimum $1/\mathrm{P}_{gm}$. The number of measurements we require to find the true global minimum with some fixed success probability P_s is $N_M = \log(1-\mathrm{P}_s)/\log(1-\mathrm{P}_{gm})$. We

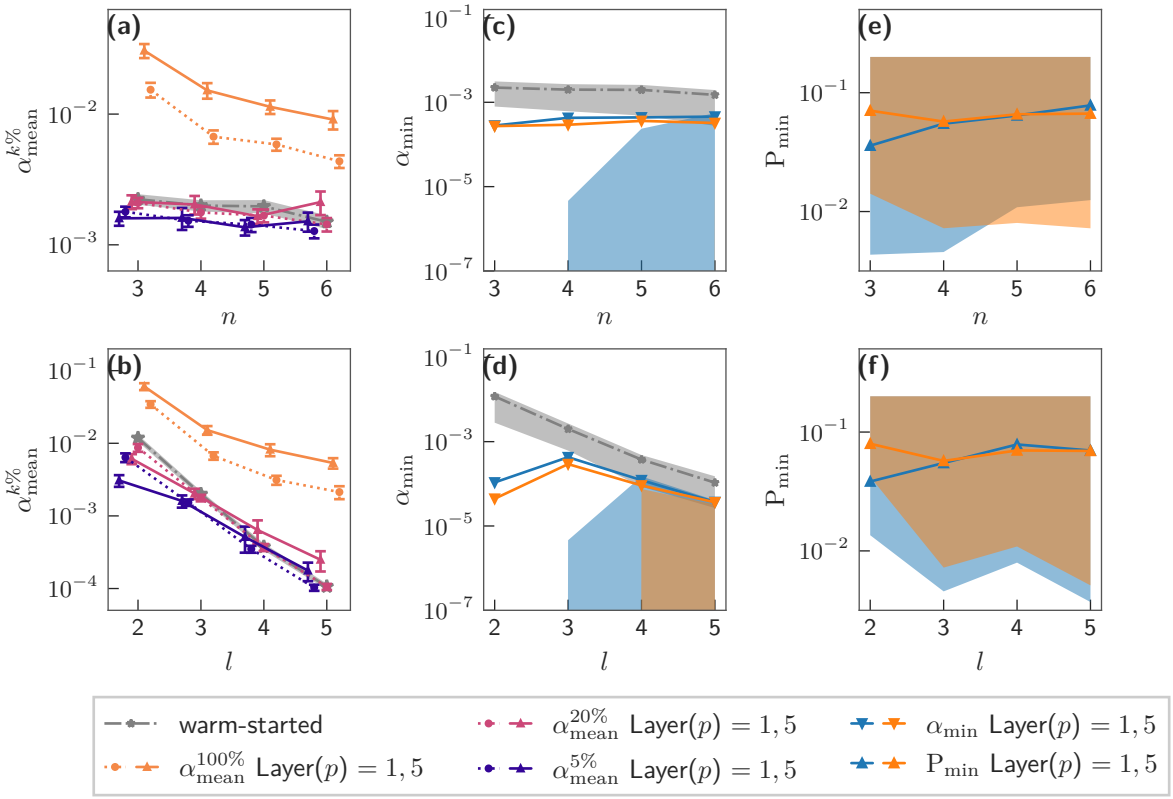


Figure 10. The $\alpha^{k\%}_{\text{mean}}$ (a) and (b), α_{min} (c) and (d) and P_{min} (e) and (f) of the stochastic optimisation over 100 randomly generated DGMVP instances as a function of n with $l = 3$ (the top three panels) and l with $n = 4$ (the bottom three panels). The simulations use the initial state $|\boldsymbol{w}\rangle_{ws}$ and the mixing operator $\hat{M}_1(\beta)$, and the initial ansatz parameters are randomly generated. The grey lines are the approximation ratio of warm-started classical benchmark portfolios (also considered initial states). The coloured lines are the average data of $\alpha^{k\%}_{\text{mean}}$, α_{min} and P_{min} of the post-optimised ansatz. The shade regions and error bars in (a) and (b) represent the standard error of the data. The shaded regions in (c) and (d) are 25%-75% percentiles of the data, and the shaded regions in (e) and (f) are 50%-100% percentiles of the data. The classical optimiser is DA with $\mathcal{J} = 2000$ and $N_m = 16$, and the optimisation method is unfrozen-layerwise optimisation.

benchmark this against the probability of measuring the global minimum when drawing uniformly random samples from both the constrained and unconstrained feasible region. The figure shows that the optimised quantum circuit has a significantly lower $1/\text{P}_{gm}$ than the constrained feasible region size, meaning the circuit can find the optimal DGMVP solution with much higher probability for a single shot compared to a constrained uniform random sampling method.

We also fit the inverse of the mean of P_{gm} (the crosses in Fig. 11) with the function $1/\bar{\text{P}}_{gm} = a \cdot B(n,l)^b$ where $B(n,l)$ is chosen as the constrained feasible region size (describe in Appendix B). We find $b = 0.76 \pm 0.30$ for the warm-started initial state, and $b = 0.51 \pm 0.09$ for the maxbias initial state—see Fig. 11(a). The ratio of the exponents of $1/\bar{\text{P}}_{gm}$ for the constrained uniform sampling method to our quantum algorithm with warm-started and maxbias initial states are $1/0.76 \approx 1.32$ and $1/0.51 \approx 1.96$, respectively. This indicates our quantum algorithm has a more favourable power-law scaling with n. Similarly, we find $b = 0.66 \pm 0.01$ and $b = 1.11 \pm 0.05$ for the warm-started and maxbias initial states, respectively—see Fig. 11(b). Once again, we can consider the ratio of the exponents: $1/0.66 \approx 1.52$ and $1/1.11 \approx 0.90$ for the warm-started and maxbias initial states, respectively. Therefore, the scaling with l of our quantum algorithm appears to be more favourable than constrained uniform sampling when using the warm-started initial state. The scalings in Fig. 11 indicate our quantum algorithm utilising the warm-started initial state may asymptotically obtain a reduction in the number of samples required to find the optimal DGMVP solution over constrained uniform sampling method. However, the constrained uniform sampling method only provides a lower bound for the classical sampling complexity. We leave comparisons to improved classical algorithms for future work.

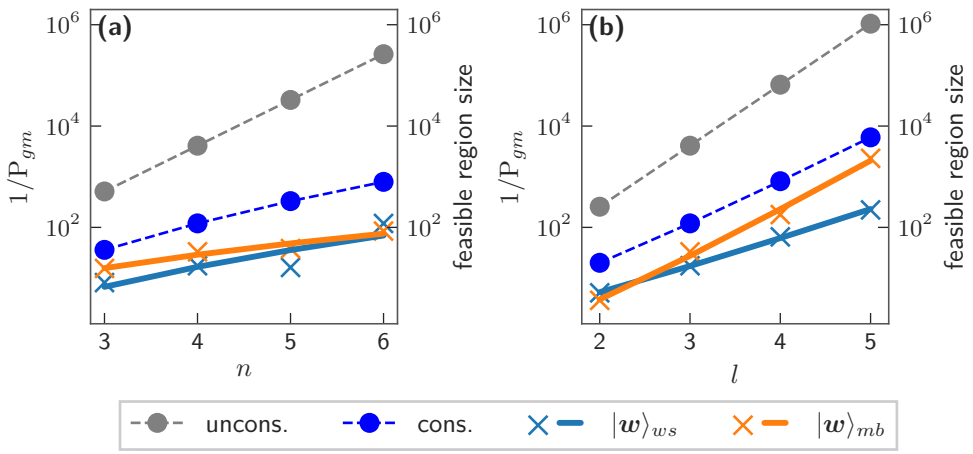


Figure 11. The inverse of the probability to measure the optimal DGMVP solution ($1/\mathrm{P}_{gm}$) and the feasible region scaling of the maxbias $|\boldsymbol{w}\rangle_{mb}$ and warm-started $|\boldsymbol{w}\rangle_{ws}$ initial states as a function of (a) n with $l = 3$, (b) l with $n = 4$. The maxbias optimisation data is the same as in Fig. 9, and the warm-started optimisation data is the same as in Fig. 10. In both panels, the left vertical axes are the $1/\mathrm{P}_{gm}$ for the crosses and solid lines. Each cross is the average of the measured data, and the solid lines with the same colour fit the cross points with a function of $1/\bar{\mathrm{P}}_{gm} = a \cdot B(n,l)^b$ where $B(n,l)$ is the constraint feasible region size function. The right vertical axes denote the feasible region size as in Eq. (B5) for the grey (unconstraint feasible region) and the blue (constraint feasible region) dash line and round solid dots.

VI. QUANTUM NOISE

This section compares the performance of our algorithm in the presence of thermal-relaxation noise with and without post-selection during optimisation. We utilise Qiskit's thermal-relaxation noise model parameterised by an ensemble of T_1 and T_2 times drawn from normal distributions such that $T_2 \leq 2T_1$ with $T_1 \sim \mathcal{N}(50\mu s, 10\mu s)$ and $T_2 \sim \mathcal{N}(70\mu s, 10\mu s)$ [55]. To evaluate the influence of the quantum noise on our algorithm, we compare two methods: First, we use measurements on a simulated noisy quantum processor to estimate the expected value directly during the optimisation. Second, we use a post-selection method to enforce the constraint in Eq. (2a) when estimating the expectation value during the optimisation.

Noise-induced errors can invalidate the constraint in Eq. (2a), which will increase the difficulty of optimisation when we do not apply post-selection. However, even when we apply post-selection, optimisation will still be hampered by noise-induced errors that leave the constraint in Eq. (2a) satisfied. Further, the application of post-selection reduces the number of shots that can be utilised for optimisation, which either increases the stochastic noise or the runtime. These effects compound to deteriorate the quality of the optimisation as shown in Fig. 12(a) and (b). Finally, the average post-selection probability for both methods during the optimisation is presented in Fig. 12(c) and (d), demonstrating roughly an order of magnitude increase in the number of shots required to maintain a fixed level of stochastic noise.

These results corroborate theoretical [56, 57] and numerical [43, 54, 58] results, suggesting only limited quantum advantage can be obtained by utilising variational quantum algorithms unless quantum noise levels are decreased by orders of magnitude. In a similar manner to stochastic noise, in Fig. 12(a), increasing measurement shots N_m can reduce the influence of quantum noise. In particular, the filtered method shows a small improvement in Fig. 12(a) compared to the unfiltered one as N_m is increased. However, when increasing p, the gap of α_{mean} closes and the probability of post-selection P_{ps} dropping sharply [see Fig. 12(b) and (d)].

VII. CONCLUSION

In this article, we have provided an end-to-end QAOA method to solve the DGMVP model. Further, we analysed the advantages and issues with the QAOA method for financial applications. Based on the DGMVP model, we designed the max-biased, ranked warm-started, approximated equal-weighted, and random-weighted initial states. Our simulations showed the max-biased and warm-started initial states outperform the others. We mapped the DGMVP model to a Hamiltonian $\hat{C}$ and designed corresponding cost operators $\hat{U}(\hat{C}, \gamma)$ with depth linear in the number of qubits. We designed a new hard-constraint mixing operator $\hat{M}_L(\beta)$ utilising two- and three-qubit excitations with efficient use of

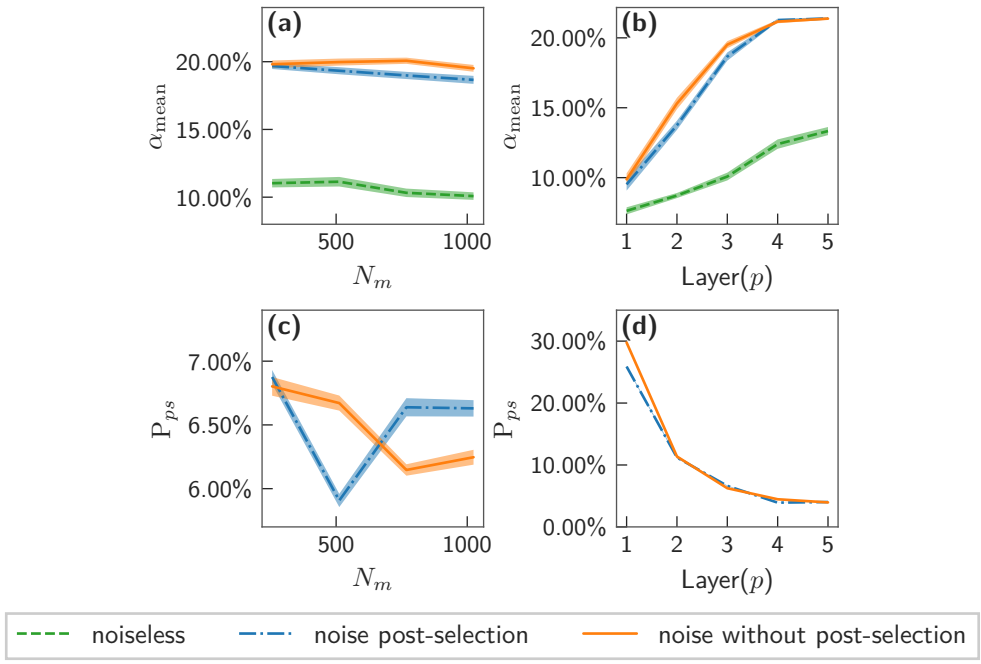


Figure 12. Performance of our QAOA in the presence of thermal relaxation noise with and without post-selection during the optimisation with optimiser COBYLA and optimisation method fixed-p ansatz approach. We present both α_{mean} [(a) and (b)] and the average probability of post-selection during parameter optimisation P_{ps} [(c) and (d)] as functions of N_m with ansatz layer $p = 3$ [(a) and (c)] and layer p with $N_m = 1024$ [(b) and (d)]. The shaded regions in (a), (b), and (d) are the 25% to 75% quantile, and the shaded region in (c) is the 10% to 90% percentile of the data. These simulations use the mixing operator $\hat{M}_1(\beta)$ and the initial state $|\boldsymbol{w}\rangle_{mb}$ with $n = l = 4$. The final expectation value measurement shots $N_M = 2^{16} = 65536$. The results are a summarisation of over 100 simulations with random initial ansatz parameters.

quantum gates using a phenomenon we call the quantum bridge. We numerically demonstrated that lower coupling distances between assets in the mixing operator makes the ansatz more readily optimisable.

Through comparative simulations, we found using DA with the limited number of expectation value access is better than one of the best local optimisation methods, COBYLA, even with large stochastic noise. We analyse two layerwise optimisation methods, frozen-layerwise, and unfrozen-layerwise, to improve the optimisability. Both methods outperform the fixed-p ansatz optimisation approach, but unfrozen-layerwise indicates a better optimisation result in general.

In the scaling analysis section, we present our quantum algorithm's performance as a function of asset numbers n and block lengths l. On the one hand, we found our quantum algorithm can decrease the mean and minimal value approximation ratio from the maxbias initial state. On the other hand, the mean approximation ratio is not improved by optimisation from the warm-started initial state. However, the distribution is skewed towards the global minimum, which allows an improved solution to be measured with constant probability as a function of n and l. Moreover, further investigation shows that our quantum algorithm utilising the warm-started initial state may asymptotically (in n and l) outperform the constrained uniform sampling in the number of samples required to find the optimal DGMVP solution.

Unfortunately, our quantum algorithm performs poorly in the presence of thermal relaxation noise. The impact can be mitigated by post-selecting out states not within the constrained feasible region. However, post-selection requires significantly more measurements to maintain a constant stochastic noise in the presence of quantum noise. This indicates drastic improvements in error rates or fault-tolerant quantum computation will be required for viable quantum finance applications.

In short, we showed that our quantum approximation algorithm allows us to predict the DGMVP. Our simulations and analyses reveal the potential advantages and disadvantages of implementing quantum algorithms in the financial industry in the future.

ACKNOWLEDGMENT

The authors thank Daniel Stilck França for valuable discussions on the DGMVP models and scaling figures. We thank Hanqing Wu and Binren Chen for the discussion of the binary encoding method, Jedrzej Burkat and Jonathan J. Thio for suggestions on mathematical notations, and David R. M. Arvidsson-Shukur and Normann Mertig for giving helpful suggestions and comments.

DATA AVAILABILITY STATEMENT

The data collected for all the figures and tables in the article are available in reference [59].

CODE AVAILABILITY STATEMENT

The data and figures can be reproduced using the code available in references [60] and [61], respectively.

[1] M. A. Nielsen and I. Chuang, *Quantum Computation and Quantum Information: 10th Anniversary Edition* (Cambridge University Press, Cambridge, 2010).
[2] S. McArdle, T. Jones, S. Endo, Y. Li, S. C. Benjamin, and X. Yuan, npj Quantum Information **5**, 75 (2019).
[3] M. Cerezo, A. Arrasmith, R. Babbush, S. C. Benjamin, S. Endo, K. Fujii, J. R. McClean, K. Mitarai, X. Yuan, L. Cincio, and P. J. Coles, Nature Reviews Physics **3**, 625 (2021).
[4] E. Farhi, J. Goldstone, and S. Gutmann, A quantum approximate optimization algorithm (2014), arXiv:1411.4028 [quant-ph].
[5] S. Hadfield, Z. Wang, B. O'gorman, E. G. Rieffel, D. Venturelli, and R. Biswas, Algorithms **12**, 34 (2019).
[6] D. J. Egger, J. Marecek, and S. Woerner, Quantum **5**, 479 (2021), arXiv:2009.10095 [quant-ph].
[7] S. Bravyi, A. Kliesch, R. Koenig, and E. Tang, Quantum **6**, 678 (2022).
[8] E. Farhi, J. Goldstone, S. Gutmann, and L. Zhou, Quantum **6**, 759 (2022), arXiv:1910.08187 [cond-mat, physics:quant-ph].
[9] S. Fernández-Lorenzo, D. Porras, and J. J. García-Ripoll, Quantum Science and Technology **6**, 034010 (2021).
[10] A. Peruzzo, J. McClean, P. Shadbolt, M. H. Yung, X. Q. Zhou, P. J. Love, A. Aspuru-Guzik, and J. L. O'Brien, Nature Communications **5**, 4213 (2014).
[11] N. Slate, E. Matwiejew, S. Marsh, and JB. Wang, Quantum **5**, 513 (2021).
[12] F. G. S. L. Brandão, R. Kueng, and D. S. França, Quantum **6**, 625 (2022), arxiv:1909.04613 [quant-ph].
[13] P. Rebentrost and S. Lloyd, KI - Künstliche Intelligenz 10.1007/s13218-024-00870-9 (2024).
[14] M. Hodson, B. Ruck, H. Ong, D. Garvin, and S. Dulman, Portfolio rebalancing experiments using the quantum alternating operator ansatz (2019), arXiv:1911.05296 [quant-ph].
[15] S. Brandhofer, D. Braun, V. Dehn, G. Hellstern, M. Hüls, Y. Ji, I. Polian, A. S. Bhatia, and T. Wellens, Quantum Information Processing **22**, 25 (2022).
[16] D. Herman, C. Googin, X. Liu, A. Galda, I. Safro, Y. Sun, M. Pistoia, and Y. Alexeev, A survey of quantum computing for finance (2022), arxiv:2201.02773 [quant-ph].
[17] B. Chen, H. Wu, H. Yuan, L. Wu, and X. Li, Quasi-binary encoding based quantum alternating operator ansatz (2024), arXiv:2304.06915 [quant-ph].
[18] L. Zhou, S.-T. Wang, S. Choi, H. Pichler, and M. D. Lukin, Physical Review X **10**, 021067 (2020).
[19] H. Markowitz, The Journal of Finance **7**, 77 (1952).
[20] P. N. Kolm, R. Tütüncü, and F. J. Fabozzi, European Journal of Operational Research **234**, 356 (2014).
[21] E. Jondeau and M. Rockinger, European Financial Management **12**, 29 (2006).
[22] R. A. Haugen, *The New Finance: The Case against Efficient Markets* (Prentice Hall, 1994).
[23] R. G. Clarke, H. De Silva, and S. Thorley, The journal of portfolio management **33**, 10 (2006).
[24] R. Clarke, H. De Silva, and S. Thorley, Journal of Portfolio Management **37**, 31 (2011).
[25] L. K. C. Chan, J. Karceski, and J. Lakonishok, The Review of Financial Studies **12**, 937 (2015), https://academic.oup.com/rfs/article-pdf/12/5/937/24420002/120937.pdf.
[26] W. A. Chaovalitwongse, I. P. Androulakis, and P. M. Pardalos, Quadratic integer programming: complexity and equivalent formsquadratic integer programming: Complexity and equivalent forms, in *Encyclopedia of Optimization* (Springer US, Boston, MA, 2009) pp. 3153–3159.
[27] G. A. Quantum, Collaborators*†, F. Arute, K. Arya, R. Babbush, D. Bacon, J. C. Bardin, R. Barends, S. Boixo, M. Broughton, B. B. Buckley, D. A. Buell, B. Burkett, N. Bushnell, Y. Chen, Z. Chen, B. Chiaro, R. Collins, W. Courtney, S. Demura, A. Dunsworth, E. Farhi, A. Fowler, B. Foxen, C. Gidney, M. Giustina, R. Graff, S. Habegger, M. P. Harrigan, A. Ho, S. Hong, T. Huang, W. J. Huggins, L. Ioffe, S. V. Isakov, E. Jeffrey, Z. Jiang, C. Jones, D. Kafri,

K. Kechedzhi, J. Kelly, S. Kim, P. V. Klimov, A. Korotkov, F. Kostritsa, D. Landhuis, P. Laptev, M. Lindmark, E. Lucero, O. Martin, J. M. Martinis, J. R. McClean, M. McEwen, A. Megrant, X. Mi, M. Mohseni, W. Mruczkiewicz, J. Mutus, O. Naaman, M. Neeley, C. Neill, H. Neven, M. Y. Niu, T. E. O'Brien, E. Ostby, A. Petukhov, H. Putterman, C. Quintana, P. Roushan, N. C. Rubin, D. Sank, K. J. Satzinger, V. Smelyanskiy, D. Strain, K. J. Sung, M. Szalay, T. Y. Takeshita, A. Vainsencher, T. White, N. Wiebe, Z. J. Yao, P. Yeh, and A. Zalcman, Science **369**, 1084 (2020), https://www.science.org/doi/pdf/10.1126/science.abb9811.

[28] H. R. Grimsley, G. S. Barron, E. Barnes, S. E. Economou, and N. J. Mayhall, npj Quantum Information **9**, 19 (2023).

[29] J. Cook, S. Eidenbenz, and A. Bärtschi, in *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)* (IEEE, 2020) pp. 83–92.

[30] R. Tate, J. Moondra, B. Gard, G. Mohler, and S. Gupta, Quantum **7**, 1121 (2023).

[31] Z. Wang, N. C. Rubin, J. M. Dominy, and E. G. Rieffel, Physical Review A **101**, 012320 (2020).

[32] A. Bärtschi and S. Eidenbenz, in *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)* (IEEE, 2020) pp. 72–82.

[33] P. K. Barkoutsos, G. Nannicini, A. Robert, I. Tavernelli, and S. Woerner, Quantum **4**, 256 (2020), arxiv:1907.04769 [quant-ph].

[34] D.-B. Zhang, B.-L. Chen, Z.-H. Yuan, and T. Yin, Chinese Physics B **31**, 120301 (2022).

[35] D. Wierichs, C. Gogolin, and M. Kastoryano, Physics Review Research **2**, 043246 (2020).

[36] M. Streif and M. Leib, Quantum Science and Technology **5**, 034008 (2020).

[37] M. Fernández-Pendás, E. F. Combarro, S. Vallecorsa, J. Ranilla, and I. F. Rúa, Journal of Computational and Applied Mathematics **404**, 113388 (2022).

[38] A. Pellow-Jarman, S. McFarthing, I. Sinayskiy, D. K. Park, A. Pillay, and F. Petruccione, Scientific Reports **14**, 16011 (2024).

[39] N. Chancellor, Quantum Science and Technology **4**, 045004 (2019).

[40] S. P. Boyd and L. Vandenberghe, *Convex optimization* (Cambridge university press, 2004).

[41] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to algorithms* (MIT press, 2022).

[42] P. G. Anastasiou, Y. Chen, N. J. Mayhall, E. Barnes, and S. E. Economou, Physical Review Research **6**, 013254 (2024).

[43] C. K. Long, K. Dalton, C. H. W. Barnes, D. R. M. Arvidsson-Shukur, and N. Mertig, Physics Review A **109**, 042413 (2024).

[44] Y. S. Yordanov, D. R. M. Arvidsson-Shukur, and C. H. W. Barnes, Physical Review A **102**, 062612 (2020), arxiv:2005.14475 [quant-ph].

[45] Y. Xiang, DY. Sun, W. Fan, and XG. Gong, Physics Letters A **233**, 216 (1997).

[46] M. J. D. Powell, A direct search optimization method that models the objective and constraint functions by linear interpolation, in *Advances in Optimization and Numerical Analysis* (Springer Netherlands, Dordrecht, 1994) pp. 51–67.

[47] A. Pellow-Jarman, I. Sinayskiy, A. Pillay, and F. Petruccione, Quantum Information Processing **20**, 202 (2021), arxiv:2106.08682 [quant-ph].

[48] W. Lavrijsen, A. Tudor, J. Müller, C. Iancu, and W. de Jong, in *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)* (2020) pp. 267–277.

[49] P. Gokhale, O. Angiuli, Y. Ding, K. Gui, T. Tomesh, M. Suchara, M. Martonosi, and F. T. Chong, IEEE Transactions on Quantum Engineering **1**, 1 (2020).

[50] A. Skolik, J. R. McClean, M. Mohseni, P. van der Smagt, and M. Leib, Quantum Machine Intelligence **3**, 5 (2021).

[51] L. Zhu, H. L. Tang, G. S. Barron, F. A. Calderon-Vargas, N. J. Mayhall, E. Barnes, and S. E. Economou, Physics Review Research **4**, 033029 (2022).

[52] E. Campos, D. Rabinovich, V. Akshay, and J. Biamonte, Physics Review A **104**, L030401 (2021).

[53] X. Lee, X. Yan, N. Xie, D. Cai, Y. Saito, and N. Asai, Physics Review A **109**, 052406 (2024).

[54] N. Yanakiev, N. Mertig, C. K. Long, and D. R. M. Arvidsson-Shukur, Physics Review A **109**, 032420 (2024).

[55] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, and J. M. Gambetta, Quantum computing with Qiskit (2024), arXiv:2405.08810 [quant-ph].

[56] D. Stilck França and R. García-Patrón, Nature Physics **17**, 1221 (2021).

[57] G. De Palma, M. Marvian, C. Rouzé, and D. S. França, PRX Quantum **4**, 010309 (2023).

[58] K. Dalton, C. K. Long, Y. S. Yordanov, C. G. Smith, C. H. W. Barnes, N. Mertig, and D. R. M. Arvidsson-Shukur, npj Quantum Information **10**, 18 (2024).

[59] H. Yuan, The supporting data for "Quantifying the advantages of applying quantum approximate algorithms to portfolio optimisation", figshare (2025).

[60] H. Yuan, Quantum solver to the Global Minimal Variance Portfolio (QGMVP), GitHub repository (2026), retrieved 04 March 2026.

[61] H. Yuan, Quantifying the advantages of applying quantum approximate algorithms to portfolio optimisation source code, GitHub repository (2026), retrieved 04 March 2026.

[62] R. Malladi and F. Fabozzi, Journal of Asset Management **18**, 188 (2017).

Appendix A: Covariance matrix for portfolio optimisation

A financial asset i has typically a time series of price data. If we represent it by a random variable $P_i(\boldsymbol{T})$, where $\boldsymbol{T} = (T_1, T_2, \ldots, T_n)$ is the time series, a σ_{ij} can be calculated as

$$\sigma_{ij} = \mathrm{cov}(P_i(\boldsymbol{T}), P_j(\boldsymbol{T})) \tag{A1}$$

where cov is the covariance. This article generates a portfolio basket among 32 stocks from the NASDAQ and NYSE, dated from "2023-03-01" to "2023-06-30". The price data uses their daily adjust-close price over the same 100 days as raw data. An example of a basket of 8 selected stocks covariance matrix is shown in Fig. 13.

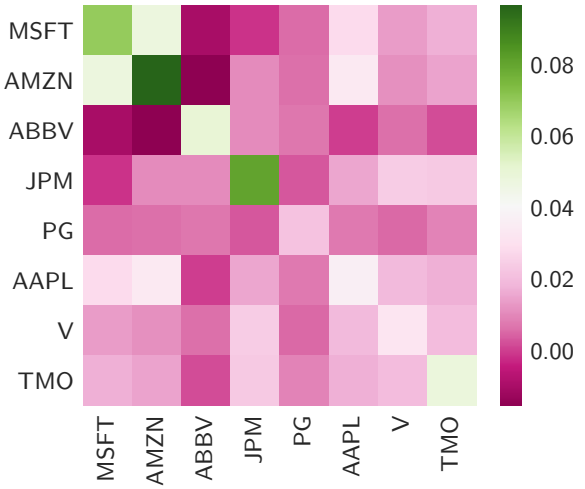


Figure 13. The covariance matrix of 8 selected stocks. MSFT means Microsoft Corp, AMZN means Amazon.com Inc, ABBV means AbbVie Inc, JPM means JPMorgan Chase & Co, PG means Procter & Gamble Co, APPL means Apple Inc, V means Visa Inc, and TMO means Thermo Fisher Scientific Inc.

Appendix B: Feasible region size of the discrete global minimum variance portfolio (DGMVP) with and without the budget constraint

This appendix reveals the method to calculate the feasible region size of the DGMVP with and without the budget constraint, which also represents the total number of portfolio strategies. The feasible region size for the DGMVP [Eq. (1)] without the constraint is 2^{ln}, where l is the number of binary variables for encoding each asset and n is the number of assets.

For the constrained DGMVP as in Eq. (1), the total number of portfolio strategies can be calculated inductively. Let $A(n, \mathrm{N})$ be the number of strategies on n assets with N trading lots. Suppose we spend m trading lots on the nth asset then we will have $A(n-1, \mathrm{N}-m)$ remaining strategies on the remaining $n-1$ assets. Therefore, the total number of strategies will be:

$$A(n, \mathrm{N}) = A(n-1, \mathrm{N}) + A(n-1, \mathrm{N}-1) + \cdots + A(n-1, 1) + A(n-1, 0) \tag{B1}$$

Inductively, $A(n, \mathrm{N})$ can be calculated as proven in Lemma 1.

Lemma 1. *The number of strategies* $A(n, \mathrm{N})$ *of* n *assets with* N *trading lots is*

$$A(n, \mathrm{N}) = \binom{\mathrm{N}+n-1}{n-1}. \tag{B2}$$

Proof. Suppose Eq. (B2) holds true for some specific n and all N', then

$$\begin{aligned} A(n+1,\mathrm{N}') &= \sum_{m=0}^{\mathrm{N}'} A(n,m) \\ &= \sum_{m=0}^{\mathrm{N}'} \binom{m+n-1}{n-1} \\ &= \binom{\mathrm{N}'+n+1-1}{n+1-1}, \end{aligned} \tag{B3}$$

where the last equality follows from the rules of Pascal's triangle:

$$\binom{j}{k} \equiv \binom{j+1}{k+1} - \binom{j}{k+1}. \tag{B4}$$

Note that $A(1,\mathrm{N}') = 1 \equiv \binom{\mathrm{N}'}{0}$ for all N'. Thus, by induction Eq. (B2) must hold for all n and N'. □

When considering a binary encoding, we define a new notation of the number of strategies:

$$B(n,l) = \binom{2^l+n-2}{n-1}, \tag{B5}$$

where l is the binary block length for encoding each asset.

In Fig. 11, we observe both the constrained and unconstrained feasible reason sizes grow exponentially. However, the size of the feasible region with the budget constraint grows much slower than the one without constraint. The gaps become more significant on an exponential scale, which indicates our hard-constraint mixing operator will exclude drastic amounts of unqualified solutions in a larger n, l configuration.

Appendix C: Initial state extra preparation methods and simulations

In this appendix, we compare the initial states defined in Section III B and below. To facilitate this comparison, we present both α_{mean} and $\alpha_{\min}$ as a function of p for each initial state in Fig. 14. We find distinctly different trends in α_{mean} and $\alpha_{\min}$: On the one hand for α_{mean}, there is an optimal p before which the stochastic measurement noise causes the optimiser to return increasingly poor values for α_{mean}—for a thorough analysis of stochastic measurement noise see Section IV A. On the other hand $\alpha_{\min}$ generally decreases with increasing p and is impacted less by the stochastic measurement noise. Fortunately, despite optimising with respect to estimated averaged values, it is $\alpha_{\min}$ that defines the quality of our final solution to the DGMVP model. Thus, we conclude that $|\boldsymbol{w}\rangle_{ws}$ performs the best followed by $|\boldsymbol{w}\rangle_{mb}$. Henceforth, we will restrict our investigations to $|\boldsymbol{w}\rangle_{ws}$ and $|\boldsymbol{w}\rangle_{mb}$; not only for their superior performance but also as they represent two disjoint areas for potential quantum advantage: $|\boldsymbol{w}\rangle_{ws}$ represents a *prior-knowledge* initialisation as a good classical approximation obtainable in polynomial time and allows us to probe the potential for improved quantum-enabled approximations in the non-asymptotic regime. Alternatively, $|\boldsymbol{w}\rangle_{mb}$ represents a *zero-knowledge* initialisation allowing us to investigate possible quantum-enabled accelerations in obtaining an approximation at least as good as $|\boldsymbol{w}\rangle_{ws}$.

The approximated equal-weighted state $|\boldsymbol{w}\rangle_{ew}$:—This state represents an approximation to an equal-weighted portfolio strategy (50/50 portfolio strategy) and consists of buying each asset in equal weight w.r.t. the total capital. Researchers often use an equal-weighted portfolio strategy as a benchmark for other strategies [62]. An approximated equal-weighted strategy of the DGMVP can be found by iteratively buying the unit trading lot (binary precision a) from the first asset to the end. For a loop of buying one trading lot for n assets, the total spent is na. The maximal repetition of completing this buying loop is at most $\lfloor 1/(na) \rfloor$ times because of the budget constraint. Then, we assign remaining budgets to buy one trading lot a sequentially from the first asset to the pth until no money left and get an equation as

$$1 - \left\lfloor \frac{1}{na} \right\rfloor na - pa = 0. \tag{C1}$$

An approximated equal-weighted state $|\boldsymbol{w}\rangle_{ew}$ is to prepare the asset weights as

$$w_i = \begin{cases} a\tilde{z} + a, i \leq p \\ a\tilde{z}, i > p \end{cases}, \tag{C2}$$

where $\tilde{z} = \lfloor 1/(na) \rfloor$.

The random-weighted state $|\boldsymbol{w}\rangle_{rd}$*:*— This state is prepared by generating a uniformly random set of weights satisfying the budget constraint and the binary block encoding.

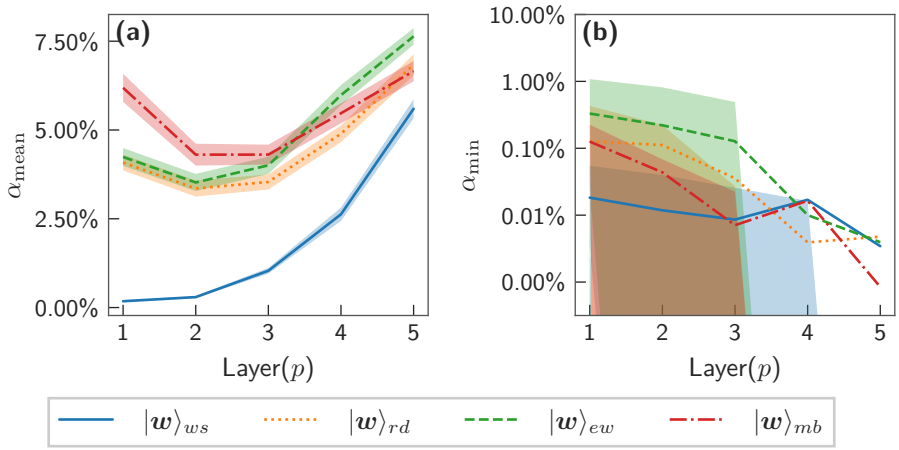


Figure 14. A comparison of the impact of initial states on the performance of our QAOA ansatz: (a) α_{mean}, (b) α_{min}. We present the optimised α_{mean} and corresponding α_{min} over randomly selected GMVP instances from the market as a function of the number of the layer p. We compare the warm-started $|\boldsymbol{w}\rangle_{ws}$, randomly initialised $|\boldsymbol{w}\rangle_{rd}$, the equally weighted $|\boldsymbol{w}\rangle_{ew}$, and the maxbias $|\boldsymbol{w}\rangle_{mb}$ initial states. The shaded region in (a) indicates the standard error of the data, and the shaded region in (b) indicates the 50% to 90% percentile of data. We only use warm-started states that are not already equal to the global minimum to facilitate a fair comparison. The QAOA ansatz uses the mixing operator $\hat{M}_1(\beta)$ with the ansatz initial parameters randomly generated. The optimiser is DA, and the optimisation method is the fixed-p ansatz approach with the number of shots during optimisation $N_m = 2^4 = 16$ and the final expectation value measurement shots $N_M = 2^{16} = 65536$.

Appendix D: Quantum bridge on two-qubit excitations

In this appendix, we analyse the action of two-qubit excitations in a specific design pattern. We define the following pair of operator to exchange qubit excitation between two qubits:

$$\begin{aligned} \hat{S}^-_{AB} &\coloneqq i\left(\hat{Q}^\dagger_A \hat{Q}_B - \hat{Q}_A \hat{Q}^\dagger_B\right) \\ \hat{S}^+_{AB} &\coloneqq i\left(\hat{Q}^\dagger_A \hat{Q}_B + \hat{Q}_A \hat{Q}^\dagger_B\right), \end{aligned} \tag{D1}$$

where A and B are the qubits sub-systems acted upon non-trivially. By expanding them with Pauli operators, we have

$$\hat{S}^+_{AB} = \frac{i}{2}(\hat{X}_A\hat{X}_B + \hat{Y}_A\hat{Y}_B) \tag{D2}$$

$$\hat{S}^-_{AB} = -\frac{1}{2}(\hat{X}_A\hat{Y}_B - \hat{Y}_A\hat{X}_B). \tag{D3}$$

If we apply a Pauli-Z operator to one of the qubit sub-systems of $\hat{S}^\pm_{AB}$, we have,

$$\begin{aligned} &\hat{S}^-_{AB}\hat{Z}_A = -\hat{S}^-_{AB}\hat{Z}_B = -\frac{1}{2}(\hat{X}_A\hat{Y}_B - \hat{Y}_A\hat{X}_B)\hat{Z}_A = \frac{i}{2}(\hat{X}_A\hat{X}_B + \hat{Y}_A\hat{Y}_B) = \hat{S}^+_{AB}, \\ &\hat{S}^+_{AB}\hat{Z}_A = \hat{S}^+_{AB}\hat{Z}_B = \frac{i}{2}(\hat{X}_A\hat{X}_B + \hat{Y}_A\hat{Y}_B)\hat{Z}_A = -\frac{1}{2}(\hat{X}_A\hat{Y}_B - \hat{Y}_A\hat{X}_B) = \hat{S}^-_{AB}, \\ &\hat{Z}_A\hat{S}^-_{AB} = -\hat{Z}_B\hat{S}^-_{AB} = -\hat{S}^-_{AB}\hat{Z}_A = -\hat{S}^+_{AB}, \\ &\hat{Z}_A\hat{S}^+_{AB} = \hat{Z}_B\hat{S}^+_{AB} = -\hat{S}^+_{AB}\hat{Z}_A = \hat{S}^-_{AB}. \end{aligned} \tag{D4}$$

If we apply $\hat{Z}_A\hat{Z}_B$, then,

$$\begin{aligned}\hat{S}^-_{AB}\hat{Z}_A\hat{Z}_B &= \hat{Z}_A\hat{Z}_B\hat{S}^-_{AB} = -\frac{1}{2}(\hat{X}_A\hat{Y}_B - \hat{Y}_A\hat{X}_B)\hat{Z}_A\hat{Z}_B = -\frac{1}{2}(\hat{Y}_A\hat{X}_B - \hat{X}_A\hat{Y}_B) = -\hat{S}^-_{AB}\\ \hat{S}^+_{AB}\hat{Z}_A\hat{Z}_B &= \hat{Z}_A\hat{Z}_B\hat{S}^+_{AB} = \frac{i}{2}(\hat{X}_A\hat{X}_B + \hat{Y}_A\hat{Y}_B)\hat{Z}_A\hat{Z}_B = -\hat{S}^+_{AB}.\end{aligned} \tag{D5}$$

We can expand the exponential of $\hat{S}^-_{AB}$ as

$$\begin{aligned}e^{-i\beta\hat{S}^-_{AB}} =& e^{\frac{i\beta}{2}(\hat{X}_A\hat{Y}_B-\hat{Y}_A\hat{X}_B)} = e^{\frac{i\beta}{2}\hat{X}_A\hat{Y}_B}e^{-\frac{i\beta}{2}\hat{Y}_A\hat{X}_B}\\ =&(\cos\frac{\beta}{2}\hat{\mathbb{1}} + i\sin\frac{\beta}{2}\hat{X}_A\hat{Y}_B)(\cos\frac{\beta}{2}\hat{\mathbb{1}} - i\sin\frac{\beta}{2}\hat{Y}_A\hat{X}_B)\\ =&\cos^2\frac{\beta}{2}\hat{\mathbb{1}} + \sin^2\frac{\beta}{2}\hat{Z}_A\hat{Z}_B + i\sin\frac{\beta}{2}\cos\frac{\beta}{2}\hat{X}_A\hat{Y}_B - i\sin\frac{\beta}{2}\cos\frac{\beta}{2}\hat{Y}_A\hat{X}_B\\ =&\cos^2\frac{\beta}{2}\hat{\mathbb{1}} + \sin^2\frac{\beta}{2}\hat{Z}_A\hat{Z}_B + i\frac{\sin\beta}{2}(\hat{X}_A\hat{Y}_B - \hat{Y}_A\hat{X}_B)\\ =&\cos^2\frac{\beta}{2}\hat{\mathbb{1}} + \sin^2\frac{\beta}{2}\hat{Z}_A\hat{Z}_B - i\sin\beta\hat{S}^-_{AB}\end{aligned} \tag{D6}$$

where $\hat{\mathbb{1}}$ and $\hat{Z}_A\hat{Z}_B$ leave the occupations of the computational basis states invariant, $\hat{S}^-_{AB}$ varies the occupations, and the parameter β controls the magnitude of the change.

If we apply another operator $e^{-i\beta\hat{S}^-_{BC}}$ to the right side of $e^{-i\beta\hat{S}^-_{AB}}$ [see Fig. 15(a)], we have

$$\begin{aligned}&e^{-i\beta\hat{S}^-_{AB}}e^{-i\beta\hat{S}^-_{BC}}\\ =&e^{\frac{i\beta}{2}(\hat{X}_A\hat{Y}_B-\hat{Y}_A\hat{X}_B)}e^{\frac{i\beta}{2}(\hat{X}_B\hat{Y}_C-\hat{Y}_B\hat{X}_C)}\\ =&\left(\cos^2\left(\frac{\beta}{2}\right)\hat{\mathbb{1}} + \sin^2\left(\frac{\beta}{2}\right)\hat{Z}_A\hat{Z}_B - i\sin(\beta)\hat{S}^-_{AB}\right)\left(\cos^2\left(\frac{\beta}{2}\right)\hat{\mathbb{1}} + \sin^2\left(\frac{\beta}{2}\right)\hat{Z}_B\hat{Z}_C - i\sin(\beta)\hat{S}^-_{BC}\right)\\ =&\cos^4\left(\frac{\beta}{2}\right)\hat{\mathbb{1}} + \frac{1}{4}\sin^2(\beta)\hat{Z}_B\hat{Z}_C + \frac{1}{4}\sin^2(\beta)\hat{Z}_A\hat{Z}_B + \sin^4\left(\frac{\beta}{2}\right)\hat{Z}_A\hat{Z}_C - i\sin(\beta)\cos^2\left(\frac{\beta}{2}\right)\hat{S}^-_{BC} - i\sin(\beta)\cos^2\left(\frac{\beta}{2}\right)\hat{S}^-_{AB}\\ &+ i\sin^2\left(\frac{\beta}{2}\right)\sin(\beta)\hat{Z}_A\hat{S}^+_{BC} + i\sin^2\left(\frac{\beta}{2}\right)\sin(\beta)\hat{S}^+_{AB}\hat{Z}_C - \sin^2(\beta)\hat{S}^-_{AB}\hat{S}^-_{BC}.\end{aligned} \tag{D7}$$

If we continue to add another operator $e^{-i\beta\hat{S}^-_{BC}}$ to the left of $e^{-i\beta\hat{S}^-_{AB}}$ [see Fig. 15(b)], we have

$$
\begin{aligned}
&e^{-i\beta\hat{S}^-_{BC}}e^{-i\beta\hat{S}^-_{AB}}e^{-i\beta\hat{S}^-_{BC}}\\
=&\left(e^{-i\beta\hat{S}^-_{BC}}e^{-i\beta\hat{S}^-_{AB}}\right)e^{-i\beta\hat{S}^-_{BC}}\\
=&\cos^6\left(\frac{\beta}{2}\right)\hat{\mathbb{1}}+\frac{1}{4}\cos^2\left(\frac{\beta}{2}\right)\sin^2(\beta)\hat{Z}_A\hat{Z}_B-i\sin(\beta)\cos^4\left(\frac{\beta}{2}\right)\hat{S}^-_{AB}\\
&+\frac{1}{4}\sin^2(\beta)\cos^2\left(\frac{\beta}{2}\right)\hat{Z}_B\hat{Z}_C+\frac{1}{4}\sin^2(\beta)\sin^2\left(\frac{\beta}{2}\right)\hat{Z}_A\hat{Z}_C-\frac{i}{4}\sin^3(\beta)\hat{Z}_C\hat{S}^+_{AB}\\
&+\frac{1}{4}\sin^2(\beta)\cos^2\left(\frac{\beta}{2}\right)\hat{Z}_A\hat{Z}_B+\frac{1}{4}\sin^2(\beta)\sin^2\left(\frac{\beta}{2}\right)\hat{\mathbb{1}}+\frac{i}{4}\sin^3\beta\hat{S}^-_{AB}\\
&+\sin^4\left(\frac{\beta}{2}\right)\cos^2\left(\frac{\beta}{2}\right)\hat{Z}_A\hat{Z}_C+\sin^6\left(\frac{\beta}{2}\right)\hat{Z}_B\hat{Z}_C+i\sin^4\left(\frac{\beta}{2}\right)\sin(\beta)\hat{Z}_C\hat{S}^+_{AB}\\
&-i\sin(\beta)\cos^4\left(\frac{\beta}{2}\right)\hat{S}^-_{BC}-i\frac{1}{4}\sin^3(\beta)\hat{S}^+_{BC}\hat{Z}_A-\sin^2(\beta)\cos^2\left(\frac{\beta}{2}\right)\hat{S}^-_{BC}\hat{S}^-_{AB}\\
&-i\sin(\beta)\cos^4\left(\frac{\beta}{2}\right)\hat{S}^-_{AB}+i\frac{1}{4}\sin^3(\beta)\hat{S}^-_{AB}-\sin^2(\beta)\cos^2\left(\frac{\beta}{2}\right)\hat{S}^-_{AB}\hat{S}^-_{AB}\\
&+i\frac{1}{4}\sin^3(\beta)\hat{Z}_A\hat{S}^+_{BC}-i\sin^4\left(\frac{\beta}{2}\right)\sin(\beta)\hat{S}^-_{BC}-\sin^2\left(\frac{\beta}{2}\right)\sin^2(\beta)\hat{S}^+_{BC}\hat{S}^+_{AB}\\
&+i\frac{1}{4}\sin^3(\beta)\hat{S}^+_{AB}\hat{Z}_C-i\sin^4\left(\frac{\beta}{2}\right)\sin(\beta)\hat{S}^+_{AB}\hat{Z}_C+\sin^2\left(\frac{\beta}{2}\right)\sin^2(\beta)\hat{S}^+_{AB}\hat{S}^-_{AB}\hat{Z}_C\\
&-\sin^2(\beta)\cos^2\left(\frac{\beta}{2}\right)\hat{S}^-_{AB}\hat{S}^-_{BC}-\sin^2(\beta)\sin^2\left(\frac{\beta}{2}\right)\hat{S}^-_{AB}\hat{S}^-_{BC}\hat{Z}_A\hat{Z}_B\\
=&\left(\cos^6\frac{\beta}{2}+\frac{1}{4}\sin^2\beta\sin^2\frac{\beta}{2}\right)\hat{\mathbb{1}}+\frac{1}{2}\cos^2\frac{\beta}{2}\sin^2\beta\hat{Z}_A\hat{Z}_B\\
&+\frac{1}{2}\sin^2(\beta)\sin^2\left(\frac{\beta}{2}\right)\hat{Z}_A\hat{Z}_C+\left(\frac{1}{4}\sin^2(\beta)\cos^2\left(\frac{\beta}{2}\right)+\sin^6\left(\frac{\beta}{2}\right)\right)\hat{Z}_B\hat{Z}_C\\
&-i\left(2\sin(\beta)\cos^4\left(\frac{\beta}{2}\right)-\frac{1}{2}\sin^3(\beta)\right)\hat{S}^-_{AB}-i\left(\sin(\beta)\cos^4\left(\frac{\beta}{2}\right)+\sin^4\left(\frac{\beta}{2}\right)\sin(\beta)\right)\hat{S}^-_{BC}\\
&-\sin^2(\beta)\cos^2\left(\frac{\beta}{2}\right)\hat{S}^-_{AB}\hat{S}^-_{AB}+\sin^2\left(\frac{\beta}{2}\right)\sin^2(\beta)\hat{S}^+_{AB}\hat{S}^-_{AB}\hat{Z}_C-\sin^2(\beta)\cos^2\left(\frac{\beta}{2}\right)\hat{S}^-_{BC}\hat{S}^-_{AB}\\
&-\sin^2\left(\frac{\beta}{2}\right)\sin^2(\beta)\hat{S}^+_{BC}\hat{S}^+_{AB}-\sin^2(\beta)\cos^2\left(\frac{\beta}{2}\right)\hat{S}^-_{AB}\hat{S}^-_{BC}-\sin^2(\beta)\sin^2\left(\frac{\beta}{2}\right)\hat{S}^-_{AB}\hat{S}^-_{BC}\hat{Z}_A\hat{Z}_B\\
=&k_1^\beta\hat{\mathbb{1}}+k_2^\beta\hat{Z}_A\hat{Z}_B+k_3^\beta\hat{Z}_B\hat{Z}_C-ik_4^\beta\hat{S}^-_{AB}-ik_5^\beta\hat{S}^-_{BC}-ik_6^\beta\hat{S}^+_{AC},
\end{aligned}
\tag{D8}
$$

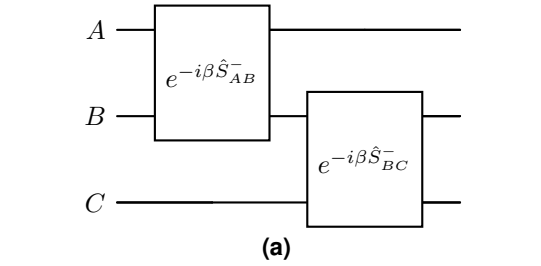
where

$$
\begin{aligned}
k_1^\beta&=\cos^6\frac{\beta}{2}+\frac{1}{4}\sin^2\beta\sin^2\frac{\beta}{2}-\frac{1}{2}\sin^2(\beta)\cos^2\left(\frac{\beta}{2}\right)\\
k_2^\beta&=\cos^2\frac{\beta}{2}\sin^2\beta\\
k_3^\beta&=\frac{1}{4}\sin^2(\beta)\cos^2\left(\frac{\beta}{2}\right)+\sin^6\left(\frac{\beta}{2}\right)+\frac{1}{2}\sin^2\left(\frac{\beta}{2}\right)\sin^2(\beta)\\
k_4^\beta&=2\sin(\beta)\cos^4\left(\frac{\beta}{2}\right)-\frac{1}{2}\sin^3(\beta)\\
k_5^\beta&=\sin(\beta)\cos^4\left(\frac{\beta}{2}\right)-\sin^4\left(\frac{\beta}{2}\right)\sin(\beta)\\
k_6^\beta&=\sin^2(\beta).
\end{aligned}
\tag{D9}
$$

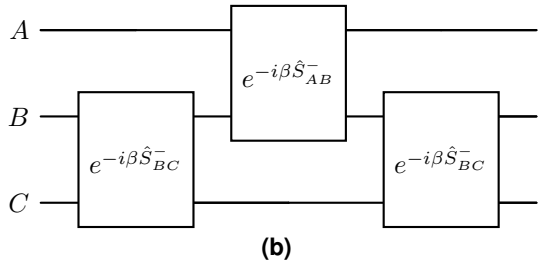
In the above derivation, we have used the following identities in addition to those in Eqs. (D4) and (D5):

$$\begin{aligned}
\hat{S}^+_{AB}\hat{S}^-_{AB} &= -\hat{S}^-_{AB}\hat{S}^+_{AB} = -\frac{1}{2}\left(\hat{Z}_A - \hat{Z}_B\right), \\
\hat{S}^+_{AB}\hat{S}^+_{AB} &= -\hat{S}^-_{AB}\hat{S}^-_{AB} = -\frac{1}{2}\left(\hat{\mathbb{1}} - \hat{Z}_A\hat{Z}_B\right) \\
\hat{S}^-_{BC}\hat{S}^-_{AB} &= \frac{1}{2}\hat{S}^+_{AC} - \frac{1}{2}\hat{S}^-_{AC}\hat{Z}_B, \quad \hat{S}^+_{BC}\hat{S}^+_{AB} = \frac{1}{2}\hat{S}^+_{AC} - \frac{1}{2}\hat{S}^-_{AC}\hat{Z}_B \\
\hat{S}^-_{AB}\hat{S}^-_{BC} &= \hat{S}^-_{AB}\hat{S}^-_{BC}\hat{Z}_A\hat{Z}_B = \frac{1}{2}\hat{S}^-_{AC}\hat{Z}_B + \frac{1}{2}\hat{S}^+_{AC} \\
\hat{S}^-_{AB}\hat{S}^-_{BC}\hat{S}^-_{AB} &= 0.
\end{aligned} \tag{D10}$$

In Eq. (D8), the support on $\hat{\mathbb{1}}, \hat{Z}_A\hat{Z}_B$, and $\hat{Z}_B\hat{Z}_C$ leave the occupations of the computational basis states invariant, the support on $\hat{S}^-_{AB}, \hat{S}^-_{BC}$, and $\hat{S}^+_{AC}$ varies the occupations, and the parameter β controls the magnitude of the change of total superposition. The decomposed supports are linearly independent of each other. In Eq. (D8), we observe in addition to the expected support on $\hat{S}^-_{AB}$ and $\hat{S}^-_{BC}$ as shown directly in the circuit Fig. 15(b), the two $e^{-i\beta\hat{S}^-_{BC}}$ operators in $e^{-i\beta\hat{S}^-_{BC}}e^{-i\beta\hat{S}^-_{AB}}e^{-i\beta\hat{S}^-_{BC}}$ act as a bridge to generate support on $\hat{S}^+_{AC}$, and we name this phenomenon as the quantum bridge on two-qubit excitations. In fact, this result, generating new support, can also be extended to additional ancilla by repeatedly sandwiching the circuit with additional two-qubit excitation operators.



$$\left\{\hat{\mathbb{1}}, \hat{Z}_B\hat{Z}_C, \hat{Z}_A\hat{Z}_B, \hat{Z}_A\hat{Z}_C\right\} \cup \left\{\hat{S}^-_{AB}, \hat{S}^-_{BC}, \hat{Z}_A\hat{S}^+_{BC}, \hat{S}^+_{AB}\hat{Z}_C, \hat{S}^-_{AB}\hat{S}^-_{BC}\right\}$$



$$\left\{\hat{\mathbb{1}}, \hat{Z}_A\hat{Z}_B, \hat{Z}_B\hat{Z}_C\right\} \cup \left\{\hat{S}^-_{AB}, \hat{S}^-_{BC}, \hat{S}^+_{AC}\right\}$$

Figure 15. Quantum circuit diagrams of (a) $e^{-i\beta\hat{S}^-_{AB}}e^{-i\beta\hat{S}^-_{BC}}$ and (b) $e^{-i\beta\hat{S}^-_{BC}}e^{-i\beta\hat{S}^-_{AB}}e^{-i\beta\hat{S}^-_{BC}}$. The set on the right of each circuit is the linear independent support of the unitary generated by the quantum circuit. These operators form an orthogonal basis for the subspace with respect to the Hilbert-Schmidt inner product. The first set includes operators that leave the occupation of computational basis states invariant. The second set includes operators that vary the occupation.

Appendix E: Quantum bridge on three-qubit excitations

In this appendix, we analyse the action of three-qubit excitations. Similar to the two-qubit excitation, for the three-qubit excitation operator as in Eq. (25), we can define a pair of operators:

$$\hat{P}^{\pm}_{ABC} \coloneqq i\left(\hat{Q}^\dagger_A\hat{Q}_B\hat{Q}_C \pm \hat{Q}_A\hat{Q}^\dagger_B\hat{Q}^\dagger_C\right). \tag{E1}$$

In the Pauli basis, these operators are

$$\hat{P}^+_{ABC} = -\frac{i}{4}(\hat{X}_A\hat{X}_B\hat{X}_C + \hat{Y}_A\hat{X}_B\hat{Y}_C - \hat{X}_A\hat{Y}_B\hat{Y}_C + \hat{Y}_A\hat{Y}_B\hat{X}_C), \tag{E2}$$

$$\hat{P}^-_{ABC} = \frac{1}{4}(\hat{X}_A\hat{X}_B\hat{Y}_C + \hat{X}_A\hat{Y}_B\hat{X}_C - \hat{Y}_A\hat{X}_B\hat{X}_C + \hat{Y}_A\hat{Y}_B\hat{Y}_C). \tag{E3}$$

Notice $\hat{P}^{\pm}_{ABC}$ is invariant under swapping the B and C indices. Applying Pauli-Z operators we find the following identities:

$$\begin{aligned}
&\hat{P}^{+}_{ABC}\hat{Z}_A = -\hat{Z}_A\hat{P}^{+}_{ABC} = \hat{P}^{-}_{ABC}\\
&\hat{P}^{+}_{ABC}\hat{Z}_B = \hat{P}^{+}_{ABC}\hat{Z}_C = -\hat{Z}_B\hat{P}^{+}_{ABC} = -\hat{Z}_C\hat{P}^{+}_{ABC} = -\hat{P}^{-}_{ABC}\\
&\hat{P}^{+}_{ABC}\hat{Z}_A\hat{Z}_B = \hat{P}^{+}_{ABC}\hat{Z}_A\hat{Z}_C = \hat{Z}_A\hat{Z}_B\hat{P}^{+}_{ABC} = \hat{Z}_A\hat{Z}_C\hat{P}^{+}_{ABC} = -\hat{P}^{+}_{ABC}\\
&\hat{P}^{+}_{ABC}\hat{Z}_B\hat{Z}_C = \hat{Z}_B\hat{Z}_C\hat{P}^{+}_{ABC} = \hat{P}^{+}_{ABC}\\
&\hat{P}^{+}_{ABC}\hat{Z}_A\hat{Z}_B\hat{Z}_C = \hat{Z}_A\hat{Z}_B\hat{Z}_C\hat{P}^{+}_{ABC} = \hat{P}^{-}_{ABC}.
\end{aligned} \tag{E4}$$

Now we will use these identities to express the circuit in Fig. 16(a):

$$\begin{aligned}
&e^{-i\beta\hat{S}^{-}_{CD}}e^{-i\beta\hat{P}^{-}_{ABC}}e^{-i\beta\hat{S}^{-}_{CD}}\\
&= \left(\cos^2\left(\frac{\beta}{2}\right)\hat{\mathbb{1}} + \sin^2\left(\frac{\beta}{2}\right)\hat{Z}_C\hat{Z}_D - i\sin(\beta)\hat{S}^{-}_{CD}\right)\\
&\quad \left(\frac{1}{4}\left(3+\cos(\beta)\right)\hat{\mathbb{1}}_{ABC} + \frac{1}{2}\sin^2\left(\frac{\beta}{2}\right)\left(\hat{Z}_A\hat{Z}_B + \hat{Z}_A\hat{Z}_C - \hat{Z}_B\hat{Z}_C\right) - i\sin(\beta)\hat{P}^{-}_{ABC}\right)\\
&\quad \left(\cos^2\left(\frac{\beta}{2}\right)\hat{\mathbb{1}} + \sin^2\left(\frac{\beta}{2}\right)\hat{Z}_C\hat{Z}_D - i\sin(\beta)\hat{S}^{-}_{CD}\right)\\
&= \left(\frac{1}{4}\cos^2\left(\frac{\beta}{2}\right)\left(3+\cos(\beta)\right)\hat{\mathbb{1}} + \frac{1}{4}\left(3+\cos(\beta)\right)\sin^2\left(\frac{\beta}{2}\right)\hat{Z}_C\hat{Z}_D - i\frac{1}{4}(3+\cos(\beta))\sin(\beta)\hat{S}^{-}_{CD}\right.\\
&\quad + \frac{1}{8}\sin^2(\beta)(\hat{Z}_A\hat{Z}_B + \hat{Z}_A\hat{Z}_C - \hat{Z}_B\hat{Z}_C) + \frac{1}{2}\sin^4\left(\frac{\beta}{2}\right)(\hat{Z}_A\hat{Z}_B\hat{Z}_C\hat{Z}_D + \hat{Z}_A\hat{Z}_D - \hat{Z}_B\hat{Z}_D)\\
&\quad - i\frac{1}{2}\sin^2\left(\frac{\beta}{2}\right)\sin(\beta)(\hat{S}^{-}_{CD}\hat{Z}_A\hat{Z}_B + \hat{S}^{+}_{CD}\hat{Z}_A - \hat{S}^{+}_{CD}\hat{Z}_B)\\
&\quad \left. - i\cos^2\left(\frac{\beta}{2}\right)\sin(\beta)\hat{P}^{-}_{ABC} - i\sin^2\left(\frac{\beta}{2}\right)\sin(\beta)\hat{P}^{+}_{ABC}\hat{Z}_D - \sin^2(\beta)\hat{S}^{-}_{CD}\hat{P}^{-}_{ABC}\right)\\
&\quad \left(\cos^2\left(\frac{\beta}{2}\right)\hat{\mathbb{1}} + \sin^2\left(\frac{\beta}{2}\right)\hat{Z}_C\hat{Z}_D - i\sin(\beta)\hat{S}^{-}_{CD}\right)\\
&= m^{\beta}_1\hat{\mathbb{1}} + m^{\beta}_2\hat{Z}_A\hat{Z}_B + m^{\beta}_3\hat{Z}_C\hat{Z}_D + m^{\beta}_4\hat{Z}_A\hat{Z}_C + m^{\beta}_5\hat{Z}_B\hat{Z}_C + m^{\beta}_6\hat{Z}_A\hat{Z}_B\hat{Z}_C\hat{Z}_D\\
&\quad - m^{\beta}_7 i\hat{S}^{-}_{CD} - m^{\beta}_8 i\hat{Z}_A\hat{Z}_B\hat{S}^{-}_{CD} - m^{\beta}_9 i\hat{P}^{-}_{ABC} - m^{\beta}_{10} i\hat{P}^{+}_{ABD},
\end{aligned} \tag{E5}$$

where

$$\begin{aligned}
m^{\beta}_1 &= \frac{1}{4}(3+\cos(\beta))(\frac{1}{4}(3+\cos(2\beta) - \frac{1}{2}\sin^2(\beta)),\\
m^{\beta}_2 &= \frac{1}{2}\sin^6\left(\frac{\beta}{2}\right) + \frac{1}{8}\sin^2(\beta)\cos^2\left(\frac{\beta}{2}\right) - \frac{1}{4}\sin^2\left(\frac{\beta}{2}\right)\sin^2(\beta),\\
m^{\beta}_3 &= \frac{1}{4}\sin^2(\beta)(3+\cos(\beta)),\\
m^{\beta}_4 &= \frac{1}{8}\sin^2(\beta)\cos^2\left(\frac{\beta}{2}\right) + \frac{1}{2}\sin^6\left(\frac{\beta}{2}\right) + \frac{1}{4}\sin^2\left(\frac{\beta}{2}\right)\sin^2(\beta),\\
m^{\beta}_5 &= -\frac{1}{8}\sin^2(\beta)\cos^2\left(\frac{\beta}{2}\right) - \frac{1}{2}\sin^6\left(\frac{\beta}{2}\right) - \frac{1}{4}\sin^2\left(\frac{\beta}{2}\right)\sin^2(\beta),\\
m^{\beta}_6 &= \frac{1}{2}\sin^2\left(\frac{\beta}{2}\right)\sin^2(\beta), m^{\beta}_7 = \frac{1}{4}(3+\cos(\beta))\sin(2\beta)\\
m^{\beta}_8 &= \frac{1}{4}\sin^3(\beta) - \sin^4\left(\frac{\beta}{2}\right)\sin(\beta), m^{\beta}_9 = \cos(\beta)\sin(\beta), m^{\beta}_{10} = \sin^2(\beta).
\end{aligned} \tag{E6}$$

In the above derivation, we have used the following identities in addition to those in Eq. (E4):

$$\begin{aligned}
\hat{S}^-_{CD}\hat{P}^-_{ABC} &= \frac{1}{2}(\hat{P}^+_{ABD} - \hat{Z}_C\hat{P}^-_{ABD}) \\
\hat{P}^-_{ABC}\hat{S}^-_{CD} &= \frac{1}{2}(\hat{P}^+_{ABD} + \hat{Z}_C\hat{P}^-_{ABD}) \\
\hat{P}^+_{ABC}\hat{S}^+_{CD} &= (-\hat{P}^-_{ABC}\hat{Z}_C)(-\hat{Z}_C\hat{S}^-_{CD}) = \hat{P}^-_{ABC}\hat{S}^-_{CD} = \frac{1}{2}(\hat{P}^+_{ABD} + \hat{Z}_C\hat{P}^-_{ABD}) \\
\hat{S}^-_{CD}\hat{P}^-_{ABC}\hat{S}^-_{CD} &= \frac{1}{2}(\hat{P}^+_{ABD} - \hat{Z}_C\hat{P}^-_{ABD})\hat{S}^-_{CD} \\
&= \frac{1}{2}(\hat{P}^+_{ABD}\hat{S}^-_{CD} - \hat{P}^-_{ABD}\hat{Z}_C\hat{S}^-_{CD}) \\
&= \frac{1}{2}(-\hat{P}^-_{ABD}\hat{Z}_D\hat{S}^-_{CD} - \hat{P}^-_{ABD}\hat{Z}_C\hat{S}^-_{CD}) \\
&= \frac{1}{2}(-\hat{P}^-_{ABD}\hat{S}^+_{CD} - \hat{P}^-_{ABD}(-\hat{S}^+_{CD})) = 0.
\end{aligned} \tag{E7}$$

In Eq. (E5), the support on $\hat{\mathbb{1}}, \hat{Z}_A\hat{Z}_B, \hat{Z}_C\hat{Z}_D, \hat{Z}_A\hat{Z}_C, \hat{Z}_B\hat{Z}_C$, and $\hat{Z}_A\hat{Z}_B\hat{Z}_C\hat{Z}_D$ leave the occupations of the computational basis states invariant, the support on $\hat{S}^-_{CD}, \hat{Z}_A\hat{Z}_B\hat{S}^-_{CD}, \hat{P}^-_{ABC}$, and $\hat{P}^+_{ABD}$ varies the occupations, and the parameter β controls the magnitude of the change of the total superposition. The decomposed supports are linearly independent of each other. Thus, $e^{-i\beta\hat{S}^-_{CD}}e^{-i\beta\hat{P}^-_{ABC}}e^{-i\beta\hat{S}^-_{CD}}$ has support on $\hat{P}^+_{ABD}$ in addition to the expected support on $\hat{P}^-_{ABC}$ and $\hat{S}^-_{CD}$ and local phase generating terms. The same derivation follows directly for $e^{-i\beta\hat{S}^-_{BD}}e^{-i\beta\hat{P}^-_{ABC}}e^{-i\beta\hat{S}^-_{BD}}$ [see Fig. 16(b)] as $\hat{P}^-_{ABC}$ is invariant under exchange of the indices B and C.

Next we consider the circuit in Fig. 16(c):

$$\begin{aligned}
&e^{-i\beta\hat{S}^-_{AD}}e^{-i\beta\hat{P}^-_{ABC}}e^{-i\beta\hat{S}^-_{AD}} \\
&= \left(\cos^2\left(\frac{\beta}{2}\right)\hat{\mathbb{1}} + \sin^2\left(\frac{\beta}{2}\right)\hat{Z}_A\hat{Z}_D - i\sin(\beta)\hat{S}^-_{AD}\right) \\
&\quad \left(\frac{1}{4}\left(3+\cos(\beta)\right)\hat{\mathbb{1}}_{ABC} + \frac{1}{2}\sin^2\left(\frac{\beta}{2}\right)\left(\hat{Z}_A\hat{Z}_B + \hat{Z}_A\hat{Z}_C - \hat{Z}_B\hat{Z}_C\right) - i\sin(\beta)\hat{P}^-_{ABC}\right) \\
&\quad \left(\cos^2\left(\frac{\beta}{2}\right)\hat{\mathbb{1}} + \sin^2\left(\frac{\beta}{2}\right)\hat{Z}_A\hat{Z}_D - i\sin(\beta)\hat{S}^-_{AD}\right) \\
&= \left(\frac{1}{4}\cos^2\left(\frac{\beta}{2}\right)(3+\cos(\beta))\hat{\mathbb{1}} + \frac{1}{4}(3+\cos(\beta))\sin^2\left(\frac{\beta}{2}\right)\hat{Z}_A\hat{Z}_D + \frac{1}{4}(3+\cos(\beta))\sin(\beta)\hat{S}^-_{AD}\right. \\
&\quad + \frac{1}{8}\sin^2(\beta)(\hat{Z}_A\hat{Z}_B + \hat{Z}_A\hat{Z}_C - \hat{Z}_B\hat{Z}_C) + \frac{1}{2}\sin^4\left(\frac{\beta}{2}\right)(\hat{Z}_B\hat{Z}_D - \hat{Z}_C\hat{Z}_D - \hat{Z}_A\hat{Z}_B\hat{Z}_C\hat{Z}_D) \\
&\quad - i\frac{1}{2}\sin^2\left(\frac{\beta}{2}\right)\sin(\beta)(\hat{S}^+_{AD}\hat{Z}_B + \hat{S}^+_{AD}\hat{Z}_C - \hat{S}^-_{AD}\hat{Z}_B\hat{Z}_C) \\
&\quad \left. - i\cos^2\left(\frac{\beta}{2}\right)\sin(\beta)\hat{P}^-_{ABC} + i\sin^2\left(\frac{\beta}{2}\right)\sin(\beta)\hat{P}^+_{ABC}\hat{Z}_D - \sin^2(\beta)\hat{S}^-_{AD}\hat{P}^-_{ABC}\right) \\
&\quad \left(\cos^2\left(\frac{\beta}{2}\right)\hat{\mathbb{1}} + \sin^2\left(\frac{\beta}{2}\right)\hat{Z}_A\hat{Z}_D - i\sin(\beta)\hat{S}^-_{AD}\right) \\
&= m^\beta_1\hat{\mathbb{1}} + m^\beta_2\hat{Z}_B\hat{Z}_C + m^\beta_3\hat{Z}_A\hat{Z}_D + m^\beta_4\hat{Z}_A\hat{Z}_B + m^\beta_5\hat{Z}_A\hat{Z}_C + m^\beta_6\hat{Z}_A\hat{Z}_B\hat{Z}_C\hat{Z}_D \\
&\quad - m^\beta_7 i\hat{S}^-_{AD} - m^\beta_8 i\hat{Z}_B\hat{Z}_C\hat{S}^-_{AD} - m^\beta_9 i\hat{P}^-_{ABC} - m^\beta_{10} i\hat{P}^+_{DBC},
\end{aligned} \tag{E8}$$

where

$$
\begin{aligned}
m_1^\beta &= \frac{1}{4}(3+\cos(\beta))(\frac{1}{4}(3+\cos(2\beta)-\frac{1}{2}\sin^2(\beta)),\\
m_2^\beta &= -\frac{1}{2}\sin^6\left(\frac{\beta}{2}\right)-\frac{1}{8}\sin^2(\beta)\cos^2\left(\frac{\beta}{2}\right)+\frac{1}{4}\sin^2\left(\frac{\beta}{2}\right)\sin^2(\beta),\\
m_3^\beta &= \frac{1}{4}\sin^2(\beta)(3+\cos(\beta)),\\
m_4^\beta &= \frac{1}{8}\sin^2(\beta)\cos^2\left(\frac{\beta}{2}\right)+\frac{1}{2}\sin^6\left(\frac{\beta}{2}\right)+\frac{1}{4}\sin^2\left(\frac{\beta}{2}\right)\sin^2(\beta),\\
m_5^\beta &= \frac{1}{8}\sin^2(\beta)\cos^2\left(\frac{\beta}{2}\right)+\frac{1}{2}\sin^6\left(\frac{\beta}{2}\right)+\frac{1}{4}\sin^2\left(\frac{\beta}{2}\right)\sin^2(\beta),\\
m_6^\beta &= -\frac{1}{2}\sin^2\left(\frac{\beta}{2}\right)\sin^2(\beta), m_7^\beta=\frac{1}{4}(3+\cos(\beta))\sin(2\beta)\\
m_8^\beta &= -\frac{1}{4}\sin^3(\beta)+\sin^4\left(\frac{\beta}{2}\right)\sin(\beta), m_9^\beta=\cos(\beta)\sin(\beta), m_{10}^\beta=-\sin^2(\beta).
\end{aligned}
\tag{E9}
$$

In the above derivation, we have used the following identities in addition to those in Eq. (E4) and Eq. (E7):

$$
\begin{aligned}
\hat{P}^-_{ABC}\hat{S}^-_{AD} &= \frac{1}{2}(\hat{Z}_A\hat{P}^-_{DBC}-\hat{P}^+_{DBC})\\
\hat{S}^-_{AD}\hat{P}^-_{ABC} &= -\frac{1}{2}(\hat{Z}_A\hat{P}^-_{DBC}+\hat{P}^+_{DBC})\\
\hat{P}^+_{ABC}\hat{S}^+_{AD} &= (\hat{P}^-_{ABC}\hat{Z}_A)(-\hat{Z}_A\hat{S}^-_{AD}) = -\hat{P}^-_{ABC}\hat{S}^-_{AD}\\
\hat{S}^+_{AD}\hat{P}^+_{ABC} &= (-\hat{S}^-_{AD}\hat{Z}_A)(-\hat{Z}_A\hat{P}^-_{ABC}) = \hat{S}^-_{AD}\hat{P}^-_{ABC}\\
\hat{S}^-_{AD}\hat{P}^-_{ABC}\hat{S}^-_{AD} &= 0.
\end{aligned}
\tag{E10}
$$

In Eq. (E8), the support on $\hat{\mathbb{1}}, \hat{Z}_B\hat{Z}_C, \hat{Z}_A\hat{Z}_D, \hat{Z}_A\hat{Z}_B$, and $\hat{Z}_A\hat{Z}_C, \hat{Z}_A\hat{Z}_B\hat{Z}_C\hat{Z}_D$ leave the occupations of the computational basis states invariant, the support on $\hat{S}^-_{AD}, \hat{Z}_B\hat{Z}_C\hat{S}^-_{AD}, \hat{P}^-_{ABC}$, and $\hat{P}^+_{DBC}$ vary the occupations, and the parameter β controls the magnitude of the change the total superposition. The decomposed supports are linearly independent of each other. Thus, $e^{-i\beta\hat{S}^-_{AD}}e^{-i\beta\hat{P}^-_{ABC}}e^{-i\beta\hat{S}^-_{AD}}$ has support on $\hat{P}^+_{DBC}$ in addition to the expected support on $\hat{P}^-_{ABC}$ and $\hat{S}^-_{AD}$ and local phase generating terms.

In this appendix, we have shown that by sandwiching a three-qubit excitation with two-qubit excitations that act non-trivially on an ancilla qubit, the resulting unitary will have support on a three-qubit excitation acting upon this ancilla qubit. We name this phenomenon as the quantum bridge on three-qubit excitations.

Appendix F: Mixing operator with a implement of quantum bridges

In this appendix, we show how the design of the mixing operator $\hat{H}_{tt'}(\beta)$ (see in Section III D) utilises the quantum bridges to reduce the cost of quantum resources and generate sufficient support for binary arithmetic operations between asset blocks. By using the same notation as in Section III D, we have

$$
\begin{aligned}
&\left[\hat{S}^k_{tt'},\hat{P}^k_{ttt'}\right]\\
=&\left[\frac{1}{2}\left(\hat{X}^k_t\hat{Y}^k_{t'}-\hat{Y}^k_t\hat{X}^k_{t'}\right),\frac{1}{4}\left(\hat{X}^{k+1}_t\hat{X}^k_t\hat{Y}^k_{t'}+\hat{X}^{k+1}_t\hat{Y}^k_t\hat{X}^k_{t'}-\hat{Y}^{k+1}_t\hat{X}^k_t\hat{X}^k_{t'}+\hat{Y}^{k+1}_t\hat{Y}^k_t\hat{Y}^k_{t'}\right)\right]\\
=&\frac{1}{8}\left\{\left[\hat{X}^k_t\hat{Y}^k_{t'},\hat{X}^{k+1}_t\hat{X}^k_t\hat{Y}^k_{t'}\right]+\left[\hat{X}^k_t\hat{Y}^k_{t'},\hat{X}^{k+1}_t\hat{Y}^k_t\hat{X}^k_{t'}\right]-\left[\hat{X}^k_t\hat{Y}^k_{t'},\hat{Y}^{k+1}_t\hat{X}^k_t\hat{X}^k_{t'}\right]+\left[\hat{X}^k_t\hat{Y}^k_{t'},\hat{Y}^{k+1}_t\hat{Y}^k_t\hat{Y}^k_{t'}\right]\right.\\
&\left.-\left[\hat{Y}^k_t\hat{X}^k_{t'},\hat{X}^{k+1}_t\hat{X}^k_t\hat{Y}^k_{t'}\right]-\left[\hat{Y}^k_t\hat{X}^k_{t'},\hat{X}^{k+1}_t\hat{Y}^k_t\hat{X}^k_{t'}\right]+\left[\hat{Y}^k_t\hat{X}^k_{t'},\hat{Y}^{k+1}_t\hat{X}^k_t\hat{X}^k_{t'}\right]-\left[\hat{Y}^k_t\hat{X}^k_{t'},\hat{Y}^{k+1}_t\hat{Y}^k_t\hat{Y}^k_{t'}\right]\right\}\\
=&\frac{1}{8}\left\{\hat{X}^{k+1}_t+\hat{X}^{k+1}_t\left[\hat{X}^k_t\hat{Y}^k_{t'},\hat{Y}^k_t\hat{X}^k_{t'}\right]-\hat{Y}^{k+1}_t\left[\hat{Y}^k_{t'},\hat{X}^k_{t'}\right]+\hat{Y}^{k+1}_t\left[\hat{X}^k_t,\hat{Y}^k_t\right]\right.\\
&\left.-\hat{X}^{k+1}_t\left[\hat{Y}^k_t\hat{X}^k_{t'},\hat{X}^k_t\hat{Y}^k_{t'}\right]-\hat{X}^{k+1}_t+\hat{Y}^{k+1}_t\left[\hat{Y}^k_t,\hat{X}^k_t\right]-\hat{Y}^{k+1}_t\left[\hat{X}^k_{t'},\hat{Y}^k_{t'}\right]\right\}\\
=&0
\end{aligned}
\tag{F1}
$$

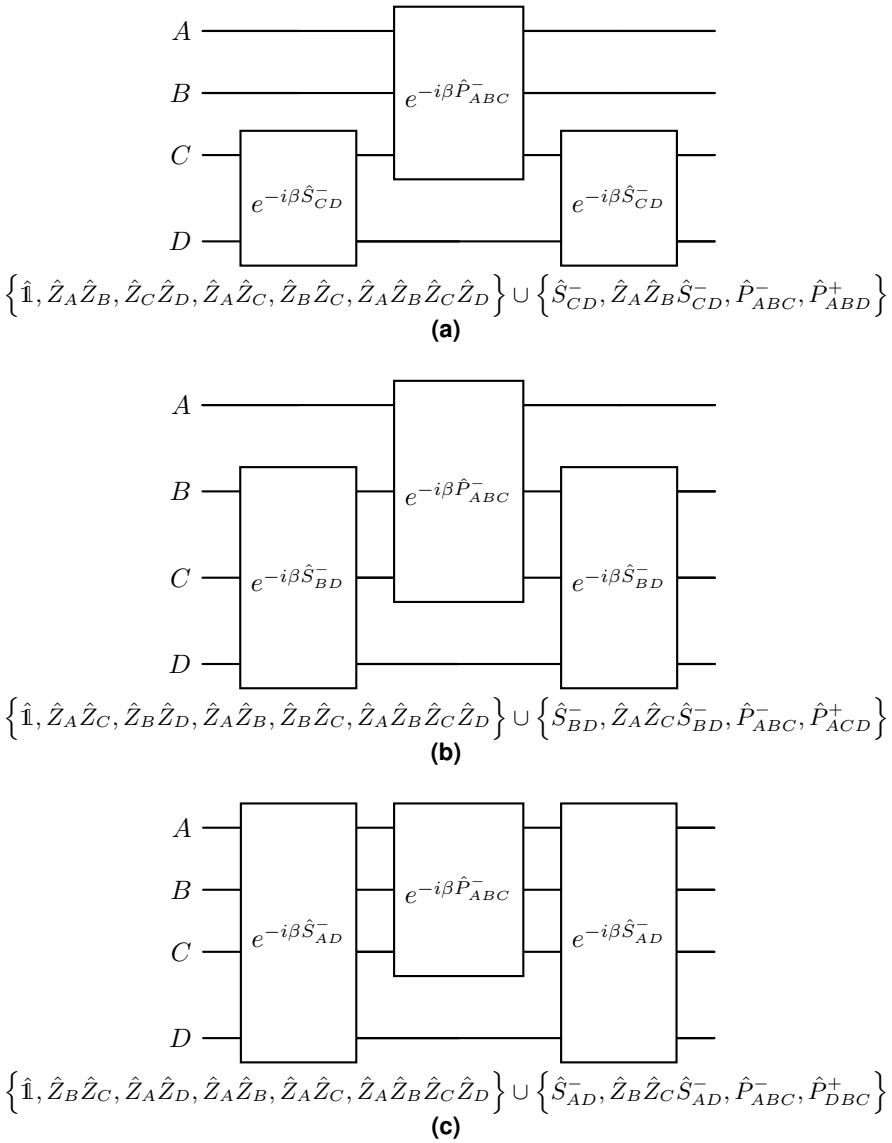


Figure 16. Quantum circuits of (a) $e^{-i\beta\hat{S}^-_{CD}}e^{-i\beta\hat{P}^-_{ABC}}e^{-i\beta\hat{S}^-_{CD}}$, (b) $e^{-i\beta\hat{S}^-_{BD}}e^{-i\beta\hat{P}^-_{ABC}}e^{-i\beta\hat{S}^-_{BD}}$, and (c) $e^{-i\beta\hat{S}^-_{AD}}e^{-i\beta\hat{P}^-_{ABC}}e^{-i\beta\hat{S}^-_{AD}}$. The set on the right of each circuit is the linear independent support of the unitary generated by the quantum circuit. These operators form an orthogonal basis for the subspace with respect to the Hilbert-Schmidt inner product. The first set includes operators that leave the occupation of computational basis states invariant. The second set includes operators that vary the occupation.

where the second equality follows from $[\hat{A}+\hat{B}, \hat{C}+\hat{D}] = [\hat{A}, \hat{C}] + [\hat{B}, \hat{C}] + [\hat{A}, \hat{D}] + [\hat{B}, \hat{D}]$ and $[c_1\hat{A}, c_2\hat{B}] = c_1c_2[\hat{A}, \hat{B}]$ with $c_1, c_2 \in \mathbb{C}$, for the third equality we use the fact that Pauli operators are involutions, and the last equality follows from $[\hat{X}\hat{Y}, \hat{Y}\hat{X}] = [\hat{Y}\hat{X}, \hat{X}\hat{Y}]$. Thus, we have

$$\begin{aligned} & e^{-i\beta\hat{S}^k_{tt'}}e^{-i\beta\hat{P}^k_{ttt'}} \\ =& e^{-i\beta\hat{S}^k_{tt'}-i\beta\hat{P}^k_{ttt'}} \\ =& e^{-i\beta\hat{P}^k_{ttt'}}e^{-i\beta\hat{S}^k_{tt'}} \end{aligned} \tag{F2}$$

Mixing operator	Average gradient	Largest gradient	Min	Max−Min	Valleys
$\hat{M}_1(\beta)$	0.0298 ± 0.0025	0.132 ± 0.011	$\mathbf{0.220 \pm 0.028}$	0.051 ± 0.003	4.2 ± 0.1
$\hat{M}_2(\beta)$	0.0226 ± 0.0024	0.101 ± 0.010	$\mathbf{0.220 \pm 0.028}$	0.022 ± 0.002	6.3 ± 0.1

Table II. Tabulated properties of the loss landscapes of mixing operators $\hat{M}_L(\beta)$ with $L = 1, 2$. The data is collected from 100 simulations each using randomly generated DGMVP instances with the asset number $n = 4$ and the asset binary block length $l = 5$. In our simulations, we use the $|\boldsymbol{w}\rangle_{mb}$ as the initial state and a $p = 10$ layer ansatz. We scan the landscape of the fifth mixing operator's parameter $\beta_5 \in [0, \pi]$ with a resolution of $\pi/500$ and fix all the other cost and mixing rotation parameters as $\pi/4$ and $\pi/8$, respectively. Each data point is estimated with $N_M{=}2^{22} = 4194304$ measurement shots. The average (second column) and largest (third column) gradient magnitudes of the approximation ratio are calculated using finite differences. The minimum (fourth column), maximum−minimum (fifth column), and the number of valleys (sixth column) are averaged over the 100 instances. All values for $\hat{M}_1(\beta)$ and $\hat{M}_2(\beta)$ are separated by more than the standard error in the mean except for the minimum (bold).

Then, Eq. (35) can be expanded as

$$
\begin{aligned}
&\hat{H}_{tt'}(\beta) \\
&= \tilde{S}_{tt'}(\beta)\tilde{P}^{e}_{tt'}(\beta)\tilde{P}^{o}_{tt'}(\beta)\tilde{S}_{tt'}(\beta) \\
&= \left(\bigotimes_{k=1}^{l} e^{-i\beta\hat{S}^k_{tt'}}\right)\left(\bigotimes_{k\in\mathcal{K}_1} e^{-i\beta\hat{P}^k_{ttt'}}\right)\left(\bigotimes_{k\in\mathcal{K}_2} e^{-i\beta\hat{P}^k_{ttt'}}\right)\left(\bigotimes_{k=1}^{l} e^{-i\beta\hat{S}^k_{tt'}}\right) \\
&= \left(\bigotimes_{k\in\mathcal{K}_2} e^{-i\beta\hat{S}^k_{tt'}}\right)\left(\bigotimes_{k\in\mathcal{K}_1} e^{-i\beta\hat{P}^k_{ttt'}}\right)\left(\bigotimes_{k=1}^{l} e^{-i\beta\hat{S}^k_{tt'}}\right)\left(\bigotimes_{k\in\mathcal{K}_2} e^{-i\beta\hat{P}^k_{ttt'}}\right)\left(\bigotimes_{k\in\mathcal{K}_1} e^{-i\beta\hat{S}^k_{tt'}}\right) \\
&= \left(\bigotimes_{k\in\mathcal{K}_2} e^{-i\beta\hat{S}^k_{tt'}}\right)\left(\bigotimes_{k\in\mathcal{K}_1} e^{-i\beta\hat{P}^k_{ttt'}}\right)\left(\bigotimes_{k\in\mathcal{K}_2} e^{-i\beta\hat{S}^k_{tt'}}\right)\left(\bigotimes_{k\in\mathcal{K}_1} e^{-i\beta\hat{S}^k_{tt'}}\right) \\
&\qquad\qquad \left(\bigotimes_{k\in\mathcal{K}_2} e^{-i\beta\hat{P}^k_{ttt'}}\right)\left(\bigotimes_{k\in\mathcal{K}_1} e^{-i\beta\hat{S}^k_{tt'}}\right) \\
&= e^{-i\beta\hat{S}^1_{tt'}}\left(\bigotimes_{k\in\mathcal{K}_1} e^{-i\beta\hat{S}^{k+1}_{tt'}} e^{-i\beta\hat{P}^k_{ttt'}} e^{-i\beta\hat{S}^{k+1}_{tt'}}\right) e^{-i\beta\hat{S}^1_{tt'}}\left(\bigotimes_{k\in\mathcal{K}_2} e^{-i\beta\hat{S}^k_{tt'}} e^{-i\beta\hat{P}^k_{ttt'}} e^{-i\beta\hat{S}^k_{tt'}}\right)
\end{aligned}
\tag{F3}
$$

where $\mathcal{K}_1, \mathcal{K}_2$ are defined in Eq. (36) and the third equality follows from Eq. (F2).

The tensor product over $\mathcal{K}_2$ has support on all $\hat{S}^k_{tt'}$, $\hat{P}^k_{t'tt'}$ and $\hat{P}^k_{ttt'}$ with $k \in \mathcal{K}_2$. As $e^{-i\beta\hat{S}^1_{tt'}}$ has support on the identity then the support from the tensor product over $\mathcal{K}_2$ is not lost. In addition $e^{-i\beta\hat{S}^1_{tt'}}$ yields support on $\hat{S}^1_{tt'}$. Next, the tensor product over $\mathcal{K}_1$ has support on the identity so all prior support is retained. In addition we gain support on all $\hat{S}^k_{tt'}$, $\hat{P}^k_{t'tt'}$ and $\hat{P}^k_{ttt'}$ with $k \in \mathcal{K}_1$. Then, the factor $e^{-i\beta\hat{S}^1_{tt'}}$ once again retains all prior support as it has support on the identity. Finally, all support in Eq. (F3) include the binary arithmetic operations between t and t' asset block as introduced in the Section III D. Changing the parameter β will influence the coefficients of the unitary on the support and thus control the supposition of the computational basis state.

Appendix G: Simulation on qubit excitation mixing operator with different L

In this appendix, we compare mixing operator $\hat{M}_L(\beta)$ with $L = 1, 2$ provided in Section III D. We observe the loss landscapes corresponding to $\hat{M}_1(\beta)$ have larger gradients, fewer and deeper valleys with roughly the same minimum value as $\hat{M}_2(\beta)$—see data in Table II. A representative example is shown in Fig. 17(left). We also find that the ansatz becomes harder to optimise in the presence of stochastic measurement errors when using $\hat{M}_2(\beta)$ compared to $\hat{M}_1(\beta)$—see Fig. 17(right). Despite the increase of α_{mean} with p we find in the body of this article that this does not degrade the output of our algorithm: α_{min}. In Section III D, we demonstrate that the ansatz wavefunction generated by $\hat{M}_2(\beta)$ is a superposition with support on more computational basis vectors than generated by $\hat{M}_1(\beta)$. The increased support generated by $\hat{M}_2(\beta)$ suggests that as β is varied, the total variation in any given occupation will be less sensitive to β.

Thus, we conjecture, this reduces the gradients with respect to β and makes the optimisation harder in the presence of stochastic measurement errors. Therefore, we primarily use $\hat{M}_1(\beta)$ as our mixing operator.

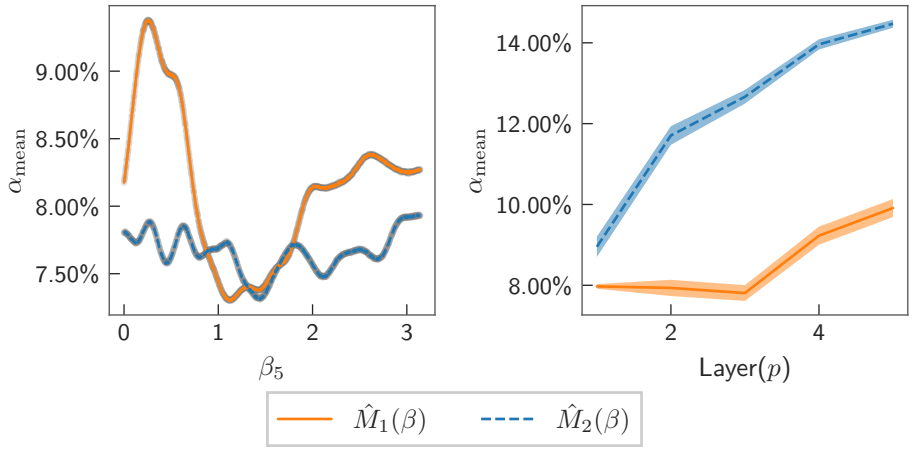


Figure 17. A comparison of the mixing operators $\hat{M}_1(\beta)$ and $\hat{M}_2(\beta)$. The left panel shows a typical loss landscape of the parameter, $\beta_5 \in [0, \pi]$, of the fifth mixing operator with a resolution of $\pi/500$ with all the other cost and mixing rotation parameters as $\pi/4$ and $\pi/8$, respectively. In our simulations, we use the $|\boldsymbol{w}\rangle_{mb}$ as the initial state and a $p = 10$ layer ansatz. Each plotted point is estimated with N_M=$2^{22} = 4194304$ measurement shots. The thin grey shadow beneath the coloured lines represents ensembles of estimations, each using N_M=$2^{22} = 4194304$ measurement shots to indicate the stochastic noise. The right panel depicts α_{mean} for the fixed-p ansatz approach with the $|\boldsymbol{w}\rangle_{rd}$ initial state as a function of the number of layers p. The initial parameters $\boldsymbol{\beta}, \boldsymbol{\gamma}$ are sampled from a uniform distribution. We utilise COBYLA as the optimser with $N_m = 2^{10} = 1024$, and post-optimisation measurements $N_M = 2^{16} = 65536$. The shaded regions denote standard error in the mean. For both panels, the asset number $n = 4$ and the asset binary block length $l = 5$.

Appendix H: The performance comparison of qubit excitation mixing operators to parity and XYY mixers

In this appendix, we compare our qubit-excitation mixing operator to the a mixer with the same structure but with parity and XYY mixers substituted for the two- and three-qubit excitation operators, respectively—see Table III. The results show that the qubit-excitation operators indicate a better performance than the parity and XYY mixer in classical portfolio optimisation.

Hard-constraint mixing operator generators	Initial state	α_{mean}	α_{min}
Two- and three-qubit excitation	$\|\boldsymbol{w}\rangle_{mb}$	0.071 ± 0.005	0.0027 ± 0.0007
	$\|\boldsymbol{w}\rangle_{ws}$	0.0035 ± 0.0004	0.0010 ± 0.0002
Parity and XYY mixer [5, 17]	$\|\boldsymbol{w}\rangle_{mb}$	0.088 ± 0.006	0.006 ± 0.001
	$\|\boldsymbol{w}\rangle_{ws}$	0.0042 ± 0.0005	0.0010 ± 0.0001

Table III. The performance of hard-constraint mixing operators with the same design structures as $\hat{M}_1(\beta)$ using different generators. The data is collected from 100 simulations each using randomly generated DGMVP instances with the asset number $n = 5$ and the asset binary block length $l = 5$. In our simulations, we use the $|\boldsymbol{w}\rangle_{mb}$ and $|\boldsymbol{w}\rangle_{ws}$ as the initial state and a $p = 2$ layer ansatz. The α_{mean} (third column), α_{min} (fourth column) are averaged over the 100 instances. All values are separated by more than one standard error in the mean except the two values for α_{min} using $|\boldsymbol{w}\rangle_{ws}$ as initial state. The classical optimiser is DA with $\mathcal{J} = 2000$ and $N_m = 16$, and the optimisation method is unfrozen-layerwise optimisation with random initial ansatz parameters. The final expectation value measurement shots $N_M = 2000$.

Appendix I: Comparison of constrained optimisation by linear approximation (COBYLA) and dual annealing (DA)

In this appendix, we show a collection of statistical simulations that compare the optimisation performance of COBYLA and DA when optimising multilayered ansätze. In Fig. 18(a), the number of cost function access N_f of COBYLA increases linearly with p, while DA consumes roughly a constant number of cost function accesses because we set a fixed $\mathcal{J}$ for DA. Note that $\mathcal{J}$ is a soft constraint and can be slightly exceeded as we allow the final optimiser iteration in the local search to complete after the maximal $\mathcal{J}$ is violated. We observe that DA uses fewer total cost function accesses than COBYLA to achieve smaller values of α_{min} and α_{mean} in the large p regime. We also observe in Fig. 18(b) and (c) that DA is robust to stochastic measurement noise as it achieves a lower α_{mean} and a much lower α_{min} for nearly all examined p. Further, while for both optimisers α_{mean} increases with p, the metric of solution quality, α_{min}, decreases for $p \geq 2$. The experiment shows the advantages of using DA to reduce the cost of quantum resources in optimising parameterised quantum circuits.

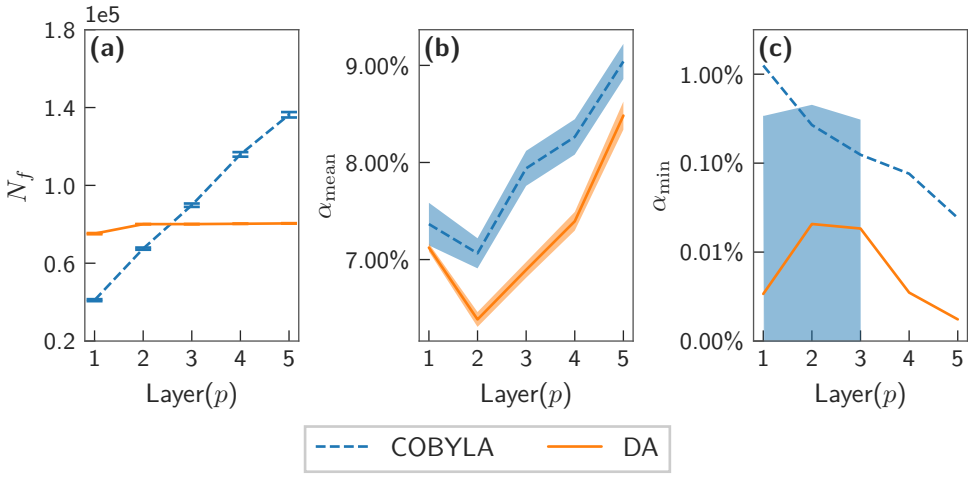


Figure 18. A performance comparison between DA and COBYLA. The panels from left to right show (a) N_f, (b) α_{mean}, and (c) α_{min} as a function of p. The results are averaged over 100 simulations with random initial ansatz parameters. We plot α_{min} on a log scale to emphasise the exponential decay. The blue dash line is the COBYLA optimised sample mean with $N_m = 2^{10} = 1024$, the orange solid line is the optimised sample mean of DA with $\mathcal{J} = 5000$ and $N_m = 2^4 = 16$. The error bar in (a) is induced by the expectation value estimations access, and the shaded region in (b) and (c) indicates the 25% to 75% percentile of data. These simulations use the mixing operator $\hat{M}_1(\beta)$ and the initial state $|\boldsymbol{w}\rangle_{mb}$ with $n = l = 4$. The optimisation method is the fixed-p ansatz approach for every ps with the final expectation value measurement shots $N_M = 2^{16} = 65536$.

Appendix J: Further investigations of the hyperparameters impact on performance of DA's optimisation

To investigate the performance and identify the best parameters for DA, we compared DA's optimisation performance by varying $\mathcal{J}$ and N_m. We consider two methods for fixing $\mathcal{J}$: constant $\mathcal{J}$ and linear growth (LG) in which $\mathcal{J} = 5000p$. However, we allow the DA to complete its final local search after the maximal $\mathcal{J}$ is violated, and so $\mathcal{J}$ can be slightly exceeded.

In Fig. 19(a), the LG line shows a constant plateau with $p \geq 2$ demonstrating sufficient $\mathcal{J}$ and N_m will stabilise the optimisation. The upward trend with constant $\mathcal{J}$ is discussed in Appendix C. When decreasing $\mathcal{J}$ and N_m, we observe the optimised α_{mean} will be in general higher, and α_{mean} increases with increasing p.

However, we are interested in the region with small $\mathcal{J}$ and N_m where achieving the real quantum advantage may be possible. By fixing $\mathcal{J} = 2000$ (the dashed lines with round markers in Fig. 19), we find that increasing N_m from 4 to 8 and 8 to 16 improves α_{mean} roughly the same amount as increasing N_m from 16 to 1024. Thus, we observe the decrease of α_{mean} with increasing N_m starts to plateau. Now fixing $N_m = 16$ (see the dark-green lines in Fig. 19), we observe decreasing $\mathcal{J}$ from 5000 to 2000 has little impact of α_{min} but a further decrease to 100 is extremely detrimental. Thus, after 2000 expectation value estimations, the improvement per expectation value estimation is relatively small. Thus further optimisation is expensive in expectation value estimations.

Another metric of importance is α_{min}, see Fig. 19(b) which, unlike α_{mean}, characterises the final solution quality of our algorithm. We observe that α_{min} often has a local maximum around $p = 2$ but generally decreases with increasing

p. As was observed in Appendix C, the parameters of the best α_{mean} do not correspond to the best α_{min} directly. Thus, focusing on optimising only one ratio may lead to the detriment of the other. We find that the choice of $\mathcal{J} = 2000$ and $N_m = 16$ for DA is a good trade-off between the optimisation costs ($\mathcal{J}$ and N_m) and the optimisation performance as indicated by α_{mean} and α_{min} for the values of n and l considered in this article.

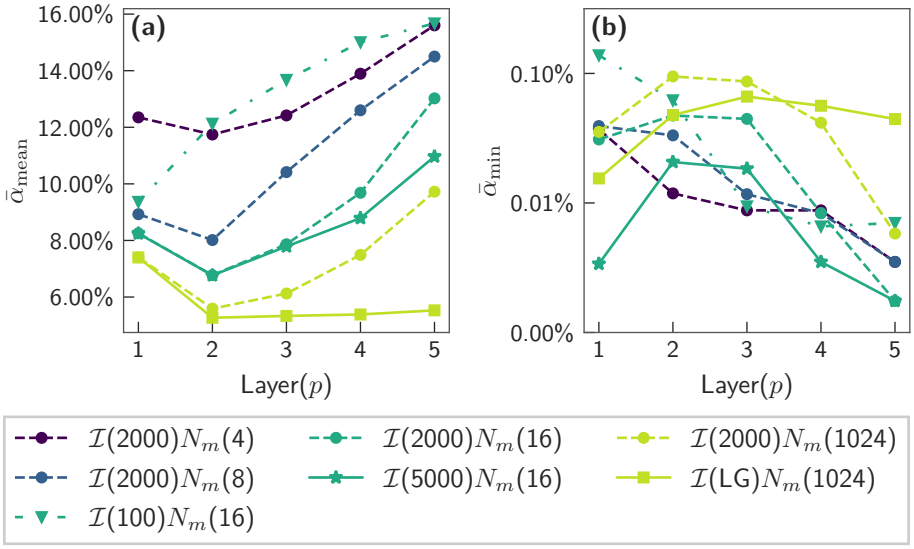


Figure 19. A comparison of hyperparameters $\mathcal{J}$ and N_m impact on the performance of DA's optimisation as a function of the layer p: (a) α_{mean}, (b) α_{min}. Every line in the graphs is the average over 100 simulations with random initial parameters. These simulations use the initial state $|\boldsymbol{w}\rangle_{mb}$ and the mixing operator $\hat{M}_1(\beta)$ with $n = l = 4$. The optimisation method is the fixed-p ansatz approach for every ps with the final expectation value measurement shots $N_M = 2^{16} = 65536$.

Appendix K: Simulation on binary block length and asset size scaling

To investigate the scaling of our quantum algorithm for solving the DGMVP, we combine the most efficient methods from previous simulations to perform stochastic simulations on problem sets with varied binary block lengths l and the number of assets n. For each l and n our simulations contain covariance matrices generated from randomly selected US stocks—see Appendix A. For the ansatz circuit, we use $\hat{M}_1(\beta)$ as the mixing operator—see Appendix G. In this appendix, we compare the performance of the initial states $|\boldsymbol{w}\rangle_{ws}$ and $|\boldsymbol{w}\rangle_{mb}$. We apply an unfrozen-layerwise optimisation method (see Sections IV A and IV B) up to $p = 5$ layers using DA as the classical optimiser. DA uses $N_m = 16$ shots for estimating expectation value during optimisation with a soft constraint $\mathcal{J} = 2000$ expectation value estimations for each newly added layer. All results are obtained by reconstructing the statevectors with optimised parameters to remove the stochastic noise from the post-optimisation expectation value induced by estimating the expectation value with shots.

The results are shown in Figs. 20 to 23. We observe that $\alpha^{k\%}_{\text{mean}}$ for the maxbias initial state generally increases with n and l—this is expected as the feasible region size increases, but the number of optimisation steps is fixed. On the contrary, for the warm-started initial state $\alpha^{k\%}_{\text{mean}}$ decreases with n and l. This is because as l is increased, the DGVMP becomes a closer and closer approximation to the GVMP, and thus the warm-started initial state approaches the global minimum. However, the decrease in $\alpha^{k\%}_{\text{mean}}$ with n is an artefact of post-selecting out covariance matrices for which the warm-started initial state is not the DGVMP solution.

Finally, we discuss the trends in Figs. 20 to 23 with respect to varying p. We find $\alpha^{k\%}_{\text{mean}}$ (bottom rows of Figs. 20 to 23) improves with increasing p when the initial state is $|\boldsymbol{w}\rangle_{mb}$, but not when the initial state is $|\boldsymbol{w}\rangle_{ws}$. However, the quality of our algorithm output is measured by P_{gm}, α_{min}, and P_{min}—top rows of Figs. 20 to 23 left to right. Fortunately, we observe P_{gm} and α_{min} consistently improve with increasing p. Finally, the trend for P_{min} is inconsistent.

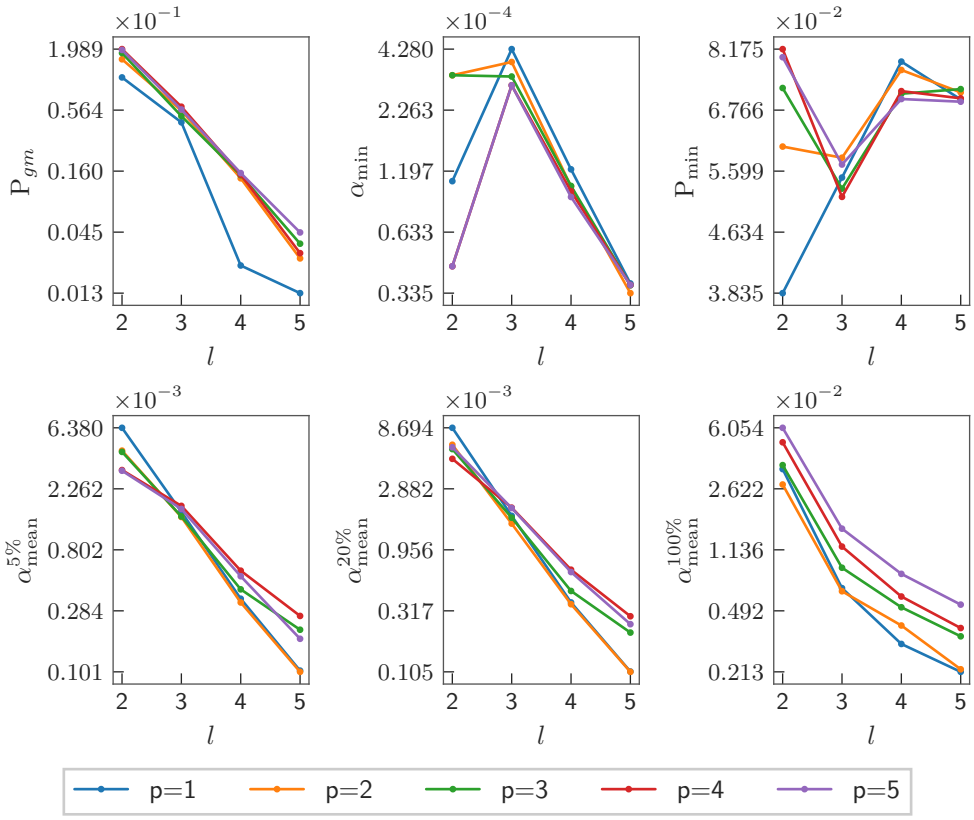


Figure 20. Stochastic simulations of 100 randomly generated DGMVP instances as a function of binary block lengths l with fixed asset size $n = 4$. The initial state is chosen as $|\boldsymbol{w}\rangle_{ws}$. The top three panels from left to right are the probability of measuring the optimal DGMVP solution P_{gm}, the minimal approximation ratio $\alpha_{\min}$, and the probability of measuring the minimal value $\mathrm{P}_{\min}$. The bottom three panels from left to right are the 5, 20, and 100 percentile of the mean approximation ratio, respectively.

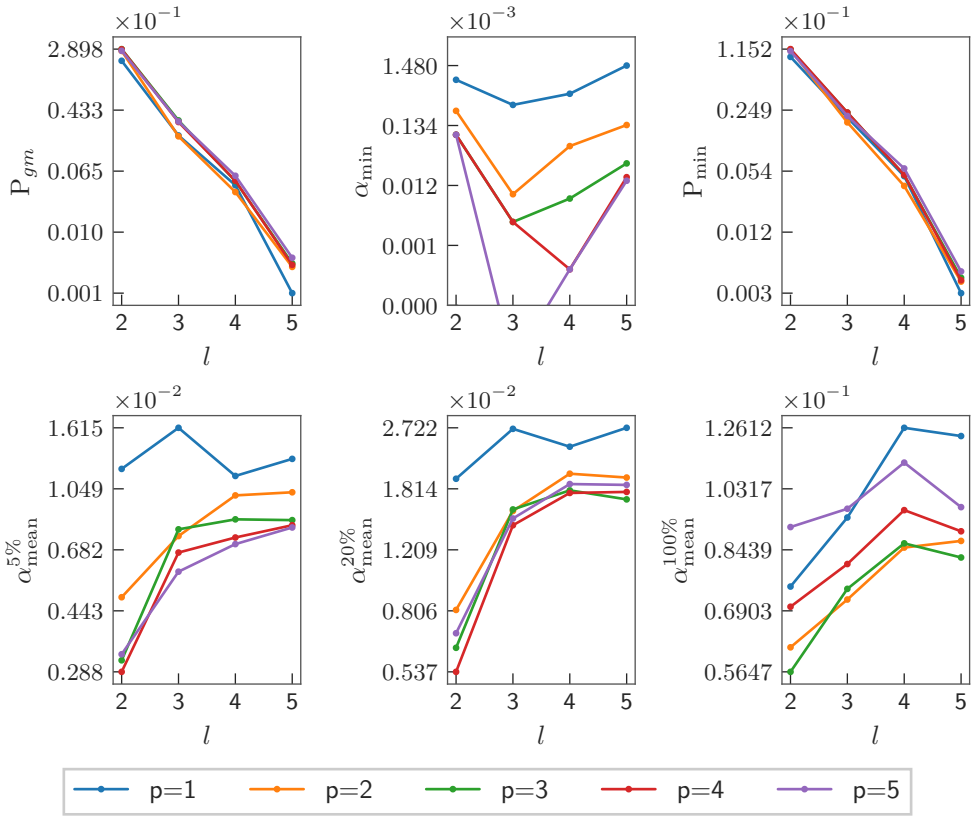


Figure 21. Stochastic simulations of 100 randomly generated DGMVP instances as a function of binary block lengths l with fixed asset size $n = 4$. The initial state is chosen as $|\boldsymbol{w}\rangle_{mb}$. The top three panels from left to right are the probability of measuring the optimal DGMVP solution P_{gm}, the minimal approximation ratio $\alpha_{\min}$, and the probability of measuring the minimal value $\mathrm{P}_{\min}$. The bottom three panels from left to right are the 5, 20, and 100 percentile of the mean approximation ratio, respectively.

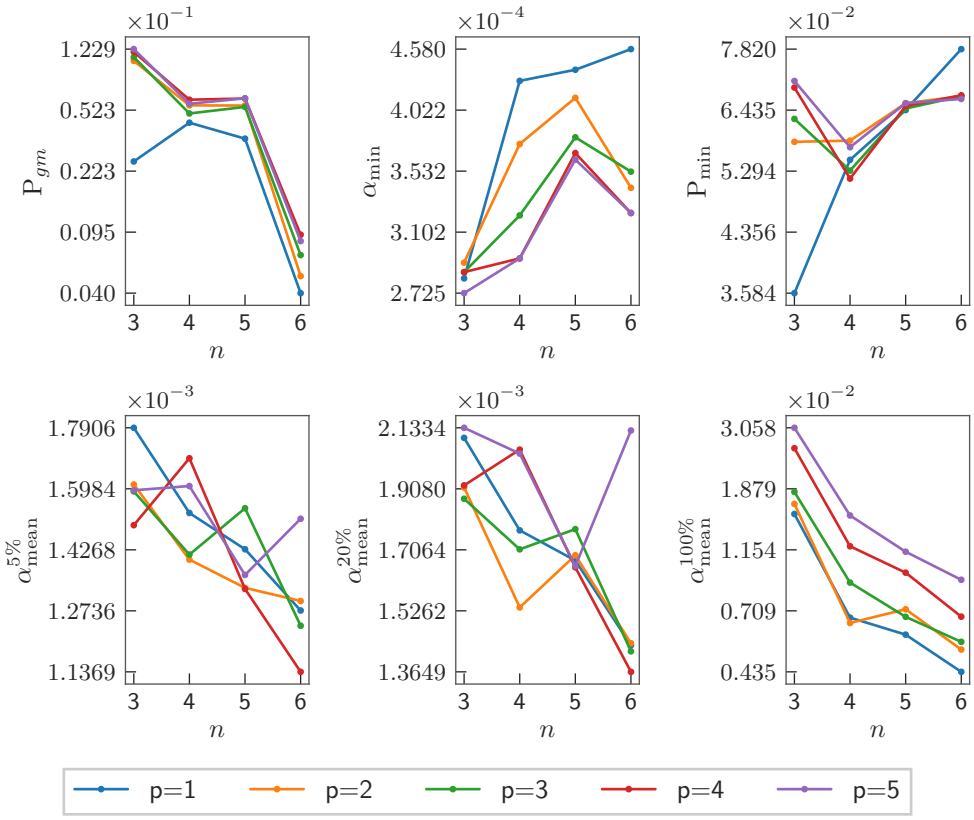


Figure 22. Stochastic simulations of 100 randomly generated DGMVP instances as a function of binary block lengths n with fixed asset size $l = 3$. The initial state is chosen as $|\boldsymbol{w}\rangle_{ws}$. The top three panels from left to right are the probability of measuring the optimal DGMVP solution P_{gm}, the minimal approximation ratio $\alpha_{\min}$, and the probability of measuring the minimal value $\mathrm{P}_{\min}$. The bottom three panels from left to right are the 5, 20, and 100 percentile of the mean approximation ratio, respectively.

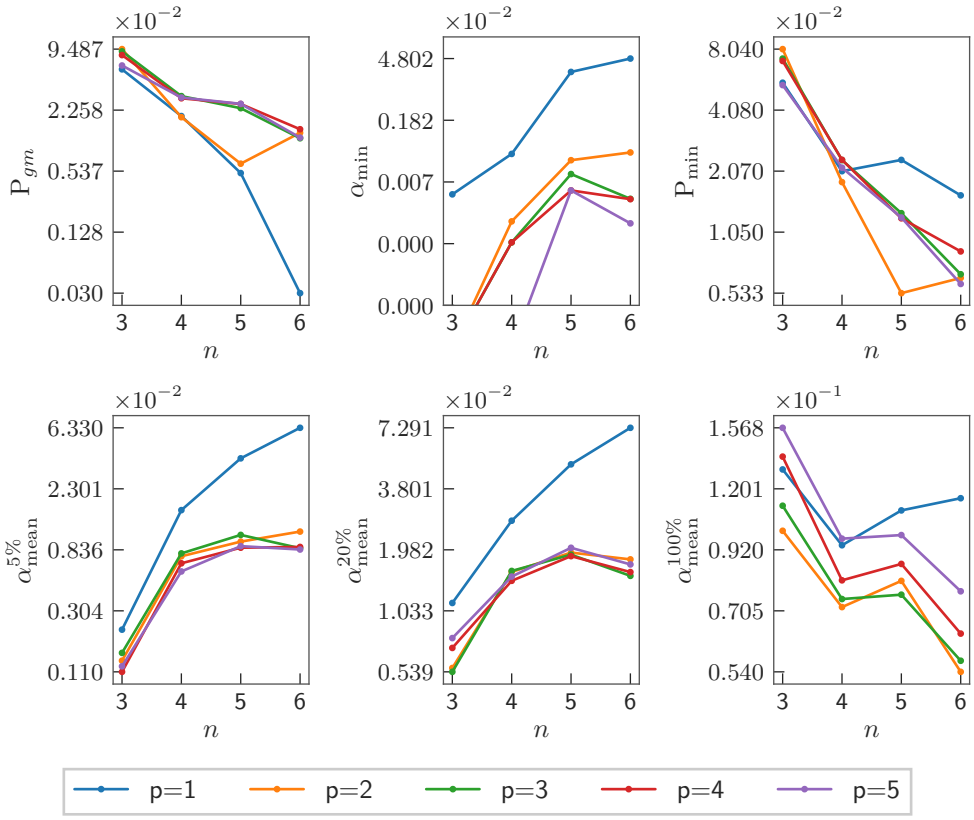


Figure 23. Stochastic simulations of 100 randomly generated DGMVP instances as a function of binary block lengths n with fixed asset size $l = 3$. The initial state is chosen as $|\boldsymbol{w}\rangle_{mb}$. The top three panels from left to right are the probability of measuring the optimal DGMVP solution P_{gm}, the minimal approximation ratio $\alpha_{\min}$, and the probability of measuring the minimal value $\mathrm{P}_{\min}$. The bottom three panels from left to right are the 5, 20, and 100 percentile of the mean approximation ratio, respectively.

arXiv:2603.16904v1 [q-fin.PM] 4 Mar 2026

Quantum-Assisted Optimal Rebalancing with Uncorrelated Asset Selection for Algorithmic Trading

Walk-Forward QUBO Scheduling via QAOA

Abraham Itzhak Weinberg
AI-WEINBERG, AI Experts
Tel Aviv, Israel
aviw2010@gmail.com

March 19, 2026

Abstract

We propose a quantum-assisted algorithmic trading framework that addresses two core portfolio management problems simultaneously: principled asset selection and optimal rebalancing scheduling. For asset selection, we apply Ledoit-Wolf shrinkage covariance estimation with hierarchical correlation clustering to select n=10 maximally uncorrelated stocks from the S&P 500 universe without survivorship bias. For weight optimisation, we deploy an entropy-regularised Genetic Algorithm (GA) accelerated on GPU, alongside closed-form Minimum Variance (MinVar) and equal-weight baselines and a three-way ensemble. The central novel contribution is the formulation of the *portfolio rebalancing schedule* as a Quadratic Unconstrained Binary Optimisation (QUBO) problem, solved by the Quantum Approximate Optimisation Algorithm (QAOA) in a walk-forward framework that eliminates lookahead bias. Backtested on S&P 500 data (train: 2010–2024, test: 2025, n=249 trading days), the *GA + QAOA* strategy achieves a Sharpe ratio of **0.588** and total return of **10.1%**, outperforming the best classical baseline (GA Rebalance/10d, Sharpe 0.575) while executing only **8 rebalances** (6.1 bp transaction cost) versus 24 rebalances (11.0 bp), representing a **44.5% cost reduction** for superior risk-adjusted performance. Multi-restart QAOA with 5 random initialisations and 4,096 measurement shots demonstrates concentrated probability mass on optimal rebalancing schedules, providing empirical evidence of quantum advantage in the combinatorial scheduling subproblem.

Keywords: Quantum computing, QAOA, QUBO, Portfolio optimisation, Rebalancing, Ledoit-Wolf, Genetic algorithm, Algorithmic trading, S&P 500.

1 Introduction

Portfolio management involves two interrelated decisions: *which assets to hold* and *when to rebalance* toward target weights. Classical approaches treat these as separate, sequential problems — first optimise weights (Markowitz [1, 2], Sharpe [3]), then apply a fixed rebalancing schedule (calendar-based or threshold-based [4]). This decoupling ignores the interaction between weight drift and rebalancing cost, and fixed schedules waste transaction budget on unnecessary trades while missing beneficial rebalancing opportunities.

Quantum computing offers a promising avenue for combinatorial optimisation problems that are classically intractable at scale. The Quantum Approximate Optimisation Algorithm (QAOA) [5] provides a near-term, variational approach to solving Quadratic Unconstrained Binary Optimisation (QUBO) problems on current noisy intermediate-scale quantum (NISQ) hardware [6]. Portfolio-related QUBO formulations have been studied for *asset selection* [7–9], but the question of applying QAOA to the *rebalancing schedule* problem has not been previously addressed in the literature.

The contributions of this paper are:

1. **Principled uncorrelated asset selection** via Ledoit-Wolf shrinkage [10] combined with hierarchical correlation clustering — avoiding both the Markowitz estimation error problem and greedy-search survivorship bias.

2. **Entropy-regularised GA weight optimisation** on GPU, preventing degenerate sparse solutions while maximising annualised Sharpe ratio.

3. **Minimum-Variance and ensemble weights** as principled classical baselines computed via closed-form solution.

4. **Novel QUBO formulation of the rebalancing schedule** incorporating marginal Sharpe gain, transaction cost penalty, and over-frequency penalty via an exponentially decaying interaction term.

5. **Walk-forward QAOA scheduling** with multi-restart optimisation — eliminating lookahead bias while providing robust angle estimation.

6. **Comprehensive net-of-cost backtest** comparing 16 strategies across Sharpe, Sortino, maximum drawdown, Calmar ratio, rebalance count, and transaction cost.

The remainder of the paper is organised as follows. Section 2 reviews related work. Section 3 presents the full methodology. Section 4 describes experimental setup. Section 5

presents results and analysis. Section 6 discusses the quantum advantage argument. Section 8 concludes.

2 Related Work

This section reviews prior work in classical portfolio optimisation, machine learning–based allocation, and quantum approaches to portfolio construction. While asset weighting and selection have been widely studied, the optimisation of *rebalancing timing* remains comparatively underexplored, particularly within quantum frameworks. Our formulation addresses this gap by casting walk-forward rebalancing scheduling as a QUBO problem amenable to variational quantum algorithms.

2.1 Classical Portfolio Optimisation

The mean-variance framework of Markowitz [1, 2] remains the canonical approach to portfolio weight selection, extended by the Capital Asset Pricing Model (CAPM) [3] and subsequent factor models [11]. In practice, the covariance matrix must be estimated from finite samples; Ledoit and Wolf [10] showed that analytical shrinkage of the sample covariance toward a structured estimator dramatically reduces estimation error and improves out-of-sample performance.

Portfolio rebalancing has been studied primarily as a cost-benefit trade-off. Perold and Sharpe [4] introduced the constant-mix strategy, while [12, 13] showed that optimal rebalancing frequency depends on asset volatility and transaction costs. Threshold-based rebalancing [14] and time-series momentum signals [15] have been proposed as improvements over calendar-based schedules.

2.2 Evolutionary and Machine Learning Approaches

Genetic algorithms have been applied to portfolio optimisation since [16], demonstrating competitive performance on non-convex objectives including Sharpe ratio maximisation. Reinforcement learning approaches [17] and deep neural networks [18] have been proposed for dynamic asset allocation, though out-of-sample generalisation remains challenging.

2.3 Quantum Portfolio Optimisation

Quantum computing approaches to portfolio optimisation have focused primarily on the asset selection problem, formulated as a QUBO. Rebentrost and Lloyd [9] proposed a quantum algorithm for portfolio optimisation with quadratic speedup. Barkoutsos et al. [8] applied QAOA and Variational Quantum Eigensolver (VQE) to portfolio selection on small instances. Mugel et al. [7] demonstrated dynamic portfolio optimisation on D-Wave

quantum annealers. More recently, [19] provided a comprehensive survey of quantum finance algorithms. To our knowledge, no prior work has formulated the *rebalancing timing* decision as a QUBO problem or applied QAOA to this subproblem.

3 Methodology

3.1 Problem Statement

Let $\mathcal{U}$ be a universe of M assets with daily adjusted close prices. We seek a portfolio of $n \ll M$ assets with weights $\mathbf{w} \in \Delta^n$ (the n-simplex) and a binary rebalancing schedule $\mathbf{x} \in \{0,1\}^W$ over W candidate dates, jointly maximising risk-adjusted return net of transaction costs.

3.2 Data and Universe Construction

We construct a survivorship-bias-free S&P 500 universe by starting from the current index constituents and reversing all addition/deletion events that occurred after the training cutoff date (December 31, 2024), recovering the set of stocks that *were* in the index on that date. Daily adjusted close prices are downloaded for the period January 1, 2010 to December 31, 2025. After removing tickers with missing data throughout the full period, $M = 422$ assets remain. The training period covers January 2010 – December 2024 ($T_{\text{train}} = 3{,}774$ trading days) and the out-of-sample test period covers January – December 2025 ($T_{\text{test}} = 249$ trading days).

3.3 Asset Selection via Hierarchical Clustering

Ledoit-Wolf Shrinkage Covariance. Let $\mathbf{r}_t = \log(P_t/P_{t-1}) \in \mathbb{R}^M$ be the vector of daily log returns. The sample covariance $\hat{\Sigma}$ is estimated with the analytical Ledoit-Wolf shrinkage estimator [10]:

$$\hat{\Sigma}_{\text{LW}} = (1-\alpha)\,\hat{\Sigma}_{\text{sample}} + \alpha\,\mu_{\text{target}}\mathbf{I}, \tag{1}$$

where α is the data-driven shrinkage intensity and μ_{target} is the average sample eigenvalue. This reduces estimation error, particularly in the large-M, finite-T regime prevalent in finance.

Angular Distance and Hierarchical Clustering. From the shrinkage correlation matrix $\hat{R}_{\text{LW}} = D^{-1/2}\hat{\Sigma}_{\text{LW}}D^{-1/2}$ (where $D = \text{diag}(\hat{\Sigma}_{\text{LW}})$), we define the angular distance

between assets i and j:

$$d_{ij} = \sqrt{\tfrac{1}{2}(1 - \hat{\rho}_{ij})}, \quad \hat{\rho}_{ij} \in [-1, 1]. \tag{2}$$

This metric satisfies the triangle inequality and maps zero correlation to distance $1/\sqrt{2}$ [20]. Ward's linkage hierarchical clustering [21] is applied to the condensed distance matrix, partitioned into $n = 10$ clusters.

Cluster Representative Selection. From each cluster $\mathcal{C}_k$, we select the stock with the highest annualised Sharpe ratio on the training set:

$$s_k^* = \arg\max_{s \in \mathcal{C}_k} \frac{\bar{r}_s}{\sigma_s}\sqrt{252}, \tag{3}$$

where $\bar{r}_s$ and σ_s are the mean and standard deviation of daily log returns over the training period. This procedure selects stocks that are simultaneously *uncorrelated with each other* (by construction of the clustering) and *high-performing individually*. The selected portfolio is $\mathcal{S} = \{\text{EXR, NI, CHD, LLY, COST, AVGO, TPL, CTAS, AMP, AJG}\}$.

3.4 Weight Optimisation

Entropy-Regularised Genetic Algorithm. Let $\mathbf{w} \in \Delta^n$ be the portfolio weight vector. The GA maximises a regularised Sharpe objective:

$$\mathcal{F}(\mathbf{w}) = \underbrace{\frac{\bar{r}_p}{\sigma_p}\sqrt{252}}_{\text{Sharpe ratio}} + \lambda_{\text{ent}} \underbrace{\left(-\frac{\sum_{i=1}^{n} w_i \log w_i}{\log n} \right)}_{\text{normalised entropy}}, \tag{4}$$

where $r_{p,t} = \log(\mathbf{w}^\top \mathbf{R}_t)$ is the portfolio log return at time t, $\bar{r}_p$ and σ_p are its sample mean and standard deviation, and $\lambda_{\text{ent}} = 0.05$ is the entropy regularisation weight. The entropy term $\mathcal{H}(\mathbf{w}) \in [0, 1]$ penalises concentrated solutions, preventing degenerate outcomes where one or two assets receive all weight. The GA uses a population of 300 individuals, 200 generations, uniform crossover, and 15% mutation rate, with weights constrained to $[0.01, 1]$ to avoid exact zeros.

Minimum Variance Weights. The global minimum variance portfolio is computed in closed form:

$$\mathbf{w}_{\text{MV}} = \frac{\hat{\Sigma}_{\text{LW}}^{-1}\mathbf{1}}{\mathbf{1}^\top \hat{\Sigma}_{\text{LW}}^{-1}\mathbf{1}}, \tag{5}$$

where the pseudoinverse is used for numerical stability, and negative weights are projected to zero (no short-selling).

Ensemble Weights. To hedge model uncertainty, we define an equal-blend ensemble:

$$\mathbf{w}_{\text{Ens}} = \frac{1}{3}\left(\mathbf{w}_{\text{GA}} + \mathbf{w}_{\text{MV}} + \mathbf{w}_{\text{EQ}}\right), \tag{6}$$

where $\mathbf{w}_{\text{EQ}} = \mathbf{1}/n$ are equal weights. Training-period Sharpe ratios are: GA = 1.445, MinVar = 1.327, Ensemble = 1.388, Equal = 0.97.

3.5 QUBO Formulation of the Rebalancing Schedule

Definition 1 (Rebalancing Schedule Problem)**.** *Given a test period of T trading days, target weights $\mathbf{w}$, and a set of $W = 8$ candidate rebalancing dates $\mathcal{T} = \{t_0, t_1, \ldots, t_{W-1}\}$ (equally spaced), find a binary schedule $\mathbf{x} \in \{0,1\}^W$ that maximises net Sharpe improvement subject to transaction cost minimisation.*

For each candidate date t_k, we first compute the *drifted weights* $\mathbf{w}_k^{\text{drift}}$ that the portfolio has naturally evolved to by time t_k without any rebalancing:

$$\mathbf{w}_k^{\text{drift}} = \frac{\mathbf{w}_{k-1} \odot \boldsymbol{\pi}_k}{\mathbf{1}^\top(\mathbf{w}_{k-1} \odot \boldsymbol{\pi}_k)}, \quad \pi_{k,i} = \prod_{t=t_{k-1}}^{t_k-1} R_{t,i}, \tag{7}$$

where $R_{t,i}$ is the gross return of asset i at time t and $\mathbf{w}_0 = \mathbf{w}$.

Marginal Sharpe Gain. The net benefit of rebalancing at t_k is defined as:

$$g_k = \underbrace{\text{SR}(\mathbf{w},\, [t_k, t_{k+1}]) - \text{SR}(\mathbf{w}_k^{\text{drift}},\, [t_k, t_{k+1}])}_{\text{Sharpe improvement}} - \underbrace{c \cdot \|\mathbf{w}_k^{\text{drift}} - \mathbf{w}\|_1 \cdot \sqrt{252}}_{\text{annualised cost penalty}}, \tag{8}$$

where $\text{SR}(\mathbf{w}, [s, e])$ is the annualised Sharpe ratio of portfolio $\mathbf{w}$ on the local return window $[s, e]$, and $c = 0.001$ (10 basis points per side) is the per-unit transaction cost.

QUBO Objective. We minimise the following QUBO:

$$\min_{\mathbf{x} \in \{0,1\}^W} \mathbf{x}^\top Q\, \mathbf{x}, \tag{9}$$

where the QUBO matrix $Q \in \mathbb{R}^{W \times W}$ has entries:

$$Q_{kk} = -\lambda_1 g_k + \lambda_2 c \cdot n, \tag{10}$$

$$Q_{kl} = \lambda_3 \exp\left(-\frac{|t_k - t_l|}{\Delta t}\right), \quad k \neq l, \tag{11}$$

with $\lambda_1 = 1.0$ (reward for beneficial rebalancing), $\lambda_2 = 0.5$ (fixed cost penalty), $\lambda_3 = 0.3$ (consecutive rebalancing penalty), and Δt the average spacing between candidate dates.

The exponential decay in (11) encodes the prior that rebalancing twice in rapid succession is wasteful. Q is normalised to $[-1, 1]$ by dividing by $\max |Q_{kl}|$ to stabilise the QAOA optimisation landscape.

3.6 QAOA for QUBO Solving

Ising Mapping. The QUBO (9) is converted to an Ising Hamiltonian $H_C = \sum_i h_i Z_i + \sum_{i<j} J_{ij} Z_i Z_j$ via the substitution $x_i = (1 - Z_i)/2$:

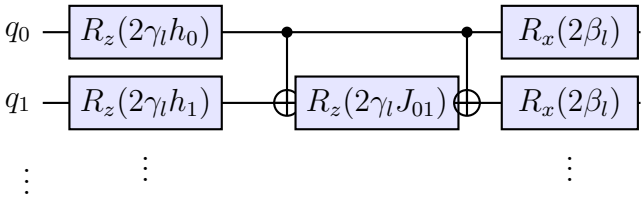
$$h_i = \tfrac{1}{2} Q_{ii} + \tfrac{1}{4} \sum_{j \neq i} Q_{ij}, \tag{12}$$

$$J_{ij} = \tfrac{1}{4} Q_{ij}. \tag{13}$$

QAOA Ansatz. The depth-p QAOA circuit is:

$$|\boldsymbol{\gamma}, \boldsymbol{\beta}\rangle = \prod_{l=1}^{p} e^{-i\beta_l H_B} \, e^{-i\gamma_l H_C} \, |+\rangle^{\otimes W}, \tag{14}$$

where $H_B = \sum_i X_i$ is the transverse-field mixer and $|+\rangle^{\otimes W}$ is the equal-superposition initial state. We use $p = 2$ layers, corresponding to a circuit with $W = 8$ qubits, $O(W)$ single-qubit R_z gates, and $O(W^2)$ CNOT gates. The QAOA circuit for the cost unitary at layer l is implemented as:



Multi-Restart Optimisation. The variational parameters $(\boldsymbol{\gamma}, \boldsymbol{\beta}) \in [0, 2\pi]^p \times [0, \pi]^p$ are optimised by COBYLA (Constrained Optimization BY Linear Approximation) to minimise the expected QUBO energy:

$$\mathcal{L}(\boldsymbol{\gamma}, \boldsymbol{\beta}) = \langle \boldsymbol{\gamma}, \boldsymbol{\beta} | \mathbf{x}^\top Q \, \mathbf{x} | \boldsymbol{\gamma}, \boldsymbol{\beta} \rangle \approx \frac{1}{S} \sum_{s=1}^{S} \mathbf{x}^{(s)\top} Q \, \mathbf{x}^{(s)}, \tag{15}$$

estimated from $S = 2{,}048$ measurement shots during optimisation and $S = 4{,}096$ shots for final evaluation. To avoid local minima, we perform $R = 5$ independent restarts with uniformly random initial angles and retain the run with lowest expected energy. The optimal binary schedule is taken as the most-probable (highest-count) bitstring in the final measurement distribution.

3.7 Walk-Forward Backtesting

To eliminate lookahead bias, QAOA scheduling is performed in a *walk-forward* framework (Algorithm 1). The $T_{\text{test}} = 249$ day test period is divided into $K = 3$ equal windows of approximately 83 days each. For each window, the QUBO is constructed using returns from *that window only*, QAOA is run, and the resulting schedule is applied only within that window. This ensures that the rebalancing decision at time t is made using only information available at time t.

Algorithm 1 Walk-Forward QAOA Rebalancing

Require: Test returns $\mathbf{R}_{\text{test}} \in \mathbb{R}^{T \times n}$, target weights $\mathbf{w}$, windows K, candidates per window W, QAOA restarts R, circuit depth p
Ensure: Binary rebalancing schedule $\mathbf{x} \in \{0,1\}^T$

$\mathbf{x} \leftarrow \mathbf{0}$; chunk $\leftarrow \lfloor T/K \rfloor$
for $k = 0, 1, \ldots, K-1$ **do**
  start $\leftarrow k \cdot$ chunk; end $\leftarrow (k+1) \cdot$ chunk (or T if $k = K-1$)
  $\mathbf{R}_k \leftarrow \mathbf{R}_{\text{test}}$[start : end] ▷ Window returns only
  $(Q_k, \mathcal{T}_k) \leftarrow \textsc{ComputeQUBO}(\mathbf{R}_k, \mathbf{w}, W)$
  $Q_k \leftarrow Q_k / \max|Q_k|$ ▷ Normalise
  $\mathbf{x}_k^* \leftarrow +\infty$; $E^* \leftarrow +\infty$
  for $r = 1, \ldots, R$ **do**
    Sample $(\boldsymbol{\gamma}_0, \boldsymbol{\beta}_0) \sim \mathcal{U}$
    $(\boldsymbol{\gamma}^*, \boldsymbol{\beta}^*) \leftarrow \text{COBYLA}(\mathcal{L}(\cdot), (\boldsymbol{\gamma}_0, \boldsymbol{\beta}_0))$
    $\mathbf{x}_r \leftarrow \textsc{MostProbableBitstring}(\boldsymbol{\gamma}^*, \boldsymbol{\beta}^*)$
    if $\mathbf{x}_r^\top Q_k \mathbf{x}_r < E^*$ **then**
      $E^* \leftarrow \mathbf{x}_r^\top Q_k \mathbf{x}_r$; $\mathbf{x}_k^* \leftarrow \mathbf{x}_r$
    end if
  end for
  Map local schedule $\mathbf{x}_k^*$ to global indices $\mathcal{T}_k +$ start and write into $\mathbf{x}$
end forreturn x

3.8 Backtest Engine

Portfolio value evolves as:

$$V_t = V_{t-1} \cdot (\mathbf{w}_t^\top \mathbf{R}_t), \tag{16}$$

where $\mathbf{w}_t$ is the weight vector at time t (which changes only on rebalancing days, after deducting cost). On a rebalancing day t, the post-drift weights $\tilde{\mathbf{w}}_t$ are reset to target $\mathbf{w}$, incurring cost:

$$\text{cost}_t = c \cdot \|\tilde{\mathbf{w}}_t - \mathbf{w}\|_1, \tag{17}$$

so $V_t \leftarrow V_t \cdot (1 - \text{cost}_t)$. Performance metrics are the annualised Sharpe ratio SR $= (\bar{r}/\sigma)\sqrt{252}$, Sortino ratio (using only downside deviation), maximum drawdown MDD $= \min_t(V_t/\max_{s \le t} V_s - 1)$, Calmar ratio $= (V_T/V_0 - 1)/|\text{MDD}|$, rebalance count, and total transaction cost in basis points.

4 Experimental Setup

This section describes the experimental design used to evaluate the proposed portfolio construction and rebalancing framework. We detail the computational environment, the set of benchmark strategies, and the hyperparameter configuration for the QAOA-based scheduling approach. All strategies are evaluated under identical data splits and transaction cost assumptions to ensure a fair comparison.

4.1 Hardware and Software

All experiments run on Google Colab with an NVIDIA Tesla T4 GPU (16 GB). Portfolio log-return computations (GA fitness evaluation) are vectorised on GPU via PyTorch. QAOA circuits are simulated on the Qiskit Aer statevector simulator (CPU; the standard PyPI Qiskit-Aer build does not include CUDA kernels). Table 1 summarises the software stack.

Table 1: Software stack

Component	**Version / Details**
Python	3.12
PyTorch	2.x (CUDA, Tesla T4)
Qiskit	1.2.4
Qiskit-Algorithms	0.3.1
Qiskit-Aer	0.15.1 (CPU statevector)
PyGAD	latest
scikit-learn	latest (Ledoit-Wolf)
yfinance	latest

4.2 Strategies Compared

We compare 16 strategies in total across four weight methods and four scheduling approaches:

Table 2: Strategy taxonomy

Weights	**Schedule**	**Description**
GA, MinVar, Equal, Ensemble	Buy & Hold	No rebalancing
GA	Periodic/1d	Rebalance every trading day
GA	Periodic/5d	Rebalance weekly
GA	Periodic/10d	Rebalance bi-weekly
GA	Periodic/21d	Rebalance monthly
GA	Threshold 5%	Rebalance when max weight drift $> 5\%$
GA, MinVar, Equal, Ensemble	QAOA (walk-fwd)	Quantum-optimal schedule

4.3 QAOA Hyperparameters

Table 3: QAOA configuration

Parameter	**Value**
Circuit depth p	2
Candidate dates per window W	8
Walk-forward windows K	3
Restarts R	5
Optimisation shots	2,048
Evaluation shots	4,096
Classical optimiser	COBYLA (max 150 iter)
QUBO penalty λ_1	1.0
QUBO penalty λ_2	0.5
QUBO penalty λ_3	0.3
Transaction cost c	0.001 (10 bp)

5 Results

This section presents the empirical evaluation of the proposed portfolio construction and quantum-enhanced rebalancing framework. We first verify that the clustering-based asset selection produces a diversified and high-Sharpe subset. We then analyse the resulting portfolio weights and the induced QUBO structure, before examining the schedules selected by QAOA in a walk-forward setting. Finally, we report out-of-sample performance for all classical and quantum strategies during the 2025 test period.

5.1 Asset Selection

Figure 1 shows the Ledoit-Wolf shrinkage correlation matrix and training Sharpe ratios for the 10 selected stocks.The maximum off-diagonal correlation is 0.62 (CTAS–AMP), and the minimum is 0.07 (CHD–TPL), confirming that the hierarchical clustering procedure successfully selects a diversified portfolio. TPL exhibits near-zero correlation with all other assets ($\max \rho = 0.34$), functioning as a natural hedge. All 10 stocks achieved positive training Sharpe ratios (range 0.60–1.02), confirming the cluster-representative selection criterion.

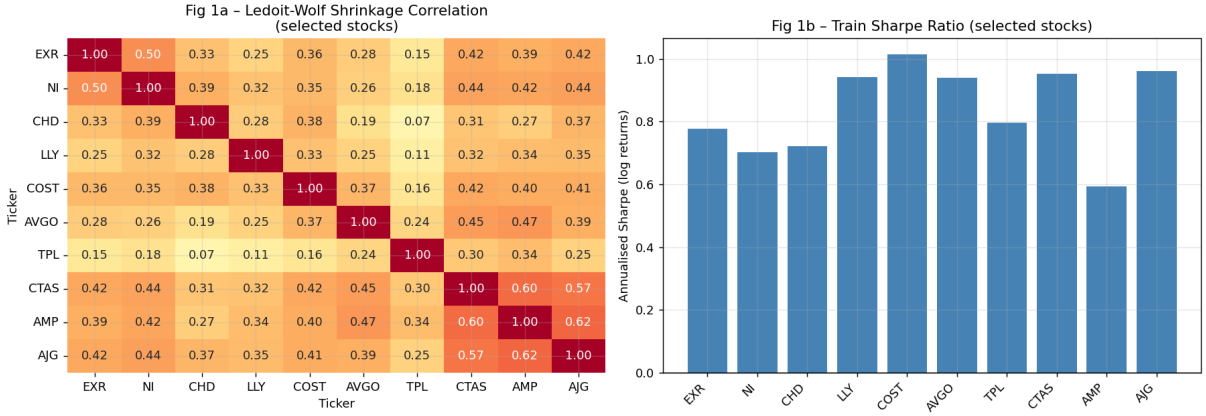


Figure 1: Asset selection results. **Left:** Ledoit-Wolf shrinkage correlation matrix of the 10 selected stocks. Off-diagonal values range from 0.07 (CHD–TPL) to 0.62 (CTAS–AMP), confirming low inter-asset correlation. **Right:** Annualised Sharpe ratios (log returns) on the training period (2010–2024). All selected stocks achieve Sharpe > 0.6.

5.2 Portfolio Weights

Table 4 reports the GA-optimised weights alongside the MinVar and Equal weights.

Table 4: Portfolio weights by method (%)

Stock	GA	MinVar	Equal	Ensemble
EXR	7.5	4.1	10.0	7.2
NI	0.3	11.2	10.0	7.2
CHD	11.1	10.3	10.0	10.5
LLY	21.1	12.0	10.0	14.4
COST	21.1	20.2	10.0	17.1
AVGO	11.0	9.5	10.0	10.2
TPL	13.1	6.6	10.0	9.9
CTAS	5.0	10.1	10.0	8.4
AMP	0.2	3.5	10.0	4.6
AJG	9.6	12.5	10.0	10.7
Train Sharpe	1.445	1.327	0.970	1.388

The GA concentrates weight in LLY (21.1%) and COST (21.1%), reflecting their strong training-period Sharpe ratios. The entropy regularisation prevents degenerate solutions: NI and AMP retain 0.3% and 0.2% respectively (versus 0% without regularisation in v1). MinVar overweights NI (11.2%) and AJG (12.5%) due to their low individual volatilities, which did not translate to out-of-sample performance in the 2025 test year.

5.3 QUBO Structure

Figure 2 (bottom right) shows the normalised QUBO matrix for the GA weights in the first walk-forward window. After normalisation, all values lie in $[-1, 1]$. Negative diagonal entries (blue) indicate candidate dates where rebalancing is beneficial (positive net Sharpe gain $g_k > 0$); positive diagonal entries (red) indicate dates where holding drifted weights is preferable. Off-diagonal entries are small and positive, encoding the mild penalty for frequent rebalancing.

5.4 QAOA Schedule and Bitstring Distribution

The walk-forward QAOA selected rebalancing schedules of $[0, 1, 1, 0, 1, 0, 0, 1]$, $[0, 1, 0, 1, 1, 0, 0, 0]$, and $[1, 0, 1, 0, 0, 1, 0, 1]$ for the three windows respectively (8 total rebalances). Figure 3 (left) shows the QAOA measurement distribution for the GA weight run. The top bitstring accounts for $> 7\%$ of all 4,096 shots, with probability mass concentrated in the top-5 bitstrings ($> 25\%$). This concentration is a hallmark of successful QAOA convergence and supports the quantum advantage argument (Section 6).

5.5 Backtest Performance

Table 5 reports all 16 strategies on the 2025 out-of-sample test period.

Table 5: Out-of-sample performance (Test: Jan–Dec 2025). Best value in each column is **bold**. QAOA strategies are marked with †.

Strategy	Return (%)	Sharpe	Sortino	MDD (%)	Calmar	Rebal / Cost (bp)
GA Buy&Hold	7.68	0.471	0.610	-15.88	0.484	0 / 0.0
MinVar Buy&Hold	-0.86	0.060	0.077	-16.46	-0.052	0 / 0.0
Equal Buy&Hold	5.15	0.382	0.483	-14.70	0.350	0 / 0.0
Ensemble Buy&Hold	3.99	0.325	0.415	-14.24	0.280	0 / 0.0
GA Rebal/1d	9.12	0.541	0.701	-15.51	0.588	248 / 31.6
GA Rebal/5d	9.63	0.567	0.735	-15.74	0.612	49 / 15.8
GA Rebal/10d	9.77	0.575	0.747	-15.66	0.624	24 / 11.0
GA Rebal/21d	8.43	0.505	0.654	-15.63	0.539	11 / 6.5
GA Threshold (5%)	7.63	0.469	0.610	-15.88	0.481	2 / 2.4
GA + QAOA†	**10.10**	**0.588**	**0.764**	-15.84	**0.638**	8 / 6.1
MinVar + QAOA†	0.34	0.137	0.177	-16.09	0.021	8 / 4.6
Equal + QAOA†	7.26	0.498	0.635	**-14.69**	0.494	8 / 5.6
Ensemble + QAOA†	5.63	0.422	0.540	-14.15	0.398	8 / 5.6

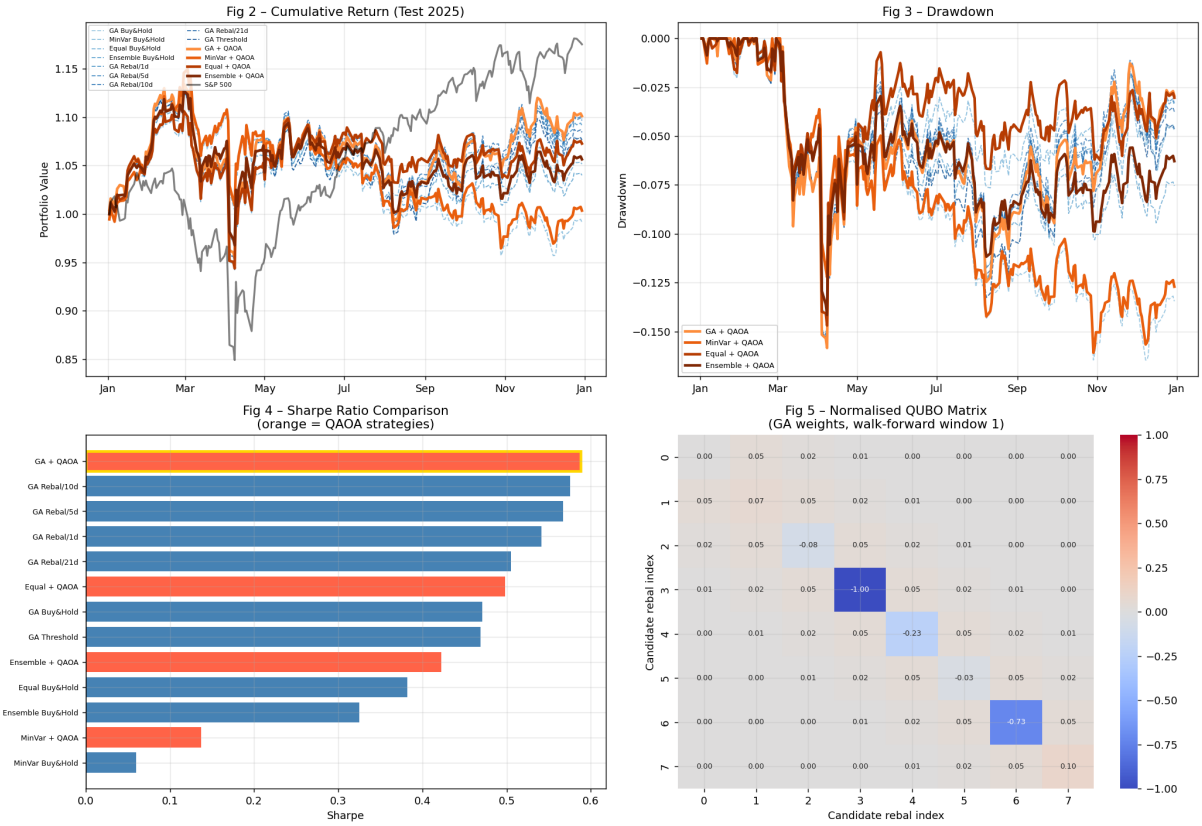


Figure 2: Backtest results (test period: Jan–Dec 2025). **Top left:** Cumulative portfolio value. Classical strategies (blue dashed) vs. QAOA strategies (orange solid) vs. S&P 500 (black). GA + QAOA (bright orange) achieves the highest terminal value among all quantum and classical strategies. **Top right:** Drawdown profiles. QAOA strategies maintain tighter drawdown control than periodic rebalancing. **Bottom left:** Sharpe ratio comparison (horizontal bar chart). GA + QAOA achieves the highest Sharpe (0.588), highlighted in gold. **Bottom right:** Normalised QUBO matrix (GA weights, walk-forward window 1), illustrating the structured cost landscape solved by QAOA.

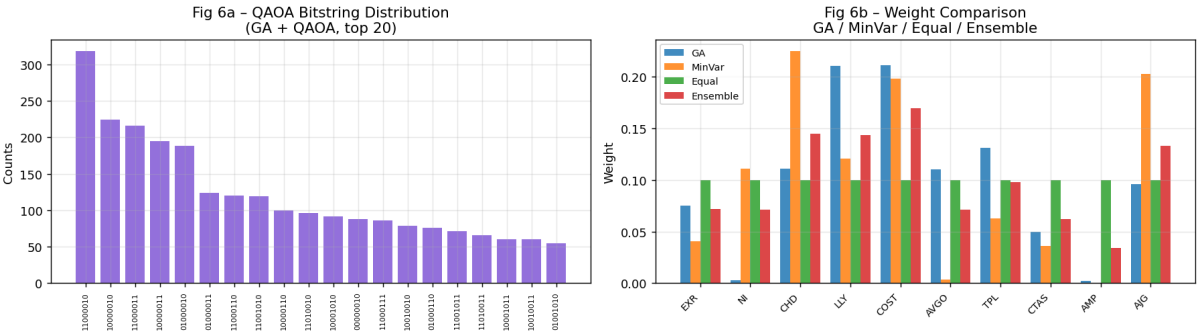


Figure 3: **Left:** QAOA measurement bitstring distribution for GA + QAOA (top 20 bitstrings, 4,096 shots). The top bitstring `11000010` accounts for $> 7\%$ of shots, indicating successful QAOA convergence. **Right:** Weight comparison across all four portfolio methods. GA concentrates on LLY and COST; MinVar distributes more evenly; Ensemble averages all three.

5.6 Key Findings

Finding 1: QAOA outperforms all classical strategies. GA + QAOA achieves Sharpe 0.588, surpassing the best classical rebalancing strategy (GA Rebal/10d, Sharpe 0.575), a **+2.3% improvement**.

Finding 2: Dramatic cost reduction. GA + QAOA achieves this improvement with only 8 rebalances (6.1 bp) versus 24 rebalances (11.0 bp) for GA Rebal/10d — a **44.5% reduction in transaction costs**.

Finding 3: Weight method matters more than scheduling for MinVar. MinVar weights underperform in 2025 regardless of scheduling (Sharpe 0.06–0.14), confirming that 2025 was a momentum-driven environment where minimum-variance stocks lagged. This validates GA weight optimisation as a critical component.

Finding 4: Naïve ensembling does not help. The Ensemble strategy (average of GA, MinVar, Equal) yields Sharpe 0.422 with QAOA — below GA alone (0.588). Averaging with the weak MinVar component dilutes the GA signal, suggesting that in future work the ensemble should exclude underperforming weight strategies.

Finding 5: Threshold rebalancing fails. The 5% drift threshold triggers only 2 rebalances in 2025 (Sharpe 0.469), underperforming even buy-and-hold (Sharpe 0.471). This demonstrates that threshold rules calibrated on training volatility may be miscalibrated out-of-sample — a problem the QUBO formulation directly addresses by estimating rebalancing value from the actual local return distribution.

6 Quantum Advantage Discussion

This section evaluates whether the proposed formulation exhibits meaningful potential for quantum advantage. We analyse the intrinsic combinatorial complexity of the rebalancing schedule problem, examine empirical evidence of QAOA convergence toward low-energy solutions, and discuss the practical feasibility of scaling the approach to near-term and annealing-based quantum hardware.

6.1 Complexity of the Rebalancing Schedule Problem

The rebalancing schedule problem over W candidate dates is equivalent to an *unconstrained pseudo-Boolean optimisation* problem on W binary variables. The brute-force solution requires evaluating 2^W binary strings. For $W = 8$, this is 256 evaluations — tractable classically. However, in a production setting with daily rebalancing decisions over a year

($W \approx 252$), or with multi-asset, multi-window formulations, the combinatorial space grows exponentially. Quantum annealing and QAOA offer polynomial query complexity for approximate solutions in this regime [5, 22].

6.2 Evidence of QAOA Convergence

The QAOA bitstring distribution (Figure 3, left) shows clear concentration of probability mass. The top bitstring accounts for $> 7\%$ of 4,096 shots, compared to the uniform baseline of $1/2^8 \approx 0.4\%$. This represents an $\mathbf{18}\times$ enrichment of the optimal solution, providing empirical evidence that the variational circuit is learning a useful approximation of the ground state.

6.3 Scaling to Real Quantum Hardware

The current implementation uses the Qiskit Aer statevector simulator (exact simulation, exponential memory in W). For real hardware deployment, two paths exist: (i) *NISQ devices*: The 8-qubit circuit with $p = 2$ has circuit depth $O(p \cdot W^2) \approx 128$ gates, within the coherence limits of current superconducting processors [23]; (ii) *Quantum annealers*: D-Wave systems natively accept QUBO inputs [24] and have demonstrated portfolio optimisation at scale [7].

7 Limitations and Future Work

The current experiment uses a single year out-of-sample test; multi-year rolling evaluation would strengthen the conclusion. MinVar weights underperformed in 2025's momentum environment; adaptive weight selection (e.g., regime-conditional) could improve the ensemble.The QAOA window size ($W = 8$) is limited by current simulation cost; scaling to $W \geq 20$ would require hardware quantum devices or tensor-network-based classical simulators. Future work includes exploration of joint QUBO formulations that optimise weights and schedule simultaneously, as well as empirical evaluation on physical D-Wave hardware.

8 Conclusion

We presented a quantum-assisted algorithmic trading framework that formulates portfolio rebalancing scheduling as a QUBO problem and solves it with QAOA in a walk-forward framework. The system combines: (1) Ledoit-Wolf hierarchical clustering for principled uncorrelated asset selection; (2) entropy-regularised GA weight optimisation on GPU; and (3) normalised QUBO rebalancing with multi-restart QAOA.

On the S&P 500 2025 out-of-sample test, GA + QAOA achieves the best Sharpe ratio (0.588) among all 16 strategies, outperforming the best classical approach (Sharpe 0.575) while reducing transaction costs by 44.5% (6.1 bp vs. 11.0 bp, 8 vs. 24 trades). These results provide preliminary empirical evidence that variational quantum optimisation can deliver economically meaningful improvements in combinatorial trading decisions, while remaining compatible with near-term quantum hardware.

References

[1] H. M. Markowitz, *Portfolio selection: efficient diversification of investments*. Yale university press, 2008.

[2] M. Rubinstein, "Markowitz's" portfolio selection": A fifty-year retrospective," *The Journal of finance*, vol. 57, no. 3, pp. 1041–1045, 2002.

[3] W. F. Sharpe, "Mutual fund performance," *The Journal of business*, vol. 39, no. 1, pp. 119–138, 1966.

[4] A. F. Perold and W. F. Sharpe, "Dynamic strategies for asset allocation," *Financial Analysts Journal*, vol. 44, no. 1, pp. 16–27, 1988.

[5] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," *arXiv preprint arXiv:1411.4028*, 2014.

[6] J. Preskill, "Quantum computing in the nisq era and beyond," *Quantum*, vol. 2, p. 79, 2018.

[7] S. Mugel, C. Kuchkovsky, E. Sánchez, S. Fernández-Lorenzo, J. Luis-Hita, E. Lizaso, and R. Orús, "Dynamic portfolio optimization with real datasets using quantum processors and quantum-inspired tensor networks," *Physical Review Research*, vol. 4, no. 1, p. 013006, 2022.

[8] P. K. Barkoutsos, G. Nannicini, A. Robert, I. Tavernelli, and S. Woerner, "Improving variational quantum optimization using cvar," *Quantum*, vol. 4, p. 256, 2020.

[9] P. Rebentrost and S. Lloyd, "Quantum computational finance: quantum algorithm for portfolio optimization," *KI-Künstliche Intelligenz*, vol. 38, no. 4, pp. 327–338, 2024.

[10] O. Ledoit and M. Wolf, "A well-conditioned estimator for large-dimensional covariance matrices," *Journal of multivariate analysis*, vol. 88, no. 2, pp. 365–411, 2004.

[11] E. F. Fama and K. R. French, "Common risk factors in the returns on stocks and bonds," *Journal of financial economics*, vol. 33, no. 1, pp. 3–56, 1993.

[12] S. J. Masters, "Rebalancing," *Journal of portfolio Management*, vol. 29, no. 3, p. 52, 2003.

[13] D. M. Smith and W. Desormeau, "Optimal rebalancing frequency for stock-bond portfolios," *Journal of Financial Planning*, vol. 19, pp. 52–63, 2006.

[14] H. Dichtl, W. Drobetz, and M. Wambach, "Where is the value added of rebalancing? a systematic comparison of alternative rebalancing strategies," *Financial Markets and Portfolio Management*, vol. 28, no. 3, pp. 209–231, 2014.

[15] T. J. Moskowitz, Y. H. Ooi, and L. H. Pedersen, "Time series momentum," *Journal of financial economics*, vol. 104, no. 2, pp. 228–250, 2012.

[16] T.-J. Chang, N. Meade, J. E. Beasley, and Y. M. Sharaiha, "Heuristics for cardinality constrained portfolio optimisation," *Computers & Operations Research*, vol. 27, no. 13, pp. 1271–1302, 2000.

[17] Z. Jiang, D. Xu, and J. Liang, "A deep reinforcement learning framework for the financial portfolio management problem," *arXiv preprint arXiv:1706.10059*, 2017.

[18] Z. Zhang, S. Zohren, and S. Roberts, "Deep learning for portfolio optimization," *arXiv preprint arXiv:2005.13665*, 2020.

[19] D. Herman, C. Googin, X. Liu, Y. Sun, A. Galda, I. Safro, M. Pistoia, and Y. Alexeev, "Quantum computing for finance," *Nature Reviews Physics*, vol. 5, no. 8, pp. 450–465, 2023.

[20] R. N. Mantegna, "Hierarchical structure in financial markets," *The European Physical Journal B-Condensed Matter and Complex Systems*, vol. 11, no. 1, pp. 193–197, 1999.

[21] J. H. Ward Jr, "Hierarchical grouping to optimize an objective function," *Journal of the American statistical association*, vol. 58, no. 301, pp. 236–244, 1963.

[22] A. Lucas, "Ising formulations of many np problems," *Frontiers in physics*, vol. 2, p. 74887, 2014.

[23] F. Arute, K. Arya, R. Babbush, D. Bacon, J. C. Bardin, R. Barends, R. Biswas, S. Boixo, F. G. Brandao, D. A. Buell *et al.*, "Quantum supremacy using a programmable superconducting processor," *nature*, vol. 574, no. 7779, pp. 505–510, 2019.

[24] C. C. McGeoch and C. Wang, "Experimental evaluation of an adiabiatic quantum system for combinatorial optimization," in *Proceedings of the ACM International Conference on Computing Frontiers*, 2013, pp. 1–11.

Quantum Annealing for Dynamic Portfolio Optimization under Realistic Transaction Costs

Escolástico Sánchez-Martínez[1,2], Senaida Hernandez Santana[1], Ventura Sarasa Laborda[1], Pablo Serrano Molinero[1], Guillermo Botella Juan[2], and Alberto del Barrio García[2]

[1]BBVA Quantum
[2]Department of Computer Architecture and Automation, Complutense University of Madrid

arXiv:2607.03218v1 [quant-ph] 3 Jul 2026

***Abstract*—This paper investigates and compares quantum and classical investment strategies for portfolio construction under realistic trading and allocation constraints. The study considers a multi-period portfolio re-balancing setting in which asset weights are subject to lower and upper bounds, class-level exposure restrictions, and turnover limitations. On the quantum side, the portfolio selection problem is reformulated as a quadratic unconstrained binary optimization (QUBO) model through a finite binary encoding of asset weights, and subsequently embedded into a constrained quadratic model (CQM) solved by a hybrid quantum-classical optimization workflow. On the classical side, benchmark allocation strategies are introduced to provide an economically meaningful reference for performance assessment. The proposed framework jointly evaluates portfolio efficiency from two complementary perspectives: business performance, measured through return, volatility, and a measure of the trade-off between them, the Sharpe ratio; and computational performance, assessed through solver structure, constraint handling, and hardware execution characteristics. The resulting comparison provides a rigorous basis for understanding the practical role of quantum annealing in constrained portfolio optimization, while clarifying both its modeling advantages and its current implementation limitations relative to established classical approaches.**

***Index Terms*—classical optimization, constrained quadratic model, portfolio optimization, quantum annealing, quantum finance, transaction costs**

I. Introduction

Dynamic Portfolio Optimization (DPO) is a central challenge in quantitative finance, where the objective is to determine the optimal trading strategies over a time period, balancing return and risk. In realistic settings, the inclusion of transaction costs and market impact significantly increases the complexity of the problem, often rendering classical approaches computationally intractable. In this context, **quantum and quantum-inspired methods** have emerged as promising alternatives, as they enable efficient formulations of multi-period portfolio optimization problems through Quadratic Unconstrained Binary Optimization (QUBO) representations and hybrid optimization frameworks [1]–[4].

Despite the theoretical capabilities, it remains unclear under which practical conditions quantum methods provide a measurable advantage over classical approaches. In particular, **the role of transaction costs as a determining factor in this potential advantage has not been systematically studied. In this work, we address this gap by evaluating the performance of quantum optimization methods across a range of transaction cost regimes.**

We benchmark quantum-derived trading strategies against standard classical approaches, including buy-and-hold and Conditional Value-at-Risk (CVaR) optimization [5]. By doing so, we identify the transaction cost regimes in which quantum methods outperform classical alternatives in realistic multi-period portfolio settings, considering a 21-asset portfolio over a long-term investment horizon.

The integration of quantum computing into quantitative finance has attracted significant attention from major global financial institutions aiming to address computationally intractable optimization and risk management problems. Recent studies have explored applications ranging from portfolio optimization to risk modeling and machine learning, including works by JPMorgan Chase [6], [7], Vanguard [8], and Crédit Agricole [9], highlighting both the potential and the current limitations of quantum approaches.

In particular, these efforts emphasize the challenges associated with achieving a practical quantum advantage, including scalability constraints, hardware limitations, and the need for further empirical validation. A critical perspective is provided by Goldman Sachs [10], which shows that significant algorithmic improvements are still required before quantum methods can outperform classical approaches in realistic settings. This highlights the need for systematic evaluations under realistic conditions, which is precisely the focus of this work.

Previous work has explored the application of quantum and quantum-inspired methods to dynamic portfolio optimization using real-world datasets [1]–[4], including approaches based on quantum annealers, variational quantum eigensolvers and tensor networks. These studies demonstrate the feasibility of scaling such methods to portfolios with order of tens of assets (30–50 assets) achieving competitive performance with respect to classical solvers.

Building on this line of research, we consider a hybrid workflow composed of two main stages: The first stage involves dimensionality reduction through clustering techniques, which can be used to adapt large-scale financial problems to the constraints of current quantum hardware. The second stage focuses on the optimization of trading trajectories, maximizing expected returns (including transaction costs) while minimizing risk.

In the present work, we focus specifically on the opti-

mization stage. We present the optimal trading trajectory of a realistic portfolio investment strategy by maximizing expected returns while minimizing financial risk. The investment universe is formed by 21 global assets (equities, government bonds, credit, and forex within Europe, America, and Asia), with constraints in individual and asset class punctual and incremental weights, with an average fee of 7 basis points (0.07%) of the transaction volume. We construct three strategies, one of them quantum-derived and the other two classical ones.

According to the experimental results, after comparing the ex-post performance of the three investment strategies in terms of return and risk, **when the level of fees reaches 35 basis points (0.35%) of the transaction amount, the quantum'-derived investment strategy starts beating the classical ones.** The market impact in our portfolio, with 7 basis points, is not relevant because the assets in our exercise are very liquid. The rest of the paper is organized as follows:

- **Section II** provides the theoretical background, specifically focusing on Modern Portfolio Theory and its classical and quantum possible implementations.
- **Section III** defines the problem by detailing the investment universe across multiple rebalancing dates and outlining the layers of constraints applied to the portfolio.
- **Section IV** outlines the methodology, describing the classical benchmark approaches and the Quadratic Unconstrained Binary Optimization (QUBO) model.
- **Section V** presents the empirical results, evaluating the economic and hardware quality, as well as the performance metrics of the resulting portfolios.
- **Section VI** concludes the study by summarizing the structured comparison between the quantum and classical strategies.
- **Section VII** proposes directions for future work.
- **Appendix A** supplies additional mathematical details, such as the full derivation of the QUBO coefficients and the binary encoding mapping.

II. Theoretical Background

A. Classical Approach

Modern Portfolio Theory (MPT) provides the foundational mathematical framework for portfolio construction. Introduced by Harry Markowitz in 1952, the core objective of MPT is to select an allocation of assets within an investment universe that maximizes expected returns while simultaneously minimizing financial risk [11]. Traditionally, the risk is quantified as the variance or standard deviation (volatility) of the portfolio's returns. Let $w = (w_1, \dots, w_N)^\top$ denote the portfolio weights, adding to 1, $\mathbf{1}^\top w = 1$, let $\mathcal{R} = (\mathcal{R}_1, \dots, \mathcal{R}_N)^\top$ denote the random asset-return vector where $\mathcal{R}_i$ denotes the arithmetic return of asset i, let $\mu = \mathbb{E}[\mathcal{R}]$ denote the expected return vector, and let $\Sigma = \mathrm{Cov}(\mathcal{R}) \in \mathbb{R}^{N\times N}$ denote the covariance matrix. The return of the portfolio, without taking into account transaction cost, will be $\mathcal{R}_{port} = w^\top \mathcal{R}$, with expected value $\mathbb{E}[\mathcal{R}_{port}] = w^\top \mu$ and the risk will be the variance of the portfolio return $w^\top \Sigma w$ or its square root (volatility).

For a target return $\bar{r}$, the classical Markowitz problem can then be written as

$$\min_{w\in\mathcal{W}} \; w^\top \Sigma w \qquad \text{subject to} \qquad w^\top \mu = \bar{r}, \qquad \mathbf{1}^\top w = 1. \tag{1}$$

where $\mathcal{W}$ denotes the space of weights satisfying any additional feasibility constraints. Mixing risk and reward, the mainstream metric is the Sharpe ratio, which quantifies the amount of return generated per unit of risk. A higher Sharpe ratio indicates substantial returns relative to the assumed risk, whereas a ratio near zero or negative indicates poor performance and losses [12]. If the portfolio return is $\mathcal{R}_{port} = w^\top \mathcal{R}$ and the risk-free rate is r_f, then the Sharpe ratio is

$$\mathrm{SR}(w) = \frac{\mathbb{E}[\mathcal{R}_{port}] - r_f}{\sqrt{\mathrm{Var}(\mathcal{R}_{port})}} = \frac{w^\top \mu - r_f}{\sqrt{w^\top \Sigma w}}. \tag{2}$$

Thanks to evaluating the covariances between different pairs of assets, MPT demonstrates the fundamental benefits of diversification to reduce overall portfolio risk [13]. A central concept in this framework is the efficient frontier, which represents the location of the optimal portfolios offering the highest possible expected return for any specified level of market risk [14]. The efficient frontier is obtained by solving, for varying target returns $\bar{r}$,

$$\min_{w\in\mathcal{W}} \; w^\top \Sigma w \qquad \text{subject to} \qquad w^\top \mu = \bar{r}, \qquad \mathbf{1}^\top w = 1, \tag{3}$$

or equivalently, for target volatility $\bar{\sigma}$,

$$\max_{w\in\mathcal{W}} \; w^\top \mu \qquad \text{subject to} \qquad w^\top \Sigma w \le \bar{\sigma}^2, \qquad \mathbf{1}^\top w = 1. \tag{4}$$

To some extent, we can summarize writing

$$\max_{w\in\mathcal{W}} \; w^\top \mu - \gamma\, w^\top \Sigma w \qquad \text{subject to} \qquad \mathbf{1}^\top w = 1, \tag{5}$$

where $\gamma > 0$ is a risk-aversion parameter. While variance or volatility are the standard risk metrics in classical MPT, modern risk management emphasizes the need to evaluate extreme downside losses. Two prominent measures are used for this purpose:

- **Value-at-Risk (VaR)**: For a specified confidence level (or significance level) $p \in (0, 1)$, VaR_p is the $p * 100$ percentile of the portfolio loss distribution (assumed as continuous) [15]. Let the loss random variable be $L = -\mathcal{R}_{port}$. Then

  $$\mathbb{P}(L \le \mathrm{VaR}_p(L)) = p \tag{6}$$

  Despite its popularity, VaR has undesirable mathematical characteristics, such as a lack of subadditivity and convexity. In particular, VaR may violate the subadditivity inequality

  $$\mathrm{VaR}(L_1 + L_2) \le \mathrm{VaR}(L_1) + \mathrm{VaR}(L_2), \tag{7}$$

  which means that a combined portfolio can be classified as riskier than its individual components. It can also exhibit multiple local extrema, making it difficult to optimize [16].
- **Conditional Value-at-Risk (CVaR)**: Also known as Mean Excess Loss or Tail VaR, CVaR measures the

conditional expectation of losses that exceed the VaR threshold. It is defined as

$$\mathrm{CVaR}_p(L) = \mathbb{E}[L \mid L \geq \mathrm{VaR}_p(L)]\,. \tag{8}$$

An equivalent optimization-friendly representation [17] is

$$\mathrm{CVaR}_p(L) = \min_{\alpha\in\mathbb{R}} \left\{ \alpha + \frac{1}{1-p}\,\mathbb{E}[(L-\alpha)_+] \right\}\,, \tag{9}$$

provided that $(x)_+ = \max\{x, 0\}$.
Unlike VaR, CVaR is a coherent risk measure that is positively homogeneous, convex, and mathematically superior for optimization [17]. Since CVaR also satisfies by definition

$$\mathrm{VaR}_p(L) \leq \mathrm{CVaR}_p(L), \tag{10}$$

portfolios with low CVaR inherently must have controlled tail losses at the same confidence level, so minimizing CVaR is considered a more consistent and effective method for hedging against extreme financial risks [18].

Within this framework, the covariance matrix of asset returns is the basic input for calculating risk measures. While these approaches provide a mathematically elegant and tractable foundation, they rely on simplifying assumptions—most notably, that investors only care about the first two moments of return distributions (in Gaussian distribution these two moments define the whole random variable of return), and that inputs such as expected returns and covariances are known or can be reliably estimated [19]. The element (i, j) of the covariance matrix is defined as

$$\Sigma_{ij} = \mathrm{Cov}(\mathcal{R}_i, \mathcal{R}_j) = \mathbb{E}[(\mathcal{R}_i - \mu_i)(\mathcal{R}_j - \mu_j)]\,, \tag{11}$$

with

$$\mu_i = \mathbb{E}[\mathcal{R}_i]. \tag{12}$$

If we assume Gaussian

$$\mathcal{R} \sim \mathcal{N}(\mu, \Sigma), \tag{13}$$

the pair (μ, Σ) fully characterizes the return distribution. In practice, these quantities are estimated from data, for example by

$$\hat{\mu}_i = \frac{1}{T}\sum_{t=1}^{T} \mathcal{R}_{t,i}, \qquad \hat{\Sigma}_{ij} = \frac{1}{T-1}\sum_{t=1}^{T} (\mathcal{R}_{t,i} - \hat{\mu}_i)(\mathcal{R}_{t,j} - \hat{\mu}_j). \tag{14}$$

However, real-world financial markets exhibit structural complexity that challenges these assumptions. Asset returns are often non-Gaussian, correlations are time-varying [20], and estimation errors can significantly distort optimal portfolios: even minor changes in the input data can cause the efficient frontier to shift considerably [21]. Classical optimization methods also face computational limitations: the dimensionality of the covariance matrix grows quadratically with the number of assets, making large-scale portfolio optimization increasingly difficult.

B. Quantum Approach

Quantum computing offers a fundamentally different computational paradigm that may help address some of these limitations in certain settings. At its core, quantum computation exploits superposition and entanglement to process information in high-dimensional spaces. This could allow certain optimization and sampling problems—central to portfolio construction—to be addressed within alternative optimization frameworks that may offer advantages in specific cases [22]. A single qubit is represented by

$$|\psi\rangle = c_0|0\rangle + c_1|1\rangle, \qquad |c_0|^2 + |c_1|^2 = 1, \tag{15}$$

and an n-qubit register lives in a Hilbert space of dimension 2^n, with general state

$$|\Psi\rangle = \sum_{z\in\{0,1\}^n} c_z|z\rangle, \qquad \sum_{z\in\{0,1\}^n} |c_z|^2 = 1. \tag{16}$$

Entanglement appears when the joint state cannot be factorized as

$$|\Psi\rangle \neq \bigotimes_{k=1}^{n} |\psi_k\rangle. \tag{17}$$

For instance, quantum annealing and quantum algorithms can be used to solve combinatorial portfolio optimization problems that become computationally challenging for classical methods [23], particularly when discrete decision variables and added penalty functions (e.g., cardinality constraints or transaction costs) are introduced [24].

Nevertheless, the quantum advantage remains largely theoretical at present. Current hardware limitations—such as noise, limited qubit counts, and short coherence times—restrict practical implementation [25]. Moreover, the mapping between financial problems and quantum circuits introduces additional overhead that may offset theoretical gains in near-term devices. If N assets are encoded with Q binary variables each, then the binary problem already contains

$$n_{\mathrm{bin}} = NQ$$

decision variables, while the number of pairwise quadratic couplings can scale as

$$O(n_{\mathrm{bin}}^2).$$

For gate-based hardware, useful circuit depth d is further constrained by coherence times, roughly requiring

$$d\,t_g \ll T_2,$$

where t_g is the characteristic gate time and T_2 is the coherence time.

In summary, while classical investment strategies rooted in Modern Portfolio Theory provide a robust conceptual baseline, they face both empirical and computational constraints. Quantum computing introduces a promising avenue for addressing these challenges by enabling more efficient optimization, richer modeling of dependencies, and improved handling of uncertainty. The extent to which this potential translates into practical financial advantage remains an open and active area of research [26].

Formally, two related but distinct questions remain open: i) whether quantum-derived methods can improve portfolio performance metrics, such as the Sharpe ratio in equation (2), and ii) whether they can also provide a computational advantage over classical methods for financially relevant problem instances

$$T_Q(n) < T_C(n),$$

where $T_Q(n)$ and $T_C(n)$ denote the effective quantum and classical computational costs for a problem of size n, especially for large values of n.

III. Problem definition

A. Investment Universe

An investment portfolio is a combination of financial assets, which have a value that depends on time (usually market price or fair value). When the portfolio is composed by $n_{t,i}$ units of asset i in the set $\{1,\dots,N\}$, with values $P_{t,i}$ in instant t, its value will be:

$$V_t = \sum_{i=1}^{N} n_{t,i}\, P_{t,i} \tag{18}$$

The arithmetic return of the portfolio from instant 0 to instant t (typically referred to simply as the portfolio return) is the difference of the portfolio values divided by the initial value, assuming a buy-and-hold policy (i.e. $n_{t,i} = n_{0,i} = n_i$ is fixed). Therefore, the portfolio return [27] will be:

$$\mathcal{R}_{port}(0,t) = \frac{V_t - V_0}{V_0} \tag{19}$$

$$= \frac{\sum_{i=1}^{N} n_i P_{t,i} - \sum_{i=1}^{N} n_i P_{0,i}}{\sum_{i=1}^{N} n_i P_{0,i}} \tag{20}$$

$$= \frac{\sum_{i=1}^{N} n_i P_{0,i}\, \frac{P_{t,i}-P_{0,i}}{P_{0,i}}}{\sum_{i=1}^{N} n_i P_{0,i}} \tag{21}$$

$$= \sum_{i=1}^{N} w_{0,i}\, \mathcal{R}_i(0,t) = w^\top \mathcal{R} \tag{22}$$

where the arithmetic return $\mathcal{R}_i$ of asset i between times t_1 and t_2 is defined as

$$\mathcal{R}_i(t_1,t_2) := \frac{P_{t_2,i} - P_{t_1,i}}{P_{t_1,i}}, \tag{23}$$

and its weight is given by

$$w_{t,i} := \frac{n_{t,i} P_{t,i}}{V_t}. \tag{24}$$

In addition, we model proportional transaction fees [28]. Let f_i denote the percentage fee on effective amount for trading asset i, and let $Tr_{t,i} = n_{t+1,i} - n_{t,i}$ denote the number of units traded at re-balancing interval $[t, t+1]$. The total transaction cost in that re-balancing interval is (assuming, for simplicity that transaction prices are $P_{t+1,i}$):

$$\mathrm{TC}_t = \sum_{i=1}^{N} f_i\, |Tr_{t,i}|\, P_{t+1,i}. \tag{25}$$

Most of the times, for technical purposes, with little loss of accuracy, as can be seen in [29], instead of working with the simple arithmetic returns $\mathcal{R}_i(t_1,t_2)$, we will work with logarithmic returns, defined as

$$R_i(t_1,t_2) = \ln \frac{P_{t_2,i}}{P_{t_1,i}}. \tag{26}$$

Consider an investment universe with N assets observed over T rebalancing dates. Let $t = 0,\dots,T-1$ index time and $i = 1,\dots,N$ index assets. For each period t, let $R_{t,i}$ denote the expected or realized log-return input associated with asset i (if $i = port$ we are talking about the whole portfolio) from time t to time $t+1$, and let $\Sigma_t \in \mathbb{R}^{N\times N}$denote the covariance matrix used for risk penalization. For convenience, from now on, we will be working with log returns along all the different implementations.

Assets are partitioned into a finite set of categories or asset classes. Let $\mathcal{A}_k \subseteq \{1,\dots,N\}$ denote the set of assets belonging to class k, for $k = 1,\dots,K_c$. This structure is introduced to capture strategic allocation rules at the class level, such as minimum and maximum exposure [30], [31]. The investment universe comprises the following asset classes: **equities**, represented by MSCI indices for the United States, Europe, Japan, and Emerging Markets; **government bonds**, including short-, medium-, and long-term sovereign bonds from the United States and Europe, as well as hard-currency Emerging Markets sovereign bonds; **credit**, including U.S., European, and pan-European corporate bonds; **foreign exchange (FX)**, consisting of EUR, JPY, CAD, AUD, and MXN, all quoted against the USD; and **cash**, represented by the euro (EUR). More details about the investment universe in Appendix C.

In this problem, we analyze 21 assets grouped into $K_c = 5$ asset classes, using data from 418 biweekly periods spanning from January 6^{th}, 2003 to December 31^{st}, 2018. At each date t, the decision variable to be optimized is the portfolio weight vector

$$w_t = (w_{t,1},\dots,w_{t,N})^\top \in \mathbb{R}^N, \tag{27}$$

where $w_{t,i}$ corresponds to the portfolio weight of asset i. Moreover, we consider an average of transaction fees of $f = 0.07\% = 0.0007$. **The objective is to maximize the return of our portfolio, taking into account transaction cost, and minimize the risk of the return (its variance). That is,**

$$\max_{w} \quad \mathbb{E}[R_{port}] - \mathbf{Vol}^2(R_{port}) - TC(w) \;\text{ s.t } \mathcal{W} \text{ are satisfied.} \tag{28}$$

Being $\mathbf{Vol}^2(R_{port})$ the variance of the portfolio returns.

B. Constraints

The optimization problem is subject to a set of feasibility constraints $\mathcal{W}$. These constraints fall in three categories: asset-level constraints, which impose lower and upper bounds on individual asset weights; asset-class constraints, which define minimum and maximum exposures for each asset class; and trading constraints, which limit the quantity that can be bought or sold between rebalancing dates [30], [31].

First, each asset weight is bounded:

$$l_i \leq w_{t,i} \leq u_i, \quad i = 1, \ldots, N, \quad t = 0, \ldots, T-1. \quad (29)$$

Second, the portfolio must satisfy the budget constraint. Since FX assets can have both long (buying) and short (selling) positions, this category of assets are excluded of this constraint.

$$\sum_{i \notin FX} w_{t,i} = 1, \qquad t = 0, \ldots, T-1. \quad (30)$$

Third, class-level exposures must remain within admissible bounds

$$L_k \leq \sum_{i \in \mathcal{A}_k} w_{t,i} \leq U_k, \;\; k = 1, \ldots, K_c, \; t = 0, \ldots, T-1. \quad (31)$$

Fourth, the re-balancing process is constrained through turnover limits. The monetary turnover of asset i form t to $t+1$ is exactly $Tr_{t,i} * P_{t+1,i}$. Considered as a percentage of the final portfolio value, the percentage turnover will be $w_{t+1,i} - w_{t,i} \frac{V_t}{V_{t+1}} \frac{P_{t+1,i}}{P_{t,i}}$. Let τ_i denote the admissible percentage turnover bound for asset i. Then the weight assigned to asset i at time t must remain sufficiently close to the drifted previous allocation:

$$-\tau_i \leq w_{t,i} - w_{t,i}^{\text{drift}} \leq \tau_i, \quad (32)$$

where the drift weight $w_{t,i}^{\text{drift}}$ is the resulting weight after price evolution from $t-1$ to t and before rebalancing, given by:

$$w_{t,i}^{\text{drift}} = w_{t-1,i} e^{R_{t,i} - R_{t,port}} \quad (33)$$

as seen above.

Similarly, class-level turnover may be restricted through bounds Θ_k:

$$-\Theta_k \leq \sum_{i \in \mathcal{A}_k} w_{t+1,i} - \sum_{i \in \mathcal{A}_k} w_{t,i}^{\text{drift}} \leq \Theta_k. \quad (34)$$

These constraints were advised by business experts and define a practitioners' way of portfolio construction in which the feasible set is shaped not only by static exposure limits, but also by the path-dependent mechanics of portfolio rebalancing. For additional details on the investment universe and the specific constraint set used in this case study, see Appendix C.

IV. Methodology

This section describes the benchmark strategies used in the empirical comparison and the quantum formulation adopted for dynamic portfolio optimization. The classical side includes a buy-and-hold portfolio and a downside-risk benchmark based on Conditional Value-at-Risk (CVaR). The quantum side follows the binary encoding and quadratic optimization framework developed for multi-period portfolio rebalancing in [1], [2].

A. Classical Benchmarks

The purpose of the classical benchmarks is to provide economically interpretable reference points against which the quantum strategy can be evaluated. The first benchmark isolates the value of maintaining a fixed strategic allocation without dynamic optimization, whereas the second introduces an optimization-based classical allocation rule focused on downside-risk control.

1) Buy-and-Hold Strategy: The buy-and-hold benchmark represents the simplest admissible reference policy in the multi-period setting. In this work, buy-and-hold is implemented as a fixed strategic allocation benchmark with periodic rebalancing. That is, the initial allocation $w_0^{BH} \in W$ defines the target portfolio weights, and at each rebalancing date the portfolio is brought back to these target weights. This convention makes the benchmark comparable with the active CVaR and quantum strategies, since all three strategies are evaluated under the same rebalancing frequency, transaction-cost framework, and business constraints.

Despite its simplicity, this fixed-allocation benchmark constitutes a widely adopted reference strategy in long-term asset allocation and passive portfolio management, where preserving a stable strategic allocation and controlling turnover are primary objectives [32], [33].

Between two consecutive rebalancing dates, portfolio weights evolve endogenously due to market returns. Let $R_{t,i}$ denote the realized log return of asset i between dates $t-1$ and t, and let $R_{t,port}^{BH}$ denote the corresponding log return of the buy-and-hold portfolio over the same period. Before rebalancing, remind that the drifted weight of asset i is given by Eq. (33).

The portfolio return $r_{t,p}^{BH}$ acts as the normalization term ensuring that the drifted weights sum to one. At each rebalancing date, the portfolio is reset to the target allocation,

$$w_t^{BH} = w_0^{BH}. \quad (35)$$

Therefore, $w_{t,i}^{drift}$ denotes the weight of asset i before rebalancing, whereas $w_{t,i}^{BH}$ denotes the implemented benchmark weight after rebalancing.

This benchmark should therefore be interpreted as a rebalanced buy-and-hold strategy, or equivalently as a fixed-allocation rebalanced benchmark, rather than as a pure buy-and-hold policy. It does not solve a dynamic optimization problem and does not update the target allocation using new expected-return or risk estimates. Its only active component is the mechanical rebalancing required to restore the initial strategic allocation and to keep the portfolio within the same business constraints imposed on the CVaR and quantum strategies.

As such, it provides a realistic and comparable baseline for assessing the value added by dynamic optimization. The comparison with the CVaR and quantum strategies therefore isolates the incremental effect of optimizing the allocation over time, rather than the effect of merely enforcing the common rebalancing schedule and operational constraints.

2) CVaR-Based Classical Optimization: The second classical baseline is an active strategy that, at every rebalancing

date, reallocates capital by solving a Conditional Value-at-Risk (CVaR) optimization problem in the Rockafellar–Uryasev linear formulation [5]. Following standard practice in the risk-management literature, the risk aversion of the strategy is not encoded as a convex combination of mean and CVaR in the objective; instead, the CVaR is minimized subject to a parametric lower bound on the portfolio average return, $R_{\min}$.

a) Optimization problem: Let $\mathbf{R} \in \mathbb{R}^{S\times N}$ be a matrix of S return scenarios over the N assets and let w_{t-1} denote the portfolio held at the previous rebalancing date. At date t we solve

$$\begin{aligned} \min_{\mathbf{w},\alpha,u} \ \alpha + \frac{1}{S(1-p)}\sum_{s=1}^{S}\eta_s, \\ \text{s.t.} \quad \eta_s \geq 0, \quad s=1,\dots,S, \\ \eta_s \ \geq \ -w^T R_s - \alpha, \\ \frac{1}{S}\sum_{s=1}^{S} w^T R_s \ \geq \ R_{\min}, \\ w \in \mathcal{W}, \end{aligned} \tag{36}$$

where $p \in [0,1)$ is the CVaR confidence level (set to $p = 0.95$ in all experiments), α is the auxiliary variable that, at the optimum, coincides with the Value-at-Risk at level p, and η_s are the standard slack variables that linearize the tail expectation. The set $\mathcal{W}$ collects all linear constraints defining the admissible portfolios—budget, box, and group constraints on individual assets and asset classes. Because problem (36) is formulated as a linear program, it can be solved to optimality using a deterministic Linear Programming (LP) solver.

In this exercise, we obtain the portfolio w_t^{CVaR} by minimizing the CVaR with no minimum-return constraint ($R_{\min} = -\infty$). The optimization problem in Eq. (36) is performed sequentially over time in a rolling-horizon fashion, that is, the period under consideration shifts forward. At each rebalancing date t, the portfolio is updated by solving the CVaR optimization problem using the previously implemented allocation after evolution w_{t-1} as the initial state.

This induces a path dependency through turnover constraints, effectively resulting in a rollout policy where decisions are conditionally optimal given past allocations but not globally optimal over the full horizon.

b) Scenario generation: The optimization problem in Eq. (36) is solved using scenarios generated by a classical Parametric Gaussian Monte Carlo model, calibrated over a backward estimation window of ($L_{\text{window}} = 35$) weeks ending at each rebalancing date. Over this window, the mean vector $\hat{\mu}_t$ and covariance matrix $\hat{\Sigma}_t$ of the log-returns are estimated on the backward window L_{window}, and S scenarios are drawn from $\mathcal{N}(\hat{\mu}_t, \hat{\Sigma}_t)$. This generator provides a smooth, fully parametric description of the joint distribution at the cost of imposing Gaussian tails.

c) Annualization convention: Throughout the benchmark, expected returns are annualized by linear scaling with $D = 252$ trading days per year, while the CVaR is annualized by combining a linear projection of the mean with a square-root-of-time scaling of the tail term, namely

$$\mathrm{CVaR}^{\mathrm{ann}} \ = \ \sqrt{D}\,\mathrm{CVaR}_d \ - \ (D-\sqrt{D})\,\bar{r}_d, \tag{37}$$

with $\bar{r}_d$ and CVaR_d the daily mean return and daily CVaR of the portfolio, respectively. This annualization is an approximation that assumes i.i.d. Gaussian distribution and combines linear scaling of the mean and square-root-of-time scaling of the tail component, as no closed-form scaling rule exists for CVaR [34]. More details in Annex D.

The Sharpe ratio is annualized by multiplying the daily volatility by $\sqrt{D}$, and a zero risk-free rate is assumed in all reported figures.

B. Quantum Approach

The quantum methodology is based on the dynamic portfolio optimization framework proposed in [1], where the multi-period rebalancing problem is reformulated as a quadratic unconstrained binary optimization (QUBO) model compatible with quantum annealing (5). In practice, however, the presence of constraints requires embedding the formulation into a constrained quadratic model (CQM).

1) Mathematical Formalism: The key step is to replace the continuous portfolio weights $w_{t,i}$ by a finite binary encoding. For each asset i, time index t, and bit position q, binary variables

$$x_{t,i,q} \in \{0,1\}, \qquad q = 0,\dots,N_{\mathrm{enc}}-1, \tag{38}$$

are introduced. Defining

$$M = 2^{N_{\mathrm{enc}}} - 1, \qquad \Delta_i = u_i - l_i, \tag{39}$$

the corresponding portfolio weight is decoded as

$$w_{t,i}(x) = l_i + \frac{\Delta_i}{M}\sum_{q=0}^{N_{\mathrm{enc}}-1} 2^q x_{t,i,q}, \tag{40}$$

where $w_{t,i}$ is defined as the portfolio weight of asset i, following Eq. (27).

This formulation respects the bounds $l_i \leq w_{t,i} \leq u_i$ introduced previously and, thus, preserves asset-level feasibility by construction.

Following [1], [2], we defined the dynamic portfolio optimization function as a combination of a normalized return term $\mathbf{R}^{\mathrm{QUBO}}$, a normalized variance penalty $\mathbf{Vol}^{\mathrm{QUBO}}$, a transaction-cost term $\mathbf{C}^{\mathrm{QUBO}}$, and a budget regularizer $\mathbf{B}^{\mathrm{QUBO}}$, defined in the following.

Let the boundary returns $R^{\max}_{\text{port}}(t)$ and $R^{\min}_{\text{port}}(t)$, and the maximum portfolio volatility $\mathbf{Vol}^{\max}_{\text{port}}$, be defined as:

$$\begin{aligned} R^{\max}_{\text{port}}(t) &= \max_{w_{t,i}\in\mathcal{W}} \sum_{i=1}^{N} w_{t,i} R_i(t,t+1), \\ R^{\min}_{\text{port}}(t) &= \min_{w_{t,i}\in\mathcal{W}} \sum_{i=1}^{N} w_{t,i} R_i(t,t+1), \quad \text{and} \\ \mathbf{Vol}^{\max}_{\text{port}} &= \max_{t,w_{t,i}\in\mathcal{W}} \mathbf{Vol}^{\max}_{t,\text{port}} \\ &= \max_{t,w_{t,i}\in\mathcal{W}} \sqrt{\sum_{i=1}^{N}\sum_{j=1}^{N} w_{t,i}\,\hat{\Sigma}_{ij}(t)\,w_{t,j}}. \end{aligned}$$

Then, the normalized return and variance penalty terms are given by:

$$\mathbf{R}_t^{\text{QUBO}} = \frac{\sum_{i=1}^{N} w_{t,i} R_i(t,t+1) - R_{port}^{\min}(t)}{\xi_t} \tag{41}$$

$$\mathbf{Vol}_t^{\text{QUBO}} = \frac{\sqrt{\sum_{i=1}^{N}\sum_{j=1}^{N} w_{t,i}\,\hat{\Sigma}_{ij}(t)\,w_{t,j}}}{\mathbf{Vol}_{port}^{\max}} \tag{42}$$

where $\xi_t = R_{port}^{\max}(t) - R_{port}^{\min}(t)$ denotes the range of portfolio returns at times t.

The transaction component deserves special attention because it is the only term that couples adjacent rebalancing dates. In the underlying financial problem, transaction costs are naturally proportional to turnover and therefore scale as $f_i|\Delta w_{t,i}|$, where

$$\Delta w_{t,i} = w_{t,i} - w_{t,i}^{\text{drift}}. \tag{43}$$

However, a QUBO model only admits quadratic terms in the binary variables. For that reason, the absolute transaction-cost contribution is approximated by a quadratic approximation over the feasible turnover interval. Using the turnover-bound notation introduced in the constraints section, τ_i denotes the maximum admissible weight change for asset i. The implementation then uses the effective coefficient

$$\tilde{f}_{t,i} = \frac{2^{1/3} f_i}{\tau_i \xi_{t+1}}, \tag{44}$$

so that the normalized transaction term becomes

$$\mathbf{C}_t^{\text{QUBO}} = \sum_{i=1}^{N} \tilde{f}_{t,i}\Big(w_{t+1,i} - w_{t,i}^{\text{drift}}\Big)^2. \tag{45}$$

Note that the quadratic transaction term used in the quantum formulation approximates the linear transaction costs used in the classical benchmark, which may introduce slight differences in effective penalization.

The factor $\frac{2^{1/3}}{\tau_i}$ comes from the calibration of the quadratic surrogate to the absolute-value function over that interval. More precisely, if one approximates $|x|$ on $[0,\tau_i]$ by a function of the form ax^2, the choice

$$a = \frac{2^{1/3}}{\tau_i}\ , \tag{46}$$

minimizes the integrated absolute approximation error.

$$\epsilon = \int_{-\tau_i}^{\tau_i} ||x| - ax^2| \tag{47}$$

This adjustment improves the fidelity of the quadratic transaction term relative to the original proportional trading-cost model and prevents the turnover penalty from becoming artificially weak when the admissible changes are small.

Lastly, budget's soft constraint could be expressed as

$$\mathbf{B}_t^{\text{QUBO}} = \left(\sum_{i=1}^{N} w_{t,i} - 1\right)^2\ . \tag{48}$$

Given this structure, the objective can be written as

$$f_{\text{QUBO}}(w) = \sum_{t=0}^{T-1}\Big[-\mathbf{R}_t^{\text{QUBO}} + \gamma\mathbf{Vol}_t^{\text{QUBO}} + \rho\mathbf{B}_t^{\text{QUBO}}\Big] + \lambda\sum_{t=0}^{T-2}\mathbf{C}_t^{\text{QUBO}}\ . \tag{49}$$

The quantities ξ_t, and $V_{port}^{\max}$ are normalization constants used to keep the return and risk contributions comparable. The parameter γ controls risk aversion, λ controls the strength of transaction-cost penalization, and ρ penalizes deviations from the budget condition. In the paper, the normalization is applied in order to make the different components of the objective play on equal footing and to avoid one term dominating the others purely because of scale.

After substituting the binary expansion of each weight into the objective and converting all linear terms to quadratic terms by exploiting the property of binary variables $x_i^2 = x_i$, it is possible to show that the multi-period problem takes the standard QUBO form

$$f_{\text{QUBO}}(x) = x^\top Q x + b, \tag{50}$$

where Q is the matrix of binary couplings and b is a constant offset. The entries of Q are assembled from the return contribution, the variance contribution, the budget penalty, and the expanded transaction-cost contribution.

In practice, the optimization problem considered in this work also contains feasibility requirements, introduced in Section III-B, that are more naturally enforced as explicit constraints than as soft penalties alone. For that reason, the QUBO objective is embedded into a constrained quadratic model (CQM), where asset-level turnover limits, exposure bounds, and class-level restrictions are imposed directly. This model is then solved using D-Wave's LeapHybridCQMSampler, corresponding to the hybrid quantum-classical solver for constrained quadratic models available.

2) Implementation: To make the full backtesting horizon compatible with the solver capacity, the multi-period optimization is not encoded as a single problem over all 418 rebalancing dates. Instead, the horizon is decomposed into yearly blocks, each containing 26 biweekly rebalancing dates. Within each block, the complete sequence of portfolio weights is optimized jointly, so the solver captures the intertemporal effect of transaction costs and turnover constraints over that year. Once a yearly block is solved, its final portfolio allocation is used as the initial condition for the next block, ensuring continuity across years and preserving the path-dependent rebalancing constraints. Thus, the approach combines global optimization within each yearly window with a sequential rollout across the full historical period.

Given the hardware characteristics of the LeapHybridCQMSampler and the given scenario to optimize, ~ 20 years, we defined the following workflow in order to solve the problem:

1) Define rebalancing frequency, in this case, every two weeks.
2) Encode the problem for a whole year, 26 rebalancing dates.

3) Solve the problem and save the resulting optimal weights.
4) Repeat this process for the next year using previous period optimal weights as starting weights in order to satisfy transactions limit constraints between periods.
5) Do this iteratively for all years.

With this workflow we can efficiently solve arbitrary large time windows without reaching hardware limitations.

The methodology described in this section provides two main advantages relative to the classical benchmarks introduced previously:

- The resulting portfolio weights constitute a globally optimized sequence $w_t \in \mathbb{R}^N$ over the rebalancing dates t included in the optimization, in our case one whole year. In contrast, classical approaches typically compute locally optimal allocations at each rebalancing step and connect them through transaction-cost restrictions.
- Classical methods such as CVaR optimization generally require a distributional assumption on asset returns, which in our setting is taken to be Gaussian, in order to derive the optimal portfolio weights. The proposed approach does not impose any return-distribution assumption, and therefore avoids this source of approximation.

For additional details, see Table V.

V. Results

This section presents the comparison of the proposed quantum-classical optimization and the two classical benchmarks, that is, a rebalanced buy-and-hold allocation and a CVaR-minimizing strategy.

The analysis is divided into two complementary perspectives. First, we analyze the computational characteristics of the hybrid quantum-classical optimization workflow. Second, we evaluate the economic quality of the resulting portfolios through return, volatility, and Sharpe ratio under different transaction-cost regimes.

Furthermore, hardware-level results and business-oriented metrics are reported separately to distinguish economic performance from computational performance. This separation is important because strong financial performance does not necessarily imply computational advantage, and computational advantage does not automatically translate into superior investment performance.

A. Hardware Performance

In this part, we discuss the computational aspects of the proposed framework. These include the size of the binary formulation (i.e., the number of binary variables required by the QUBO model), the role of quadratic couplings, the handling of constraints through the CQM layer, and the operational characteristics of the hybrid quantum-classical solver.

The yearly decomposition strategy described above was adopted to remain within the practical capacity limits of the CQM and hybrid solver summarized in Table V. This decomposition is required because the complete formulation exceeds the number of binary variables supported within a single CQM instance.

Since the main contribution of this work lies in the practical financial applicability of the proposed optimization framework, the hardware-related analysis focuses on operationally relevant quantities such as total wall-clock time, which directly reflects the effective time-to-solution in realistic portfolio-management scenarios.

1) Execution time: Specifically, the reported wall-clock time corresponds to the total time required by each method to complete the full portfolio optimization over the entire backtesting horizon (2003-01-06 to 2018-12-31), comprising 418 biweekly rebalancing dates.

a) Classical setup: At each of the 418 rebalancing dates, the linear program in Eq. (36) is solved using $S = 150{,}000$ Monte Carlo scenarios drawn from the parametric Gaussian generator described in Section IV. The cumulative wall-clock time for the full rolling-horizon backtest amounts to 4 hours and 15 minutes. The CVaR optimization was executed on a MacBook Pro running macOS Tahoe Version 26.4.1, equipped with an Apple M2 chip (8-core CPU) and 16 GB of unified memory.

b) Quantum setup: The optimization problem was solved using D-Wave's LeapHybridCQMSampler (version 1.13), a cloud-based hybrid quantum-classical solver designed for constrained quadratic models (CQMs) [35]. The solver integrates classical optimization techniques with quantum annealing resources provided by D-Wave's Advantage quantum processing unit (QPU), which comprises more than 5,000 qubits arranged in a Pegasus topology with 15-way qubit connectivity [36]. The hybrid workflow decomposes the problem into subproblems that are distributed between classical hardware and the QPU. As described in Section IV, the full 16-year backtest is decomposed into yearly batches of 26 rebalancing dates each in order to remain within the solver capacity limits summarized in Table V. The cumulative wall-clock time for solving the entire optimization period, including network latency and QPU access, amounts to 35 minutes.

Table I presents the computational environment specifications and the corresponding timing results obtained for both optimization approaches. The hybrid quantum-classical workflow achieves a wall-clock reduction factor of approximately $7.3\times$ relative to the classical CVaR implementation considered in this study. This comparison should be interpreted with nuance: the two methods differ not only in hardware but also in algorithmic structure. The CVaR approach solves 418 independent linear programs sequentially, each involving $150{,}000$ scenarios and the full set of portfolio constraints, whereas the quantum approach solves the problem for 26 rebalancing periods simultaneously with a single CQM instance per yearly batch, jointly optimizing multiple rebalancing periods within each yearly batch.

Moreover, the hybrid solver offloads computation to a remote cloud infrastructure with dedicated high-performance classical resources in addition to the QPU, whereas the classical benchmark runs entirely on a consumer-grade laptop. Consequently, the observed timing difference reflects differences in problem formulation, hardware architecture, and execution model, rather than a pure quantum-versus-classical comparison. A rigorous claim of quantum computational ad-

vantage would require benchmarking against equivalent discrete optimization methods on comparable hardware, which remains outside the scope of this work. Nevertheless, the result demonstrates that the hybrid quantum-classical approach is practically viable for real-world portfolio optimization at this problem scale, delivering competitive solutions in substantially less wall-clock time.

TABLE I
HARDWARE AND EXECUTION TIME COMPARISON

	Classical (CVaR)	**Quantum (CQM)**
Processor	Apple M2 (8-core)	D-Wave Advantage
Resources	16 GB unified	>5000 qubits
OS / Platform	macOS Tahoe 26.4.1	D-Wave Leap (cloud)
Solver	CVXPy	LeapHybridCQMSampler v1.13
Optimization calls	418 (sequential LP)	16 (yearly CQM batches)
Total wall-clock time	4 h 15 min	35 min
Speedup factor	—	$\approx 7.3\times$

B. Economic Performance

This section presents the empirical evaluation of the proposed framework. We evaluate the economic quality of the resulting portfolios through *return*, *volatility*, and *Sharpe ratio* under different transaction-cost regimes. In this way, the study distinguishes not only between absolute return and risk levels but the balance between them.

From a business perspective, the key question is whether the additional flexibility of the quantum formulation translates into improved risk-adjusted performance when transaction costs are incorporated. Conversely, classical baselines provide a benchmark for evaluating whether the additional modeling complexity of the quantum formulation leads to economically meaningful improvements.

The historical rolling-window backtest over the 2003–2018 period indicates that classical and quantum methods achieve relatively similar performance under low transaction-cost regimes (see Figure 1). Under low-cost regimes—where fees average approximately 7 basis points— CVaR achieves a higher Sharpe ratio than the quantum-derived portfolio (2.09 for CVaR versus 1.96). As transaction costs rise, however, the performance gap narrows progressively until both methods reach approximate parity. Beyond a break-even point of roughly 35 basis points, the quantum-derived portfolios begin to yield a more competitive risk-adjusted performance (Sharpe ratio of 1.70 versus 1.68 for CVaR at 35 basis points, and 1.38 versus 1.16 for CVaR at 70 basis points). This suggests that the global multi-period optimization of the quantum strategy becomes competitive with CVaR as transaction costs increase.

Notably, this threshold is well within realistic market conditions: for reference, the stamp duty levied on equity purchases in the United Kingdom amounts to 50 basis points. Consequently, the 35 basis-point level at which quantum-derived portfolios begin to outperform represents a realistic and economically relevant scenario rather than an extreme edge case. More specifically, we can see:

- In Table II, that the quantum-derived portfolio achieves the highest annualized return. Focusing on the break-even case of 35 basis points, 10.57% vs 8.06% for the best classical approach (CVaR).
- In Table III, that the quantum-derived portfolio ranks second in terms of risk, with an annual volatility of 5.58% vs 4.18% of CVaR strategy (the best for risk-averse investors) or 8.14% of buy-and-hold strategy (the riskiest).
- and finally, in Table IV and in Figure 1, that the quantum-derived portfolio achieves a higher Sharpe ratio than all classical methods when transaction costs are above 35 basis points.

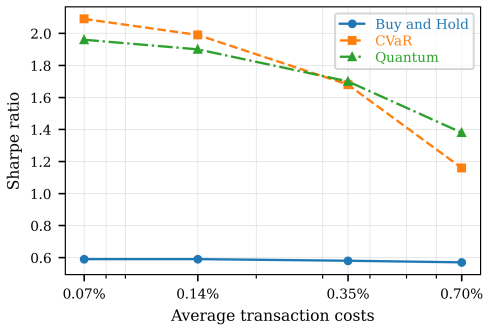


Fig. 1. Average Sharpe ratio across different transaction-cost levels. Quantum-derived portfolios become competitive with CVaR above 35 *bp*.

VI. CONCLUSION

This paper presents an empirical comparison between a quantum strategy for dynamic portfolio optimization and two classical benchmarks, a rebalanced buy-and-hold allocation and a CVaR-minimizing strategy, under realistic trading constraints. The study considers a diversified investment universe of 21 global assets spanning equities, government bonds, credit, foreign exchange, and cash, subject to asset-level, class-level, and turnover constraints over 418 biweekly rebalancing periods (2003–2018).

The quantum formulation encodes portfolio weights into binary variables and constructs a constrained quadratic model (CQM) solved by D-Wave's hybrid quantum-classical sampler. In contrast to the classical CVaR benchmark, which optimizes sequentially at each rebalancing date under a Gaussian return assumption while using previous portfolio weights as the initial condition, the quantum approach jointly optimizes the full trajectory over yearly windows without imposing a parametric distributional model on asset returns.

The key result is that the quantum-derived strategy improves monotonically as transaction costs increase, eventually overtaking the CVaR benchmark. Under low-transaction-costs regimes (average fees of 7 basis points), CVaR achieves a higher Sharpe ratio; however, as costs rise, the performance gap gradually narrows until both strategies converge near 35 basis points. Beyond this threshold, the quantum-derived portfolio achieves a higher Sharpe ratio than the CVaR benchmark.

Importantly, this crossover occurs well within realistic market conditions —for reference, UK stamp duty alone amounts to 50 basis points on equity purchases— confirming the practical relevance of the observed advantage.

Additionally, the quantum strategy achieves the highest annualized return across all fee scenarios, while maintaining intermediate volatility. This pattern further suggests that the sequential nature of the CVaR optimization becomes progressively less effective at managing cumulative rebalancing frictions than the multi-period trajectory optimization enabled by the quantum formulation.

Several limitations must be acknowledged. First, the QUBO formulation approximates the linear transaction-cost function using a quadratic surrogate, introducing a modeling error relative to the true transaction costs. Second, the problem was decomposed into yearly subproblems to ensure computational feasibility on current quantum hardware, yielding a tractable, albeit non-exact, approximation to the full multi-year optimization problem.

Despite these caveats, the results show that quantum-derived dynamic portfolio optimization becomes increasingly competitive as realistic transaction costs are incorporated, with the break-even occurring at transaction-cost levels commonly observed in institutional practice. These findings suggest that quantum methods may be particularly well suited to constrained, multi-period portfolio optimization problems where rebalancing costs play a significant role.

VII. Future Work

Several research extensions arise naturally from this work. First, the sensitivity of the quantum solution to the discretization depth, penalty calibration, and normalization scheme should be studied systematically. Second, future work may compare the hybrid constrained quadratic model approach with alternative classical discrete optimization techniques, such as mixed-integer quadratic programming or metaheuristic search methods, under identical problem instances and conditions. Third, the portfolio design can be extended to encompass richer risk measures, alternative transaction-cost models, and scenario-based uncertainty representations. Finally, a more exhaustive computational study is essential before any claim regarding potential quantum advantage in portfolio optimization can be responsibly formulated.

Additionally, while the empirical results suggest that the quantum-derived optimal portfolios exhibit favorable risk-adjusted performance, the current reliance on offline historical data limits their direct use in real-time industrial settings.

To bridge this gap, a promising direction involves utilizing the quantum-optimized allocations as training targets for a supervised learning surrogate model. Once trained on relevant macro and microeconomic market variables, such a model could generate real-time allocation recommendations at minimal operational latency, while potentially uncovering interpretable market regimes that inform adaptive investment strategies. This line of research is currently under active development and constitutes the immediate continuation of the present work.

Acknowledgment

The authors thank BBVA Corporate and Investment Banking and Asset Management for helpful discussions and technical support.

This work was developed within the framework of the Q-MIND consortium, co-funded by the European Union through the European Regional Development Fund (ERDF), under the Programa de la Comunidad de Madrid for the 2021–2027 period promoted by the Consejería de Educación, Ciencia e Universidades.

References

[1] S. Mugel, C. Kuchkovsky, E. Sánchez, S. Fernández-Lorenzo, J. Luis-Hita, E. Lizaso, and R. Orús, "Dynamic portfolio optimization with real datasets using quantum processors and quantum-inspired tensor networks," *Phys. Rev. Research*, vol. 4, p. 013006, Jan 2022. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevResearch.4.013006

[2] G. Rosenberg, P. Haghnegahdar, P. Goddard, P. Carr, K. Wu, and M. L. de Prado, "Solving the optimal trading trajectory problem using a quantum annealer," *IEEE J. Sel. Top. Sig. Proc.*, vol. 10, no. 6, pp. 1053–1060, 2016.

[3] I. De León *et al.*, "Variational Quantum Eigensolver for Real-World Finance: Scalable Solutions for Dynamic Portfolio Optimization Problems," 12 2025.

[4] Á. Nodar *et al.*, "Scaling the Variational Quantum Eigensolver for Dynamic Portfolio Optimization," 12 2024.

[5] R. T. Rockafellar and S. Uryasev, "Optimization of conditional value-at risk," *Journal of Risk*, vol. 3, pp. 21–41, 2000. [Online]. Available: https://api.semanticscholar.org/CorpusID:854622

[6] R. Yalovetzky, P. Minssen, D. Herman, and M. Pistoia, "Solving linear systems on quantum hardware with hybrid hhl++," *Scientific Reports*, vol. 14, no. 1, p. 20610, 2024. [Online]. Available: https://doi.org/10.1038/s41598-024-69077-0

[7] A. Acharya *et al.*, "Decomposition pipeline for large-scale portfolio optimization with applications to near-term quantum computing," *Phys. Rev. Res.*, vol. 7, no. 2, p. 023142, 2025.

[8] G. Agliardi, D. Alevras, V. Kumar, R. L. Nardo, G. Compostella, S. Kumar, M. Proissl, and B. Mehta, "Portfolio Construction using a Sampling-Based Variational Quantum Scheme," in *2025 IEEE International Conference on Quantum Artificial Intelligence*, 8 2025.

[9] L. Leclerc *et al.*, "Financial risk management on a neutral atom quantum processor," *Phys. Rev. Res.*, vol. 5, no. 4, p. 043117, 2023.

[10] A. M. Dalzell, B. D. Clader, G. Salton, M. Berta, C. Y.-Y. Lin, D. A. Bader, N. Stamatopoulos, M. J. A. Schuetz, F. G. S. L. Brandão, H. G. Katzgraber, and W. J. Zeng, "End-to-end resource analysis for quantum interior-point methods and portfolio optimization," *PRX Quantum*, vol. 4, p. 040325, Nov 2023. [Online]. Available: https://link.aps.org/doi/10.1103/PRXQuantum.4.040325

[11] H. Markowitz, "Portfolio selection," *The Journal of Finance*, vol. 7, pp. 77–91, Mar 1952. [Online]. Available: https://doi.org/10.1111/j.1540-6261.1952.tb01525.x

[12] W. F. Sharpe, "The sharpe ratio," *The Journal of Portfolio Management*, vol. 21, pp. 49–58, Fall 1994. [Online]. Available: https://doi.org/10.3905/jpm.1994.409501

[13] H. M. Markowitz, *Portfolio Selection: Efficient Diversification of Investments*. Yale University Press, 1959. [Online]. Available: http://www.jstor.org/stable/j.ctt1bh4c8h

[14] R. C. Merton, "An analytic derivation of the efficient portfolio frontier," *Journal of Financial and Quantitative Analysis*, vol. 7, pp. 1851–1872, Sep 1972. [Online]. Available: https://doi.org/10.2307/2329621

[15] P. Jorion, *Value at Risk*, 1st ed. New York: The McGraw-Hill Companies, 1997.

[16] P. Artzner, F. Delbaen, J.-M. Eber, and D. Heath, "Coherent measures of risk," *Mathematical Finance*, vol. 9, pp. 203–228, Jul 1999. [Online]. Available: https://doi.org/10.1111/1467-9965.00068

[17] R. T. Rockafellar and S. Uryasev, "Optimization of conditional value-at-risk," *Journal of Risk*, vol. 2, pp. 21–41, 2000. [Online]. Available: https://doi.org/10.21314/JOR.2000.038

[18] J. Lintner, "The valuation of risk assets and the selection of risky investments in stock portfolios and capital budgets: A reply," *The Review of Economics and Statistics*, vol. 51, no. 2, pp. 222–224, 1969. [Online]. Available: http://www.jstor.org/stable/1926735

[19] H. Levy and H. M. Markowitz, "Approximating expected utility by a function of mean and variance," *American Economic Review*, vol. 69, pp. 308–317, Jun 1979. [Online]. Available: https://www.jstor.org/stable/1807366

[20] R. Cont, "Empirical properties of asset returns: Stylized facts and statistical issues," *Quantitative Finance*, vol. 1, pp. 223–236, 2001. [Online]. Available: https://doi.org/10.1080/713665670
[21] R. O. Michaud, "The markowitz optimization enigma: Is 'optimized' optimal?" *Financial Analysts Journal*, vol. 45, pp. 31–42, 1989. [Online]. Available: https://doi.org/10.2469/faj.v45.n1.31
[22] A. Montanaro, "Quantum algorithms: An overview," *npj Quantum Information*, vol. 2, p. 15023, 2016. [Online]. Available: https://doi.org/10.1038/npjqi.2015.23
[23] H. Munoz-Bauza and D. Lidar, "Scaling advantage in approximate optimization with quantum annealing," *Phys. Rev. Lett.*, vol. 134, p. 160601, Apr 2025. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevLett.134.160601
[24] D. Venturelli and A. Kondratyev, "Reverse quantum annealing approach to portfolio optimization problems," *Quantum Machine Intelligence*, vol. 1, pp. 17–30, 2019. [Online]. Available: https://doi.org/10.1007/s42484-019-00001-w
[25] J. Preskill, "Quantum computing in the nisq era and beyond," *Quantum*, vol. 2, p. 79, Aug 2018. [Online]. Available: https://doi.org/10.22331/q-2018-08-06-79
[26] D. Herman, C. Googin, X. Liu, Y. Sun, A. Galda, I. Safro, M. Pistoia, and Y. Alexeev, "Quantum computing for finance," *Nature Reviews Physics*, vol. 5, pp. 450–465, 2023. [Online]. Available: https://doi.org/10.1038/s42254-023-00603-1
[27] C. S. Ang, "Portfolio returns," in *Analyzing Financial Data and Implementing Financial Models Using R*. Cham: Springer, 2021, pp. 75–102.
[28] M. S. Lobo, M. Fazel, and S. Boyd, "Portfolio optimization with linear and fixed transaction costs," *Annals of Operations Research*, vol. 152, no. 1, pp. 341–365, 2007.
[29] E. Sánchez Martínez, "BBVA's Quantum Journey," Master Thesis, University of Nebrija, 2022, quantum Computing University Master.
[30] G. Abate, T. Bonafini, and P. Ferrari, "Portfolio constraints: An empirical analysis," *International Journal of Financial Studies*, vol. 10, no. 1, p. 9, 2022.
[31] P. N. Kolm, R. Tütüncü, and F. J. Fabozzi, "60 years of portfolio optimization: Practical challenges and current trends," *European Journal of Operational Research*, vol. 234, no. 2, pp. 356–371, 2014.
[32] B. G. Malkiel, *A Random Walk Down Wall Street*, 1st ed. New York: W. W. Norton & Company, 1973.
[33] J. C. Bogle, *Common Sense on Mutual Funds: New Imperatives for the Intelligent Investor*. New York: John Wiley & Sons, 1999.
[34] J. Daníelsson and J.-P. Zigrand, "On time-scaling of risk and the square-root-of-time rule," *Journal of Banking & Finance*, vol. 30, no. 10, pp. 2701–2713, 2006.
[35] D-Wave Quantum Inc., "D-wave hybrid solver service: An overview," https://www.dwavequantum.com/media/bymjm15f/d-wave_hybrid_solver_service_an_overview.pdf, 2024, technical report.
[36] ——, "Advantage performance update: Overview and key results," https://www.dwavequantum.com/media/34zhplmx/14-1122a-a_advantage_performance_update.pdf, 2024, technical report.

APPENDIX A
RESULTS TABLES

In this appendix we present the key financial performance metrics used to evaluate the optimized portfolios:

- Annualized return
- Annualized volatility
- Annualized Sharpe ratio

These metrics are reported under four transaction-cost regimes. The column "Avg. TC" denotes the portfolio-weighted average two-way transaction cost, expressed as a percentage of portfolio value (e.g., 0.07% corresponds to 7 basis points). Volatility is approximately invariant across cost regimes; transaction costs affect only net returns and, consequently, the Sharpe ratio.

TABLE II
RETURN COMPARISON UNDER DIFFERENT TRANSACTION COST LEVELS

Avg. TC	Buy & Hold	CVaR	Quantum
0.07%	5.86%	9.82%	12.00%
0.14%	5.84%	9.38%	11.64%
0.35%	5.79%	8.06%	10.57%
0.70%	5.69%	5.87%	8.78%

TABLE III
VOLATILITY COMPARISON UNDER DIFFERENT TRANSACTION COST LEVELS

Avg. TC	Buy & Hold	CVaR	Quantum
0.07%	8.14%	4.19%	5.58%
0.14%	8.14%	4.19%	5.58%
0.35%	8.14%	4.18%	5.58%
0.70%	8.14%	4.18%	5.58%

TABLE IV
SHARPE RATIO COMPARISON UNDER DIFFERENT TRANSACTION COST LEVELS

Avg. TC	Buy & Hold	CVaR	Quantum
0.07%	0.59	2.09	1.96
0.14%	0.59	1.99	1.90
0.35%	0.58	1.68	1.70
0.70%	0.57	1.16	1.38

APPENDIX B
HARDWARE TECH SPECS

In this appendix you can find the detailed technical specifications of the Quantum Annealer utilized for the development of this work.

TABLE V
CONSTRAINED HYBRID SOLVER TECHNICAL SPECS.

Parameter	**Value**
Category	Hybrid
Maximum number of biases	2,000,000,000
Maximum number of constraints	100,000
Maximum number of quadratic variables	100,000,000
Maximum number of variables	500,000
Supported problem types	CQM
Version	1.13

APPENDIX C
ASSET UNIVERSE

In this appendix you can find all the details about the assets used in this work.

The column headers are defined as follows: **Min** and **Max** denote the lower and upper bounds on individual asset weights; **Chg.** denotes the maximum absolute change in weight permitted at each biweekly rebalancing date (i.e., $|w_{t,i} - w_{t-1,i}| \leq$ Chg.); f denotes the two-way transaction cost in basis points; and **B&H** denotes the fixed target allocation used in the buy-and-hold benchmark.

All indices are sourced from Bloomberg and represent total-return series denominated in EUR.

TABLE VI
EQUITY ASSET CHARACTERISTICS AND CONSTRAINTS

Asset / ticker	Min	Max	Chg.	f	B&H
US eq. / GDDUUS	5%	45%	8%	3	25%
Europe eq. / GDDLE15	7%	23%	3%	3	15%
Japan eq. / GDDLJN	0%	6%	2%	3	3%
EM eq. / M2EF	2%	12%	2%	3	7%

Notes: Portfolio weights are reported in Min, Max, Chg., and B&H. f denotes two-way transaction costs in basis points.

TABLE VII
GOVERNMENT-BOND ASSET CHARACTERISTICS AND CONSTRAINTS

Asset / ticker	Min	Max	Chg.	f	B&H
US gov. 7–10y / G4O2	0%	10%	3%	4.5	4%
US gov. 5–7y / G3O2	0%	10%	4%	3.75	3%
US gov. 1–3y / G1O2	0%	10%	4%	1.5	3%
Euro gov. 7–10y / EG04	0%	10%	3%	5.25	4%
Euro gov. 5–7y / EG03	0%	10%	4%	4.38	3%
Euro gov. 1–3y / EG01	0%	10%	3%	1.75	3%
EM gov. HC / EMGB	0%	10%	3%	10	5%

Notes: Portfolio weights are reported in Min, Max, Chg., and B&H. f denotes two-way transaction costs in basis points. HC stands for hard currency.

TABLE VIII
CREDIT ASSET CHARACTERISTICS AND CONSTRAINTS

Asset / ticker	Min	Max	Chg.	f	B&H
US credit / C0A0	0%	18%	4%	10	9%
EUR credit / ERL0	0%	18%	4%	10	9%
Pan-Euro HY / HE00	0%	10%	3%	18	3.5%
US HY / H0A0	0%	10%	3%	15	3.5%

Notes: Portfolio weights are reported in Min, Max, Chg., and B&H. f denotes two-way transaction costs in basis points. HY stands for high yield.

TABLE IX
CASH ASSET CHARACTERISTICS AND CONSTRAINTS

Asset / ticker	Min	Max	Chg.	f	B&H
EUR cash o/n / L0EC	0%	60%	11%	0	0%

Notes: Portfolio weights are reported in Min, Max, Chg., and B&H. f denotes two-way transaction costs in basis points.

TABLE X
FX-FORWARD ASSET CHARACTERISTICS AND CONSTRAINTS

Asset / ticker	Min	Max	Chg.	f
EUR/USD 45d / BBLIXEUE	-20%	20%	4%	0
USD/JPY 45d / BBLIXJPE	-10%	10%	2%	0
USD/CAD 45d / BBLIXCAE	-10%	10%	2%	0
USD/AUD 45d / BBLIXAUE	-10%	10%	2%	0
USD/MXN 45d / BBLIXMXE	-10%	10%	2%	0

Notes: Portfolio weights are reported in Min, Max, and Chg. f denotes two-way transaction costs in basis points.

TABLE XI
AGGREGATE LIMITS BY ASSET CLASS

Class	Min	Max	Chg.	B&H
Equity	30%	70%	8%	50%
Gov.	5%	45%	8%	25%
Cred.	5%	45%	8%	25%
Cash	0%	60%	12%	0%

Notes: Asset class weights are reported in Min, Max, and Chg. B&H denotes the asset class weights used in the buy-and-hold portfolio.

APPENDIX D
ANNUALIZATION CONVENTION FOR CVAR

This appendix describes the annualization convention used for CVaR, where we use a Gaussian time-aggregation approximation. Let $\mathrm{CVaR}_{d,\alpha}$ denote the daily CVaR at confidence level α, expressed as a positive loss measure. Under the assumption that daily returns are independent and identically distributed with constant mean and volatility, the annualized CVaR is computed as

$$\mathrm{CVaR}_{\mathrm{ann},\alpha} = \sqrt{D}\,\mathrm{CVaR}_{d,\alpha} - (D - \sqrt{D})\,\bar{r}_d. \tag{51}$$

This expression combines linear scaling of the mean return component with square-root-of-time scaling of the tail-risk component. Its derivation is provided below. The formula is exact under the i.i.d. Gaussian additive-return assumption and is used here as an annualization convention outside that setting.

a) Derivation: The expression above follows from a Gaussian time-aggregation argument. Let r_t denote the daily portfolio return and define the daily loss as $L_t = -r_t$. Assume that daily returns are independent and identically distributed as

$$r_t \sim \mathcal{N}(\mu_d, \sigma_d^2). \tag{52}$$

Then

$$L_t \sim \mathcal{N}(-\mu_d, \sigma_d^2). \tag{53}$$

For a normally distributed loss variable $L = m + sZ$, with $Z \sim \mathcal{N}(0,1)$, the CVaR at confidence level α is

$$\mathrm{CVaR}_\alpha(L) = m + s\kappa_\alpha, \qquad \kappa_\alpha = \frac{\phi(z_\alpha)}{1-\alpha}, \qquad z_\alpha = \Phi^{-1}(\alpha), \tag{54}$$

where Φ and ϕ are the standard normal distribution function and density, respectively. Therefore, the daily CVaR expressed as a positive loss measure is

$$\mathrm{CVaR}_{d,\alpha} = -\mu_d + \sigma_d\kappa_\alpha. \tag{55}$$

Now consider the annual additive return

$$R_D = \sum_{t=1}^{D} r_t. \tag{56}$$

Under the i.i.d. Gaussian assumption,

$$R_D \sim \mathcal{N}(D\mu_d, D\sigma_d^2), \tag{57}$$

and the corresponding annual loss $L_D = -R_D$ satisfies

$$L_D \sim \mathcal{N}(-D\mu_d, D\sigma_d^2). \tag{58}$$

Hence,

$$\mathrm{CVaR}_{\mathrm{ann},\alpha} = -D\mu_d + \sqrt{D}\sigma_d\kappa_\alpha. \tag{59}$$

Using Eq. (55),

$$\sigma_d\kappa_\alpha = \mathrm{CVaR}_{d,\alpha} + \mu_d. \tag{60}$$

Substituting this into the previous expression gives

$$\mathrm{CVaR}_{\mathrm{ann},\alpha} = -D\mu_d + \sqrt{D}\,(\mathrm{CVaR}_{d,\alpha} + \mu_d) \tag{61}$$

$$= \sqrt{D}\,\mathrm{CVaR}_{d,\alpha} - (D - \sqrt{D})\mu_d. \tag{62}$$

In the empirical implementation, μ_d is replaced by the sample mean $\bar{r}_d$. The result is exact under the i.i.d. Gaussian additive-return assumption and is used as an annualization convention outside that setting.

arXiv:2509.17876v3 [quant-ph] 7 Oct 2026

Quantum Portfolio Optimization: An Extensive Benchmark

Eric Stopfer[1,*] and Friedrich Wagner[1]

[1]Fraunhofer Institute for Integrated Circuits, Nürnberg

[*]eric.stopfer@iis.fraunhofer.de

October 2026

Abstract

Recently, several researchers proposed portfolio optimization as a potential use case for quantum optimization. However, the literature is lacking an extensive benchmark quantifying the potential of quantum computers for portfolio optimization. In this work, we contribute to closing this gap. We provide a computational study, comparing quantum approaches against state-of-the-art classical methods on a meaningful, real-world instance set. In particular, we compare quantum annealing and the quantum approximate optimization algorithm against classical mixed-integer programming, simulated annealing, steepest descent local search, tabu search and a problem-tailored heuristic. We consider a variance-minimizing variant of portfolio optimization which we show to be more difficult to solve for classical optimizers than return-maximizing or multi-objective formulations. Our benchmark data set comprises 270 instances with up to 1,000 assets from actual stock data. Due to hardware limitations, quantum methods could only be tested for instances with at most 30 assets. The results show that all instances can be solved to proven optimality by mixed-integer programming in the order of seconds. Moreover, the problem-tailored heuristic consistently outperforms quantum approaches in terms of solution quality for fixed runtime. Thus, we conclude that there is only very limited room for a potential quantum advantage for the considered variant of portfolio optimization.

1 Introduction

Combinatorial optimization problems play a central role in various fields such as logistics and finance [1]. In practice, these problems are often tackled by mathematical optimization solvers, which can solve large-scale mixed-integer and quadratic optimization problems, including many NP-hard instances, to proven optimality [2, 3]. Nevertheless, some practically relevant problems remain intractable for state-of-the-art classical methods [4]. This fact has motivated researchers to develop quantum algorithms for such problems [5]. However, the limitations of existing quantum computers prohibit the implementation of exact quantum algorithms for optimization problems [6]. As a result, researchers focus on heuristic quantum algorithms, which require significantly less resources than exact approaches [7]. Well-known examples of such quantum heuristics are quantum annealing [8, 9] and the quantum approximate optimization algorithm (QAOA) [10, 11]. Recently, several works proposed portfolio optimization as a suitable problem class for achieving quantum advantage [12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22]. In this work, we provide experimental evidence suggesting that even large instances of portfolio optimization can be solved to proven optimality by modern classical solvers in the order of seconds. Moreover, our problem-tailored classical heuristic outperforms quantum approaches in terms of solution quality for fixed runtime. Our results thus set the bar high for a practical quantum advantage for the considered variant of portfolio optimization.

Related work. Various variants of the portfolio optimization problem exist in literature. The authors of [15, 16, 18, 19] consider a variant that can be described as *choose-asset-or-not*. Therein, as the name suggests, the task is to decide for each asset if we add it to the portfolio or not, equally weighted. In this work, we consider an extended variant where the goal is to find optimal *weights* of assets in the portfolio, that is, we assign a fraction of the invested money to each asset. This variant can be further subdivided with respect to the considered objective. Some researchers focus on maximizing return with limited return variance [13, 17]. Others aim to minimize the variance with a fixed return [18, 23]. Several authors combine both objectives by maximizing a weighted sum of return and variance [15, 19, 24].

There also exist approaches in literature which include further practical aspects of stock trading into the model, for example, short-selling-options, risk-free investments and costs that are associated with asset acquisitions [12, 20, 24]. The authors of [12] consider a data test set consisting of 10, 50, 200 and 400 assets of the S&P500 stock market which is similar in structure but less granular than the data used in this work.
Classical solution approaches to the portfolio optimization problem include both heuristics [24, 25, 26, 27] and exact methods [28, 29, 30]. Notably, for fixed runtime, the heuristic developed in [30] finds solutions of better quality for instances of the choose-asset-or-not problem than the commercial classical solver CPLEX [31].
Recently, researchers proposed quantum approaches to the portfolio problem, including QAOA [19, 20], Variational Quantum Eigensolver (VQE) [22] and quantum annealing [13, 16, 17, 18, 21]. The authors of [13], [16] and [17] conclude that quantum annealing may outperform classical methods for portfolio optimization. However, their studies employ a closed-source hybrid quantum-classical solver, which makes it impossible to divide classical from quantum contributions to the solution. On the contrary, in this work, we stick to methods with known classical and quantum components. Regarding QAOA, the authors of [20] report near-optimal results for small problem instances of 8 assets with an idealized simulator of a gate-based quantum computer. Ref. [19] provides guidelines on the selection of penalty and circuit parameters as well as on the classical optimizer. In [18, 21], the authors use reverse quantum annealing to improve upon conventional quantum annealing. Ref. [22] studies the influence of penalty coefficients in VQE as well as the effects of different quantum devices. Other approaches cover problem sparsification [26], problem decomposition [15], warm-starts [32] and quantum circuit cutting techniques [33]. For a comprehensive review on solution approaches to the portfolio optimization, we refer the interested reader to [34].

Our contribution. In this work, we provide a computational study, including 270 problem instances with up to $1,000$ assets. The instances are generated from real-world stock data providing instances with realistic return and covariance. However, we note that we do not make any tests based on actual investment performance. Moreover, we find optimal portfolio weights instead of only deciding whether to select an asset or not. Thus, our approach resembles practical scenarios. We compare QAOA, quantum annealing, two exact classical solvers, three meta-heuristics and one problem-specific heuristic. Our contribution is twofold. First, the large instance set allows us to draw conclusions that are consistent across different instances and instance sizes. Second, applying standard implementations of state-of-the-art quantum and classical solution methods, we provide a comprehensive and fair benchmark. In particular, our methods have well-defined quantum and classical parts such that we can precisely divide quantum from classical contributions.
The remainder of this work is organized as follows. In section 2, we define three variants of portfolio optimization which we consider in this work. Section 3 studies the classical hardness of these variants to identify the one with the largest potential for quantum advantage. In section 4, we develop a quadratic unconstrained binary optimization (QUBO) model for the most difficult portfolio optimization variant. Section 5 introduces the solution methods we compare in our benchmark. In Section 6, we report on the results of our benchmark study. Finally, in section 7, we summarize our findings, draw a conclusion and state open questions.

2 The Portfolio Optimization Problem

In this section, we introduce three different variants of the portfolio optimization problem, which we consider in our computational study. As originally formulated by Markowitz in [35], the portfolio optimization problem asks for a selection of an asset portfolio which maximizes the expected portfolio return while minimizing the variance of the return..
More formally, given a set of n assets, the task is to find optimal *asset weights* $\omega_i \in [0,1]$, $i \in \{1,\dots,n\}$. The asset weight ω_i defines the share of the invested money allocated to the i-th asset. Accordingly, it holds

$$\sum_{i=1}^{n} \omega_i = 1 \ .$$

We impose upper bounds $u_i \in (0,1]$ on the asset weights,

$$\omega_i \leq u_i \quad \forall i \in 1,\dots,n.$$

Each asset i has an *expected return* $r_i \in \mathbb{R}$. Moreover, for each pair of assets (i, j), we are given a *return covariance* $\sigma_{ij} \in \mathbb{R}$.
The original problem formulation by Markowitz [35] includes the objectives of maximizing the expected portfolio return

$$\mu(\omega) \coloneqq \sum_{i=1}^{n} \omega_i r_i$$

and minimizing the portfolio variance

$$\sigma^2(\omega) \coloneqq \sum_{i=1}^{n} \sum_{j=1}^{n} \omega_i \omega_j \sigma_{ij} \ .$$

In general, the two objectives of maximizing return and minimizing variance cannot be fulfilled at the same time. Thus, several problem variants exist, which we summarize in the following. In the first variant, we aim at maximizing the return with an upper bound $\nu > 0$ on the variance. This can be formulated by the quadratic program

$$\begin{aligned} \textbf{MaxRet}: \quad \max_{\omega} \quad & \mu(\omega) \\ \text{s.t.} \quad & \sigma^2(\omega) \leq \nu \\ & \sum_{i=1}^{n} \omega_i = 1 \\ & 0 \leq \omega_i \leq u_i \quad \forall i \in \{1, \dots, n\} \ . \end{aligned}$$

The second variant minimizes the variance while ensuring a minimum return $\epsilon \in \mathbb{R}$. This is formalized by the quadratic program

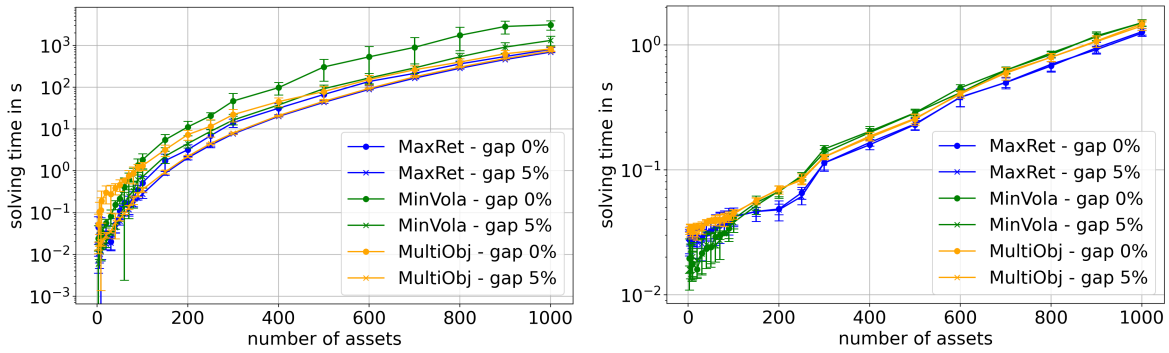
$$\begin{aligned} \textbf{MinVola}: \quad \min_{\omega} \quad & \sigma^2(\omega) \\ \text{s.t.} \quad & \mu(\omega) \geq \epsilon & (1) \\ & \sum_{i=1}^{n} \omega_i = 1 & (2) \\ & 0 \leq \omega_i \leq u_i \quad \forall i \in \{1, \dots, n\} \ . \end{aligned}$$

The name of the QP reflects its objective of minimizing portfolio volatility. Finally, the third variant combines both objectives by a fixed factor $\lambda > 0$, which depends on the risk aversion of the investor. This gives rise to the quadratic program

$$\begin{aligned} \textbf{MultiObj}: \quad \min_{\omega} \quad & \sigma^2(\omega) - \lambda \mu(\omega) \\ \text{s.t.} \quad & \sum_{i=1}^{n} \omega_i = 1 \\ & 0 \leq \omega_i \leq u_i \quad \forall i \in \{1, \dots, n\} \ . \end{aligned}$$

The mathematical programs MaxRet, MinVola and MultiObj belong to the class of convex quadratically constrained quadratic programs, which can be solved in polynomial time by interior point methods [36, 37]. The convexity is due to the fact that $(\sigma_{ij}) \in \mathbb{R}^{n \times n}$ is a sample covariance and thus positive semidefinite by definition (6) in Appendix A. Although being efficiently solvable in theory, portfolio optimization is often tackled via heuristics in practice.

Test set generation. In order to generate a realistic set of instances, we consider the daily prices of 1,978 assets in the Nasdaq stock exchange [38] during the years 2020 to 2023, available in [39] and accessible with [40]. All those assets have no missing price data during this timeframe. From the daily prices, we calculate the expected returns and the return covariances. For details on the calculation of returns and covariances from price data, we refer to Appendix A. To generate an instance with a given number n of assets, we randomly draw n assets from the data base.
We generate 10 instances for each problem variant MaxRet, MinVola and MultiObj for each size $n \in \{3, 5, 7, 8, 10, 15, 20, 25, 30, 40, 50, 60, 70, 80, 90, 100, 150, 200, 250, 300, 400, 500, 600, 700, 800, 900, 1000\}$. We still



(a) SCIP (b) Gurobi

Figure 1: Average solver runtime for different problem variants. We compare SCIP (a) and Gurobi (b) with relative optimality gaps of 0 % and 5 %. Data points are averages over 10 instances and error bars show the empirical standard deviation.

need to define values for the maximum portfolio variance ν in MaxRet, the minimum portfolio return ϵ in MinVola and the risk aversion λ in MultiObj. As we strive towards an above-average performing portfolio, in Model MaxRet, we choose the maximum portfolio variance ν as the 30 % quantile of variances of randomly generated portfolios. Similarly, in Model MinVola, we choose the minimum portfolio return ϵ as the 70 % quantile of all asset returns. Moreover, in order to account for the different magnitudes of μ and σ^2, we set the risk aversion factor λ of Model MultiObj to the average of fraction $\frac{\sigma^2}{\mu}$ of randomly generated portfolios. We remark that, in general, the selection of the parameters ϵ, ν, λ is subject to the investor's perspective. Finally, we set the asset limits to $u_i = \max\left(\frac{1}{10}, \frac{3}{n}\right)$ for $i \in \{1, ..., n\}$. The asset limits thus form a monotonically decreasing series, which for 5 assets has a value of 60%, for 10 assets 30%, for 20 assets 15% and stays at 10% from $n = 30$ onward. By this formula, we try to emulate real-world fund compositions, which can be monitored in [39].
In principle, some generated test instances of MinVola and MaxRet could be infeasible. However, our computational experiments revealed that all considered test instances are indeed feasible.

3 Classical Solution Time for Different Variants

In this section, we study the classical computational difficulty of the three problem variants MaxRet, MinVola and MultiObj. Our goal is to identify the hardest variant for classical solvers which thus bears the largest potential for a quantum advantage. To this end, we solve all instances of the test set described in Section 2 with classical mathematical optimization solvers to proven optimality and compare the overall runtime. For each instance, we solve the three problem variants with the open-source solver SCIP 9.2 [41, 42] and the commercial solver Gurobi 11.0.3 [43]. Moreover, we impose a time limit of 3,600 s and relative optimality gaps of 0 % and 5 %. Here, the relative optimality gap, short gap, is defined as

$$g := \frac{|b - c|}{|c|}$$

where b is the best bound on the objective and c is the objective value of the best available solution.
The average solving times are visualized in Figure 1.
First, we observe that Gurobi solves most of the problems in less than one second and thus is considerably faster than the open-source solver SCIP, with a speed-up factor of over 1,000 for large problem instances with $n = 1,000$ assets. The speed-up can partly be explained by the fact that Gurobi detects the convexity of the problem and consequently solves it using the Barrier algorithm, a polynomial-time interior-point method. In contrast, as observed in the solver logs, SCIP relies on outer approximation with spatial branching, which is a method for general, non-convex QPs and has exponential worst case running time.
Second, the influence of the duality gap on the solving time is larger for SCIP than for Gurobi.
Third, the Model MinVola takes the longest time to be solved in each of the four solver-gap configurations. Thus, MinVola is the most promising variant for a potential quantum speedup. Consequently, in the following we focus on the MinVola variant that minimizes variance while still maintaining a certain portfolio return level.

4 QUBO Transformation

Existing implementations of both quantum annealing and QAOA require the input problem to be modeled as QUBO, that is, problems of the form

$$\min_{x\in\{0,1\}^n} x^t Q x$$

where $n \in \mathbb{N}$ and $Q \in \mathbb{R}^{n\times n}$ is a real-valued matrix. In this section, we derive a QUBO model of the MinVola variant. To this end, we define the penalty-based objective function

$$f(\omega) \coloneqq \sigma^2(\omega) + \phi\left(\mu(\omega) - \hat{\epsilon}\right)^2 + \psi\left(\sum_{i=1}^{n}\omega_i - 1\right)^2, \tag{3}$$

where $\omega \in [0,1]^n$ are the asset weights and $\phi, \psi \in \mathbb{R}_{\geq 0}$ are penalty factors that penalize violations of the return constraint (1) and the normalization constraint (2), respectively. Here, we replace the "$\geq$" in the return constraint (1) by "=". According to Markowitz portfolio theory the variance-minimal portfolio for $\mu(\omega) \geq \xi$ has exactly return $\mu(\omega) = \xi$ for a sufficiently large ξ. We elaborate on this in appendix B. Also, we select the return target as $\hat{\epsilon} = \epsilon + 1\%$, as sampling-based methods return a distribution of portfolios whose returns scatter around the penalised target. Since a feasibility analysis accepts any solution satisfying the original inequality $\mu(\omega) \geq \epsilon$, shifting the penalised target slightly upwards increases the number of samples that are feasible for the original problem. The same target $\hat{\epsilon}$ is used for all QUBO-based methods. Thus, the comparison between the solving methods in the following chapter remains unaffected. From a theoretical point of view, finding optimal penalty factors is itself an NP-hard problem [27]. Consequently, we employ a heuristic penalty approach in this work, which we detail in section 6.
In order to derive a valid QUBO model, the continuous variables $\omega_i \in [0,1]$ in (3) have to be transformed to binaries. As proposed in [13, 17], we write the portfolio weights $\omega_i \in [0,1]$ as a linear combination of binary variables $\omega_{ij} \in \{0,1\}$, $j \in \{1,\dots,d+1\}$, via

$$\omega_i = u_i \cdot \left(\frac{1}{2^d}\omega_{i,d+1} + \sum_{j=1}^{d}\frac{1}{2^j}\omega_{i,j}\right) \quad \forall i \in \{1,...,n\}. \tag{4}$$

Here, $d \in \mathbb{N}$ controls the precision of discretization. An advantage of this discretization is the implicit incorporation of the upper bounds u_i on the asset weights. Inserting (4) in (3) yields the QUBO model for MinVola.

5 Methods

In this section, we introduce the solution methods that we compare in our benchmark. Furthermore, we explain how associated parameters are chosen. For all methods, we leave parameters to default values if not stated otherwise.

Exact mathematical optimization solver. In this benchmark, we chose Gurobi [43] as the classical solver that provably solves the problem to optimality. In order to ensure fairness when comparing the other methods to the exact solver, we solve the discretized version of the formulation MinVola.

Quantum annealing. Quantum annealing is a heuristic algorithm that runs on analog quantum computers [44, 45]. The concept of quantum annealing is based on the adiabatic theorem of quantum mechanics [46]. Quantum annealing prepares a quantum mechanical system in the ground state of some initial Hamiltonian H_0. Then, it evolves the system according to

$$H(t) = A(t)H_0 + B(t)H_1\,.$$

Here, H_1 is the problem Hamiltonian whose ground state encodes the solution to some optimization problem. $A : [0,\tau] \to [0,1]$ is chosen such that $A(0) = 1$ and $A(\tau) = 0$, where $\tau > 0$ is called *annealing time*. Similarly, $B : [0,\tau] \to [0,1]$ is chosen such that $B(0) = 0$ and $B(\tau) = 1$. Now, the adiabatic theorem states that the system is in the ground state of H_1 at $t = \tau$ for $\tau \gg 1$ if the energy gap between the ground state and the first excited state is non-zero for all $t \in [0,\tau]$ and if $\partial_t H(t)$ is finite for all $t \in [0,\tau]$.

In our computational experiments between the months of June and September of 2025, we use a D-Wave Advantage 2 processor which has more than 4,400 physical qubits, arranged in the Zephyr topology [47]. However, the limited device connectivity requires an embedding of each QUBO variable into possibly multiple physical qubits [48, 49]. This embedding overhead increases with the density of the QUBO problem. All considered QUBO instances have a density of 100%. For embedding, we therefore use the `DWaveCliqueSampler` [50] from the python package *dwave-system* [51], which employs pre-defined embeddings for fully connected QUBO problems and is thus more suited for dense problems than constructive embedding algorithms like the default `MinorMiner` [52]. Also, we test annealing times of 1 µs, 5 µs, 20 µs and 50 µs. The *annealing schedule*, which defines $A(t)$ and $B(t)$, is kept as default. However, we vary the *chain strength* parameter which can prevent the breaking of a chain of connected qubits representing a single QUBO variable. We explicitly decided against more sophisticated quantum annealing concepts like warm-starts [32] or special annealing schedules [18, 21, 53] because our goal is to test the quality of solutions returned by the basic provided infrastructure without focusing on presolving and fine-tuning the methodology and the associated parameters.

QAOA. The quantum approximate optimization algorithm is a quantum-classical hybrid algorithm, originally proposed by Farhi et al. [10]. Like quantum annealing, QAOA is conceptually based on the adiabatic theorem of quantum mechanics [46]. However, in contrast to quantum annealing, QAOA is designed for gate-based, universal quantum computers. QAOA depends on real-valued parameters $\gamma = (\gamma_1, ..., \gamma_p)$ and $\beta = (\beta_1, ..., \beta_p)$. The hyper-parameter $p \in \mathbb{N}$ controls the complexity of the algorithm. QAOA starts with the uniform superposition quantum state $|+\rangle^N$ and evolves it according to

$$|\psi(\beta,\gamma)\rangle = e^{-i\beta_p H_M} e^{-i\gamma_p H_C} \ ... \ e^{-i\beta_1 H_M} e^{-i\gamma_1 H_C} |+\rangle^N . \tag{5}$$

Here H_C is the so-called *problem Hamiltonian*, defined by

$$H_C |x\rangle = C(x) |x\rangle \quad \forall x \in \{0,1\}^N$$

where $C(x)$ is the QUBO cost function. H_M is called the *mixing Hamiltonian* and is defined by

$$H_M = \sum_{i=1}^{N} X_i$$

where X_i is the Pauli X gate acting on the i-th qubit. For given parameters β and γ, solutions to the QUBO problem are sampled from the quantum circuit implementing (5). In this work, we employ three different methods to compute the parameters β and γ. First, for $p = 1$, we perform a grid search by analytically calculating the expectation value of the resulting QAOA circuit via the formula developed in [54]. We choose the parameter values that result in the minimum expectation value. Second, we use the linear ramp QAOA (LR-QAOA) protocol introduced in [55]. Accordingly, we set $\beta_i = (1 - \frac{i}{p})\Delta_\beta$ and $\gamma_i = (\frac{i+1}{p})\Delta_\gamma$ for $i \in \{1, \dots, p\}$. In our experiments, we choose $\Delta_\beta = 0.3$ and $\Delta_\gamma = 0.6$ as proposed in [55]. Finally, we optimize the parameters with the local optimizer COBYLA [56, 57] using a noiseless quantum simulator [58, 59]. Here, we initialize the parameters with the LR-QAOA-formula.
After selecting parameters, we execute QAOA on the gate-based quantum computer *ibm_strasbourg* based on a 127 qubit *Eagle* processor [60]. The experiments on *ibm_strasbourg* were conducted in July 2025. Circuits were transpiled by *generate_preset_pass_manager* [61] with *optimization_level*= 1. No particular error mitigation techniques are set. We also run a benchmark with an ideal quantum simulator [58]. Exponentially increasing memory requirements limit the classical simulation of QAOA to roughly 30 variables. We note that our method of calculating parameters has no optimality guarantee and that better QAOA parameters might exist. Finally, we remark that like for quantum annealing, we decided against more advanced concepts like warm-starts [62] or constraint-preserving mixers [63] since we aim at testing the existing quantum architecture without method fine-tuning.

Steepest Descent. The steepest descent local search tries to improve the current solution by investigating its neighborhood. Here, the neighborhood is defined as all solutions having a Hamming distance of one to the current solution, which is equivalent to one-bit-flips. If the best solution in the neighborhood improves the objective value, it is accepted as the new current solution. Otherwise, the algorithm terminates. The main advantage of the steepest descent search is its fast runtime. A disadvantage is its inability to escape local minima. In our benchmark study, we employ the open-source implementation available in [64, 65].

Simulated Annealing. Simulated annealing is a popular classical heuristic. The name of this algorithm is based on its similarity to the process of annealing in metallurgy, which aims to harden a metal by controlled heating and cooling. In its general form, simulated annealing starts with a random solution. Then, simulated annealing iteratively considers a random solution in the neighborhood of the current solutions. If this solution improves upon the current objective value, it is accepted as the new current solution. Otherwise, it is still accepted as the new current solution with a probability decreasing with the number of iterations. By this methodology, the algorithm explores a large fraction of the solution space at the start, while at the end, objective-improving solutions are favored. The algorithm returns the solution with the best objective value [66]. Several parameters can be modified, for example, the definition of the neighborhood function and the acceptance probability function. In our computational study, we use an open-source implementation of simulated annealing [67, 68].

Tabu Search. Tabu search is a classical heuristic conceptually similar to steepest descent. Tabu search starts with a random solution, then explores the neighborhood of this solution and chooses the best solution in this neighborhood while putting the previous solution on a tabu list. Solutions on the tabu list are excluded from any neighborhood. The tabu list has a limited length and solutions are removed in a first-in-first-out manner. The tabu list allows the algorithm to escape local minima. Termination criteria can be runtime, number of steps or objective value. Adjustable parameters are the selection of the neighborhood function, the termination criterion and the length of the tabu list. In our computational study we use an open-source implementation of tabu search [69, 70].

Problem-specific heuristic. We also developed a problem-specific heuristic for the MinVola version of the portfolio optimization problem which does not require building the mathematical model. For an instance with n assets, it creates up to n feasible solutions, but does not guarantee feasibility. The heuristic starts with an empty portfolio and then increases the asset weight ω_i of some asset $i \in \{1, \dots, n\}$ by a fixed amount of $\delta > 0$. Then, it iteratively adds weight δ to the asset that reduces the portfolio variance the most while still satisfying constraint (1). In our case, we use $\delta = \frac{u_i}{2^d}$ to apply the same discretization as for the other methods. If the return constraint can not be satisfied, the algorithm increases the weight of the asset with the largest return, i.e., it moves towards return feasibility rather than terminating. The heuristic stops when the sum of weights is greater or equal to 1. Finally, the weights are normalized and the solution is checked for feasibility For a more detailed description and pseudocode of the problem specific heuristic, we refer to appendix C.

6 Computational Experiments

In this section, we benchmark the solution methods discussed in Section 5 on the portfolio optimization problem test set introduced in Section 2. The goal of our computational experiments is to quantify the potential of currently available quantum computers for portfolio optimization. Our study follows the general rules on good-practice for benchmarks from [71], which are relevance, reproducibility, fairness, verifiability and usability. Our code and data are publicly available at [72].

Benchmark procedure. The benchmark study was conducted on a standard notebook equipped with a 10-core, 16 GB RAM Intel Core i7 CPU. We compare the solution methods introduced in Section 5, that is, QAOA, quantum annealing, simulated annealing, steepest descent, tabu search and the problem-specific heuristic on the MinVola problem variant.
In the QUBO model (3), we set the penalty factors $\phi = \psi = 1000$ for violations of the return constraint (1) and the normalization constraint (2), respectively. We determine these factors by analyzing the magnitude of the objective and constraint terms. We detail the derivation of the penalty factors in appendix D, where we conduct a magnitude-analysis of the objective terms for the optimization problem.
With these penalty factors, a small absolute constraint violation of 0.01 in the constraints (1) or (2) will lead to a penalization of $1000 \cdot 0.01^2 = 0.1$ which is in the same order of magnitude as the objective value. Additionally, we choose the discretization coarseness in (4) as $d = 3$. This leads to $d + 1 = 4$ times as many variables in the discretized model as in the continuous model. Consequently, for an instance with n assets, the resulting QUBO has $4n$ variables. The value of $d = 3$ results from a trade-off between QUBO size and discretization precision.
To ensure fairness among the methods, we enforce a time limit of 60 seconds on the execution time of the algorithm on the device, i.e., the sampling time. We do not include the model construction into

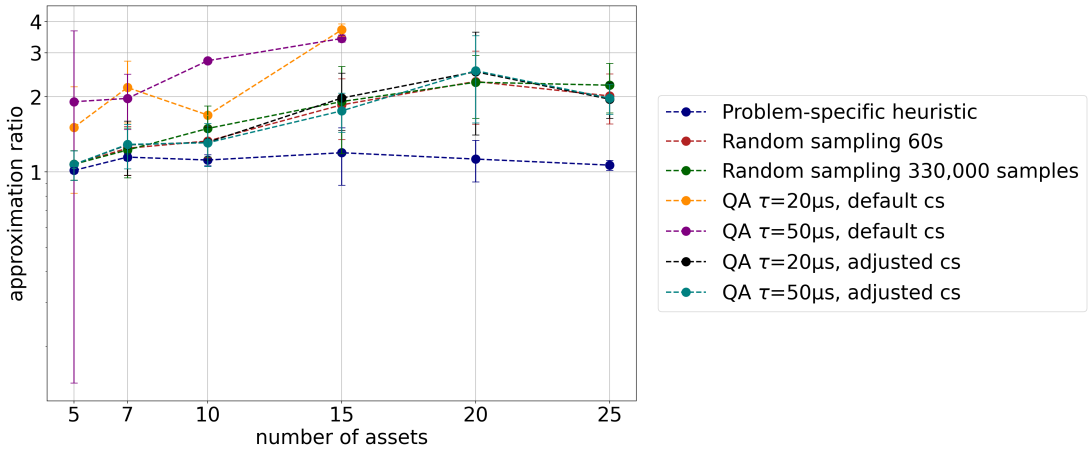
the time limit since the time for model construction varies depending on which library is used (like qiskit_optimization or dimod). Moreover, our process of first building the Quadratic Program and then transforming it into a QUBO could be sped up by directly building the QUBO, possibly in a fast programming language like C++. For the same reason, we do not include the result analysis time, i.e., calculating the QUBO objective values, which could be sped up by a faster implementation. We also do not include the embedding time into the time limit since the problem is always fully connected and thus, ideally, the embedding for a problem is calculated just once and reused for every other problem of the same size. Also, cloud connection, job submission and queueing times are not considered because we are interested in analyzing the capabilities of the quantum computer itself rather than access overhead. For the grid-search QAOA, we split up the 60 seconds into 30 seconds of training and 30 seconds of sampling with the best found parameters. We configure the size of the grid such that its execution time takes roughly 30 seconds. For the QAOA with optimized parameters, we spent a maximum of 52 seconds on training while the rest is used to exclusively sample with the best found parameters. This ensures that we have at least a minimum amount of time for sampling with high-quality parameters. Of course, the sampling results found during the training of QAOA are also considered in the final result. We remark that the accounting of runtime is deliberately favourable to the quantum methods since there is some infrastructure overhead involved like cloud connection, job submission and queueing. Finally, we note that a slightly smaller budget than 60 seconds was configured in the implementation as a safety margin to ensure that the limit is never exceeded.
After sampling within the boundaries of the time limit, we calculate the QUBO objective values of the entire sample. We take the 100 best solutions and execute a postprocessing step by scaling the weights such that they sum up to 1 in order to satisfy the normalization constraint (2). Then, we analyze these solutions with respect to feasibility and objective value for the original MinVola problem.
We then return the best found solution. This resembles a practical setting where one is usually interested in the best solution rather than statistical performance measures like expectation values or quantiles.
As our primary solution quality metric, we consider the *approximation ratio* which is defined as

$$\Theta := \frac{f_m}{f_{\text{opt}}} \geq 1.$$

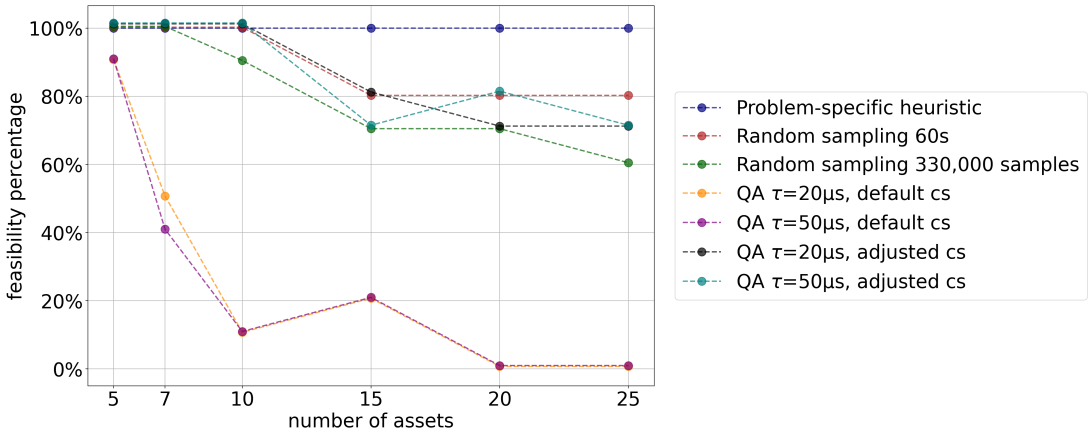
Here f_m is the objective value returned by the method m, and f_{opt} is the value of the optimal solution of the discretized version of the problem. which we calculate by an exact mathematical optimization solver. We have $f_m \geq f_{\text{opt}} \geq 0$ since MinVola is a minimization problem and the portfolio variance is always non-negative. Thus, a value of Θ close to 1 indicates near-optimal solutions.
In addition, we measure the *feasibility percentage*, which we define as the fraction of the 10 test instances per problem size for which the considered method returns at least a single feasible solution.
Finally, the *number of samples* measures how many samples are generated by the respective algorithm within the time limit.

Benchmark Results. We first discuss the results of quantum annealing, shown in Figures 2a, 2b and 2c. We observe that quantum annealing with the default calculation of the chain strength cs yields the worst approximation ratio (Figure 2a) and the lowest feasibility percentage (Figure 2b) in the red and pink lines. In particular, even random sampling has a lower average approximation ratio on the same number of samples while finding more feasible solutions (compare green and yellow lines in Figures 2a, 2b, 2c).
After observing high chain break fractions, we tested different chain strength parameters. Since all QUBO instances of equal size are fully connected and embedded using the same clique-embedding strategy, the resulting embeddings exhibit identical chain structures and chain lengths across instances of the same size. Empirically, the observed chain break fractions were nearly identical across different problem instances of equal size, indicating that chain break behavior is dominated primarily by problem size and embedding geometry rather than by instance-specific QUBO coefficients. For problems of $5, 7, 10$ and 15 assets we set $cs = 3$, for 20 assets we set $cs = 4$ and for 25 assets we set $cs = 5$. These parameters result in chain break fractions in the desired range of 0% to 5% as recommended by D-wave during a private correspondence. We would like to point out that such chain strength tuning can only improve the quantum annealing results, but it adds a bias to the reported performance estimate. The results of quantum annealing with the customized chain strengths are shown in the black and gray lines in Figures 2a, 2b and 2c. We observe that quantum annealing with adjusted chain strength is roughly on par with random sampling in terms of both feasibility percentage and approximation ratio. Nevertheless, we observe in Figures 2a and 2b that quantum annealing performs significantly worse than the problem-specific heuristic. We conjecture that the high density of the QUBO problem is a reason for the suboptimal performance of

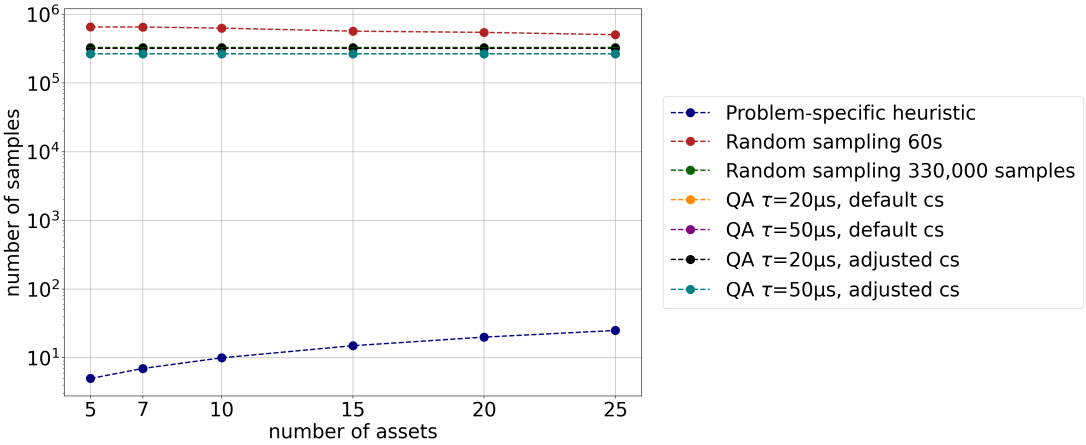
quantum annealing. Embedding the required all-to-all connectivity leads to long chains of qubits for each variable. Emphasizing the large embedding overhead, we note that the largest problem size that can still be embedded on the quantum annealing processor with $4,400+$ physical qubits has 25 assets, which corresponds to 100 QUBO variables. Moreover, varying the annealing time between 20 μs and 50 μs did not significantly improve the results, compare pink versus red and grey versus black in Figures 2a and 2b. We remark that we have conducted additional experiments with annealing times of 1 μs and 5 μs. Those results did not differ significantly from the results with annealing times of 20 μs and 50 μs. Finally, in Figure 2c, we observe that the number of samples generated in 60 s is independent of the problem size. From this, we conclude that the runtime of quantum annealing does not significantly increase with problem size, which might lead to an advantage of future annealing hardware for very large instances.
Next, we turn to the results for QAOA, shown in Figures 3a, 3b, 3c. Similar to quantum annealing, QAOA often falls short on finding feasible solutions, which results in decaying feasibility percentages in Figure 3b. Even when QAOA returns feasible solutions, their approximation ratios quickly turn suboptimal when increasing the problem size. We observe approximation values of ≥ 2 for problems with 20 or more assets in Figure 3a. In general, QAOA does not perform significantly better than random sampling with respect to feasibility and approximation ratio. Compared to the problem-specific heuristic, all QAOA configurations deliver lower feasibility percentages and worse approximation ratios. The QAOA configuration with the best results is the 1-layer linear-ramp (red lines) which returns a lower approximation ratio with a higher feasibility percentage on a similar number of samples than the remaining QAOA configuration. Moreover, we observe that increasing the number of layers worsens the performance although, in theory, the performance improves with the number of layers. Other experiments with 2 and 4 layers affirm this observation while we do not show their results here for clarity. We attribute this paradox mainly to two reasons. First, more layers increase the circuit execution time. Consequently, less shots can be executed in the same time, which can be seen in Figure 3c. Second, the effect of noise increases with the circuit size. A reason for the decaying performance of QAOA with larger problem instances is the rapidly increasing number of swap gates that are required to transpile the QAOA circuit with all-to-all-connectivity to the quantum computer with limited connectivity. For example, for 20 assets, the circuit depth of 1-layer QAOA increases from 161 to $\approx 11,000$ during transpilation.
The results for the classically simulated QAOA are only reported in appendix E, as the largest instance which can still be simulated has only 7 assets. Finally, we remark that we did not run QAOA experiments in which we optimize the parameters on the hardware since the communication and queuing times are prohibitively large which is highly impractical for a parameter optimization loop. We remark that IBM offers a session mode which reduces queuing, however, it drastically consumes more hardware access time than for the jobs themselves.
Lastly, we compare the results for classical heuristics in Figures 4a, 4b and 4c. All open-source implementations (steepest descent, simulated annealing and tabu search) are executed with default settings. In general, all heuristics are able to find feasible solutions for nearly all problem instances, see Figure 4b. Only tabu search does not return feasible solutions for all instances with 800 assets. On the other hand, random sampling failed to return any feasible solutions starting from 40 assets. The approximation ratios of simulated annealing, steepest descent and tabu search exceed the value of 2 starting at 50 assets and the value of 10 at 250 or more assets. The problem-specific heuristic performs best with respect to the average approximation ratio. The approximation ratios of the other heuristics are comparable despite differing significantly in the numbers of samples (Figure 4c). We remark that all heuristics rely on objective evaluation which can take a significant amount of time on large instances. This explains the decreasing number of samples for larger problem instances. For example, for 500 assets, the problem-specific heuristic is only able to return roughly 100 out of 500 possible solutions in 60 s. Thus, quantum methods might be beneficial for even higher asset numbers since they require classical objective evaluation only during the sample evaluation process and not during sampling itself. Nevertheless, the performance of the problem-specific heuristic sets the bar high for a possible quantum advantage.
In Figures 5a, 5b and 5c, we compare the most promising approaches from quantum annealing, QAOA and classical heuristics in order to rank them. In particular, we consider quantum annealing with 50 µs annealing time and adjusted chain strength, the single-layer QAOA with linear ramp parameters and the problem-specific heuristic.
We observe that the problem-specific heuristic has a larger feasibility percentage and a better approximation ratio than the best configurations of both QAOA and quantum annealing. Furthermore, both QAOA and quantum annealing perform roughly as good as random sampling. Notably, random sampling generates significantly more solutions in 60 s (Figure 5c). Finally, we conclude that quantum annealing slightly outperforms QAOA since quantum annealing generates more samples, finds more feasible solu-



(a) quantum annealing approximation ratio

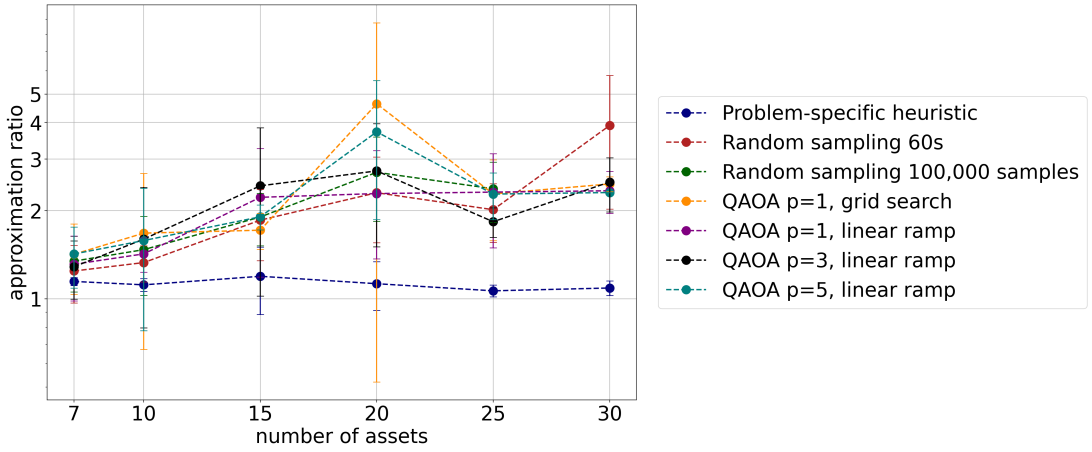


(b) quantum annealing feasibility percentage

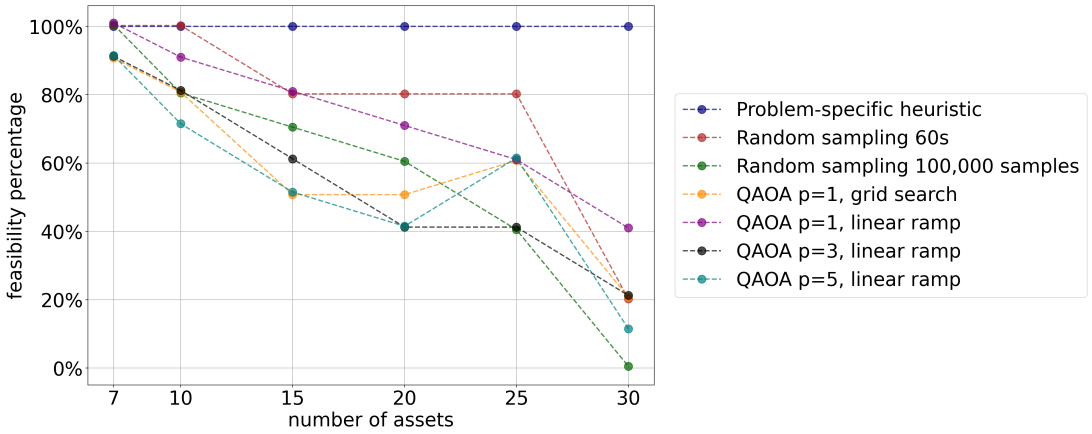


(c) quantum annealing number of samples

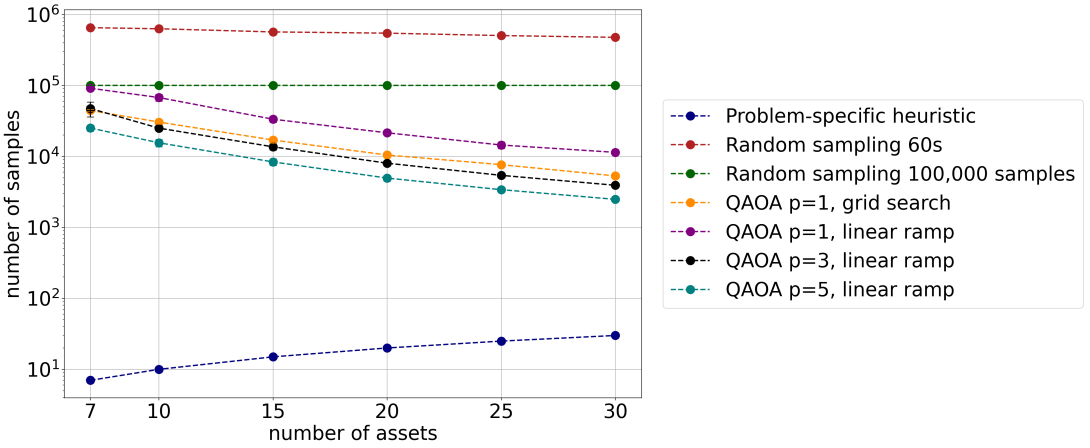
Figure 2: We report the approximation ratio, feasibility percentage and number of samples for quantum annealing. We compare against the problem-specific heuristic and random sampling as baselines. Data points are averages over 10 instances and error bars show the empirical standard deviation. Additionally, distributions of the approximation ratios over the 10 instances can be viewed in the appendix chapter F. Missing data points in the approximation ratio are due to the absence of feasible solutions.



(a) QAOA approximation ratio

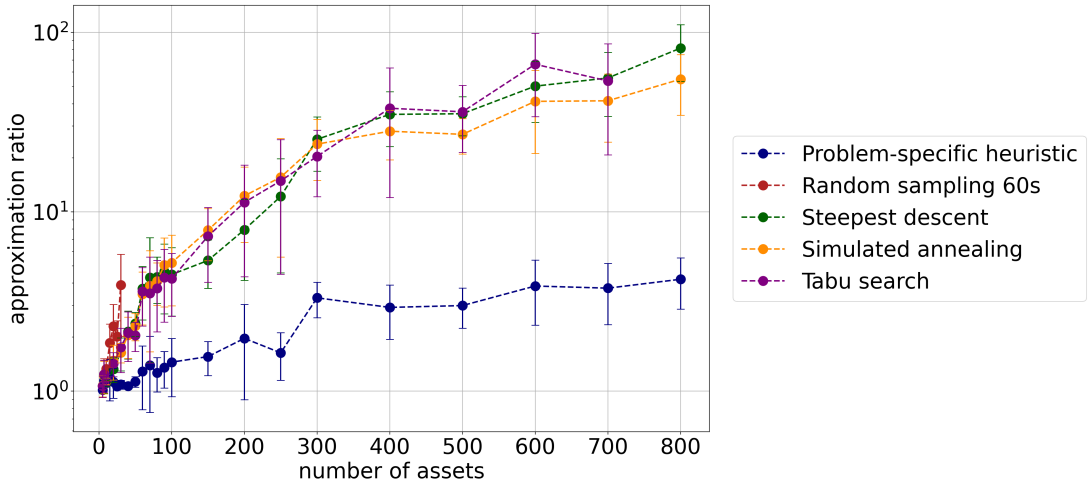


(b) QAOA feasibility percentage

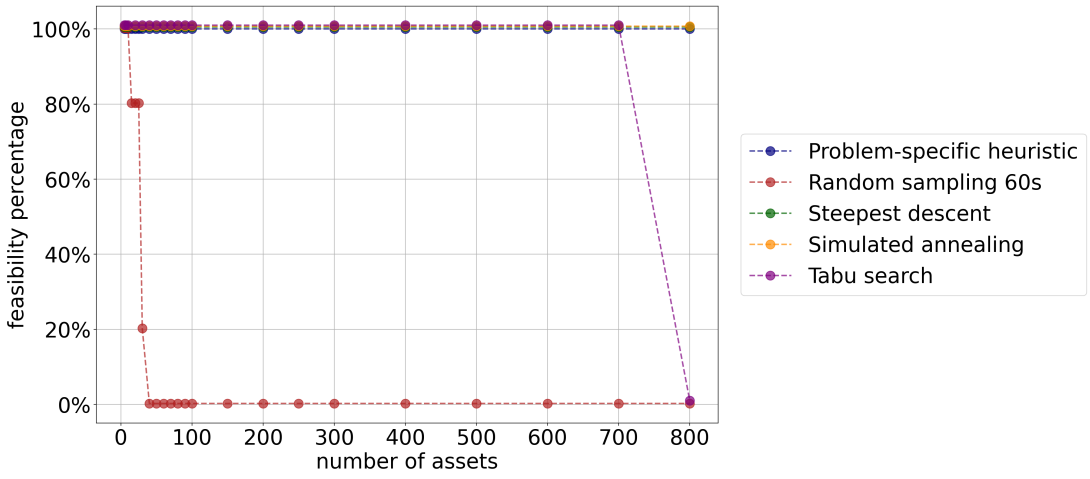


(c) QAOA number of samples

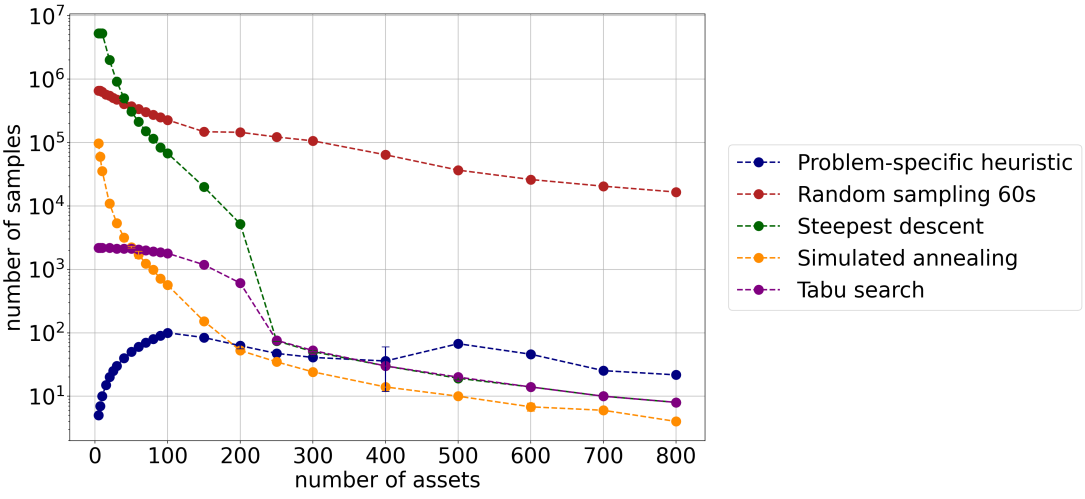
Figure 3: We report the approximation ratio, feasibility percentage and number of samples for QAOA. We compare against the problem-specific heuristic and random sampling as baselines. Data points are averages over 10 instances and error bars show the empirical standard deviation. Additionally, distributions of the approximation ratios over the 10 instances can be viewed in the appendix chapter F. Missing data points in the approximation ratio are due to the absence of feasible solutions.



(a) heuristics approximation ratio

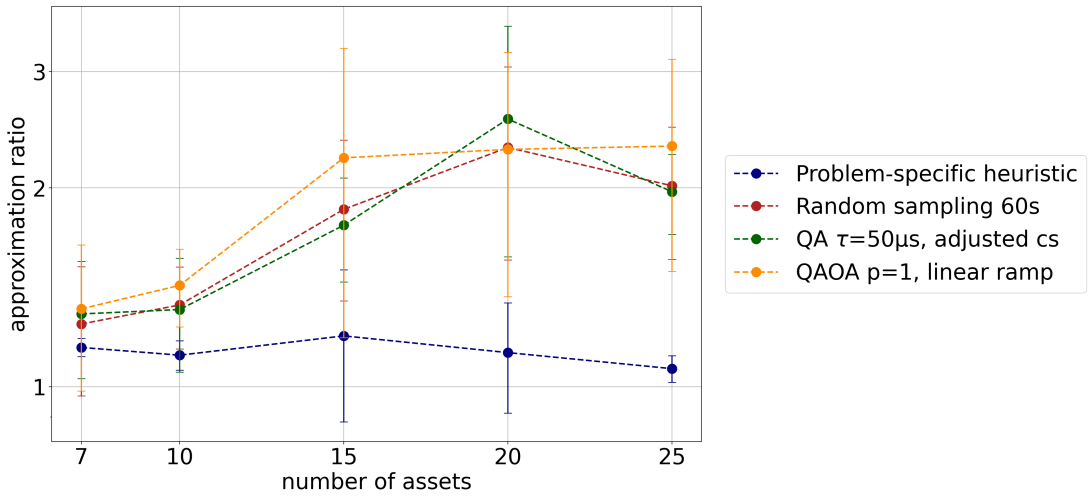


(b) heuristics feasibility percentage

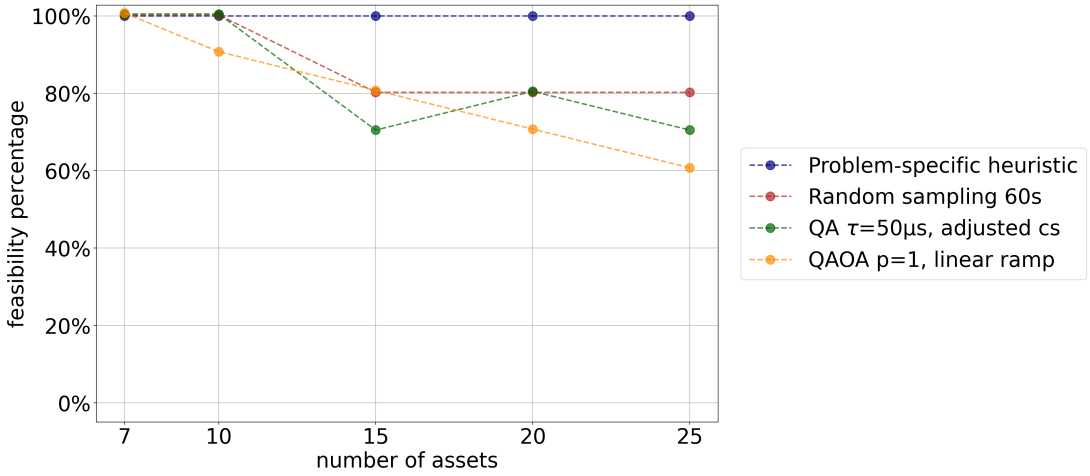


(c) heuristics number of samples

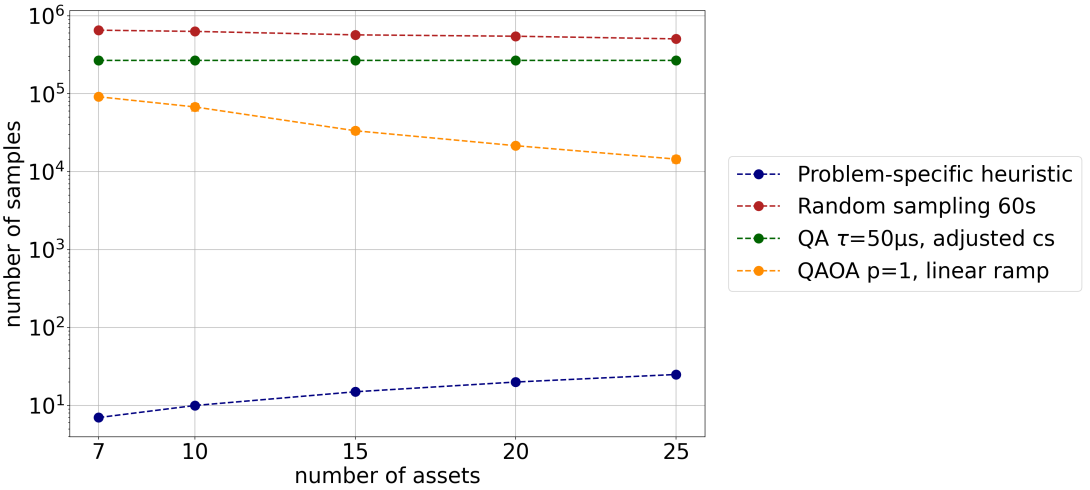
Figure 4: We report the approximation ratio, feasibility percentage and number of samples for the heuristics. We compare against the problem-specific heuristic and random sampling as baselines. Data points are averages over 10 instances and error bars show the empirical standard deviation. Additionally, distributions of the approximation ratios over the 10 instances can be viewed in the appendix chapter F. Missing data points in the approximation ratio are due to the absence of feasible solutions.



(a) Best methods: approximation ratio



(b) Best methods: feasibility percentage



(c) Best methods: number of samples

Figure 5: Comparison of the best performing configurations from our computational study. The best performing configurations are the Problem-specific heuristic, quantum annealing with annealing time of $50\mu s$ and adjusted chain strengths as well as QAOA with 1 layer and linear-ramp parameters. For comparison purposes, random sampling for 60 seconds is also visualized. We report the approximation ratio (a), the feasibility percentage (b) and the number of samples generated during the given time limit of 60 seconds (c). Data points are averages over 10 instances and error bars show the empirical standard deviation.

tions and those solutions are also of slightly higher quality. The difference in the number of generated samples would get larger for increasing problem size since the QAOA circuit length and thus its execution time grows with the number of variables whereas the annealing time stays constant.

7 Conclusion

In this work, we conducted an extensive benchmark of quantum computing for portfolio optimization. Recently, several works suggested portfolio optimization as a suitable candidate for a possible quantum advantage. Our experiments, however, highlight the challenges of achieving such an advantage in practice. We consider a variance-minimizing variant of portfolio optimization which we have shown to be more difficult for classical optimizers than return-maximizing or multi-objective formulations. We then compared both classical and quantum methods on 270 problem instances from real-world stock data. In our benchmark, we impose a time limit of 60 seconds on the sampling and result analysis to ensure fairness among the methods. Our main conclusion is that classical heuristics like simulated annealing, steepest descent, tabu search and a problem-specific heuristic clearly outperform QAOA and quantum annealing regarding solution feasibility and quality. Regarding classical exact optimizers, Gurobi and SCIP differ significantly in their solution times. Here, Gurobi is more than 1000 times faster for large problem instances, solving problems with 1000 assets in the order of seconds. Comparing quantum methods, we observed that quantum annealing slightly outperformed the gate-based QAOA in terms of feasibility and quality. However, this superiority of quantum annealing only appeared after fine-tuning chain strengths. Because the adjusted chain strengths improve feasibility and approximation quality, any residual overfitting would bias the results in favor of quantum annealing and therefore does not affect the main conclusion of this study. While being outperformed by classical heuristics, QAOA and quantum annealing also did not clearly differ from random sampling within the time limitation of 60 seconds. Here, we emphasize that we observed a clear difference in the distribution of objective values when comparing QAOA and quantum annealing to random sampling. However, in this work we considered only the best solution instead of statistical measures of the solution distribution. We conjecture the all-to-all connectivity of the problem to be the main reason for the poor performance of QAOA and quantum annealing. Dense problems generate a large qubit overhead during embedding and a large gate overhead during transpilation. Finally, we remark that the considered problem variant is one of the simpler formulations of portfolio optimization. For more complicated problem variants with additional constraints and variables, further experiments are required to study the performance of quantum methods.

Acknowledgements

We thank D-Wave for their efforts in reviewing our experiments and for their recommendations on parameter settings. This research was conducted within the Bench-QC project, a lighthouse project of the Munich Quantum Valley initiative, which is supported by the Bavarian state with funds from the Hightech Agenda Bayern Plus.

Appendix

A Generating Stock Data

For our benchmark, we create portfolio problem instances from real world stock data. We use the Python package yfinance [40] to retrieve historical NASDAQ closing prices $p_{i,t}$ of asset i at time t. From the price data, we calculate the *daily asset returns*

$$rd_{i,t} = \frac{p_{i,t}}{p_{i,t-1}} - 1 \quad \forall i \in N,\ t \in T$$

and the *average daily asset returns*

$$rav_i = \frac{1}{|T|} \sum_{t \in T} rd_{i,t} \quad \forall i \in N.$$

For the portfolio optimization problem, we work with *annualized asset returns*, which are defined by

$$r_i \coloneqq \left(\prod_{t \in T} (1 + rd_{i,t}) \right)^{\frac{252}{|T|}} - 1 \quad \forall i \in N.$$

The value of 252 is the average yearly amount of business days at the stock exchange. The estimated *annualized covariances* for all asset combinations of asset (i, j) are calculated by

$$\sigma_{ij} \coloneqq \frac{252}{|T|} \sum_{t \in T} (rd_{i,t} - rav_i)(rd_{j,t} - rav_j) \quad \forall i, j \in N. \tag{6}$$

The covariance matrix is used in its raw form without regularization because of the large number of observations in our test data set of 4 years. Now, the annualized asset returns r_i and the annualized asset covariances σ_{ij} are used to create instances of the portfolio optimization problem.

B Efficient Frontier

Our argument for replacing the $\geq$ with $=$ in model MinVola relies on the *efficient frontier* together with the selection of ϵ. According to the Markowitz Portfolio Theory [35], when displaying the minimum portfolio variance for given portfolio returns, a hyperbola arises [73]. The upper part of this hyperbola is called the efficient frontier. We visualize this concept by a schematic illustration in Fig. 6. The efficient frontier is a hyperbola where the horizontal axis represents risk (standard deviation) and the vertical axis represents expected return.

The portfolio which produces the leftmost point on this hyperbola is the global variance-minimal portfolio p_* with return and variance (μ_*, σ_*^2). Its return splits up the curve into two parts, $\mu < \mu_*$ and $\mu \geq \mu_*$. The upper part of this hyperbola consists of efficient portfolios whereas the lower part consists of inefficient portfolios, because for these, a portfolio with a higher return and the same variance also exists. Now, if we select $\epsilon \geq \mu_*$, the minimum variance portfolio for a larger-than-ϵ return has exactly return ϵ. In our benchmark test set we select ϵ as the 70 % quantile of its asset returns. Thus, we do not explicitly enforce $\epsilon \geq \mu_*$. Nonetheless, we assume this to be mostly the case, because with the 70 % quantile, we have a higher-than-average returning portfolio. We checked this assumption by solving the "$\mu \geq \epsilon$" quadratic program for all instances of our benchmark set. We observe that 93, 75 % of the variance-minimal portfolio displayed $\mu = \epsilon$. Furthermore, we observe that this assumption tends to be more often violated on small instances. When considering only instances that we used in the quantum benchmarks, i.e., from 5 to 30 assets, $\mu = \epsilon$ holds only with a percentage of 80 %. To further analyze the magnitude of the incurred inaccuracy, we calculate the difference in variance between optimizing with "$\geq$" and "$=$" for the instances where $\mu = \epsilon$ did not hold. The mean relative difference amounts to 0.025. To also report absolute values, the mean absolute difference in optimal variances for instances from 5 to 30 assets turns out to be 0.0015, the maximum difference is 0.004. As a result of these small deviations, we argue that it is indeed reasonable to replace $\geq$ by $=$, as in most cases, the optimum solution to the "$\geq$" quadratic program is attained at $\mu = \epsilon$. For instances where this is not the case, the incurred error in optimum variance is small. Replacing $\geq$ by $=$ also benefits the QUBO model because no additional slack variables have to be introduced, which in turn would have to be discretized and thus would heavily increase the qubit count and further reduce the accessible instance sizes. We acknowledge that this is a trade-off between model-size and modelling accuracy. The same argumentation holds for a one-sided penalty construction, which introduces additional non-quadratic structure or auxiliary variables.

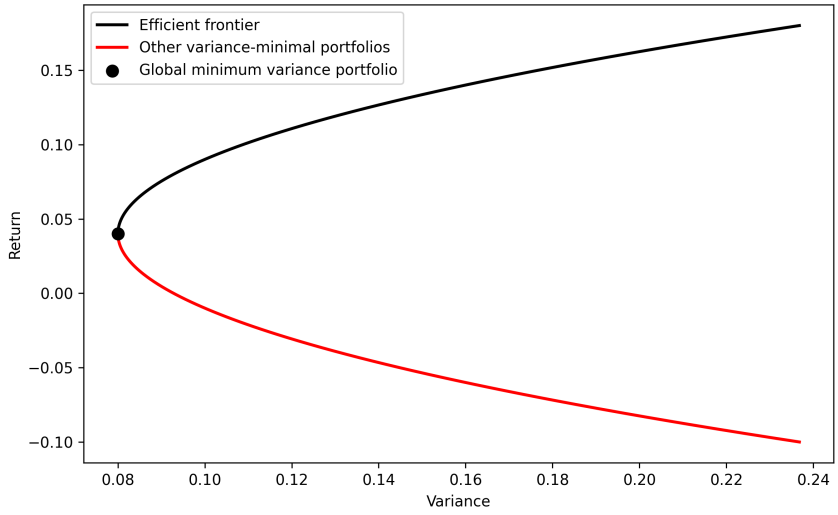


Figure 6: Visualization of the concept of the efficient frontier. The axes have arbitrary units.

C Pseudocode Problem-specific Heuristic

The Problem-specific heuristic for the MinVola problem formulation with $n \in \mathbb{N}$ possible assets generates up to n feasible solutions.

Algorithm 1 MinVola - Problem-specific Heuristic

Let $\mu_i \in \mathbb{R} \leftarrow$ expected return for asset $i \in \{1, \ldots, n\}$
Let $u_i \in (0, 1] \leftarrow$ upper bound of asset weight for asset $i \in \{1, \ldots, n\}$
Let $\sigma_{ij} \in \mathbb{R} \leftarrow$ covariance of returns of assets $i, j \in \{1, \ldots, n\}$
Let $\epsilon \in \mathbb{R} \leftarrow$ minimum required portfolio return
Let $\delta \in (0, 1] \leftarrow$ constant weight parameter that gets added to the samples
Initialize `Sols` $\leftarrow$ empty list of solutions
for $i \leftarrow 1$ **to** n **do**
    Initialize $x \leftarrow [0, \ldots, 0]$ empty asset weight vector of length n
    $x_i \leftarrow \delta$ add weight to asset i
    while $\sum_{k=1}^{n} x_k < 1 - \frac{\delta}{2}$ **do**
        $x \leftarrow$ AddNewAssetWeightSteepestDesc$(x, \mu, ub, \sigma, \delta, \epsilon)$
    end while
    $x \leftarrow$ NormalizeAssetWeights(x)
    Add x to `Sols`
end for
return `Sols`

function AddNewAssetWeightSteepestDesc$(x, \mu, ub, \sigma, \delta, \epsilon)$
    Let big_M $\leftarrow 1000$
    Let small_M $\leftarrow -1000$
    Initialize $Variances \leftarrow [\,]$ variances
    Initialize $Rets \leftarrow [\,]$ returns
    for $j \leftarrow 1$ **to** n **do**
        Let $x' \leftarrow$ copy of x
        $x'_j \leftarrow x'_j + \delta$
        if $x'_j \leq ub_j$ **then**
            $r \leftarrow \frac{\sum_{k=1}^{n} x'_k \mu_k}{\sum_{k=1}^{n} x'_k}$ calculate normalized return
            Append r to $Rets$
            if $r \geq \epsilon$ **then**
                v $\leftarrow \sum_{k=1}^{n} \sum_{l=1}^{n} x'_k \sigma_{kl} x'_l$ calculate return variance
                Append v to $Variances$
            else
                Append big_M to $Variances$
            end if
        else
            Append small_M to $Rets$
            Append big_M to $Variances$
        end if
    end for
    if $\min(Variances) =$ big_M **then**
        $j^* \leftarrow \arg\max Rets$
    else
        $j^* \leftarrow \arg\min Variances$
    end if
    $x_{j^*} \leftarrow x_{j^*} + \delta$
    return x
end function

function NormalizeAssetWeights(x)
    sum_of_weights $\leftarrow \sum_{i=1}^{n} x_i$
    for $i \leftarrow 1$ **to** n **do**
        $x_i \leftarrow x_i \cdot$ sum_of_weights
    end for
    return x
end function

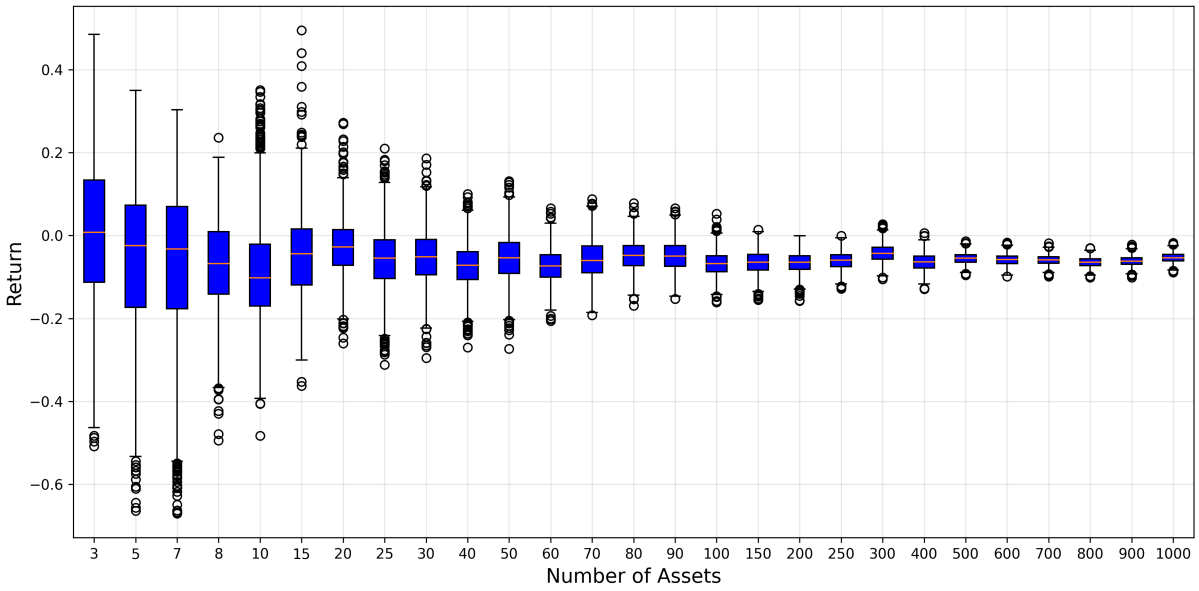


Figure 7: Distribution of returns of random portfolios for different instance sizes. For each instance size, we sampled 100 random portfolios for each of the 10 problem instances in our test data set.

D Selection of Penalty Factors

In the QUBO model (3), we set the penalty factors $\phi = \psi = 1000$ for violations of the return constraint (1) and the normalization constraint (2), respectively. Thus, an absolute constraint violation of 0.01 leads to a penalization of $1000 \cdot 0.01^2 = 0.1$. The goal is to make constraint violations unfavorable by increasing the objective value. Thus, we require that a constraint violation of 0.01 is small compared to the typical right-hand side of (1) and (2) and, at the same time, that a penalization of 0.1 is comparable to a typical objective value of MinVola. Clearly, 0.01 is small compared to the right-hand-side 1 of the normalization constraint (2). To analyze the domain of the right-hand-side of the return constraint (1), i.e. the 70%-quantile of the return $\mu(\omega)$, and the domain of the objective, i.e. the variance $\sigma^2(\omega)$, we calculate the returns and variances for 100 random portfolios for each of the 10 instances for each problem size (asset number). Thus, we analyze 28,000 random portfolios in total. The results are visualized in the boxplots in Fig. 7 and Fig. 8.

The boxplots visualize the statistical distribution of numerical data points by showing the median (red line), the 25th and 75th percentiles (blue box boundaries), the range of non-outlier values within 1.5 times the interquartile range (whiskers), and individual outlier points outside the whiskers. From this data, we estimate the typical magnitudes of portfolio returns and variances. The 75th percentiles (upper boundaries) of the portfolio returns are mostly negative and take values in the range from -0.1 to 0.1. We see that a return constraint violation of 0.01 is indeed small compared to a typical absolute value of ϵ, especially for instances with a small number of assets. These violations happen with decreasing frequency with increasing problem size. In particular, for $n \leq 30$ assets, which is the maximum number of assets that we are able to execute on a quantum computers, these frequently appearing high violations are penalized heavily, which is desired.

For the variances, the number of outliers is larger than for the returns. Some outliers have values above $10,000$ (e.g. for 40 assets). This is due to the high variance of the return variances of the individual assets which can cause a huge return variance in a random portfolio. Nonetheless, all medians have values below 0.35. Moreover, the optimal solutions to the instances show decreasing variances with increasing problem sizes, all below 0.5. This effect is visualized in Fig. 9. Thus, a penalty of 0.1 for a return-constraint violation of 0.01, is in a comparable range to the objective function as desired.

E Benchmark Study Results: Simulated QAOA

As mentioned in Section 5, a classically simulated QAOA, e.g. with AerSimulator [58], can only be executed for up to around 30 variables. As a result, the largest problem size that can be run with our configuration on the simulator is 7 assets, requiring 28 variables. For $3, 5$ and 7 assets we show the results

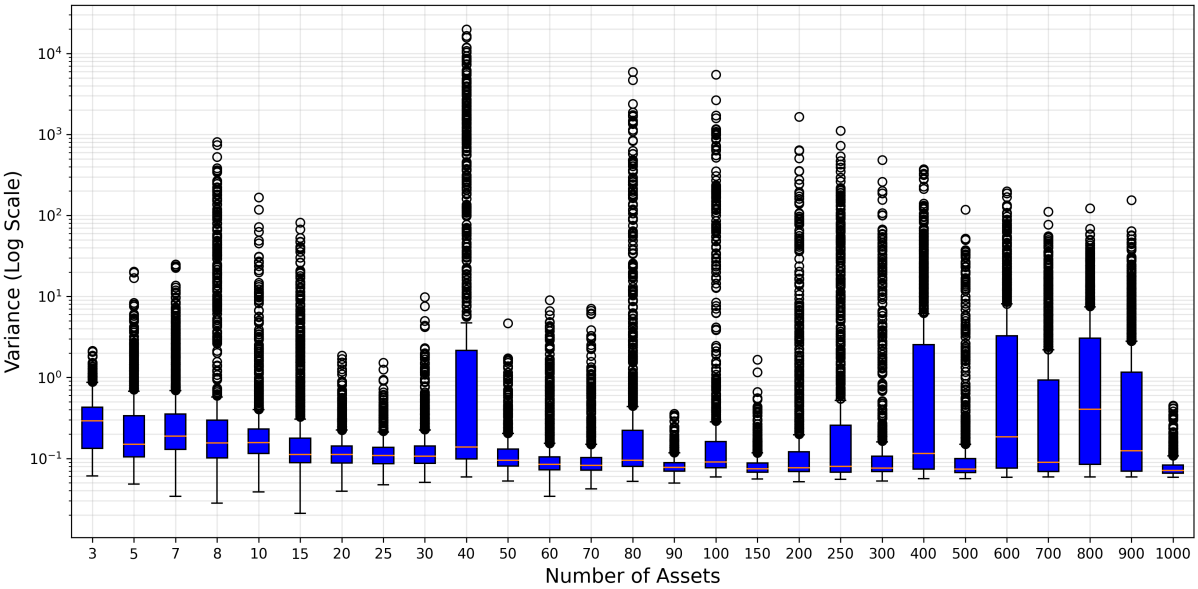


Figure 8: Distribution of variances of random portfolios for different instance sizes. For each instance size, we sampled 100 random portfolios for each of the 10 problem instances in our test data set.

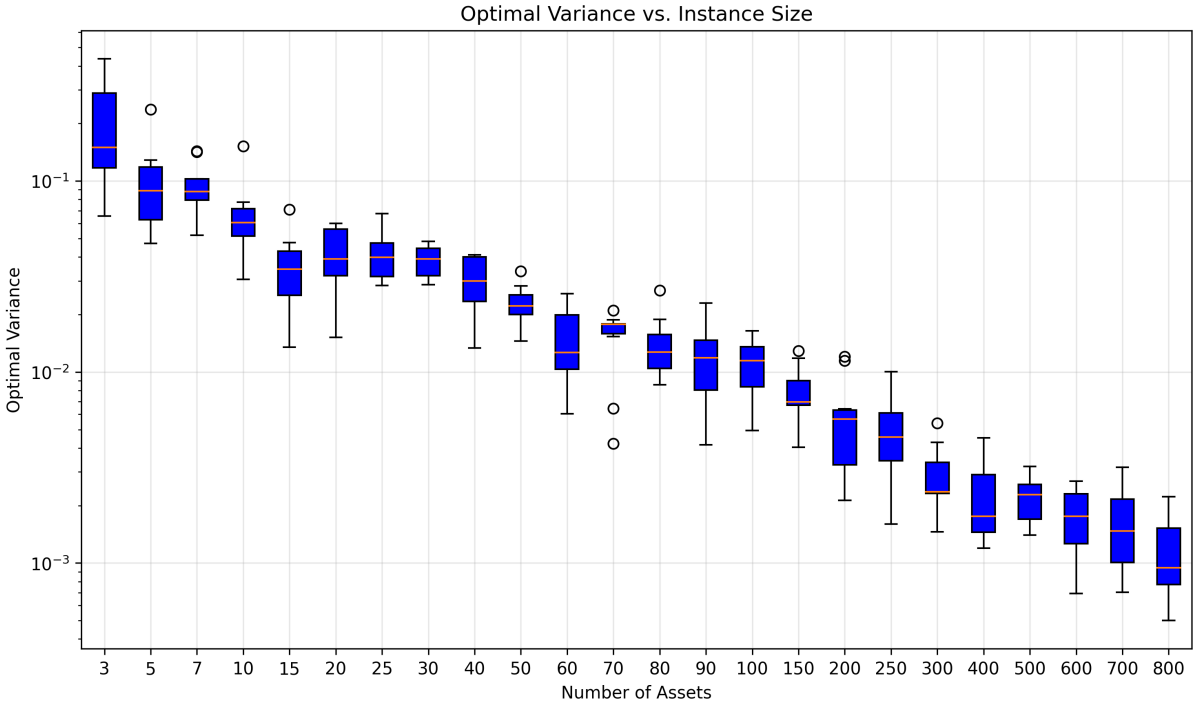


Figure 9: Distribution of optimal variances of the 10 discretized instances of each problem size in the benchmark data set.

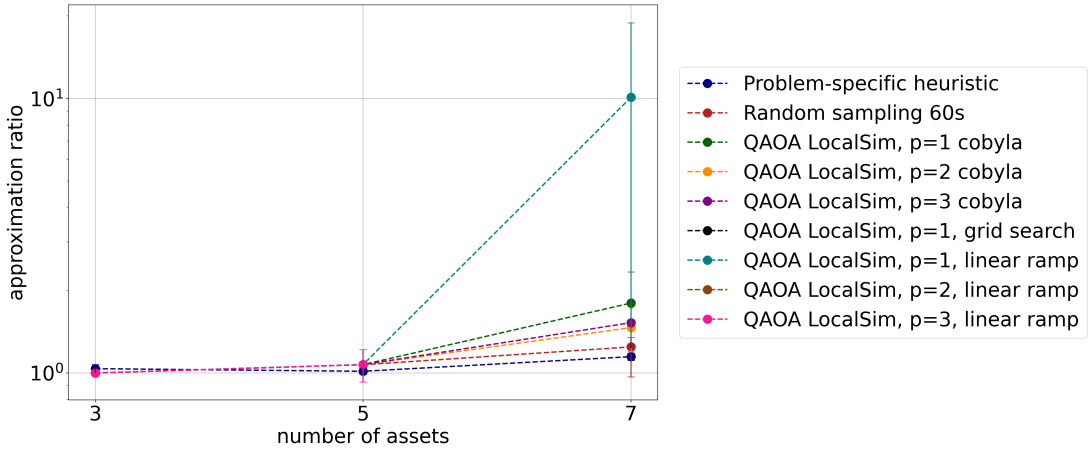
in Fig. 10. Therein, we compare different configurations of the locally simulated QAOA against random sampling and the problem-specific heuristic. We differentiate between the number of layers $p = 1, 2, 3$ and the way of optimizing the QAOA parameters. First, we use the COBYLA optimizer to optimize the parameters classically. Second, we perform a grid search and third, we initialize the parameters with the linear ramp formula. For further details on the grid search and the linear ramp, we refer to the QAOA section 5.

F Benchmark Study Results: Distribution of Approximation Ratios

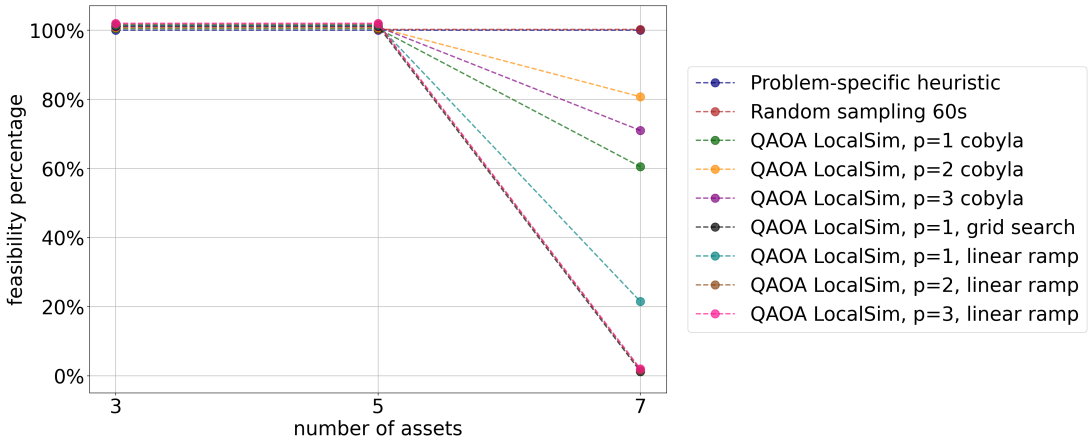
In the result section 6, we show the mean and empirical standard deviation of the achieved approximation ratios. In this appendix, we additionally visualize the distributions by boxplots in Fig. 11.

G Benchmark Study Results: Approximation Ratios With Respect to Continuous Problem

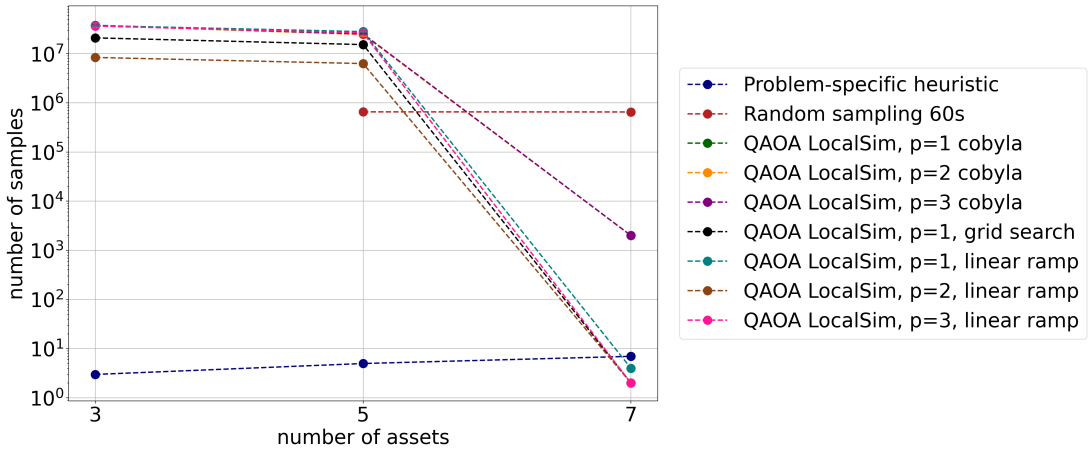
In section 6, we report the approximation ratios based on the objective value of the optimal solution to the discretized formulation of the MinVola problem. In Fig. 12, we additionally report the approximation ratios with respect to the optimal solution to the continuous MinVola problem. We observe similar results for the continuous case as for the discretized case. This similarity is due to the small difference between the optimal solution values of the discretized and the continuous problem, which further justifies the selection of the discretization factor of $d = 3$.



(a) Simulated QAOA approximation ratio

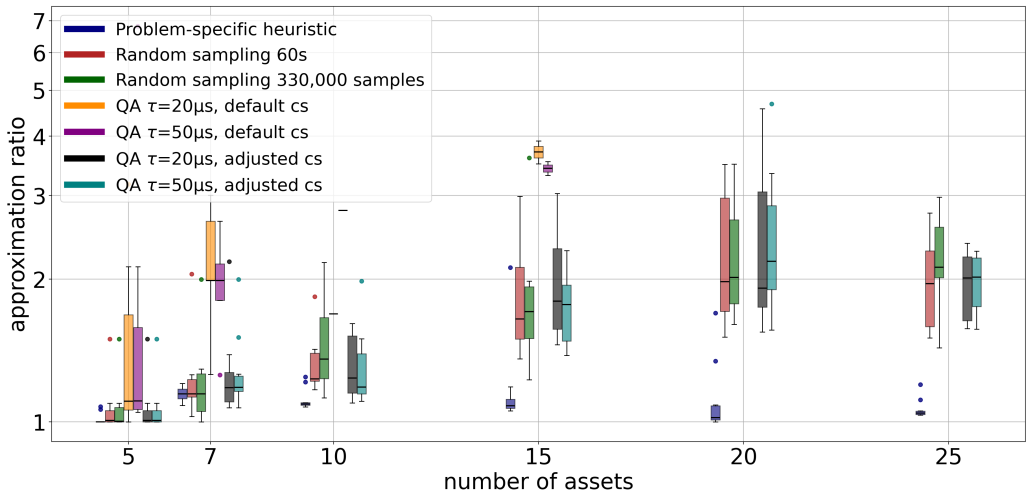


(b) Simulated QAOA feasibility percentage

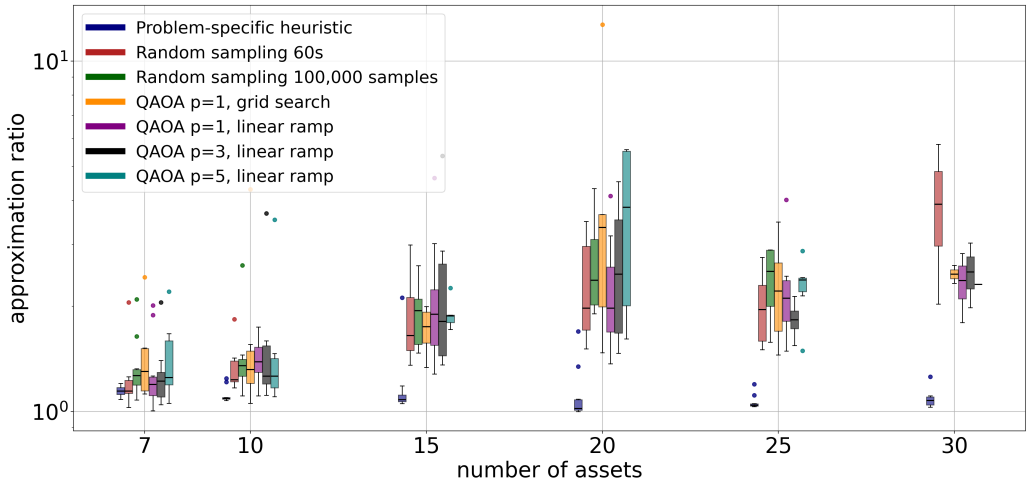


(c) Simulated QAOA number of samples

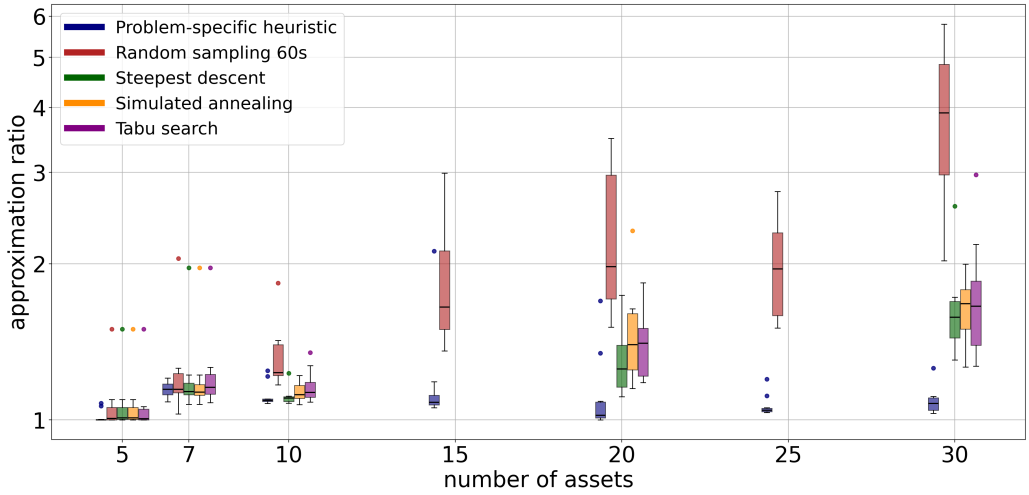
Figure 10: Approximation ratio, feasibility percentage and number of samples for the simulated QAOA. We compare against the problem-specific heuristic and random sampling as baselines. Data points are averages over 10 instances and error bars show the empirical standard deviation. Missing data points in the approximation ratio are due to the absence of feasible solutions.



(a) Boxplots of the quantum annealing approximation ratios of solving the 10 instances of each problem size in the benchmark data set.

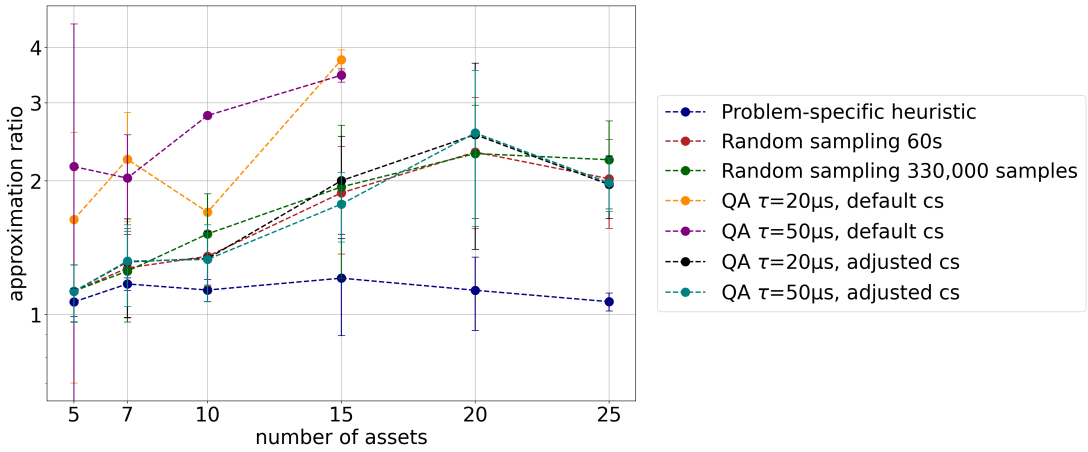


(b) Boxplots of the QAOA approximation ratios of solving the 10 instances of each problem size in the benchmark data set.

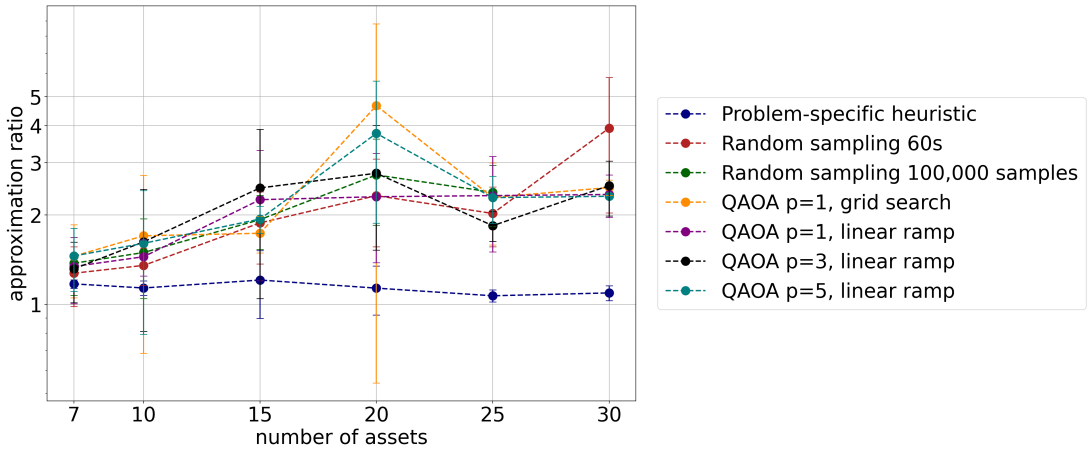


(c) Boxplots of the heuristics approximation ratios of solving the 10 instances of each problem size in the benchmark data set.

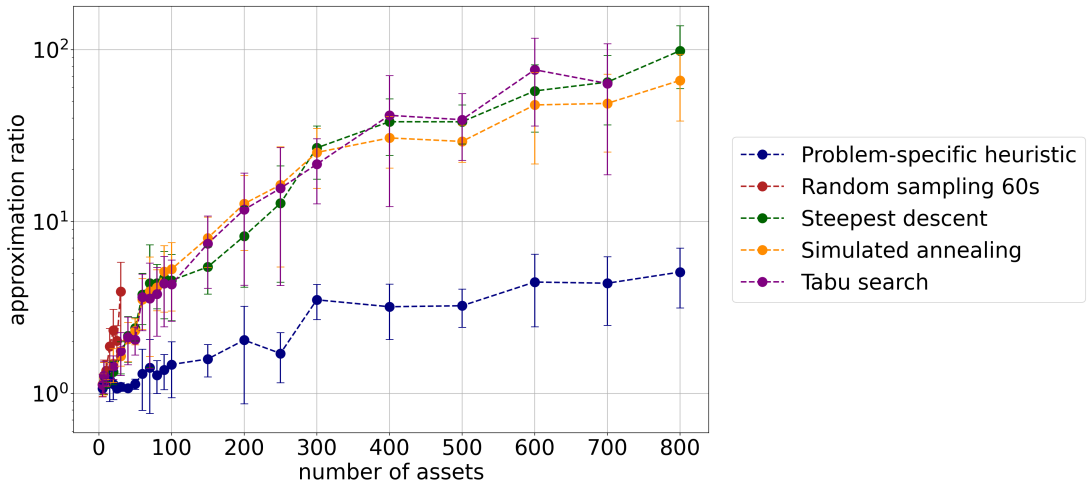
Figure 11: Boxplots of the approximation ratios of quantum annealing, QAOA, and the heuristics. They visualize the median (black lines inside the boxes), the 25th and 75th percentiles (box boundaries), the range of non-outlier values within 1.5 times the inter-quartile range (whiskers), and individual outlier points outside the whiskers.



(a) Quantum annealing approximation ratios w.r.t. the continuous MinVola problem.



(b) QAOA approximation ratios w.r.t. the continuous MinVola problem.



(c) Heuristics approximation ratios w.r.t. the continuous MinVola problem.

Figure 12: The approximation ratios of quantum annealing, QAOA, and the heuristics, where f_{opt} is selected as the optimal solution of the continuous version of the Minvola problem.

References

[1] Bernhard Korte and Jens Vygen. *Combinatorial Optimization: Theory and Algorithms*. Vol. 21. Algorithms and Combinatorics. Berlin, Heidelberg: Springer, 2018. URL: https://link.springer.com/10.1007/978-3-662-56039-6 (visited on 22/09/2025).

[2] Thorsten Koch, Timo Berthold, Jaap Pedersen and Charlie Vanaret. "Progress in mathematical programming solvers from 2001 to 2020". In: *EURO Journal on Computational Optimization* 10 (2022), p. 100031. DOI: https://doi.org/10.1016/j.ejco.2022.100031.

[3] Michael Jünger, Thomas M. Liebling, Denis Naddef, George L. Nemhauser, William R. Pulleyblank, Gerhard Reinelt, Giovanni Rinaldi and Laurence A. Wolsey, eds. *50 Years of Integer Programming 1958-2008: From the Early Years to the State-of-the-Art*. Berlin, Heidelberg: Springer, 2010. URL: http://link.springer.com/10.1007/978-3-540-68279-0 (visited on 22/09/2025).

[4] Ambros Gleixner, Gregor Hendel, Gerald Gamrath, Tobias Achterberg, Michael Bastubbe, Timo Berthold, Philipp M. Christophel, Kati Jarck, Thorsten Koch, Jeff Linderoth, Marco Lübbecke, Hans D. Mittelmann, Derya Ozyurt, Ted K. Ralphs, Domenico Salvagnin and Yuji Shinano. "MIPLIB 2017: Data-Driven Compilation of the 6th Mixed-Integer Programming Library". In: *Mathematical Programming Computation* 13.3 (Sept. 2021), pp. 443–490. DOI: 10.1007/s12532-020-00194-3.

[5] Amira Abbas, Andris Ambainis, Brandon Augustino, Andreas Bärtschi, Harry Buhrman, Carleton Coffrin, Giorgio Cortiana, Vedran Dunjko, Daniel J. Egger, Bruce G. Elmegreen, Nicola Franco, Filippo Fratini, Bryce Fuller, Julien Gacon, Constantin Gonciulea, Sander Gribling, Swati Gupta, Stuart Hadfield, Raoul Heese, Gerhard Kircher, Thomas Kleinert, Thorsten Koch, Georgios Korpas, Steve Lenk, Jakub Marecek, Vanio Markov, Guglielmo Mazzola, Stefano Mensa, Naeimeh Mohseni, Giacomo Nannicini, Corey O'Meara, Elena Peña Tapia, Sebastian Pokutta, Manuel Proissl, Patrick Rebentrost, Emre Sahin, Benjamin C. B. Symons, Sabine Tornow, Víctor Valls, Stefan Woerner, Mira L. Wolf-Bauwens, Jon Yard, Sheir Yarkoni, Dirk Zechiel, Sergiy Zhuk and Christa Zoufal. "Challenges and opportunities in quantum optimization". In: *Nature Reviews Physics* 6.12 (Dec. 2024). Publisher: Nature Publishing Group, pp. 718–735. URL: https://www.nature.com/articles/s42254-024-00770-9 (visited on 22/09/2025).

[6] Sabrina Ammann, Maximilian Hess, Debora Ramacciotti, Sándor P. Fekete, Paulina L. A. Goedicke, David Gross, Andreea Lefterovici, Tobias J. Osborne, Michael Perk, Antonio Rotundo, S. E. Skelton, Sebastian Stiller and Timo de Wolff. *Realistic Runtime Analysis for Quantum Simplex Computation*. arXiv:2311.09995 [quant-ph]. Nov. 2023. URL: http://arxiv.org/abs/2311.09995 (visited on 05/05/2025).

[7] Alexey Bochkarev, Raoul Heese, Sven Jäger, Philine Schiewe and Anita Schöbel. "Quantum computing for discrete optimization: A highlight of three technologies". In: *European Journal of Operational Research* 329.3 (2026), pp. 747–766. DOI: https://doi.org/10.1016/j.ejor.2025.07.063. URL: https://www.sciencedirect.com/science/article/pii/S0377221725005880.

[8] Tameem Albash and Daniel A. Lidar. "Demonstration of a Scaling Advantage for a Quantum Annealer over Simulated Annealing". In: *Phys. Rev. X* 8.3 (July 2018). Publisher: American Physical Society, p. 031016. DOI: 10.1103/PhysRevX.8.031016.

[9] Catherine C. McGeoch and Pau Farré. "Milestones on the Quantum Utility Highway: Quantum Annealing Case Study". In: *ACM Transactions on Quantum Computing* 5.1 (Dec. 2023). Place: New York, NY, USA Publisher: Association for Computing Machinery. DOI: 10.1145/3625307.

[10] Edward Farhi, Jeffrey Goldstone and Sam Gutmann. *A Quantum Approximate Optimization Algorithm*. arXiv:1411.4028 [quant-ph]. Nov. 2014. URL: http://arxiv.org/abs/1411.4028 (visited on 22/09/2025).

[11] Matthew P. Harrigan, Kevin J. Sung, Matthew Neeley, Kevin J. Satzinger, Frank Arute, Kunal Arya, Juan Atalaya, Joseph C. Bardin, Rami Barends, Sergio Boixo, Michael Broughton, Bob B. Buckley, David A. Buell, Brian Burkett, Nicholas Bushnell, Yu Chen, Zijun Chen, Ben Chiaro, Roberto Collins, William Courtney, Sean Demura, Andrew Dunsworth, Daniel Eppens, Austin Fowler, Brooks Foxen, Craig Gidney, Marissa Giustina, Rob Graff, Steve Habegger, Alan Ho, Sabrina Hong, Trent Huang, L. B. Ioffe, Sergei V. Isakov, Evan Jeffrey, Zhang Jiang, Cody Jones, Dvir Kafri, Kostyantyn Kechedzhi, Julian Kelly, Seon Kim, Paul V. Klimov, Alexander N. Korotkov, Fedor Kostritsa, David Landhuis, Pavel Laptev, Mike Lindmark, Martin Leib, Orion Martin, John

M. Martinis, Jarrod R. McClean, Matt McEwen, Anthony Megrant, Xiao Mi, Masoud Mohseni, Wojciech Mruczkiewicz, Josh Mutus, Ofer Naaman, Charles Neill, Florian Neukart, Murphy Yuezhen Niu, Thomas E. O'Brien, Bryan O'Gorman, Eric Ostby, Andre Petukhov, Harald Putterman, Chris Quintana, Pedram Roushan, Nicholas C. Rubin, Daniel Sank, Andrea Skolik, Vadim Smelyanskiy, Doug Strain, Michael Streif, Marco Szalay, Amit Vainsencher, Theodore White, Z. Jamie Yao, Ping Yeh, Adam Zalcman, Leo Zhou, Hartmut Neven, Dave Bacon, Erik Lucero, Edward Farhi and Ryan Babbush. "Quantum approximate optimization of non-planar graph problems on a planar superconducting processor". In: *Nature Physics* 17.3 (Feb. 2021). Publisher: Springer Science and Business Media LLC, pp. 332–336. DOI: 10.1038/s41567-020-01105-y.

[12] Thorsten Koch, David E. Bernal Neira, Ying Chen, Giorgio Cortiana, Daniel J. Egger, Raoul Heese, Narendra N. Hegade, Alejandro Gomez Cadavid, Rhea Huang, Toshinari Itoko, Thomas Kleinert, Pedro Maciel Xavier, Naeimeh Mohseni, Jhon A. Montanez-Barrera, Koji Nakano, Giacomo Nannicini, Corey O'Meara, Justin Pauckert, Manuel Proissl, Anurag Ramesh, Maximilian Schicker, Noriaki Shimada, Mitsuharu Takeori, Victor Valls, David Van Bulck, Stefan Woerner and Christa Zoufal. "Quantum Optimization Benchmarking Library - The Intractable Decathlon". In: *Nature Computational Science* 6.6 (June 2026). arXiv:2504.03832 [quant-ph], pp. 653–671. DOI: 10.1038/s43588-026-00991-1. URL: http://arxiv.org/abs/2504.03832 (visited on 01/10/2026).

[13] Wolfgang Sakuler, Johannes M. Oberreuter, Riccardo Aiolfi, Luca Asproni, Branislav Roman and Jürgen Schiefer. *A real world test of Portfolio Optimization with Quantum Annealing*. arXiv:2303.12601 [quant-ph]. Mar. 2023. URL: http://arxiv.org/abs/2303.12601 (visited on 11/06/2024).

[14] Chen-Yu Liu and Hsi-Sheng Goan. *Hybrid Gate-Based and Annealing Quantum Computing for Large-Size Ising Problems*. arXiv:2208.03283 [quant-ph]. Aug. 2022. URL: http://arxiv.org/abs/2208.03283 (visited on 22/09/2025).

[15] Atithi Acharya, Romina Yalovetzky, Pierre Minssen, Shouvanik Chakrabarti, Ruslan Shaydulin, Rudy Raymond, Yue Sun, Dylan Herman, Ruben S. Andrist, Grant Salton, Martin J. A. Schuetz, Helmut G. Katzgraber and Marco Pistoia. "Decomposition Pipeline for Large-Scale Portfolio Optimization with Applications to Near-Term Quantum Computing". In: *Physical Review Research* 7.2 (May 2025). arXiv:2409.10301 [math.OC], p. 023142. DOI: 10.1103/PhysRevResearch.7.023142. URL: http://arxiv.org/abs/2409.10301 (visited on 01/10/2026).

[16] Frank Phillipson and Harshil Singh Bhatia. "Portfolio Optimisation Using the D-Wave Quantum Annealer". In: *Computational Science – ICCS 2021*. Ed. by Maciej Paszynski, Dieter Kranzlmüller, Valeria V. Krzhizhanovskaya, Jack J. Dongarra and Peter M. A. Sloot. Cham: Springer International Publishing, 2021, pp. 45–59. DOI: 10.1007/978-3-030-77980-1_4.

[17] Samuel Palmer, Serkan Sahin, Rodrigo Hernandez, Samuel Mugel and Roman Orus. *Quantum Portfolio Optimization with Investment Bands and Target Volatility*. arXiv:2106.06735 [q-fin]. Aug. 2021. URL: http://arxiv.org/abs/2106.06735 (visited on 30/12/2024).

[18] Davide Venturelli and Alexei Kondratyev. "Reverse Quantum Annealing Approach to Portfolio Optimization Problems". In: *Quantum Machine Intelligence* 1.1-2 (May 2019). arXiv:1810.08584 [quant-ph], pp. 17–30. DOI: 10.1007/s42484-019-00001-w. URL: http://arxiv.org/abs/1810.08584 (visited on 01/10/2026).

[19] Sebastian Brandhofer, Daniel Braun, Vanessa Dehn, Gerhard Hellstern, Matthias Hüls, Yanjun Ji, Ilia Polian, Amandeep Singh Bhatia and Thomas Wellens. "Benchmarking the performance of portfolio optimization with QAOA". In: *Quantum Information Processing* 22.1 (Dec. 2022). arXiv:2207.10555 [quant-ph], p. 25. DOI: 10.1007/s11128-022-03766-5. URL: http://arxiv.org/abs/2207.10555 (visited on 30/12/2024).

[20] Mark Hodson, Brendan Ruck, Hugh Ong, David Garvin and Stefan Dulman. *Portfolio rebalancing experiments using the Quantum Alternating Operator Ansatz*. arXiv:1911.05296 [quant-ph]. Nov. 2019. URL: http://arxiv.org/abs/1911.05296 (visited on 21/08/2025).

[21] Zhijie Tang, Alex Lu Dou and Arit Kumar Bishwas. *Comparative analysis of diverse methodologies for portfolio optimization leveraging quantum annealing techniques*. arXiv:2403.02599 [quant-ph]. July 2024. URL: http://arxiv.org/abs/2403.02599 (visited on 22/09/2025).

[22] Giuseppe Buonaiuto, Francesco Gargiulo, Giuseppe De Pietro, Massimo Esposito and Marco Pota. "Best practices for portfolio optimization by quantum computing, experimented on real quantum devices". In: *Scientific Reports* 13.1 (Nov. 2023), p. 19434. URL: https://www.nature.com/articles/s41598-023-45392-w (visited on 04/08/2025).

[23] Francesco Cesarone, Andrea Scozzari and Fabio Tardella. "Efficient Algorithms for mean-variance portfolio optimization with Hard Real -World Constraints". In: *Giornale dell'Istituto Italiano degli Attuari* (2009), pp. 37–56.

[24] Álvaro Rubio-García, Juan José García-Ripoll and Diego Porras. *Portfolio optimization with discrete simulated annealing*. arXiv:2210.00807 [cond-mat]. Oct. 2022. URL: http://arxiv.org/abs/2210.00807 (visited on 30/12/2024).

[25] T.-J. Chang, N. Meade, J.E. Beasley and Y.M. Sharaiha. "Heuristics for cardinality constrained portfolio optimisation". In: *Computers & Operations Research* 27.13 (Nov. 2000), pp. 1271–1302. URL: https://linkinghub.elsevier.com/retrieve/pii/S030505489900074X (visited on 05/06/2025).

[26] Cassidy K. Buhler and Hande Y. Benson. *Efficient Solution of Portfolio Optimization Problems via Dimension Reduction and Sparsification*. arXiv:2306.12639 [q-fin]. June 2023. URL: http://arxiv.org/abs/2306.12639 (visited on 07/01/2025).

[27] Edoardo Alessandroni, Sergi Ramos-Calderer, Ingo Roth, Emiliano Traversi and Leandro Aolita. "Alleviating the quantum Big-M problem". In: *npj Quantum Information* 11.1 (July 2025), p. 125. URL: https://www.nature.com/articles/s41534-025-01067-0 (visited on 30/07/2025).

[28] Mahdi Moeini. *A Continuous Optimization Approach for the Financial Portfolio Selection under Discrete Asset Choice Constraints*. arXiv:1404.3286 [cs]. Apr. 2014. URL: http://arxiv.org/abs/1404.3286 (visited on 13/06/2025).

[29] Francesco Cesarone, Andrea Scozzari and Fabio Tardella. "Portfolio selection problems in practice: a comparison between linear and quadratic optimization models". In: *Computational Management Science* 12.3 (July 2015). arXiv:1105.3594 [q-fin], pp. 345–370. DOI: 10.1007/s10287-014-0210-1. URL: http://arxiv.org/abs/1105.3594 (visited on 13/06/2025).

[30] Sarat Moka, Matias Quiroz, Vali Asimit and Samuel Muller. *A Scalable Gradient-Based Optimization Framework for Sparse Minimum-Variance Portfolio Selection*. arXiv:2505.10099 [stat]. May 2025. URL: http://arxiv.org/abs/2505.10099 (visited on 13/06/2025).

[31] *IBM ILOG CPLEX Optimization Studio*. Dec. 2022. URL: https://www.ibm.com/docs/en/icos/22.1.1?topic=cplex-meet (visited on 21/08/2025).

[32] Sebastian Schlütter, Tomislav Maras, Alexander Dotterweich and Nico Piatkowski. *Hot-Starting Quantum Portfolio Optimization*. 2025. arXiv: 2510.11153 [quant-ph]. URL: https://arxiv.org/abs/2510.11153.

[33] Vicente P. Soloviev, Antonio Márquez Romero, Josh Kirsopp and Michal Krompiec. *Scaling Portfolio Diversification with Quantum Circuit Cutting Techniques*. arXiv:2506.08947 [quant-ph]. June 2025. URL: http://arxiv.org/abs/2506.08947 (visited on 13/06/2025).

[34] Zi Xuan Loke, Say Leng Goh, Graham Kendall, Salwani Abdullah and Nasser R. Sabar. "Portfolio Optimization Problem: A Taxonomic Review of Solution Methodologies". In: *IEEE Access* 11 (2023), pp. 33100–33120. URL: https://ieeexplore.ieee.org/document/10087257/ (visited on 07/01/2025).

[35] Harry Markowitz. "Portfolio Selection". In: *The Journal of Finance* 7.1 (1952), pp. 77–91. URL: https://onlinelibrary.wiley.com/doi/abs/10.1111/j.1540-6261.1952.tb01525.x (visited on 05/03/2025).

[36] Stephen P. Boyd and Lieven Vandenberghe. *Convex optimization*. Cambridge, UK ; New York: Cambridge University Press, 2004.

[37] Margaret Wright. "The interior-point revolution in optimization: History, recent developments, and lasting consequences". In: *Bulletin of the American Mathematical Society* 42.1 (Sept. 2004), pp. 39–56. URL: https://www.ams.org/bull/2005-42-01/S0273-0979-04-01040-7/ (visited on 22/09/2025).

[38] *Stock Screener NASDAQ*. URL: https://www.nasdaq.com/market-activity/stocks/screener (visited on 26/05/2026).

[39] *Yahoo Finance - Stock Market Live, Quotes, Business & Finance News*. URL: https://finance.yahoo.com/ (visited on 13/06/2025).

[40] *yfinance: Python Package to download market data from Yahoo! Finance API*. Sept. 2025. URL: https://pypi.org/project/yfinance/0.2.66/ (visited on 01/10/2026).

[41] Suresh Bolusani, Mathieu Besancon, Ksenia Bestuzheva, Antonia Chmiela, Joao Dionısio, Tim Donkiewicz, Jasper van Doornmalen, Leon Eifler, Mohammed Ghannam, Ambros Gleixner, Christoph Graczyk, Katrin Halbig, Ivo Hedtke, Alexander Hoen and Christopher Hojny. *The SCIP Optimization Suite 9.0*. arXiv:2402.17702 [math.OC]. 2024. URL: https://arxiv.org/abs/2402.17702v2 (visited on 26/05/2026).

[42] *PySCIPOpt (Python Package for SCIP)*. Aug. 2025. URL: https://pypi.org/project/PySCIPOpt/5.6.0/ (visited on 01/10/2026).

[43] *The Leader in Decision Intelligence Technology - Gurobi Optimization*. URL: https://www.gurobi.com/ (visited on 04/03/2025).

[44] Philipp Hauke, Helmut G. Katzgraber, Wolfgang Lechner, Hidetoshi Nishimori and William D. Oliver. "Perspectives of quantum annealing: Methods and implementations". In: *Reports on Progress in Physics* 83.5 (May 2020). arXiv:1903.06559 [quant-ph], p. 054401. DOI: 10.1088/1361-6633/ab85b8. URL: http://arxiv.org/abs/1903.06559 (visited on 30/05/2025).

[45] Tameem Albash and Daniel Lidar. "Adiabatic quantum computation". In: *Reviews of Modern Physics* 90.1 (2018). DOI: 10.1103/RevModPhys.90.015002.

[46] Albert Messiah. *Quantum Mechanics*. Google-Books-ID: VR93vUk8d_8C. Elsevier, 1961.

[47] *The Advantage™ Quantum Computer | D-Wave*. URL: https://www.dwavequantum.com/solutions-and-products/systems/ (visited on 22/09/2025).

[48] Vicky Choi. "Minor-Embedding in Adiabatic Quantum Computation: I. The Parameter Setting Problem". In: *Quantum Information Processing, 7* (Apr. 2008). arXiv:0804.4884 [quant-ph], pp193–209. DOI: 10.48550/arXiv.0804.4884. URL: http://arxiv.org/abs/0804.4884 (visited on 01/10/2026).

[49] Vicky Choi. "Minor-embedding in adiabatic quantum computation: II. Minor-universal graph design". In: *Quantum Information Processing* 10.3 (June 2011). arXiv:1001.3116 [quant-ph], pp. 343–353. URL: http://arxiv.org/abs/1001.3116 (visited on 22/09/2025).

[50] *Samplers — dwave-system 0.9.12 documentation*. URL: https://dwave-systemdocs.readthedocs.io/en/latest/reference/samplers.html#dwavecliquesampler (visited on 24/06/2025).

[51] *dwave-system (Python Package for Dwave)*. Sept. 2025. URL: https://pypi.org/project/dwave-system/1.33.0/ (visited on 01/10/2026).

[52] Jun Cai, William G. Macready and Aidan Roy. *A practical heuristic for finding graph minors*. arXiv:1406.2741 [quant-ph]. June 2014. URL: http://arxiv.org/abs/1406.2741 (visited on 22/09/2025).

[53] Mostafa Khezri, Xi Dai, Rui Yang, Tameem Albash, Adrian Lupascu and Daniel A. Lidar. "Customized Quantum Annealing Schedules". In: *Physical Review Applied* 17.4 (Apr. 2022). DOI: 10.1103/physrevapplied.17.044005. URL: http://dx.doi.org/10.1103/PhysRevApplied.17.044005.

[54] Asier Ozaeta, Wim Van Dam and Peter L McMahon. "Expectation values from the single-layer quantum approximate optimization algorithm on Ising problems". In: *Quantum Science and Technology* 7.4 (Oct. 2022), p. 045036. URL: https://iopscience.iop.org/article/10.1088/2058-9565/ac9013 (visited on 11/06/2024).

[55] J. A. Montanez-Barrera and Kristel Michielsen. "Towards a Linear-Ramp QAOA protocol: Evidence of a scaling advantage in solving some combinatorial optimization problems". In: *npj Quantum Information* 11.1 (Aug. 2025). arXiv:2405.09169 [quant-ph], p. 131. DOI: 10.1038/s41534-025-01082-1. URL: http://arxiv.org/abs/2405.09169 (visited on 01/10/2026).

[56] *minimize(method='COBYLA') — SciPy v1.15.3 Manual*. URL: https://docs.scipy.org/doc/scipy/reference/optimize.minimize-cobyla.html (visited on 30/05/2025).

[57] *scipy (open-source software for mathematics, science, and engineering)#*. Sept. 2025. URL: https://pypi.org/project/scipy/1.16.2/ (visited on 01/10/2026).

[58] *AerSimulator - Qiskit Aer 0.16.1*. URL: https://qiskit.github.io/qiskit-aer/stubs/qiskit_aer.AerSimulator.html (visited on 30/05/2025).

[59] *qiskit-aer (Python Package for quantum circuit simulation)*. Sept. 2025. URL: https://pypi.org/project/qiskit-aer/0.17.2/ (visited on 01/10/2026).

[60] M. AbuGhanem. "IBM Quantum Computers: Evolution, Performance, and Future Directions". In: *The Journal of Supercomputing* 81.5 (Apr. 2025). arXiv:2410.00916 [quant-ph], p. 687. DOI: 10.1007/s11227-025-07047-7. URL: http://arxiv.org/abs/2410.00916 (visited on 01/10/2026).

[61] *generate_preset_pass_manager (v2.0)*. URL: https://quantum.cloud.ibm.com/docs/en/api/qiskit/2.0/quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.transpiler.generate_preset_pass_manager (visited on 21/09/2026).

[62] Daniel J. Egger, Jakub Mareček and Stefan Woerner. "Warm-starting quantum optimization". In: *Quantum* 5 (2021), p. 479. DOI: 10.22331/q-2021-06-17-479. URL: http://dx.doi.org/10.22331/q-2021-06-17-479.

[63] J. V. S. Scursulim, Gabriel M. Langeloh, Victor L. Beltran and Samuraí Brito. "Multiclass portfolio optimization via variational quantum Eigensolver with Dicke state ansatz". In: *Scientific Reports* 16.1 (Feb. 2026). DOI: 10.1038/s41598-026-36333-4. URL: http://dx.doi.org/10.1038/s41598-026-36333-4.

[64] *dwavesystems/dwave-greedy*. original-date: 2019-08-02T10:10:41Z. Sept. 2024. URL: https://github.com/dwavesystems/dwave-greedy (visited on 30/05/2025).

[65] *dwave-greedy (Python Package for Dwave Greedy)*. Nov. 2022. URL: https://pypi.org/project/dwave-greedy/0.3.0/ (visited on 01/10/2026).

[66] S. Kirkpatrick, C. D. Gelatt and M. P. Vecchi. "Optimization by Simulated Annealing". In: *Science*. New Series 220.4598 (1983), pp. 671–680. DOI: 10.1126/science.220.4598.671. URL: http://www.jstor.org/stable/1690046.

[67] *dwave-samplers — Python documentation*. URL: https://docs.dwavequantum.com/en/latest/ocean/api_ref_samplers/index.html (visited on 30/05/2025).

[68] *dwave-samplers (Python Package for Dwave samplers)*. Oct. 2025. URL: https://pypi.org/project/dwave-samplers/1.7.0/ (visited on 01/10/2026).

[69] *dwavesystems/dwave-tabu*. original-date: 2018-08-28T02:14:30Z. Sept. 2024. URL: https://github.com/dwavesystems/dwave-tabu (visited on 30/05/2025).

[70] *dwave-tabu (Python Package for Dwave Tabu)*. Nov. 2022. URL: https://pypi.org/project/dwave-tabu/0.5.0/ (visited on 01/10/2026).

[71] Arturo Acuaviva, David Aguirre, Rubén Peña and Mikel Sanz. "Benchmarking quantum computers: towards a standard performance evaluation approach". In: *Quantum Science and Technology* 11.2 (Feb. 2026), p. 025004. DOI: 10.1088/2058-9565/ae4269. URL: https://doi.org/10.1088/2058-9565/ae4269 (visited on 01/10/2026).

[72] *stopfereric/portfolio_opt_benchmark*. URL: https://github.com/stopfereric/portfolio_opt_benchmark (visited on 22/09/2025).

[73] Robert C. Merton. "An Analytic Derivation of the Efficient Portfolio Frontier". In: *The Journal of Financial and Quantitative Analysis* 7.4 (1972), pp. 1851–1872. URL: http://www.jstor.org/stable/2329621 (visited on 03/07/2026).

arXiv:2207.10555v1 [quant-ph] 21 Jul 2022

Benchmarking the performance of portfolio optimization with QAOA

Sebastian Brandhofer[1], Daniel Braun[2], Vanessa Dehn[4], Gerhard Hellstern[3], Matthias Hüls[2], Yanjun Ji[1], Ilia Polian[1], Amandeep Singh Bhatia[2] and Thomas Wellens[4*]

[1]Institut für Technische Informatik, Universität Stuttgart, Pfaffenwaldring 47, D-70569 Stuttgart, Germany.
[2]Institut für Theoretische Physik, Eberhard Karls Universität Tübingen, Auf der Morgenstelle 14, D-72076 Tübingen, Germany.
[3]Zentrum für Digitale Innovationen (ZDI), DHBW Ravensburg, Marktstr. 28, D-88214 Ravensburg, Germany.
[4]Fraunhofer-Institut für Angewandte Festkörperphysik IAF, Tullastr. 72, D-79108 Freiburg, Germany.

*Corresponding author(s). E-mail(s): thomas.wellens@iaf.fraunhofer.de;

Abstract

We present a detailed study of portfolio optimization using different versions of the quantum approximate optimization algorithm (QAOA). For a given list of assets, the portfolio optimization problem is formulated as quadratic binary optimization constrained on the number of assets contained in the portfolio. QAOA has been suggested as a possible candidate for solving this problem (and similar combinatorial optimization problems) more efficiently than classical computers in the case of a sufficiently large number of assets. However, the practical implementation of this algorithm requires a careful consideration of several technical issues, not all of which are discussed in the present literature. The present article intends to fill this gap and thereby provide the reader with a useful guide for applying QAOA to the portfolio optimization problem (and similar problems). In particular, we will discuss several possible choices of the variational form and of different classical algorithms for finding the corresponding optimized parameters. Viewing at the application of QAOA on error-prone NISQ hardware,

we also analyze the influence of statistical sampling errors (due to a finite number of shots) and gate and readout errors (due to imperfect quantum hardware). Finally, we define a criterion for distinguishing between 'easy' and 'hard' instances of the portfolio optimization problem.

Keywords: Quantum computing, Quantum algorithm, QAOA, Quantum optimization

1 Introduction

The solution of combinatorial optimization problems with QAOA, i.e., the quantum approximate optimization algorithm [1] or its generalized version known as quantum alternating operator ansatz [2], is often mentioned as one of the most promising candidates for realizing an advantage of quantum computers with respect to classical computers in a relatively near future. This expectation is based upon the fact the quantum circuits required for QAOA are shorter than those of other algorithms, e.g the Shor [3] or HHL [4] algorithm, for which an exponential advantage compared to classical algorithms can be proven (under the assumption that quantum computers with sufficiently many error-corrected qubits are available). In contrast to those cases, a quantum advantage of QAOA has not yet been rigorously proven. Since QAOA involves a numerical classical optimization of variational circuit parameters (which, in itself, is an NP-hard problem [5]), analytical results concerning the performance of QAOA for a large number of qubits are not available. Nevertheless, extrapolations of results obtained for small numbers of qubits towards larger numbers suggest that a quantum speed-up (for the Max-Cut problem) could be expected with several hundred qubits [6]. As pointed out there, the main limitation of QAOA is imposed by the exponential cost required for optimizing the variational parameters.

This conclusion, however, might change if the exponential cost can be reduced by finding more efficient heuristic strategies for performing the classical optimization of these parameters. Indeed, there are indications that this is possible: for example, instead of starting the optimization from randomly chosen initial values, as it is done in [6], a strategy making use of observed patterns in the optimal parameters requires exponentially less time in order to achieve a similar performance [7]. In general, the performance of QAOA critically depends on how exactly the parameter optimization is carried out, in particular how the initial values fed into the classical optimization routine are chosen. Whereas [7] contains an in-depth study of the performance of QAOA on rather abstract MaxCut problems, the present paper provides a similar analysis for the use case of portfolio optimization, which constitutes one of the most discussed problems in finance [8–11].

In addition to previous works on portfolio optimization by QAOA [12–14], the present paper therefore focuses on a number of technical issues relevant

for the practical implementation of QAOA, not all of which are discussed in the present literature. By presenting the details of our implementation, which we have found to yield consistently good results, we provide a useful guide for all readers who want to apply QAOA to portfolio optimization or similar optimization problems. Furthermore, since the QAOA performance varies for different instances (i.e., different assets from which the portfolio is chosen), we present a new criterion for distinguishing between 'easy' and 'hard' instances of the portfolio optimization problem. Finally, viewing at the application of QAOA on error-prone hardware, we also analyze how the QAOA performance is modified by the presence of errors.

Correspondingly, the paper is organized as follows: In Sec. 2, we define the portfolio optimization problem as a quadratic binary optimization problem including a constraint given by the total number of assets to be included in the portfolio. In order to measure the performance of a given portfolio, we modify the standard definition of the so-called 'approximation ratio' r such that it is independent of the particular way how the constraint is implemented (see Sec. 2.2). Since one such way consists of adding a penalty term, we explain our method for appropriately choosing the corresponding penalty factor A (see Sec. 2.3).

The following Sec. 3 exposes the required concepts for solving the portfolio optimization problem using QAOA. In general, this method consists of a p-fold alternating application of two different operators (phase separating and mixing operators) depending on respective parameters $\vec{\gamma} = (\gamma_1, \ldots, \gamma_p)$ and $\vec{\beta} = (\beta_1, \ldots, \beta_p)$. Apart from the standard implementation [1] (using a 'soft constraint' based on the above-mentioned penalty term), we outline several possible modifications for incorporating the constraint in an alternative way ('hard constraint') by using so-called XY mixing operators [2] (see Sec. 3.1). Furthermore, we present in detail our strategy for performing the optimization of the QAOA parameters (see Sec. 3.2). This includes the suitable choice of a global scaling factor λ (the relevance of which has, to our knowledge, not been recognized in the present literature) as well as four different heuristic methods for choosing appropriate initial values of the parameters $\vec{\gamma}$ and $\vec{\beta}$ for increasing depth p.

The results obtained by applying our algorithm to different instances of portfolio problems based on real market data (taken from the German index DAX) are presented in Sec. 4. Starting with ideal, error-free simulations (using Qiskit's statevector simulator), we show that two of the five variants of XY mixers outperform the other ones by achieving the fastest convergence towards the optimal solution for increasing depth p (see Sec. 4.1). Similar results are obtained when including statistical sampling noise due to a finite number of shots (using the qasm simulator). In the following, we compare the algorithm's performance on different problem instances (see Sec. 4.2). We define characteristic measures for an instance and show that the performance correlates with these. An explanation for this performance dependencies is given by analyzing the energy landscape of the respective optimization problem.

To account for the noise-affected nature of today's quantum computers, we systematically investigate the impact of errors on QAOA, thus adding a further dimension to its analysis (see Sec. 4.3). Considering, both, the error-affected optimization landscapes and the final outcome of the optimization, we find, for the first time, that noise of varying strengths affects different mixers differently. We conclude that the expected noise levels on the quantum platform to be used should be taken into account when choosing a specific version of QAOA.

2 Definition of the portfolio optimization problem

Portfolio optimization is one of the classic problems of finance: How should a financial investor put together a portfolio of assets (e.g. stocks) so that it generates the highest possible return on the one hand, but contains the lowest possible risk on the other? The solution to this problem was formulated in 1952 by H. Markowitz [15] as a mathematical optimization problem. In its original form, continuous portfolio weights are allowed, i.e., it is assumed that, e.g., a stock is also tradable 'in parts'. For a more realistic formulation, however, it is necessary to assume binary or at least integer portfolio weights. In the binary formulation, the portfolio weight of a stock is 1 or 0, depending on whether the stock is part of the portfolio or not. The portfolio optimization problem can then be formulated as follows: Consider the cost function

$$F(z_1, z_2, \ldots, z_n) = q \sum_{i,j=1}^{n} z_i z_j \sigma_{ij} - (1-q) \sum_{i=1}^{n} z_i \mu_i, \quad z_i = 0, 1 \tag{1}$$

which consists of the portfolio weights $z_i = 0$ or 1 of stock i, the covariance matrix of the stock returns σ_{ij} and the expected return μ_i. Furthermore, n is the number of available assets and the factor q sets the risk preference of the investor: $q = 1$ would be used if the investor is fully risk-averse, i.e. he wants to choose the portfolio with the lowest risk, irrespective of the return. $q = 0$ would be used in case of an aggressive investor who takes only the return of the stocks into account. In real-world applications, however, q is set between 0 and 1.

Usually, an investment strategy consists in investing a fixed budget of money into a portfolio of securities. This budget constraint can be formulated in the binary optimization problem as follows:

$$\sum_{i=1}^{n} z_i = B \tag{2}$$

The parameter B defines the number of assets to be chosen for the portfolio and thereby constrains the money invested. In the following, we will refer to those portfolios $z_1, z_2, \ldots, z_n$ which fulfill the constraint as 'feasible' portfolios, whereas the remaining ones will be called 'unfeasible' [12].

2.1 Covariances and returns

In order to solve the portfolio optimization problem, the covariance matrix and the return vector must be provided. Here, we use the following prescription to calculate these values: If the portfolio consists of n assets, we use $m+1$ historical daily prices $p_k^{(i)}$ where $i = 1, 2, \ldots, n$ and $k = 0, 1, \ldots, m$. Out of these values we compute the daily percentage price change via

$$r_k^{(i)} = \left(p_k^{(i)} - p_{k-1}^{(i)}\right) / p_k^{(i)}, 1 \leq k \leq m, \tag{3}$$

and the annualized return of the portfolio:

$$\mu_i = \left[\prod_{k=1}^{m} \left(1 + r_k^{(i)}\right)\right]^{\frac{252}{m}} \tag{4}$$

The annualized covariance matrix is calculated with the equation

$$\sigma_{ij} = \frac{252}{m} \sum_{k=1}^{m} \left(r_k^{(i)} - \overline{r^{(i)}}\right)\left(r_k^{(j)} - \overline{r^{(j)}}\right) \tag{5}$$

where $\overline{r^{(i)}} = \frac{1}{m}\sum_{k=1}^{m} r_k^{(i)}$ is the mean of the daily price changes of asset i.

To investigate the portfolio optimization problem for different market segments, we calculated the annualized returns and covariance matrices for the German DAX 30 (as of 1/2/2021) between 01/01/2016 and 31/12/2020 (see supplementary material). In the following, we will investigate different instances of the portfolio optimization problem with given total number of assets $n = 5$ (with budget $B = 2$) or $n = 10$ (with budget $B = 5$). For given n, different instances of the problem are obtained by randomly selecting n out of the 30 Dax assets. The risk preference factor will be set to $q = 1/3$.

2.2 Performance measures

In order to quantify which values of the function F correspond to 'good' or 'bad' portfolios, we first determine the minimum and maximum value of F among all feasible portfolios:

$$F_{\min} = \min_{\sum_i z_i = B} F(z_1, \ldots, z_n) \tag{6}$$

$$F_{\max} = \max_{\sum_i z_i = B} F(z_1, \ldots, z_n) \tag{7}$$

Then we define the approximation ratio of a given portfolio:

$$r(z_1, \ldots, z_n) = \begin{cases} \frac{F(z_1,\ldots,z_n) - F_{\max}}{F_{\min} - F_{\max}} & \text{if } \sum_i z_i = B \\ 0 & \text{if } \sum_i z_i \neq B \end{cases} \tag{8}$$

Note that the optimal portfolio yields $r = 1$, whereas $r = 0$ corresponds to the worst feasible or an unfeasible portfolio. Let us note that this definition of the approximation ratio is independent of the particular method used to enforce the constraint (see below) [14].

The QAOA algorithm returns different portfolios with different probabilities, with corresponding average approximation ratio r. An alternative measure for the performance of QAOA is the probability P of obtaining the optimal portfolio. In contrast to the approximation ratio, however, the latter does not take into account the possibility that alternative portfolios might exhibit values of F very close to $F_{\min}$, which would also be acceptable for the portfolio manager.

2.3 Penalty factor

In the standard version of QAOA (see below), the minimization is performed with respect to all states, including those which do not fulfill the budget constraint, Eq. (2). In order to deal with the latter, a penalty term can be added to the function F as follows:

$$F^{(A)}(z_1, \ldots, z_n) = F(z_1, \ldots, z_n) + A \left(\sum_{i=1}^{n} z_i - B \right)^2 \tag{9}$$

The prefactor A should be chosen large enough such that, on the one hand, all unfeasible states (i.e. those which do not fulfill the constraint) yield $F^{(A)} > F_{\min}$. On the other hand, if A is chosen too large, the performance of QAOA is expected to deteriorate. It has been suggested to choose A such that all unfeasible states yield $F^{(A)} > F_{\max}$ [12, 14]. As we have found, however, this choice of A is unnecessarily large: after all, it is not our primary objective to separate feasible from unfeasible states, but to identify those states which exhibit values very close to $F_{\min}$. For this purpose, it is sufficient to choose A such that

$$F_{\min}^{(\mathrm{nf})} = \min_{\sum_i z_i \neq B} F^{(A)}(z_1, \ldots, z_n) \tag{10}$$

i.e., the minimum value of $F^{(A)}$ among all unfeasible states, is not 'too close' to $F_{\min}$. More precisely, we use the following procedure to ensure that

$$F_{\min}^{(\mathrm{nf})} \geq \frac{1}{2}(F_{\min} + \overline{F}) \tag{11}$$

where

$$\overline{F} = \frac{1}{\binom{n}{B}} \sum_{z_1 + \cdots + z_n = B} F(z_1, \ldots, z_n) \tag{12}$$

denotes the mean value of F over all feasible states:

(i) Use a classical algorithm to determine $F_{\min}$ and $\overline{F}$. Start with $A = 0$.
(ii) Use a classical algorithm to determine $F_{\min}^{(\mathrm{nf})}$, see Eq. (10).

(iii) Check whether Eq. (11) is fulfilled. If yes, return the value A. If not, increase A by ΔA, where

$$\Delta A = \frac{\frac{1}{2}(F_{\min} + \overline{F}) - F_{\min}^{(\mathrm{nf})}}{\left(\sum_i z_i^* - B\right)^2} \tag{13}$$

and $z_1^*, \ldots, z_n^*$ denotes the state where the minimum in Eq. (10) is reached. Repeat (ii) and (iii) until the condition, Eq. (11), is fulfilled.

In this paper, we will only consider examples with a small number n of assets (limited by the number of qubits in present NISQ devices or simulators), where $F_{\min}$, $F_{\min}^{(\mathrm{nf})}$ and $\overline{F}$ can be exactly determined with a classical brute force search. For a larger number of assets (when a larger number of qubits will be available in the future), the brute force search will have to be replaced by an efficient classical algorithm in order to determine these quantities at least approximately. If the value of A determined as described above then turns out to be too small (i.e., if the optimal solution found by the QAOA algorithm, see Sec. 3, violates the constraint), the algorithm can then be re-run with a larger value of A.

3 QAOA algorithm

To solve the above optimization problem on a quantum computer, we convert the function $F^{(A)}$ into a quantum operator:

$$\hat{F} = \lambda\, F^{(A)}\left(\frac{\hat{I}_1 + \hat{Z}_1}{2}, \ldots, \frac{\hat{I}_n + \hat{Z}_n}{2}\right) \tag{14}$$

where $\hat{I}_j$ and $\hat{Z}_j$ denote the identity and the Pauli-$\hat{Z}$ operator, respectively, acting on qubit j. Additionally, we scale the function by a constant factor $\lambda > 0$, the significance of which will be explained below. Taking into account Eqs. (1,9), we can write $\hat{F}$ as a sum of two-qubit and single-qubit operators (plus an irrelevant constant term c)

$$\hat{F} = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} W_{ij} \hat{Z}_i \hat{Z}_j - \sum_{i=1}^{n} w_i \hat{Z}_i + c \tag{15}$$

where $W_{ij} = \frac{\lambda}{2}(q\sigma_{ij} + A)$ and $w_i = \frac{\lambda}{2}\left[(1-q)\mu_i + A(2B-n) - q\sum_{j=1}^{n}\sigma_{ij}\right]$.

The QAOA variational quantum circuit then generates the following quantum state $|\psi_{\vec{\gamma},\vec{\beta}}\rangle$ depending on the parameters $\vec{\gamma} = (\gamma_1, \ldots, \gamma_p)$ and $\vec{\beta} = (\beta_1, \ldots, \beta_p)$ (with number of iterations p, also referred to as 'QAOA depth' in the following):

$$|\psi_{\vec{\gamma},\vec{\beta}}\rangle_M = \hat{U}_M(\beta_p) e^{-i\gamma_p \hat{F}} \ldots \hat{U}_M(\beta_2) e^{-i\gamma_2 \hat{F}} \hat{U}_M(\beta_1) e^{-i\gamma_1 \hat{F}_A} |\psi_0\rangle_M \tag{16}$$

where the initial state $|\psi_0\rangle_M$ and the operator $\hat{U}_M(\beta)$ depend on the choice of the mixer M, see below. Finally, all qubits are measured with respect to the standard basis in order to determine the mean value

$$\langle F\rangle_{\vec{\gamma},\vec{\beta}} = \langle\psi_{\vec{\gamma},\vec{\beta}}|\hat{F}|\psi_{\vec{\gamma},\vec{\beta}}\rangle \tag{17}$$

This information is then passed to a classical optimization routine, which suggests new values for the parameters $\vec{\gamma}$ and $\vec{\beta}$ in order to minimize the expectation value $\langle F\rangle_{\vec{\gamma},\vec{\beta}}$.

As explained in [1], this particular choice, Eq. (16), of the QAOA circuit is motivated by the principle of adiabatic quantum computing. With $\hat{U}_M(\beta) = e^{-i\beta\hat{M}}$, the circuit can be interpreted as a Trotterized time evolution transferring the eigenstate $|\psi_0\rangle_M$ of the mixing operator $\hat{M}$ to the desired eigenstate of the operator $\hat{F}$. For a finite depth p, however, the optimal choice of the angles $\vec{\beta}$ and $\vec{\gamma}$ may also amount to a non-adiabatic annealing path [7].

3.1 Mixer

3.1.1 Standard QAOA

In the originally proposed [1] version of QAOA (M = standard), the mixing operation is realized by single-qubit rotations applied to each qubit:

$$\hat{U}_{\text{standard}}(\beta) = e^{i\beta\sum_{i=1}^{n}\hat{X}_i} \tag{18}$$

The corresponding initial state is $|\psi_0\rangle_{\text{standard}} = |+\cdots+\rangle$, where $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$. This state is the eigenstate of the mixing operator $\hat{M}_{\text{standard}} = -\sum_i \hat{X}_i$ associated to its smallest eigenvalue $(-n)$.

3.1.2 XY-mixers

Alternatively, the initial state and the mixer can be adapted such that only states $|\boldsymbol{z}\rangle = |z_1,\ldots,z_n\rangle$ fulfilling the budget constraint, see Eq. (2), are populated during the algorithm [2, 16]. Correspondingly, the initial state is chosen as a uniform superposition (Dicke state [17])

$$|\psi_0\rangle_{M_{XY}} = |D_B^n\rangle = \frac{1}{\sqrt{\binom{n}{B}}}\sum_{\substack{i_1,\ldots,i_n=0,1\\ i_1+\cdots+i_n=B}}|i_1\ldots i_n\rangle \tag{19}$$

This state is an eigenstate of the operator $(\hat{X}_i\hat{X}_j + \hat{Y}_i\hat{Y}_j)$ (for all pairs of qubits i and j), which essentially swaps two qubits populating different states (i.e., $|01\rangle \leftrightarrow |10\rangle$), thereby keeping the total budget (or number of excitations) constant. Therefore, the basic ingredient of the XY-mixers is given by the following two-qubit operation:

$$\hat{R}_{i,j}^{(XY)}(\beta) = e^{i\beta(\hat{X}_i\hat{X}_j+\hat{Y}_i\hat{Y}_j)} \tag{20}$$

Different versions M_{XY} of the XY-mixer are now obtained by applying this operation to different sets $S_{M_{XY}}$ of qubit pairs:

$$\hat{U}_{M_{XY}}(\beta) = \prod_{(i,j)\in S_{M_{XY}}} \hat{R}_{i,j}^{(XY)}(\beta) \tag{21}$$

We will investigate the following four versions:

(i) **Ring mixer** ($M_{XY} = \text{ring}$)
In the *ring mixer*, the operation $\hat{R}_{i,j}^{(XY)}(\beta)$ is applied only to neighbouring pairs of qubits, i.e., $S_{\text{ring}} = \{(1,2),(2,3),\ldots,(n-1,n),(n,n+1)\}$, where qubit $n+1$ is identified with qubit 1. Note that, since $[\hat{R}_{i,j}^{(XY)}, \hat{R}_{j,k}^{(XY)}] \neq 0$ for mutually distinct qubits i, j and k, the ordering in which the individual operations $\hat{R}_{i,j}^{(XY)}$ are applied in Eq. (21) is relevant. It is given by the order in which the corresponding qubit pairs appear in the above set S_{ring}, i.e., first (1,2), then (2,3) etc.

(ii) **Parity ring mixer** ($M_{XY} = \text{par_ring}$)
In the *parity ring mixer* , this ordering is changed such as to obtain two subsets of commuting operations $\hat{R}_{i,i+1}^{(XY)}$ with odd or even i, respectively, i.e., $S_{\text{par_ring}} = \{(1,2),(3,4),\ldots,(n_o,n_o+1),(2,3),(4,5),\ldots,(n_e,n_e+1)\}$, where n_e (or n_o) is the largest even (or odd) number $n_e \leq n$ (or $n_o \leq n$) (and, again, qubit $n+1$ is identified with qubit 1).

(iii) **Full mixer** ($M_{XY} = \text{full}$)
In the *full XY-mixer* ($M_{XY} = \text{full}$), the set S_{full} contains *all* pairs of qubits, where the ordering is chosen such that as many gates as possible can be performed in parallel, thus minimizing the depth of the circuit. If n is odd, the $\hat{R}_{i,j}^{(XY)}$'s are arranged into n subsets of $(n-1)/2$ commuting operations, where $i+j \mod n = k$ in subset k. (E.g., for $n=5$: $S_{\text{full}} = \{(1,5),(2,4),(2,5),(3,4),(2,1),(3,5),(3,1),(4,5),(3,2),(4,1)\}$ with subsets $\{(1,5),(2,4)\}$, $\{(2,5),(3,4)\}$ etc.) If n is even, we first generate the subsets for $n-1$ as described above, and add the missing pair of qubits to each subset. (For example, for $n=6$, we add $(3,6)$ to the first subset $\{(1,5),(2,4)\}$, $(1,6)$ to the second, etc., i.e., $S_{\text{full}} = \{(1,5),(2,4),(3,6),(2,5),(3,4),(1,6),...\}$.)
Together with the phase separation operator $e^{-i\gamma\hat{F}}$, a single iteration step in the full XY-mixer QAOA algorithm, see Eq. (16), reads as follows:

$$\hat{U}_{\text{full}}(\beta)e^{-i\gamma\hat{F}} = \prod_{(i,j)\in S_{\text{full}}} \hat{R}_{i,j}^{(XY)}(\beta) \prod_{(i,j)\in S_{\text{full}}} e^{-i\gamma W_{ij}\hat{Z}_i\hat{Z}_j} \prod_{i=1}^{n} e^{-i\gamma w_i \hat{Z}_i} \tag{22}$$

(iv) **Quantum alternate mixer-phase ansatz** ($M_{XY} = \text{QAMPA}$)
Finally, we modify the full mixer according to the recently proposed quantum alternate mixer-phase ansatz (QAMPA) [18]. Instead of applying the operator $e^{-i\gamma\hat{F}}$ to all qubits before applying the mixer, we reorder the terms such that, for each pair of qubits, the gates corresponding to the mixer and

the phase separation are contracted as follows:

$$\hat{U}_{\mathrm{MP}}(\beta,\gamma) = \prod_{(i,j)\in S_{\mathrm{full}}} e^{i\beta(\hat{X}_i\hat{X}_j+\hat{Y}_i\hat{Y}_j)-i\gamma W_{ij}\hat{Z}_i\hat{Z}_j} \prod_{i=1}^{n} e^{-i\gamma w_i\hat{Z}_i} \tag{23}$$

Thereby, the QAOA circuit, see Eq. (16) is modified as follows:

$$|\psi_{\vec{\gamma},\vec{\beta}}\rangle_M = \hat{U}_{MP}(\beta_p,\gamma_p)\ldots\hat{U}_{MP}(\beta_2,\gamma_2)\hat{U}_{MP}(\beta_1,\gamma_1)|\psi_0\rangle_{M_{XY}} \tag{24}$$

As evident from Fig. 8 in Appendix A, this reduces the number of CNOT gates by a factor 3/4. Since the CNOT gates constitute the most important source of errors in present NISQ devices, we may expect that this version is more resilient against errors than the full XY mixer.

3.2 Optimization of the QAOA parameters

The most critical step in the actual implementation of QAOA consists of choosing the classical optimization procedure used to minimize the expectation value $\langle F\rangle_{\vec{\gamma},\vec{\beta}}$, see Eq. (17). Before outlining our procedure, let us discuss the significance of the scaling factor λ introduced above in Eq. (14), which, to our knowledge, has not yet been pointed out in the present literature.

3.2.1 Global scaling factor

At first sight, the scaling $\hat{F} \to \lambda\hat{F}$ appears to have no relevant effect. Indeed, due to the phase separation operators $e^{i\gamma_j\hat{F}}$ $(j = 1,\ldots,p)$ in Eq. (16), this factor only leads to a rescaling $\gamma_j \to \gamma_j/\lambda$ of the angles $\vec{\gamma}$, whereas the angles $\vec{\beta}$ remain unchanged. Nevertheless, we have observed that such a scaling can have an important influence on the classical optimization. Since the optimizer used for this purpose is usually imported from a standard numerical library, it, a priori, treats all the parameters $\vec{\gamma}$ and $\vec{\beta}$ on the same footing. Numerical difficulties may arise if the optimal values of the parameters $\vec{\gamma}$ differ by orders of magnitude from those of $\vec{\beta}$. In contrast, the best performance is expected if all parameters exhibit approximately the same order of magnitude.

For this reason, our idea is to choose λ such that the operator $\hat{F}$ exhibits a similar spectral width as the mixing operator $\hat{M}$. The latter reads $\hat{M}_{\mathrm{standard}} = -\sum_i \hat{X}_i$ (standard mixer) or $\hat{M}_{XY} = -\sum_{(i,j)\in S_{M_{XY}}}(\hat{X}_i\hat{X}_j+\hat{Y}_i\hat{Y}_j)$ (XY mixers), respectively. (Note that, in the latter case $\hat{U}_{M_{XY}}(\beta) \neq e^{-i\beta\hat{M}_{XY}}$ due to non-commuting terms. Nevertheless, we use $\hat{M}_{XY}$ for obtaining a rough estimate of a suitable λ value). The corresponding spectral width, defined as the difference between largest and smallest eigenvalue of $\hat{M}$, is $\Delta M = 2n$ (standard, XY ring and XY ring_par) or $\Delta M = n(n-1)$ (XY full and XY QAMPA), respectively.

Without scaling (i.e. for $\lambda = 1$), the largest eigenvalue of $\hat{F}$ is either $F_{\max}$ or $F_{\max}^{(\mathrm{nf})}$, see Eq. (10) with 'max' instead of 'min'. Since, in case of the XY-mixers, states which do not fulfill the constraint are not populated, we define $\Delta F = F_{\max} - F_{\min}$ as effective spectral width. In case of the standard mixer, we take into account, both, $F_{\max}$ and $F_{\max}^{(\mathrm{nf})}$ (where the latter is typically much larger and determined by one of the few states that violate the constraint most strongly) and define $\Delta F = \left[(F_{\max} - F_{\min})\left(F_{\max}^{(\mathrm{nf})} - F_{\min}\right)\right]^{1/2}$. Finally, to make the spectra of $\hat{M}$ and $\hat{F}$ comparable, we set $\lambda = \Delta M/\Delta F$.

3.2.2 Initial values of parameters for $p = 1$

Another important aspect concerns the initial values of $(\vec{\gamma}, \vec{\beta})$ fed into the classical optimization routine. The latter usually converges to a local minimum in the vicinity of the initial point. Therefore, the quality of the solution can be enhanced by providing a good starting point which is close to the desired global minimum. In previous work [7], it has been found that a suitable initial point can be extracted from the optimal values determined previously for QAOA depth $p - 1$.

Concerning the lowest depth $p = 1$, it is possible to examine a relatively large range of possible parameters γ and β and plot the corresponding landscape $\langle \hat{F} \rangle_{\gamma,\beta}$. Due to the periodicity of the mixers introduced above, values of β can be restricted to the interval $[0, \pi]$, but the same is not true for γ. An example of the landscape for an exemplary portfolio optimization instance with $n = 10$ assets is shown in Fig. 1(a). We see several local minima, where three of them achieve similar values (two on the left, i.e., at small m_1, and one at $m_1 \simeq 3$, $m_2 \simeq 4$). As we have checked, however, not all of them are equally suitable as starting points for the extrapolation towards larger depths p. This can be seen by employing the following simple, linear ansatz for $(\vec{\gamma}, \vec{\beta})$:

$$\vec{\gamma}^{(\mathrm{lin},p)}(m_1) = m_1(x_1^{(p)}, x_2^{(p)}, \ldots, x_p^{(p)}) \tag{25}$$

$$\vec{\beta}^{(\mathrm{lin},p)}(m_2) = m_2(1 - x_1^{(p)}, 1 - x_2^{(p)}, \ldots, 1 - x_p^{(p)}) \tag{26}$$

where

$$x_i^{(p)} = \frac{2i - 1}{2p} \tag{27}$$

For $m_1, m_2 > 0$, the angles $\gamma_i^{(\mathrm{lin},p)}(m_1)$ increase with increasing i (i.e. in the same order in which they are also applied in the QAOA circuit), whereas $\beta_i^{(\mathrm{lin},p)}(m_2)$ decrease. This agrees with the interpretation of QAOA as quantum annealing, where $\hat{F}$ is gradually switched on while $\hat{M}$ is switched off.

If we now plot

$$\langle \hat{F} \rangle_{m_1,m_2}^{(\mathrm{lin},p)} = \langle \hat{F} \rangle_{\vec{\gamma}^{(\mathrm{lin},p)}(m_1), \vec{\beta}^{(\mathrm{lin},p)}(m_2)} \tag{28}$$

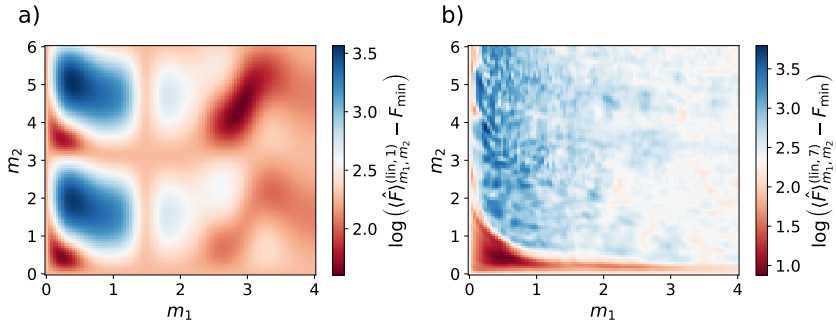


Fig. 1 (a) QAOA parameter landscape at $p = 1$, for an exemplary portfolio optimization instance with $n = 10$ assets. To enable a direct comparison with (b), the parameters γ, β are expressed in terms of $m_1 = 2\gamma$ and $m_2 = 2\beta$, respectively. The colour code shows the deviation of $\langle\hat{F}\rangle_{\gamma,\beta} = \langle\hat{F}\rangle^{(\mathrm{lin},1)}_{m_1,m_2}$ from the optimal solution $F_{\min}$ on a logarithmic scale. Three pronounced minima are visible in dark red. (b) For higher depth $p = 7$ with linear ansatz for the parameters $\vec{\gamma}$ and $\vec{\beta}$, see Eqs. (25,26), only one of these minima (close to the bottom left corner) survives

as a function of m_1, m_2 for larger p (e.g., $p = 7$), we see that only one single pronounced local minimum remains, see Fig. 1(b). To find a suitable starting point for $p = 1$, we therefore try 100 different combinations of parameters m_1, m_2 (in a logarithmically spaced grid with $\frac{1}{100} < m_1 < 100$ and $\frac{\pi}{100} < m_2 < \pi$), and choose the one yielding the smallest value of $\langle\hat{F}\rangle^{(\mathrm{lin},\mathrm{p_{max}})}_{m_1,m_2}$ (where $p_{\max}$ denotes the maximum depth, see below) as a starting point for the minimization of $\langle\hat{F}\rangle_{\gamma,\beta}$ (where $\beta = m_2/2$ and $\gamma = m_1/2$ for $p = 1$), in order to arrive at the one local minimum which is suitable for extrapolation towards larger p.

3.2.3 Initial values of parameters for larger p

The minimization of $\langle\hat{F}\rangle$ is now performed for increasing values of $p = 2, 3, \ldots, p_{\max}$. For each p, we apply the interpolation method described below for choosing a suitable initial point based on the optimal parameters previously found for depth $p-1$. Since we cannot strictly exclude the possibility to get stuck in a local minimum different from the global one (which is a generic problem in high-dimensional optimization), we repeat the optimization three times with additional starting points determined by different methods. In summary, we perform, for each depth p, four optimization runs starting from the following initial parameters:

(i) We consider the optimized parameters $\vec{\gamma}^{(p-1)} = (\gamma_1^{(p-1)}, \ldots, \gamma_{p-1}^{(p-1)})$ from the previous depth $p-1$ and want to determine angles $\vec{\gamma}^{(p)} = (\gamma_1^{(p)}, \ldots, \gamma_p^{(p)})$ as starting point for the optimization at depth p. To determine $\gamma_i^{(p)}$ $(i = 1, \ldots, p)$, we first select the two points $x_j^{(p-1)}$ and $x_{j+1}^{(p-1)}$, which are closest to $x_i^{(p)}$. Then, we define a straight line $\gamma(x) = ax + b$ such that $\gamma\left(x_j^{(p-1)}\right) =$

$\gamma_j^{(p-1)}$ and $\gamma\left(x_{j+1}^{(p-1)}\right) = \gamma_{j+1}^{(p-1)}$ and choose $\gamma_i^{(p)} = \gamma\left(x_i^{(p)}\right)$. The same is repeated with the angles β.

(ii) First determine m_1 and m_2 by minimizing $\langle\hat{F}\rangle^{(\mathrm{lin},p)}(m_1, m_2)$ (where we take m_1 and m_2 from the previous depth $p-1$ as initial point), and then use $\vec{\gamma}^{(\mathrm{lin},p)}(m_1)$ and $\vec{\beta}^{(\mathrm{lin},p)}(m_2)$ as initial point for minimizing $\langle\hat{F}\rangle_{\vec{\gamma},\vec{\beta}}$

(iii) The same is repeated using a quadratic ansatz

$$\left[\vec{\gamma}^{(\mathrm{quad},p)}(a_1, b_1, c_1)\right]_i = a_1 + b_1 x_i^{(p)} + c_1\left(x_i^{(p)}\right)^2 \tag{29}$$

$$\left[\vec{\beta}^{(\mathrm{quad},p)}(a_2, b_2, c_2)\right]_i = a_2 + b_2 x_i^{(p)} + c_2\left(x_i^{(p)}\right)^2 \tag{30}$$

First, $\langle\hat{F}\rangle^{(\mathrm{quad},p)}(a_1, b_1, c_1, a_2, b_2, c_2) = \langle\hat{F}\rangle_{\vec{\gamma}^{(\mathrm{quad},p)}(a_1,b_1,c_1),\vec{\beta}^{(\mathrm{quad},p)}(a_2,b_2,c_2)}$ is minimized to determine optimal values for the six parameters $a_1, \ldots, c_2$ (where, for $p > 2$, the optimized values from the previous depth $p-1$ are taken as initial values, whereas for $p = 2$, we choose initial values $(a_1, b_1, c_1, a_2, b_2, c_2) = (0, m_1, 0, m2, -m2, 0)$ with m_1 and m_2 as determined by the grid search described in Sec. 3.2.2), then the corresponding angles $\vec{\gamma}^{(\mathrm{quad},p)}$ and $\vec{\beta}^{(\mathrm{quad},p)}$ are taken as initial point for minimizing $\langle\hat{F}\rangle_{\vec{\gamma},\vec{\beta}}$

(iv) Finally, we take the optimal angles for depth $p-1$ and add $\gamma_p = \beta_p = 0$. Thereby, we recover at least the same value as for $p-1$ and ensure that $\langle\hat{F}\rangle$ monotonically decreases with increasing depth p.

Among these four optimization runs, we select the one achieving the smallest expectation value $\langle\hat{F}\rangle$. Before proceeding to the next depth $(p+1)$, we finally rescale $\hat{F} \to \mu\hat{F}$ together with $\vec{\gamma} \to \vec{\gamma}/\mu$ (see Sec. 3.2.1), with μ chosen such that $\sum_i |\gamma_i| = \sum_i |\beta_i|$, to ensure that the parameters γ and β retain a similar order of magnitude.

3.2.4 Classical optimizers

Apart from trying different initial values $(\vec{\gamma}, \vec{\beta})$ as explained above, we can choose between different routines for optimizing these values. Concerning the simulation of the QAOA circuits, we distinguish between two different methods: on the one hand, Qiskit's 'statevector simulator' allows us to precisely calculate the expectation value, Eq. (17), without any statistical sampling error. On the other hand, the 'qasm simulator' includes the measurement process which, for a finite number of shots (in our case: 1000), leads to a statistical sampling error due to the random measurement outcomes.

According to our experience, the gradient-based optimization routine 'SLSQP' (which is included in the function 'minimize' of the Python package 'scipy.optimize') yields good results in case of the statevector simulator. The gradient-based optimization routines 'gradient-descent' (GD) and 'stochastic gradient descent' (SGD) show a similar performance for the standard mixer, but perform less well than SLSQP for the XY-mixers. In case of the qasm simulator, more robust gradient-free methods (COBYLA and Nelder-Mead) are

preferable, since the sampling noise prevents a precise evaluation of the gradient. Here, we choose the method 'Nelder-Mead' with the following parameters: the size of the initial simplex is set to 0.5 (i.e., the simplex consists of the initial point plus $2p$ additional points obtained from the initial point by adding 0.5 to one of the $2p$ parameters) and the maximum number of iterations is limited to 10 times the number of parameters ($2p$).

4 Results

4.1 Different Mixers

In this section, we give a presentation and discussion of our QAOA simulation results for the different mixers. For this purpose, we discuss the approximation ratio and the ground state probability plotted as a function of the QAOA depth p for a given number of assets that, in turn, coincides with the number of qubits in the circuit. The results are generated using both simulator methods (without and with measurement noise) with respective optimization routines as previously introduced.

For the comparison, we generate an ensemble of 20 different portfolio optimization instances with given total number $n = 5$ or $n = 10$ of randomly selected assets as described in Sec. 2.1. We compare the convergence of approximation ratio and ground state probability of each mixer and simulation method with increasing QAOA depth p (up to $p_{\max} = 7$).

Fig. 2 shows the approximation ratio r as well as the ground state population P for 20 instances with $n = 5$ assets as a function of the QAOA depth p using the statevector, Fig. 2(a,b), and qasm simulator, Fig. 2(c,d). Already at $p = 1$, the XY-mixers outperform the standard mixer. This can simply be explained by the fact that, in case of the XY mixers, the optimization is restricted to a smaller space [19], since, in contrast to the standard mixer, all unfeasible portfolios (i.e., those not satisfying the budget constraint) are never populated during the algorithm (see Sec. 3.1.2). With increasing p, the full and the QAMPA mixer show the fastest convergence towards the optimal solution, achieving approximation ratios $r > 99\%$ without measurement noise, see Fig. 2(a). The superiority of the full XY mixer (and its variant QAMPA) as compared to the ring mixers can be traced back to their higher degree of symmetry [20]. Indeed, the full XY and QAMPA mixer contain all pairs of qubits, whereas only pairs of neighbouring qubits occur in the ring mixers. The full mixers therefore completely mix across all basis states at the first attempt, whereas the ring mixers take several rounds [19].

Concerning the probability P of obtaining the optimal portfolio, see Fig. 2(b), the above discussed difference between the various mixers is less clearly visible. Taking into account the different scales (logarithmic scale for $1 - r$ vs. linear scale for P), this is due to larger fluctuations among the 20 random instances (see error bars), such that small differences between values of P close to 1 cannot be resolved. These larger fluctuations, in turn, originate from the fact that the probability P depends on the population of only

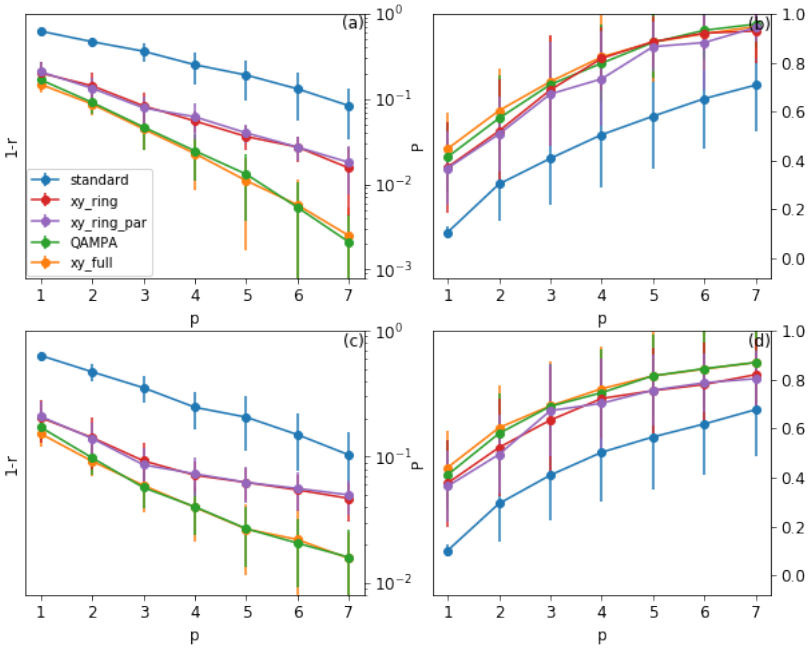


Fig. 2 (a) Mean deviation $1-r$ of the approximation ratio from the optimal solution as a function of the QAOA depth p for 20 randomly chosen portfolio optimization instances with $n = 5$ assets (and budget $B = 2$), using different mixers (see legend) and the statevector simulator (without measurement noise). The error bars display the standard deviation among the 20 random instances. The full XY mixer and QAMPA (which requires less CNOT gates) show the fastest convergence towards the optimal solution. (b) Mean probability P of obtaining the optimal portfolio with standard deviation (error bars) as a function of QAOA depth p for the statevector simulator. (c,d) Same as (a,b), but using the qasm simulator including measurement noise due to a finite number of 1000 shots instead of the statevector simulator. Significant differences with respect to the statevector simulator are only visible in the vicinity of the optimal solution (r and P close to 1)

a single state (i.e., the optimal one), whereas the approximation ratio results from an expectation value taking into account also other feasible states, which might achieve values of the cost function close to the optimal value $F_{\min}$.

The results of the qasm simulator (with 1000 shots per circuit) in Fig. 2(c,d) show a similar behavior. Only in case of the XY mixers at larger values of p, the statistical fluctuations due to the measurement noise lead to a slower convergence towards the optimal solution.

Fig. 3 displays the results of a similar analysis with larger instances consisting of $n = 10$ assets with budget $B = 5$ (instead of $n = 5$ and $B = 2$, as in Fig. 2). Basically, they confirm our above conclusions concerning, e.g., the superiority of the full XY and the QAMPA mixer. Not surprisingly, the

values of r and P are smaller than in Fig. 2, since the total number of possible portfolios is much larger (i.e., $\binom{10}{5=252}$ instead of $\binom{5}{2=10}$ feasible portfolios). Nevertheless, using one of the two best mixers (full XY or QAMPA), we still find approximation ratios as large as $r \simeq 0.98$, with corresponding probability $P \simeq 0.6$ of obtaining the optimal solution.

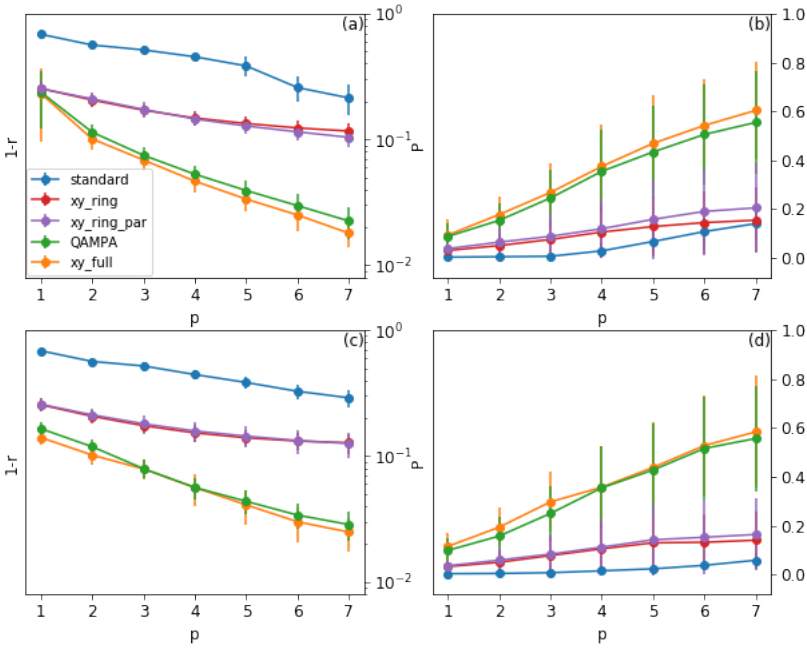


Fig. 3 Same as in Fig. 2 with ensemble of portfolio instances with $n = 10$ and budget $B = 5$. The main conclusions, in particular concerning the superiority of the full XY and the QAMPA mixer, are similar as in Fig. 2.

4.2 Easy and Hard Instances

As we have seen in the previous section, the QAOA performance, especially the probability P of obtaining the optimal solution, varies a lot over a set of different instances of the portfolio optimization problem. Some instances are easier to optimize than others. A possible cause for this performance differences could lie in the respective distributions of correlations and returns characterizing an instance. We examine performance dependencies on these instance statistics. For this purpose, we use the variances of the returns and correlation distributions as a measure of 'hardness' of a given instance:

$$s^2_{\text{ret}} = \text{var}\left(\{\mu_i\}\right)$$

$$s^2_{\text{cor}} = \text{var}\left(\left\{\frac{\sigma_{ij}}{\sqrt{\sigma_{ii}}\sqrt{\sigma_{jj}}} \mid i \leq j\right\}\right)$$

The correlation is normalized to a value between -1 and 1 (Pearson coefficients). This normalization resulted in more pronounced differences in statistics than using the raw correlations.

Each instance generates an energy landscape, composed of the objective function values $F(\boldsymbol{z})$ of all possible portfolios represented by the string $\boldsymbol{z} = z_1, z_2, \ldots, z_n$. We characterize the energy landscape by the mean and variance of the energy value distribution:

$$\begin{aligned}\mu_{\text{energy}} &= \text{mean}\left(\{F(\boldsymbol{z}) \mid \boldsymbol{z} \text{ satisfying the investment constraint}\}\right)\\ s^2_{\text{energy}} &= \text{var}\left(\{F(\boldsymbol{z}) \mid \boldsymbol{z} \text{ satisfying the investment constraint}\}\right)\end{aligned}$$

In the course of a Monte Carlo walk, we optimized 2400 randomly generated baskets and evaluated the performance of each instance with the measure

$$\text{perf} = \sqrt{G^2 + R^2}.$$

We kept the 20 best and the 20 worst performing instances and examined their statistics. The instances were generated as described in Sec. 2.1. We restricted the size of the instances to $n = 10$ and the investment constraint to $B = 5$ assets. For the optimization, we used a QAOA version with standard mixer. To enable a clearer comparison, we chose the same penalty factor $A = 0.2$ and scaling factor $\lambda = 6$ for all instances. We further implemented 'linear interpolation' as described in [7] for angle initialization and optimized up to $p = 6$. The algorithm was simulated using the statevector simulator and the SLSQP method for the classical optimization step.

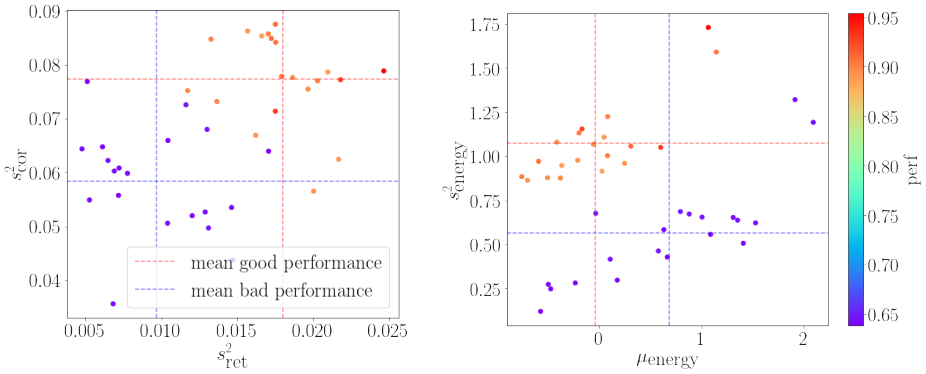


Fig. 4 Variance of returns s^2_{ret} and normalized correlations s^2_{cor} for instances of good/bad performance (left). Mean μ_{energy} and variance s^2_{energy} of associated energy landscape (right)

As can be seen in Fig. 4, the performance of an instance correlates with the variance of its asset correlations and returns, as well as with the mean and variance of its associated energy landscape. We repeated this procedure with historical data from MDAX 50 and found similar distributions.

These results can be interpreted as follows. Instances consisting of stocks with broadly distributed correlations are easier to optimize than those with more similar correlations. The same holds for returns. A possible reason can be found by looking at the energy landscapes created by the cost function 1. High variances of correlations σ_{ij} and returns μ_i get propagated to high variance of $F(\boldsymbol{z})$ over possible $\boldsymbol{z}$. This could make the optimization step easier since the energy landscape becomes more distinct. Or, to put it figuratively: when the possible portfolios that can be built from an instance look more different in terms of correlations and returns, it is easier to choose the best one. Consequently, this dependency is an effect of the classical part of QAOA and not of the quantumness of the algorithm. It should therefore be interesting to look for this behaviour in purely classical portfolio optimization as well. Besides the characterizing statistics shown, we looked for dependencies in the eigenvalue spectra of the covariance matrices. Here, we could not find any differences for good or bad performing instances. Nevertheless, there could be other measures characterizing the instances which lead to the same, or even a more extreme performance dependency.

4.3 Evaluation of QAOA under Noise

In this section, we study the effect of noise on a given QAOA instance. In particular, we first investigate the optimization landscape of QAOA to outline convergence issues with an increasing noise strength and then discuss the probability P of obtaining the optimal solution and the approximation ratio r of QAOA subject to varying noise strengths.

The evaluations in this section were performed on a QAOA instance of a portfolio optimization problem while varying the QAOA parameter p, the employed QAOA mixer (see section 3.1) and noise strength. The used portfolio optimization problem consists of $n = 5$ assets, i.e. five qubits, out of which $B = 2$ assets must be optimally selected with a risk-return factor $q = 1/3$. The corresponding covariances σ_{ij} and returns μ_i are listed in Appendix B.

4.3.1 Employed Noise Model

In order to compute QAOA subject to noise, we employ the depolarizing channel [21]. The depolarizing channel $\mathcal{E}$ acting on n qubits described by a density matrix ρ is defined as

$$\mathcal{E}(\rho) = \frac{\eta I}{2^n} + (1 - \eta)\rho$$

where η is the depolarizing noise strength and I denotes the identity operator. After each single-qubit or two-qubit (CNOT) gate occurring in the QAOA ansatz circuit, the depolarizing channel is applied to the respective qubit(s).

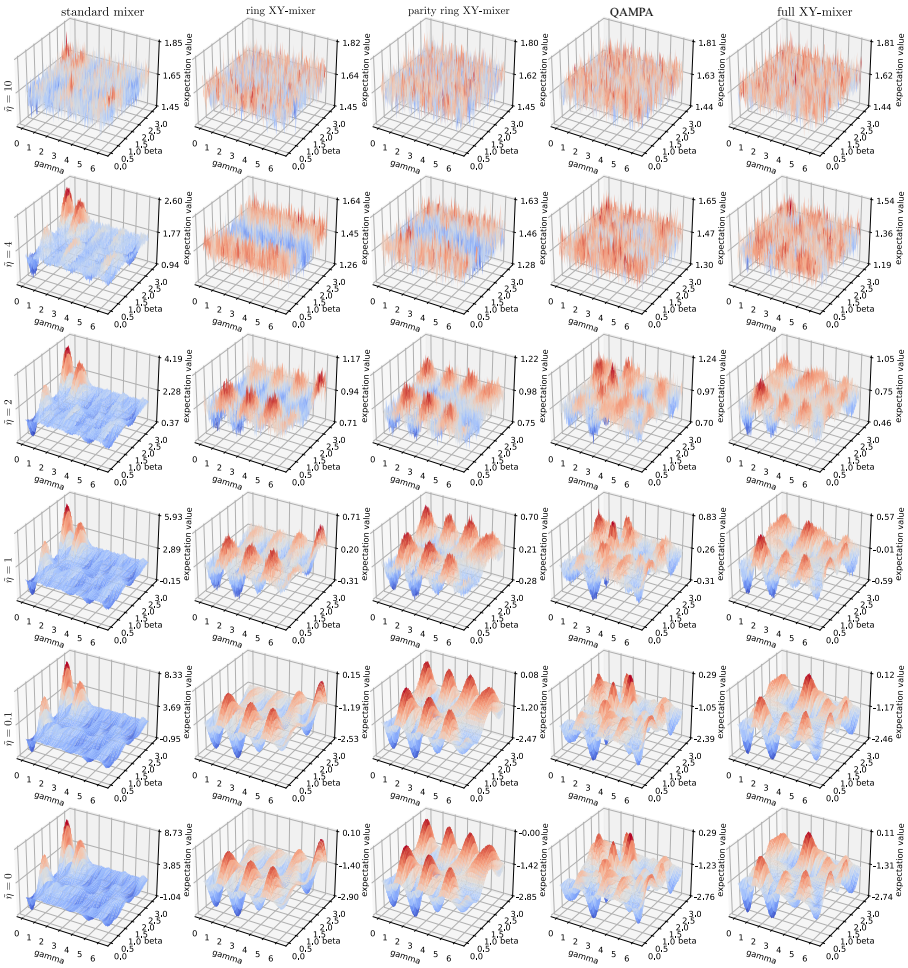


Fig. 5 Expectation value of the investigated QAOA instance for the standard mixer, the ring XY-mixer, the parity ring XY-mixer, the QAMPA, and the full XY-mixer (from left to right) and noise strength $\tilde{\eta} \in \{0, 0.1, 1, 2, 4, 10\}$ (from bottom to top), where $\tilde{\eta}$ is the noise strength normalized by the size of the quantum computation, i.e. the number of quantum gates in the QAOA ansatz circuit.

4.3.2 Optimization Landscape of QAOA under Noise

In this section, we show the optimization landscape of the investigated QAOA instance for depth $p = 1$ subject to depolarizing noise for varying noise strengths and QAOA mixers. Fig. 5 shows the optimization landscape of the

investigated QAOA instance, i.e. for a each pair (γ, β) of parameters, the expectation value of the optimization function (see equation 17) is plotted. The parameter γ is varied from 0.025 to 2π while β is varied from 0.025 to π in steps of size 0.025. From the left to the right column, the optimization landscape of QAOA is shown for the standard mixer, the ring XY-mixer, the parity ring XY-mixer, the QAMPA and the full XY-mixer. From the bottom to the top row, the normalized noise strength is varied as $\tilde{\eta} \in \{0, 0.1, 1, 2, 4, 10\}$. Here, the normalized noise strength $\tilde{\eta}$ is the noise strength η divided by the size of the quantum computation, i.e. the number of quantum gates in the QAOA ansatz circuit. This normalization is used to better study the vulnerability of specific structures within different ansatz circuits to noise. Without such normalization, the performance of circuits under a constant noise level would be dominated by the circuit size, as a circuit with more gates will tend to exhibit more errors. The x- and y-axes of the figures in Fig. 5 show the parameters γ and β of QAOA, while the expectation value of the optimization function is shown on z-axis.

In the error-free case (bottom row of Fig. 5), the ring XY-mixer finds the best (lowest) value of the expectation value (-2.90), followed by the parity ring XY-mixer (-2.85), QAMPA (-2.76), full XY-mixer (-2.74) and lastly the standard mixer (-1.04). However, with increasing noise strength, the order of the mixers with respect to the yielded minimal expectation value first becomes constant at $\tilde{\eta} = 0.1$ and then reverts for $\tilde{\eta} = 1$, such that the full XY-mixer is able to yield a lower expectation value than all other mixers. At a noise strengths of $\tilde{\eta} = 2$ and $\tilde{\eta} = 4$, the standard mixer yields the best expectation value until, at $\tilde{\eta} \geq 10$, the optimization landscape is dominated by noise for all mixers.

For higher noise strengths, e.g. $\tilde{\eta} \geq 1$, the optimization landscape begins to flatten and to deteriorate for each mixer. The expectation value tends to become less regular and more sensitive to small changes in the parameters γ and β. These effects impair the ability of the classical optimization algorithms to converge to the global minimum of the expectation value, eventually leading to a noise-induced barren plateau that incurs an exponential runtime overhead [22]. As demonstrated in the top rows of Fig. 5, the optimization landscape deteriorates stronger for XY-mixers than for the standard mixer at the same noise strength $\tilde{\eta}$.

The results in Fig. 5 demonstrate that XY-mixers yield a better expectation value compared to the standard mixer as long as the noise strength remains below some threshold. After that threshold is exceeded, the standard mixer tends to yield a better expectation value, making the standard mixer a better option.

Overall, the optimization landscape of the standard mixer tends to be less complex, with less local optima compared to the XY-mixers. With an increasing noise strength, the deterioration of such simpler landscapes tends to be less pronounced. This may imply a faster convergence, requiring less

optimization steps, to reach the respective global minimum with the standard mixer compared to XY-mixers for higher noise levels.

4.3.3 Evaluation of QAOA at specific noise strengths

In this section, the investigated QAOA instance is computed subject to the depolarizing channel for different QAOA depth p, mixers and noise strength $\eta \in \{0, 0.001, 0.002, ..., 0.01\}$. Note that η is not normalized, in contrast to $\tilde{\eta}$ employed to generate optimization landscapes in Fig. 5. Therefore, larger circuits are more affected by noise. We use the approximation ratio r and the probability P of obtaining the optimal solution (also called 'ground state probability' in the following) as a metric to evaluate the performance of QAOA subject to noise. A higher value of the approximation ratio or the ground state probability indicates a better performance.

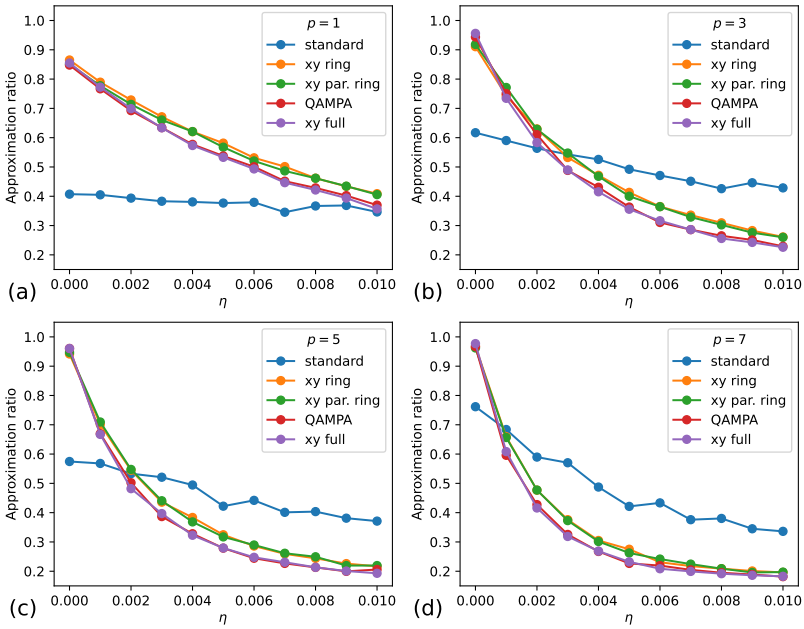


Fig. 6 The approximation ratio of the investigated QAOA instance as a function of depolarizing noise strength η for different mixers using a density matrix simulator with 8192 measurement samples. The classical optimizer COBYLA [23] was used.

Fig. 6 shows the approximation ratio of QAOA for the investigated mixers and QAOA depth p. For $p = 1$, the XY-mixers outperform the standard mixer

for all considered values of η. For larger p, the standard mixer outperforms XY-mixers starting with a specific larger depolarizing noise strength, and this threshold goes down with increasing p (around 0.003, 0.002 and 0.001 for p equal to 3, 5 and 7, respectively). One reason for such behavior is the larger size of the XY-mixers, where each of the additional gates is associated with a constant noise contribution. This explanation is consistent with the dependency on p, where circuits for a higher p, both, are larger in absolute terms and have a larger size difference between standard and XY-mixers.

The ring XY-mixer and parity ring XY-mixer show better results subject to noise compared to the QAMPA and full XY-mixer at the same noise strength. This again can be explained by their smaller sizes.

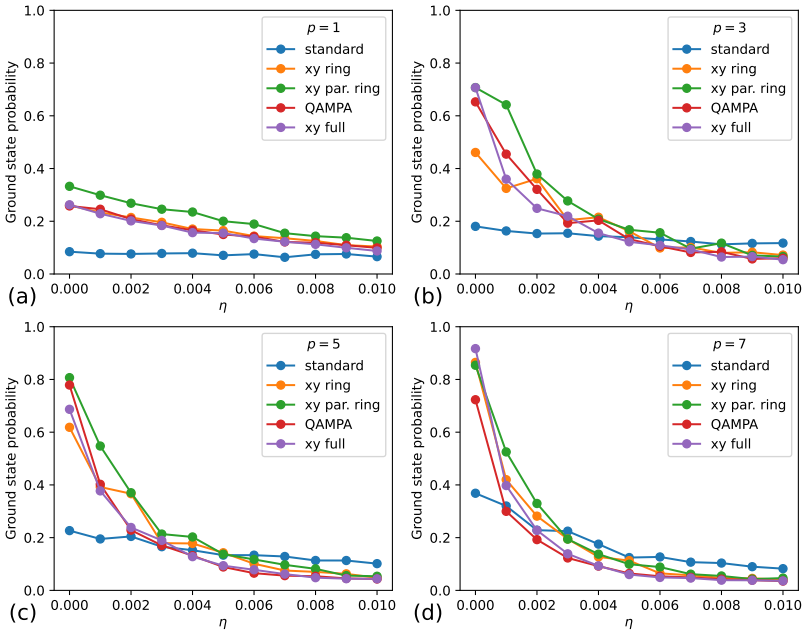


Fig. 7 The ground state probability of the investigated QAOA instance as a function of depolarizing noise strength η for different mixers using a density matrix simulator with 8192 measurement samples. The classical optimizer COBYLA [23] was used.

The results for ground state probability are shown in Fig. 7. Again, for $p = 1$, the parity ring XY-mixer outperforms all other mixers. Similar to the observations for the approximation ratio, the standard mixer is able to achieve a higher ground state probability as compared to the XY-mixers for larger p and higher noise strengths. For $p = 3$, $p = 5$ and $p = 7$, the standard mixer

yields a better result than the XY-mixers at noise strengths $0.008, 0.005$, and 0.003. Out of the XY-mixers, the parity ring XY-mixer performs better subject to noise than others.

Another reason for a worsening performance of XY-mixers under higher noise levels in terms of both approximation ratio and ground state probability is their reliance on Dicke states [17] (see section 3.1). This restriction makes them more efficient in the error-free case, but noise occurring during the circuit execution can gradually destroy the Dicke state, thus violating the algorithm's assumptions. Moreover, the standard mixer requires an equal superposition as an initial state that can be prepared efficiently with a linear number of single-qubit gates, while the Dicke state needed as an input for XY-mixers must be prepared by more complex gate sequences, including multiple layers of two-qubit gates [24].

5 Conclusion

The present paper contains a detailed study of portfolio optimization using different versions of the quantum approximate optimization algorithm (QAOA). The portfolio optimization problem is formulated as a quadratic binary optimization problem with a constraint corresponding to a given number B (budget) of assets to be chosen from a list of n assets (Sec. 2). We discuss all the technical aspects which are necessary to run different versions of the QAOA algorithm for solving the portfolio optimization problem with good performance: first, the magnitude of the penalty factor A (needed to implement the constraint in case of the standard mixer) should not be chosen larger than necessary (see Sec. 2.3). Furthermore, we propose to appropriately scale the cost function F such as to align the spectra of F and of the mixer M with each other. Of crucial importance are the initial values of the circuit parameters $\vec{\gamma}$ and $\vec{\beta}$ fed into the classical optimization routine: similarly to [7], we perform the optimization for increasing values of the QAOA depth p, using an extrapolation method to deduce initial values from the optimized values of $\vec{\gamma}$ and $\vec{\beta}$ obtained previously for depth $p-1$. Additionally (to avoid getting trapped in a local minimum), we perform, at each depth p, three additional optimization runs with initial values obtained from a linear ansatz, a quadratic ansatz, and adding zero angles (to ensure monotonicity, see Sec. 3.2.3). Concerning the parameter landscape $\langle \hat{F} \rangle_{\gamma,\beta}$ at depth $p = 1$, we observe several local minima and show a method for selecting one which is most suitable for extrapolation towards larger p (see Sec. 3.2.2). Finally, the proper choice of the classical optimization method depends on whether the simulation takes into account noise: in the ideal, noise-free case (using the statevector simulator), we use a gradient-based method (e.g. SLSQP), whereas more robust, gradient-free methods like Nelder-Mead or COBYLA are preferable in the presence of measurement noise due to a finite number of shots (see Sec. 3.2.4).

The performance of different mixers in solving 20 randomly chosen instances of the portfolio optimization problem with $n = 5$ and $n = 10$ assets

is investigated in Sec. 4.1. Not surprisingly, the XY mixers are superior to the standard mixer, since they restrict the optimization to the subspace of 'feasible' portfolios (i.e., to those that satisfy the budget constraint). The best results are achieved by the full XY mixer and the QAMPA (quantum alternate mixer-phase ansatz, see Sec. 3.1), which differs from the full XY mixer in the ordering of mixing and phase-separating operators applied to all pairs of qubits. For selecting $B = 5$ out of $n = 10$ assets with QAOA depth $p = 7$, both find the optimal portfolio with probability $P \simeq 0.6$. Their better performance as compared to the two ring mixers is attributed to their higher degree of symmetry with respect to qubit permutations. Moreover, we find that the measurement noise (with 1000 shots) does not significantly reduce the performance as compared to the ideal case corresponding to an infinite number of shots, except in cases with extremely high approximation ratio (close to about 99%).

When considering noise, the XY-mixers are in general demonstrated to be more sensitive than the standard mixer. This underscores the importance of considering the noise levels expected on the target quantum hardware when selecting the best variety of QAOA. From our specific results, purely noise-free analysis demonstrates a clear superiority of XY-mixers. However, this finding is only valid for sufficiently low noise levels, and the decision should be reverted if the actual available quantum hardware is more noisy.

Concerning the performance dependency on external factors we find clear connections with the statistics of the optimized instance, namely the variance of the correlation's and return's distributions. Broader distributions lead to a more pronounced energy landscape, simplifying the classical optimization step. Similar behaviour could be found for other optimization problems. Therefore, when benchmarking QAOA's performance the instances should be carefully picked regarding their statistics.

Supplementary information. The files 'DAX30_Daily_Covar_5y.csv' and 'DAX30_Daily_Returns_5y.csv' attached to this submission as ancillary files contain the annualized covariance matrices and returns for the German DAX 30, calculated from data between 2016 and 2021 as described in Sec. 2.1.

Acknowledgements. This work is funded by the Ministry of Economic Affairs, Labour and Tourism Baden Württemberg in the frame of the Competence Center Quantum Computing Baden-Württemberg (project 'QORA').

A Decomposition of mixer operators

Here, we show that using the reshuffled XY mixer reduces the number of CNOT gates by a factor 3/4 as compared to the full XY mixer. For this purpose, Fig. 8 shows quantum circuit representations of the two-qubit operations applied to a given pair of qubits (i, j) during a single QAOA iteration step. The full XY mixer, see Eq. (22), contains the circuits shown in Fig. 8a) and b), amounting to 4 CNOT gates for each pair of qubits. In contrast, the reshuffled XY mixer, see Eq. (23), only requires 3 CNOT gates, see Fig. 8c).

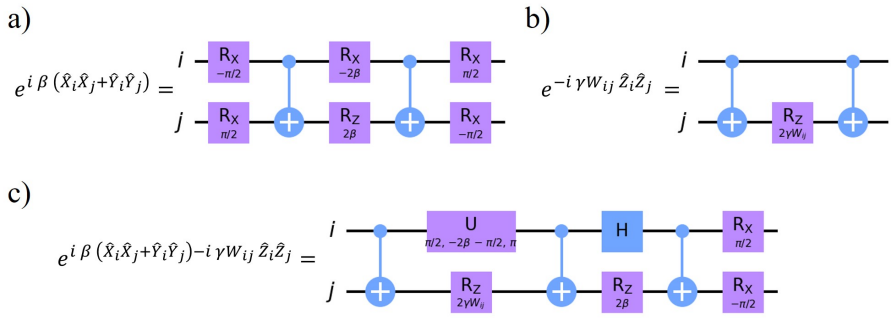


Fig. 8 Quantum circuits of the two-qubit operations a) $e^{i\beta(\hat{X}_i\hat{X}_j+\hat{Y}_i\hat{Y}_j)}$ with 2 CNOT gates, b) $e^{-i\gamma W_{ij}\hat{Z}_i\hat{Z}_j}$ with 2 CNOT gates and c) $e^{i\beta(\hat{X}_i\hat{X}_j+\hat{Y}_i\hat{Y}_j)-i\gamma W_{ij}\hat{Z}_i\hat{Z}_j}$ with 3 CNOT gates. Apart from the CNOT gates, the following single-qubit gates are used: H (Hadamard gate), $R_X(\phi) = e^{-i\frac{\phi}{2}\hat{X}}$, $R_Z(\phi) = e^{-i\frac{\phi}{2}\hat{Z}}$ and the general single-qubit gate $U(\theta,\phi,\lambda) = R_Z(\phi)R_Y(\theta)R_Z(\lambda)$

B Data for covariances and returns

The covariances and returns of the portfolio optimization instance used in Sec. 4.3 are provided in the tables below.

LIN.DE	BAYN.DE	VNA.DE	MTX.DE	MUV2.DE
0.26801758	-0.11724968	0.2109537	0.21523688	0.1128935

Table 1 Return vector μ_i for 5 assets chosen from the German DAX 30.

	LIN.DE	BAYN.DE	VNA.DE	MTX.DE	MUV2.DE
LIN.DE	0.21117209	0.03030933	0.00941277	0.02972179	0.02922818
BAYN.DE	0.03030933	0.08796365	0.01833403	0.0465302	0.04069187
VNA.DE	0.00941277	0.01833403	0.04971719	0.02303918	0.02051608
MTX.DE	0.02972179	0.0465302	0.02303918	0.13717214	0.05638483
MUV2.DE	0.02922818	0.04069187	0.02051608	0.05638483	0.06765634

Table 2 Covariance matrix σ_{ij} for 5 assets chosen from the German DAX 30.

References

[1] Farhi, E., Goldstone, J., Gutmann, S.: A Quantum Approximate Optimization Algorithm. http://arxiv.org/pdf/1411.4028v1

[2] Hadfield, S., Wang, Z., O'Gorman, B., Rieffel, E.G., Venturelli, D., Biswas, R.: From the quantum approximate optimization algorithm to a quantum

alternating operator ansatz. Algorithms **12**(2) (2019). https://doi.org/10.3390/a12020034

[3] Shor, P.W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. SIAM Journal on Computing **26**(5), 1484–1509 (1997) https://arxiv.org/abs/https://doi.org/10.1137/S0097539795293172. https://doi.org/10.1137/S0097539795293172

[4] Harrow, A.W., Hassidim, A., Lloyd, S.: Quantum algorithm for linear systems of equations. Phys. Rev. Lett. **103**, 150502 (2009). https://doi.org/10.1103/PhysRevLett.103.150502

[5] Bittel, L., Kliesch, M.: Training variational quantum algorithms is NP-hard. Phys. Rev. Lett. **127**, 120502 (2021). https://doi.org/10.1103/PhysRevLett.127.120502

[6] Guerreschi, G.G., Matsuura, A.Y.: QAOA for max-cut requires hundreds of qubits for quantum speed-up. Sci. Rep. **9**, 6903 (2019)

[7] Zhou, L., Wang, S.-T., Choi, S., Pichler, H., Lukin, M.D.: Quantum approximate optimization algorithm: Performance, mechanism, and implementation on near-term devices. Phys. Rev. X **10**, 021067 (2020). https://doi.org/10.1103/PhysRevX.10.021067

[8] Mansini, R., Speranza, M.G.: Heuristic algorithms for the portfolio selection problem with minimum transaction lots. Eur. J. Oper. Res. **114**, 219 (1999)

[9] Detemple, J.: Portfolio selection: A review. Journal of Optimization Theory and Applications **161**(1), 1–21 (2014)

[10] Mansini, R., Ogryczak, W., Speranza, M.G.: Linear and Mixed Integer Programming for Portfolio Optimization. Springer International Publishing, Cham (2015). https://doi.org/10.1007/978-3-319-18482-1

[11] Kalayci, C.B., Ertenlice, O., Akbay, M.A.: A comprehensive review of deterministic models and applications for mean-variance portfolio optimization. Expert Systems with Applications **125**, 345–368 (2019). https://doi.org/10.1016/j.eswa.2019.02.011

[12] Hodson, M., Ruck, B., Ong, H., Garvin, D., Dulman, S.: Portfolio rebalancing experiments using the Quantum Alternating Operator Ansatz (2019)

[13] Egger, D.J., Gambella, C., Marecek, J., McFaddin, S., Mevissen, M., Raymond, R., Simonetto, A., Woerner, S., Yndurain, E.: Quantum computing for finance: state of the art and future prospects. IEEE Transactions

on Quantum Engineering **1**, 1–24 (2020). https://doi.org/10.1109/TQE.2020.3030314

[14] Baker, J.S., Radha, S.K.: Wasserstein Solution Quality and the Quantum Approximate Optimization Algorithm: A Portfolio Optimization Case Study. arXiv (2022). https://doi.org/10.48550/ARXIV.2202.06782. https://arxiv.org/abs/2202.06782

[15] Markowitz, H.: Portfolio selection. The Journal of Finance **7**(1), 77–91 (1952) https://arxiv.org/abs/https://onlinelibrary.wiley.com/doi/pdf/10.1111/j.1540-6261.1952.tb01525.x. https://doi.org/10.1111/j.1540-6261.1952.tb01525.x

[16] Wang, Z., Rubin, N.C., Dominy, J.M., Rieffel, E.G.: XY mixers: Analytical and numerical results for the quantum alternating operator ansatz. Phys. Rev. A **101**, 012320 (2020). https://doi.org/10.1103/PhysRevA.101.012320

[17] Dicke, R.H.: Coherence in spontaneous radiation processes. Physical review **93**(1), 99 (1954)

[18] LaRose, R., Rieffel, E., Venturelli, D.: Mixer-Phaser Ansätze for Quantum Optimization with Hard Constraints. arXiv (2021). https://doi.org/10.48550/ARXIV.2107.06651. https://arxiv.org/abs/2107.06651

[19] Cook, J., Eidenbenz, S., Bärtschi, A.: The quantum alternating operator ansatz on maximum k-vertex cover. In: 2020 IEEE International Conference on Quantum Computing and Engineering (QCE), pp. 83–92 (2020). https://doi.org/10.1109/QCE49297.2020.00021

[20] Fuchs, F.G., Lye, K.O., Nilsen, H.M., Stasik, A.J., Sartor, G.: Constraint preserving mixers for the quantum approximate optimization algorithm. Algorithms **15**(6), 202 (2022). https://doi.org/10.3390/a15060202

[21] Nielsen, M.A., Chuang, I.L.: Quantum Computation and Quantum Information: 10th Anniversary Edition. Cambridge University Press, Cambridge (2010). https://doi.org/10.1017/CBO9780511976667

[22] Wang, S., Fontana, E., Cerezo, M., Sharma, K., Sone, A., Cincio, L., Coles, P.J.: Noise-induced barren plateaus in variational quantum algorithms. Nature communications **12**(1), 1–11 (2021)

[23] Powell, M.J.: A direct search optimization method that models the objective and constraint functions by linear interpolation. In: Gomez, S., Hennart, J.-P. (eds.) Advances in Optimization and Numerical Analysis, pp. 51–67. Springer, Dordrecht (1994)

[24] Mukherjee, C.S., Maitra, S., Gaurav, V., Roy, D.: Preparing Dicke states on a quantum computer. IEEE Transactions on Quantum Engineering **1**, 1–17 (2020)

Limitations of optimization algorithms on noisy quantum devices

Daniel Stilck França[1,*] and Raul Garcia-Patron[2,†]

[1] *QMATH, Department of Mathematical Sciences, University of Copenhagen, Denmark*
[2] *School of Informatics, University of Edinburgh, Edinburgh EH8 9AB, UK*

(Dated: September 14, 2020)

Recent technological developments have focused the interest of the quantum computing community on investigating how near-term devices could outperform classical computers for practical applications. A central question that remains open is whether their noise can be overcome or it fundamentally restricts any potential quantum advantage. We present a transparent way of comparing classical algorithms to quantum ones running on near-term quantum devices for a large family of problems that include optimization problems and approximations to the ground state energy of Hamiltonians. Our approach is based on the combination of entropic inequalities that determine how fast the quantum computation state converges to the fixed point of the noise model, together with established classical methods of Gibbs state sampling. The approach is extremely versatile and allows for its application to a large variety of problems, noise models and quantum computing architectures. We use our result to provide estimates for a variety of problems and architectures that have been the focus of recent experiments, such as quantum annealers, variational quantum eigensolvers, and quantum approximate optimization. The bounds we obtain indicate that substantial quantum advantages are unlikely for classical optimization unless the current noise rates are decreased by orders of magnitude or the topology of the problem matches that of the device. This is the case even if the number of qubits increases substantially. We reach similar but less stringent conclusions for quantum Hamiltonian problems.

arXiv:2009.05532v1 [quant-ph] 11 Sep 2020

Quantum computation experiments have reached the classical complexity frontier, where the devices are hard or impossible to simulate with classical resources [1]. A question that has focused the attention of the quantum computation community in the last years is whether these near-term quantum computing devices, despite their inherent noise and lack of quantum error-correction, have any potential to demonstrate superiority over classical computing devices for practically relevant problems [2].

A growing body of work has suggested using near-term devices to solve both optimization problems relevant in a large portfolio of industrial applications and approximating the ground state energy of physically relevant Hamiltonian [3–8]. As opposed to quantum advantage tests based on sampling of random circuits [1], optimization has a clear practical application and at least one straightforward certification protocol of its superiority over a classical computer: whichever device outputs the state with the lowest value of the cost function or Hamiltonian energy wins. This makes them ideal candidates for a convincing practical quantum advantage demonstration. The central question that remains open is whether a quantum advantage for practical problems is at all possible, especially taking into consideration the unavoidable presence of noise.

In this article, we develop a new technique to provide sufficient conditions for when a noisy quantum device cannot significantly outperform a classical computer when it comes to optimizing a cost function given a noise model. Additionally, we develop a technique to certify that a classical algorithm outperforms a given noisy quantum device running for a certain depth or number of cycles. This is shown using a nontrivial lower bound on the achievable value of the cost function or energy by the noisy quantum device. Both techniques can be adapted to a plethora of problems, noise models and computing architectures, opening the room for analytical proofs and concrete analysis of the limitations of near-term quantum computing devices. As a demonstration of its potential, we apply them to discuss the effect of local depolarizing noise on quantum circuits used to solve classical Ising models, which are equivalent to well-know NP-complete optimization problems, and variational quantum eigensolvers for local Hamiltonians problems [9–12]. We also generalize the technique to include noise models beyond depolarizing noise and develop a continuous time version, which allows us to study the effect of noise on quantum annealers [13, 14].

Our analysis suggests that, for classical optimization problems, speedups may be impossible with current noise rates and are unlikely even at the boundary of fault-tolerant errors rates, especially when the architecture does not match the topology of the problem. The problems where near-term devices could have an opportunity window are most probably quantum many-body physics motivated, where the architecture topology has been tailored-made to match the Hamiltonian of the problem and the correlation length in the system is bounded. Whether examples of this kind exist and cannot be simulated efficiently classically is, up to our knowledge, an important and non-trivial open problem. Furthermore, our bounds show that it is unlikely that increasing the number of qubits of current quantum devices without reducing the noise of the gates will lead to quantum speedups.

* dsfranca@math.ku.dk
† rgarcia3@exseed.ed.ac.uk

I. AN INTRODUCTION TO THE TECHNIQUE

In this work we are interested in solving optimization problems that can be recast as the minimization of a cost function $\mathrm{tr}\,(\rho H)$, where ρ is a quantum state of n qubits, which therefore includes sub-problems defined over probability distributions on bit strings, and H is a Hermitian operator characterizing the cost function of the problem. Many well-studied optimization problems, such as MAX-CUT, can be recast as optimizing classical Ising Hamiltonians [15], a connection exploited in most of the proposal of quantum algorithms for those problems and behind the architecture design of quantum annealers. In the case of a many-body ground-state problem, H would correspond to the Hamiltonian of the system [9].

Every Hamiltonian H has an associated family of Gibbs states, $\sigma_\beta = e^{-\beta H}/\mathcal{Z}_\beta$, where $\mathcal{Z}_\beta = \mathrm{tr}\left(e^{-H}\right)$ is the partition function and β is the inverse temperature parameter, i.e, σ_β corresponds to the thermal equilibrium state with temperature $1/\beta$. The infinite temperature case $\beta = 0$ corresponds to a fully mixed state of n qubits, the case $\beta = \infty$ provides the ground-state of H, which for optimization problems has its support over the set of solutions.

In what follows we present an approach to construct a sufficient condition that guarantees that a noisy quantum computation solving a given optimization problem does not perform better than its classical counterpart. As sketched in Figure 1, the techniques combines relative entropy convergence methods (subsection I A), mirror descent (subsection I B) and results on efficient algorithms for Gibbs state sampling. The Gibbs sampling result being dependent on the specific problem the quantum algorithm is solving, we delay its presentation to the sections where specific examples are presented (sections II and III). In addition, we construct in subsection I C a technique that allows to certify that a classical algorithm does better than a given noisy device.

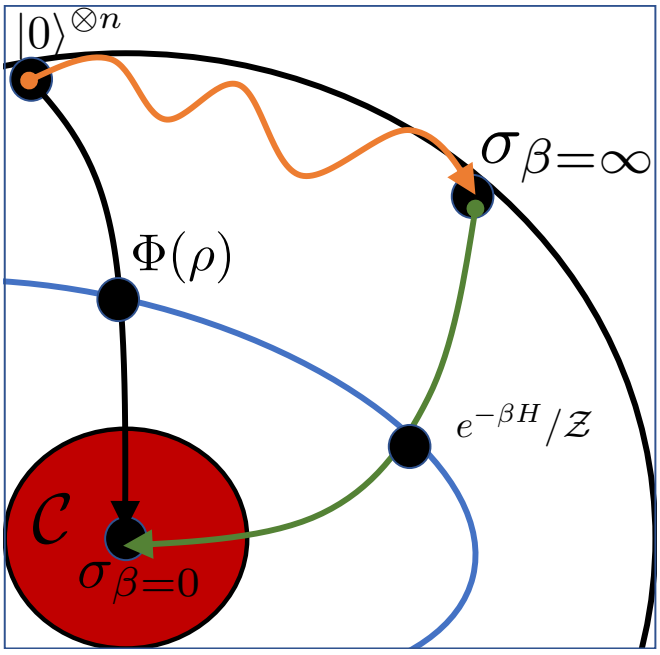


FIG. 1. The noisy quantum computer initialized at $|0\rangle^{\otimes n}$ attempts to follow a path (orange arrow) over the hyper-surface of the hyper-Bloch-sphere of n qubits until reaching the targeted solution of the problem with Hamiltonian H, $\sigma_{\beta=\infty}$, which corresponds to the Gibbs state at zero temperature. The noise imposes an alternative path (black arrow) that directs the computation toward the fixed point of the noise, the maximally mixed state (Gibbs state with $\beta = 0$). Mirror descent allows us to associate to every intermediate state of the computation $\Phi(\rho)$ an equivalent Gibbs state $e^{-\beta H}/\mathcal{Z}$ that provides the same cost function estimate as $\Phi(\rho)$ up to relative error ϵ. Therefore, there is a one-to-one correspondence between the noisy computation path and an equivalent Gibbs state path (green arrow). More precisely, there is a manifold of states with approximately the same energy as $e^{-\beta H}/\mathcal{Z}$ (blue line). For many classes of problems, for any $\beta \leq \beta_c$ there will exist an efficient classical algorithm that samples from the Gibbs state, demarcating a region of efficient classical simulation (red circle). Relative entropy convergence methods can then be used to certify that the noisy quantum computer has entered the classically efficient region.

A. The effect of noise

We will assume that we wish to implement a quantum circuit on n qubits consisting of D layers of unitaries $\mathcal{U}_1, \ldots, \mathcal{U}_D$ acting on n qubits. However, due to imperfections in the implementation, they are interspersed by a quantum channel T that describes the noise, which for simplicity we will assume to be local and uniform over the circuit and time-invariant. The output of the noisy quantum circuit is then described by the quantum channel $\Phi = \bigcirc_{t=1}^{D} T \circ \mathcal{U}_t$. We consider noise models that drive the system to a fixed point σ s.t. $T(\sigma) = \sigma$, which is the generic case. For our framework, it will be important to quantify how fast this happens when measured in the relative entropy $D(\rho\|\sigma)$. The following lemma exemplifies this for local depolarizing noise.

Lemma 1. *For T being a layer of $1-$qubit depolarizing channels on n qubits with depolarizing probability p, the channel $\Phi = \bigcirc_{t=1}^{D} T \circ \mathcal{U}_t$ satisfies*

$$D(\Phi(\rho)||\sigma) \leq (1-p)^{2D} D(\rho||\sigma) \leq (1-p)^{2D} n \qquad (1)$$

for all states ρ [16].

B. An alternative path

The key tool to achieving our goal is to decouple our result from the simulation of the noisy quantum circuit. We achieve this with mirror descent [17, 18], which was recently used in several results in quantum computing [19–23], and the variational formulation of the relative entropy [24].

Lemma 2. *Let $\Phi(\rho)$ be the output of a noisy quantum device, $\sigma = I/2^n$, and the relative entropy $D(\Phi(\rho)||\sigma) = n - S(\Phi(\rho))$, where $S(\Phi(\rho)) = -tr((\Phi(\rho)\,(\log(\Phi(\rho)))$. Then for any Hamiltonian H there is a $\lambda \in [0, D(\Phi(\rho)||\sigma)]$ such that the state*

$$\widetilde{\sigma} = exp\left(-\frac{\lambda}{\|H\|\epsilon}H\right)/\mathcal{Z} \tag{2}$$

satisfies:

$$tr\left(H\left(\widetilde{\sigma} - \Phi(\rho)\right)\right) \leq \|H\|\epsilon. \tag{3}$$

This means that for every (noisy) quantum circuit output $\Phi(\rho)$, solving the optimization problem with cost function H, one can assign a Gibbs state of temperature

$$\beta = \frac{\lambda}{\|H\|\epsilon} \tag{4}$$

which provides a solution that approximates with $\epsilon\|H\|$ accuracy that of the quantum circuit. Note that typically an absolute error of $\epsilon\|H\|$ reflects a relative error of ϵ of the energy. Results like Lemma 1 allow us to bound λ given the depth.

In subsection VI C 1 we argue why we should take ϵ to be approximately $10^{-1} - 10^{-2}$ for the results we present here, and in Sec. VIII we generalize to noisy channels with fixed points other than maximally mixed qubits. It is important to stress that our analysis is fundamentally different from analysing how close the output state is to the maximally mixed in trace distance, as discussed in more detail in Sec. VI A 1 of the supplemental material.

C. Certifying classical superiority

Exploiting the variational formulation of the relative entropy it possible to lower-bound the energy of the output of the noisy quantum device:

Proposition 1. *Let $\Phi(\rho)$ be the output of a noisy quantum device and $\sigma = I/2^n$ the fixed point of the noise. Then $\Phi(\rho)$ satisfies:*

$$\begin{aligned} &tr(\Phi(\rho)H) \\ &\geq \sup_{\beta>0} \beta^{-1}\left(n - \log(tr\left(e^{-\beta H}\right)) - D(\Phi(\rho)||\sigma)\right). \end{aligned} \tag{5}$$

Thus, if we can approximate the partition function for some values of β and bound the relative entropy with a relative entropy convergence method, we immediately obtain a lower bound on the achievable energy of the circuit through (5), which can be used to certify that the output of a given classical method outperforms the noisy quantum device solving the same problem over a given number of computation cycles D. In Sec. VI B of the supplemental material we present a detailed proof of this proposition and its generalization beyond the maximally mixed state, while also discussing the scaling of the bound in more detail. But the best way of obtaining an intuitive grasp of its consequences is to perform a Taylor expansion of $f(\beta) = \log(\mathrm{tr}\left(e^{-\beta H}\right))$ around $\beta = 0$. Assuming that f is analytical for a neighbourhood around 0, we see that the bound in Eq. (5) reads that for any β

$$\mathrm{tr}\left(\Phi(\rho)H\right) \geq \mathrm{tr}\left(\sigma H\right) - \mathcal{O}(\beta f^{(2)}(0)) - \beta^{-1}D(\Phi(\rho)||\sigma),$$

where $f^{(2)}(0) = 2^{-n}\mathrm{tr}\left(H^2\right) - 2^{-2n}\mathrm{tr}\left(H\right)^2$. One can readily check that for κ-local Hamiltonians we have $f^{(2)}(0) = \mathcal{O}(\kappa n)$. We conclude that if $D(\Phi(\rho)||\sigma) \leq \epsilon^2 n$, then picking $\beta = \epsilon$ yields $\mathrm{tr}\left(\Phi(\rho)H\right) \geq \mathrm{tr}\left(\sigma H\right) - \mathcal{O}(\kappa\epsilon n)$. That is, the output energy density will be close to the one of the fixed point.

II. CLASSICAL ISING HAMILTONIAN PROBLEMS

By mapping NP-complete optimization problems such as MAXCUT to Ising Hamiltonian problems [15], we aim at finding the ground state and energy of the Ising Hamiltonian

$$H_I = -\sum_{i\sim j} a_{i,j} Z_i Z_j - \sum_i b_i Z_i. \tag{6}$$

defined on a graph $G = (V, E)$ with maximum degree Δ with n vertices, where $i \sim j$ stands for the edge of graph G between vertices i and j and $a_{i,j}, b_i$ real. Monte Carlo algorithms to sample from σ_β for Δ-regular graphs have been extensively studied, see e.g. [25, Chapter 15.1]. There is a vast literature dedicated to determining for which range of β it is possible to sample efficiently depending on the structure of the coupling coefficients and graph. To the best of our knowledge, the state of the art general condition that asserts rapid mixing for Ising models is the recent [26, Theorem 11], which reads that as long as

$$\beta\|A\| < 1, \tag{7}$$

where $(A)_{i,j} = a_{i,j}$ is the $n \times n$ coupling matrix, it is possible to approximately sample from the Gibbs state σ_β in polynomial time. For instance, if we have a Δ-regular graph and $|a_{i,j}| \leq 1$, note that $\|A\| \leq \Delta$ and the bound above yields $\beta < \Delta^{-1}$. On the other hand, for the SK model, where we have a complete interaction graph and $a_{i,j} \sim \mathcal{N}(0, n^{-1})$, [26] shows that rapid mixing still holds at $\beta < 1/4$.

A. Bounding the number of cycles

Let us now bound the maximum depth for MAXCUT on a noisy computer and Δ-regular graph. Let's consider a circuit of depth D, where f_1 is the fraction of layers of single qubits gates with error p_1, f_2 the fraction of layer of two-qubit gate of error p_2, and p_m the probability of

error of the measurements. Replacing every ideal component by its noisy counterpart followed by local depolarizing noise with rate p_i, combining Lemma 2, eq. (4), the bound eq. (7), and the approximation $-\log(1-p) \simeq p^{-1}$ for small p we obtain:

Proposition 2. *Let H_I be an Ising Hamiltonian as in Eq. (6). Moreover, let $\Phi(\rho)$ be the output of a noisy quantum circuit on n qubits consisting of D layers of unitaries. Suppose further that the circuit consists of a fraction of f_1 layers of one-qubit gates and f_2 layers of 2-qubit gates, affected by local depolarizing noise with probability p_1, p_2 respectively and the measurement is affected with depolarizing probability p_m. Then for $D \geq D_{\max}$ with*

$$D_{\max} = \frac{\log \epsilon^{-1} + \log\left(\|H_I\|^{-1}\|A\|n\right) - p_m}{2(f_1 p_1 + f_2 p_2)}, \qquad (8)$$

there is a Gibbs state $\widetilde{\sigma}$ satisfying

$$tr\left(H_I\left(\widetilde{\sigma} - \Phi(\rho)\right)\right) \leq \epsilon \|H\|.$$

that can be sampled from in polynomial time.

For instance, for Δ regular graphs with $a_{i,j} \in \{0,1\}$ the energy of the ground state is of order Δn. As $\|A\| \leq \Delta$, up to a relative error of ϵ we can sample from a comparable state after at most $\log(\epsilon^{-1})/2(f_1 p_1 + f_2 p_2)$ depth.

We have now everything at hand to make some estimates on the capability for realistic architectures to solve practical problems. We will assume the values $p_1 = 1.6 \times 10^{-3}$, $p_2 = 6.2 \times 10^{-3}$ and $p_m = 3.8 \times 10^{-2}$ taken from the recent experiment [1]. A simple calculation using eq. (8) with $\epsilon = 10^{-1}$ shows that those numbers limit the depth of any quantum circuit to $D_c \approx 300$ for circuits dominated by $2-$qubit gate layers and $D_c \approx 700$ for $1-$qubit gate dominated circuits.

Most current quantum computing architectures have a planar design. However, usually interesting optimization problems are defined over non-planar graphs, which imposes the need for SWAP circuits that consume a lot of cycles of computation. For example, the SK model studied in [1] requires $7n$ layers cycles of computation per application of the QAOA unitary and implements 3 layers of QAOA. The bound in Eq. (8), for which we have $\beta_c \geq 1/4$, predicts a limit of roughly 20 qubits before their systems lose any potential advantage at three rounds. Indeed, experimental observations show that for systems of size 17 their device is not better than random guessing.

Furthermore, it is also possible to approximately compute the partition function for the aforementioned β range. We evaluated the bound in Eq. (5) for several instances of the SK model on up to 30 qubits and consistently observed that already depths $D_c \approx 150$ are outperformed by Gibbs states in the polynomial-time range. Translating this bound to number of qubits, it predicts that our classical method should outperform the noisy quantum computer with more than 10 qubits. In Sec. VI D of the supplementary material we explain how we perform such computations in more detail and that the variational bound allows for comparison to any other algorithm to approximately solve optimization problems beyond Gibbs sampling. Also note that quantum computers of this size or smaller can be easily simulated on a laptop.

We remark that equation (8) has the advantage of providing a rigorous guarantee for arbitrary system sizes and instances, where (5) provides sharper estimates for specific instances of a problem. Thus, both methods seem to complement each other in providing a good bound on the range of cycles of computation a noisy quantum computers can sustain before it is outperformed by polynomial time classical methods.

Even for optimization problems on sparser graphs, such as the MAXCUT problems also studied in [1], the depth of every QAOA layer is proportional to n. From our previous discussion, we readily infer that, as long as the depth of each QAOA layer scales with n, also for sparser problems the noise rate would have to decrease roughly two orders of magnitude, well below the fault-tolerant threshold, for quantum computers to have a chance of being competitive with classical methods. This is because current heuristic methods yield good results for graphs with numbers of nodes in the $10^3 - 10^4$ range [27, 28] and state of the art SDP solvers can solve 10^4-node instances in seconds on a laptop [29].

III. VARIATIONAL QUANTUM EIGENSOLVERS

Variational quantum eigensolvers (VQE) are one of the most studied families of quantum algorithms for near-term quantum devices [9–11]. This family of hybrid classical-quantum algorithms supplements a traditional variational technique on a classical computer with a call to a quantum circuit that encodes the quantum state and measures its energy. The technique allows to circumvent the cost of encoding and evolving quantum states in a traditional classical computer, but suffers from similar limitation as any traditional variational approach, among them the fact that we restrict the set of solution to a manifold of the Hilbert space. Here we exemplify how to derive limitations on VQE for finding the ground state of local Hamiltonians, although it is possible to extend the results here to other classes of problems.

There are several results and techniques for the classical approximation of quantum Gibbs states [30–38]. For instance, in the recent [31, Corollary 21] the authors show that on a Hamiltonian on a d dimensional lattice and κ local interactions of strength at most J, for all inverse temperatures $\beta \leq \beta_c$ with

$$\beta_c \geq \left(5e\kappa d^{\kappa} J\right)^{-1}, \qquad (9)$$

we can approximately compute the partition function of $\mathcal{Z}_\beta = e^{-\beta H}$ up to a multiplicative error δ in $\mathcal{O}(n^{\log(n\delta^{-1})})$

time. Let us normalize $\|H\|$ in such a way that $\kappa Jn \leq \|H\| \leq 2\kappa Jn$. By our choice of normalization of $\|H\|$ we obtain the following proposition:

Proposition 3. *Let H be a local Hamiltonian on a d dimensional lattice with κ local interactions of strength at most J. Moreover, let $\Phi(\rho)$ be the output of a noisy quantum circuit on n qubits consisting of D layers of unitaries interspersed with local depolarizing channels with depolarizing rate p. Then for $D \geq D_{\max}$ with*

$$D_{\max} = \frac{\log\left(20e\left(d^{\kappa}\epsilon\right)^{-1}\right)}{2p} \tag{10}$$

there is a Gibbs state $\widetilde{\sigma}$ satisfying

$$tr\left(H\left(\widetilde{\sigma} - \Phi(\rho)\right)\right) \leq 2\kappa Jn\epsilon.$$

whose energy w.r.t. H can be computed in quasi-polynomial time.

This upper-bound on the maximum depth of a VQE algorithm implies also a limitation on the maximum correlation length ξ of ground states we can prepare with a near-term device. As detailed in Section VII of the supplementary material, a light cone argument can be used to argue that whenever $D < \xi$ the quantum circuit will provide a solution that is far in trace distance from the actual ground state. Thus, the noise imposes limits on the correlation length present in the quantum ansatz generated by near-term devices.

Comparing the bound in Eq. (8) and Eq. (10), we see that they exhibit the same scaling. Thus, we believe that this warrants similar conclusions: as long as the depth of a VQE layer scales with the system size, substantially decreasing the noise is more important than increasing system size. However, as quantum Hamiltonians are inherently more complex than Ising problems, and also less studied, smaller system sizes and higher noise rates might be tolerable for a quantum advantage demonstration.

IV. NOISY QUANTUM ANNEALERS

Quantum annealers have been proposed as an alternative architecture to the circuit model of computation to solve classical Ising problems [39]. They follow a quantum computation approach to optimization of Ising Hamiltonians aiming at preparing the ground state $|\psi_I\rangle$ of the classical Hamiltonian H_I of eq. (6) adiabatically. We start by first preparing the ground state of $H_0 = -\sum \Gamma_i X_i$, which is easily seen to be the state $|\psi_0\rangle = |+\rangle^{\otimes n}$. Then by adiabatically shifting from the Hamiltonian H_0 toward H_I, through the family of transverse-field Ising Hamiltonians

$$H_s = g_I(s)H_I + g_0(s)H_0$$

where g_0, g_I are smooth functions of s s.t. $g_0(0) = A, g_I(0) = 0$ and $g_0(T) = 0, g_I(T) = B$, we converge to $|\psi_I\rangle$ by evolving under the Hamiltonian $H_s$over time T.

The spectral gap of H_s along the curve determines how slow the evolution has to be to guarantee a good solution. Furthermore, to the best of our knowledge, most rigorous theorems yield a runtime that scales at least linearly in the number of qubits and inverse quadratically on the gap [40]. There are, however, heuristic bounds that only scale quadratically inverse with the gap [41]. We refer to [14] for a discussion of adiabatic quantum computation and these issues.

A. A continuous time contraction bound

Quantum adiabatic computation being a continuous time computational model, we need to generalize Lemma 1 to this framework. Therefore, we assume that the evolution of the system at time s is described by the time-dependent Lindbladian

$$\mathcal{S}_s(\rho) = -i[\rho, H_s] + \mathcal{L}(\rho).$$

Here $\mathcal{L}$ is a purely dissipative term, which we assume to be time-independent and that is supposed to model the thermalization of the system. The time evolution of the system at time t is described by the quantum channel $\mathcal{T}_t$ given by

$$\mathcal{T}_t = \lim_{n\to\infty} \bigcirc_{l=1}^{n} e^{\frac{t}{n}\mathcal{S}_{t_l}}.$$

We will consider a model of local noise combining amplitude damping, dephasing and control error for which the fixed point of the Lindbladian $\mathcal{S}_s$ is the product state $\sigma = \sigma_{\gamma}^{\otimes n}$, where $\sigma_\gamma = (e^{\gamma} + e^{-\gamma})^{-1} e^{-\gamma Z}$.

The continuous time version of the relative entropy decay of Eq. (1) is that $\mathcal{L}$ satisfies a *modified logarithmic Sobolev inequality* (MLSI) with constant $\alpha > 0$ [42, 43]. This implies that $\mathcal{L}$ satisfies

$$D\left(e^{t\mathcal{L}}(\rho)||\sigma\right) \leq e^{-\alpha t} D\left(\rho||\sigma\right). \tag{11}$$

for all $n-$qubit states ρ and times $t \geq 0$ and a state $\sigma > 0$.

Exploiting the generalization to an arbitrary fixed point of Lemma 2 in Section VIII and its adaptation to the continuous evolution setting in Section IX, both in the supplemental material, we obtain:

Theorem 1. *Let $\mathcal{S}_s(\rho) = -i[\rho, H_s] + \mathcal{L}(\rho)$ be a time-dependent Linbladian s.t. $\mathcal{L}$ satisfies MLSI with constant $\alpha > 0$. Then the evolution from time 0 to t satisfies $D(\mathcal{T}_t(\rho)||\sigma) \leq e^{-\alpha t} D\left(\rho||\sigma\right) + X(t, H_s, \sigma)$, where*

$$X(t, H_s, \sigma) = \int_0^t d\tau e^{-\alpha(t-\tau)} \|\sigma^{-\frac{1}{2}}[H_t, \sigma]\sigma^{-\frac{1}{2}}\|. \tag{12}$$

B. Noise model

For current implementations of annealers, amplitude damping, control errors and dephasing are the main sources of noise [39]. As shown in the supplementary material in Sec. IX B, assuming an amplitude damping rate of r_1, a dephasing r_2 and r_3 as the standard deviation on the control error on the parameter b_i and Γ_i, one can show that the fixed point reads $\sigma = \sigma_\gamma^{\otimes n}$, where

$$\sigma_\gamma = \begin{pmatrix} \frac{r_1+r_3}{r_1+2r_3} & 0 \\ 0 & \frac{r_3}{r_1+2r_3} \end{pmatrix}. \tag{13}$$

Thus, we see that when amplitude damping dominates the control error, i.e, the regime $r_1 \gg r_3$, the fixed point is essentially the $|0\rangle$ state, where for $r_3 \gg r_1$ we have a state close to maximally mixed state. One can also obtain a MLSI for $\mathcal{L}$ with $\alpha(r_1, r_3) = r_1 + 2r_3$ [44, Theorem 19].

C. Example: linear adiabatic path

Let us exemplify how inequality (12) behaves for the path $H_s = \frac{s}{T}H_0 + \left(1 - \frac{t}{T}\right) H_1$ if we evolve from the initial state $\rho = |+\rangle\langle+|^{\otimes n}$ under $\mathcal{S}_s$ from 0 to T.

Using the noise models given in Eq. (13), we conclude that the relative entropy after evolving the system for time T is bounded by

$$D(\mathcal{T}_T(|+\rangle\langle+|^{\otimes n})||\sigma) \leq n f(\gamma, r, T, \overline{\Gamma}) \tag{14}$$

where

$$f(\gamma, r, T, \overline{\Gamma}) = e^{-rT}\log(2\cosh(\gamma)) + \frac{2\sinh(\gamma)\left(1 - e^{-rT}rT - e^{-rT}\right)}{r^2 T}\overline{\Gamma}, \tag{15}$$

and $\gamma = \frac{1}{2}\log\left(1 + \frac{r_1}{r_3}\right)$ and $\overline{\Gamma} = n^{-1}\sum_i \Gamma_i$. We refer to Sec. IX C of the supplementary material for a detailed derivation.

Using those results we conclude that we need to simulate the classical Gibbs state

$$\widetilde{\sigma} \propto \exp\left(-\gamma \sum_{i=1}^{n} Z_i - \frac{4\lambda}{\|H_I\|\epsilon} H_I\right)$$

for $\lambda \leq f(\gamma, r, T, \overline{\Gamma})n$ to approximate the energy of the output $\mathcal{T}_t(|+\rangle\langle+|^{\otimes n})$ of the noisy quantum computer up to ϵH_I. Note that $\widetilde{\sigma}$ is just the Gibbs state of a classical Ising model.

Thus, the same mixing time of Ising used in section II can be used here again. Note that the mixing time bounds in Eq. (7) is independent of the external field and that adding the term $\sum_{i=1}^{n} b_i' Z_i$ corresponding to the fixed point amounts to changing the external field strength. Translating this bound to our setting, we see we can sample from $\widetilde{\sigma}$ in polynomial time for

$$f(\gamma, r, T, \overline{\Gamma}) \leq \frac{4\|A\|n}{\|H_I\|}\epsilon. \tag{16}$$

This puts stringent constraints on the potential of noisy adiabatic quantum computers to outperform classical computers optimizing classical Ising Hamiltonians. We exemplify the behaviour of the bound in Fig. 2.

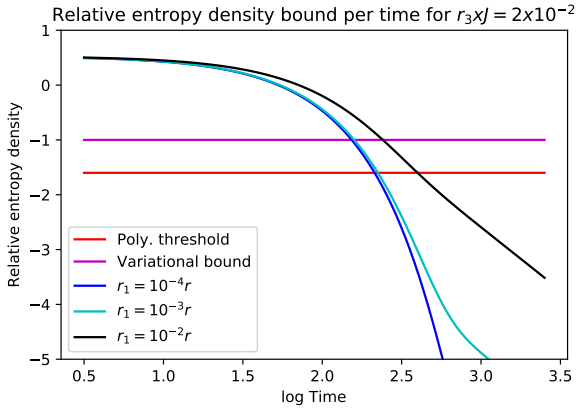


FIG. 2. Relative entropy density $D(\mathcal{T}_T(|+\rangle\langle+|^{\otimes n})||\sigma)/n$ as bounded by Eq. (14) for a control error noise rate of $r_3 \times J = 2 \times 10^{-2}$. The polynomial threshold line depicts when the device provably does not significantly outperform polynomial methods and is based on Eq. (16). The variational bound is an extrapolation based on various instances of up to 30 qubits and is based on Eq. (18) of the supplemental material. The graph that models the interactions available in the device has maximum degree $\Delta = 6$. The interaction strengths $a_{i,j}, b_i, \Gamma_i$ of the device are unit-less and ≈ 1, where the modulation functions satisfy $g_i(s) \in [10^{-2}, 10]$, measured in GHz. Specifications of the device [45, Fig. 3.1, Fig. 3.2] suggest that the standard deviation of the control noise for $a_{i,j}, b_i, \Gamma_i$ is not constant in time, but one can see that their product with $g_i(s)$ remains in a reasonable interval of values always larger than 10MHz. Therefore, we choose to re-scale the problem to $g_0(0) = g_1(T) = 1$ and $r_3 \times J \simeq 2 \times 10^{-2}$, J the maximum interaction strength. Unfortunately, the value of r_1 is not provided but [46] suggests that $r_1/r_3 \in [10^{-4}, 10^{-2}]$ should cover a realistic parameter range.

Let us now discuss our bound given relevant parameters in current implementations. We will consider a commercially available quantum annealer with the parameters released in 2019 [45]. The results and technical details of the choice of parameters is shown in Fig. 2. We conclude that the maximal annealing time for which the device is useful is of order $200\mu s$ and the window for a potential advantage lies on problems that only require a very small annealing time. This is consistent with the recent findings of a potential advantage of commercial annealers versus the best classical solvers [47]. For instance, in the recent demonstration of a constant speedup of quantum annealers in [48], the authors used

the minimal annealing time available for their findings, $5\mu s$. The same is true for the results of [49], which used $20\mu s$ annealing times. However, it is not clear to what extent this advantage does not result from purely engineering reasons disconnected to the presence of quantum effects, such as the high sampling rate on such devices. An argument that would further substantiate the claims of [50] that a new generation of special-purpose classical annealing chips have the potential to outperform current quantum annealing devices. Furthermore, we again conclude that increasing the number of qubits without substantially decreasing the control errors of annealers is unlikely to increase their potential to achieve an unquestionable quantum advantage.

V. CONCLUSION AND OUTLOOK

In this work we have developed a framework to quantify the effect of noise on near-term quantum computers when performing optimization. It is based on the combination of three main ingredients: firstly, contractivity results that quantify how the noise makes the quantum system deviate from the ideal solution and gets closer to the fixed point of the noise process; secondly, the use of mirror descent that allows us to assign to every step of the quantum computation a Gibbs state achieving the same approximation of the cost function; thirdly, exploiting existing efficient algorithms to approximate the specific cost function of the problem at high noise levels. The techniques proved to be extremely versatile and applicable to a large variety of problems, noise models and quantum computing architectures. Our work confirms that a bound on the depth that scales inversely proportional to the quantum gate error is indeed a universal bound and provides means of estimating the maximum depth rigorously.

Additionally, we have developed a technique to certify that a classical algorithms outperforms a given noisy quantum device running for a certain depth or number of cycles. This is shown using a nontrivial lower bound on the achievable energies by the noisy quantum device. Interestingly, it can also be used to provides sharper estimates for specific instances of a problem. Thus, a combination of both methods provides a good estimate of the limitations of a noisy near-term quantum computer.

Our result suggests that there is little chance near-term quantum devices can provide an advantage over classical ones for classical optimization problems, especially if they do not match the architecture's topology. This is because this would result in depths scaling with system size for current algorithmic proposals, which in turn imply that noise rates would have to decrease two orders of magnitude before being potentially competitive against state of the art classical methods.

The problems where near-term devices could have an opportunity window are most probably quantum many-body physics motivated, where the architecture topology has been tailored to the Hamiltonian of the problem and the correlation length in the system is bounded. Whether examples exist that cannot be simulated efficiently classically is, up to our knowledge, an important open problem.

It is important to stress that our bounds guarantee the efficient classical simulation, but at the price of being extremely conservative. Practical implementations of Gibbs samplers are expected to reach higher β than predicted by our bounds [51], especially using clusters or supercomputers. Furthermore, it is possible that the relative entropy contracts significantly more for states of interest than predicted by our state-independent bound. Moreover, relative entropy contraction techniques struggle when the fixed point of the system is close to a pure state, i.e. when amplitude damping is the main source of noise. This can be seen in our bounds for quantum annealers in the $r_1 \simeq r_3$ regime for our quantum annealing bound. Noise that does not uniformly contract the relative entropy, like dephasing, also requires a more detailed analysis, but technical tools are already available [52]. Therefore, improvements to our technique for both additional types of noises can only improve over the estimates obtained in this work.

Noise mitigation techniques based on post-processing of the quantum computation measurement outcomes, despite being useful to filter the data from noise, would not change the predictions of our work. One can easily reach this conclusion through a simple argument using the data-processing inequality of relative entropy. It would be interesting to see if primitive error correction tools can be used to reduce the effect of noise, in the spirit of [53], or how embedding a problem into the annealer topology or quantum annealing correction may be both limited by the technique presented here [54].

Finally, we believe that the technique developed here can be adapted to analog quantum simulators, where our continuous time result can be seen as a first step. This would allow to make predictions about the resilience of complex phases of matter to experimental imperfections. Similarly, extending our result to Fermionic problems is something we plan to address in future work.

DSF was supported by VILLUM FONDEN via the QMATH Centre of Excellence under Grant No. 10059. RGP was supported by the Quantum Computing and Simulation Hub, an EPSRC-funded project, part of the UK National Quantum Technologies Programme. We thank Heng Guo, Jonah Brown-Cohen and Pierre-Luc Dallaire-Demers for helpful discussions.

[1] F. Arute, K. Arya, R. Babbush, D. Bacon, J. C. Bardin, R. Barends, R. Biswas, S. Boixo, F. G. S. L. Brandao, D. A. Buell, B. Burkett, Y. Chen, Z. Chen,

B. Chiaro, R. Collins, W. Courtney, A. Dunsworth, E. Farhi, B. Foxen, A. Fowler, C. Gidney, M. Giustina, R. Graff, K. Guerin, S. Habegger, M. P. Harrigan, M. J. Hartmann, A. Ho, M. Hoffmann, T. Huang, T. S. Humble, S. V. Isakov, E. Jeffrey, Z. Jiang, D. Kafri, K. Kechedzhi, J. Kelly, P. V. Klimov, S. Knysh, A. Korotkov, F. Kostritsa, D. Landhuis, M. Lindmark, E. Lucero, D. Lyakh, S. Mandrà, J. R. McClean, M. McEwen, A. Megrant, X. Mi, K. Michielsen, M. Mohseni, J. Mutus, O. Naaman, M. Neeley, C. Neill, M. Y. Niu, E. Ostby, A. Petukhov, J. C. Platt, C. Quintana, E. G. Rieffel, P. Roushan, N. C. Rubin, D. Sank, K. J. Satzinger, V. Smelyanskiy, K. J. Sung, M. D. Trevithick, A. Vainsencher, B. Villalonga, T. White, Z. J. Yao, P. Yeh, A. Zalcman, H. Neven, and J. M. Martinis, Nature **574**, 505 (2019).

[2] J. Preskill, Quantum **2**, 79 (2018).

[3] A. Peruzzo, J. McClean, P. Shadbolt, M.-H. Yung, X.-Q. Zhou, P. J. Love, A. Aspuru-Guzik, and J. L. O'Brien, Nature Communications **5** (2014), 10.1038/ncomms5213.

[4] C. Kokail, C. Maier, R. van Bijnen, T. Brydges, M. K. Joshi, P. Jurcevic, C. A. Muschik, P. Silvi, R. Blatt, C. F. Roos, and P. Zoller, Nature **569**, 355 (2019).

[5] T. Jones, S. Endo, S. McArdle, X. Yuan, and S. C. Benjamin, Phys. Rev. A **99**, 062304 (2019).

[6] R. LaRose, A. Tikku, É. O'Neel-Judy, L. Cincio, and P. J. Coles, npj Quantum Information **5** (2019), 10.1038/s41534-019-0167-6.

[7] C. Bravo-Prieto, R. LaRose, M. Cerezo, Y. Subasi, L. Cincio, and P. J. Coles, "Variational quantum linear solver," (2019), arXiv:1909.05820 [quant-ph].

[8] S. McArdle, T. Jones, S. Endo, Y. Li, S. C. Benjamin, and X. Yuan, npj Quantum Information **5** (2019), 10.1038/s41534-019-0187-2.

[9] N. Moll, P. Barkoutsos, L. S. Bishop, J. M. Chow, A. Cross, D. J. Egger, S. Filipp, A. Fuhrer, J. M. Gambetta, M. Ganzhorn, *et al.*, arXiv preprint arXiv:1710.01022 (2017).

[10] A. Kandala, A. Mezzacapo, K. Temme, M. Takita, M. Brink, J. M. Chow, and J. M. Gambetta, Nature **549**, 242 (2017).

[11] D. Wang, O. Higgott, and S. Brierley, Physical review letters **122**, 140504 (2019).

[12] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," (2014), arXiv:1411.4028 [quant-ph].

[13] E. Farhi, J. Goldstone, S. Gutmann, and M. Sipser, arXiv preprint quant-ph/0001106 (2000).

[14] T. Albash and D. A. Lidar, Rev. Mod. Phys. **90**, 015002 (2018).

[15] A. Lucas, Frontiers in Physics **2** (2014), 10.3389/fphy.2014.00005.

[16] A. Mller-Hermes, D. S. França, and M. M. Wolf, Journal of Mathematical Physics **57**, 022202 (2016).

[17] K. Tsuda, G. Rätsch, and M. K. Warmuth, J. Mach. Learn. Res. **6**, 995 (2005).

[18] S. Bubeck, Foundations and Trends® in Machine Learning **8**, 231 (2015).

[19] S. Aaronson, X. Chen, E. Hazan, S. Kale, and A. Nayak, Journal of Statistical Mechanics: Theory and Experiment **2019**, 124019 (2019).

[20] S. Aaronson, in *Proceedings of the 50th Annual ACM SIGACT Symposium on Theory of Computing - STOC 2018* (ACM Press, 2018).

[21] F. G. Brandão, A. Kalev, T. Li, C. Y.-Y. Lin, K. M. Svore, and X. Wu, arXiv preprint arXiv:1710.02581 (2017).

[22] F. G. S. L. Brando, R. Kueng, and D. S. Frana, "Faster quantum and classical sdp approximations for quadratic binary optimization," (2019), arXiv:1909.04613 [cs.DS].

[23] A. Youssry, C. Ferrie, and M. Tomamichel, New Journal of Physics **21**, 033006 (2019).

[24] D. Petz, Communications in Mathematical Physics **114**, 345 (1988).

[25] D. A. Levin and Y. Peres, *Markov chains and mixing times*, Vol. 107 (American Mathematical Soc., 2017).

[26] R. Eldan, F. Koehler, and O. Zeitouni, "A Spectral Condition for Spectral Gap: Fast Mixing in High-Temperature Ising Models," (2020), arXiv:2007.08200v1, 2007.08200.

[27] G. A. Kochenberger, J.-K. Hao, Z. L, H. Wang, and F. Glover, Journal of Heuristics **19**, 565 (2011).

[28] I. Dunning, S. Gupta, and J. Silberholz, INFORMS Journal on Computing **30**, 608 (2018).

[29] A. Yurtsever, J. A. Tropp, O. Fercoq, M. Udell, and V. Cevher, arXiv preprint arXiv:1912.02949 (2019).

[30] E. Crosson and S. Slezak, "Classical Simulation of High Temperature Quantum Ising Models," (2020), arXiv:2002.02232v1, 2002.02232.

[31] A. Harrow, S. Mehraban, and M. Soleimanifar, "Classical algorithms, correlation decay, and complex zeros of partition functions of quantum many-body systems," (2019), arXiv:1910.09071v1, 1910.09071.

[32] T. Kuwahara, K. Kato, and F. G. Brando, Physical Review Letters **124** (2020), 10.1103/physrevlett.124.220601.

[33] F. Verstraete, J. J. García-Ripoll, and J. I. Cirac, Phys. Rev. Lett. **93**, 207204 (2004).

[34] W. M. C. Foulkes, L. Mitas, R. J. Needs, and G. Rajagopal, Rev. Mod. Phys. **73**, 33 (2001).

[35] M. Rigol, T. Bryant, and R. R. P. Singh, Phys. Rev. Lett. **97**, 187202 (2006).

[36] B. Tang, E. Khatami, and M. Rigol, Computer Physics Communications **184**, 557 (2013).

[37] S. R. White, Phys. Rev. Lett. **102**, 190601 (2009).

[38] A. Molnar, N. Schuch, F. Verstraete, and J. I. Cirac, Phys. Rev. B **91**, 045138 (2015).

[39] M. W. Johnson, M. H. S. Amin, S. Gildert, T. Lanting, F. Hamze, N. Dickson, R. Harris, A. J. Berkley, J. Johansson, P. Bunyk, E. M. Chapple, C. Enderud, J. P. Hilton, K. Karimi, E. Ladizinsky, N. Ladizinsky, T. Oh, I. Perminov, C. Rich, M. C. Thom, E. Tolkacheva, C. J. S. Truncik, S. Uchaikin, J. Wang, B. Wilson, and G. Rose, Nature **473**, 194 (2011).

[40] S. Jansen, M.-B. Ruskai, and R. Seiler, Journal of Mathematical Physics **48**, 102111 (2007).

[41] M. H. S. Amin, Phys. Rev. Lett. **102**, 220401 (2009).

[42] M. J. Kastoryano and K. Temme, Journal of Mathematical Physics **54**, 052202 (2013).

[43] R. Olkiewicz and B. Zegarlinski, Journal of Functional Analysis **161**, 246 (1999).

[44] S. Beigi, N. Datta, and C. Rouzé, Communications in Mathematical Physics **376**, 753 (2020).

[45] D.-W. S. Inc., *Technical Description of the D-Wave Quantum Processing Unit*, D-Wave Systems Inc. (2019).

[46] T. Albash and D. A. Lidar, Physical Review A **91** (2015), 10.1103/physreva.91.062320.

[47] P. Hauke, H. G. Katzgraber, W. Lechner, H. Nishimori, and W. D. Oliver, Reports on Progress in Physics **83**, 054401 (2020).
[48] S. Mandrà and H. G. Katzgraber, Quantum Science and Technology **3**, 04LT01 (2018).
[49] V. S. Denchev, S. Boixo, S. V. Isakov, N. Ding, R. Babbush, V. Smelyanskiy, J. Martinis, and H. Neven, Physical Review X **6** (2016), 10.1103/physrevx.6.031015.
[50] M. Aramon, G. Rosenberg, E. Valiante, T. Miyazawa, H. Tamura, and H. G. Katzgraber, Frontiers in Physics **7** (2019), 10.3389/fphy.2019.00048.
[51] S. Isakov, I. Zintchenko, T. Rønnow, and M. Troyer, Computer Physics Communications **192**, 265 (2015).
[52] I. Bardet and C. Rouzé, arXiv preprint arXiv:1803.05379 (2018).
[53] M. J. Bremner, A. Montanaro, and D. J. Shepherd, Quantum **1**, 8 (2017).
[54] J. Job and D. Lidar, Quantum Science and Technology **3**, 030501 (2018).
[55] M. Mller-Lennert, F. Dupuis, O. Szehr, S. Fehr, and M. Tomamichel, Journal of Mathematical Physics **54**, 122203 (2013).
[56] N. Datta, IEEE Transactions on Information Theory **55**, 2816 (2009).
[57] F. Pastawski, A. Kay, N. Schuch, and I. Cirac, Phys. Rev. Lett. **103**, 080501 (2009).
[58] M. J. Kastoryano, D. Reeb, and M. M. Wolf, Journal of Physics A: Mathematical and Theoretical **45**, 075307 (2012).
[59] L. Lami and M. Huber, Journal of Mathematical Physics **57**, 092201 (2016).
[60] E. P. Hanson, C. Rouzé, and D. S. França, Annales Henri Poincaré (2020), 10.1007/s00023-020-00906-4.
[61] A. Harrow and J. Napp, "Low-depth gradient measurements can improve convergence in variational hybrid quantum-classical algorithms," (2019), arXiv:1901.05374 [quant-ph].
[62] Y. Ge, A. Molnár, and J. I. Cirac, Physical Review Letters **116** (2016), 10.1103/physrevlett.116.080503.
[63] E. Farhi, D. Gamarnik, and S. Gutmann, "The Quantum Approximate Optimization Algorithm Needs to See the Whole Graph: Worst Case Examples," (2020), arXiv:2005.08747v1, 2005.08747.
[64] S. Bravyi, A. Kliesch, R. Koenig, and E. Tang, "Obstacles to State Preparation and Variational Optimization from Symmetry Protection," (2019), arXiv:1910.08980v1, 1910.08980.
[65] M. X. Goemans and D. P. Williamson, Journal of the ACM (JACM) **42**, 1115 (1995).
[66] C. Bravo-Prieto, J. Lumbreras-Zarapico, L. Tagliacozzo, and J. I. Latorre, Quantum **4**, 272 (2020).
[67] M. B. Hastings and T. Koma, Communications in Mathematical Physics **265**, 781 (2006).
[68] R. Ors, Annals of Physics **349**, 117158 (2014).
[69] I. H. Kim and B. Swingle, "Robust entanglement renormalization on a noisy quantum computer," (2017), arXiv:1711.07500 [quant-ph].
[70] I. H. Kim, "Noise-resilient preparation of quantum many-body ground states," (2017), arXiv:1703.00032 [quant-ph].
[71] J. Borregaard, M. Christandl, and D. S. Frana, "Noise-robust exploration of quantum matter on near-term quantum devices," (2019), arXiv:1909.04786v1, 1909.04786.
[72] F. G. S. L. Brandao and K. M. Svore, in *58th Annual IEEE Symposium on Foundations of Computer Science—FOCS 2017* (IEEE Computer Soc., Los Alamitos, CA, 2017) pp. 415–426.
[73] M. Christandl and A. Mller-Hermes, Communications in Mathematical Physics **353**, 821 (2017).
[74] A. A. Budini, Physical Review A **64** (2001), 10.1103/physreva.64.052110.

SUPPLEMENTARY INFORMATION

VI. QUANTUM CIRCUITS WITH DEPOLARIZING NOISE

In this section we will prove and discuss our bounds in the special case of local depolarizing noise. Furthermore, we will review some fundamental properties of the relative entropy more generally and discuss some qualitative aspects of our results, like how they differ from analysing convergence in trace distance to the fixed point. Whenever the proof is essentially the same in the case of the maximally mixed state and more generally, like it is the case for Prop. 4, we will prove it in full generality.

A. Relative entropy fundamentals

Most of our results rely on the relative entropy $D(\rho\|\sigma)$ between two states. In this part of the supplemental material we collect some basic facts about it and then move on to a more detailed proof of Lemma 2. First, recall that for a full-rank state σ and an arbitrary state ρ, the relative entropy $D(\rho\|\sigma)$ is defined as

$$D(\rho\|\sigma) = \operatorname{tr}\left(\rho(\log(\rho) - \log(\sigma)\right).$$

One of the most important properties of the relative entropy is that it satisfies a data-processing inequality. That is, for any quantum channel T and states ρ, σ we have:

$$D(T(\rho)\|T(\sigma)) \leq D(\rho\|\sigma).$$

In this work we also exploited extensively the case in which the channel contracts the relative entropy strictly, which we call a strong data-processing inequality.

In the main text we exploited upper bounds on the relative entropy with a fixed second argument. The most straightforward way to derive them is through the max-relative entropy. Indeed, one can show that the relative entropy is upper-bounded by the so-called max-relative entropy D_∞ [55, 56], which is defined as:

$$D_\infty(\rho\|\sigma) = \log(\|\sigma^{-\frac{1}{2}}\rho\sigma^{-\frac{1}{2}}\|),$$

where $\|\cdot\|$ denotes the operator norm. From this, we see that for all states ρ:

$$D(\rho\|\sigma) \leq \log(\|\sigma^{-1}\|).$$

Indeed, we have

$$D(\rho\|\sigma) \leq D_\infty(\rho\|\sigma) \\ \leq \log(\|\rho\|\|\sigma^{-\frac{1}{2}}\|^2) \leq \log(\|\sigma^{-1}\|),$$

where we used the submultiplicativity of the operator norm and the fact that $\|\rho\| \leq 1$ for all states.

We also have the following bound on the max relative entropy:

$$D_\infty(\mathcal{U}(\sigma)\|\sigma) \leq \log(\|\sigma^{-1}\|\|\sigma\|)$$

for all unitary channels $\mathcal{U}$. This also easily follows from the submultiplicativity of the operator norm, as we have:

$$D_\infty(\mathcal{U}(\sigma)\|\sigma) = \log(\|\sigma^{-\frac{1}{2}}\mathcal{U}(\sigma)\sigma^{-\frac{1}{2}}\|) \\ \leq \log(\|\sigma^{-1}\|\|\mathcal{U}(\sigma)\|) = \log(\|\sigma^{-1}\|\|\sigma\|).$$

If σ is a Gibbs state $\sigma = e^{-\beta H}/\mathcal{Z}$, then the inequality above reads:

$$D_\infty(\mathcal{U}(\sigma)\|\sigma) \leq \log(\|e^{-\beta H}\|\|e^{\beta H}\|) \\ \leq 2\beta\|H\|.$$

The last inequality follows from the fact that spectrum of $e^{-\beta H}$ and $e^{\beta H}$ are both contained in the interval $[e^{-\beta\|H\|}, e^{\beta\|H\|}]$.

1. From relative entropy convergence to trace distance

Let us now discuss the connection between bounds on the relative entropy and closeness in trace distance. This is established by Pinsker's inequality, which asserts that:

$$D(\rho||\sigma) \geq \frac{1}{2}\|\rho - \sigma\|_{\text{tr}}^2.$$

Thus, in particular, it follows from a strong data processing inequality for a channel T with constant α that applying a channel n times to any initial state is enough to ensure that the trace distance to σ is at most:

$$\frac{1}{2}\|T^n(\rho) - \sigma\|_{\text{tr}}^2 \leq D(T^n(\rho)\|\sigma) \leq (1-\alpha)^n \log(\|\sigma^{-1}\|).$$

Thus, if we apply the channel more than

$$N_{\max} \geq \frac{\log(2\epsilon^2 \log(\|\sigma^{-1}\|))}{\log((1-\alpha)^{-1})}, \tag{17}$$

times then the trace distance between any input and output is at most ϵ. This is to be compared with e.g. Eq. (10) of the main text, which discussed the maximum distance after which we can approximate the energy of the output of a noisy circuit. If we take e.g. $\sigma = I/2^n$ and we take α and ϵ to be constant, then we are ϵ-close to σ after $\log(n)$ depth. On the other hand, Eq. (10) asserts that we can already approximate the energies of the output with a classically simulatable state after constant depth. This difference in the scaling clarifies that our analysis differs from estimating when the output of the circuit is essentially maximally mixed.

Although one might at first think that this is just a consequence of the method we used to bound the trace distance, it is possible to show that logarithmic time is really necessary before the output of the circuit becomes close to maximally mixed [57, 58]. The same holds when analysing how long it takes for the output to become separable [59, 60], again highlighting how our analysis is qualitatively different.

B. Variational formulation of the relative entropy

Another fundamental ingredient to concretely assess the potential of noisy quantum computers is the variational formulation of the relative entropy. As proved by Petz [24], we have that:

$$D(\rho\|\sigma) = \\ \sup_{\omega>0, \text{tr}(\omega)=1} \text{tr}\left(\rho \log(\omega)\right) - \log\left(\text{tr}(\exp\left(\log(\omega) + \log(\sigma)\right)\right). \tag{18}$$

From this we can immediately derive:

Proposition 4 (Lower bound of output energy of noisy device)**.** *Let $\Phi(\rho)$ be the output of a noisy quantum circuit and H a Hamiltonian. For any $\sigma > 0$ and $\beta > 0$ define $\mathcal{Z}_{\beta,\sigma} = tr\left(e^{-\beta H + \log(\sigma)}\right)$. Then:*

$$tr\left(\Phi(\rho)H\right) \geq \sup_{\beta>0} \beta^{-1}\left(\log(\mathcal{Z}_{\beta,\sigma}) - D(\Phi(\rho)\|\sigma)\right). \tag{19}$$

Proof. Picking $\omega = e^{-\beta H}/tr\left(e^{-\beta H}\right)$ in Eq. (18) we obtain:

$$D(\Phi(\rho)\|\sigma) \geq -\beta \text{tr}\left(\rho H\right) - \log(\mathcal{Z}_\beta) - \log(\mathcal{Z}_{\beta,\sigma}/\mathcal{Z}_\beta).$$

The claim then follows from a straightforward manipulation of the terms above and noting that it holds for all $\beta > 0$. □

As usual, it is instructive to take the case $\sigma = I/2^n$ into consideration to gain some intuition on the scaling of the bound above. It can then be rewritten as

$$\text{tr}\left(\Phi(\rho)H\right) \geq \beta^{-1}\left(n - \log(\mathcal{Z}_\beta) - D(\Phi(\rho)\|I/2^n)\right).$$

Note that in the limit $\beta \to \infty$ we obtain the inequality $\text{tr}\left(\rho H\right) \geq E_0$, where E_0 is the ground state energy. Moreover, note that

$$\lim_{\beta\to 0} \beta^{-1}\left(n - \log(\mathcal{Z}_\beta)\right) = \text{tr}\left(H\frac{I}{2^n}\right).$$

That is, for β small, the first term is given by the energy of the maximally mixed state. Let us now once again resort to relative entropy convergence techniques. We

know that if Φ is a depth D quantum circuit affected by depolarizing noise with probability p we have:

$$\mathrm{tr}\left(\rho H\right) \geq \beta^{-1}\left(n - \log(\mathcal{Z}_\beta) - (1-p)^{2D} n\right).$$

Now assume that $(1-p)^{2D} \ll \beta$ for some small β. Then the second term will be under control for that β and we have the promise that the first term is close to the energy of the maximally mixed state, ensuring that we do not deviate much from it. However, this inequality also provides nontrivial bounds even for moderate values of $(1-p)^{2D}$. Note that evaluating this bound requires us to estimate the partition function in some range of parameters β. However, as remarked in the main text, there are many efficient algorithms that achieve this for β below a critical temperature.

It is also worth noting that for local Hamiltonians one has

$$\beta^{-1}\left(n - \log(\mathcal{Z}_\beta)\right) = \mathrm{tr}\left(H\frac{I}{2^n}\right) - \mathcal{O}(\beta n). \tag{20}$$

This implies that usually taking $D_{\max} = -\log(\epsilon^{-1})/\log(1-p)$ should be enough to ensure that the energy of the output of the circuit is $\mathcal{O}(\epsilon n)$ away from the energy of the maximally mixed state. Indeed, we have:

$$\begin{aligned}&\mathrm{tr}\left(\rho H\right) \geq \epsilon^{-1}\left(n - \log(\mathcal{Z}_\epsilon) - (1-p)^{2D_{\max}} n\right)\\ &= \mathrm{tr}\left(H\frac{I}{2^n}\right) - \mathcal{O}(\beta n) - \epsilon^{-1}\epsilon^2 n = \mathrm{tr}\left(H\frac{I}{2^n}\right) - \mathcal{O}(\epsilon n).\end{aligned}$$

Although using Prop. 4 does not immediately give us classical simulability thresholds, as it still requires us to evaluate the partition function of the model, it is very useful to assess the potential of a quantum computer for a given problem. Indeed, suppose that a classical algorithm, not necessarily simulated annealing, provides us an energy E_c. By evaluating the partition function for some range of β and estimating the relative entropy with a strong data processing inequality it then immediately gives us access to concrete lower bounds on the outputs of noisy quantum devices at a certain depth. In particular, we can estimate the maximal depth before the output of the device is larger than E_c and the quantum computer becomes useless. We exemplify this in Section VI D.

C. Proof of Lemma 2 for maximally mixed states

However, in the case of $\sigma = I/2^n$, it is straightforward to obtain approximate classical simulability bounds from the variational formulation of the relative entropy:

Corollary 1. *Let ρ be a n-qubit state such that $D(\rho\|I/2^n) \leq \beta\epsilon\|H\|$ for some $\epsilon, \beta > 0$ and Hamiltonian H. Then we have:*

$$tr(H\rho) \geq tr(\sigma_\beta H) - \epsilon\|H\|.$$

Proof. Note that the log-partition function $f(\beta) = \log(\mathcal{Z}_\beta)$ is always differentiable for finite dimensional systems. Thus, by the mean value theorem we have that:

$$\frac{f(0) - f(\beta)}{\beta} = -f'(\widetilde{\beta})$$

for some $\widetilde{\beta} \in (0, \beta)$. It is well-known that $-f'(\widetilde{\beta}) = \mathrm{tr}\left(\sigma_{\widetilde{\beta}} H\right)$. As the energy is monotone decreasing in β, we have $\mathrm{tr}\left(\sigma_{\widetilde{\beta}} H\right) \geq \mathrm{tr}\left(\sigma_\beta H\right)$. Thus, it follows from Prop. 4 that

$$\mathrm{tr}\left(H\rho\right) \geq \mathrm{tr}\left(\sigma_{\widetilde{\beta}} H\right) - \beta^{-1} D(\rho\|I/2^n) \geq \mathrm{tr}\left(\sigma_{\widetilde{\beta}} H\right) - \epsilon\|H\|,$$

where in the last line we used our assumed bound on $D(\rho\|I/2^n)$. □

We will later generalize the statement above to arbitrary stationary states in Lemma 5. However, the corollary above improves the estimate of Lemma 5 by a factor of 4.

1. *Choice of error parameter ϵ*

Let us now discuss the scaling and the choice of the parameter ϵ in Lemma 2 and similar statements throughout the text. It controls the precision with which we approximate the energy, as in Eq. (32). Throughout this article the reader should think of ϵ in the range $10^{-1} - 10^{-2}$. This is justified for several reasons. In the setting of the variational quantum eigensolver [9–11], note that most algorithms have an inverse polynomial scaling in ϵ in the sample complexity for each iteration [61]. In most of the cases, the dependency is quadratic, while other proposals get a better dependence by proposing circuits of larger depth or more complex measurements [11, 61]. Thus, for most of these proposals, a precision beyond $\epsilon \sim 10^{-2}$ would require a significant number of measurements for each iteration or larger circuit depths. In the adiabatic setting, the more general theorems [40] have a scaling of the runtime that has an ϵ^{-1} dependency on the error ϵ as defined in Eq. (32). Thus, obtaining high precision solutions may take very long evolution times, although some adiabatic theorems have a logarithmic dependency on the error under stronger assumptions [14, 62]. More importantly, through the variational bound of Eq. 18 we numerically observe that picking ϵ of order $10^{-2} - 10^{-1}$ is actually sufficient to ensure that the energy of $\widetilde{\sigma}$ is strictly smaller than that of $\Phi(\rho)$. This behaviour is justified by eq. (20).

D. Worked through example of the variational bound

Let us now exemplify how to apply the lower bound given in Lemma 4. We do this with MAXCUT instances

on 3-regular graphs. We will compare the performance of the device with the SDP relaxation and Gibbs sampling with inverse temperatures in the polynomial time range. See Fig. 3 for an example. As soon as one of the

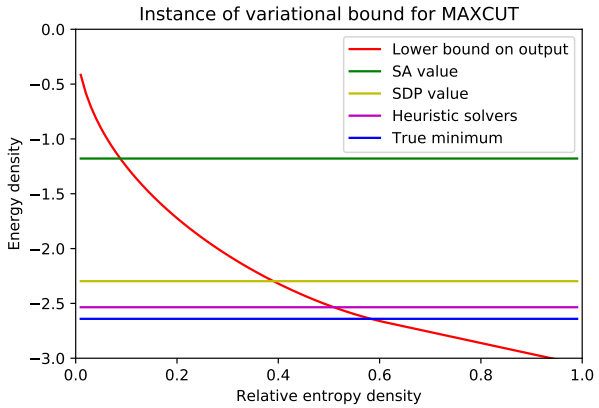


FIG. 3. Performance of Lemma 4 in one instance of MAXCUT of a 3-regular graph with 20 nodes. The lower bound on curve gives the maximum energy density predicted by Eq. (19) when evaluated on $\beta \in (0, \beta_c)$ and different values of the relative entropy density. The SA value line gives the energy achieved by simulated annealing in the polynomial time range, while the SDP line gives the expected value of the SDP relaxation randomized rounding. Furthermore, the heuristic solver line gives the bound assuming that heuristic solvers achieve a 0.95 approximation ratio. From this graph we see that as long as the depth is such that the relative entropy density of the output states is at most $\simeq 0.4$, then SDP methods will yield better approximations than the noisy circuit. On the other hand, SA will outperform the noisy quantum computer at densities $\simeq 0.1$.

horizontal lines corresponding to a method crosses the lower bound line and the lower bound is then above it, it means that for relative entropy densities smaller than that threshold the noisy quantum computer is guaranteed to be outperformed by said method. From Fig. 3, we infer that the SDP threshold is around $0.4n$. Thus, we get the condition $D(\Phi(\rho)\|\sigma) \leq 0.4n$ for $\sigma = I/2^n$ for SDPs to outperform the noisy quantum computer. Making the approximation $\log(0.4) \simeq -0.92$ and resorting to the usual bound $D(\Phi(\rho)\|\sigma) \leq (1-p_1)^{2Df_1}(1-p_2)^{2Df_2}$, where f_1 is the fraction of $1-$qubit layers with depolarizing noise p_1 and f_2, p_2 defined analogously for 2 qubits, we conclude that the maximum depth satisfies

$$D \leq 0.46(f_1 p_1 + f_2 p_2)^{-1}.$$

Note that this estimate is superior to the naive $1/p$ estimate of the maximal depth one can achieve. Assuming the fractions of layers to be the same and the noise parameters as in Sec. II A, we get a total depth of roughly $D \simeq 70$ of when the noisy quantum computer is guaranteed to be outperformed by the SDP relaxation. We note that we observe a threshold relative entropy density for varying graph sizes and random instances of up to 20 qubits of around $0.35 - 0.4$. Thus, we will extrapolate the findings to larger system sizes. Assuming the slightly more conservative relative entropy density bound threshold of 0.3 scales to larger system sizes and that each round of QAOA requires a depth scaling like n, we see that we expect the SDP to outperform three rounds of QAOA already at systems of size 30, orders of magnitude below what current solvers can handle.

VII. CORRELATION LENGTH AND REQUIRED CIRCUIT DEPTH

In Eq. (10) we gave a bound on the maximal circuit depth before a noisy quantum device is outperformed by classical methods that only depended on the geometry of the lattice and noise rate. We will now show that this also implies that noisy quantum devices performing VQE are unlikely to find the ground states with long correlation length. First, recall that given a graph $G = (V, E)$, a state is said to have correlation length ξ if for all observables O_x and O_y supported on sites $x, y \in V$ we have that

$$|\mathrm{tr}\left(|\psi\rangle\langle\psi| O_x \otimes O_y\right) - \mathrm{tr}\left(|\psi\rangle\langle\psi| O_x\right)\mathrm{tr}\left(|\psi\rangle\langle\psi| O_y\right)| \quad (21)$$

$$\leq 2\|O_x\|\|O_y\| e^{-\frac{d(x,y)}{\xi}}. \quad (22)$$

Let us assume a negation of Eq (27), i.e. the existence of observables on O_x, O_y at x, y at distance ξ s.t.

$$|\mathrm{tr}\left(|\psi\rangle\langle\psi| O_x \otimes O_y\right) - \mathrm{tr}\left(|\psi\rangle\langle\psi| O_x\right)\mathrm{tr}\left(|\psi\rangle\langle\psi| O_y\right)| \quad (23)$$

$$\geq \|O_x\|\|O_y\| C, \quad (24)$$

where $C \gg 0$ is some constant. That is, there are significant correlations in the state at distance $d(x, y)$. Suppose further for simplicity that the graph G also reflects the connectivity of the noisy quantum device and let $U = U_1 U_2 \cdots U_D$ be a circuit of depth D. Let us recall the well-known result that, starting from a product state, a depth proportional to the correlation length is required to prepare a state:

Lemma 3. *Let $|\psi\rangle$ be the ground state of κ-local Hamiltonian H on a d-dimensional lattice such that there exist observables O_x, O_y with $\|O_x\|, O_y\| = 1$ supported at sites x, y at distance ξ fulfilling*

$$tr\left(|\psi\rangle\langle\psi| O_x \otimes O_y\right) - tr\left(|\psi\rangle\langle\psi| O_x\right) tr\left(|\psi\rangle\langle\psi| O_y\right) \geq C. \quad (25)$$

Then the output $\Phi\left(|0\rangle\langle 0|^{\otimes n}\right)$ of any depth $\frac{\xi}{2}$ circuit Φ with the same locality as the lattice satisfies:

$$\|\Phi(|0\rangle\langle 0|^{\otimes n}) - |\psi\rangle\langle\psi|\|_{tr} \geq \frac{C}{4}$$

Proof. We will prove the claim by contradiction. Assume that

$$\|\Phi(|0\rangle\langle 0|^{\otimes n}) - |\psi\rangle\langle\psi|\|_{tr} < \frac{C}{4} \quad (26)$$

By the variational formulation of the trace distance, it suffices to find an observable O with $\|O\| \leq 1$ and such that

$$\mathrm{tr}\left(O\left(\Phi(|0\rangle\langle 0|^{\otimes n}) - |\psi\rangle\langle\psi|\right)\right) \geq \frac{C}{4}$$

for a contradiction. Consider

$$O = O_x \otimes O_y$$

Note that as the circuit has the same locality as the lattice and sites x and y are a distance ξ apart, we have that $\Phi^*(O_x)$ and $\Phi^*(O_y)$ have disjoint support. Here Φ^* corresponds to the circuit in the Heisenberg picture. This is because the circuit has depth at most $\frac{\xi}{2}$. Let $a_x = \mathrm{tr}\left(O_x\Phi(|0\rangle\langle 0|^{\otimes n})\right)$ and $a_y = \mathrm{tr}\left(O_y\Phi(|0\rangle\langle 0|^{\otimes n})\right)$. Thus,

$$\mathrm{tr}(O\Phi(|0\rangle\langle 0|^{\otimes n})) = a_x a_y$$

Let $b_{x,y} = \mathrm{tr}\left(O_x \otimes O_y|\psi\rangle\langle\psi|\right)$, $b_x = \mathrm{tr}\left(O_x|\psi\rangle\langle\psi|\right)$ and $b_y = \mathrm{tr}\left(O_y|\psi\rangle\langle\psi|\right)$. By Eq. (25), we have that:

$$b_{x,y} \geq C + b_x b_y.$$

Moreover, as we assumed Eq. (26), we have that

$$|b_x - a_x| < \frac{C}{4}, \quad |b_y - a_y| < \frac{C}{4}.$$

Thus:

$$\begin{aligned}&\mathrm{tr}\left(O\left(\Phi(|0\rangle\langle 0|^{\otimes n}) - |\psi\rangle\langle\psi|\right)\right) \geq C + b_x b_y - a_x a_y \\ &\geq C - (|b_x| + |b_y|)\frac{C}{4} \geq \frac{C}{2}.\end{aligned}$$

This yields the claim. □

Thus, we see that for depths $D \leq \xi/2$, the quantum circuit is guaranteed to be a constant distance away from the ground state and conclude that a depth which scales at least linearly with the correlation length of the state is necessary to faithfully reproduce the ground state. This, however, does not exclude the possibility that a quantum device can outperform classical optimization methods at smaller depths if there are excited states with significantly smaller correlation lengths and small energies. But then, if we are interested in properties of the ground state, it is arguable that the quantum computer is outperforming classical methods for the wrong reason. This is because the output of the circuit will necessarily be far away from the ground state. And, at larger depths, unless the noise rate is sufficiently small, the device is outperformed by classical methods in optimizing the energy. This is captured in the next proposition:

Proposition 5. *Let $|\psi\rangle$ be the ground state of a κ local Hamiltonian H on a d-dimensional lattice such that there exist observables O_x, O_y at distance ξ fulfilling*

$$\begin{aligned}&|tr(|\psi\rangle\langle\psi|O_x \otimes O_y) - tr(|\psi\rangle\langle\psi|O_x)\, tr(|\psi\rangle\langle\psi|O_y)| \\ &\geq \|O_x\|\|O_y\|C. \end{aligned} \tag{27}$$

Suppose that $\Phi(|0\rangle\langle 0|^{\otimes n})$ is the output of a noisy quantum circuit with depth D respecting the locality of the lattice. Furthermore, suppose that the circuit undergoes local depolarizing noise with rate p that satisfies:

$$p \geq \frac{2\log(20ed^\kappa\epsilon^{-1})}{\xi}.$$

Then $\Phi(|0\rangle\langle 0|^{\otimes n})$ either satisfies:

$$\|\Phi(|0\rangle\langle 0|^{\otimes n}) - |\psi\rangle\langle\psi|\|_{tr} \geq \frac{C}{4} \tag{28}$$

or there is a quasi-polynomial time classical algorithm that outputs a state $\widetilde{\sigma}$ that satisfies:

$$tr\left(H\left(\widetilde{\sigma} - \Phi(\rho)\right)\right) \leq \|H\|\epsilon. \tag{29}$$

Proof. If the depth of the circuit Φ is less than $\frac{\xi}{2}$, then Eq. (28) holds. On the other hand, for depths $D \geq \frac{\xi}{2}$, we have a quasi-polynomial algorithm by Proposition 3 and the bound on the noise rate. □

Unfortunately, it is a hard task to show that for a given Hamiltonian there are no low-energy states that have a small correlation length and, thus, we cannot exclude the possibility of there being minima of the energy at low depths. However, for some models, one can show that a quantum circuit of at least logarithmic depth is required to beat polynomial time classical algorithms. For instance, it has been shown that the quantum approximate optimization algorithm requires at least logarithmic depth to outperform classical polynomial time approximation algorithms to optimize the energy of certain classical Ising Hamiltonians [63, 64]. This implies that even noise rates that decay with the number of qubits are not enough to ensure that QAOA outperforms classical methods:

Proposition 6 (Inverse logarithmic noise QAOA for classical Ising models never outperforms polynomial time classical for certain instances)**.** *Suppose a quantum device with n qubits suffers from 1-local depolarizing noise after each layer of QAOA is applied with probability*

$$p \geq \frac{\log\left(\epsilon^{-1}\right)\log(\Delta - 1)}{2\log(n)}$$

for some $\Delta \geq 3$ and $\epsilon > 0$. Then there exist Δ-regular graphs $G = (V, E)$ on n vertices such that for the Hamiltonian

$$H = -\sum_{i\sim j} Z_i Z_j$$

performing QAOA on the noisy device never outperforms polynomial time randomized classical algorithms by more than $\epsilon\Delta n$.

Proof. In [63, 64] the authors identify instances such that at least

$$D_{\min} = \log(n)/\log(\Delta - 1) \tag{30}$$

applications of the QAOA unitary operator are required to achieve an approximation ratio that is better than 1/2. Thus, for depths smaller than $D_{\min}$, QAOA is outperformed by the Goemanson-Williamson algorithm [65], as it achieves an approximation ratio greater than 0.87. As we assumed local depolarizing noise, if Φ is a noisy depth D quantum channel we have:

$$D\left(\Phi(\rho)\|\frac{I}{2^n}\right) \leq (1-p)^{2D} n.$$

Thus, to obtain an approximation up to $\epsilon n \Delta$, we need to sample from the inverse temperature

$$\beta = \frac{(1-p)^{2D}}{\Delta\epsilon}.$$

The results of [26] assert that we may sample efficiently from this inverse temperature as long as

$$\beta \leq \frac{1}{\Delta}.$$

Thus, if p is such that for $D_{\min}$ we have

$$\left(\frac{(1-p)^{2D_{\min}}}{\epsilon}\right) \leq 1,$$

then also for depths larger than $D_{\min}$ the noisy quantum device will not substantially outperform classical methods. Solving for p we then obtain the constraint:

$$\log(1-p) \leq \frac{\log(\Delta-1)}{2\log(n)} \log\left(\epsilon^{-1}\right)$$

Using $\log(1-p) \geq -p$ yields the claim. □

Thus, we see that without error correction, for some instances QAOA is outperformed by classical methods even with noise rates decaying logarithmically with system size. To the best of our knowledge, this is the first result that asserts that a noisy quantum algorithm will be outperformed by polynomial time-classical algorithms even with local noise rates converging to 0. However, as pointed out in [63], the logarithmic dependency in system size on the depth of the circuit is not too restrictive in practice and this bound would admittedly only be relevant for very large system sizes. However, as remarked in the main text, if the depth of each QAOA layer scales with system size due to the limited connectivity of the device, then the bound above can actually be used to obtain rigorous results on how noisy QAOA will never substantially outperform polynomial time algorithms for some instances. Indeed, evaluating Eq. (30) for $n = 10^3$ qubits and $\Delta = 3$ gives that we need roughly 10 rounds of QAOA to outperform SDPs. Note that for the estimates in Sec. II A we only assumed three rounds of QAOA. Thus, performing 10 rounds of QAOA with each QAOA unitary requiring circuit depth scaling mildly with n would require extremely low noise rates in order for results like Prop. 2 not to guarantee classical superiority.

Although we cannot prove the lack of advantage at any depth for noisy quantum algorithms beyond the example above, a striking phenomenon was observed in [66]. There, the authors show numerical evidence that for critical systems, which have long correlation lengths, noiseless VQE and QAOA algorithms do not make significant progress in lowering the energy before depths proportional to the system size. This suggests that an even stronger variation of Prop. (5) may be true for such systems. That is, the noisy quantum computer will not output lower energy states for any depth unless the noise rate scales at least inverse linearly with system size.

On the other hand, for gapped systems, which have exponentially decaying correlations [67], the authors of [66] show that noiseless VQE algorithms already start exponentially converging to the ground state for small depths.

Thus, if the behaviour observed in [66] for the preparation of states with long correlation length is indeed universal for VQE algorithms, it may be difficult to demonstrate any advantage in optimizing the energy for states with long correlation lengths. And, if there indeed is one, then it may also happen because the VQE algorithm is stuck in a local minimum, as discussed in Prop. 5.

We conclude that our results indicate that it is more realistic to expect noisy quantum computers to prepare states with a small correlation length. However, these are exactly the class of states classical simulation techniques such as tensor networks [68] excel at. Moreover, as argued in [69–71], for such states it is more noise robust to prepare them in a sequential fashion instead of through a quantum circuit. Thus, when it comes to optimizing the ground state of local Hamiltonians on noisy devices, it seems advisable to focus on Hamiltonians that are expected to be gapped, but still out of reach for current classical methods. Furthermore, it may be advisable to focus on sequential preparation instead of a circuit ansatz.

VIII. GENERAL CIRCUITS

In this section we show how to generalize Lemma 2 beyond the maximally mixed case. Furthermore, we develop some tools that lead to the qubit circuit equivalent of Lemma 1 in the main manuscript and that is a preliminary step to derive a variation of entropic convergence for the quantum annealers beyond the case of a maximally mixed fixed point. This is the content of Lemma 6.

A. Mirror descent

Now that we discussed some basic properties of the relative entropy, let us discuss Lemma 2 in more detail and prove an analogue beyond the maximally mixed state. As mentioned in the main text, the goal of our framework is to find a state that approximates the energy of the noisy quantum device. To that end, given a quantum state $\rho \in \mathcal{D}_{2^n}$, an n-qubit Hamiltonian H and an error parameter $\epsilon > 0$, define the sets:

$$\mathcal{C}(\rho, H, \epsilon) = \{\sigma \in \mathcal{D}_{2^n} : \mathrm{tr}\left(H\left(\sigma - \rho\right)\right) \leq \epsilon \|H\|\}.$$

That is, $\sigma \in \mathcal{C}(\rho, H, \epsilon)$ if its energy is at most $\epsilon\|H\|$ larger than that of ρ. We will then say that σ performs approximately as well as $\Phi(\rho)$ to optimize H, where Φ is as usual a noisy quantum device with input ρ, if $\sigma \in \mathcal{C}(\Phi(\rho), H, \epsilon)$. The goal of our framework will be to find a Gibbs state $\widetilde{\sigma}$ that is in $\mathcal{C}(\Phi(\rho), H, \epsilon)$ for a given device and noise levels. We will now prove the convergence of the update rule of mirror descent for completeness:

Lemma 4. *Let ρ be a quantum state. Fix a Hamiltonian H_0 and let $\sigma_0 = e^{-H_0}/\mathcal{Z}_0$. Suppose that for some other Hamiltonian H we have:*

$$tr\left(H\left(\sigma_0 - \rho\right)\right) \geq \|H\|\epsilon.$$

Set $H_1 = H_0 + \frac{\epsilon}{2\|H\|}H$. Then, the Gibbs states $\sigma_1 = e^{-H_1}/\mathcal{Z}_1$ obeys

$$D(\rho\|\sigma_1) - D(\rho\|\sigma_0) \leq -\frac{\epsilon^2}{4}$$

Proof. We have:

$$D(\rho\|\sigma_1) - D(\rho\|\sigma_0) = \mathrm{tr}\left(\rho(H_1 - H_0) + \log\left(\frac{\mathrm{tr}(\exp(-H_1))}{\mathrm{tr}(\exp(-H_0))}\right)\right)$$

By construction, $H_1 - H_0 = \frac{\epsilon}{2\|H\|}H$ and the first term equals $\frac{\epsilon}{2\|H\|}\mathrm{tr}(H\rho)$. The logarithmic ratio can be bounded by $\frac{\epsilon}{2\|H\|}\mathrm{tr}(H\sigma_1)$ and we refer to [22][Proof of Lemma 3.1] for a derivation based on the Peierls-Bogoliubov inequality. Adding and subtracting $\frac{\epsilon}{2\|H\|}\mathrm{tr}(H\sigma_0)$, we conclude

$$\begin{aligned} &D(\rho\|\sigma_1) - D(\rho\|\sigma_0) \\ \leq &\frac{\epsilon}{2}\left(\mathrm{tr}\left(\frac{H}{\|H\|}(\sigma_1 - \sigma_0)\right) + \mathrm{tr}\left(\frac{H}{\|H\|}(\rho - \sigma_0)\right)\right) \\ \leq &\frac{\epsilon}{2}\left(\tfrac{1}{2}\|\sigma_1 - \sigma_0\|_{\mathrm{tr}} + \mathrm{tr}\left(\frac{H}{\|H\|}(\rho - \sigma_0\right)\right), \end{aligned}$$

Here we used Hellstrom's bound to get

$$\tfrac{1}{2}\|\sigma_1 - \sigma_0\|_{\mathrm{tr}} \geq \mathrm{tr}\left(\frac{H}{\|H\|}(\sigma_1 - \sigma_0)\right).$$

In terms of The claim now follows from noticing that σ_0 and σ_1 must be close in trace distance. [72, Lem. 16] implies $\frac{1}{2}\|\sigma_1 - \sigma_0\|_{tr} \leq \exp(\frac{\epsilon}{4}) - 1 \leq \frac{\epsilon}{2}$. □

That is, as long as we have not converged to a state in $\widetilde{\sigma} \in \mathcal{C}(\rho, H, \epsilon)$, updating the Hamiltonian of the Gibbs state gives rise to a state that is closer to the target state in relative entropy. Let us now see how this can be used to prove Lemma 2, which we now restate here for the reader's convenience. Before stating the Lemma, let us remark that the statement below is worse by a factor of 1/4 for the required relative entropy compared to that of Lemma 2. That is, here we required the Gibbs state with Hamiltonian $\frac{4\lambda}{\|H\|\epsilon}H$ instead of $\frac{\lambda}{\|H\|\epsilon}H$. This is because in the case of σ being maximally mixed state it is straightforward to obtain this slightly improved statement from the formulation of the relative entropy, as we show later in Cor. 1.

Lemma 5. *Let $\Phi(\rho)$ be the output of a noisy quantum device and for some quantum state $\sigma > 0$ let $H_\infty = \log(\sigma)$ and $D(\Phi(\rho)||\sigma) = tr(\Phi(\rho)\left(\log(\Phi(\rho)) - \log(\sigma)\right))$ be their relative entropy. Then for any Hamiltonian H there is a $\lambda \in [0, D(\Phi(\rho)||\sigma)]$ such that the state $\widetilde{\sigma}$ defined as*

$$\widetilde{\sigma} = exp\left(-H_\infty - \frac{4\lambda}{\|H\|\epsilon}H\right)/\mathcal{Z} \tag{31}$$

satisfies:

$$tr\left(H\left(\widetilde{\sigma} - \Phi(\rho)\right)\right) \leq \|H\|\epsilon. \tag{32}$$

That is, $\widetilde{\sigma} \in \mathcal{C}(\rho, H, \epsilon)$.

Proof. We will prove this by contradiction. Suppose that for no $\lambda \in [0, D(\Phi(\rho)||\sigma)]$ we have that $\widetilde{\sigma}$ satisfies Eq. (32). Define the sequence of states

$$\sigma_t = \exp\left(-H_\infty - \frac{t\epsilon}{4\|H\|}H\right)/\mathcal{Z}_t$$

for $t \geq 0$. Note that by our assumption, we have for all $t \in [0, 4\epsilon^{-2}D(\Phi(\rho)||\sigma)]$ that:

$$\mathrm{tr}\left(H\left(\sigma_t - \Phi(\rho)\right)\right) > \|H\|\epsilon.$$

Thus, it follows from 4 that:

$$D(\Phi(\rho)\|\sigma_{t+1}) - D(\Phi(\rho)\|\sigma_t) \leq -\frac{\epsilon^2}{4}.$$

We conclude that

$$D(\Phi(\rho)||\sigma_t) \leq -t\frac{\epsilon^2}{4} + D(\Phi(\rho)||\sigma_0). \tag{33}$$

If we pick $t = \lceil 4\epsilon^{-2}D(\Phi(\rho)||\sigma)\rceil$, we get that

$$D(\Phi(\rho)||\sigma_t) < 0,$$

which contradicts the positivity of the relative entropy. □

Note that the Lemma above only ensures that the state $\widetilde{\sigma}$ reproduces approximately the same energy as the output of the noisy circuit. It does not promise us that

the state $\widetilde{\sigma}$ approximates the output $\Phi(\rho)$ for other observables. This is because as soon as $\widetilde{\sigma} \in \mathcal{C}(\Phi(\rho), H, \epsilon)$, we do not have any information on how to update the Hamiltonian of $\widetilde{\sigma}$ solely based on the energy and further expectation values are required to ensure convergence.

However, a close inspection of the proof above shows that if the value of t we need to observe comparable energies to $\Phi(\rho)$ is high, then we are ensured that $\widetilde{\sigma}$ is a good approximation to $\Phi(\rho)$. Indeed, we see from Eq. (33) that:

$$\begin{aligned}\frac{1}{2}\|\Phi(\rho) - \sigma_t\|_{\mathrm{tr}}^2 &\leq D(\Phi(\rho)||\sigma_t) \\ &\leq -t\frac{\epsilon^2}{4} + D(\Phi(\rho)||\sigma_0).\end{aligned}$$

Thus, if the λ required to observe similar energies to $\Phi(\rho)$ is of order $D(\Phi(\rho)||\sigma_0)$, then we are also guaranteed a good global approximation.

B. Entropic convergence in quantum circuits

The goal of this subsection is to obtain the entropic convergence required for our framework. Let us start by bounding the convergence of the entropy in discrete time, i.e. in quantum circuits. The main technical result we need for this is the data-processed inequality of [73, Theorem III.1]. It asserts that for a quantum channel P and states ρ, σ and σ' we have:

$$D(P(\rho)\|\sigma) \leq D(\rho\|\sigma') + D_\infty(P(\sigma')\|\sigma).$$

We then have:

Lemma 6. *Let $T : \mathcal{M}_{2^n} \to \mathcal{M}_{2^n}$ be a quantum channel with fixed point σ that satisfies a strong data-processing inequality with constant $\alpha > 0$. That is,*

$$D(T(\rho)||\sigma) \leq (1-\alpha)D(\rho||\sigma)$$

for all states ρ. Then for any other quantum channels $\Phi_1, \ldots, \Phi_m : \mathcal{M}_{2^n} \to \mathcal{M}_{2^n}$ we have:

$$\begin{aligned}&D(\textstyle\bigcirc_{t=1}^m(\ \Phi_t \circ T)(\rho)||\sigma) \leq (1-\alpha)^m D(\rho||\sigma) + \\ &\sum_{t=0}^{m-1}(1-\alpha)^{m-t} D_\infty(\Phi_t(\sigma)||\sigma),\end{aligned} \tag{34}$$

where

$$D_\infty(\rho||\sigma) = \log(\|\sigma^{-\frac{1}{2}}\rho\sigma^{-\frac{1}{2}}\|).$$

Proof. We will prove this by mathematical induction. For $m = 1$, this follows from the data-processed triangle inequality of [73, Theorem III.1]. Following their notation with $P = \ \Phi$ and $\sigma' = \sigma$ we have:

$$D(\ \Phi_1 \circ T)(\rho)||\sigma) \leq D(T(\rho)||\sigma) + D_\infty(\Phi_1(\sigma)||\sigma). \tag{35}$$

Let us now assume the claim to be true for $m = k$ and let us show it for $m = k+1$. We have:

$$\begin{aligned}&D((\textstyle\bigcirc_{t=1}^{k+1}(\ \Phi_t \circ T)(\rho)||\sigma) \leq (1-\alpha)^k D(\ \Phi_{k+1} \circ T)(\rho)||\sigma) + \\ &\sum_{t=0}^{k-1}(1-\alpha)^{m-t} D_\infty(\Phi_t(\sigma)||\sigma)\end{aligned} \tag{36}$$

by our induction hypothesis. Applying Eq. (35) to the first term in Eq. (36) and applying the strong data-processing inequality we obtain the claim. □

The lemma above yields a good estimate on the decay of the relative entropy in two (not mutually exclusive) scenarios. First, when the state $\sigma = e^{-\gamma H_1}/\mathrm{tr}(e^{-\gamma H_1})$ is a high temperature Gibbs state. Indeed, in this case we have, as mentioned before, that $D_\infty(\Phi(\sigma)||\sigma) \leq 2\gamma\|H_1\|$. Thus, the second term in (34) will be small irrespective of the unitaries being implemented if $\gamma \ll \|H_1\|^{-1}$. In particular, in the doubly stochastic case, i.e. $\sigma = \frac{I}{2^n}$, we have $D_\infty(\Phi(\sigma)||\sigma) = 0$. The second scenario is the one in which the quantum circuit approximately preserves the fixed point at late times of the evolution, i.e. $\Phi_t(\sigma) \simeq \sigma$ for all $t \geq m_0$. In this case, the $(1-\alpha)^{m-t}$ prefactor makes sure that the contribution of $D_\infty(\Phi_t(\sigma)||\sigma)$ is small for $t \leq m_0$ and $D_\infty(\Phi_t(\sigma)||\sigma)$ is small by our assumption that $\Phi_t(\sigma) \simeq \sigma$ for $t > m_0$. For instance, this could be the case for a circuit consisting overwhelmingly of diagonal gates at the end of the computation under decoherence, that is, σ a diagonal state. A concrete example of such a circuit would be the quantum Fourier transform.

IX. QUANTUM ANNEALERS

A. Entropic convergence in quantum annealers

The statement of Lemma 6 allows us to analyse the entropic convergence of circuits in discrete time. Let us now generalize these results to quantum annealers under uniform noise. The first step will be to generalize Lemma 6 for a fixed Lindbladian and a fixed Hamiltonian:

Lemma 7. *Let $\mathcal{L} : \mathcal{M}_{2^n} \to \mathcal{M}_{2^n}$ be a Lindbladian with fixed point σ satisfying a MLSI with constant $\alpha > 0$. Moreover, let $\mathcal{H} : \mathcal{M}_{2^n} \to \mathcal{M}_{2^n}$ be given by $\mathcal{H}(X) = i[H, X]$ for some Hamiltonian H. Then for all states ρ and times $t > 0$:*

$$\begin{aligned}D(e^{t(\mathcal{L}+\mathcal{H})}(\rho)||\sigma) &\leq e^{-\alpha t} D(\rho||\sigma) \\ &+ \alpha^{-1}(1-e^{-\alpha t})\|\sigma^{-\frac{1}{2}}[H,\sigma]\sigma^{-\frac{1}{2}}\|.\end{aligned}$$

Proof. Denote by $T_m = e^{\frac{t}{m}\mathcal{L}}$ and $\Phi_m = e^{\frac{t}{m}\mathcal{H}}$ for some $m \in \mathbb{N}$. By the Lie- Trotter formula we have:

$$\lim_{m\to\infty}(\Phi_m T_m)^m = e^{t(\mathcal{L}+\mathcal{H})}.$$

By Lemma 6 we have for all m:

$$D((\Phi_m \circ T_m)^m(\rho)||\sigma) \leq e^{-t\alpha} D(\rho||\sigma) + \sum_{t=1}^{m} e^{-\frac{\alpha t}{m}} D_\infty(\Phi_m(\sigma)||\sigma),$$

as each T_m contracts the relative entropy by $e^{-\frac{\alpha t}{m}}$. First, we can sum the geometric series and see that

$$\sum_{t=1}^{m} e^{-\frac{\alpha t}{m}} = \frac{(1-(e^{-t\alpha}))}{1-e^{-\alpha t/m}}$$

Let us now bound the limit:

$$\lim_{m\to\infty} \frac{(1-(e^{-t\alpha}))}{1-e^{-\alpha t/m}} D_\infty(\Phi_m(\sigma)||\sigma).$$

By a Taylor expansion:

$$\frac{(1-(e^{-t\alpha/2}))}{1-e^{-\alpha t/m}} = \frac{1-(e^{-t\alpha})}{\alpha t + \mathcal{O}(m^{-1})} m.$$

Thus, let us bound the limit

$$\lim_{m\to\infty} m D_\infty(\Phi_m(\sigma)||\sigma).$$

As $\log(1+x) \leq x$, we have:

$$m D_\infty(\Phi_m(\sigma)||\sigma) \leq m\left(\|\sigma^{-\frac{1}{2}} e^{\frac{t}{m}\mathcal{H}}(\sigma)\sigma^{-\frac{1}{2}}\| - 1\right).$$

Performing a Taylor expansion of $e^{\frac{t}{m}\mathcal{H}}$ followed by a triangle inequality we see that

$$\|\sigma^{-\frac{1}{2}} e^{\frac{t}{m}\mathcal{H}}(\sigma)\sigma^{-\frac{1}{2}}\| \leq 1 + \frac{t}{m}\|\sigma^{-\frac{1}{2}}[H,\sigma]\sigma^{-\frac{1}{2}}\| + \mathcal{O}\left(\frac{t^2}{m^2}\right).$$

We conclude that

$$m\left(\|\sigma^{-\frac{1}{2}} e^{\frac{t}{m}\mathcal{H}}(\sigma)\sigma^{-\frac{1}{2}}\| - 1\right) \leq t\|\sigma^{-\frac{1}{2}}[H,\sigma]\sigma^{-\frac{1}{2}}\| + \mathcal{O}\left(\frac{t^2}{m}\right),$$

which gives:

$$\lim_{m\to\infty} \frac{(1-(e^{-t\alpha}))}{1-e^{-\alpha t/m}} D_\infty(\Phi_m(\sigma)||\sigma) \leq \frac{(1-e^{-t\alpha})}{\alpha}\|\sigma^{-\frac{1}{2}}[H,\sigma]\sigma^{-\frac{1}{2}}\|$$

□

This allows us to control the scaling of the relative entropy with a fixed Hamiltonian term and gives the following simple corollary:

Corollary 2. *Let $\mathcal{S}_1, \mathcal{S}_2, \ldots, \mathcal{S}_m$ be Lindbladians of the form $\mathcal{S}_l = \mathcal{L} + \mathcal{H}_l$. Here $\mathcal{L}$ is a fixed Lindbladian with fixed point $\sigma > 0$ satisfying a MLSI with constant $\alpha > 0$. Then for all states ρ and times $t_1, \ldots, t_m$:*

$$D(\bigcirc_{l=1}^{m} e^{t_l \mathcal{S}_{t_l}}(\rho)||\sigma) \leq e^{-\alpha T} D(\rho||\sigma) + \alpha^{-1} \sum_{l=1}^{m} e^{-\alpha T_l}(1-e^{-\alpha t_l})\|\sigma^{-\frac{1}{2}}[H_l,\sigma]\sigma^{-\frac{1}{2}}\| \quad (37)$$

with $T_l = \sum_{j=1}^{l} t_l$.

Proof. We will prove the claim by mathematical induction. For $m = 1$, the claim is just Lemma 7. Assume the claim is true for all $n \leq k$. It follows from our induction hypothesis that:

$$D(\bigcirc_{l=1}^{k+1} e^{t_l \mathcal{S}_{t_l}}(\rho)||\sigma) \leq e^{-T_k} D(e^{t_{k+1}\mathcal{S}_{k+1}}(\rho)||\sigma) + \alpha^{-1} \sum_{l=1}^{k} e^{-\alpha T_l}(1-e^{-\alpha t_l})\|\sigma^{-\frac{1}{2}}[H_l,\sigma]\sigma^{-\frac{1}{2}}\|$$

It again follows from Lemma 7 that:

$$e^{-T_k} D(e^{t_{k+1}\mathcal{S}_{k+1}}(\rho)||\sigma) \leq e^{-T_{k+1}} D(\rho||\sigma) + \alpha^{-1} \sum_{l=1}^{k+1} e^{-\alpha T_l}(1-e^{-\alpha t_l})\|\sigma^{-\frac{1}{2}}[H_l,\sigma]\sigma^{-\frac{1}{2}}\|,$$

which yields the claim. □

The next step is to consider the evolution of a Lindbladian with a time-dependent Hamiltonian but uniform dissipative part and bound its relative entropy. That is, let $\mathcal{L} : \mathcal{M}_{2^n} \to \mathcal{M}_{2^n}$ be a Lindbladian with fixed point σ. Moreover, let $\mathcal{H}_t : \mathcal{M}_{2^n} \to \mathcal{M}_{2^n}$ be given by $\mathcal{H}_t(X) = i[H_t, X]$ for some time-dependent Hamiltonian H_t. Letting $\rho(t)$ be the solution to

$$\frac{d}{dt}\rho(t) = \mathcal{S}_t(\rho(t)), \quad \rho(0) = \rho,$$

for some initial state ρ one can show that $\rho(t) = \mathcal{T}_t(\rho)$ with

$$\mathcal{T}_t(\rho) = \lim_{n\to\infty} \bigcirc_{l=1}^{n} e^{\frac{t}{n}\mathcal{S}_{t_l}}(\rho).$$

Our strategy to estimate the convergence of the relative entropy $\mathcal{T}_t$ will be to apply the previous bounds for each $e^{\frac{t}{n}\mathcal{S}_{t_l}}$ individually and then take the $n \to \infty$ limit. We then obtain

Theorem 2. *Let $\mathcal{L} : \mathcal{M}_{2^n} \to \mathcal{M}_{2^n}$ be a Lindbladian with fixed point σ satisfying a MLSI with constant $\alpha > 0$. Moreover, let $\mathcal{H}_t : \mathcal{M}_{2^n} \to \mathcal{M}_{2^n}$ be given by $\mathcal{H}_t(X) = i[H_t, X]$ for some time-dependent Hamiltonian H_t. Moreover, let $\mathcal{T}_t$ be the evolution of the system under the Lindbladian $\mathcal{S}_t = \mathcal{L} + \mathcal{H}_t$ from time 0 to t. Then for all states ρ and times $t > 0$:*

$$D(\mathcal{T}_t(\rho)||\sigma) \leq e^{-\alpha t} D(\rho||\sigma) + \int_0^t d\tau e^{-\alpha(t-\tau)}\|\sigma^{-\frac{1}{2}}[H_t,\sigma]\sigma^{-\frac{1}{2}}\|. \quad (38)$$

Proof. We have

$$\mathcal{T}_t(\rho) = \lim_{n\to\infty} \bigcirc_{l=1}^{n} e^{\frac{t}{n}\mathcal{S}_{t_l}}(\rho)$$

with $t_l = \frac{tl}{n}$. It follows from Cor. 2 for any fixed n,time $t \geq 0$ and initial state ρ we have:

$$D(\bigcirc_{l=1}^{n} e^{\frac{t}{n}\mathcal{L}_{t_l}}(\rho)||\sigma) \leq e^{-\alpha t} D(\rho||\sigma)$$
$$+ \alpha^{-1} \sum_{l=1}^{n} e^{-\alpha\frac{nt-tl}{n}}(1-e^{-\frac{\alpha}{n}})\|\sigma^{-\frac{1}{2}}[H_{t_l},\sigma]\sigma^{-\frac{1}{2}}\|. \quad (39)$$

We will now take the limit $n \to \infty$ for a fixed t. First note that:

$$\alpha^{-1} \sum_{l=1}^{n} e^{-\alpha\frac{nt-tl}{n}}(1-e^{-\frac{\alpha}{n}})\|\sigma^{-\frac{1}{2}}[H_{t_l},\sigma]\sigma^{-\frac{1}{2}}\| =$$
$$\sum_{l=1}^{n} \frac{1}{n} e^{-\alpha\frac{nt-tl}{n}}\|\sigma^{-\frac{1}{2}}[H_{t_l},\sigma]\sigma^{-\frac{1}{2}}\| + \mathcal{O}\left(\frac{1}{n^2}\right)$$

by performing a Taylor expansion of the $(1-e^{-\frac{\alpha}{n}})$ term. We conclude that

$$\lim_{n\to\infty} \alpha^{-1} \sum_{l=1}^{n} e^{-\alpha\frac{nt-tl}{n}}(1-e^{-\frac{\alpha}{n}})\|\sigma^{-\frac{1}{2}}[H_{t_l},\sigma]\sigma^{-\frac{1}{2}}\| =$$
$$\lim_{n\to\infty} \sum_{l=1}^{n} \frac{1}{n} e^{-\alpha\frac{nt-tl}{n}}\|\sigma^{-\frac{1}{2}}[H_{t_l},\sigma]\sigma^{-\frac{1}{2}}\| =$$
$$\int_0^t d\tau e^{-\alpha(t-\tau)}\|\sigma^{-\frac{1}{2}}[H_\tau,\sigma]\sigma^{-\frac{1}{2}}\|.$$

This yields the claim. □

As Lemma 6 was the main building block to prove this bound, it should come as no surprise that Theorem 2 performs well in the regimes in which Lemma 6 does too. That is, either for σ a high temperature Gibbs state or $[H_t,\sigma] = 0$. In the first case, the commutator $\|\sigma^{-\frac{1}{2}}[H_{t_l},\sigma]\sigma^{-\frac{1}{2}}\|$ is small for all times. In the second case we, have that the evolution will leave σ approximately invariant at the end of the computation, resulting in small values $\|\sigma^{-\frac{1}{2}}[H_{t_l},\sigma]\sigma^{-\frac{1}{2}}\|$ irrespective of the temperature. On the other hand, the contribution of $\|\sigma^{-\frac{1}{2}}[H_{t_l},\sigma]\sigma^{-\frac{1}{2}}\|$ at the beginning of the computation is exponentially suppressed in the integral in Eq. (38), even for σ a Gibbs state at very high inverse temperature. This phenomenon is illustrated in the main text with annealers supposed to prepare the ground state of a classical Hamiltonian and σ a diagonal quantum state.

B. Noise model for adiabatic quantum computation

To the best of our knowledge, spontaneous emissions, decoherence and control errors are the main sources of noise in superconducting qubits [45], the leading platform for state of the art implementations of quantum annealers. We will now demonstrate our technique on a Lindbladian reflecting these sources of noise. In order not to overcomplicate the presentation, we will assume that there is no cross-talk between the qubits and that each term of the Lindbladian only acts on one qubit. Moreover, we will model the control error as follows. At each time s, instead of implementing the Hamiltonian H_s we implement the random Hamiltonian

$$\widetilde{H}_s = \sum_{i\sim j}(a_{i,j} + \xi^{z,z}_{i,j}(s))Z_iZ_j$$
$$+ \sum_i (b_i + \xi^z_i(s))Z_i - \sum(\Gamma_i(s) + \xi^x_i(s))X_i,$$

where ξ^z_i, ξ^x_i is white noise with the same standard deviation r_3 and mean 0. But it is straightforward to adapt our techniques to other noise models. Moreover, as we will see later, the $\xi^{z,z}_{i,j}$ control errors will only lead to extra dephasing that does not influence our analysis. Thus, we will set the Z_iZ_j control errors to 0. All these assumptions lead to a Lindbladian with $\mathcal{L}$ given by $\mathcal{L} = \sum_i \mathcal{L}_i$, where $\mathcal{L}_i$ acts on qubit i and has the terms:

$$\mathcal{L}_i = r_1\mathcal{L}_{amp} + r_2\mathcal{L}_{deph} + r_3\mathcal{L}_{cont}, \quad (40)$$

where $\mathcal{L}_{amp}$ is the amplitude damping term, $\mathcal{L}_{deph}$ the dephasing and $\mathcal{L}_{cont}$ the control error. One can show that under our white noise assumption, the control error is given by:

$$\mathcal{L}_{cont}(\rho) = X\rho X + Z\rho Z - 2\rho.$$

We refer to e.g. [74] for a derivation of this term. Each local term $\mathcal{L}_i$ then acts as:

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} \mapsto$$
$$\begin{pmatrix} r_1 d + r_3(d-a) & -(3r_3 + r_2 + \frac{r_1}{2})b + r_3 c \\ -(3r_3 + r_2 + \frac{r_1}{2})c + r_3 b & -r_1 d + r_3(a-d) \end{pmatrix}$$

on 2×2 matrices. Moreover, we assume for simplicity that the noise rates r_1, r_2 and r_3 are the same on each qubit. One can then easily solve $\mathcal{L}_i(\sigma) = 0$ for a state σ and conclude that the fixed point of this evolution is given by

$$\sigma = \begin{pmatrix} \frac{r_1+r_3}{r_1+2r_3} & 0 \\ 0 & \frac{r_3}{r_1+2r_3} \end{pmatrix}. \quad (41)$$

Thus, we see that in the regime $r_1 \gg r_3$, the fixed point is essentially the $|0\rangle$ state, for $r_3 \gg r_1$ we have a state close to maximally mixed state and for $r_1 \simeq r_3$ we have the state $\frac{2}{3}|0\rangle\langle 0| + \frac{1}{3}|1\rangle\langle 1|$. Note that we can write $\mathcal{L}_i$ as $\mathcal{L}_i = (r_1+2r_3)\mathcal{L}_\sigma + \mathcal{L}'$, where $\mathcal{L}_\sigma(\rho) = \mathrm{tr}\,(\rho)\,\sigma - \rho$ and $\mathcal{L}'$ is a Lindbladian s.t. $\mathcal{L}'(\sigma) = 0$. This implies that $\mathcal{L}_i$ satisfies a MLSI with the same constant as $(r_1+2r_3)\mathcal{L}_\sigma$. This gives a MLSI for $\mathcal{L}$ with $\alpha(r_1,r_3) = r_1 + 2r_3$ [44, Theorem 19]. Note that the constant r_2 does not influence the fixed point or the relative entropy convergence. This is why

we have not added the extra dephasing generated by the control errors of the Z_iZ_j terms. This also indicates that our convergence analysis is suboptimal for small times. Indeed, as we pick our initial state as $|\psi_0\rangle = |+\rangle^{\otimes n}$, it is not difficult to see that the initial convergence rate scales like $r_1 + 2r_3 + r_2$. However, at the end of the evolution we expect the state to mostly be diagonal and, thus, the action of the dephasing noise is negligible. We conclude that for large times the convergence rate is indeed of the order $r_1 + 2r_3$. It should also be noted that some works suggest that the dephasing noise actually occurs in the eigenbasis of H_s [46]. As argued in this work, if the state at time s is close to the ground state of H_s, the effect of dephasing in the eigenbasis of H_s is negligible. On the other hand, note that the dephasing obtained because of the Z_iZ_j terms is necessarily in the computational basis.

C. Computations required for linear path

Here we gather the computations required to exemplify how inequality (12) behaves for the path $H_s = \frac{s}{T}H_0 + \left(1 - \frac{t}{T}\right)H_I$ if we evolve from the initial state $\rho = |+\rangle\langle+|^{\otimes n}$ under $\mathcal{S}_s$ from 0 to T. We will assume that the noise models is given as in Eq. (40) with fixed point σ given by the tensor product of the state in Eq. (13). Note that as H_I commutes with σ, we obtain from Eq. (12) that:

$$D(\mathcal{T}_T(\rho)||\sigma) \leq e^{-\alpha T}D(\rho||\sigma) + \int_0^T d\tau e^{-\alpha(T-\tau)}\left(1 - \frac{\tau}{T}\right)\|\sigma^{-\frac{1}{2}}[H_0,\sigma]\sigma^{-\frac{1}{2}}\|. \tag{42}$$

Integrating Eq. (42), we obtain:

$$D(\mathcal{T}_T(\rho)||\sigma) \leq e^{-\alpha T}D(\rho||\sigma) + \frac{1}{\alpha^2 T}\left(1 - e^{-\alpha T}\alpha T - e^{-\alpha T}\right)\|\sigma^{-\frac{1}{2}}[H_0,\sigma]\sigma^{-\frac{1}{2}}\|. \tag{43}$$

By a triangle inequality and noting that the state σ commutes with the Z_i terms, we can further bound the commutant by:

$$\|\sigma^{-\frac{1}{2}}[H_0,\sigma]\sigma^{-\frac{1}{2}}\| \leq \sum_i |\Gamma_i|\|\sigma^{-\frac{1}{2}}[X_i,\sigma]\sigma^{-\frac{1}{2}}\|.$$

It will be convenient to reparametrize the state as $\sigma = (e^{\gamma} + e^{-\gamma})^{-n} \otimes_{i=1}^n e^{\gamma Z_i}$ for $\gamma = \frac{1}{2}\log\left(\frac{r_1+r_3}{r_3}\right)$ and let $r = r_1 + 2r_3$. One can then show that $\|\sigma^{-\frac{1}{2}}[X_i,\sigma]\sigma^{-\frac{1}{2}}\| = 2\sinh(\gamma)$. Putting all these elements together, we conclude that with this noise model the relative entropy after evolving the system for time T is bounded by

$$D(\mathcal{T}_T(|+\rangle\langle+|^{\otimes n})||\sigma) \leq e^{-rT}n\log(2\cosh(\gamma)) + \frac{2\sinh(\gamma)\left(1 - e^{-rT}rT - e^{-rT}\right)}{r^2T}n\overline{\Gamma}. \tag{44}$$

For the sake of conciseness, denote the r.h.s. of Eq. (44) by $nf(\gamma, r, T, \overline{\Gamma})$ with

$$f(\gamma, r, T, \overline{\Gamma}) = e^{-rT}\log(2\cosh(\gamma)) + \frac{2\sinh(\gamma)\left(1 - e^{-rT}rT - e^{-rT}\right)}{r^2T}\overline{\Gamma}. \tag{45}$$

In terms of the noise rates and simplifying the expressions a bit we have:

$$f(r_1, r_3, T, \overline{\Gamma}) \leq e^{-(r_1+2r_3)T}\log\left(\frac{r_1 + 2r_3}{\sqrt{r_3(r_1+r_3)}}\right) + r_1\frac{1 - e^{-(r_1+2r_3)T}(r_1+2r_3)T - e^{-(r_1+2r_3)T}}{\sqrt{r_1+r_3}(r_1+2r_3)^2T}\overline{\Gamma}.$$

Error mitigation for short-depth quantum circuits

Kristan Temme, Sergey Bravyi and Jay M. Gambetta
IBM T.J. Watson Research Center, Yorktown Heights NY 10598
(Dated: November 7, 2017)

Two schemes are presented that mitigate the effect of errors and decoherence in short-depth quantum circuits. The size of the circuits for which these techniques can be applied is limited by the rate at which the errors in the computation are introduced. Near-term applications of early quantum devices, such as quantum simulations, rely on accurate estimates of expectation values to become relevant. Decoherence and gate errors lead to wrong estimates of the expectation values of observables used to evaluate the noisy circuit. The two schemes we discuss are deliberately simple and don't require additional qubit resources, so to be as practically relevant in current experiments as possible. The first method, extrapolation to the zero noise limit, subsequently cancels powers of the noise perturbations by an application of Richardson's deferred approach to the limit. The second method cancels errors by resampling randomized circuits according to a quasi-probability distribution.

arXiv:1612.02058v3 [quant-ph] 6 Nov 2017

From the time quantum computation generated wide spread interest, the strongest objection to its viability was the sensitivity to errors and noise. In an early paper, William Unruh [1] found that the coupling to the environment sets an ultimate time and size limit for any quantum computation. This initially curbed the hopes that the full advantage of quantum computing could be harnessed, since it set limits on the scalability of any algorithm. This problem was, at least in theory, remedied with the advent of quantum error correction [2–4]. It was proven that if both the decoherence and the imprecision of gates could be reduced below a finite threshold value, then quantum computation could be performed indefinitely [5, 6]. Although it is the ultimate goal to reach this threshold in an experiment that is scalable to larger sizes, the overhead that is needed to implement a fully fault-tolerant gate set with current codes [7] seems prohibitively large [8, 9]. In turn, it is expected that in the near term the progress in quantum experiments will lead to devices with dynamics, which are beyond what can be simulated with a conventional computer. This leads to the question: what computational tasks could be accomplished with only limited, or no error correction?

The suggestions of near-term applications in such quantum devices mostly center around quantum simulations with short-depth circuit [10–12] and approximate optimization algorithms [13]. Furthermore, certain problems in material simulation may be tackled by hybrid quantum-classical algorithms [14]. In most such applications, the task can be abstracted to applying a short-depth quantum circuits to some simple initial state and then estimating the expectation value of some observable after the circuit has been applied. This estimation must be accurate enough to achieve a simulation precision comparable or exceeding that of classical algorithms. Yet, although the quantum system evolves coherently for the most part of the short-depth circuit, the effects of decoherence already become apparent as an error in the estimate of the observable. For the simulation to be of value, the effect of this error needs to be mitigated.

In this paper we introduce two techniques for *quantum error mitigation* that increase the quality of any such short-depth quantum simulations. We find that the accuracy of the expectation value can be increased significantly in the presence of noise. We are looking for error mitigation techniques that are as simple as possible and don't require additional quantum resources. Both techniques require that some noise parameter taken together with system size and circuit depth can be considered a small number. The first scheme does not make any assumption about the noise model other than it being weak and constant in time. In comparison, the second scheme can tolerate stronger noise; however, it requires detailed knowledge of the noise model.

Extrapolation to the zero noise limit: It is our goal to estimate the expectation value of some quantum observable A with respect to an evolved state $\rho_\lambda(T)$ after time T that is subject to noise characterized by the parameter λ in the limit where $\lambda \to 0$. To achieve this, we apply Richardson's deferred approach to the limit to cancel increasingly higher orders of λ [15].

Although gates are typically used to describe quantum circuits, for our analysis it is more convenient to consider the time-dependent Hamiltonian dynamics implementing the circuit. The time-dependent multi-qubit Hamiltonian is denoted by $K(t)$. It can be expanded into N - qubit Pauli operators $P_\alpha \in \langle \mathbb{1}, X_j, Y_j, Z_j \rangle_{j=1\ldots N}$, where X_j, Y_j, Z_j acts as a single-qubit Pauli matrix on site j and trivially elsewhere. We allow for time-dependent coupling coefficients $J_\alpha(t) \in \mathbb{R}$. The circuit is encoded as $K(t) = \sum_\alpha J_\alpha(t) P_\alpha$. The total evolution of the open system with initial state ρ_0 will be described by an equation of the following form:

$$\frac{\partial}{\partial t}\rho(t) = -i[K(t), \rho(t)] + \lambda \mathcal{L}(\rho(t)) \tag{1}$$

for time $t \in [0, T]$. We do not specify the exact form of the generator $\mathcal{L}(\rho)$ but only require that it is invariant under time rescaling and independent from the parameters $J_\alpha(t)$ in $K(t)$. The noise term $\mathcal{L}(\rho)$ could be given as a Lindblad operator, or it could correspond to a Hamiltonian that couples to a bath to model non-Markovian dynamics. We ask that there is a parameter $\lambda \ll 1$ that indicates a weak action of the noise and that we can bound $\|\mathcal{L}_{I,t_1} \circ \mathcal{L}_{I,t_2} \circ \ldots \circ \mathcal{L}_{I,t_n}(\rho)\|_1 \leq l_n$, where at most $l_n = \mathcal{O}(N^n)$. The map $\mathcal{L}_{I,t}$ is short-hand notation for the transformation of $\mathcal{L}$ into the interaction frame generated

by $K(t)$.

The expectation value of the observable A is obtained from the final state $\rho_\lambda(T)$ as $E_K(\lambda) = \mathrm{tr}\,(A\rho_\lambda(T))$. The function $E_K(\lambda)$ can be expressed as a series in λ where the contribution with λ^0 corresponds to the noise-free evolution. This can be seen by transforming the evolution into the interaction frame w.r.t $K(t)$ and expanding the Born series, c.f. supp. mat. sec I. Starting from the noise-free expectation value $E^* = \mathrm{tr}\,(A\rho_0(T))$, the expansion is given by

$$E_K(\lambda) = E^* + \sum_{k=1}^{n} a_k \lambda^k + R_{n+1}(\lambda, \mathcal{L}, T). \tag{2}$$

The a_k are model-specific constants typically growing like $a_k \sim N^k T^k$. Here $R_{n+1}(\lambda, \mathcal{L}, T)$ is the remainder of the expansion and can be bounded by $|R_{n+1}(\lambda, \mathcal{L}, T)| \leq \|A\| l_{n+1} (\lambda T)^{n+1}/(n+1)!$ by standard arguments. Since we assumed an extensive scaling of l_n, such an expansion is only meaningful whenever $NT\lambda$ is small. We are of course interested in $\lim_{\lambda\to 0} E_K(\lambda) = E^*$; however, we are faced with a small but finite parameter λ. Since we only have access to $E_K(\lambda)$, our estimate of E^* will be off by $\mathcal{O}(\lambda)$.

This estimate can be improved by Richardson's deferred approach to the limit [15, 16]. To explain the idea, let us assume we can run the quantum circuit at different noise rates λ_j, with $j = 0, \ldots, n$ and obtain experimental estimates $\hat{E}_K(\lambda_j) = E_K(\lambda_j) + \delta_j$. Here the $\lambda_j = c_j\lambda$ are appropriately rescaled values of the experimental noise rate λ. The estimate deviates from the actual expectation value due to experimental inaccuracies and finite sampling errors by an error δ_j. The estimate of E^* can be significantly improved by considering the approximation $\hat{E}^n_K(\lambda)$, which is written as the linear combination

$$\hat{E}^n_K(\lambda) = \sum_{j=0}^{n} \gamma_j \hat{E}_K(c_j\lambda). \tag{3}$$

Here we require the coefficients γ_j to satisfy the linear system of equations [16].

$$\sum_{l=0}^{n} \gamma_j = 1 \quad \text{and} \quad \sum_{j=0}^{n} \gamma_j\, c_j^k = 0 \quad \text{for } k = 1 \ldots n. \tag{4}$$

The linear combination Eq. (3) will be an approximation to E^* up to an error of order $\mathcal{O}(\lambda^{n+1})$.

To obtain estimates at different noise rates λ_j, we use a rescaling trick. We run the same circuit $n+1$ times with rescaled parameters in $K(t)$. We follow the protocol:

1. For $j = 0, \ldots, n$
   (a) choose a rescaling coefficient $c_j > 1$ ($c_0 = 1$) and evolve ρ_0 with rescaled Hamiltonian $K^j(t) = \sum_\alpha J^j_\alpha(t) P_\alpha$, where
   $$J^j_\alpha(t) = c_j^{-1} J_\alpha\left(c_j^{-1} t\right), \tag{5}$$
   for time $T_j = c_j T$.
   (b) Estimate observable A to obtain $\hat{E}_K(c_j\lambda)$.
2. Solve equations (4) and compute $\hat{E}^n_K(\lambda)$ as in Eq. (3).

A rescaling of the equations shows that the state $\rho^j_\lambda(T_j)$, which evolves under $\dot{\rho}^j_\lambda = -i[K^j(t), \rho^j] + \lambda\mathcal{L}(\rho^j)$ for time T_j, satisfies $\rho^j_\lambda(T_j) = \rho_{c_j\lambda}(T)$, c.f. supp. mat. sec. II. Hence the estimates $\hat{E}_K(c_j\lambda) = \mathrm{tr}\left(A\rho^j_\lambda(T_j)\right) + \delta_j$ can be obtained from the $n+1$ runs rescaled according to the protocol.

If the protocol is performed for $n+1$ steps, the error between the exact expectation value E^* and the estimator $\hat{E}^n_K(\lambda)$ can be bounded by

$$|E^* - \hat{E}^n_K(\lambda)| \leq \Gamma_n \left(\delta^* + \|A\| \frac{l_{n+1}(\lambda T)^{n+1}}{(n+1)!}\right). \tag{6}$$

Here $\Gamma_n = \sum_{j=0}^n |\gamma_j| c_j^{n+1}$ and $\delta^* = \max_j |\delta_j|$ is the largest experimental error.

This follows from repeated application of the triangle inequality, c.f. supp. mat. sec. III. The equations (4) can be solved, and one finds that the coefficients $\gamma_j = \prod_{m\neq j} c_m (c_j - c_m)^{-1}$, so that the constant Γ_n can be evaluated. In the literature [16], several choices for progression of c_j are common. The two most frequent series are exponential decrease (Bulirsch - Stoer) and harmonic decay. In our experiments we are actually increasing the noise rate starting from the optimal value, wheres it is common in the numerical literature to improve the small parameter. The result is, of course, the same.

Examples : To demonstrate this method we will consider three numerical examples. In all the examples the time evolution is given by a Hamiltonian $K(t)$ that encodes a control problem. For a single *drift step* we evolve with a Hamiltonian $K_R(t) = U_N(\theta) K_0 U_N^\dagger(\theta)$, where the single qubit product unitary $U_N(\theta) \in SU(2)^{\otimes N}$ is chosen Haar-random, and the drift Hamiltonian $K_0 = \sum_{i,j} J_{i,j} X_i Z_j$ is chosen with respect to a random graph and Gaussian distributed couplings $J_{i,j}$. The evolution is subject to three different noise models: first Fig 1(a), we evolve in the presence of depolarizing noise described by the sum of single qubit generators $\mathcal{L}_i = -\lambda(2^{-1}\mathrm{tr}_i(\rho) - \rho)$ acting on all N qubits. Second, Fig 1(b), we consider dephasing and amplitude damping noise on every qubit, where we have chosen a ratio of $\lambda_1/\lambda_2 = 1.5$ with a generator $\mathcal{L}_i = \lambda_1\left(\sigma_i^- \rho \sigma_i^+ - \frac{1}{2}\{\sigma_i^+\sigma_i^-, \rho\}\right) + \lambda_2\left(Z_i\rho Z_i - \rho\right)$ and $\sigma_i^\pm = 2^{-1}(X_i \pm iY_i)$. Third, Fig 1(c), we consider a highly non-Markovian setting, where each of the N qubits i is coupled to its own single-qubit bath b_i via the Hamiltonian $V_i = 1/2\; X_i \otimes X_{b_i} + 1/2\; Z_{b_i}$ and the bath is prepared in the initial state $\rho_B = (2\cosh(\beta/2))^{-N} \exp(-\beta \sum_{b_i} \sigma^z_{b_i})$. Then, after the evolution of each noisy circuit $T = td$ we measure a randomly chosen multi-qubit Pauli operator P_α.

The graphs in Fig 1 show that with modest effort very high precisions can be obtained. In the low noise range $\epsilon \sim 10^{-3}$

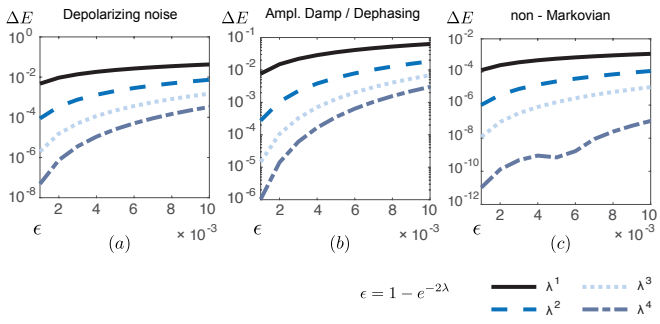


FIG. 1. (color online) The plots show a random Hamiltonian evolution for $N = 4$ system qubits and $d = 6$ drift steps, each for time $t = 2$. For all systems plot the error $\Delta E = |E^* - \hat{E}^n_K(\lambda)|$ for $n = 0, 1, 2, 3$. Here $\lambda^1, n = 0$ corresponds to the uncorrected error. The noise parameter $\lambda = -1/2 \log(1 - \epsilon)$ is chosen so that all plots have the same perturbation measured in the depolarizing strength $\epsilon = 10^{-3} \dots 10^{-2}$. The plot shows the mitigation of (a) Depolarizing noise (b) Amplitude damping / dephasing noise and (c) non-Markovian noise, for $\{c_j\}$ chosen as random partition of in the interval $[1, 4]$.

the relative error can be reduced to $\Delta E \sim 10^{-6} - 10^{-11}$. The precision is then essentially determined by the sampling error δ^*, which we have neglected in the plots.

Probabilistic error cancellation: Here we discuss a noise reduction scheme for quantum circuits subject to a Markovian noise. First let us state our assumptions on the noise model. A noisy N-qubit device will be described by a basis set of noisy operations $\Omega = \{\mathcal{O}_1, \dots, \mathcal{O}_m\}$ that can be implemented on this device. Each operation $\mathcal{O}_\alpha$ is a trace-preserving completely positive (TPCP) map on N qubits that acts non-trivially only on a small subset of qubits, say at most two. For example, $\mathcal{O}_\alpha$ could be a noisy unitary gate applied to a specified pair of qubits or a noisy qubit initialization. We assume that noise in the system can be fully characterized such that the map $\mathcal{O}_\alpha$ is known for each α. A circuit of length L in the basis Ω is a sequence of L operations from Ω. Let Ω_L be a set of all length-L circuits in the basis Ω. A circuit $\boldsymbol{\alpha} = (\alpha_1, \dots, \alpha_L)$ implements a map $\mathcal{O}_{\boldsymbol{\alpha}} = \mathcal{O}_{\alpha_L} \cdots \mathcal{O}_{\alpha_2} \mathcal{O}_{\alpha_1}$. The expectation value of an observable A on the final state produced by a noisy circuit $\boldsymbol{\alpha}$ is

$$E(\boldsymbol{\alpha}) = \mathrm{Tr}\big[A\, \mathcal{O}_{\boldsymbol{\alpha}}(|0\rangle\langle 0|^{\otimes n})\big].$$

For simplicity, we ignore errors in the initial state preparation and in the final measurement. Such errors can be accounted for by adding dummy noisy operations before each measurement and after each qubit initialization. Furthermore, we shall assume that A is diagonal in the Z-basis and $\|A\| \le 1$.

Below we show that under certain conditions the task of simulating an ideal quantum circuit can be reduced to estimating the expectation value $E(\boldsymbol{\alpha})$ for a suitable random ensemble of noisy quantum circuits $\boldsymbol{\alpha}$. Moreover, the ideal and the noisy circuits act on the same number of qubits and have the same depth.

Let $\Gamma = \{\mathcal{U}_1, \dots, \mathcal{U}_k\}$ be a fixed basis set of ideal gates. Each gate $\mathcal{U}_\beta(\rho) = U_\beta \rho U_\beta^\dagger$ is described by a unitary TPCP map on N qubits that acts non-trivially on a small subset of qubits. An ideal length-L circuit in the basis Γ is a sequence of L gates from Γ. A circuit $\boldsymbol{\beta} = (\beta_1, \dots, \beta_L)$ implements a map $\mathcal{U}_{\boldsymbol{\beta}} = \mathcal{U}_{\beta_L} \cdots \mathcal{U}_{\beta_2} \mathcal{U}_{\beta_1}$. Define an ideal expectation value

$$E^*(\boldsymbol{\beta}) = \mathrm{Tr}\big[A\, \mathcal{U}_{\boldsymbol{\beta}}(|0\rangle\langle 0|^{\otimes n})\big].$$

We consider a simulation task where the goal is to estimate $E^*(\boldsymbol{\beta})$ with a specified precision δ.

The key idea of our scheme is to represent the ideal circuit as a quasi-probabilistic mixture of noisy ones. Let us say that a noisy basis Ω simulates an ideal circuit $\boldsymbol{\beta}$ with the overhead $\gamma_{\boldsymbol{\beta}} \ge 1$ if there exists a probability distribution $P_{\boldsymbol{\beta}}(\boldsymbol{\alpha})$ on the set of noisy circuits $\boldsymbol{\alpha} \in \Omega_L$ such that

$$\mathcal{U}_{\boldsymbol{\beta}} = \gamma_{\boldsymbol{\beta}} \sum_{\boldsymbol{\alpha} \in \Omega_L} P_{\boldsymbol{\beta}}(\boldsymbol{\alpha}) \sigma_{\boldsymbol{\beta}}(\boldsymbol{\alpha}) \mathcal{O}_{\boldsymbol{\alpha}} \tag{7}$$

for some coefficients $\sigma_{\boldsymbol{\beta}}(\boldsymbol{\alpha}) = \pm 1$. We also require that the distribution $P_{\boldsymbol{\beta}}(\boldsymbol{\alpha})$ is sufficiently simple so that one can efficiently sample $\boldsymbol{\alpha}$ from $P_{\boldsymbol{\beta}}(\boldsymbol{\alpha})$. The coefficients $\gamma_{\boldsymbol{\beta}}, \sigma_{\boldsymbol{\beta}}(\boldsymbol{\alpha})$ must be efficiently computable. We shall refer to Eq. (41) as a *quasi-probability representation* (QPR) of the ideal circuit $\boldsymbol{\beta}$. Note that $\gamma_{\boldsymbol{\beta}} \ge 1$ because $\mathcal{U}_{\boldsymbol{\beta}}$ and $\mathcal{O}_{\boldsymbol{\alpha}}$ are trace-preserving. Quasi-probability distributions have been previously used to construct classical algorithms for simulation of quantum circuits [17, 18]. Our work can be viewed as an application of these methods to the problem of simulating ideal quantum circuits by noisy ones.

Substituting Eq. (41) into the definition of $E^*(\boldsymbol{\beta})$ gives

$$E^*(\boldsymbol{\beta}) = \gamma_{\boldsymbol{\beta}} \sum_{\boldsymbol{\alpha} \in \Omega_L} P_{\boldsymbol{\beta}}(\boldsymbol{\alpha}) \sigma_{\boldsymbol{\beta}}(\boldsymbol{\alpha}) E(\boldsymbol{\alpha}). \tag{8}$$

Let $\boldsymbol{\alpha} \in \Omega_L$ be a random variable drawn from $P_{\boldsymbol{\beta}}(\boldsymbol{\alpha})$ and $x \in \{0, 1\}^n$ be the final readout of the noisy circuit $\boldsymbol{\alpha}$ obtained by measuring each qubit of the final state $\mathcal{O}_{\boldsymbol{\alpha}}(|0\rangle\langle 0|^{\otimes n})$ in the Z-basis. Note that $\langle x|A|x\rangle$ is an unbiased estimator of $E(\boldsymbol{\alpha})$ with the variance $O(1)$. Thus from Eq. (42) one infers that $\gamma_{\boldsymbol{\beta}} \sigma_{\boldsymbol{\beta}}(\boldsymbol{\alpha}) \langle x|A|x\rangle$ is an unbiased estimator of the ideal expectation value $E^*(\boldsymbol{\beta})$ with the variance $O(\gamma^2_{\boldsymbol{\beta}})$. We can now estimate $E^*(\boldsymbol{\beta})$ with any desired precision δ by the Monte Carlo method. Define

$$M = (\delta^{-1} \gamma_{\boldsymbol{\beta}})^2 \tag{9}$$

and generate M samples $\boldsymbol{\alpha}^1, \dots, \boldsymbol{\alpha}^M \in \Omega_L$ drawn from $P_{\boldsymbol{\beta}}(\boldsymbol{\alpha})$. By Hoeffding's inequality, $E^*(\boldsymbol{\beta})$ is approximated within error $O(\delta)$ w.h.p. by a random variable

$$\hat{E}(\boldsymbol{\beta}) = \frac{\gamma_{\boldsymbol{\beta}}}{M} \sum_{a=1}^{M} \sigma_{\boldsymbol{\beta}}(\boldsymbol{\alpha}^a) \langle x^a|A|x^a\rangle, \tag{10}$$

where $x^a \in \{0, 1\}^n$ is the final string of the noisy circuit $\boldsymbol{\alpha}^a$. Computing the estimator $\hat{E}(\boldsymbol{\beta})$ requires M runs of the noisy circuits, with each run producing a single readout string

x^a. Estimating $E^*(\boldsymbol{\beta})$ with a precision δ in the absence of noise by Monte Carlo method would require approximately δ^{-2} runs. Thus the quantity $\gamma_{\boldsymbol{\beta}}^2$ determines the simulation overhead (see Eq. (9)).

A systematic method of constructing QPRs with a small overhead is given in supp. mat. sec. IV. Here we illustrate the method using toy noise models usually studied in the quantum fault-tolerance theory: the depolarizing noise and the amplitude damping noise. For concreteness, we choose the ideal gate set Γ as the standard Clifford+T basis.

Let $\mathcal{D}_k$ be the depolarizing noise on $k = 1, 2$ qubits that returns the maximally mixed state with probability ϵ and does nothing with probability $1-\epsilon$. Define a noisy version of a k-qubit unitary gate $\mathcal{U}$ as $\mathcal{D}_k\mathcal{U}$. The noisy basis Ω is obtained by multiplying ideal gates on the left by arbitrary Pauli operators and adding the depolarizing noise. Thus Ω is a set of operations $\mathcal{O}_\alpha = \mathcal{D}_k\mathcal{P}\mathcal{U}$, where $\mathcal{U} \in \Gamma$ is a k-qubit ideal gate and $\mathcal{P} \in \{\mathcal{I}, \mathcal{X}, \mathcal{Y}, \mathcal{Z}\}^{\otimes k}$ is a Pauli TPCP map. The random ensemble of noisy circuits $\mathcal{O}_{\boldsymbol{\alpha}}$ that simulates an ideal circuit $\mathcal{U}_{\boldsymbol{\beta}}$ is constructed in three steps: (1) Start from the ideal circuit, $\mathcal{O}_{\boldsymbol{\alpha}} = \mathcal{U}_{\boldsymbol{\beta}}$. (2) Modify $\mathcal{O}_{\boldsymbol{\alpha}}$ by adding a Pauli X, Y, Z after each single-qubit gate with probability $p_1 = \epsilon/(4+2\epsilon)$. The gate is unchanged with probability $1-3p_1$. (3) Modify $\mathcal{O}_{\boldsymbol{\alpha}}$ by adding a Pauli $IX, IY, \ldots, ZZ$ after each CNOT with probability $p_2 = \epsilon/(16+14\epsilon)$. The CNOT is unchanged with probability $1-15p_2$. The resulting circuit is then implemented on a noisy device (which adds the depolarizing noise after each gate) and the final readout string x is recorded. By generating M samples of x one can estimate $E^*(\boldsymbol{\beta})$ from Eq. (10). The sign function $\sigma_{\boldsymbol{\beta}}(\boldsymbol{\alpha})$ is equal to $(-1)^r$, where r is the number of Pauli operators added to the ideal circuit $\mathcal{U}_{\boldsymbol{\beta}}$. As shown in supp. mat. sec. IV, the above defines a QPR of the ideal circuit $\mathcal{U}_{\boldsymbol{\beta}}$ with the overhead $\gamma_{\boldsymbol{\beta}} \approx 1 + \epsilon(3L_1/2 + 15L_2/8)$, where L_1 is the number of single-qubit gates and L_2 is the number of CNOTs in the ideal circuit. The method has been tested numerically for random noisy Clifford+T circuits, see Fig. 2.

A more interesting example is the noise described by the amplitude-damping channel $\mathcal{A}$ that resets every qubit to its ground state with probability ϵ. A noisy version of a k-qubit unitary gate $\mathcal{U}$ is defined as $\mathcal{A}^{\otimes k}\mathcal{U}$. In contrast to the previous example, noisy unitary gates $\mathcal{A}^{\otimes k}\mathcal{U}$ alone cannot simulate any ideal unitary gate since $\mathcal{A}$ is not a unital map. To overcome this, we extend the noisy basis Ω by adding noisy versions of single-qubit state preparations $\mathcal{A}\mathcal{P}_{|\psi\rangle}$, where $\mathcal{P}_{|\psi\rangle}$ maps any input state to $|\psi\rangle\langle\psi|$. Our scheme requires state preparations for single qubit states $|\psi\rangle = |+\rangle, |-\rangle, |0\rangle, |1\rangle$ that can be performed at any time step (not only at the beginning). In supp. mat. sec. V we show how to construct a QPR of the ideal Clifford+T circuit $\mathcal{U}_{\boldsymbol{\beta}}$ with the overhead $\gamma_{\boldsymbol{\beta}} \approx 1 + \epsilon(2L_1 + 4L_2)$. The examples considered above suggest that well-characterized noisy circuits can simulate ideal ones with overhead $\gamma \approx (1+c\epsilon)^L$, where ϵ is the typical error rate and c is a small constant. The value of c can be determined by performing quantum process tomography [19] and finding the QPR for each ideal gate. Using Eq. (9), one can estimate the number of noisy circuit runs of length L as $M \sim \exp(2c\epsilon L)$. Assuming error rates in the range $\epsilon \sim 10^{-3}$, it may be possible to simulate ideal circuits with $O(10^3)$ gates.

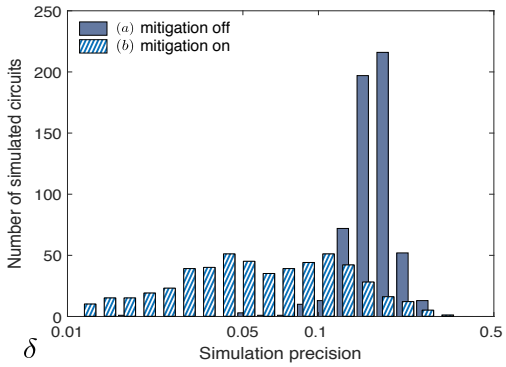


FIG. 2. Simulation precision $\delta(\beta) = |\hat{E}(\boldsymbol{\beta}) - E^*(\boldsymbol{\beta})|$ for 500 randomly generated ideal Clifford+T circuits on $N = 6$ qubits with depth $d = 20$. The gates are subject to single- and two-qubit depolarizing noise $\epsilon = 10^{-2}$. The figure shows results for simulations without (a) and with (b) error cancellation. In both cases each ideal circuit was simulated by $M = 4000$ runs of the noisy circuit. For each circuit $\mathcal{U}_{\boldsymbol{\beta}}$ we defined the observable A as a projector Π_{out} onto the subset of 2^{N-1} basis vectors with the largest weight in the final state. The results are consistent with $\gamma_\beta \approx 4.3$ so that $\gamma_\beta M^{-1/2} \approx 0.07$.

Conclusions: Both error mitigation schemes require no additional quantum hardware such as ancilla or code qubits and work directly with the physical qubits. The zero-noise extrapolation requires sufficient control of the time evolution to implement the rescaled dynamics and hinges on the assumption of a large time-scale separation between the dominant noise and the controlled dynamics. For the probabilistic error cancellation a full characterization of the noisy computational operations is necessary. To obtain this to a precision of $\sim 10^{-3}$ is challenging in practice. However, if one is willing to sacrifice optimality, a Pauli- or Clifford-twirling [20, 21] can be applied that converts any noise channel into a simple mixture of Pauli errors or depolarizing noise, making the characterization task much more manageable. A very recent independent paper by Li and Benjamin [22] discusses similar issues to those addressed here.

Acknowledgements: We thank Antonio Mezzacapo for insightful discussions, and we acknowledge support from the IBM Research Frontiers Institute

[1] W. G. Unruh, Physical Review A **51**, 992 (1995).
[2] P. W. Shor, Physical review A **52**, R2493 (1995).
[3] A. M. Steane, Physical Review Letters **77**, 793 (1996).
[4] A. R. Calderbank and P. W. Shor, Physical Review A **54**, 1098 (1996).

[5] D. Aharonov and M. Ben-Or, in *Proceedings of the twenty-ninth annual ACM symposium on Theory of computing* (ACM, 1997) pp. 176–188.
[6] A. Y. Kitaev, Russian Mathematical Surveys **52**, 1191 (1997).
[7] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, Physical Review A **86**, 032324 (2012).
[8] N. C. Jones, R. Van Meter, A. G. Fowler, P. L. McMahon, J. Kim, T. D. Ladd, and Y. Yamamoto, Physical Review X **2**, 031007 (2012).
[9] S. J. Devitt, A. M. Stephens, W. J. Munro, and K. Nemoto, Nature communications **4** (2013).
[10] A. Peruzzo, J. McClean, P. Shadbolt, M.-H. Yung, X.-Q. Zhou, P. J. Love, A. Aspuru-Guzik, and J. L. O?Brien, Nature communications **5** (2014).
[11] J. R. McClean, J. Romero, R. Babbush, and A. Aspuru-Guzik, arXiv preprint arXiv:1509.04279 (2015).
[12] D. Wecker, M. B. Hastings, and M. Troyer, Physical Review A **92**, 042303 (2015).
[13] E. Farhi, J. Goldstone, and S. Gutmann, arXiv preprint arXiv:1411.4028 (2014).
[14] B. Bauer, D. Wecker, A. J. Millis, M. B. Hastings, and M. Troyer, arXiv preprint arXiv:1510.03859 (2015).
[15] L. F. Richardson and J. A. Gaunt, Philosophical Transactions of the Royal Society of London. Series A, containing papers of a mathematical or physical character **226**, 299 (1927).
[16] A. Sidi, "Practical extrapolation methods: Theory and applications, volume 10 of cambridge monographs on applied and computational mathematics," (2003).
[17] H. Pashayan, J. J. Wallman, and S. D. Bartlett, Physical review letters **115**, 070501 (2015).
[18] N. Delfosse, P. A. Guerin, J. Bian, and R. Raussendorf, Physical Review X **5**, 021003 (2015).
[19] M. Mohseni, A. Rezakhani, and D. Lidar, Phys. Rev. A **77**, 032322 (2008).
[20] M. Silva, E. Magesan, D. W. Kribs, and J. Emerson, Physical Review A **78**, 012347 (2008).
[21] E. Magesan, J. M. Gambetta, and J. Emerson, Physical Review A **85**, 042311 (2012).
[22] Y. Li and S. C. Benjamin, arXiv preprint arXiv:1611.09301 (2016).
[23] G. Lindblad, Communications in Mathematical Physics **48**, 119 (1976).
[24] H.-P. Breuer and F. Petruccione, *The theory of open quantum systems* (Oxford University Press on Demand, 2002).
[25] C. Schneider, Numerische Mathematik **24**, 177 (1975).

SUPPLEMENTAL MATERIAL

Reducing noise by Richardson extrapolation

In numerical analysis, Richardson extrapolation [15, 16] is a sequence acceleration method, used to improve the rate of convergence of a sequence. We use the same technique to extrapolate to the zero-noise limit in short-depth quantum circuits in the presence of noise. We assume that the noise process is constant in time and does not depend on the rescaling of the system Hamiltonian parameters. We consider various noise models in continuous time.

It is our goal to estimate the expectation value of some observable A with respect to the evolved state $\rho_\lambda(T)$. The actual computation is now encoded in the time-dependent Hamiltonian $K(t)$, and the full evolution is given by the equation

$$\frac{\partial}{\partial t}\rho = -i[K(t),\rho] + \lambda\mathcal{L}(\rho). \tag{11}$$

Here we have that

$$K(t) = \sum_\alpha J_\alpha(t) P_\alpha \tag{12}$$

is some multi-qubit Pauli Hamiltonian with time-dependent coupling constant $J_\alpha(t) \in \mathbb{R}$ and Pauli operators $P_\alpha \in \langle \mathbb{1}, X_i, Y_i, Z_i\rangle_i$. Starting from some initial state ρ_0 the system is evolved for some time T. We consider different forms of noise $\mathcal{L}(\rho)$. As the simplest form of noise, we assume a time-independent Lindblad operator [23] of the form

$$\mathcal{L}(\rho) = \sum_\beta L_\beta \rho L_\beta^\dagger - \frac{1}{2}\{L_\beta^\dagger L_\beta, \rho\}. \tag{13}$$

However, we can also imagine other forms of errors, such as

$$\mathcal{L}(\rho) = -i[V,\rho], \tag{14}$$

where V is some Hamiltonian. This setting in useful when we want to consider more general, possibly non-Markovian noise models, or a noisy evolution derived from first principle [24]. One can make the assumption that the initial state is given by $\rho_0 = \rho_S(0)\otimes\rho_B(0)$ and give the most general form of an interaction Hamiltonian between system and bath, such as

$$V = \sum_\alpha S_\alpha \otimes B_\alpha + H_B. \tag{15}$$

Here we take the point of view that a small λ indicates a separation of time scales, and $\rho_S(0) = |\psi_0\rangle\langle\psi_0|$ may be the initial state of the computation. We assume that $\rho(0) = \rho_S(0) \otimes \rho_B$, where the bath state is a steady state w.r.t the bath Hamiltonian $[H_B, \rho_B] = 0$. The observable we want to estimate $A = A_S \otimes \mathbb{1}$ is then only supported on the system degrees of freedom.

We assume that $\rho_\lambda(T)$ is the state we obtain after the noisy evolution for time T. From this state we can estimate the expectation value of the observable A by various methods. Typically we will sample the expectation value

$$E_K(\lambda) = \mathrm{tr}\left(A\rho_\lambda(T)\right) \tag{16}$$

so that an additional sampling error δ is introduced, and we obtain from out measurement the statistic $\hat{E}_K(\lambda) = E_K(\lambda) + \delta$. The error can assumed to be asymptotically Gaussian $\delta = \mathcal{O}\left(M^{-1/2}\sqrt{\mathrm{tr}\left(\rho_\lambda(T)(A - E_K(\lambda))^2\right)}\right)$ since one typically repeats the experiment $M \gg 1$ times and the i.i.d hypothesis holds.

I Series expansion in the noise parameter

We now show that the function $E_K(\lambda)$ can be expressed as a series in λ where the contribution with λ^0 corresponds to the noise-free evolution. We also provide a bound on the error term. To this end, we transform into the interaction picture of $K(t)$. We define $U_K(t) = \mathcal{T}\left\{\exp(-i\int_0^t K(t')dt')\right\}$, where $\mathcal{T}\{\cdot\}$ defines the time order expansion. We define the interaction picture through

$$\rho_I(t) = U_K(t)\rho(t)U_K^\dagger(t) \quad \text{and} \quad \mathcal{L}_{I,t}(\circ) = U_K(t)\mathcal{L}\left(U_K^\dagger(t) \circ U_K(t)\right)U_K^\dagger(t), \tag{17}$$

where now the generator $\mathcal{L}_{I,t}$ has become time-dependent. The evolution equation in the interaction picture now reads

$$\partial_t \rho_I(t) = \lambda\mathcal{L}_{I,t}\left(\rho_I(t)\right). \tag{18}$$

Recall that every first-order differential equation can be reformulated as an integral equation

$$\rho_I(T) = \rho_I(0) + \lambda\int_0^T \mathcal{L}_{I,t}\left(\rho_I(t)\right)dt. \tag{19}$$

This equation can be recursively solved to increasing order in λ so that

$$\begin{aligned}\rho_I(T) = \rho_I(0) + \lambda\int_0^T \mathcal{L}_{I,t}\left(\rho_I(0)\right)dt + \lambda^2\int_0^T\int_0^t \mathcal{L}_{I,t}\circ\mathcal{L}_{I,t'}\left(\rho_I(0)\right)dtdt' \\ + \lambda^3\int_0^T\int_0^t\int_0^{t'} \mathcal{L}_{I,t}\circ\mathcal{L}_{I,t'}\circ\mathcal{L}_{I,t''}\left(\rho_I(0)\right)dtdt'dt'' \ldots.\end{aligned} \tag{20}$$

Recall that $\rho_I(0) = \rho(0)$. Furthermore, we can conjugate the full expression on both sides with the unitary $U_K(T)$. Let us for notational convenience define $\rho_\lambda(T)$ as the resulting state after evolution with noise rate λ. We observe that $U_K(T)^\dagger\rho(0)U_K(T) = \rho_0(T)$, whereas $U_K(T)^\dagger\rho_I(T)U_K(T) = \rho_\lambda(T)$, so that we obtain the expression in the Schrödinger picture as

$$\begin{aligned}\rho_\lambda(T) = \rho_0(T) + \sum_{k=1}^n \lambda^k\int_0^T\int_0^{t_1}\ldots\int_0^{t_{k-1}} U_K^\dagger(T)\mathcal{L}_{I,t_1}\circ\mathcal{L}_{I,t_2}\circ\ldots\circ\mathcal{L}_{I,t_k}\left(\rho(0)\right)U_K(T)dt_1dt_2\ldots dt_k \\ + \lambda^{n+1}\int_0^T\int_0^{t_1}\ldots\int_0^{t_n} U_K^\dagger(T)\mathcal{L}_{I,t_1}\circ\mathcal{L}_{I,t_2}\circ\ldots\circ\mathcal{L}_{I,t_n+1}\left(\rho_I(t_{n+1})\right)U_K(T)dt_1dt_2\ldots dt_{n+1}.\end{aligned} \tag{21}$$

The expectation value $E_K(\lambda) = \mathrm{tr}\left(A\rho_\lambda(T)\right)$ for the observable A can immediately be expanded in a series with parameter λ of the form

$$E_K(\lambda) = \mathrm{tr}\left(A\rho_0(T)\right) + \sum_{k=1}^n a_k\lambda^k + R_{n+1}(\lambda, \mathcal{L}, T), \tag{22}$$

where the constants a_k and the remainder $R_{n+1}(\lambda, \mathcal{L}, T)$ are obtained by pairing the integrals with the trace $\mathrm{tr}\,(A\cdot)$ and $\mathrm{tr}\,(A\rho_0(T)) = E^*$ corresponds to the noise-free evolution to which we seek to extrapolate. We read off that

$$a_k = \int_0^T \int_0^{t_1} \dots \int_0^{t_{k-1}} \mathrm{tr}\left(U_K(T) A U_K^\dagger(T) \mathcal{L}_{I,t_1} \circ \mathcal{L}_{I,t_2} \circ \dots \circ \mathcal{L}_{I,t_k}\left(\rho(0)\right)\right) dt_1 dt_2 \dots dt_k, \tag{23}$$

as well as

$$R_{n+1}(\lambda, \mathcal{L}, T) = \lambda^{n+1} \int_0^T \int_0^{t_1} \dots \int_0^{t_n} \mathrm{tr}\left(U_K(T) A U_K^\dagger(T) \mathcal{L}_{I,t_1} \circ \dots \circ \mathcal{L}_{I,t_n+1}\left(\rho_I(t_{n+1})\right)\right) dt_1 \dots dt_{n+1}. \tag{24}$$

We can bound $|R_{n+1}(\lambda, \mathcal{L}, T)|$ by a simple application of Cauchy's mean value theorem and Hölder's inequality. We observe by first applying the midpoint Theorem that there exist $\xi_1, \dots, \xi_{n+1}$ so that

$$R_{n+1}(\lambda, \mathcal{L}, T) = \frac{\lambda^{n+1} T^{n+1}}{(n+1)!} \mathrm{tr}\left(U_K(T) A U_K^\dagger(T) \mathcal{L}_{I,\xi_1} \circ \dots \circ \mathcal{L}_{I,\xi_n+1}\left(\rho_I(\xi_{n+1})\right)\right). \tag{25}$$

We can then of course immediately bound the inner product

$$|\mathrm{tr}\left(U_K(T) A U_K^\dagger(T) \mathcal{L}_{I,t_1} \circ \dots \circ \mathcal{L}_{I,t_n+1}\left(\rho_I(t_{n+1})\right)\right)| \leq \|A\| \|\mathcal{L}_{I,t\xi_1} \circ \dots \circ \mathcal{L}_{I,\xi_n+1}\left(\rho_I(\xi_{n+1})\right)\|_1 \tag{26}$$

by a direct application of Hölder's inequality. Note that all Schatten norms are unitarily invariant, so when the map $\mathcal{L}$ is bounded, we can apply the subsequent operator norm inequalities

$$\|\mathcal{L}_{I,t\xi_1} \circ \dots \circ \mathcal{L}_{I,\xi_n+1}\left(\rho_I(\xi_{n+1})\right)\|_1 \leq \|\mathcal{L}\|_{1\to 1}^{n+1}. \tag{27}$$

It is safe to assume that a Lindblad operator $\mathcal{L}$ acting on a finite dimensional system, such as a collection of qubits, is bounded. However, we also consider the case of a first-principle noise model that can even be non-Markvoian. In this setting the operator $\mathcal{L}(\rho) = -[V, \rho]$ is expected to couple to an arbitrary large bath and V may contain unbounded operators, such as bosonic operators. In such a setting an upper bound in terms of an operator norm of $\mathcal{L}$ is a moot point. Yet, in this case we can transform the evolution into the Heisenberg picture $\mathcal{L}^*$, for the observable $A(0) = A_S \otimes \mathbb{1}$, and look at the equations for $A(t)$ instead. The almost identical analysis as performed above can be carried through, but this time we can obtain a bound on

$$|\mathrm{tr}\left(\rho_I(0) \mathcal{L}^*_{I,\xi_{n+1}} \circ \dots \circ \mathcal{L}^*_{I,t\xi_1}(A(t))\right)| \leq \|A(t)\| \|\mathcal{L}_{I,t\xi_1} \circ \dots \circ \mathcal{L}_{I,\xi_{n+1}}\left(\rho_I(0)\right)\|_1. \tag{28}$$

We obtain almost the same type of bound from Hölder's inequality since $\|A(t)\| \leq \|A\|$ for contractive evolutions, where the sole difference is now that $\|\mathcal{L}_{I,t\xi_1} \circ \dots \circ \mathcal{L}_{I,\xi_{n+1}}\left(\rho_I(0)\right)\|_1 \leq l_{n+1}$ only depends on the initial state. Since we now consider the action of the operators in V on a well-behaved initial state $\rho(0)$, we can assume that l_{n+1} is a reasonable bound. In either case, we will now write for the bound on $|R_{n+1}(\lambda, \mathcal{L}, T)|$ from now on:

$$|R_{n+1}(\lambda, \mathcal{L}, T)| \leq \|A\|\, l_{n+1}\, \frac{\lambda^{n+1} T^{n+1}}{(n+1)!}. \tag{29}$$

The coefficients a_k can be bounded in a similar fashion. Note that, if we assume that noise acts locally on each qubit, such as for instance, when the dissipator $\mathcal{L}$ corresponds to single-qubit depolarizing noise, so that $\mathcal{L}(\rho) = \sum_{i=1}^N (\frac{1}{2}\mathrm{tr}_{[i]}\,(\rho) - \rho)$. We have that $||\mathcal{L}||_{1\to 1} = \mathcal{O}(N)$ is extensive in the system size. A similar argument holds for the case when the individual qubits couple to a bath. From this we can deduce that for local noise we typically find $l_k = \mathcal{O}(N^k)$ as mentioned in the main text, and that $|a_k| \leq \mathcal{O}((N\,T)^k)$.

It is also worthwhile to point out the following observation. For different types of error terms $\mathcal{L}$ it may happen that not all powers of λ are present in the expansion. It is conceivable that some system bath interactions could lead to an expansion in only even powers of λ. If this occurs, the Richardson extrapolation method is particularly efficient, since a higher order of precision can be obtained with fewer values of λ_j.

II Experimental rescaling of the noise parameter

In order to apply Richardson extrapolation, we have to be able to evaluate $E_K(\lambda)$ for different values of λ. In an actual experiment, we can't directly control the parameter λ: however, we may control the evolution $K(t)$. To this end, we introduce a rescaling. We redefine

$$T \to T' = cT \quad \text{as well as} \quad J_\alpha(t) \to J'_\alpha(t) = c^{-1} J_\alpha(c^{-1}t) \quad \text{from which also} \quad \rho(t) \to \rho'(t) = \rho(c^{-1}t). \tag{30}$$

We claim that this rescaling maps $\rho'_\lambda(T') = \rho_{c\lambda}(T)$ if the noise operator $\mathcal{L}$ does not depend on the Hamiltonian couplings $J_\alpha(t)$ and is constant in time. This rescaled density matrix then leads to a new evaluation $E'_K(\lambda) \to E_K(c\lambda)$ of the expectation value. To see that the rescaling has the desired effect, we again make use of the integral representation of $\rho_\lambda(T)$, for which we can write now in the Schrödinger picture

$$\rho_\lambda(T) = \rho(0) - i\int_0^T [K(t), \rho(t)]dt + \lambda \int_0^T \mathcal{L}(\rho(t))dt. \tag{31}$$

We can now choose a re-parametrization of the evolution $c^{-1}J_\alpha(c^{-1}t)$ and an increased runtime cT, and write

$$\rho'_\lambda(T') = \rho(0) - i\int_0^{cT} [K'(t), \rho'(t)]dt + \lambda \int_0^{cT} \mathcal{L}(\rho'(t))dt \tag{32}$$

with $K'(t) = \sum_\alpha c^{-1}J_\alpha(c^{-1}t)P_\alpha$. If we now substitute the integration variable according to $t = ct'$, we have that $dt = cdt'$, which leads to

$$\begin{aligned}\rho'_\lambda(T') &= \rho(0) - i\int_0^T \sum_\alpha c^{-1}J_\alpha(t')[P_\alpha, \rho(t')]cdt' + \lambda\int_0^T \mathcal{L}(\rho'(t))cdt' \\ &= \rho(0) - i\int_0^T \sum_\alpha [K(t'), \rho(t')]dt' + \lambda c\int_0^T \mathcal{L}(\rho(t'))dt' \\ &= \rho_{c\lambda}(T).\end{aligned} \tag{33}$$

Hence, rescaling the evolution according to equation (30) leads to an effective rescaling of the dissipative rate λ. This can be done for any constant dissipator $\mathcal{L}$ and allows the experimenter to evaluate $E_K(\lambda)$ for different values of $c\lambda$ so that we can apply the Richardson extrapolation procedure.

Note that for different experimental circumstances, other rescaling methods of the parameter λ may actually be easier to implement. For example, in an optical experiment that is plagued by photon loss, it may be suitable to consider different methods of directly changing the photon loss rate. The only requirement is that the modification of λ_j can be performed sufficiently accurately so the extrapolation can be performed.

III Error bounds on the noise-free estimator

Let us now show that the protocol leads to the desired error bound on the estimated expectation value as claimed in the main text. Recall that we first choose a set of $n+1$ rescaling parameters $c_0 = 1 < c_1 < \ldots < c_n$, to evolve with respect to the rescaled Hamiltonian $K^j(t)$ for time $T_j = c_j T$. As discussed in the previous section, this evolution leads to a state $\rho^j_\lambda(T_j) = \rho_{c_j\lambda}(T)$, c.f. Eq. (33) as was discussed in section II. If we now measure the observable A on these states we obtain for $j = 0 \ldots n+1$ the estimates $\hat{E}_K(c_j\lambda) = E_K(c_j\lambda) + \delta_j$. Recall the set of equations for γ_j defined in [25] and given in the main text, which requires for the $\{c_j\}$ that

$$\begin{aligned}&\sum_{l=0}^n \gamma_j = 1 \\ &\sum_{j=0}^n \gamma_j\ c_j^k = 0 \quad \text{for } k = 1 \ldots n.\end{aligned} \tag{34}$$

Now we observe that estimators $\hat{E}_K(c_j\lambda)$ can be expressed as

$$\hat{E}_K(c_j\lambda) = E^* + \sum_{k=1}^n a_k c_j^k \lambda^k + R(c_j\lambda, \mathcal{L}, T) + \delta_j \tag{35}$$

due to the expansion (22) discussed in section I. Recall now the definition of our improved estimator $\hat{E}^n_K(\lambda)$ as given in the

main text, $\hat{E}_K^n(\lambda) = \sum_{j=0}^n \gamma_j \hat{E}_K(c_j\lambda)$, for which then

$$\begin{aligned}\hat{E}_K^n(\lambda) &= \sum_{j=0}^n \left(\gamma_j E^* + \sum_{k=1}^n a_k c_j^k \lambda^k + R(c_j\lambda, \mathcal{L}, T) + \delta_j \right) \\ &= E^* \left(\sum_{j=0}^n \gamma_j \right) + \sum_{k=1}^n a_k \lambda^k \left(\sum_{j=0}^n \gamma_j c_j^k \right) + \left(\sum_{j=0}^n \gamma_j R(c_j\lambda, \mathcal{L}, T) + \delta_j \right). \end{aligned} \tag{36}$$

Recall the equations for γ_j from which we can then infer after the application of the triangle inequality

$$|E^* - \hat{E}_K^n(\lambda)| \leq \sum_{j=0}^n |\gamma_j| \left(|R(c_j\lambda, \mathcal{L}, T)| + |\delta_j| \right). \tag{37}$$

After the application of the bound $|R(c_j\lambda, \mathcal{L}, T)| \leq \|A\| \, l_{n+1} \, c_j^{n+1} \lambda^{n+1} T^{n+1} ((n+1)!)^{-1}$ and the observation that $c_j \geq 1$, we can bound the difference with $\Gamma_n = \sum_{j=0}^n |\gamma_j| c_j^{n+1}$ and obtain the final bound

$$|E^* - \hat{E}_K^n(\lambda)| \leq \Gamma_n \left(\delta^* + \|A\| \frac{l_{n+1} \lambda^{n+1} T^{n+1}}{(n+1)!} \right), \tag{38}$$

with $\delta^* = \max_j |\delta_j|$.

In the Richardson extrapolation literature [16], two types of sequences c_j are considered frequently. In the Bulirsch - Stoer series the rescalings are chosen so that $c_j = h^j c_0$ constitutes an exponential series, which is typically chosen at base $h = 1/2$; but harmonic series have also been frequently applied, e.g. for parameters $q > 1, \eta \geq 0$ one can choose $c_j = (j + \eta)^{-q} c_0$. Note that in our experiments we are actually increasing the noise rate starting from the optimal value, whereas it is common in the numerical literature to improve the small parameter, so that $c_{j+1} \leq c_j$. The result here is of course the same, and just corresponds to a reordering of the labels when n is finite. For both of the aforementioned cases, a bound on Γ_n has been derived. One is mostly interested in the asymptotic behavior of Γ_n as $n \to \infty$ in order to analyze the numerical stability of the method. In current experiments we only expect to go to third or forth order, making the stability analysis less relevant.

Probabilistic error cancellation by resampling

The key idea of our scheme is to represent the ideal circuit as a quasi-probabilistic mixture of noisy ones. Central to this approach is the quasi probability representation (QPR) of the noise-free circuit $\mathcal{U}_{\boldsymbol{\beta}}$. We note that quasi-probability distributions have been previously used to construct classical algorithms for simulation of quantum circuits [17, 18]. Our work can be viewed as an application of these methods to the problem of simulating ideal quantum circuits by noisy ones.

The general approach to constructing a QPR for a quantum circuit is the following: Suppose you are given a set of noisy operations $\Omega = \{\mathcal{O}_1, \ldots, \mathcal{O}_m\}$ that can be implemented on a noisy N-qubit device. We assume that we can perform gate tomography [19] to specify the gates with an accuracy that is comparable to the desired accuracy of the ideal circuit. These noisy operations are noisy versions of ideal quantum gates and are assumed to form a full basis of TPCP operations, i.e. an element $\mathcal{O}_k \in \Omega$ is always of the form

$$\mathcal{O}_k(\rho) = \sum_i O_{i,k} \rho O_{i,k}^\dagger \quad \text{with} \quad \sum_i O_{i,k}^\dagger O_{i,k} = \mathbb{1}. \tag{39}$$

A crucial condition is that the set of noisy operators Ω constitutes a basis in the space of TPCP operations that is sufficiently large, so that any ideal, unitary gate $\mathcal{U}(\rho) = U\rho U^\dagger$ can be expressed as a linear combination of noisy gates in Ω. Hence, there have to be coefficients $\eta_\alpha \in \mathbb{R}$, and noisy operations in $\mathcal{O}_\alpha \in \Omega$ so that we can write for any ideal gate in the circuit

$$\mathcal{U}(\rho) = \sum_\alpha \eta_\alpha \mathcal{O}_\alpha(\rho), \quad \forall \rho. \tag{40}$$

This linear expansion of $\mathcal{U}$ can then be cast into the form of a quasi-probability representation.

On real quantum devices we can only apply the noisy operations Ω. We say that a circuit of length L in the basis Ω is a sequence of L operations from Ω. Such a circuit, c.f. Fig 3(b), indexed by $\boldsymbol{\alpha} = (\alpha_1, \ldots, \alpha_L)$ implements a noisy map $\mathcal{O}_{\boldsymbol{\alpha}} = \mathcal{O}_{\alpha_L} \cdots \mathcal{O}_{\alpha_2} \mathcal{O}_{\alpha_1}$.

The expectation value of an observable A on the final state produced by a noisy circuit $\boldsymbol{\alpha}$ is

$$E(\boldsymbol{\alpha}) = \mathrm{Tr}\big[A\,\mathcal{O}_{\boldsymbol{\alpha}}(|0\rangle\langle 0|^{\otimes n})\big].$$

For simplicity, we ignore errors in the initial state preparation and in the final measurement. Such errors can be accounted for by adding dummy noisy operations before each measurement and after each qubit initialization. Furthermore, we shall assume that A is diagonal in the Z-basis and $\|A\| \leq 1$.

The task of simulating an ideal quantum circuit $\mathcal{U}_{\boldsymbol{\beta}}$, c.f. Fig. 3(a), of ideal gates $\mathcal{U}_{\boldsymbol{\beta}} = \mathcal{U}_{\beta_L} \ldots \mathcal{U}_{\alpha_2}\mathcal{U}_{\alpha_1}$, can be reduced to estimating the expectation values $E(\boldsymbol{\alpha})$ for a suitable random ensemble of noisy quantum circuits $\boldsymbol{\alpha}$. That is, we can obtain estimates for the ideal expectation values

$$E^*(\boldsymbol{\beta}) = \mathrm{Tr}\big[A\,\mathcal{U}_{\boldsymbol{\beta}}(|0\rangle\langle 0|^{\otimes n})\big],$$

after the application of the circuit $\mathcal{U}_{\boldsymbol{\beta}}$ by estimating noisy circuit outputs. Moreover, the ideal and the noisy circuits act on the same number of qubits and have the same length.

We say that the noisy basis Ω simulates an ideal circuit $\boldsymbol{\beta}$ if there exists a probability distribution $P_{\boldsymbol{\beta}}(\boldsymbol{\alpha})$ on the set of noisy circuits $\boldsymbol{\alpha} \in \Omega_L$ such that

$$\mathcal{U}_{\boldsymbol{\beta}} = \gamma_{\boldsymbol{\beta}} \sum_{\boldsymbol{\alpha}\in\Omega_L} P_{\boldsymbol{\beta}}(\boldsymbol{\alpha})\sigma_{\boldsymbol{\beta}}(\boldsymbol{\alpha})\mathcal{O}_{\boldsymbol{\alpha}} \tag{41}$$

for some coefficients $\sigma_{\boldsymbol{\beta}}(\boldsymbol{\alpha}) = \pm 1$. We require that the distribution $P_{\boldsymbol{\beta}}(\boldsymbol{\alpha})$ is sufficiently simple so that one can efficiently sample $\boldsymbol{\alpha}$ from $P_{\boldsymbol{\beta}}(\boldsymbol{\alpha})$. The coefficients $\gamma_{\boldsymbol{\beta}}, \sigma_{\boldsymbol{\beta}}(\boldsymbol{\alpha})$ must be efficiently computable.

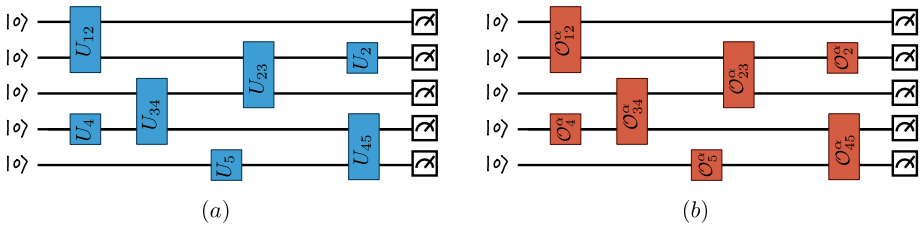


FIG. 3. (color online) The figure (a) represents the ideal circuit we want to simulate. It is comprised of single- and two-qubit gates $\{U_{12}, \ldots, U_5\}$. We assume that a complete set of noisy gates exist $\Omega = \{\mathcal{O}_{12}^{\alpha_{12}}, \ldots, \mathcal{O}_5^{\alpha_5}\}$, which serve as an operator basis in which the action of the ideal set can be expanded. It is then sufficient to sample circuits, as given in figure (b), where the gates are drawn from the probability distribution $P_{\boldsymbol{\beta}}$ in Eq. (41).

We can see that the estimates of the noisy circuit are related to the ideal circuit probability by substituting Eq. (41) into the definition of $E^*(\boldsymbol{\beta})$. This gives

$$E^*(\boldsymbol{\beta}) = \gamma_{\boldsymbol{\beta}} \sum_{\boldsymbol{\alpha}\in\Omega_L} P_{\boldsymbol{\beta}}(\boldsymbol{\alpha})\sigma_{\boldsymbol{\beta}}(\boldsymbol{\alpha})E(\boldsymbol{\alpha}). \tag{42}$$

The construction of QPRs with optimal overhead of general operation-dependent noise is an interesting open problem. A preliminary analysis shows that a noisy basis Ω that includes noisy versions of all single-qubit and two-qubit Clifford gates, T-gates, and noisy qubit initializations in the X, Y, Z basis can simulate any ideal gate $\mathcal{U}_\beta$ from the Clifford+T gate set with the overhead $\gamma_\beta \leq 1 + O(\epsilon)$, provided that each noisy operation is ϵ-close to its ideal analogue. Unfortunately, the constant coefficient in this upper bound is far too large to have any practical implications.

Furthermore, the full Clifford group on two-qubits contains 11520 gates. It may not be feasible to perform process tomography for each of those gates. We shall see that for certain noise models, such as the amplitude damping noise, QPRs can be constructed only if the noisy basis Ω includes some qubit initialization maps. In particular, noisy circuits $\boldsymbol{\alpha}$ that appear in Eq. (41) may apply qubit initializations at intermediate time steps, even though the ideal circuit $\boldsymbol{\beta}$ initializes all the qubits at the very first step. All QPRs constructed below preserve the circuit depth. That is, if the ideal circuit $\boldsymbol{\beta}$ has depth d then all noisy circuits $\boldsymbol{\alpha}$ in Eq. (41) have depth at most d.

IV Minimal overhead decomposition of noise free circuit

Let us discuss how to construct QPRs with a small overhead. For concreteness, we choose the ideal gate set Γ as the Clifford+T basis. It includes the identity gate I, the Hadamard gate H, phase-shift gates $S = \mathrm{diag}[1, i]$ and $T = \mathrm{diag}[1, e^{i\pi/4}]$, and the CNOT. For technical reasons, we shall assume that each CNOT is followed by single-qubit gates (that could be identity gates). We shall consider toy noise models usually studied in the quantum fault-tolerance theory: the depolarizing noise and the amplitude damping noise.

First let us describe product QPRs that can be constructed independently for each gate in the ideal circuit. Consider a fixed ideal gate $\mathcal{U}_\beta \in \Gamma$. Let $\mathcal{O}_1, \ldots, \mathcal{O}_p \in \Omega$ be the list of all noisy operations whose support is contained in the support of $\mathcal{U}_\beta$. Consider the following linear program with $2p$ real variables $\mu_1, \ldots, \mu_p, \eta_1, \ldots, \eta_p$.

$$\textbf{minimize} \quad \sum_{\alpha=1}^{p} \mu_\alpha \tag{43}$$

$$\textbf{subject to} \quad \begin{cases} \eta_\alpha \le \mu_\alpha \\ -\eta_\alpha \le \mu_\alpha \\ \mathcal{U}_\beta = \sum_{\alpha=1}^{p} \eta_\alpha \mathcal{O}_\alpha. \end{cases} \tag{44}$$

Suppose $\{\mu_\alpha, \eta_\alpha\}$ is the optimal solution of the program. Note that $\mu_\alpha = |\eta_\alpha|$ for all α since otherwise the objective function can be decreased. Define $\gamma_\beta = \sum_{\alpha=1}^{p} \mu_p$, $P_\beta(\alpha) = \mu_\alpha/\gamma_\beta$, and $\sigma_\beta(\alpha) = \mathrm{sgn}(\eta_\alpha)$. Then

$$\mathcal{U}_\beta = \gamma_\beta \sum_{\alpha=1}^{p} P_\beta(\alpha)\sigma_\beta(\alpha)\mathcal{O}_\alpha, \tag{45}$$

which is a gate-wise version of the QPR Eq. (41). We shall say that a noisy basis Ω simulates a gate $\mathcal{U}_\beta$ with the overhead γ_β if the linear program Eqs. (43,44) has a feasible solution with value γ_β. A product QPR of the ideal circuit $\boldsymbol{\beta}$ is defined as a product of all gate-wise QPRs Eq. (45). It gives $\gamma_{\boldsymbol{\beta}} = \gamma_{\beta_1} \cdots \gamma_{\beta_L}$, $P_{\boldsymbol{\beta}}(\boldsymbol{\alpha}) = P_{\beta_1}(\alpha_1) \cdots P_{\beta_L}(\alpha_L)$ and $\sigma_{\boldsymbol{\beta}}(\boldsymbol{\alpha}) = \sigma_{\beta_1}(\alpha_1) \cdots \sigma_{\beta_L}(\alpha_L)$. The assumption that all noisy operations $\mathcal{O}_\alpha$ in Eq. (44) act non-trivially only within the support of $\mathcal{U}_\beta$ allows one to restrict Eq. (44) to operations acting on at most two qubits. Such operations can be represented by real matrices of size 16×16 by computing matrix elements of $\mathcal{O}_\alpha$ and $\mathcal{U}_\beta$ in the Pauli basis. Thus the program Eqs. (43,44) can be solved in time $O(1)$. Since the ideal gate set has size $O(n^2)$, product QPRs can be computed in time $O(n^2)$. Furthermore, if two ideal gates have disjoint supports, then the gate-wise QPRs defined in Eq. (45) have disjoint supports. Thus product QPRs preserve the circuit depth.

V Depolarizing noise cancellation and numerical results

Let us illustrate the construction of product QPRs using the depolarizing noise as an example. Let $\mathcal{D}_k$ be the ϵ-depolarizing channel on k qubits that returns the maximally mixed state with probability ϵ and does nothing with probability $1 - \epsilon$. Define a noisy version of a k-qubit unitary gate $\mathcal{U}$ as $\mathcal{D}_k\mathcal{U}$. Define a noisy basis Ω by multiplying ideal gates on the left by arbitrary Pauli operators and adding the depolarizing noise. Thus Ω is a set of operations $\mathcal{O}_\alpha = \mathcal{D}_k\mathcal{P}\mathcal{U}$, where $\mathcal{U} \in \Gamma$ is a k-qubit ideal gate and $\mathcal{P} \in \{\mathcal{I}, \mathcal{X}, \mathcal{Y}, \mathcal{Z}\}^{\otimes k}$ is a Pauli TPCP map. A Pauli map $\mathcal{P}$ corresponding to a Pauli operator $P \in \{I, X, Y, Z\}$ is defined by $\mathcal{P}(\rho) = P\rho P$. Here $k = 1, 2$. We claim that Ω simulates ideal single-qubit gates $\mathcal{U}_\beta \in \Gamma$ with the overhead $\gamma_\beta = (1 + \epsilon/2)/(1 - \epsilon)$ and simulates CNOTs with the overhead $\gamma_\beta = (1 + 7\epsilon/8)/(1 - \epsilon)$.

Indeed, suppose $\mathcal{U}_\beta \in \Gamma$ is a single-qubit gate. Let us look for a solution of Eq. (44) in the form $\mathcal{O}_\alpha = \mathcal{D}_1\mathcal{P}\mathcal{U}_\beta$, where $\mathcal{P} \in \{\mathcal{I}, \mathcal{X}, \mathcal{Y}, \mathcal{Z}\}$. Then Eq. (44) is equivalent to

$$\mathcal{D}_1^{-1} = \eta_1\mathcal{I} + \eta_2\mathcal{X} + \eta_3\mathcal{Y} + \eta_4\mathcal{Z}.$$

One can easily check that the optimal solution minimizing $\sum_\alpha |\eta_\alpha|$ is $\eta_1 = 1 + 3\epsilon/4(1 - \epsilon)$ and $\eta_\alpha = -\epsilon/4(1 - \epsilon)$ for $\alpha = 2, 3, 4$. Therefore $\gamma_\beta = \sum_\alpha |\eta_\alpha| = (1 + \epsilon/2)/(1 - \epsilon)$. The CNOT is simulated in a similar fashion by representing $\mathcal{D}_2^{-1}$ as a linear combination of two-qubit Pauli maps. The random ensemble of noisy circuits $\mathcal{O}_{\boldsymbol{\alpha}}$ that simulates an ideal circuit $\mathcal{U}_{\boldsymbol{\beta}}$ is constructed in three steps:

1. Start from the ideal circuit, $\mathcal{O}_{\boldsymbol{\alpha}} = \mathcal{U}_{\boldsymbol{\beta}}$.

2. Modify $\mathcal{O}_{\boldsymbol{\alpha}}$ by adding a Pauli X, Y, Z after each single-qubit gate with probability $p_1 = \epsilon/(4+2\epsilon)$. The gate is unchanged with probability $1 - 3p_1$.

3. Modify $\mathcal{O}_{\boldsymbol{\alpha}}$ by adding a Pauli $IX, IY, \ldots, ZZ$ after each CNOT with probability $p_2 = \epsilon/(16 + 14\epsilon)$. The CNOT is unchanged with probability $1 - 15p_2$.

The resulting circuit is then implemented on a noisy device (which adds the depolarizing noise after each gate) and the final readout string x is recorded. By generating M samples of x one can estimate $E^*(\boldsymbol{\beta})$ using Eq. (10) of the main text. The sign function $\sigma_{\boldsymbol{\beta}}(\boldsymbol{\alpha})$ is equal to $(-1)^r$, where r is the number of Pauli operators added to the ideal circuit $\mathcal{U}_{\boldsymbol{\beta}}$ to obtain $\mathcal{O}_{\boldsymbol{\alpha}}$.

Numerical simulations

The error cancellation method was tested numerically for small Clifford+T circuits subject to the depolarizing noise. We choose the ideal circuit $\mathcal{U}_{\boldsymbol{\beta}}$ as a composition of d alternating layers of gates, with each layer being either a tensor product of n single-qubit gates I, H, S, T (for odd layers) or a tensor product of $n/2$ CNOTs (for even layers). The resulting circuit $\mathcal{U}_{\boldsymbol{\beta}}$ has depth d. Simulations were performed for 500 random circuits $\mathcal{U}_{\boldsymbol{\beta}}$ as above with the initial state $|+\rangle^{\otimes n}$. Each single-qubit gate was picked randomly from the set $\{I, H, S, T\}$. Control and target qubits for each CNOT were picked at random.

For each ideal circuit $\mathcal{U}_{\boldsymbol{\beta}}$ we choose the observable A as a projector onto the subset of 2^{n-1} basis states $x \in \{0,1\}^n$ whose probability in the final state of $\mathcal{U}_{\boldsymbol{\beta}}$ is above the median value. In other words,

$$A = \sum_{x \in S} |x\rangle\langle x|, \qquad S = \arg \min_{\substack{S \subseteq \{0,1\}^n \\ |S| = 2^{n-1}}} \sum_{x \in S} \langle x|\mathcal{U}_{\boldsymbol{\beta}}(|+\rangle\langle +|^{\otimes n})|x\rangle.$$

By construction, $E^*(\boldsymbol{\beta}) \geq 1/2$ for any circuit $\boldsymbol{\beta}$. Furthermore, we observed that $E^*(\boldsymbol{\beta})$ is well separated from $1/2$ for most of the circuits see Fig. 4. Recall that we define a noisy version of a k-qubit unitary gate $\mathcal{U}$ as $\mathcal{D}_k\mathcal{U}$, where

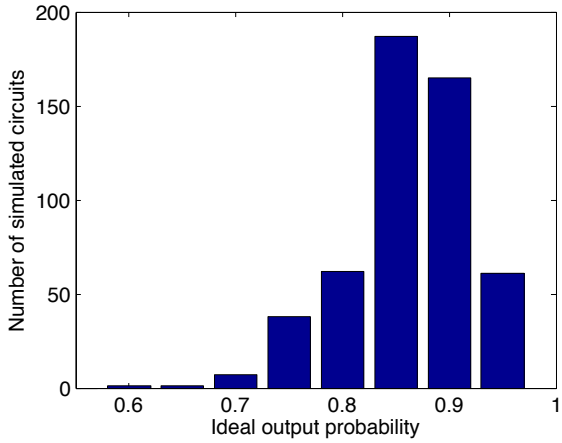


FIG. 4. Distribution of the ideal circuits according to their output probability $E^*(\boldsymbol{\beta})$.

$$\mathcal{D}_k(\rho) = (1-\epsilon)\rho + \frac{\epsilon I}{2^k}\mathrm{Tr}(\rho)$$

is the depolarizing channel on k qubits. Noise was added after all gates including the identity gates. In this case the total simulation overhead $\gamma_{\boldsymbol{\beta}}$ depends only on the number of qubits and the circuit depth, namely

$$\gamma_{\boldsymbol{\beta}} = \left[\frac{1+\epsilon/2}{1-\epsilon}\right]^{nd/2} \cdot \left[\frac{1+7\epsilon/8}{1-\epsilon}\right]^{nd/4}.$$

Consider a fixed ideal circuit $\mathcal{U}_{\boldsymbol{\beta}}$ and let $P_{\boldsymbol{\beta}}(\boldsymbol{\alpha})$, $\mathcal{O}_{\boldsymbol{\alpha}}$ be the random ensemble of noisy circuits obtained from $\mathcal{U}_{\boldsymbol{\beta}}$ by inserting random Pauli operators and adding noise as described in the main text. Instead of using the estimate Eq. (10) of the main text

for the ideal output probability $E^*(\boldsymbol{\beta})$ we opted for a slightly optimized estimate. It is defined by dividing the total budget of M runs into K groups such that the j-th group contains M_j runs

$$M = \sum_{j=1}^{K} M_j.$$

Define a random variable

$$\hat{E}(\boldsymbol{\beta}) \equiv \gamma_{\boldsymbol{\beta}} K^{-1} \sum_{j=1}^{K} \sigma_{\boldsymbol{\beta}}(\boldsymbol{\alpha}^j) \frac{1}{M_j} \sum_{a=1}^{M_j} \langle x_j^a | A | x_j^a \rangle, \tag{46}$$

where $\boldsymbol{\alpha}^1, \ldots, \boldsymbol{\alpha}^K$ are independent samples drawn from the distribution $P_{\boldsymbol{\beta}}(\boldsymbol{\alpha})$ and $x_j^a \in \{0,1\}^n$ are readout strings obtained by measuring each qubit of the final state $\mathcal{O}_{\boldsymbol{\alpha}^j}(\rho_{in})$ in the Z-basis. We prepare a fresh copy of the final state to generate each string x_j^a. Thus computing $\hat{E}(\boldsymbol{\beta})$ requires M runs of the noisy circuits with each run producing a single readout string. One can easily check that $\hat{E}(\boldsymbol{\beta})$ is an unbiased estimator of $E^*(\boldsymbol{\beta})$ for any choice of $\{M_j\}$. Our goal is to choose $\{M_j\}$ that minimize the variance of $\hat{E}(\boldsymbol{\beta})$ for a fixed M. One can easily check that the optimal choice is

$$M_j \approx \frac{M\sigma_j}{\sum_{i=1}^{K} \sigma_i}$$

where $\sigma_j^2 = E(\boldsymbol{\alpha}^j) - E(\boldsymbol{\alpha}^j)^2$. In order to choose optimal values of M_j, one has to run each circuit $\boldsymbol{\alpha}^j$ at least a few times, which gives a rough estimate of $E(\boldsymbol{\alpha}^j)$ and thus σ_j. Numerical simulations were performed for the following parameters:

number of qubits	$n = 6$
circuit depth	$d = 20$
error rate	$\epsilon = 0.01$
total number of runs	$M = 4,000$
simulation overhead	$\gamma_{\boldsymbol{\beta}} \approx 4.3$

Our results are presented on the left panel of Figure 5. For each of ≈ 500 ideal circuits $\boldsymbol{\beta}$ generated at random we computed a simulation precision $\delta(\boldsymbol{\beta}) \equiv |\hat{E}(\boldsymbol{\beta}) - E^*(\boldsymbol{\beta})|$ where $\hat{E}(\boldsymbol{\beta})$ is the estimate defined in Eq. (46). The plot on Figure 5, left, shows distribution of the ideal circuits $\boldsymbol{\beta}$ according to their simulation precision $\delta(\boldsymbol{\beta})$. The median value of $\delta(\boldsymbol{\beta})$ is approximately 0.05. This is consistent with the estimate

$$\delta(\boldsymbol{\beta}) \approx \frac{\gamma_{\boldsymbol{\beta}}}{\sqrt{M}} = \frac{4.3}{\sqrt{4000}} \approx 0.07.$$

We also computed a simulation precision $\delta_0(\boldsymbol{\beta})$ that one would obtain by running the circuit $\boldsymbol{\beta}$ directly on a noisy device without error cancellation, see the right panel of Figure 5. It is defined as $\delta_0(\boldsymbol{\beta}) \equiv |E(\boldsymbol{\beta}) - E^*(\boldsymbol{\beta})|$. For each circuit the output probability $E(\boldsymbol{\beta})$ was estimated using $M = 4,000$ circuit runs. Thus the simulations presented on the left and the right panels of Figure 5 have access to exactly the same resources. The median value of $\delta_0(\boldsymbol{\beta})$ is approximately 0.15. We conclude that error cancellation significantly improves the simulation precision.

VI Quasi-probability representation for amplitude-damping noise

A more interesting example is the noise described by the amplitude-damping channel $\mathcal{A}(\rho) = A_0 \rho A_0^\dagger + A_1 \rho A_1^\dagger$, where

$$A_0 = \begin{bmatrix} 1 & 0 \\ 0 & (1-\epsilon)^{1/2} \end{bmatrix} \quad \text{and} \quad A_1 = \epsilon^{1/2} \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}.$$

A noisy version of a k-qubit unitary gate $\mathcal{U}$ is defined as $\mathcal{A}^{\otimes k}\mathcal{U}$. In contrast to the previous example, noisy unitary gates $\mathcal{A}^{\otimes k}\mathcal{U}$ alone cannot simulate any ideal unitary gate. Indeed, assume the contrary. Suppose $\mathcal{U}_\beta$ is a single-qubit gate that has a QPR Eq. (45) with $\mathcal{O}_\alpha = \mathcal{A}\mathcal{V}_\alpha$ for some unitary maps $\mathcal{V}_\alpha$. Rewrite Eq. (45) as

$$\mathcal{A}^{-1} = \gamma_\beta \sum_{\alpha=1}^{p} P_\beta(\alpha)\sigma_\beta(\alpha)\mathcal{V}_\alpha \mathcal{U}_\beta^{-1}.$$

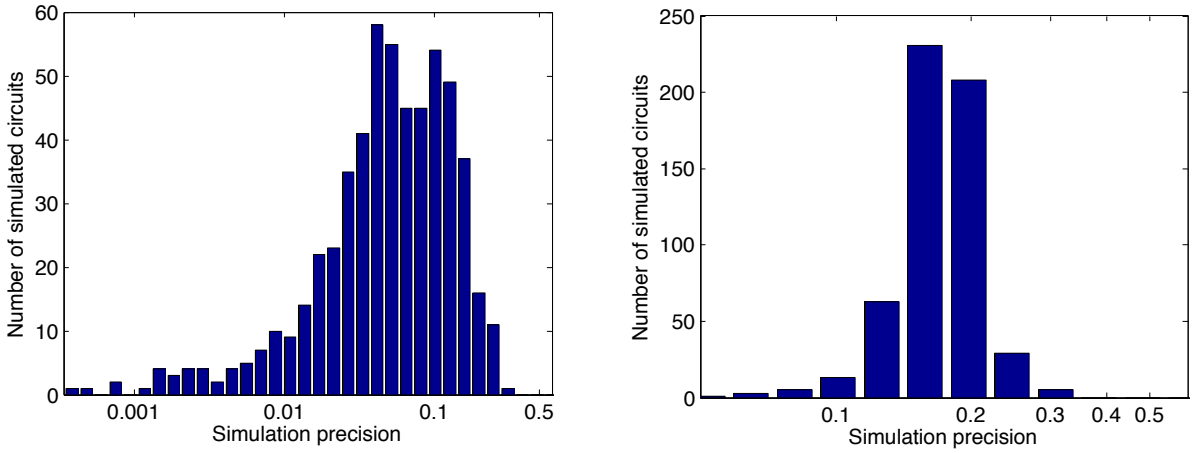


FIG. 5. Simulation precision for ≈ 500 randomly generated ideal Clifford+T circuits on $n = 6$ qubits with depth $d = 20$. The left and the right panels show results for simulations with and without error cancellation. In both cases each ideal circuit was simulated by $M = 4000$ runs of the noisy circuit.

Since the maps $\mathcal{V}_\alpha \mathcal{U}_\beta^{-1}$ are unital, we infer that $\mathcal{A}^{-1}$ and $\mathcal{A}$ are unital which is false. Thus Eq. (45) has no solutions.

To overcome this problem we shall extend the noisy basis by adding state preparations. Also we shall employ non-product QPRs. Given a single-qubit state $|\psi\rangle$, define a state preparation map

$$\mathcal{P}_{|\psi\rangle}(\rho) = \mathrm{Tr}(\rho) \cdot |\psi\rangle\langle\psi|. \tag{47}$$

Let $\mathcal{S}(\rho) = S\rho S^{-1}$ be the S-gate considered as a TPCP map. Define a noisy basis Ω that includes noisy state preparations $\mathcal{A}\mathcal{P}_{|\psi\rangle}$ with $|\psi\rangle = |+\rangle, |-\rangle, |0\rangle, |1\rangle$, noisy single-qubit gates $\mathcal{A}\mathcal{U}_\beta$, $\mathcal{A}\mathcal{S}^{\pm 1}\mathcal{U}_\beta$ for each ideal single-qubit gate $\mathcal{U}_\beta \in \Gamma$, and noisy two-qubit gates

$$\mathcal{A}_c \mathcal{A}_t \mathcal{S}_c^y \mathcal{S}_t^z \mathcal{U}_{\mathrm{cnot}}, \quad y, z \in \{0, \pm 1\}$$

where c, t are the control and the target qubits of a CNOT gate $\mathcal{U}_{\mathrm{cnot}} \in \Gamma$. Here the subscripts indicate qubits acted upon by each map. We claim that this noisy basis Ω simulates any ideal Clifford+T circuit $\boldsymbol{\mathcal{B}}$ with the overhead

$$\gamma_{\boldsymbol{\mathcal{B}}} \leq \gamma^{L_1 + 2L_2}, \quad \gamma \equiv \frac{1+\epsilon}{1-\epsilon}, \tag{48}$$

where L_k is the number of k-qubit gates in $\boldsymbol{\mathcal{B}}$. The corresponding QPR Eq. (41) preserves the circuit depth, although it does not have a simple product form as above.

Indeed, consider a single-qubit gate $\mathcal{U}_\beta \in \Gamma$. Let us look for a solution of Eq. (44) with $p = 4$ and

$$\mathcal{O}_1 = \mathcal{A}\mathcal{U}_\beta, \ \mathcal{O}_2 = \mathcal{A}\mathcal{S}\mathcal{U}_\beta, \ \mathcal{O}_3 = \mathcal{A}\mathcal{S}^{-1}\mathcal{U}_\beta, \ \mathcal{O}_4 = \mathcal{P}_{|0\rangle}.$$

Note that $\mathcal{P}_{|0\rangle} \in \Omega$ since $\mathcal{A}\mathcal{P}_{|0\rangle} = \mathcal{P}_{|0\rangle}$. Furthermore, since $\mathcal{P}_{|0\rangle} = \mathcal{A}\mathcal{P}_{|0\rangle}\mathcal{U}_\beta$, one can rewrite Eq. (44) as

$$\mathcal{A}^{-1} = \eta_1 \mathcal{I} + \eta_2 \mathcal{S} + \eta_3 \mathcal{S}^{-1} + \eta_4 \mathcal{P}_{|0\rangle}. \tag{49}$$

One can easily check that the optimal solution minimizing $\sum_\alpha |\eta_\alpha|$ is

$$\eta_1 = \frac{1}{\sqrt{1-\epsilon}}, \ \eta_2 = \eta_3 = \frac{1-\sqrt{1-\epsilon}}{2(1-\epsilon)}, \ \eta_4 = -\frac{\epsilon}{1-\epsilon}.$$

Therefore, Ω simulates $\mathcal{U}_\beta$ with the overhead $\sum_\alpha |\eta_\alpha| = \gamma$, where γ is defined in Eq. (48).

Next consider the CNOT gate $\mathcal{U}_{\mathrm{cnot}} \in \Gamma$. Consider a decomposition of $\mathcal{A}_c^{-1}\mathcal{A}_t^{-1}$ obtained by applying Eq. (49) twice. Multiplying this decomposition on the right by $\mathcal{U}_{\mathrm{cnot}}$ and on the left by $\mathcal{A}_c\mathcal{A}_t$ one obtains

$$\mathcal{U}_{\mathrm{cnot}} = \sum_\alpha \eta_\alpha \mathcal{O}'_\alpha \mathcal{O}_\alpha, \quad \sum_\alpha |\eta_\alpha| = \gamma^2 \tag{50}$$

where $\mathcal{O}_\alpha = \mathcal{A}_c\mathcal{A}_t\mathcal{S}_c^y\mathcal{S}_t^z\mathcal{U}_{\rm cnot} \in \Omega$ is a valid noisy operation and $\mathcal{O}'_\alpha$ is either identity or a state preparation map $\mathcal{P}_{|0\rangle}$ applied to the control and/or target qubits. Here we noted that $\mathcal{A}\mathcal{P}_{|0\rangle} = \mathcal{P}_{|0\rangle}\mathcal{A}$. Although $\mathcal{O}'_\alpha\mathcal{O}_\alpha$ might not be a valid noisy operation from Ω, we may merge $\mathcal{O}'_\alpha$ with the next gate applied after the CNOT. Indeed, by assumption, each CNOT in the ideal circuit is followed by some single-qubit gates $\mathcal{U}_c$ and $\mathcal{U}_t$ applied to the control and the target qubits. The gates I, S, T can be absorbed into $\mathcal{P}_{|0\rangle}$ since they act trivially on the state $|0\rangle$. The only non-trivial case is when $\mathcal{P}_{|0\rangle}$ is merged with the Hadamard gate. In this case the latter is replaced by the state preparation $\mathcal{P}_{|+\rangle}$.

Since $\mathcal{P}_{|+\rangle}$ can now appear in the ideal circuit, we must be able to use noisy operations from Ω to simulate $\mathcal{P}_{|+\rangle}$. Let us look for a solution of Eq. (44) with $\mathcal{U}_\beta \equiv \mathcal{P}_{|+\rangle}$ in the form

$$\mathcal{P}_{|+\rangle} = \eta_1\mathcal{A}\mathcal{P}_{|+\rangle} + \eta_2\mathcal{A}\mathcal{P}_{|-\rangle} + \eta_3\mathcal{A}\mathcal{P}_{|1\rangle}. \tag{51}$$

Note that the righthand side of Eq. (51) contains only noisy operations from Ω. One can rewrite Eq. (51) as

$$|+\rangle\langle+| = \eta_1\mathcal{A}(|+\rangle\langle+|) + \eta_2\mathcal{A}(|-\rangle\langle-|) + \eta_3\mathcal{A}(|1\rangle\langle1|).$$

The optimal solution minimizing $\sum_\alpha |\eta_\alpha|$ is

$$\eta_{1,2} = \pm\frac{1}{2}\left(\frac{1}{\sqrt{1-\epsilon}} \pm \frac{1-2\epsilon}{1-\epsilon}\right), \quad \eta_3 = \frac{\epsilon}{1-\epsilon}.$$

Therefore Ω simulates the ideal state preparation $\mathcal{P}_{|+\rangle}$ with the overhead $\gamma' = \sum_\alpha |\eta_\alpha| \le \gamma$, where γ is defined in Eq. (48).

A QPR of the ideal circuit $\boldsymbol{\beta}$ with the overhead Eq. (48) is constructed in two steps. First, one applies the decomposition Eq. (50) to each CNOT of $\boldsymbol{\beta}$ and merges state preparation maps $\mathcal{P}_{|0\rangle}$ that appear in $\mathcal{O}'_\alpha$ (if any) with the single-qubit gates of $\boldsymbol{\beta}$ following the CNOT. Now all CNOT gates are replaced by noisy gates from Ω. The rest of the circuit consists of single-qubit gates $\mathcal{U}_\beta \in \Gamma$ and state preparations $\mathcal{P}_{|+\rangle}$. At the second step, each of these ideal operations is replaced by its QPR constructed above. Note that each CNOT contributes γ^2 to the total overhead $\gamma_{\boldsymbol{\beta}}$, see Eq. (50). Each single-qubit gate $\mathcal{U}_\beta \in \Gamma$ or a state preparation $\mathcal{P}_{|+\rangle}$ contributes at most γ to the total overhead. This proves Eq. (48).

arXiv:2103.02202v3 [quant-ph] 18 Jun 2021

Stim: a fast stabilizer circuit simulator

Craig Gidney

Google Inc., Santa Barbara, California 93117, USA

June 21, 2021

This paper presents "Stim", a fast simulator for quantum stabilizer circuits. The paper explains how Stim works and compares it to existing tools. With no foreknowledge, Stim can analyze a distance 100 surface code circuit (20 thousand qubits, 8 million gates, 1 million measurements) in 15 seconds and then begin sampling full circuit shots at a rate of 1 kHz. Stim uses a stabilizer tableau representation, similar to Aaronson and Gottesman's CHP simulator, but with three main improvements. First, Stim improves the asymptotic complexity of deterministic measurement from quadratic to linear by tracking the *inverse* of the circuit's stabilizer tableau. Second, Stim improves the constant factors of the algorithm by using a cache-friendly data layout and 256 bit wide SIMD instructions. Third, Stim only uses expensive stabilizer tableau simulation to create an initial reference sample. Further samples are collected in bulk by using that sample as a reference for batches of Pauli frames propagating through the circuit.

Readers who want to try Stim can find its source code in this paper's ancillary files or on github at `https://github.com/quantumlib/stim`. *Stim is also available as a Python 3 pypi package installed via "*`pip install stim`*". There is also a pypi package "stimcirq" that exposes stim as a cirq sampler.*

1 Introduction

In the field of human computer interaction, an important metric is the delay between human action and computer reaction. Quoting Jakob Nielsen [25, 16]:

- 0.1 seconds is about the limit for having the user feel that the system is reacting instantaneously [...]
- 1.0 second is about the limit for the user's flow of thought to stay uninterrupted [...]
- 10 seconds is about the limit for keeping the user's attention [...]

I bring this up because the underlying motivation driving this paper is that I want to tinker with huge stabilizer circuits, and I want to stand in the 0.1s bucket while I do it. For example, consider the single level 15-to-1 surface code T state factory shown in figure 4 of [11]. It corresponds to a circuit with millions of gates spanning 15 thousand qubits. What would it take to simulate *that* in 0.1 seconds, with no built-in preconceptions about the surface code, while a user makes iterative changes that break and restore the functionality of the circuit? There are several obstacles that make this difficult.

First, if a circuit has a million gates, and your computer's clock speed is 2 gigahertz, then the simulator has a budget of 200 clock cycles per gate. That's a tight budget.

Second, describing an arbitrary n-qubit stabilizer state takes at least $\frac{1}{2}n^2$ bits [14, 22, 18]. This means that, in the worst case, a general stabilizer simulator that works in an online fashion will

Craig Gidney: craiggidney@google.com

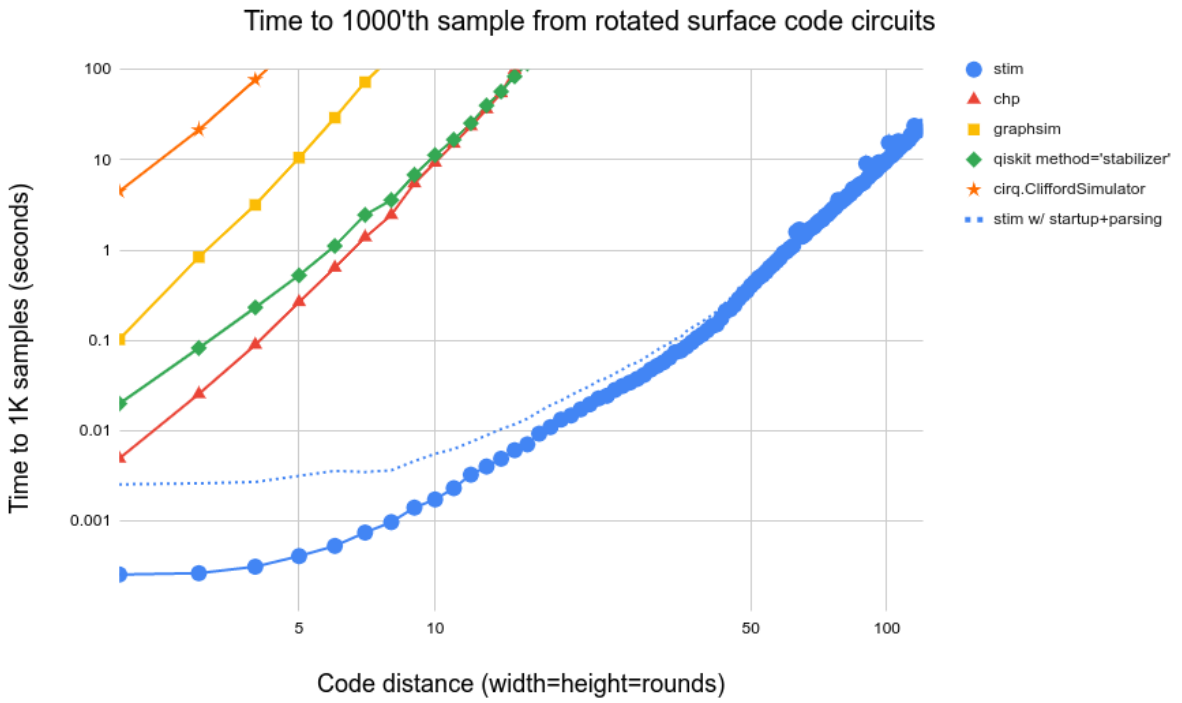


Figure 1: Delay until one thousandth sample on $d \times d \times d$ surface code memory circuits [15]. The intention of this benchmark is to highlight simulators that can perform some sort of compilation or analysis that lowers the cost of taking multiple samples. Stim does particularly well on this benchmark, with timings almost identical to its time-to-first-sample. This is because it uses its first sample as a reference for batches of hundreds or thousands of Pauli frame simulations that are updated in parallel using SIMD instructions. The per-sample performance of Qiskit's simulator also improves when collecting more than one sample.

need megabytes of information to track the state of a 15000 qubit system. Storing this amount of information is not a problem. The problem is repeatedly touching that information without blowing the budget on cache misses.

Third, typically, simulating a Clifford gate has complexity $\Omega(n)$ where n is the number of qubits in the circuit. Given the aforementioned cycle budget, the constant factor hiding behind that Ω has to be roughly 0.01 cycles.

Fourth, the surface code is chock full of measurements and, basically, historically, the cost of measurements in stabilizer simulators started at $\Theta(n^3)$ [13], improved to $\Theta(n^2)$ [1], and then stayed there. Aaronson and Gottesman's [1] CHP simulator? Measurement takes $\Theta(n^2)$ time in the worst case. Anders and Briegel's [3] graph state simulator? Measurement takes $\Theta(n^2)$ time in the worst case. Bravyi et al's [6] CH form simulator? Measurement takes $\Theta(n^2)$ time in the worst case. We can't afford $\Theta(n^2)$ time.

Fifth, the circuit I've been talking about isn't even technically a stabilizer circuit. It contains magic state injections. This obstacle has actually been the subject of a lot of recent work [6, 7, 20, 19]. I'll be completely ignoring it in this paper.

Sixth, when simulating an error correcting circuit, it's common to sample the circuit millions or even trillions of times. For example, going to large code distances can allow a lone half distance error mechanism to shine through the background haze of full distance errors. But logical errors become rare at large code distances, and so many samples have to be taken to see them. Based on discussions I've had with co-workers who have done these sorts of large scale simulations [12], the main bottleneck is not decoding the errors but rather (surprisingly) generating the samples. So it's not just the latency until the first sample that matters; the sustained bulk sampling rate is also important.

With all those obstacles listed I should note that, of course, by using hardcoded knowledge about magic state distillation and the surface code, it's possible to make a specialized simulator that could solve my example case much faster than a general stabilizer circuit simulator. But my underlying motivation is one of exploration, and I would strongly prefer to create a tool whose performance and correctness doesn't require understanding the behavior of places I haven't been yet. So, in this paper, I present my failed attempt to reach my ridiculous goal. The result is a software tool I've named "Stim". Although Stim can't simulate a 15 thousand qubit surface code circuit with millions of gates in under a tenth of a second, it can at least get that job done in roughly ten seconds. Then, with that initial sample acquired, it can begin producing batches of hundreds of samples in a tenth of the time.

Stim makes three key improvements over previous stabilizer simulators. These improvements address obstacle 4 (the measurement obstacle), obstacle 3 (the constant factor obstacle), and obstacle 6 (the mega sampling obstacle).

The measurement obstacle is addressed by making an algorithmic improvement that decreases the time complexity of *deterministic* measurements from quadratic to linear. This is beneficial in contexts, such as the surface code, where a circuit has many measurements but those measurements are almost entirely deterministic.

The constant factor obstacle is addressed by using SIMD (Same Instruction Multiple Data) instructions in tight loops iterating over contiguous memory. Using SIMD is not a novel concept (e.g. Aaronson et al's CHP simulator [1] packs the bits from Pauli terms into bytes, which allows 8 to be operated on by each instruction) but it's technically challenging to implement well and Stim does it at larger register sizes than previous work. Specifically, Stim uses 256 bit wide Advanced Vector Extensions (AVX) [30] and also I spent a lot of effort minimizing the number of operations in key loops. For example, a core operation for most gates is multiplying together strings of Pauli terms, which requires accumulating scalar factors from individual Pauli multiplications. I found a way to do this using only 11 bitwise operations per group of Paulis (see Figure 2). Modern Intel CPUs can apply 3 bitwise AVX operations per clock cycle [21], and 256 is much larger than 11, suggesting that a budget of 0.01 cycles per qubit during a Clifford gate isn't quite as ridiculous as it seems.

The mega sampling obstacle is addressed by including a Pauli frame simulator that uses SIMD instructions to operate on the state of hundreds of simulations in parallel. This amortizes the cost of cache misses and branch mispredictions and other performance killers. This is a simple enough idea,

but it gives Stim an enormous advantage over previous work when bulk sampling (see Figure 1).

This paper is divided into sections. In Section 2, I explain key background concepts that are needed in order to understand the simulation strategies being described. Then in Section 3 and Section 4 I describe the Pauli frame simulator and stabilizer tableau simulators implemented by Stim. Section 5 describes several of the software engineering details that went into Stim, such as why I decided to use a dense representation instead of a sparse representation and how I went about testing that operations were implemented correctly. Stim's performance is compared to other stabilizer simulators in Section 6. Finally, Section 7 wraps everything up. There are also a few short usage examples in Appendix A.

2 Key Concepts

2.1 Pauli Products

There are four Pauli gates: the identity $I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$, the bit flip $X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$, the phase flip $Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$, and the other one $Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$. Sometimes the identity gate is not considered to be a Pauli gate, but in this paper it is.

A Pauli product is a set of Pauli gates applied to qubits. For example, $X_1 \cdot Y_2 \cdot Z_3$ is a Pauli product that applies X to qubit 1, Y to qubit 2, Z to qubit 3, and I to everything else. The set of all Pauli products form a group under matrix multiplication. This group is called the Pauli group.

Paulis can be encoded in a variety of ways. One encoding, which I'll be calling the "xz encoding", represents a Pauli by decomposing it into an x bit representing the presence of an X gate, a z bit representing the presence of a Z gate, and an ignored global phase:

$$\texttt{encode_xz}(P) = (x, z) = (P \notin \{I, Z\}, P \notin \{I, X\}) = \texttt{encode_xz}(X^x Z^z)$$

Meaning:

$$\texttt{encode_xz}(I) = (0, 0)$$
$$\texttt{encode_xz}(X) = (1, 0)$$
$$\texttt{encode_xz}(Y) = (1, 1)$$
$$\texttt{encode_xz}(Z) = (0, 1)$$

In the xz encoding (and most others), two Paulis can be multiplied up to global phase by xoring their encoded representations:

$$\texttt{encode_xz}(P_1 P_2) = \texttt{encode_xz}(P_1) \oplus \texttt{encode_xz}(P_2) = (x_1 \oplus x_2, z_1 \oplus z_2)$$

In Stim, Pauli products are represented by the `stim::PauliString` class.

2.2 Stabilizer Generators

When a quantum state is in the $+1$ eigenstate of an operation, that operation is said to be a *stabilizer* of the state. For example: $+Z$ is a stabilizer of the computational basis state $|0\rangle$, $+Z$ is *not* a stabilizer of the computational basis state $|1\rangle$, $+X$ is a stabilizer of the superposed state $|0\rangle + |1\rangle$, and $-YY$ is a stabilizer of the entangled state $|00\rangle + |11\rangle$. Stabilizer operations that are Pauli products are of particular interest, as they form the basis of the *stabilizer formalism* [13] (easily one of the most important and foundational ideas in quantum error correction). In this paper, whenever I say "stabilizer" I always mean "Pauli product stabilizer".

The stabilizers of a state form a group under multiplication. Let A and B be stabilizers of a state $|\psi\rangle$, meaning $A|\psi\rangle = |\psi\rangle = B|\psi\rangle$. AB is also a stabilizer of $|\psi\rangle$, because $AB|\psi\rangle = A|\psi\rangle = |\psi\rangle$.

The stabilizers of a state commute. Note that $AB|\psi\rangle = BA|\psi\rangle$, so $0 = (AB - BA)|\psi\rangle = [A, B]|\psi\rangle$. The commutator of A and B must project $|\psi\rangle$ to the zero vector. The commutators of Pauli products are always scalar factors of Pauli products, where only the scalar factor can affect the length of $|\psi\rangle$, so it must be the case that the scalar factor is zero, meaning $[A, B] = 0$ (i.e. A and B commute).

If you start with the set of all stabilizers of a state, and iteratively remove stabilizers that can be formed by multiplying together other stabilizers, you are left with a small set of stabilizers referred to as *generators* of the state's stabilizer group. For example, $|000\rangle$ has seven stabilizers (not counting the vacuous identity stabilizer) and these stabilizers can be generated by ZII, IZI, and IIZ. An n qubit state will have at most n stabilizer generators. An n qubit state with exactly n stabilizer generators is a *stabilizer state*.

2.3 Stabilizer Tableaus

A *Clifford operation* is a unitary quantum operation that conjugates Pauli products into Pauli products. C is Clifford if, for all pauli products P, it is the case that $C^\dagger PC$ is also a Pauli product. In fact, a Clifford operation can be uniquely identified (up to global phase) by how it conjugates Pauli products.

A stabilizer tableau is a representation of a Clifford operation that simply directly stores how the Clifford operation conjugates each generator of the Pauli group. In this paper, I use the generators X_q and Z_q for each qubit q that the operation touches.

For example, here is a stabilizer tableau for the Controlled Y gate C_Y:

$$\text{tableau}(C_Y) = \begin{array}{c|cc|cc} & X_1 & Z_1 & X_2 & Z_2 \\ \hline \pm & + & + & + & + \\ 1 & X & Z & Z & Z \\ 2 & Y & & X & Z \end{array}$$

Each column describes how C_Y conjugates one of the four generators of the two qubit Pauli group. The column labelled X_1 states that C_Y conjugates X_1 into $+X_1Y_2$, i.e. that $C_Y^{-1}X_1C_Y = X_1Y_2$.

Any qubit not mentioned by a tableau is unaffected by that tableau. For example, if X_q does not appear in the tableau then it is understood that the tableau's operation conjugates X_q into X_q.

In order for a stabilizer tableau to be valid, i.e. to represent a Clifford operation, it must preserve commutativity and anticommutativity. The Pauli products in its columns must commute or anticommute in the same way that its generators do. The column for X_a must commute with the column for X_b and the column for Z_b, but must anticommute with the column for Z_a. Also, a valid tableau doesn't have missing columns. If there is a row for qubit q, there must be columns for the generators X_q and Z_q.

Storing an n-qubit stabilizer tableau uses $4n^2 + O(n)$ bits. There are $2n$ generator outputs to store, each output has n Pauli terms, and each Pauli term is xz-encoded into two bits.

In Stim, stabilizer tableaus are represented by the class `stim::Tableau`. Storing the stabilizer tableau is Stim's dominant space cost. All other space costs are linear in the number of qubits.

2.3.1 Conjugating a Pauli Product by a Tableau

To conjugate a Pauli product observable by the Clifford operation represented by a stabilizer tableau, start by decomposing the Pauli product into the generators listed in the tableau. Then use the fact that conjugation distributes over matrix multiplication to conjugate each of the generators, and re-assemble the resulting Pauli product.

For example, suppose we want to apply the Controlled-Y tableau to the Pauli product X_1Y_2. We start by decomposing X_1Y_2 into its generators $i(X_1)(X_2)(Z_2)$. We then look up each generator in the table, replacing the generator with the result of the lookup. This produces the result $i(X_1Y_2)(Z_1X_1)(Z_1Z_2)$. We multiply these Pauli products together to get the result X_1. We conclude that $C_Y^{-1}X_1Y_2C_Y = X_1$.

When a Pauli product involves qubits that do not appear in the stabilizer tableau, and we want to conjugate the Pauli product inplace, those additional qubits do not need to be touched. If the tableau covers m qubits, and the Pauli product has c qubits in common with the tableau, then each of the $O(c)$ table lookups has cost $O(m)$ (including multiplying together terms at the end). Therefore, the complexity of inplace conjugation of an n qubit Pauli product by an m qubit stabilizer tableau with c qubits in common is $O(mc)$ with no dependence on n (this assumes that it is not necessary to determine which qubits are in common). Note that $O(mc) \subseteq O(m^2)$.

In Stim, the method `stim::Tableau::apply_within` performs inplace conjugation of a Pauli product by a tableau. Stim also supports out of place conjugation via the `stim::Tableau::operator()` operator.

2.3.2 Composing Tableaus

Given an N-qubit stabilizer tableau A and an m-qubit stabilizer tableau B where $m \leq N$, we may want to append or prepend B into A.

Inplace appending B into A, i.e. performing the mutation $A \to B \circ A$, is done by inplace conjugating each of A's columns with B. Since there are N columns, and inplace conjugation by an m qubit tableau takes $O(m^2)$ time, inplace appending takes $O(Nm^2)$ time.

Inplace prepending B into A is done by computing the result of conjugating each of B's generators' outputs by A. For each generator g_q in B, we compute $g'_q = A^{-1}B^{-1}g_qBA$. After computing all of the g'_q values, we write g'_q into A under g_q for each g_q from B. There are m generators to conjugate by B and then by A. The cost of conjugating by B is $O(m)$, since the generator has 1 qubit in common with B. The cost of then conjugating by A is $O(Nm)$, since the output from B has at most m qubits in common with A. Therefore the complexity of inplace prepending is the same as inplace appending: $O(Nm^2)$.

In Stim, inplace composition is implemented by the `stim::Tableau::inplace_scatter_append` method and the `stim::Tableau::inplace_scatter_prepend` method. There are also many specialized optimized methods for inplace composition of common gates, such as `stim::Tableau::prepend_SQRT_Z`.

2.3.3 Inverting Tableaus

The inverse T^{-1} of a stabilizer tableau T is the unique tableau that satisfies $T \circ T^{-1} = T^{-1} \circ T = I$, where I is the identity tableau.

Although Clifford circuits can contain highly nonlocal effects such as entanglement, inverting the Pauli terms in a stabilizer tableau is almost entirely a local process. For example, consider the following partially specified stabilizer tableau:

$$T = \begin{array}{c|cc|cc|cc|cc} & X_1 & Z_1 & X_2 & Z_2 & X_3 & Z_3 & X_4 & Z_4 \\ \hline \pm & ? & ? & ? & ? & ? & ? & ? & ? \\ 1 & ? & ? & ? & ? & ? & ? & ? & ? \\ 2 & ? & ? & ? & ? & ? & ? & ? & ? \\ 3 & ? & ? & ? & ? & ? & ? & ? & ? \\ 4 & ? & ? & ? & ? & X & & ? & ? \end{array}$$

This tableau specifies four bits of information. It states that conjugating X_3 by T will produce a Pauli product that has an X term on qubit 4, and that conjugating Z_3 by T produces a Pauli product with no term on qubit 4. Nothing else is specified.

What can we determine about the inverse tableau, using only these four bits of information? Well, we know that the generator X_4 commutes with $T^{-1}X_3T$. Therefore TX_4T^{-1} must also commute with $TT^{-1}X_3TT^{-1} = X_3$, since conjugation preserves commutation. This means that the term on qubit 3 of TX_4T^{-1} must be either X or I. Furthermore, because X_4 commutes with $T^{-1}Z_3T$, we know TX_4T^{-1} must also commute with Z_3. This leaves only one possibility for the term on qubit 3 resulting

from conjugating X_4 by the inverse of T: the identity gate. We can similarly solve for the term on qubit 3 of TZ_3T^{-1}. Given four bits of information about the tableau, we have derived four bits of information about the inverse tableau:

$T^{-1} =$

	X_1	Z_1	X_2	Z_2	X_3	Z_3	X_4	Z_4
$\pm$	?	?	?	?	?	?	?	?
1	?	?	?	?	?	?	?	?
2	?	?	?	?	?	?	?	?
3	?	?	?	?	?	?		Z
4	?	?	?	?	?	?	?	?

The terms on qubit b that result from applying T to X_a and Z_a always completely determine the terms on qubit a from applying T^{-1} to X_b and Z_b. To compute the Pauli terms of the inverse tableau, all that is needed is to transpose the input and output indices and then apply a few local tweaks that correspond to solving the commutation constraints.

With the Pauli terms computed, we can move on to computing the signs. Let S be the tableau with the same Pauli terms as T^{-1}, but with all signs positive. If the sign in the column for the generator g_q is negative, then round-tripping g_q through S and then T will return negative $-g_q$ instead of $+g_q$. That is to say, the sign of the g_q column in T^{-1} is equal to $TSg_qS^{-1}T^{-1}g_q$. To compute the signs, we evaluate this expression for each generator g_q.

Interestingly, computing the signs of the inverse tableau is more expensive than computing the Pauli terms. It takes $O(n^2)$ time to compute the Pauli terms of the inverse of an n qubit tableau, and $O(n^3)$ time to compute the signs. One possible way to avoid this cost is to associate a pair of signs with every row of the tableau, such that the column signs of the inverse tableau are equal to the row signs of the tableau. When appending and prepending operations into a tableau, the row signs can be updated along with the column signs at no additional cost (asymptotically speaking). However, for the use case in this paper, only the column signs are needed so I won't bother with including the row signs.

In Stim, the inverse of a tableau can be computed using the `stim::Tableau::inverse` method.

2.4 Pauli Frames

A Pauli frame stores, for each qubit, whether or not that qubit has been bit flipped and/or phase flipped relative to some reference state. A Pauli frame can be moved through a Clifford circuit in the same way that Pauli product observables are moved through the circuit: by conjugating the frame by the Clifford operations that the frame is passing through. In effect, a Pauli frame is just a Pauli product where the global phase is ignored.

2.4.1 Tracking Noise

When simulating a noisy stabilizer circuit, where the noise is composed of probabilistic Pauli operations, it is not necessary to directly apply the noise to the simulated qubits. Instead, the noise can be accumulated into a Pauli frame stored alongside the simulation [23]. As the simulation progresses, applying gates to the qubits, the Pauli frame is kept synchronized with the simulation by conjugating the frame's contents by the same gates.

When the simulation reports the measurement result from a qubit q, and the Pauli frame is storing an X or Y update for q, the measurement result is intercepted and inverted before being forwarded along. The forwarded measurement are equivalent to samples from the noisy circuit.

In other words, by using a Pauli frame, you can augment any stabilizer simulator into a noisy stabilizer simulator. This works even without access to the internal implementation details of that simulator.

2.4.2 Tracking Corrections

Pauli frames can also track corrective Pauli operations, so that those operations don't have to be applied to the underlying qubits. For example, this can be used to augment a black box non-adaptive stabilizer simulator into a simulator that supports adaptive Pauli operations that depend on previous measurement results. This even works if the simulator is replaced by a physical quantum computer [29].

The ability to track corrections from the outside using a Pauli frame is a key technique in quantum error correction [23]. It is the fundamental reason why just-in-time decoding of errors isn't needed for stabilizer codes like the surface code [10]. Any corrections that are needed can be backdated into the Pauli frame at the appropriate time, and propagated forward through the circuit to the current time by toggling recorded measurements as dictated by the Pauli frame.

3 Pauli Frame Simulation

A Pauli frame simulator works by propagating a Pauli frame through a circuit [27]. Clifford operations conjugate the frame, noise processes multiply Paulis into the frame, and collapsing operations randomize parts of the frame. The benefit of Pauli frame simulation is that the frame only takes $O(n)$ bits to store and $O(1)$ time to update per gate. The downside of Pauli frame simulation is that it doesn't tell you measurement results; it tells you whether or not measurements *were flipped*. To convert this information into actual measurement results, you need a noiseless reference sample to diff against.

In a Pauli frame simulation, noise processes must be Pauli channels (i.e. equivalent to sampling a Pauli product from a probability distribution and applying the sampled product to the system's state). For example, dephasing and depolarization are Pauli channels but relaxation to the ground state and leaking outside the computational basis aren't. A Pauli frame simulator simulates a Pauli channel noise process by sampling a Pauli product from the Pauli product distribution corresponding to the noise process and multiplying the sampled Pauli product into the Pauli frame.

Paulis can also be multiplied into the Pauli frame by collapsing operations (initializations, resets, and measurements). These operations introduce new stabilizers into the system. To force later measurements to have random results if they measure observables that anticommute with a newly introduced stabilizer, the stabilizer is multiplied into the Pauli frame with 50% probability. In other words, after each initialization and reset and measurement, a `Z_ERROR(0.5)` is applied to the target qubit. Conveniently, inserting these random Z operations is equivalent to replacing the noiseless reference sample that is being used with another independent noiseless sample from the circuit. The random Z operations make all reference samples interchangeable and reusable.

Putting all of the pieces together, a Pauli frame simulation works as follows. (Note: this summary will describe actions as if they were applied to one isolated Pauli frame. Stim uses SIMD operations to apply each action to multiple frames simultaneously.)

1. A reference sample is collected from the target circuit, with all noise processes disabled, using some other method. The reference sample can be used and reused as many times as needed.

2. A Pauli frame is initialized with a randomly chosen I or Z term on each qubit (due to initialization being a collapsing operation).

3. The Pauli frame is advanced through the circuit.

    - When the frame crosses a Clifford operation C, the frame undergoes the update $F \to C^{-1}FC$.
    - When the frame crosses a reset, the qubit's term in the frame is set to the identity operation. Then a Z on the target qubit is multiplied into the frame with 50% probability, because resets are collapsing operations.

- When the frame crosses a Pauli error channel, a Pauli product is sampled from the error channel (usually the no-error identity operation is by far the most likely) and multiplied into the frame.
- When the frame crosses a measurement, the result of that measurement (for the simulation run that the frame represents) is reported as $r_M \oplus x_q$ (where r_M is the reference measurement result and x_q is true if the Pauli frame has an X or Y term on the target qubit). Then a Z on the target qubit is multiplied into the frame with 50% probability, because measurements are collapsing operations.

4. If more samples of the circuit are needed, go to step 2. Otherwise stop.

4 Stabilizer Tableau Simulation

Consider the first measurement in a stabilizer circuit. (If there are multiple measurements tied for first, pick one of them arbitrarily.) This measurement is measuring the observable Z_q, where q is the qubit targeted by the measurement. Between this measurement and the $|0\rangle$ states at start of the circuit there are a variety of Clifford gates forming a compound Clifford operation C. By conjugating the current-time observable Z_q by the inverse Clifford operation C^{-1}, we get some observable from the start of time that is equivalent to Z_q at the current time. Measuring CZ_qC^{-1} at the start of time is equivalent to measuring Z_q at the current time.

Suppose that CZ_qC^{-1} is equal to $-Z_aZ_b$. At the start of time we know that qubits a and b are in the $|0\rangle$ state, i.e. in the $+1$ eigenstate of the Z observable. Therefore we can replace Z_a with $+1$, and we can do the same for Z_b. If we were to measure $-Z_aZ_b$ at the start of time, we would get a measurement result of -1. And since this observable is equivalent to Z_q at the current time, we can conclude that measuring Z_q now should also return a deterministic measurement result of -1.

Suppose alternatively that CZ_qC^{-1} was equal to $-X_aX_b$. This start-of-time observable anticommutes with the initial state, so the measurement result will be random. But, in order to fully resolve the measurement, we need to simplify the observable we are dealing with. The key insight to make is that, because all qubits are initialized into the $|0\rangle$ state, a controlled operation inserted at the start of time will have no effect on the state (because the control is not satisfied). However, inserting a controlled operation at the start of time can change the start-of-time observables. For example, if we insert a CNOT operation controlled by qubit a and targeting qubit b at the start of time, we will change C so that CZ_qC^{-1} is equal to $-X_a$ instead of $-X_aX_b$. We like this, because it reduces the number of terms that are present. Measuring Z_q at the current time is now equivalent to measuring $-X_a$ at the beginning of time. The latter measurement is clearly 50/50 random and forces qubit a into either the $|+\rangle$ or $|-\rangle$ state (at the start of time). We emulate the measurement collapse by inserting a Hadamard gate at the start of time (to prepare a $|+\rangle$) and then randomly inserting an X gate or not before the Hadamard (to choose between $|+\rangle$ and $|-\rangle$). After this change to the circuit, the Z_q observable at the current time is equivalent to either Z_a or $-Z_a$ at the start of time, and we have reduced the random measurement case to the deterministic measurement case.

Each measurement in the circuit, working from the start of time to the end of time, can be resolved in the same way that we resolved these two example cases. Measurements whose equivalent-start-of-time-observable contain only Z terms are deterministic, and have a result equal to the sign of their equivalent-start-of-time-observable. After reporting this result, the offending measurement gate can be deleted from the circuit. Measurements whose equivalent-start-of-time-observable contain an X or Y term trigger an elimination procedure which simplifies their observable into an observable with a single X term. This can be done by introducing no-effect controlled operations and no-effect S gates at the start of time. Once a single term remains, a Hadamard gate is inserted at the start of time to emulate the X observable collapsing and an X gate is randomly inserted at the start of time to emulate the randomness of the result. This forces the measurement under consideration to become deterministic, so it can be handled by the deterministic case.

4.1 The Asymptotic Benefits of Backwards Thinking

At this point I should note that, in previous work [1], this whole process was explained in reverse. Instead of being framed in terms of how single qubit observables at the current time mapped to compound observables at the start of time, previous work was framed in terms of single qubit stabilizers and "destabilizers" at the start of time being mapped to compound Pauli products at the current time. For example, measurements were classified as deterministic or random based on whether or not they anticommuted with any of the current-time stabilizers. This is equivalent to checking whether the Z observable mapped backward anticommutes with the initial state for the same reason that inverting a Clifford tableau is nearly a transposition.

The downside of thinking in terms of mapping forwards, instead of mapping backwards, is that the measurement result is not immediately available when a measurement is deterministic. When mapping backwards, the measurement result is the sign of the equivalent observable at the start of time. This allows a deterministic measurement to be classified as deterministic and resolved in worst case time $O(n)$, where n is the number of qubits. When mapping forwards, it is necessary to find a combination of stabilizers which multiply together to form the desired measurement's observable. That process has a worst case time of $O(n^2)$, instead of $O(n)$.

4.2 Tracking the Tableau

To perform a stabilizer tableau simulation efficiently, we need a data structure where we can efficiently append Clifford operations (as we progress further into the circuit), efficiently prepend operations (for the elimination process at the beginning of time), and efficiently determine the start-of-time observable of a measurement. In other words, we need a stabilizer tableau.

Here are the steps used to perform a stabilizer tableau simulation:

1. Initialize an identity stabilizer tableau T. To aide the intuition, imagine it being positioned at the start of the circuit being simulated, just after the qubits have been initialized into the $|0\rangle$ state.

2. Begin folding circuit operations into T, working from earliest to latest.

   - To fold a Clifford gate C into T, perform an inplace prepend of C^{-1} into T then delete C from the circuit. (We prepend the inverse of C, instead of appending C, because T is tracking the inverse of the circuit processed so far.)
   - To fold a measurement gate on qubit q into T, first check whether or not the Z_q column in T contains any X or Y terms. If it does, the measurement is random and must be resolved.
     - If the measurement is random, arbitrarily pick one of the X or Y terms to be the "pivot". For each other X or Y term, perform an inplace append into T of a CNOT operation controlled by the pivot targeting that term. (It is not necessary to remove the leftover Z terms.) To collapse the target qubit, append into T a single qubit operation targeting the pivot, changing its term to a Z (from either an X or Y). Finally, to randomize the measurement result, flip a coin to decide whether or not to inplace append an X operation on the pivot into T.

     The measurement is now deterministic. Report the sign of the Z_q column as the result, and delete the measurement gate from the circuit.
   - To fold a reset gate into T, perform a measurement on the qubit without reporting the result. The qubit is now either in the $|0\rangle$ state or the $|1\rangle$ state, determined by the sign of the Z_q column of T. To force the qubit into the $|0\rangle$ state, overwrite the Z_q column's sign with $+1$.

The asymptotic complexity of this simulation is $O(ng + nd + n^2 r)$ where n is the number of qubits, g is the number of gates, d is the number of measurements that have a deterministic result given

previous measurements, and r is the number of measurements that have random results. (Note: d and r also include contributions from operations such as resets, which implicitly perform a measurement as part of their implementation.) This is an improvement over the complexity $O(ng + n^2d + n^2r)$ from previous work.

An example of a context where this improvement provides an asymptotic advantage is simulating a distance d surface code circuit for d rounds. Typically, such a circuit will have $\Theta(d^2)$ qubits, $\Theta(d^2)$ deterministic measurements in each round, and $\Theta(d^2)$ random measurements in the first and last round. Simulating this circuit by tracking the forward tableau has a worst case simulation cost of $\Theta(d^7)$ whereas tracking the inverse tableau results in a worst case cost of $\Theta(d^6)$. (Interestingly, the $\Theta(d^6)$ cost comes entirely the random measurements in the first and last rounds. All of the intermediate rounds combined have a cost of $\Theta(d^5)$.)

5 Software Engineering

5.1 Data layout and vectorization

There are many factors that affect software speed, such as the choice of programming language, the underlying algorithm, and company culture. However, when attempting to write code that performs near the limits of what your computer is capable of, there are some factors that *have* to be considered. Data layout is one of those factors.

In a good data layout, most data accesses are sequential. They are performed while iterating across contiguous memory. This makes the memory accesses predictable, so that various caching mechanisms can hide latency by prefetching data before it is needed.

In a bad data layout, data accesses are disorganized and difficult to predict, resulting in cache misses. This can easily slow down your code by an order of magnitude (that's what it did in some quick tests I did) if not more [26].

When initially imagining Stim, I guessed that it was going to spend most of its time doing three key types of operations. (1) Applying unitary operations and deterministic measurements to the stabilizer tableau by interacting its columns. (2) Resolving random measurements by performing Gaussian elimination over the rows of the stabilizer tableau. (3) Tweaking Pauli frames as they propagated through the operations in a circuit.

Notice that there are both row-wise and column-wise operations being applied to the stabilizer tableau. This is bad, because it means we have to pick one to be fast and one to be slow. Alternatively, we can spend effort transposing the table data to switch from one being fast to the other being fast. In the circuits I care about (error correcting circuits) random measurements tend to come in large bursts. With that in mind, I decided to go with the switching strategy. Usually the stabilizer tableau is stored in column major order, so that unitary operations and deterministic measurements are efficient. When (hopefully large batches of) random measurements need to be processed, the tableau is temporarily transposed into row major order. (An example of a circuit that penalizes this strategy is a surface code circuit with a gauge covering a single missing data qubit [24].)

I did experiment with an alternative data layout where 256x256 bit blocks from the table were contiguous in memory. This significantly reduces the cost of switching from column-wise to row-wise operations (because only local transposes of the 256x256 blocks are needed). However, it also forces key operations like Pauli string multiplication to skip through memory instead of iterating through contiguous memory. Profiling showed that the costs outweighed the benefits.

For the Pauli frame simulator, I was initially worried that a good data layout didn't exist. The problem is that the gates being performed by the circuit are not predictable, and each only affects a tiny number of bits in the Pauli frame. This is a worst case scenario when it comes to cache misses. Even for circuits with well organized operations, like surface code circuits operating on local 2d grids, some of the operations would have to be running "against the grain" of memory. I spent quite a lot of time trying to think of workarounds for this problem.

```
int inplace_pauli_string_multiplication(
        int n, __m256i *x1, __m256i *z1 __m256i *x2, __m256i *z2) {
    // The 1s and 2s bits of 256 two-bit counters.
    __m256i c1{};
    __m256i c2{};
    // Iterate over data in 256 bit chunks.
    for (int k = 0; k < n; k++) {
        __m256i old_x1 = x1[k];
        __m256i old_z1 = z1[k];
        // Update the left hand side Paulis.
        x1[k] ^= x2[k];
        z1[k] ^= z2[k];
        // Accumulate anti-commutation counts.
        __m256i x1z2 = old_x1 & z2[k];
        __m256i anti_commutes = (x2[k] & old_z1) ^ x1z2;
        c2 ^= (c1 ^ x1[k] ^ z1[k] ^ x1z2) & anti_commutes;
        c1 ^= anti_commutes;
    }
    // Determine final anti-commutation phase tally.
    return (popcount(c1) + 2 * popcount(c2)) % 4;
}
```

Figure 2: Vectorized Pauli string multiplication. Multiplies one fixed length xz-encoded Pauli string into another while computing the base-i logarithm of the resulting scalar phase. The loop body contains 4 SIMD loads, 4 SIMD stores, and 11 bitwise SIMD operations (after trivial compiler optimizations). I believe the number of bitwise SIMD operations is optimal, and would be very interested to hear if anyone can achieve the same effect with fewer.

Eventually, I realized that I shouldn't be trying to vectorize within a Pauli frame simulation. Instead, the correct thing to do is to vectorize *across multiple* Pauli frame simulations. The data layout I decided on was to have 1024 frames packed together into a two dimensional table of bits, with the major axis corresponding to the qubits in the frame and the minor axis (i.e. the contiguous-in-memory axis) corresponding to the different frames being tracked. Gates can then be applied to many frames using single instructions. For example, an S gate targeting qubit q is performed by xoring x_q into z_q, where x_q and z_q are the bits of the xz-encoded Pauli for qubit q in the Pauli frame. Instead of individually operating on each x_q and z_q from each frame, there is a 256 bit word containing 256 separate x_q bits from 256 frames, and a 256 bit word containing 256 separate z_q bits from 256 frames, and the S gate is applied to all 256 frames by one `_mm256_xor_si256` instruction computing the 256 new z_q values. The story is similar for all other gates (e.g. see Figure 3 for the CNOT gate's code). With this approach each gate may still touch memory in an unpredictable way, but the cost of the cache miss will be amortized over a thousand parallel frame updates.

Another data layout decision I had to make as part of implementing Stim was whether or not to interleave the x and z bits of xz-encoded Pauli products. Initially, I decided that for locality reasons each 256 bit word should contain the 128 x bits and 128 z bits from 128 Paulis. This is a bad idea. It violates a core rule of thumb for writing fast vectorized code: don't mix different data together. For example, consider simulating an S gate (again). As part of simulating the S gate, the x bits of a Pauli string have to be xored pairwise into the z bits. If the x and z bits are in separate words, you can process the xor part of the S gate for 256 Paulis using one AVX bitwise xor instruction. If the x and z bits are in the same word, the xor will have to be accompanied by some shifting and masking and these multiple instructions will have only processed 128 Paulis instead of 256. This realization flipped my decision from "yes interleave" to "definitely do not interleave". (I also found that alternating between 256 bit x words and 256 bit z words, like `XZXZXZXZ`, was worse than doing all of one then all of the

```
void apply_CNOT_to_batched_pauli_frames(
        int n, __m256i *x1, __m256i *z1 __m256i *x2, __m256i *z2) {
    // Iterate over tracked frames in chunks of 256.
    for (int k = 0; k < n; k++) {
        z1[k] ^= z2[k];
        x2[k] ^= x1[k];
    }
}
```

Figure 3: Vectorized code to apply a CNOT operation to a batch of $256n$ Pauli frames. Note that the code is looping over multiple Pauli frames, not over multiple qubits. The argument `x1` points at data storing whether or not, for each Pauli frame, there is currently an X error on the control of the CNOT (the other arguments are similar, but for the Z errors and/or the target qubit).

other, like `XXXXZZZZ`. That being said, in this case the difference was small enough that I wouldn't be confident of it reproducing.)

5.2 Sparse vs Dense

Stim uses a dense representation for its stabilizer tableau. Every bit is stored, even if almost all of the bits are zero. In some contexts, like simulating the surface code, the stabilizer tableau representation can be made sparse and so a dense representation is wasteful. Given how much I've been talking about the surface code in this paper, shouldn't I want to use a sparse representation instead of a dense one? Well, maybe. There's three issues that complicate the situation.

First, in order for a simulator to use a sparse representation, it has to *find* and *maintain* that representation. To do this well, the simulator has to quickly solve non-local problems like "where are the logical qubits hiding in this circuit?". In the general case, these problems look very similar to problems with non-linear scaling (like Gaussian elimination).

Second, sparse representations hide huge constant factors compared to dense representations. For example, Stim has a `--detector_hypergraph` mode where it groups error mechanisms present in a circuit based on which annotated detectors and logical observables those errors flip. This error analysis does work very similar to what the tableau simulator does, but using a sparse representation. Despite that, the error analysis is slower on most of the circuits I care about. The problem is that a key bottleneck operation is computing the symmetric difference of small sets of integers, and my best efforts at writing code to do this quickly produced a result that takes on the order of ten nanoseconds to process sets with ten items. By contrast, using a dense bit packed representation where the symmetric difference is computed using 256 bit wide xor operations, ten nanoseconds is enough time to process ten thousand items. In other words, if you care more about speed than space, you may find that a sparse representation is worse until the sparsity of your data exceeds 99.9%.

Third, there is elegance in a piece of software that is *unconditionally* fast. It's bad enough that Stim's performance depends on the density of random measurements in a circuit. I would prefer not to involve even more complicated properties like the existence of a stabilizer basis with low weight generators.

Using a sparse representation would be more appropriate for circuits with hundreds of thousands or millions of qubits, where even storing the stabilizer tableau becomes problematic. Sparse representations are also more compelling in contexts where the hard or finicky parts of finding the sparse basis are embedded into the representation of the problem (e.g. explicitly annotating the locations of the logical qubits in the circuit).

5.3 GPU Experiment Failure

I did a small test to check if using a GPU would be beneficial for stabilizer tableau simulation or Pauli frame simulation. All of the expensive operations in a stabilizer simulation are embarrassingly parallel bitwise operations. They are very similar to xoring two arrays together. So I wrote a WebGL2 fragment shader to xor together two large arrays, and tested how long it took.

I found that (surprisingly) the shader was roughly as fast at xoring as my CPU. I expected the performance to be much better. In hindsight, I suspect this is because the arithmetic intensity of xoring two arrays together is *extremely* low, and doesn't use floating point arithmetic. That is to say, bitwise xoring doesn't hit on the comparative strengths of GPUs over CPUs.

This quick xor test suggests a GPU would perform poorly (or at least not significantly better than a CPU) at stabilizer simulation. Since programming GPUs is harder, and was unlikely to give much benefit, I dropped the idea.

(I caution the reader that perhaps I'm simply not familiar enough with GPU programming to have produced a fast implementation. For example, perhaps a compute shader would be more effective or perhaps I should have used a standalone GPU instead of an embedded GPU. There's certainly no fundamental obstacle to a specialized piece of hardware running huge bitwise operations significantly faster than a CPU.)

5.4 Threading

I tried adding threading to various parts of Stim. For example, when given a batch of CNOT operations to do, I tried partitioning them into two groups and running the two groups on separate threads. Surprisingly, this was less than ten percent faster. I didn't look deeply into why there was so little benefit; I simply decided based on the measurement that the complexity of threading batches of operations wasn't worth the gain.

One place where I found that it was worth adding threading was when transposing stabilizer tableaus. The stabilizer tableau's data is divided into four independent pieces (x/z bits of X/Z observables), so it's trivial to process each one on a separate thread without worries of contention. Overall I found that at large sizes this was about twice as fast as not threading the tableau transpose.

5.5 Entropy

When running noisy Pauli frame simulations, Stim can consume gigabytes of entropy per second. Stim gets this entropy by using `std::mt19937_64`, the 64 bit Mersenne twister PRNG included in C++'s standard library. The PRNG is seeded using `std::random_device`, the questionable [17] source of external entropy built into C++'s standard library.

Stim is often simulating noise that occurs with low probability (e.g. $p < 1\%$). When deciding whether or not each instance of a noise process has occurred, Stim applies an optimization that reduces waste in the conversion from max-entropy bits to low-entropy bits. Instead of sampling from `std::bernoulli_distribution(p)` n times, Stim samples the gap between errors using `std::geometric_distribution(p)` np times (on average). This reduces the expected cost of sampling n potential errors from $\Theta(n)$ to $\Theta(np + 1)$.

Sometimes noise occurs with intermediate probability (e.g. $10\% < p < 40\%$). To decide whether or not this type of noise occurs, Stim uses a hybrid sampling strategy. The desired probability p is decomposed into a truncated probability $p_{\texttt{trunc}} = \lfloor 256p \rfloor / 256$ and a refinement probability $p_{\texttt{refine}} = \frac{p - p_{\texttt{trunc}}}{1 - p_{\texttt{trunc}}}$. The truncated probability is a multiple of 2^{-8}, so bits that are true with this probability can be sampled quickly and exactly by generating a uniformly random byte, dividing it by 256, and comparing it to the truncated probability. Bits that are true with the refinement probability can also be sampled quickly, because the refinement probability is small. The bitwise OR of a truncated probability bit and a refinement probability bit is a bit that is true with the desired intermediate probability.

These optimizations, and others, allow Stim to sample from any Bernoulli distribution at gigahertz rates.

5.6 Sampling Detection Events

A "detector" is a specified set of measurement locations in a circuit, with the promise that xoring those locations' measurement results together will produce a deterministic value (under noiseless conditions). When a detector's measurements xor together into a value opposite to the expected one, that is a "detection event" indicating the presence of errors. In addition to its measurement result sampling mode `--sample=#`, Stim supports a detection event sampling mode `--detect=#`.

Note that, in principle, detection events can be sampled with better asymptotic complexity than measurements. For example, consider that, in the surface code, an X error on a data qubit will flip every single future measurement of a Z stabilizer involving that qubit. By contrast, when sampling detection events, this same error flips two nearby detectors and that's it. By sampling detection events instead of measurements, errors with unbounded non-local effects can become errors with bounded local effects. Another benefit of sampling detection events is that, since the only data that is being reported is which detectors were flipped, otherwise-distinct errors can be fused together [8]. Ultimately, by using a sparse representation for the set of flipped detectors, and sampling low-probability errors the way I described in Section 5.5, sampling the detection events in a circuit with $O(n)$ gates and detectors can be done in $O(npd + 1)$ operations (where p is the average error probability and d is the average number of detectors flipped by an error). Contrast with the $O(n)$ work needed to produce measurement samples using a Pauli frame simulation.

Currently, Stim doesn't implement the asymptotically efficient detector sampling method. Stim produces detector samples by sampling measurement results and then combining them. This is because, for the noise levels around 0.1% that are most interesting to me, the larger constant factors inherent in using a sparse representation outweigh the asymptotic gains.

5.7 Testing

Stim contains a lot of code that is very easy to get wrong. There are hundreds of opportunities for sign errors, transposition errors, substitution errors, and omissions that would go undetected without testing. For example, embarrassingly, version 1.0 had a transposition error where the effects of the `X_ERROR` and `Z_ERROR` noise channels were swapped when in `--repl` mode.

To verify the behavior of the operations in Stim, they are redundantly specified and the redundancies are checked for consistency. Each Clifford operation is specified in at least three different ways. As a unitary matrix:

$$\texttt{unitary}(\sqrt{Y}) = \frac{1}{2}\begin{bmatrix} 1-i & 1-i \\ i-1 & 1-i \end{bmatrix}$$

As a stabilizer tableau:

$$\texttt{tableau}(\sqrt{Y}) = \begin{array}{c|c|cc} & & X_1 & Z_1 \\ \hline \pm & & + & - \\ 1 & & Z & X \end{array}$$

And as hand-optimized simulator code calling vectorized subroutines:

```
void Tableau::prepend_SQRT_Y(size_t target) {
    // This method contains an intentional bug.
    xs[target].swap_with(zs[target]);
    zs[target].sign ^= 1;
}
```

The unitary matrix and stabilizer tableau are validated against each other by using a vector state simulator and the state channel duality. For each column in the tableau, the state vector simulator is initialized to contain a number of EPR pairs equal to the number of qubits that the operation under test acts on. Let S be the subsystem made up of the first qubit from each EPR pair. The state vector simulator applies the column's input observable to S as a Pauli product (including sign), applies the unitary matrix of the operation under test, and applies the column's output observable to S as a Pauli product (including sign). It then verifies that the resulting state is equal (including global phase) to the state produced when acting just the unitary matrix of the operation under test on S. This proves that the unitary matrix conjugates the input Pauli product into the output Pauli product, as specified by the tableau.

For some operations, further checks are needed before I feel confident their unitary matrix and tableau are correct. For example, consider the $\sqrt{Y}$ operation. The Y operation has two square roots (up to global phase). It is an arbitrary convention which of the two roots is the principle root. What if I mixed them up? Then I would enter the tableau for $\sqrt{Y}^\dagger$ instead of $\sqrt{Y}$, and also the unitary matrix for $\sqrt{Y}^\dagger$ instead of $\sqrt{Y}$. Comparing the two will not catch the problem. In these cases, I also included consistency tests which verify some disambiguating circuit identity. For example, I want the choice of principle root to be consistent across axes, and I follow the usual convention that $\sqrt{Z} = \texttt{diag}(1, i)$. So I can disambiguate $\sqrt{Y}$ by verifying that $\sqrt{Y} = H_{YZ} \cdot \sqrt{Z} \cdot H_{YZ} \neq \sqrt{Y}^\dagger$ where $H_{YZ} = (Y + Z)/\sqrt{2}$.

Once an operation's tableau is established as being correct, it can be used to verify the hand optimized code. The test code does this by generating several large random tableaus [5], and confirming that composing the operation's tableau into the random tableau using a general method gives the same result as applying the hand optimized code. Because of the local and discrete nature of tableau operations, this sort of fuzzing is extremely effective at catching bugs.

One important risk factor in testing is the risk of simply forgetting to test an operation in the first place. I'd normally use code coverage tools to verify that the tests are exercising everything, but I couldn't get any C++ code coverage tool that I tried to work. Instead, the test code takes advantage of the fact that every supported gate appears as data in a list. This listed data is used for crucial functionality such as parsing, so I can expect it to be complete. With this listed data, one test can verify all of the defined gates. For example:

```
TEST(gate_data, tableau_data_vs_unitary_data) {
    for (const auto &gate : GATE_DATA.gates()) {
        if (gate.flags & GATE_IS_UNITARY) {
            EXPECT_TRUE(tableau_agrees_with_unitary(
                gate.tableau(), gate.unitary())) << gate.name;
        }
    }
}
```

Noise channels are harder to test than Cliffords, because the behavior of noise channels isn't deterministic. I tested noise channels by creating tests that sampled circuits applying the noise channel to simple states. Then, statistical checks verified the consistency of the samples against independent definitions of noise channels from Cirq [28]. The `X_ERROR`/`Z_ERROR` transposition that I mentioned slipped through these tests because originally they only exercised the bulk sampling API (which was a natural choice due to needing many samples), not the single-sample interactive API.

Tests are run with compiler optimizations enabled, but also run without compilation optimizations with address and memory sanitization checks enabled. A continuous integration system verifies that the tests pass before allowing commits to be merged into the main branch.

```
[...........|*..........] 350 ns (vs 450 ns) ( 27 GBits/s) simd_bits_randomize_10K
[...........*|..........]  27 ns (vs  20 ns) (360 GBits/s) simd_bits_xor_10K
[...........*|..........] 4.7 ns (vs 4.0 ns) (210 GBits/s) simd_bits_not_zero_100K
[...........*..........] 6.0 ms (vs 6.0 ms) ( 16 GBits/s) simd_bit_table_inplace_square_transpose_diam10K
[...........*..........]  12 ms (vs  12 ms) (7.8 GBits/s) simd_bit_table_out_of_place_transpose_diam10K
[...........|*..........] 1.1 us (vs 1.5 us) (230 GBits/s) simd_compat_popcnt
[.........*<|..........] 7.5 ms (vs 5.0 ms) ( 13 GOpQubits/s) FrameSimulator_depolarize1_100Kqubits_1Ksamples_per1000
[...........*..........] 5.6 ms (vs 5.0 ms) ( 17 GOpQubits/s) FrameSimulator_depolarize2_100Kqubits_1Ksamples_per1000
[.........*<|..........] 3.2 ms (vs 2.0 ms) ( 31 GOpQubits/s) FrameSimulator_hadamard_100Kqubits_1Ksamples
[...........*..........] 2.0 ms (vs 2.0 ms) ( 49 GOpQubits/s) FrameSimulator_CX_100Kqubits_1Ksamples
[...........*..........]  69 us (vs  70 us) ( 73 MLayers/s) (3.5 GOutBits/s) FrameSimulator_sample_unrotated_surface_code_d5_samples1024
[...........*..........] 170 us (vs 170 us) ( 29 MLayers/s) (1.4 GOutBits/s) FrameSimulator_sample_unrotated_noisy_surface_code_d5_samples1024
[...........*..........]  59 ms (vs  60 ms) (700 kLayers/s) (2.3 GOutBits/s) FrameSimulator_sample_unrotated_surface_code_d41_samples1024
[...........*..........] 130 ms (vs 130 ms) (320 kLayers/s) (1.0 GOutBits/s) FrameSimulator_sample_unrotated_noisy_surface_code_d41_samples1024
[...........*..........] 160 us (vs 150 us) ( 31 kLayers/s) (1.5 MOutBits/s) TableauSimulator_sample_unrotated_surface_code_d5
[...........*..........] 630 ms (vs 650 ms) ( 64 Layers/s) (210 kOutBits/s) TableauSimulator_sample_unrotated_surface_code_d41
[...........*..........] 9.4 us (vs 8.5 us) (100 GPaulis/s) PauliString_multiplication_1M
[...........*..........] 690 ns (vs 700 ns) (140 GPaulis/s) PauliString_multiplication_100K
[...........*..........]  94 ns (vs  90 ns) (100 GPaulis/s) PauliString_multiplication_10K
[...........|*..........]  51 us (vs  60 us) tableau_random_10
[...........*..........] 1.9 ms (vs 2.0 ms) tableau_random_100
[...........*..........]  12 ms (vs  12 ms) tableau_random_256
[...........*..........] 210 ms (vs 200 ms) tableau_random_1000
[.........*<|..........] 200 ns (vs 120 ns) tableau_cnot_10Kqubits
```

Figure 4: Screenshot of results from `stim_benchmark`. Each line is the result of a small benchmark defined in Stim's source code. The ASCII bar indicators on the left show deviations from the reference time in a visual way that can easily be scanned over. The location of the asterisk, relative to the center of the ASCII bar, indicates how many decibels (factors of 1.26) slower or faster the run was (compared to the reference time). In the screenshot, a few benchmarks are running slow. That could indicate some sort of performance regression, or could just be noise.

5.8 Profiling and Optimizing

To determine the performance of individual components of Stim, I wrote a small benchmarking framework that would repeatedly run some task and compare the time it took to a reference time. For example, here is code benchmarking the multiplication of Pauli strings with ten thousand terms:

```
BENCHMARK(PauliString_multiplication_10K) {
    size_t n = 10 * 1000;
    PauliString p1(n);
    PauliString p2(n);
    benchmark_go([&]() {
        p1.ref().inplace_right_mul_returning_log_i_scalar(p2);
    }).goal_nanos(90).show_rate("Paulis", n);
}
```

Note that an important assumption here is that the compiler is inlining the `benchmark_go` method and the lambda. I found that making `benchmark_go` a template with the lambda's type as a parameter caused more consistent inlining, but I can make no guarantees.

The framework runs whatever is in the `benchmark_go` body for 0.5 seconds, counting how many iterations were finished. It then produces output comparing the inferred time to a reference time. Example benchmarking output can be seen in Figure 4. This benchmarking methodology would not work well for a team of people (e.g. because different computers require different reference times), but was sufficient for my purposes.

When I wanted to improve the performance of a benchmark, or fix a regression, I would build Stim using debug flags `-g -fno-omit-framepointer` and run it under Linux's `perf` tool to identify functions and lines of code taking disproportionate amounts of time. For example, the reason that I optimized the Bernoulli distribution sampling process to use less entropy per error is because profiling showed that generating entropy for errors was a significant bottleneck.

5.9 Portability

One of the major downsides of using AVX instructions is that they are not portable. They won't work on older CPUs, and they won't work on architectures besides x64 (e.g. most phones). Stim tries to work around this by hiding all AVX instructions behind an opaque type `simd_word`. Stim includes

one implementation of `simd_word` that uses 256-bit-wide AVX instruction, another that uses older 128-bit-wide SSE instructions, and another that uses no non-standard instructions. The presence of defines like `__AVX2__` then determine which `simd_word` implementation is used.

Because the size of a `simd_word` is not fixed, code is written in a fashion that avoids depending on this size. For example, the class `simd_bits`, which represents a bit packed array that supports vectorized instructions, will help the caller with this by automatically padding its size up to a multiple of `simd_word`'s size. `simd_bits` also exposes methods which handle the boilerplate parts of iterating over the words. `simd_bit_table` plays a similar role for two dimensional data.

5.10 Input Format

Users need some way of describing quantum circuits to Stim. I considered using an established format for this, such as OpenQASM. I dropped the idea because OpenQASM has a lot of semantics that would be time consuming to implement (such as functions, named variables, and file includes). Also OpenQASM is currently undergoing a breaking version change [2].

Consequently, I decided to make yet another input format. Stim's input format is nearly minimal. It takes a series of lines, and each line can specify a gate to apply to some targets. For example, the line "`H 0`" says to apply a Hadamard to qubit 0 and the line "`CNOT 0 1`" says to apply a Controlled-NOT operation controlled by qubit 0 targeting qubit 1. That's essentially all that's necessary to know in order to use the format, besides the names of supported gates.

There are of course several exceptions to this minimalist ideal. Blank lines are permitted. Lines can be suffixed with a comment prefixed by a hash "`# like this`". Gates can be broadcast over multiple targets like "`H 0 1 2`". And, most shamefully, there is a "`REPEAT N {...}`" macro which repeats a block of instructions a given number of times (drastically reducing some file sizes).

5.11 Output Format

By default, Stim outputs a series of '0' and '1' characters, one for each measurement result. The final measurement result is followed by a newline character '\n'. When multiple shots are taken, the measurement results (and newline) from the first shot are output, then the results (and newline) from the second shot, and so forth.

Stim also supports a binary output format, triggered by providing the command line argument `--out_format=b8`. This format packs 8 measurements results into each byte of output, ordered from least significant bit to most significant bit. The last byte output for a shot is padded to completion with 0s. There is no separator between the bytes from separate shots.

In addition to these "dense" formats, Stim supports sparse output formats like `--out_format=hits` and `--out_format=r8` that focus on the indices of non-zero bits. These output formats are useful when sampling detectors, because non-zero bits are rarer.

6 Comparison

I compared Stim's performance to Aaronson and Gottesman's `chp` simulator [1], IBM's Qiskit simulator (`qiskit method='stabilizer'`) [9], Google's Cirq simulator (`cirq.CliffordSimulator`) [28], and finally Anders and Briegel's `graphsim` [3] simulator.

I performed the comparison benchmarks on my work laptop: a ThinkPad running gLinux (a Google internal modified variant of Debian) with an Intel Core i7-8650U CPU @ 1.90GHz and 16GB of RAM. I compiled `stim` and `chp` with `g++ v10.2.1` using optimization level `-O3`. I generated circuit files understood by each simulator (samples of these files are included in the ancillary files of this paper), and then iterated through the files of each benchmark timing how long each simulator took. Output from the simulators was discarded, not recorded. I didn't do anything special to prepare the computer for benchmarking except ensure it was plugged in with a full battery with no other user programs

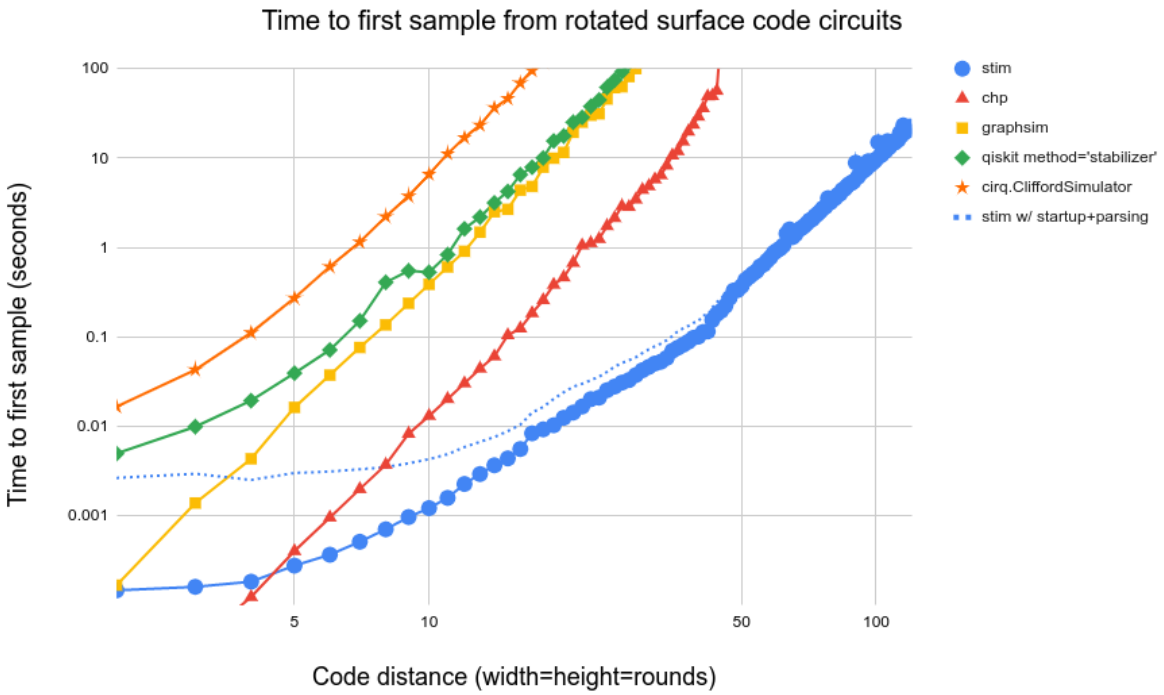


Figure 5: Delay until first sample on $d \times d \times d$ unrotated surface code memory circuits [15]. The intention of this benchmark is to highlight simulators that can take advantage of redundant structure in the surface code, such as the abundance of local stabilizers and deterministic measurements. Based on the apparent scaling, it appears that none of the simulators are making much use of the structure of the surface code. Stim scales comparatively well on this benchmark because Stim performs deterministic measurements in worst case linear time instead of quadratic time.

running. I didn't disable dynamic frequency scaling or auto-updating mechanisms (the simulators differ in performance by large enough factors that I don't think this is a problem).

Timing was done by calling `time.monotonic()` or `std::chrono::steady_clock::now()` before and after the key simulation methods and computing the difference (I edited source code to do this when necessary). Timing data doesn't include program startup or circuit parsing, except for "`stim with startup+parsing`" where I used `date +%s%N` before and after invoking Stim in Bash.

Because the various simulators I benchmarked don't consume a common format or support a common gate set, it was often necessary to decompose one gate into several for one of the simulators. For example, `chp` doesn't have a Z gate so two S gates were used instead when needed. Additionally, because `graphsim` is an API rather than an end-to-end tool, I had to write a bit of glue code to drive that API using data read from a file. For simplicity, I wrote this glue code to use `chp`'s file format which has very few gates.

After running the benchmarks I noticed that there were outlier times at the smallest sizes. I think this is because the last problem size in each benchmark was large, and so simulating it could for example consume a lot of memory and evict pages that would then fault during the next run (which would be the smallest case in the next benchmark). To correct this, I discarded the suspicious data for the three smallest cases from each benchmark and re-ran them on their own.

I chose five benchmarking tasks: bulk sampling a surface code circuit (see Figure 1), sampling a surface code circuit (see Figure 5), sampling a Bacon-Shor code circuit (see Figure 6), sampling a randomly generated circuit (see Figure 7), and sampling multi-level S state distillation (see Figure 8). See the figure captions for commentary on each benchmark task.

Overall, I would say that the benchmarks show that at larger sizes Stim outperforms everything else

Figure 6: Delay until first sample on $d \times d \times d$ Bacon-Shor code memory circuits [4]. The intention of this benchmark is to contrast with the surface code benchmark, in that the Bacon-Shor code also has a lot of structure but doesn't have lots of deterministic measurements. Note how Stim's apparent scaling on this benchmark looks more like the other simulators than it did in the surface code benchmark.

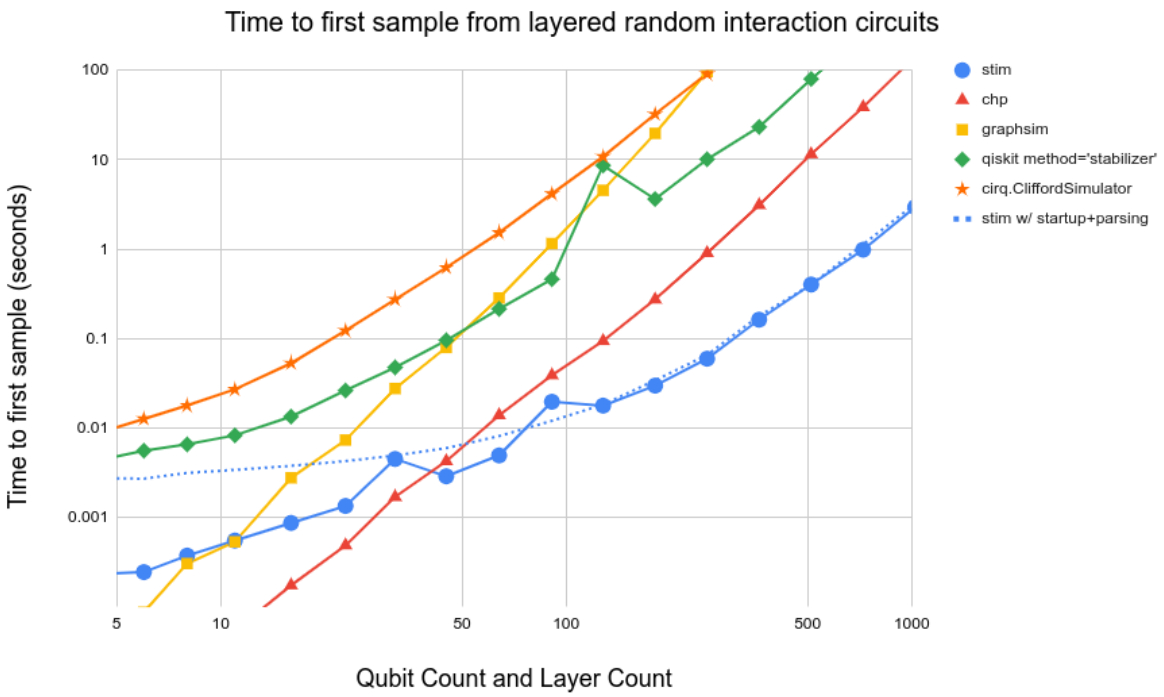


Figure 7: Delay until first sample on random circuits. The intention of this benchmark is to highlight simulators with good performance in circuits that lack much exploitable structure. Each circuit is made up of n qubits operated on for n layers. Each layer randomly applies an H, S, or I gate to each qubit, then samples a random pairing of the qubits and applies a CNOT to each pair, then samples 5% of the qubits to measure in a random basis. At the end of the circuit, every qubit is measured in a random basis. One effect I don't understand here is why Stim appears to be scaling differently from chp.

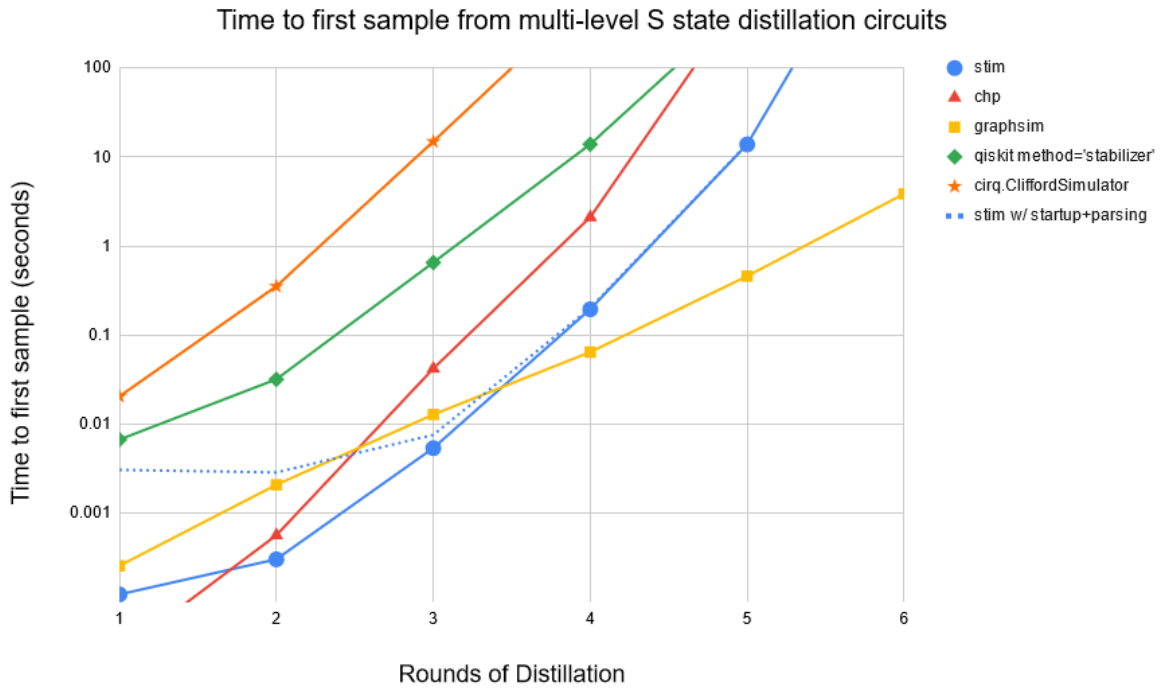


Figure 8: Delay until first sample on nested 7-to-1 S state distillation circuits [10]. The intention of this benchmark is to highlight simulators that do well on circuits made up of nearly independent pieces. For example, with 5 levels of distillation, there are initially 2401 independent pieces (each with 15 qubits) performing identical copies of the 7-to-1 distillation circuit (view in Quirk). Then the output qubits from each of these pieces are put into groups of seven and handed along to the next stage, which has 343 independent pieces. This process continues until there is a single output qubit remaining. graphsim does particularly well on this benchmark, due to the connected components of the graph state perfectly matching the independent pieces of the circuit.

by one or more orders of magnitude, and that when collecting thousands of samples Stim outperforms everything else by many orders of magnitude. A notable exception is the multi-level distillation task, where `graphsim` shines due to its ability to simulate piecewise circuits in a piecewise fashion. Actually, in most of the benchmarks, there is circuit structure present that should allow a simulator that notices and uses that structure to outperform Stim. This would be an interesting avenue for future work.

7 Conclusion

In this paper I presented Stim, a fast stabilizer circuit simulator. Stim was my attempt at simulating surface code circuits with tens of thousands of qubits and millions of operations in a tenth of a second. Although that goal wasn't reached, Stim is regardless a useful piece of software.

I still think it's possible to simulate stabilizer circuits with tens of thousands of qubits and millions of gates in a tenth of a second, at least in special cases. For example, when a person analyzes a surface code circuit, they notice the repetitiveness of the low level details of the circuit and zoom out to focus on topological details like lattice surgeries and braids. If the stabilizer simulator could do something similar, decent speedups should be possible. That is to say, one avenue of attack is to improve the performance of finding and maintaining sparse representations.

Another avenue of attack is to re-examine the assumptions going into the problem being solved. The motivating use case I described at the start of the paper was a user making small iterative changes to a large circuit. In that context the typical simulation is not a brand new circuit, but rather a slight variation of a circuit from just a moment ago. This suggests that work could be cached and reused. Alternatively, perhaps as part of creating the circuit, the user has provided information that can be used by the simulator. For example, the user may have specified the circuit in terms of repeated pieces or may have noted that a measurement is supposed to relate to previous measurements in a specific way. The simulator could verify this information (instead of having to find it) and then exploit it.

Ultimately, I hope the experimental and theoretical sides of the quantum community find Stim to be a useful tool. When you can do something ten or a hundred times faster, new and interesting use cases become possible. For example, with a fast enough simulator, you could consider brute forcing (or machine learning) the internal minutia of an error correction circuit to minimize undesirable error propagation properties. I certainly intend to take advantage of the speed in my own work.

8 Acknowledgements

I thank Michaels Broughton and Newman for putting up with excessive jokes about *brute speed*. I thank Michael Broughton for advice and discussions on profiling and optimizing code. I thank N. Cody Jones for useful discussions about simulation algorithms, and in particular for pointing out that randomizing the Z frame after resets and measurements was sufficient for sampling from the space of noiseless circuit outputs. I thank Hartmut Neven for creating an environment where this work was possible in the first place.

References

[1] Scott Aaronson and Daniel Gottesman. Improved simulation of stabilizer circuits. *Physical Review A*, 70(5):052328, 2004. DOI: 10.1103/PhysRevA.70.052328.

[2] Thomas Alexander, Lev Bishop, Andrew Cross, Jay Gambetta, Ali Javadi-Abhari, Blake Johnson, and John Smolin. "a new openqasm for a new era of dynamic circuits". `https://medium.com/qiskit/a-new-openqasm-for-a-new-era-of-dynamic-circuits-87f031cac49`, 2020. Accessed: 2021-01-26.

[3] Simon Anders and Hans J Briegel. Fast simulation of stabilizer circuits using a graph-state representation. *Physical Review A*, 73(2):022334, 2006. DOI: 10.1103/PhysRevA.73.022334.

[4] Dave Bacon. Operator quantum error-correcting subsystems for self-correcting quantum memories. *Physical Review A*, 73(1):012340, 2006. DOI: 10.1103/PhysRevA.73.012340.

[5] Sergey Bravyi and Dmitri Maslov. Hadamard-free circuits expose the structure of the clifford group. *arXiv preprint arXiv:2003.09412*, 2020. URL https://arxiv.org/abs/2003.09412.

[6] Sergey Bravyi, Dan Browne, Padraic Calpin, Earl Campbell, David Gosset, and Mark Howard. Simulation of quantum circuits by low-rank stabilizer decompositions. *Quantum*, 3:181, 2019. DOI: 10.22331/q-2019-09-02-181.

[7] Kaifeng Bu and Dax Enshan Koh. Efficient classical simulation of clifford circuits with non-stabilizer input states. *Physical review letters*, 123(17):170502, 2019. DOI: 10.1103/PhysRevLett.123.170502.

[8] Rui Chao, Michael E Beverland, Nicolas Delfosse, and Jeongwan Haah. Optimization of the surface code design for majorana-based qubits. *Quantum*, 4:352, 2020. DOI: 10.22331/q-2020-10-28-352.

[9] Qiskit Contributors. Qiskit: An open-source framework for quantum computing. 2019. DOI: 10.5281/zenodo.2562110.

[10] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland. Surface codes: Towards practical large-scale quantum computation. *Phys. Rev. A*, 86:032324, 2012. URL https://doi.org/10.1103/PhysRevA.86.032324. arXiv:1208.0928.

[11] Craig Gidney and Austin G Fowler. Efficient magic state factories with a catalyzed $|ccz\rangle$ to $2|t\rangle$ transformation. *Quantum*, 3:135, 2019. DOI: 10.22331/q-2019-04-30-135.

[12] Craig Gidney, Austin Fowler, and Michael Newman. Informal private conversations about simulation bottlenecks, 2021.

[13] Daniel Gottesman. Stabilizer codes and quantum error correction. *arXiv preprint quant-ph/9705052*, 1997. URL https://arxiv.org/abs/quant-ph/9705052.

[14] David Gross. Hudson's theorem for finite-dimensional quantum systems. *Journal of mathematical physics*, 47(12):122107, 2006. DOI: 10.1063/1.2393152.

[15] Clare Horsman, Austin G Fowler, Simon Devitt, and Rodney Van Meter. Surface code quantum computing by lattice surgery. *New Journal of Physics*, 14(12):123011, 2012. DOI: 10.1088/1367-2630/14/12/123011.

[16] Jakob Nielsen Hořeňovský. "response times: The 3 important limits". https://www.nngroup.com/articles/response-times-3-important-limits/, 1993. Accessed: 2021-01-26.

[17] Martin Hořeňovský. "generating random numbers using c++ standard library: the problems". https://codingnest.com/generating-random-numbers-using-c-standard-library-the-problems/, 2020. Accessed: 2021-01-26.

[18] "Yack" (https://quantumcomputing.stackexchange.com/users/11887/yack). Answer to "how many n-qubit stabilizer states are there?". Quantum Stack Exchange, 2020. URL https://quantumcomputing.stackexchange.com/a/11781/119. URL:https://quantumcomputing.stackexchange.com/a/11781/119 (version: 2021-01-27).

[19] Yifei Huang and Peter Love. Approximate stabilizer rank and improved weak simulation of clifford-dominated circuits for qudits. *Physical Review A*, 99(5):052307, 2019. DOI: 10.1103/PhysRevA.99.052307.

[20] Yifei Huang and Peter Love. Feynman-path-type simulation using stabilizer projector decomposition of unitaries. *Physical Review A*, 103(2), February 2021. DOI: 10.1103/physreva.103.022428. URL https://doi.org/10.1103/physreva.103.022428.

[21] Intel. Intel intrinsics guide (_mm256_and_si256). https://software.intel.com/sites/landingpage/IntrinsicsGuide/#text=_mm256_and_si256&expand=301, 2021. Accessed: 2021-01-26.

[22] Angela Karanjai, Joel J Wallman, and Stephen D Bartlett. Contextuality bounds the efficiency of classical simulation of quantum processes. *arXiv preprint arXiv:1802.07744*, 2018. URL https://arxiv.org/abs/1802.07744.

[23] Emanuel Knill. Quantum computing with realistically noisy devices. *Nature*, 434(7029):39–44, 2005. DOI: 10.1038/nature03350.

[24] Shota Nagayama, Austin G Fowler, Dominic Horsman, Simon J Devitt, and Rodney Van Meter. Surface code error correction on a defective lattice. *New Journal of Physics*, 19(2):023050, 2017. DOI: 10.1088/1367-2630/aa5918.
[25] Jakob Nielsen. *Usability engineering*. Morgan Kaufmann, 1994.
[26] Peter Norvig. "approximate timing for various operations on a typical pc". http://norvig.com/21-days.html#answers, 2014. Accessed: 2021-01-26.
[27] Patrick Rall, Daniel Liang, Jeremy Cook, and William Kretschmer. Simulation of qubit quantum circuits via pauli propagation. *Physical Review A*, 99(6):062337, 2019. DOI: 10.1103/PhysRevA.99.062337.
[28] Quantum AI team and collaborators. Cirq, October 2020. URL https://doi.org/10.5281/zenodo.4062499.
[29] Matthew Ware, Guilhem Ribeill, Diego Riste, Colm Ryan, Blake Johnson, and Marcus P da Silva. Experimental pauli-frame randomization on a superconducting qubit. In *APS March Meeting Abstracts*, volume 2017, pages L46–004, 2017. DOI: 10.1103/PhysRevA.103.042604.
[30] Wikipedia. Advanced Vector Extensions — Wikipedia, the free encyclopedia. http://en.wikipedia.org/w/index.php?title=Advanced%20Vector%20Extensions&oldid=1021841294, 2021. [Online; accessed 11-May-2021].

A Short Usage Examples

A.1 (Command Line) Sample a noisy repetition code 1024 times

```
echo -e "
    REPEAT 20 {\n
        X_ERROR(0.01) 0 1 2 3 4 5 6 7 8 9 10\n
        CNOT 0 1 2 3 4 5 6 7 8 9\n
        CNOT 10 9 8 7 6 5 4 3 2 1\n
        MR 1 3 5 7 9\n
    }
" | ./stim --sample=1024
```

A.2 (Python) Sample a noisy repetition code 1024 times

```
import stim
distance = 5
rounds = 20
bit_flip_rate_per_round = 0.01
qubits = range(2 * distance + 1)
# Build circuit.
c = stim.Circuit()
c.append_operation("X_ERROR", qubits, bit_flip_rate_per_round)
c.append_operation("CNOT", qubits[:-1])
c.append_operation("CNOT", qubits[::-1][:-1])
c.append_operation("MR", qubits[1::2])
c *= rounds
# Collect and print samples.
sampler = c.compile_sampler()
samples = sampler.sample(1024)
for sample in samples: print(sample)
```

A.3 (Python) Interactive quantum teleportation

```
import stim
s = stim.TableauSimulator()
# Share EPR pair.
s.h(1)
s.cnot(1, 9)
# Prepare state to teleport.
s.h(0)
s.s(0)
# Alice measurements.
s.cnot(0, 1)
s.h(0)
x, z = s.measure_many(1, 0)
# Bob corrections.
if x: s.x(9)
if z: s.z(9)
# Check received correctly by uncomputing preparation.
s.s_dag(9)
s.h(9)
should_be_false = s.measure(9)
print(f"Alice got {x}, {z}. Bob uncomputed to {should_be_false}")
assert should_be_false == False
```

A.4 (Python) Algebraic manipulations

```
import stim

t = stim.Tableau.random(10)
t_inverse = t**-1
assert t * t_inverse == stim.Tableau(10)
assert (t**1000000) * (t_inverse**500000)**2 == stim.Tableau(10)

x5: stim.PauliString = t.x_output(5)
z6: stim.PauliString = t.z_output(6)
assert t_inverse(x5 * z6) == stim.PauliString("+_____XZ___")

s = stim.TableauSimulator()
s.set_inverse_tableau(t)
stabilizers = s.canonical_stabilizers()
assert stabilizers[0].commutes(stabilizers[1])
```

arXiv:2303.15933v2 [quant-ph] 14 Jan 2025

Sparse Blossom: correcting a million errors per core second with minimum-weight matching

Oscar Higgott[1,2] and Craig Gidney[1]

[1]Google Quantum AI, Santa Barbara, California 93117, USA
[2]Department of Physics & Astronomy, University College London, WC1E 6BT London, United Kingdom

January 14, 2025

In this work, we introduce a fast implementation of the minimum-weight perfect matching (MWPM) decoder, the most widely used decoder for several important families of quantum error correcting codes, including surface codes. Our algorithm, which we call sparse blossom, is a variant of the blossom algorithm which directly solves the decoding problem relevant to quantum error correction. Sparse blossom avoids the need for all-to-all Dijkstra searches, common amongst MWPM decoder implementations. For 0.1% circuit-level depolarising noise, sparse blossom processes syndrome data in both X and Z bases of distance-17 surface code circuits in less than one microsecond per round of syndrome extraction on a single core, which matches the rate at which syndrome data is generated by superconducting quantum computers. Our implementation is open-source, and has been released in version 2 of the PyMatching library.

The source code for our implementation of sparse blossom in PyMatching version 2 can be found on github at `https://github.com/oscarhiggott/PyMatching`. *PyMatching is also available as a Python 3 pypi package installed via "*`pip install pymatching`*".*

1 Introduction

A surface code superconducting quantum computer with a million physical qubits will generate measurement data at a rate of around 1 terabit per second. This data must be processed at least as fast as it is generated by a *decoder*, the classical software used to predict which errors may have occurred, to prevent a backlog of data that grows exponentially in the T-gate depth of the computation [Ter15]. Moreover, fast decoders are important not only for operating a quantum computer, but also as a software tool for researching quantum error correction protocols. Estimating the resource requirements of a quantum error correction protocol (e.g. estimating the teraquop footprint below threshold [Gid+21]) can require the use of direct Monte Carlo sampling to estimate the probability of extremely rare events. These analyses can be prohibitively expensive without access to a fast decoder, capable of processing millions of shots from circuits containing $\approx 10^5$ gates in a reasonable time-frame.

Many decoders have been developed for the surface code, the oldest and most popular of which is the minimum-weight perfect matching (MWPM) decoder [Den+02], which is also the focus of this work. The MWPM decoder maps the decoding problem onto a graphical problem by decomposing the error model into X-type and Z-type Pauli errors [Den+02]. This graphical problem can then be solved with the help of Edmonds' blossom algorithm for finding a minimum-weight perfect matching in a graph [Edm65b; Edm65a]. A naive implementation of the MWPM decoder has a worst-case complexity in the number of nodes N in the graph of $O(N^3 \log(N))$, with the expected running time for typical instances found empirically to be roughly $O(N^2)$ [Hig22]. Approximations and optimisations of the MWPM decoder have led to significantly improved expected running times [FWH12a; Fow15; Hig22]. In particular, Fowler proposed a MWPM decoder with average

$O(1)$ parallel running time [Fow15]. However, published implementations of MWPM decoders have not demonstrated speeds fast enough for real-time decoding at scale.

There have been several alternatives to the MWPM decoder proposed in the literature. The Union-Find decoder has an almost-linear worst-case running time [DN21; HNB20], and fast hardware implementations have been proposed [Das+22] and implemented [Liy+23]. The Union-Find decoder is slightly less accurate than, and can be seen as an approximation of, the MWPM decoder [WLZ22]. Maximum-likelihood decoders can achieve a higher accuracy than the MWPM decoder [BSV14; Hig+23; Sun+23] but have high computational complexity, rendering them impractical for real-time decoding. Other decoders, such as correlated MWPM [Fow13], belief-matching [Hig+23] and neural network [TM17; MPT22] decoders can achieve higher accuracy than MWPM with a much more modest increase in running time. While there has been progress in the development of open-source software packages for decoding surface codes [Hig22; Tuc20], these tools are much slower than stabilizer circuit simulators [Gid21], and have therefore been a bottleneck in surface code simulations. This is perhaps one of the reasons why numerical studies of error correcting codes have often focused on estimating *thresholds* (which require decoding fewer shots), instead of resource overheads (which are more practically useful for making comparisons).

In this work, we introduce a new implementation of the MWPM decoder. The algorithm we introduce, sparse blossom, is a variant of the blossom algorithm which is conceptually similar to the approach taken in Refs. [FWH12a; FWH12b; Fow15], in that it solves the MWPM decoding problem directly on the detector graph, rather than naively breaking up the problem into multiple sequential steps and solving the traditional MWPM graph theory problem as a separate subroutine. This avoids the all-to-all Dijkstra searches often used in implementations of the MWPM decoder. Our implementation is orders of magnitude faster than alternative available tools, and can decode both X and Z bases of a distance-17 surface code circuit (for 0.1% circuit-noise) in under one microsecond per round on a single core, matching the rate at which syndrome data is generated on a superconducting quantum processor. At distance 29 with the same noise model (more than sufficient to achieve 10^{-12} logical error rates), PyMatching takes 3.5 microseconds per round to decode on a single core. These results suggest that our sparse blossom algorithm is fast enough for real-time decoding of superconducting quantum computers at scale; a real-time implementation is likely achievable through parallelisation across multiple cores, and by adding support for decoding a stream, rather than a batch, of syndrome data. Our implementation of sparse blossom has been released in version 2 of the PyMatching Python package, and can be combined with Stim [Gid21] to run simulations in minutes on a laptop that previously would have taken hours on a high-performance computing cluster.

In impressive parallel independent work, Yue Wu has also developed a new implementation of the blossom algorithm called fusion blossom [WZ23], available at [Wu22]. The conceptual similarity with our approach is that fusion blossom also solves the MWPM decoding problem directly on the detector graph. However, there are many differences in the details of our respective implementations; for example, fusion blossom explores the graph in a similar way to how clusters are grown in union-find, whereas our approach grows exploratory regions uniformly, managed by a global priority queue. While our approach has faster single-core performance, fusion blossom also supports parallel execution of the algorithm itself, which can be used to achieve faster processing speeds for individual decoding instances. When used for error correction simulations, we note that sparse blossom is already trivially parallelisable by splitting the simulation into batches of shots, and processing each batch on a separate core. However, parallelisation of the decoder itself is important for real-time decoding, to prevent an exponentially increasing backlog of data building up within a single computation [Ter15], or to avoid the polynomial slowdown imposed by relying on parallel window decoding instead [Sko+23; Tan+23]. Therefore, future work could explore combining sparse blossom with the techniques for parallelisation introduced in fusion blossom.

The paper is structured as follows. In Section 2 we give background on the decoding problem we are interested in and give an overview of the blossom algorithm. In Section 3 we explain our algorithm, sparse blossom, before describing the data structures we use for our implementation in Section 4. In Section 5 we analyse the running time of sparse blossom, and in Section 6 we benchmark its decoding time, before concluding in Section 7.

2 Preliminaries

2.1 Detectors and observables

A *detector* is a parity of measurement outcome bits in a quantum error correction circuit that is deterministic in the absence of errors. The outcome of a detector measurement is 1 if the observed parity differs from the expected parity for a noiseless circuit, and is 0 otherwise. We say that a Pauli error P *flips* detector D if including P in the circuit changes the outcome of D, and a *detection event* is a detector with outcome 1. We define a *logical observable* to be a linear combination of measurement bits, whose outcome instead corresponds to the measurement of a logical Pauli operator.

We define an independent error model to be a set of m independent error mechanisms, where error mechanism i occurs with probability $\mathbf{p}[i]$ (where $\mathbf{p} \in \mathbb{R}^m$ is a vector of priors), and flips a set of detectors and observables. An independent error model can be used to represent circuit-level depolarising noise exactly, and is a good approximation of many commonly considered error models [Cha+20; Gid20]. It can be useful to describe an error model in an error correction circuit using two binary parity check matrices: a *detector check matrix* $H \in \mathbb{F}_2^{n \times m}$ and a *logical observable matrix* $L \in \mathbb{F}_2^{n_l \times m}$. We set $H[i,j] = 1$ if detector i is flipped by error mechanism j, and $H[i,j] = 0$ otherwise. Likewise, we set $L[i,j] = 1$ if logical observable i is flipped by error mechanism j, and $L[i,j] = 0$ otherwise. By describing the error model this way, each error mechanism is defined by which detectors and observables it flips, rather than by its Pauli type and location in the circuit. From a stabilizer circuit and Pauli noise model, we can construct H and L efficiently by propagating Pauli errors through the circuit to see which detectors and observables they flip. Each prior $\mathbf{p}[i]$ is then computed by summing over the probabilities of all the Pauli errors that flip the same set of detectors or observables (or more precisely, these equivalent error mechanisms are independent, and we compute the probability that an odd number occurred). This is essentially what the error analysis tools do in Stim, where a detector error model (automatically constructed from a Stim circuit) captures the information contained in H, L and $\mathbf{p}$ [Gid21].

We can represent an error (a set of error mechanisms) by a vector $\mathbf{e} \in \mathbb{F}_2^m$. The *syndrome* of $\mathbf{e}$ is the outcome of detector measurements, given by $\mathbf{s} = H\mathbf{e}$. The detection events are the detectors corresponding to the non-zero elements of $\mathbf{s}$. An undetectable logical error is an error in $B \coloneqq \{\mathbf{e} \in \mathbb{F}_2^m \mid \mathbf{e} \in \ker H, \mathbf{e} \notin \ker L\}$, and the distance of the circuit is $d = \min_{\mathbf{e} \in B} |\mathbf{e}|$, where $|\mathbf{e}|$ is the Hamming weight of $\mathbf{e}$. Given a syndrome $\mathbf{s} = H\mathbf{e}$ of some error $\mathbf{e} \in \mathbb{F}_2^m$, as well as knowledge of H, L and $\mathbf{p}$, a decoder makes a prediction $\mathbf{c} \in \mathbb{F}_2^m$ of an error satisfying $H\mathbf{c} = H\mathbf{e}$ and succeeds if $L(\mathbf{c} \oplus \mathbf{e}) = 0$. Note that sometimes a decoder need only output the logical prediction $L\mathbf{c}$ (succeeding if it matches $L\mathbf{e}$), as we discuss later.

For a given circuit, the choice of detectors is not unique. For example, if we redefine any detector D_i as $D_i \coloneqq D_i \oplus D_j$ then we still have a valid choice of detectors, and this change corresponds to adding row j to row i of H. Since two errors $\mathbf{e}_k, \mathbf{e}_l \in \mathbb{F}_2^m$ are distinguishable if and only if $\mathbf{e}_k \oplus \mathbf{e}_l \notin \ker H$, two detector check matrices can distinguish the same errors if they have the same kernel [Der+24]. While row operations on H do not affect which errors can be distinguished, it is often necessary that the choice of basis for the detectors has a particular structure in order for a given decoding algorithm to be applicable (as discussed shortly in Section 2.2).

Similarly, the choice of logical observables is also not unique. Since detector outcomes are deterministically zero in the absence of noise, we can add any linear combination of detectors to a logical observable without effecting its expectation value for a noiseless circuit. In terms of check matrices, this change corresponds to defining some new logical observables matrix L' where we have added some linear combinations of the rows of H to each row of the original logical observables matrix L:

$$L' \coloneqq L \oplus AH. \tag{1}$$

Here $A \in \mathbb{F}_2^{n_l \times n}$ is an arbitrary binary matrix of dimensions $n_l \times n$. If we use a decoder that is guaranteed to provide a prediction consistent with the syndrome, i.e. $H(\mathbf{c} \oplus \mathbf{e}) = 0$, then adding detectors to each observable will not change whether or not the decoder succeeds, since $L'(\mathbf{c} \oplus \mathbf{e}) = L(\mathbf{c} \oplus \mathbf{e})$.

In Appendix D, we show how the detectors, observables and the matrices H and L are defined for a small example of a distance 2 repetition code circuit.

2.2 Detector graphs

In this work, we will restrict our attention to *graphlike error models*, defined to be independent error models for which each error mechanism flips at most two detectors (H has column weight at most two). Graphlike error models can be used to approximate common noise models for many important classes of quantum error correction codes including surface codes [Den+02], for which X-type and Z-type Pauli errors are both graphlike. Many related code families also have graphlike error models, such as repetition codes, 2D hyperbolic codes [BT16], some 2D subsystem codes [Bra+13; SBT11; Bac06; HB21] and Floquet codes [HH21; Gid+21], amongst others. Color codes [BM06] can be decoded by using a mapping to a graphlike error model as a subroutine [KD23; GJ23], and generalizing this approach to decode more general codes is an active area of research [Bro22].

We can represent a graphlike error model using a detector graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, also called a matching graph or decoding graph in the literature. Each node $v \in \mathcal{V}$ corresponds to a detector (a detector node, a row of H). Each edge $e \in \mathcal{E}$ is a set of detector nodes of cardinality one or two representing an error mechanism that flips this set of detectors (a column of H). We can decompose the edge set as $\mathcal{E} = \mathcal{E}_1 \cup \mathcal{E}_2$ where $\forall e \in \mathcal{E}_1 : |e| = 1$ and $\forall e \in \mathcal{E}_2 : |e| = 2$. A regular edge $(u, v) \in \mathcal{E}_2$ flips a pair of detectors $u, v \in \mathcal{V}$, whereas a *half-edge* $(u,) \in \mathcal{E}_1$ flips a single detector $u \in \mathcal{V}$. For a half-edge $(u,) \in \mathcal{E}_1$ we sometimes say that u is connected to the boundary and use the notation (u, v_b), where v_b is a virtual boundary node (which does not correspond to any detector). Therefore, when we refer to an edge $(u, v) \in \mathcal{E}$ it is assumed that u is a node and v is either a node or the boundary node v_b. Each edge $e_i \in \mathcal{E}$ is assigned a weight $w(e_i) = \log((1 - \mathbf{p}[i])/\mathbf{p}[i])$, and recall that $\mathbf{p}[i]$ is the probability that error mechanism i occurs. We also define an edge weights vector $\mathbf{w} \in \mathbb{R}^{|\mathcal{E}|}$ for which $\mathbf{w}[i] = w(e_i)$. We also label each edge $e_i = (u, v) \in \mathcal{E}$ with the set of logical observables that are flipped by the error mechanism, which we denote either by $l(e_i)$ or $l(u, v)$. We use $x \oplus y$ to denote the symmetric difference of sets x and y. For example, $l(e_1) \oplus l(e_2)$ is the set of logical observables flipped when the error mechanisms 1 and 2 are both flipped. We define the distance $D(u, v)$ between two nodes u and v in the detector graph to be the length of the shortest path between them. We give an example of a detector graph $\mathcal{G}$ for a repetition code circuit in Appendix D.

2.3 The minimum-weight perfect matching decoder

From now on, we will assume that we have an independent graphlike error model with check matrix H, logical observables matrix L, priors vector $\mathbf{p}$, as well as the corresponding detector graph $\mathcal{G}$ with edge weights vector $\mathbf{w}$. Given some error $\mathbf{e} \in \mathbb{F}_2^m$ sampled from the graphlike error model, with the observed syndrome $\mathbf{s} = H\mathbf{e}$, the minimum-weight perfect matching (MWPM) decoder finds the *most probable physical error* consistent with the syndrome. In other words, for a graphlike error model it finds a physical error $\mathbf{c} \in \mathbb{F}_2^m$ satisfying $H\mathbf{c} = \mathbf{s}$ that has maximum prior probability $P(\mathbf{c})$. For our error model the prior probability $P(\mathbf{e})$ of an error $\mathbf{e}$ is

$$P(\mathbf{e}) = \prod_i (1 - \mathbf{p}[i])^{(1-\mathbf{e}[i])} \mathbf{p}[i]^{\mathbf{e}[i]} = \prod_i (1 - \mathbf{p}[i]) \prod_i \left(\frac{\mathbf{p}[i]}{1 - \mathbf{p}[i]} \right)^{\mathbf{e}[i]}. \tag{2}$$

Equivalently we seek to maximise $\log(P(\mathbf{c})) = C - \sum_i \mathbf{w}[i]\mathbf{c}[i]$, where here $C \coloneqq \sum_i \log(1 - \mathbf{p}[i])$ is a constant and we recall that the edge weight is defined as $\mathbf{w}[i] \coloneqq \log((1 - \mathbf{p}[i])/\mathbf{p}[i])$. This corresponds to finding an error $\mathbf{c} \in \mathbb{F}_2^m$ satisfying $H\mathbf{c} = \mathbf{s}$ that minimises $\sum_i \mathbf{w}[i]\mathbf{c}[i]$, the sum of the weights of the corresponding edges in $\mathcal{G}$. We note that a maximum likelihood decoder instead finds the most probable *logical* error for any error model (graphlike or not), however maximum likelihood decoding is in general computationally inefficient.

Despite the MWPM decoder's name, the problem it solves, defined above, does not correspond to finding a MWPM in $\mathcal{G}$, but instead corresponds to solving a variation of the MWPM problem, which we refer to as the minimum weight *embedded* matching (MWEM) problem. Let us first define the traditional MWPM problem for a graph $G = (V, E)$. Here G is a weighted graph, where each edge $(u, v) \in E$ is a pair of nodes $u, v \in V$ and, unlike detector graphs, there are no half-edges. Each edge is assigned a weight $w(e) \in \mathbb{R}$. A perfect matching $M \subseteq E$ is a subset of edges such that each node $u \in V$ is incident to exactly one edge $(u, v) \in M$. For each $(u, v) \in M$ we say that u is matched to v, and vice versa. A MWPM is a perfect matching that has minimum weight

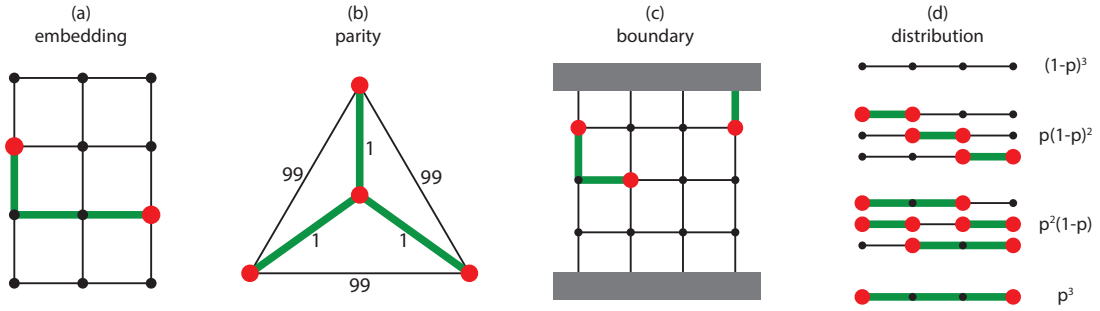


Figure 1: Key differences between the quantum decoding problem solved by PyMatching and the minimum weight perfect matching problem. In the usual MWPM problem, all nodes must be matched and they are matched using a disjoint set of edges. In the decoding problem, (a) only a subset of nodes is excited, only these nodes need to be matched, and (b) the edge set used to match them is not required to be disjoint. The excited nodes are matched by finding an edge set where excited nodes have an odd number of neighbors in the edge set, non-excited nodes have an even number of neighbors in the edge set, and (c) there may be boundary nodes that can have any number of neighbors in the edge set. (d) The expected distribution of excited nodes is not uniform. It is generated by sampling edges, where each edge is independently included with some probability, and then exciting any nodes that have an odd number of neighbors in the sampled edge set. This results in it being exponentially unlikely to see large distances between excited nodes at low error rates, which has major implications on the expected runtime of the algorithm (see Section 5).

$\sum_{e\in M} w(e)$. Clearly, not every graph has a perfect matching (a simple necessary condition is that $|V|$ must be even; a necessary *and sufficient* condition is provided by Tutte's theorem [Tut47]), and a graph may have more than one perfect matching of minimum weight.

Given a detector graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with vertex set $\mathcal{V}$, and a set of detection events (highlighted nodes) $\mathcal{D} \subseteq \mathcal{V}$, we define an *embedded* matching of $\mathcal{D}$ in $\mathcal{G}$ to be a set of edges $M_E \subseteq \mathcal{E}$ such that every node in $\mathcal{D}$ is incident to an *odd* number of edges in M_E, and every node in $\mathcal{V} \setminus \mathcal{D}$ is incident to an *even* number of edges in M_E. The minimum-weight embedded matching is an embedded matching that has minimum weight $\sum_{e\in M_E} w(e)$. We note that the minimum-weight embedded matching problem for a standard graph (containing no half-edges) is known as the minimum-weight T-join problem in the field of combinatorial optimisation (for $T \equiv \mathcal{D}$) [EJ73; KV18]. The key differences between the minimum-weight embedded matching problem we are interested in for error correction, and the traditional MWPM problem, are shown in Figure 1.

The decoder does not need to output the set of edges directly, but rather outputs $L\mathbf{c}$, a prediction of which logical observable measurements were flipped. Our decoder has succeeded if it correctly predicts which observables were flipped, i.e. if $L\mathbf{c} = L\mathbf{e}$. In other words, we apply a correction at the logical level rather than at the physical level, which is equivalent since $L(\mathbf{c} \oplus \mathbf{e}) = L\mathbf{c} \oplus L\mathbf{e}$. This output is generally much more sparse: for example, in a surface code memory experiment the prediction is simply a single bit, predicting whether or not the logical X (or Z) observable measurement outcome was flipped by the error. As we will see later, predicting logical observables $L\mathbf{c}$ rather than the full physical error $\mathbf{c}$ leads to some useful optimizations. Note that PyMatching does also support returning the full physical error (e.g. a unique observable can be assigned to each edge), but we apply these additional optimizations when the number of logical observables is small (up to 64 observables).

Note that the edge weight $\mathbf{w}[i] = \log((1 - \mathbf{p}[i])/\mathbf{p}[i])$ is negative when $\mathbf{p}[i] > 0.5$. Our sparse blossom algorithm instead assumes that edge weights are non-negative. Fortunately, it is straightforward to decode a syndrome $\mathbf{s}$ for a detector graph $\mathcal{G}$ containing negative edge weights by using efficient pre- and post-processing to instead decode a modified syndrome $\mathbf{s}'$ using a set of adjusted edge weights $\mathbf{v}$ containing only non-negative edge weights. This procedure is used to handle negative edge weights in PyMatching and is explained in Appendix C. However, from now on we will assume, without loss of generality, that we have a graph containing only non-negative edge weights.

2.4 Connection between minimum-weight perfect and embedded matching

Although minimum-weight embedded and perfect matchings are different problems, there is a close connection between them. In particular, by using a polynomial-time algorithm for solving the MWPM problem (e.g. the blossom algorithm), we can obtain a polynomial-time algorithm for solving the MWEM problem via a reduction. We will now describe this reduction for the case that the detector graph $\mathcal{G}$ has no boundary, in which case the MWEM problem is equivalent to the minimum-weight T-join problem (see [EJ73; KV18]). This reduction was used by Edmonds and Johnson for their polynomial-time algorithm for solving the Chinese postman problem [EJ73]. The boundary can also be handled with a small modification (e.g. see [Fow15]).

Given a detector graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with non-negative edge weights (and which, for now, we assume has no boundary, i.e. $\mathcal{E} = \mathcal{E}_2$), and given a set of detection events $\mathcal{D} \subseteq \mathcal{V}$, we define the *path graph* $\bar{\mathcal{G}}[\mathcal{D}] = (\mathcal{D}, \bar{\mathcal{E}})$ to be the complete graph on the vertices $\mathcal{D}$ for which each edge $(u, v) \in \bar{\mathcal{E}}$ is assigned a weight equal to the distance $D(u, v)$ between u and v in $\mathcal{G}$. Here the distance $D(u, v)$ is the length of the shortest path between u and v in $\mathcal{G}$, where here the length of the path is the sum of the weights of the edges it contains. In other words, the path graph $\bar{\mathcal{G}}[\mathcal{D}]$ is the subgraph of the metric closure of $\mathcal{G}$ induced by the vertices $\mathcal{D}$. A MWEM M of $\mathcal{D}$ in $\mathcal{G}$ can be found efficiently using the following three steps:

1. Construct the path graph $\bar{\mathcal{G}}[\mathcal{D}]$ using Dijkstra's algorithm.
2. Find the minimum-weight perfect matching $\bar{M} \subseteq \bar{\mathcal{E}}$ in $\bar{\mathcal{G}}[\mathcal{D}]$ using the blossom algorithm.
3. Use $\bar{M}$ and Dijkstra's algorithm to construct the MWEM: $M \coloneqq \bigoplus_{(u,v)\in\bar{M}} P^{\min}_{u,v}$.

where here $P^{\min}_{u,v} \subseteq \mathcal{E}$ is a minimum-length path between u and v in $\mathcal{G}$. See Theorem 12.10 of [KV18] for a proof of this reduction, where their minimum-weight T-join is our MWEM, and their set T corresponds to our $\mathcal{D}$. See also [Bar82; Ber+99] for alternative reductions and [BDL22] for a recent review.

Unfortunately, solving these three steps sequentially is quite computationally expensive; for example, just the cost of enumerating the edges in $\bar{\mathcal{G}}[\mathcal{D}]$ scales quadratically in the number of detection events $|\mathcal{D}|$, whereas we would ideally like a decoder with an expected running time that scales linearly in $|\mathcal{D}|$. This sequential approach has nevertheless been widely used by QEC researchers, despite its performance being very far from optimal.

A significant improvement was introduced by Fowler [Fow15]. A key observation made by Fowler was that, for QEC problems, typically only low-weight edges in $\bar{\mathcal{G}}[\mathcal{D}]$ are actually used by blossom. Fowler's approach exploited this fact by setting an initial exploration radius in the detector graph, within which separate searches were used to construct some of the edges in $\bar{\mathcal{G}}[\mathcal{D}]$. This exploration radius was then adaptively increased as required by the blossom algorithm. Our approach, sparse blossom, is inspired by Fowler's work but is different in many of the details. Before introducing sparse blossom, we will next give an overview of the standard blossom algorithm.

2.5 The blossom algorithm

The blossom algorithm, introduced by Jack Edmonds [Edm65b; Edm65a], is a polynomial-time algorithm for finding a minimum-weight perfect matching in a graph. In this section we will outline some of the key concepts in the original blossom algorithm. We will not explain the original blossom algorithm in full, since there is significant overlap with our sparse blossom algorithm, which we describe in Section 3. While this section provides motivation for sparse blossom, it is not a prerequisite for the rest of the paper, so the reader may wish to skip straight to Section 3. We refer the reader to references [Edm65b; Edm65a; Gal83; Kol09] for a more complete overview of the blossom algorithm.

We will first introduce some terminology. Given some matching $M \subseteq E$ in a graph $G = (V, E)$, we say that an edge in E is *matched* if it is also in M, and unmatched otherwise, and a node is matched if it is incident to a matched edge, and unmatched otherwise. A maximum cardinality matching is a matching that contains as many edges as possible. An *augmenting path* is a path $P \subseteq E$ which alternates between matched and unmatched edges, and begins and terminates at two distinct unmatched nodes.

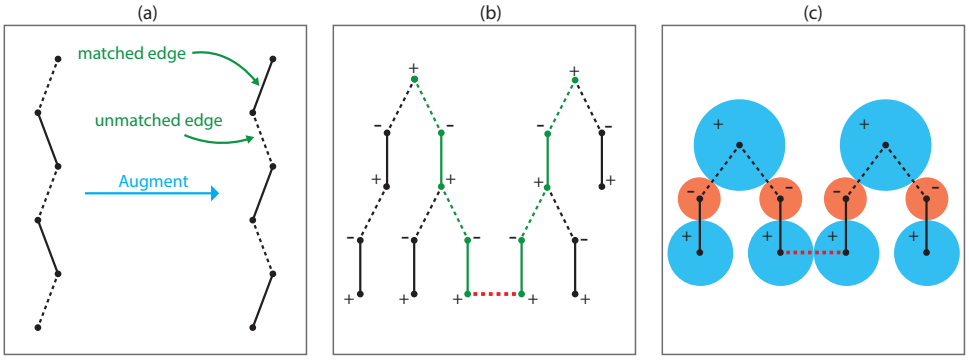


Figure 2: (a) Augmenting an augmenting path. Matched edges become unmatched, and unmatched edges become matched. (b) Examples of two alternating trees in the blossom algorithm for finding a maximum matching. Each tree has one unmatched node. The two trees have become connected via the red dashed edge. The path between the roots of the two trees, through the green edges and red edge, is an augmenting path. (c) An example of two alternating trees in the blossom algorithm for finding a minimum-weight perfect matching. Each node v now has a dual variable y_v which, when y_v is positive, we can interpret as the radius of a region centred on the node. A new edge (u, v) with weight $w_{u,v}$ can only be explored by the alternating tree if it is *tight*, meaning that the dual variables y_u and y_v satisfy $y_u + y_v = w_{u,v}$.

Given an augmenting path P in G, we can always increase the cardinality of the matching M by one by replacing M with the new matching $M' = M \oplus P$. We refer to this process, of adding each unmatched edge in P to M and removing each matched edge in P from M, as *augmenting* the augmenting path P (see Figure 2(a)). Berge's theorem states that a matching has maximum cardinality if and only if there is no augmenting path [Ber57].

2.5.1 Solving the maximum cardinality matching problem

We will now give an overview of the original *unweighted* version the blossom algorithm, which finds a maximum cardinality matching (as introduced by Edmonds in [Edm65b]). The unweighted blossom algorithm is used as a subroutine by the more general blossom algorithm for finding a minimum-weight perfect matching (discovered, also by Edmonds, in [Edm65a]), which we will outline in Section 2.5.2. The algorithm is motivated by Berge's theorem. Starting with a trivial matching, it proceeds by finding an augmenting path, augmenting the path, and then repeating this process until no augmenting path can be found, at which point we know that the matching is maximum. Augmenting paths are found by constructing *alternating trees* within the graph. An alternating tree T in the graph G is a tree subgraph of G with an unmatched node as its root, and for which every path from root to leaf alternates between unmatched and matched edges, see Figure 2(b). There are two types of nodes in T: "outer" nodes (labelled "+") and "inner" nodes (labelled "−"). Each inner node is separated from the root node by an odd-length path, whereas each outer node is separated by an even-length path. Each inner node has a single child (an outer node). Each outer node can have any number of children (all inner nodes). All leaf nodes are outer nodes.

Initially, every unmatched node is a trivial alternating tree (a root node). To find an augmenting path, the algorithm searches the neighboring nodes in G of the outer nodes in each tree T. If, during this search, an edge (u, v) is found such that u is an outer node of T and v is an outer node of some other tree $T' \neq T$ then an augmenting path has been found, which connects the roots of T and T', see Figure 2(b). This path is augmented, the two trees are removed, and the search continues. If an edge (u, v) is found between an outer node u in T and a matched node v not in any tree (i.e. v is matched), then v and its match are added to T. Finally, if an edge (u, v) is found between two outer nodes of the same tree then an odd-length cycle has been found, and forms a *blossom*. A key insight of Edmonds was that a blossom can be treated as a virtual node, which can be matched or belong to an alternating tree like any other node. However, we will explain how blossoms are handled in more detail in the context of our sparse blossom algorithm in Section 3.

2.5.2 Solving the minimum-weight perfect matching problem

The extension from finding a maximum cardinality matching to finding a minimum-weight perfect matching is motivated by formulating the problem as a linear program [Edm65a]. Constraints are added on how the alternating trees are allowed to grow, and these constraints ensure that the weight of the perfect matching is minimal once it has been found. The formulation of the problem as a linear program is not required either to understand the algorithm, or for the proof of correctness. However, it does provide useful motivation, and the constraints and definitions used in the linear program are also used in the blossom algorithm itself. We will therefore describe the linear program here for completeness.

We will denote the boundary edges of some subset of the nodes $S \subseteq V$ by $\delta(S) \coloneqq \{(u,v) \in E \mid u \in S, v \in V \setminus S\}$, and will let $\mathcal{O}$ be the set of all subsets of V of odd cardinality at least three, i.e. $\mathcal{O} \coloneqq \{o \subseteq V : |o| > 1, |o| \mod 2 = 1\}$. We denote the edges incident to a single node v by $\delta(v)$ (i.e. $\delta(v) = \delta(\{v\})$). We will use an incidence vector $\mathbf{x} \in \{0,1\}^{|E|}$ to represent a matching $M \subseteq E$ where $x_e = 1$ if $e \in M$ and $x_e = 0$ if $e \notin M$. We denote the weight of an edge $e \in E$ by w_e. The minimum-weight perfect matching problem can then be formulated as the following *integer* program:

$$\begin{aligned} \text{Minimise} \quad & \sum_{e \in E} w_e x_e && \text{(3a)} \\ \text{subject to} \quad & \sum_{e \in \delta(v)} x_e = 1 \quad \forall v \in V && \text{(3b)} \\ & x_e \in \{0,1\} \quad \forall e \in E && \text{(3c)} \end{aligned}$$

Edmonds introduced the following linear programming relaxation of the above integer program:

$$\begin{aligned} \text{Minimise} \quad & \sum_{e \in E} w_e x_e && \text{(4a)} \\ \text{subject to} \quad & \sum_{e \in \delta(v)} x_e = 1 \quad \forall v \in V && \text{(4b)} \\ & \sum_{e \in \delta(S)} x_e \geq 1 \quad \forall S \in \mathcal{O} && \text{(4c)} \\ & x_e \geq 0 \quad \forall e \in E. && \text{(4d)} \end{aligned}$$

Note that the constraints in Equation 4c are satisfied by *any* perfect matching, but Edmonds showed that adding them ensures that the linear program has an integral optimal solution. In other words, the integrality constraint (Equation 3c) can be replaced by the inequalities in Equation 4c and Equation 4d.

Every linear program (referred to as the *primal* linear program, or primal problem) has a *dual* linear program (or dual problem). The dual of the above primal problem is:

$$\begin{aligned} \text{Maximise} \quad & \sum_{v \in V} y_v + \sum_{S \in \mathcal{O}} y_S && \text{(5a)} \\ \text{subject to} \quad & \mathit{slack}(e) \geq 0 \quad \forall e \in E && \text{(5b)} \\ & y_S \geq 0 \quad \forall S \in \mathcal{O} && \text{(5c)} \end{aligned}$$

where the *slack* of an edge is defined as

$$\mathit{slack}(e) \coloneqq w_e - \sum_{u \in e} y_u - \sum_{S \in \mathcal{O} : e \in \delta(S)} y_S. \tag{6}$$

We say that an edge is *tight* if it has zero slack. Here we have defined a dual variable $y_v \in \mathbb{R}$ for each node $v \in V$, as well as a dual variable $y_S \in \mathbb{R}$ for each set $S \in \mathcal{O}$. While each variable y_S is constrained to be non-negative (Equation 5c), each y_v is permitted to take any value. Although we have an exponential number of y_S variables, this turns out not to be an issue since only $O(|V|)$ are non-zero at any given stage of the blossom algorithm.

We now recall some terminology and general properties of linear programs (see [MG07; KV18] for more details). A solution of a linear program is *feasible* if it satisfies the constraints of the linear program. Without loss of generality, we assume that the primal linear program is a minimisation problem (in which case its dual is a maximisation problem). By the *strong duality theorem*, if both the primal and the dual linear program have a feasible solution, then they both also have an *optimal* solution. Furthermore, the minimum of the primal problem is equal to the maximum of its dual, providing a "numerical" proof of optimality.

We can obtain a "combinatorial" proof of optimality for any linear program using the *complementary slackness* conditions. Each constraint in the primal problem is associated with a variable of the dual problem (and vice versa). Let us associate the ith primal constraint with the ith dual variable (and vice versa). The complementary slackness conditions state that, if and only if we have a pair of optimal solutions, then if the ith dual variable is greater than zero then the ith primal constraint is satisfied with equality. Similarly, if the ith primal variable is greater than zero then the ith dual constraint is satisfied with equality. More concretely, for the specific primal-dual pair of linear programs we are considering, the complementary slackness conditions are:

$$slack(e) > 0 \implies x_e = 0 \tag{7}$$

$$y_S > 0 \implies \sum_{e \in \delta(S)} x_e = 1 \qquad (S \in \mathcal{O}) \tag{8}$$

These conditions are used as a stopping rule in the blossom algorithm (with Equation 7 satisfied throughout) and provide a proof of optimality.

While it is convenient to use the strong duality theorem, since it applies to *any* linear program, its correctness is not immediately intuitive and its proof is quite involved (see [MG07; KV18]). Fortunately, we can obtain a simple proof of optimality of the minimum-weight perfect matching problem directly, without the need for the strong duality theorem [Gal83]. First, we note that for any feasible dual solution, we have that *any* perfect matching N satisfies

$$\sum_{e \in N} w_e = \sum_{e \in N} \left(slack(e) + \sum_{v \in e} y_v + \sum_{S \in \mathcal{O} : e \in \delta(S)} y_S \right) \geq \sum_{v \in V} y_v + \sum_{S \in \mathcal{O}} y_S, \tag{9}$$

where here the equality is from the definition of $slack(e)$ and the inequality uses Equation 5b and Equation 5c (i.e. the fact that the dual solution is feasible) and the fact that N is a perfect matching. However, if we have a perfect matching M which additionally satisfies Equation 7 and Equation 8 we instead have

$$\sum_{e \in M} w_e = \sum_{v \in V} y_v + \sum_{S \in \mathcal{O}} y_S, \tag{10}$$

and thus the perfect matching M has minimal weight.

So far in this section, we have only considered the case that each edge is a *pair* of nodes (a set of cardinality two). Let us now consider the more general case (required for decoding) where we can also have half-edges. More specifically, we now have the edge set $E = E_1 \cup E_2$ where each $(u, v) \in E_2$ is a regular edge and each $(u,) \in E_1$ is a half-edge (a node set of cardinality one). We note that a perfect matching is now defined as a subset of this more general edge set, but its definition is otherwise unchanged (a perfect matching is an edge set $M \subseteq E \coloneqq E_1 \cup E_2$ such that each node is incident to exactly one edge in M). We extend our definition of $\delta(S)$ to be

$$\delta(S) \coloneqq \{(u, v) \in E_2 \mid u \in S, v \in V \setminus S\} \cup \{(u,) \in E_1 \mid u \in S\}. \tag{11}$$

With this modification, the simple proof of correctness above still holds and the $slack(e)$ of a half-edge $e \in E_1$ is well defined by Equation 6.

The blossom algorithm for finding a minimum-weight perfect matching starts with an empty matching and a feasible dual solution, and iteratively increases the cardinality of the matching and the value of the dual objective while ensuring the dual problem constraints remain satisfied. Eventually, we will have a pair of feasible solutions to the primal and dual problem satisfying the complementary slackness conditions (Equation 7 and Equation 8) at which point we know we have a perfect matching of minimal weight. The algorithm proceeds in stages, where each stage consists

of a "primal update" and a "dual update". We repeat these primal and dual updates until no more progress can be made at which point, provided the graph admits a perfect matching, the complementary slackness conditions will be satisfied and so the minimum-weight perfect matching has been found. We will now outline the primal and dual update in more detail.

In the *primal update*, we consider only the subgraph H of G consisting of *tight* edges and try to find a matching of higher cardinality, essentially by running a slight modification to the unweighted blossom algorithm on this subgraph. In [Kol09], the four allowed operations in the primal update are referred to as "GROW", "AUGMENT", "SHRINK" and "EXPAND". The first three of these already occur in the unweighted variant of blossom discussed in Section 2.5.1. The GROW operation consists of adding a matched pair of nodes to an alternating tree. AUGMENT is the process of augmenting the path between the roots of two trees when they become connected. SHRINK is the name for the process of forming a blossom when an odd length cycle is found. The operation that differs slightly in the weighted variant is EXPAND. This EXPAND operation can occur whenever the dual variable y_S for a blossom S becomes zero; when this happens the blossom is removed, the odd-length path through the blossom is added into the alternating tree, and nodes in the even-length path become matched to their neighbours. This differs slightly from the unweighted variant as we described it, where blossoms are only expanded when a path they belong to becomes augmented (at which point *all* the nodes in a blossom cycle become matched). We refer the reader to [Kol09] for a more complete description of these operations in the primal update (and associated diagrams), although we reiterate that very similar concepts will be covered in more detail when we describe sparse blossom in Section 3.

In the *dual update*, we try to increase the dual objective by updating the value of the dual variables, ensuring that edges in alternating trees and blossoms remain tight, and also ensuring that the dual variables remain a feasible solution to the dual problem (the inequalities Equation 5b and Equation 5c must remain satisfied). Loosely speaking, the goal of the dual update is to increase the dual objective in such a way that more edges become tight, while ensuring existing alternating trees, blossoms and matched edges remain intact. The only dual variables we update are those belonging to nodes in an alternating tree. For each alternating tree T we choose a dual change $\delta_T \geq 0$ and we *increase* the dual variable of every outer node u with $y_u \coloneqq y_u + \delta_T$ but *decrease* the dual variable of every inner node u with $y_u \coloneqq y_u - \delta_T$. Recall that each node in T is either a regular node or a blossom, and if the node is a blossom then we are changing the *blossom*'s dual variable (while leaving the dual variables of the nodes it contains unchanged). Note that this change ensures that all tight edges within a given alternating tree remain tight, but since outer node dual variables are increasing, it is possible that some of their neighbouring (non-tight) edges may *become* tight (hopefully allowing us to find an augmenting path between alternating trees in the next primal update). The constraints of the dual problem (the inequalities Equation 5b and Equation 5c) impose constraints on the choice of δ_T; in particular, the slacks of all edges must remain non-negative, and *blossom* dual variables must also remain non-negative.

There are many different valid strategies that can be taken for the dual update. In a *single tree* approach, we pick a single tree T and update the dual variables only of the nodes in T by the maximum amount δ_T such that the constraints of the dual problem remain satisfied (e.g. we change the dual variables until an edge becomes tight or a blossom dual variable becomes zero). In a *multiple tree fixed* δ approach, we update the dual variables of *all* alternating trees by the same amount δ_T (again by the maximum amount that ensures the dual constraints remain satisfied). In a *multiple tree variable* δ approach, we choose a different δ_T for each tree T. Our variant of the blossom algorithm (sparse blossom) uses a multiple tree fixed δ approach. This is a key difference with Refs. [FWH12a; FWH12b; Fow15], which instead use a single tree approach. See [Kol09] for a more detailed discussion and comparison of these different strategies.

In Figure 2(c) we give an example with two alternating trees, and visualise a dual variable as the radius of a circular region centred on its node. Visualising dual variables this way, an edge between two trivial nodes is tight if the regions at its endpoints touch. In this example, we update the dual variables (radiuses) until the two alternating trees touch, at which point the edge joining the two trees becomes tight, and we can augment the path between the roots of the two trees. Note that we can only visualise dual variables as region radiuses like this when they are non-negative. While dual variables of blossoms are always non-negative (as imposed by Equation 5c), dual variables of regular nodes *can* become negative in general. However, when running the blossom algorithm

on a *path graph*, the dual variable of every regular node is also always non-negative, owing to the structure of the graph. This can be understood as follows. Consider any regular inner node v that is not a blossom, which by definition must have exactly one child outer node w in its alternating tree (its match), as well as its one parent outer node u. Recall that the path graph is a complete graph where the weight $w(x, y)$ of each edge (x, y) is the length of shortest path between nodes x and y in some other graph (e.g. in our case always the detector graph). Therefore there is also an edge (u, w) in the path graph with weight $w(u, w) \leq w(u, v) + w(v, w)$, since we know that there is at least one path from u to w of length $w(u, v) + w(v, w)$, corresponding to the union of the shortest path from u to v and the shortest path from v to w. Therefore, we cannot have $y_v < 0$ without having $slack((u, w)) < 0$ which would violate Equation 5b. More specifically, if $y_v = 0$ then we know that the edge (u, w) *must* be tight, which means we can form a new blossom from the blossom cycle (u, v, w) and this blossom can become an *outer* node in the (possibly now trivial) alternating tree.

3 Sparse Blossom

The variant of the blossom algorithm we introduce, which we call sparse blossom, directly solves the minimum-weight embedded matching problem relevant to quantum error correction. Sparse blossom does not have a separate Dijkstra step for constructing edges in the path graph $\bar{\mathcal{G}}[\mathcal{D}]$. Instead, shortest path information is recovered as part of the growth of alternating trees themselves. Put another way, we only discover and store an edge $e \in \bar{\mathcal{E}}$ in $\bar{\mathcal{G}}[\mathcal{D}]$ exactly if and when it is needed by the blossom algorithm; the edges that we track at any point in the algorithm correspond exactly to the subset of *tight* edges in $\bar{\mathcal{E}}$ being used to represent an alternating tree, a blossom or a match. This leads to very large speedups relative to the sequential approach, where all edges in $\bar{\mathcal{E}}$ are found using Dijkstra searches, despite the vast majority never becoming tight edges in the blossom algorithm. We name the algorithm *sparse* blossom, since it exploits the fact that only a small fraction of the detector nodes correspond to detection events for typical QEC problems (and detection events can be paired up locally), and for these problems our approach only ever inspects a small subset of the nodes and edges in the detector graph.

Before explaining sparse blossom and our implementation, we will first introduce and define some concepts.

3.1 Key concepts

3.1.1 Graph fill regions

A *graph fill region* R of radius y_R is an exploratory region of the detector graph. A graph fill region R contains the nodes in the detector graph which are within distance y_R of its *source*. The source of a graph fill region is either a single detection event, or the surface of other graph fill regions forming a *blossom*. We will define blossoms later on, however for the case that the graph fill region R has a single detection event u as its source, every node that is within distance y_R of u is contained in R. Recall that the distance $D(u, v)$ between two nodes u and v is the sum of the weights of edges along the weighted shortest path between them. Note that a graph fill region in sparse blossom is analogous to a node or blossom in the standard blossom algorithm, and the graph fill region's radius is analogous to a node or blossom's dual variable in standard blossom. The sparse blossom algorithm proceeds along a timeline (see Section 3.1.7), and the radius of each graph fill region can have one of three growth rates: +1 (growing), -1 (shrinking) or 0 (frozen). Therefore, at any time t we can represent the radius of a region using an equation $y_R = mt + c$, where $m \in \{-1, 0, 1\}$. We will at times refer to a graph fill region just as a *region* when it is clear from context.

We denote by $\mathcal{D}(R)$ the set of detection events in a region R. When a region contains only a single detection event, $|\mathcal{D}(R)| = 1$, we refer to it as a trivial region. A region can contain multiple detection events if it has a blossom as its source. As well as its radius equation, each graph fill region may also have blossom children and a blossom parent (both defined in Section 3.1.6). It also has a *shell area*, stored as a stack. The shell area of a region is the set of detector nodes it contains, excluding the detector nodes contained in its blossom children. We say that a graph fill

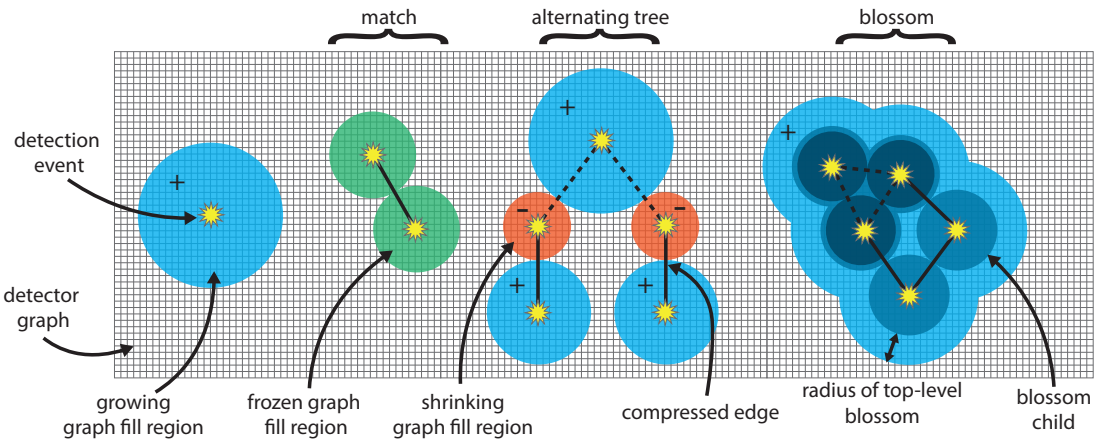


Figure 3: Key concepts in sparse blossom

region is *active* if it does not have a blossom parent. We will let $\mathcal{R}$ denote the set of all graph fill regions.

3.1.2 Compressed edges

A *compressed edge* represents a path through the detector graph between two detection events, or between a detection event and the boundary. Given a path $P_{u,v} \subseteq \mathcal{E}$ between u and v, where u is a detection event and v is either a detection event or denotes the boundary, the compressed edge $\beta(P_{u,v})$ associated with $P_{u,v}$ is the pair of nodes (u, v) at the endpoints of $P_{u,v}$, as well as the set of logical observables $l(P_{u,v}) \coloneqq \bigoplus_{e \in P_{u,v}} l(e)$ flipped by flipping all edges in $P_{u,v}$. The compressed edge $\beta(P_{u,v})$ is therefore a compressed representation of the path $P_{u,v}$ containing all the information relevant for error correction and, for a given detector graph, can be stored using a constant amount of data (independent of the path length). When the choice of path $P_{u,v} \subseteq \mathcal{E}$ for some given pair of detection events (u, v) is clear from context, we may denote the compressed edge $\beta(P_{u,v})$ instead by (u, v), and may also denote the set of logical observables $l(P_{u,v})$ by $l(u, v)$. We define the *length* of a compressed edge to be the distance $D(u, v)$ between its endpoints u and v.

Every compressed edge $\beta(P_{u,v})$ in sparse blossom corresponds to a *shortest* path $P_{u,v}$ between u and v. However, we can use a compressed edge to represent any path between u and v, and when used in union-find (see Appendix A) the path $P_{u,v}$ need not be minimum weight. Compressed edges are used in the representation of several data structures in sparse blossom (alternating trees, matches and blossoms). In particular, each compressed edge (u, v) corresponds to an edge in the path graph $\bar{\mathcal{G}}[\mathcal{D}]$, but unlike in the standard serial approach to implementing the MWPM decoder, we only every discover and construct the small subset of the edges in $\bar{\mathcal{G}}[\mathcal{D}]$ needed by sparse blossom.

We will let Γ be the set of all possible compressed edges that can represent shortest paths between detector nodes (i.e. the set Γ contains a compressed edge (u, v) for each edge in $\bar{\mathcal{G}}[\mathcal{D}]$). For a region R, we denote by $\delta_\beta(R) \subseteq \Gamma$ the *boundary-compressed-edges* of R, defined as

$$\delta_\beta(R) \coloneqq \{(u, v) \in \Gamma \mid u \in \mathcal{D}(R), v \notin \mathcal{D}(R)\}. \tag{12}$$

3.1.3 Region edges

A *region edge* describes a relationship between two graph fill regions, or between a region and the boundary. We use (A, B) to denote a region edge, where here A and B are both regions. Whenever we describe an edge between two regions, or between a region and the boundary, it is implied that it is a region edge. A region edge (A, B) comprises its endpoints A and B as well as a compressed edge $\beta(P_{u,v})$ representing the shortest path between any detection event in A and any detection event in B. We sometimes use the notation (A_u, B_v) for a region edge to explicitly specify the two

regions A and B along with the endpoints (u, v) of the compressed edge $\beta(P_{u,v})$ associated with it.

For any region edge that arises in sparse blossom, the following invariants always hold:

1. If A and B are both regions, then either A and B are both active regions (with no blossom-parent), or both have the same blossom-parent.

2. The compressed edge (u, v) associated with a region edge (A_u, B_v) is always *tight* (it would correspond to a tight edge in $\bar{\mathcal{G}}[\mathcal{D}]$ in blossom). More precisely, a compressed edge (u, v) is tight if it satisfies
$$D(u, v) = \sum_{R \in \mathcal{R} \,:\, (u,v) \in \delta_\beta(R)} y_R. \tag{13}$$
In other words, if there is a region edge (A, B), then regions A and B must be touching.

3.1.4 Matches

We call a pair of regions A and B that are matched to each other a *match*. If A and B are matched, this corresponds to the compressed edge between A and B being matched on the path graph. The matched regions A and B must be touching, assigned a growth rate of zero (they are frozen), and must be joined by a region edge (A, B) which we refer to as a *match edge*. An example of a match is shown in the middle of Figure 3. Initially, all regions are unmatched, and once the algorithm terminates, every region (either a trivial region or a blossom) is matched either to another region or to the boundary.

3.1.5 Alternating trees

An alternating tree is a tree where each node corresponds to an active graph fill region and each edge corresponds to a region edge. We refer to each region edge in the alternating tree as a *tree-edge*. Two regions connected by a tree-edge must always be touching (since every tree-edge is a region edge).

An alternating tree contains at least one growing region and can also contain shrinking regions, and always contains exactly one more growing region than shrinking region. Each growing region can have any number of children (each a *tree-child*), all of which must be shrinking regions, and can have a single parent (a *tree-parent*), also a shrinking region. Each shrinking region has a single child, a growing region, as well as a single parent, also a growing region. The leaves of an alternating tree are therefore always growing regions. An example of an alternating tree is shown in Figure 3.

3.1.6 Blossoms

When two growing regions from within the same alternating tree hit each other they form a *blossom*, which is a region containing an odd-length cycle of regions called a *blossom cycle*. More concretely, we will denote a blossom cycle as an ordered tuple of regions $(R_0, R_1, \ldots, R_{k-1})$ for some odd k and each region R_i in the blossom cycle is connected to each of its two neighbours by a region edge. In other words, the blossom cycle has region edges $\{(R_i, R_{(i+1) \mod k}) | i \in \{0, 1, \ldots, k-1\}\}$. An example of a blossom is shown on the right side of Figure 3.

The regions in the blossom cycle are called the blossom's *blossom-children*. If a blossom B has region b as one of its blossom-children, then we say that B is the *blossom-parent* of b. Neighbouring regions in a blossom cycle must be touching (as required by the fact that they are connected by a region edge). Blossoms can also be nested; each blossom-child can itself be a blossom, with its own blossom-children. For example, the top-left blossom-child of the blossom in Figure 3 is itself a blossom, with three blossom-children. A blossom *descendant* of a blossom B is a region that is either a blossom child of B or is recursively a descendant of any blossom child of B. Similarly, a blossom *ancestor* of B is a region that is either the blossom parent of B or (recursively) an ancestor of the blossom parent of B. The radius y_B of a blossom B (its dual variable in the standard blossom algorithm) is the distance it has grown since it formed (the minimum distance between a point on

its surface and any point on its source). This is visualised as the distance across the shell of the blossom in Figure 3.

We say that a detector node u is *contained* in a region R if u is in the shell area of R. A detector node can only be contained in at most one region (shell areas are disjoint). If a detector node is not contained in a region we say that the node is *empty*, and otherwise it is *occupied*. We say that a detector node u is *owned* by a region R either if u is contained in R, or if u is contained in a blossom descendant of R. The distance $D_S(R, u)$ between a detector node and the surface of a region R is

$$D_S(R, u) = \min_{x \in \mathcal{D}(R)} \left(D(u, x) - \sum_{A \in \mathcal{R} : x \in \mathcal{D}(A), A \leq R} y_A \right) \tag{14}$$

where here $A \leq R$ denotes that A is either a descendant of R or $A = R$. If $D_S(R, u) < 0$ then detector node u is owned by region R, if $D_S(R, u) > 0$ then detector node u is not owned by R, whereas if $D_S(R, u) = 0$ then u may or may not be owned by R (it is on the surface of R).

3.1.7 The timeline

The algorithm proceeds along a timeline with the time t increasing monotonically as different events occur and are processed. Examples of events include a region arriving at a node, or colliding with another region. The time that each event occurs is determined based on the radius equations of the regions involved, as well as the structure of the graph. The algorithm terminates when there are no more events left to be processed, which happens when all regions have been matched, and have therefore become frozen.

3.2 Architecture

Sparse blossom is split into different components: a *matcher*, a *flooder* and a *tracker*. Each component has different responsibilities. The matcher is responsible for managing the structure of the alternating trees and blossoms, without knowledge of the underlying structure of the detector graph. The flooder handles how graph fill regions grow and shrink in the detector graph, as well as noticing when a region collides with another region or the boundary. When the flooder notices a collision involving a region, or when a region reaches zero radius, the flooder notifies the matcher, which is then responsible for modifying the structure of the alternating trees or blossoms. The tracker is responsible for handling *when* the different events occur, and ensures that the flooder handles events in the correct order, with the help of a single priority queue. A priority queue is a type of queue, holding a collection of items, with the property that an item removed from the queue (with the "extract-min" operation) is guaranteed to have the highest priority among all items in the queue (in our case each item is an event, with earlier event times corresponding to a higher priority). The tracker uses this priority queue to inform the flooder when every event should be handled.

3.3 The matcher

At the initialisation stage of the algorithm, every detection event is initialised as the source of a growing region, a trivial alternating tree. As these regions grow and explore the graph, they can hit other (growing or frozen) regions, as well as the boundary, until eventually all regions are matched and the algorithm terminates. A growing region cannot hit a shrinking region, since shrinking regions recede exactly as quickly as growing regions expand.

When the flooder notices that a growing region R has hit another (growing or frozen) region R' or the boundary, it finds the *collision edge* and gives it to the matcher. The collision edge is a region edge between R and R' (or, if R hit the boundary, then between R and the boundary). The collision edge can be constructed by the flooder from local information at the point of collision, as will be explained in Section 3.4, and it is used by the matcher when handling events that change the structure of the alternating tree (which we refer to as *alternating tree events*). The matcher is responsible for handling alternating tree events, as well as for recovering the pairs of matched detection events once all regions have been matched.

3.3.1 Alternating tree events

There are seven different types of events that can change the structure of an alternating tree, and which are handled by the matcher, shown in Figure 4:

(a) A growing region R in an alternating tree T can hit a region M_1 that is matched to another region M_2. In this case, M_1 becomes a tree-child of R in T (and starts shrinking), and M_2 becomes a tree-child of M_1 in T (and starts growing). The collision edge (R, M_1) and match edge (M_1, M_2) both become tree-edges (R is the tree-parent of M_1, and M_1 is the tree-parent of M_2).

(b) A growing region R in an alternating tree T hits a growing region R' in a different alternating tree T'. When this happens, R is matched to R' and the remaining regions in T and T' also become matched. The collision edge (R, R') becomes a match edge, and a subset of the tree-edges also become match edges. All the regions in T and T' become frozen.

(c) A growing region R in an alternating tree T can hit another growing region R' in the *same* alternating tree T. This leads to an odd-length cycle of regions which form the blossom cycle C of a new blossom B. The region edges (blossom edges) in the blossom cycle are formed from the collision edge (R, R'), as well as the tree-edges along the paths from R and R' to their most recent common ancestor A in T. The newly formed blossom becomes a growing node in T. When forming the cycle C, we define the *orphans* O to be the set of shrinking regions in T but not C that were each a child of a growing region in C. The orphans become tree-children of B in T. The compressed edge associated with the new tree-edge (B, F) (connecting the new blossom region to its tree-parent F) is just the compressed edge that was associated with the old tree-edge (A, F). Similarly, the compressed edges connecting each orphan to its alternating tree parent remains unchanged (even though its parent *region* becomes B). In other words, if an orphan O^i had been connected to its tree-parent R^i by the tree-edge (O^i_u, R^i_v) before the blossom formed, the new tree-edge connecting it to its new tree-parent B will be (O^i_u, B_v) once the blossom B forms and the region R^i becomes part of the blossom cycle C.

(d) When a blossom B in an alternating tree T shrinks to a radius of zero, instead of the radius becoming negative the blossom must *shatter*. When the blossom shatters, the odd-length path through its blossom cycle from the tree-child of B to the tree-parent of B is added to T as growing and shrinking regions. The even length path becomes matches. The blossom-edges in the odd length path become tree-edges, and some of the blossom-edges in the even length path become match edges (the remaining blossom-edges are forgotten). Note that the endpoints of the compressed edges associated with the tree-edges joining B to its tree-parent and tree-child are used to determine where and how the blossom cycle is cut into two paths.

(e) When a trivial region R shrinks to a radius of zero, instead of the radius becoming negative a blossom forms. If R has a child C and parent P in the alternating tree T, when R has zero radius it must be that C is touching P (it is as if C has collided with P). The newly formed blossom B has the blossom cycle (P, R, C). The old tree-edges (P_u, R_v) and (R_v, C_w) become blossom edges in the blossom cycle. The blossom edge connecting C with P in the blossom cycle is computed from edges (P_u, R_v) and (R_v, C_w) and is (C_w, P_u). In other words, its compressed edge has endpoints (w, u) with logical observables $l(u, v) \oplus l(v, w)$.

(f) When a growing region R in an alternating tree T hits the boundary, R matches to the boundary and the collision edge becomes the match edge. The remaining regions in T also become matches.

(g) When a growing region R in an alternating tree T hits a region M that is matched to the boundary, then M instead becomes matched to R (and the collision edge becomes the match edge), and the remaining edges in T also become matches.

Some of these events involve changing the growth rate of regions (for example, two growing regions both become frozen regions when they match to each other). Therefore, when handling each alternating tree event, the matcher informs the flooder of any required changes to region growth.

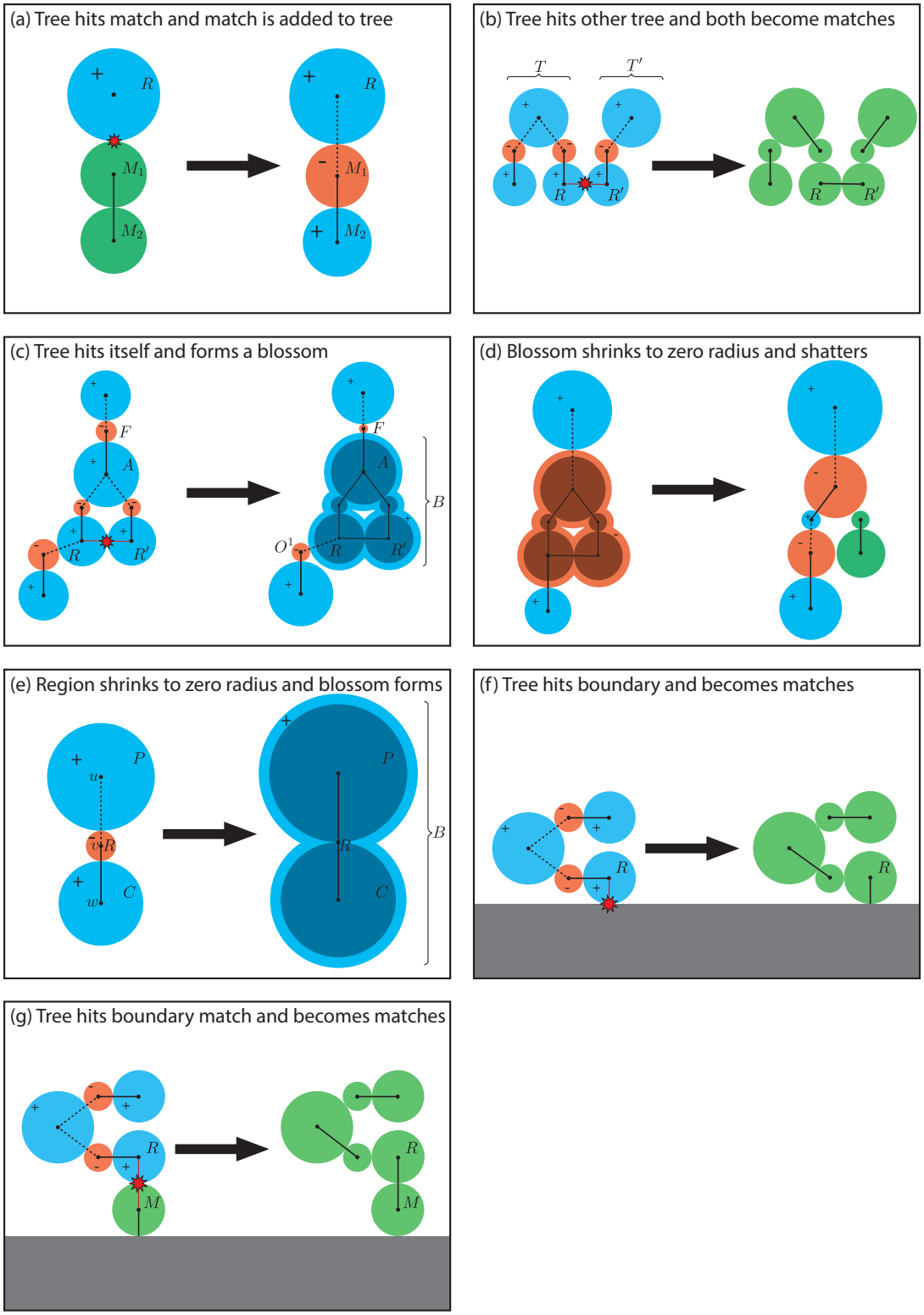


Figure 4: The main events that change the structure of alternating trees. For clarity, the background detector graph has been omitted. Each node corresponds to a detection event, and each edge corresponds to a compressed edge. Some labels (e.g. of regions) have been included in the diagrams and correspond to those referred to in the main text in Section 3.3.1.

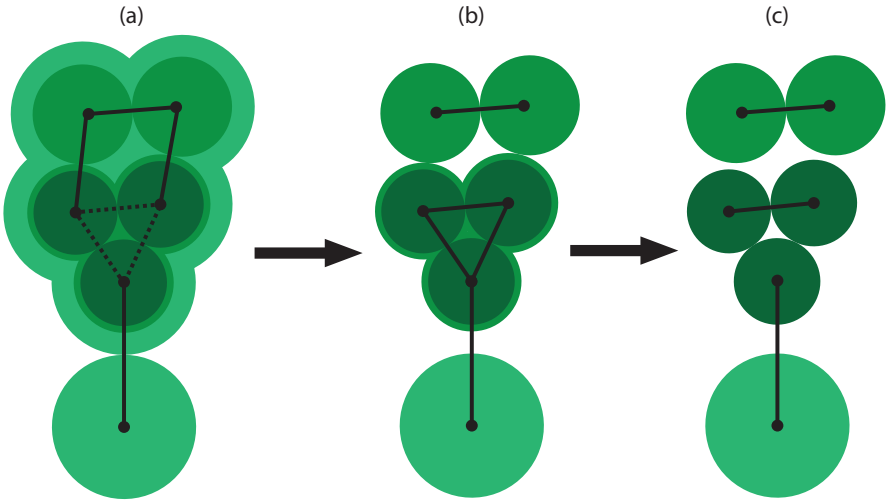


Figure 5: Shattering a matched blossom. Solid lines within a blossom are edges in the topmost blossom cycle. Dashed lines are edges in the blossom cycle of the blossom-child of the topmost blossom.

3.3.2 Matched detection events from matched regions

Provided there is a valid solution, eventually all regions become matched to other regions, or to the boundary. However, some of these matched regions may be blossoms, not trivial regions. To extract the compressed edge representing the match for each detection event instead, it is necessary to *shatter* each remaining blossom, and match its blossom children, as shown in in Figure 5. Suppose a blossom B, with blossom cycle C, is matched to some other region R with the match edge (B_u, R_v), where we recall that u is a detection event in B and v is a detection event in R. We find the blossom child $c \in C$ of B which contains the detection event u. We shatter B and match c to R with the compressed edge (u, v). The remaining regions in the blossom cycle C are then split into neighbouring pairs, which become matches. This process is repeated recursively until all matched regions are trivial regions.

3.4 The flooder

The flooder is responsible for managing how graph fill regions grow, shrink or collide in the detector graph, and is not concerned with the structure of the alternating trees and blossoms, which is instead handled by the matcher. We refer to the events handled by the flooder as *flooder events*.

Broadly speaking, we can categorise flooder events into four different types:

1. ARRIVE: A growing region R can grow into an empty detector node u.
2. LEAVE: A shrinking region R can leave a detector node u.
3. COLLIDE: A growing region can hit another region, or the boundary.
4. IMPLODE: A shrinking region can reach a radius of zero.

Let us first consider what happens for ARRIVE and LEAVE events. Neither of these types of events can change the structure of the alternating trees or blossoms, so the matcher does not need to be notified. Instead, it is the flooder's responsibility to ensure that any new flooder events get scheduled (inserted into the tracker's priority queue) after the events have been processed. When a region grows into a node u, the flooder *reschedules* the node u, by notifying the tracker of the next flooder event that can occur along an edge adjacent to u (either an ARRIVE or COLLIDE event, see Section 3.4.1). When a shrinking region leaves a node, the flooder immediately checks the top of the shell area stack and schedules the next LEAVE or IMPLODE event (see Section 3.4.2).

The flooder only needs to notify the matcher of a COLLIDE or IMPLODE event, and when a collision occurs the flooder passes the collision edge to the matcher as well. When either of these

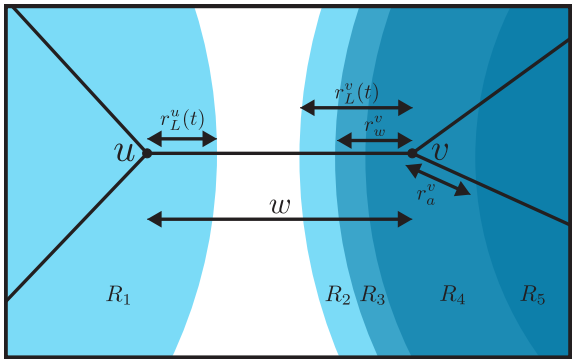


Figure 6: Two regions R_1 and R_2 colliding along an edge (u, v). Node u is contained in region R_1, which is an active region (no blossom parent). Node v is contained in region R_4, and is owned by R_4 as well as its blossom ancestors R_2 and R_3. Region R_4 also has R_5 as one of its blossom children. We have labelled the local radius $r_L^u(t)$ of node u and the local radius $r_L^v(t)$ of node v, as well as the wrapped radius r_w^v of v and the radius of arrival for v. The edge weight w of the edge (u, v) is also shown.

types of events occur, the matcher may change the growth rate of some regions when updating the structure of alternating trees or blossoms. The matcher then notifies the flooder of any change of growth rate, which may require the flooder to reschedule some flooder events. For example, if a region R was shrinking, but then becomes frozen or starts growing, the flooder reschedules all nodes contained in R (including blossom children, and their children etc.), to check for new ARRIVE or COLLIDE events. When a region starts shrinking, then the flooder informs the tracker of the next LEAVE or IMPLODE event by checking the top of the shell area stack.

The correct ordering of these flooder events is ensured by the tracker, and we create a growing region for each detection event simultaneously at time $t = 0$. Our implementation therefore uses an alternating tree growth strategy that is analogous to what's described as a "multiple tree approach with fixed δ" in [Kol09].

3.4.1 Rescheduling a node

When the flooder *reschedules* a node, it looks for an ARRIVE or COLLIDE event along each neighboring edge. There will be an ARRIVE event along an edge if one node is occupied by a growing region and the other is empty. There will be a COLLIDE event if both nodes are owned by active regions (R_1, R_2) with growth rates $(1, 1)$, $(0, 1)$ or $(1, 0)$, or if one region is growing towards a boundary, for a half-edge.

In order to calculate *when* an ARRIVE or COLLIDE event will occur along an edge, we use the *local radius* of each node. The *local radius* $r_L^v(t)$ of an node v is the amount that regions owning v have grown beyond v (see Figure 6). To define the local radius more precisely, we will need some more definitions. The *radius of arrival* r_a^v for an occupied node v contained in a region R is the radius that R had when it arrived at v. We denote the radius of a region A by $y_A(t)$ and we let $\mathcal{O}(v)$ be the set of regions that own a detector node v (the region that v is contained in, as well as its blossom ancestors). The local radius is then defined as

$$r_L^v(t) = -r_a^v + \sum_{A \in \mathcal{O}(v)} y_A(t). \tag{15}$$

This definition can be understood by considering the example in Figure 6, for which we recall that the radius of a blossom region can be visualized as the thickness of (i.e. distance across) the shell of the blossom. Both the local radius and radius of arrival of a node v are defined to be zero if v is empty. Therefore, for an edge (u, v) with weight w, the time of an ARRIVE or COLLIDE event can be found by solving $r_L^u(t) + r_L^v(t) = w$ for t. The only situation in which this involves division is when the local radius of both nodes are growing (have gradient one), in which case the collision occurs at time $t_{\text{collide}} = (w - r_L^u(0) - r_L^v(0))/2$. However, provided all edges are assigned even integer weights all flooder events, including these collisions between growing regions, occur at integer times.

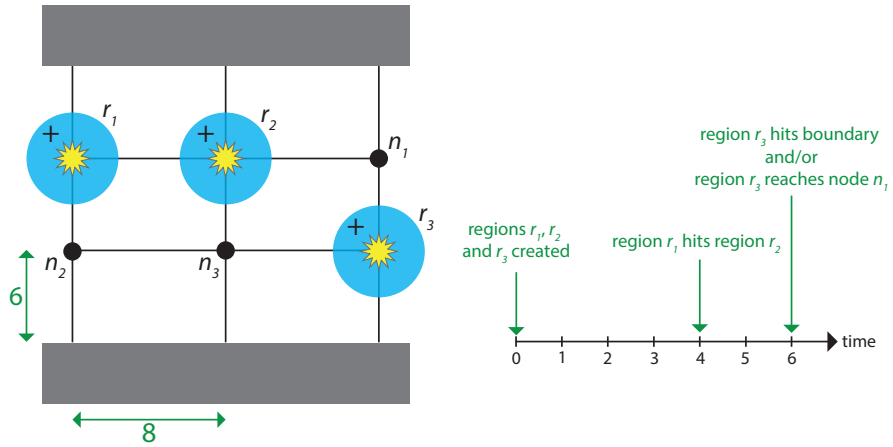


Figure 7: An example of some flooder events in a detector graph with three growing regions.

3.4.2 Rescheduling a shrinking region

When a region is shrinking, we find the time of the next LEAVE or IMPLODE event by inspecting the shell area stack of the region. If the stack is empty, or if the region has no blossom children and only a single node remains on the stack (the region's source detection event), then the next event is an IMPLODE event, the time of which can be found from the x-intercept of the region's radius equation. Otherwise, the next event is a LEAVE event, with the node u at the top of the stack leaving the region. We find the time of this next LEAVE event using the local radius of u, by solving $r_L^u(t) = 0$ for t. Using this approach, shrinking a region is much cheaper than growing a region, as it doesn't require enumerating edges in the detector graph.

3.4.3 An example

We give a small example of how the timeline of the flooder progresses in Figure 7. Since regions r_1, r_2 and r_3 are all initialised at $t = 0$, their radius equations are all equal to t. Regions r_1 and r_2 are separated by a single edge with weight 8 and therefore collide at time $t = 4$ (and recall that edge weights are always even integers to ensure collisions occur at integer times). When the matcher is informed of the collision, r_1 and r_2 are matched and become frozen regions. Region r_3 reaches empty node n_1 and the boundary at the same time ($t = 6$), and so there are two equally valid sequences of events. Either region r_3 matches to the boundary, and never reaches n_1, or r_3 reaches n_1 and then matches to the boundary. Clearly the final state of the algorithm is not unique, however there is a unique solution *weight*, and in this instance, both outcomes lead to the same set of compressed edges in the solution.

3.4.4 Compressed tracking

Whenever a collision occurs between two regions A and B, the flooder constructs a region edge (A, B), which we recall includes a compressed edge corresponding to the shortest path between a detection event in A and a detection event in B. By storing relevant information on nodes as regions grow, the compressed edge can be constructed efficiently using local information at the point of collision (i.e. using only information stored on the edge (u, v) that the collision occurs on, and on the nodes u and v).

This is explained in Figure 8. As a region R reaches an empty node v by growing along an edge (u, v) from a predecessor node u, we store on v a pointer to the detection event $\mathcal{S}(v)$ it was reached from (which can simply be copied from u). In other words, we set $\mathcal{S}(v) \coloneqq \mathcal{S}(u)$ once v is reached from u. Initially, when a search is started from a detection event w (i.e. a trivial growing region is created containing w), then we set $\mathcal{S}(w) \coloneqq w$. We refer to $\mathcal{S}(v)$ as the *source detection event* of v.

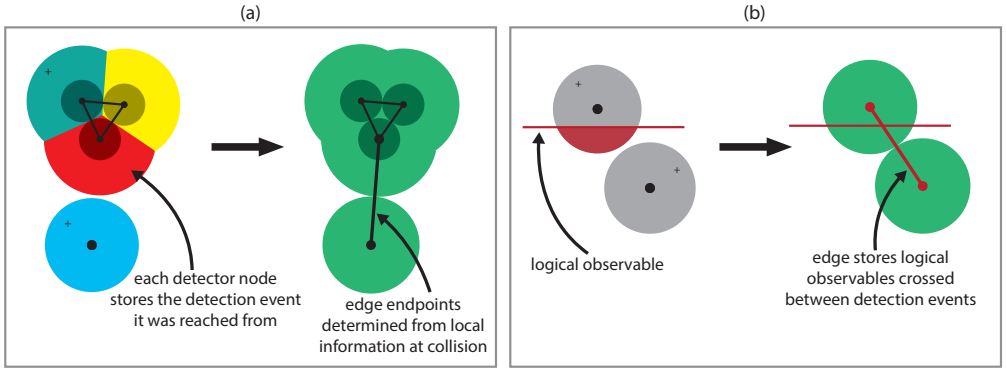


Figure 8: (a) As a region expands, each detector node it contains stores the detection event it was reached from (visualised by the Voronoi-style colouring of the blossom on the left). When a collision occurs, this allows the endpoints of the corresponding compressed edge (collision edge) to be determined from local information at the point of collision. (b) Each detector node also stores (as a 64-bit bitmask) the observables that were crossed to reach it from the detection event it was reached from. This allows the observables bitmask of the compressed edge to be recovered efficiently, also from local information at the point of collision.

We also store on v the set of observables $l(v)$ crossed during the region growth (i.e. the observables crossed along a shortest path from $\mathcal{S}(v)$ to v). This set of crossed observables $l(v)$ can be efficiently computed when v is reached from u along edge (u, v) from $l(v) \coloneqq l(u) \oplus l(u, v)$, where we implement $\oplus$ as a bitwise XOR since $l(u)$ and $l(u, v)$ are stored as bitmasks. Initially, when a trivial growing region is created at a detection event w we set $l(w)$ to the empty set.

Therefore, when a collision occurs between regions R and R' along an edge (p, q), the endpoints (x, y) of the compressed edge associated with the collision edge (R_x, R'_y) can then be determined locally as $x = \mathcal{S}(p)$ and $y = \mathcal{S}(q)$. The observables $l(x, y)$ associated with the collision edge can be computed locally as $l(x, y) \coloneqq l(p) \oplus l(q) \oplus l(p, q)$. Note that compressed tracking can also be used to remove the peeling step of the union-find decoder [DN21], as we explain in Appendix A. We note that it would also be interesting to explore how compressed tracking could be generalised to improve decoding for other families of codes that are not decodable with matching (for which the corresponding error models are not graphlike).

In PyMatching, we only fully rely on compressed tracking when there are 64 logical observables or fewer. When there are more than 64 logical observables, we don't include the observable bitmask in the compressed edges, instead only storing the detection events at its endpoints. This still allows us to use sparse blossom to find which detection events are matched to each other. Then after sparse blossom has completed, for each matched pair (u, v) we use a bi-directional Dijkstra search (implemented by adapting the flooder and tracker as required) to find the shortest path between u and v. If $C \subseteq \mathcal{E}$ is the set of all edges along the shortest paths found this way by the Dijkstra search post-processing, then the solution output by PyMatching is $\bigoplus_{e_i \in C} l(e_i)$. Note that since we are only post-processing with Dijkstra rather than constructing the full path graph, this only adds a small relative overhead (typically less than 50%) to the runtime.

3.5 Tracker

The tracker is responsible for ensuring flooder events occur in the correct order. A simple approach one *could* take to implement the tracker would just be to place every flooder event in a priority queue. However, many of the potential flooder events are chaff. For example, when a region R is growing, a flooder event would be added to the queue for each of its neighboring edges. We say an edge (u, v) is a neighbor of a region R if u is in R and v is not (or vice versa). Along each neighboring edge, there will be an event either corresponding to the region growing into an empty node, or colliding with another region or boundary. However, if the region becomes frozen or shrinking, then all of these remaining events will be invalidated.

To reduce this chaff, rather than adding every flooder event to a priority queue, the tracker instead adds *look-at-node* and *look-at-region* events to a priority queue. The flooder just finds the

time of the *next* event at a node or region, and asks the tracker for a reminder to look back at that time. As a result, at each node, we only need to add the next event to the priority queue. The remaining potential flooder events along neighboring edges will not be added if they have become invalidated.

When the flooder reschedules a node, it finds the time of the next ARRIVE or COLLIDE event along a neighboring edge, and asks the tracker for a reminder to look back at that time (a *look-at-node* event). The flooder finds the time of the next LEAVE or IMPLODE event for a shrinking region by checking the top of the shell area stack, and asks the tracker for a reminder to look back at the region at that time (a *look-at-region* event). To further reduce chaff, the tracker only adds a *look-at-node* or *look-at-region* event to the priority queue if it will occur at an earlier time than an event already in the queue for the same node or region. Once the tracker reminds the flooder to look back at a node or region, the flooder checks if it is still a valid event by recomputing the next event for the node or region, processing it if so.

3.6 Comparison between blossom and sparse blossom

In Table 1 we summarise how some of the concepts in the traditional blossom algorithm translate into concepts in sparse blossom. If the traditional blossom algorithm is run on the path graph $\bar{\mathcal{G}}[\mathcal{D}]$ using a multiple tree approach (and with all dual variables initialised to zero at the start), then a valid state of blossom at a particular stage corresponds to a valid state of sparse blossom for the same problem at the appropriate point in the timeline. The dual variables in blossom define the region radiuses in sparse blossom (and these radiuses can be used to construct the corresponding exploratory regions). Likewise the edges in the alternating trees, blossom cycles and matches in traditional blossom can all be translated into compressed edges in the corresponding entities in sparse blossom. We note, however, that when multiple alternating tree events happen at the same time in sparse blossom, *any* ordering for the processing of these events is a valid choice. So just because we can translate a valid state of one algorithm to that of the other, does not imply that two implementations of the algorithms (or the same algorithm) will have the same sequence of alternating tree manipuations (indeed it is unlikely that they will). This correspondence between sparse blossom run on the detector graph and traditional blossom run on a path graph, and the correctness of blossom itself for finding a MWPM, is one way understanding why sparse blossom correctly finds a MWEM in the detector graph.

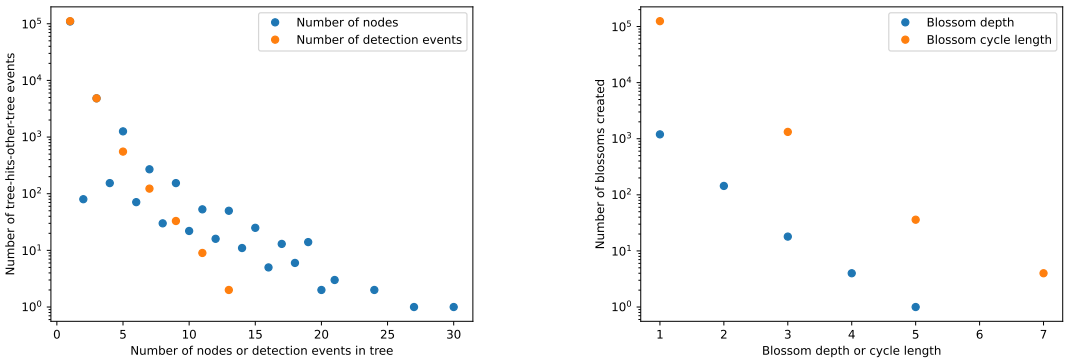
4 Data structures

In this section, we outline the data structures we use in sparse blossom. Each detector node u stores its neighbouring edges in the detector graph as an adjacency list. For each neighbouring edge (u, v), we store its weight $w((u, v))$ as a 32-bit integer, the observables $l(u, v)$ it flips as a 64-bit bitmask, as well as its neighbouring node v as a pointer (or as nullptr for the boundary). Edge weights are discretised as *even* integers, which ensures that all events (including two growing regions colliding) occur at integer times.

Each *occupied* detector node v stores a pointer to the source detection event it was reached from $\mathcal{S}(v)$, a bitmask of the observables crosssed $l(v)$ along the path from its source detection event and a pointer to the graph fill region $C(v)$ it is contained in (the region that arrived at the node). There are some properties that we cache on each occupied node whenever the blossom structure changes, in order to speed up the process of finding the next COLLIDE or ARRIVE event along an edge. This includes storing a pointer to the *active* region the occupied detector node v is owned by (the topmost blossom ancestor of the region it is contained in) as well as its radius of arrival r_a^v, which is the radius that the node's containing region $C(v)$ had when it arrived at v (see Section 3.4.1). Additionally, we cache the *wrapped radius* r_w^v of each detector node v whenever its owning regions' blossom structure changes (if a blossom is formed or shattered). The wrapped radius of an occupied node v is the local radius of the node, excluding the (potentially growing or shrinking) radius of the active region it is owned by. If we let $y_A(t)$ be the radius of the active region that owns v, we can recover the local radius from the wrapped radius with $r_L^v(t) = r_w^v + y_A(t)$.

Blossom concepts (multiple tree approach, applied to $\bar{\mathcal{G}}[\mathcal{D}]$)	Sparse blossom concepts
Dual variable y_u (for a node $u \in \mathcal{D}$) or y_S (for a set of nodes $S \subseteq \mathcal{D}$ of odd cardinality at least three)	Radius y_R of a graph fill region R, an exploratory region containing nodes and edges in the detector graph including an odd number of detection events.
A tight edge (u, v) between two nodes u and v in $\mathcal{D}$ that is in the matching or belongs to an alternating tree or a blossom cycle. Since (u, v) is an edge in the path graph, its weight is the length of a shortest path between the two detection events u and v in the detector graph $\mathcal{G}$ (found using a Dijkstra search when the path graph was constructed). It is *tight* since the dual variables associated with u and v result in zero slack as defined in Equation 6.	A compressed edge (u, v) associated with a region edge, belonging to a match, alternating tree or blossom cycle. The compressed edge (u, v) represents a shortest path between two detection events u and v in the detector graph $\mathcal{G}$. Associated with (u, v) is the set of logical observables $l(u, v)$ flipped by flipping edges in $\mathcal{G}$ along the corresponding path between u and v. The two regions in the corresponding region edge are touching (the edge is tight).
An edge (u, v) between two nodes u and v in $\mathcal{D}$ that is *not* tight. Its weight is the length of the shortest path between u and v in the detector graph $\mathcal{G}$, found using a Dijkstra search when constructing the path graph. For typical QEC problems, the vast majority of edges in $\bar{\mathcal{G}}[\mathcal{D}]$ never become tight, but for standard blossom they still must be explicitly constructed using a Dijkstra search.	There is no analogous data structure for this edge in sparse blossom. The shortest path between u and v is not currently fully explored by graph fill regions. This means that either the path has not yet been discovered by sparse blossom (and may never be), or perhaps it had previously been discovered (belonging to a region edge) but at least one of the regions owning u or v since shrunk (e.g. it was matched and then became a shrinking region in an alternating tree).
In the dual update stage, update each dual variable y_u of an outer node with $y_u \coloneqq y_u + \delta$ and update each dual variable y_u of an inner node with $y_u \coloneqq y_u - \delta$. The variable $\delta \in \mathbb{R}_{\geq 0}$ is set to the maximum value such that the dual variables remain a feasible solution to the dual problem.	As time Δt passes, each growing region R explores the graph and its radius y_R increases by Δt and each shrinking region R shrinks in the graph and its radius y_R decreases by Δt. Eventually at $\Delta t = \delta$ a collision or implosion occurs (one of the matcher events in Figure 4), which must be handled by the matcher.
An edge (u, v) between a node u in an alternating tree T and a node v in another alternating tree T' becomes tight after a dual update. The path between the root of T and the root of T' is augmented and all nodes in the two trees become matches.	A growing region R of an alternating tree T collides with a growing region R' of another alternating tree T'. The collision edge (R, R') is constructed from local information at the point of collision. R is matched to R' with the collision edge (R, R') as a match edge. All other regions in the trees also become matched. All regions in the two trees become frozen.
An edge (u, v) between two outer nodes u and v in the same tree T becomes tight after a dual update. This forms an odd-length cycle in T which becomes a *blossom*, which itself becomes an outer node in T.	Two growing regions R and R' from the same alternating tree T collide. The discovered collision edge (R, R') as well as the regions and tree-edges along the path between R and R' in T become a blossom cycle in a newly formed blossom. The blossom starts growing, and is now a growing region in T.

Table 1: The correspondence between concepts in the standard blossom algorithm and concepts in sparse blossom. For the standard blossom algorithm we assume a "multiple tree with fixed δ" approach is used, and further we assume that the algorithm is applied to the path graph $\bar{\mathcal{G}}[\mathcal{D}] = (\mathcal{D}, \bar{\mathcal{E}})$.



(a) (b)

Figure 9: Distribution of alternating tree and blossom sizes observed in sparse blossom when decoding 1000 shots of distance-11 surface code circuits with $p = 0.3\%$ circuit-level noise. (a) A histogram of the size of alternating trees observed in events where a tree hits another tree, in terms of both the number of detection events and detector nodes contained in each tree. (b) A histogram of the size of blossoms formed, in terms of both the length of each blossom's blossom cycle, as well as its recursive depth. A blossom depth or cycle of size one is a trivial blossom (a graph fill region without blossom children).

Each graph fill region R has a pointer to its blossom parent and its topmost blossom ancestor. Its blossom cycle is stored as an array L of *cycle edges*, where the cycle edge $L[i]$ stores a pointer to the ith blossom child of R, along with the compressed edge associated with the region edge joining child i to child $i + 1 \mod n_c$, where n_c is the number of blossom children of R. Its shell area stack is an array of pointers to the detector nodes it contains (in the order they were added). For its radius $y_R(t) = mt + c$ we use 62 bits to store the y-intercept c and with 2 bits used to store the gradient $c \in \{-1, 0, 1\}$. Each region also stores its match as a pointer to the region it is matched to, along with the compressed edge associated with the match edge. Finally, each growing or shrinking region has a pointer to an AltTreeNode.

An AltTreeNode is used to represent the structure of an alternating tree. Each AltTreeNode corresponds to a growing region in the tree, as well as its shrinking parent region (if it has one). Each AltTreeNode has a pointer to its growing graph fill region and its shrinking graph fill region (if it has one). It also has a pointer to its parent AltTreeNode in the alternating tree (as a pointer and a compressed edge), as well as to its children (as an array of pointers and compressed edges). We also store the compressed edge corresponding to the shortest path between the growing and shrinking region in the AltTreeNode.

Each detector node and graph fill region also has a tracker field, which stores the desired time the node or region should next be looked at, as well as the time (if any) it is already scheduled to be looked at as a result of a *look-at-node* or *look-at-region* event already in the priority queue (called the *queued time*). The tracker therefore only needs to add a new *look-at-node* or *look-at-region* event to the priority queue if its desired time is set to be *earlier* than its current queued time.

A property of the algorithm is that each event dequeued from the priority queue must have a time that is greater than or equal to all previous events dequeued. This allows us to use a radix heap [Ahu+90], which is an efficient monotone priority queue with good caching performance. Since the next flooder event can never be more than one edge length away from the current time, we use a cyclic time window for the priority, rather than the total time. We use 24, 32 and 64 bits of integer precision for the edge weights, flooder event priorities and total time, respectively.

5 Expected running time

Empirically, we observe an almost-linear running time of our algorithm for surface codes below threshold (see Figure 10). We would expect the running time to be linear at low physical error rates, since in this regime a typical error configuration will consist of small isolated clusters of

errors. Provided the clusters are sufficiently well separated from one another, each cluster is essentially handled as an independent matching problem by sparse blossom. Furthermore, using results from percolation theory [Men86], we would expect the number of clusters of a given size to decay exponentially in this size [Fow15]. Since the number of operations required to match a cluster is polynomial in its size, this leads to a constant cost per cluster, and therefore an expected running time that is linear in the size of the graph.

In more detail, suppose we highlight every edge in the detector graph $\mathcal{G}$ ever touched by sparse blossom when decoding a particular syndrome. Now consider the subgraph $\mathcal{F}$ of $\mathcal{G}$ induced by these highlighted edges. This graph will generally have many connected components, and we will refer to each connected component as a *cluster region*. Clearly, running sparse blossom on each cluster region separately will give an identical solution to running the algorithm on $\mathcal{G}$ as a whole. Let us *assume* that the probability that a detector node in $\mathcal{G}$ is within a cluster region of n_c detector nodes is at most Ae^{-bn_c} for some $b > 0$. Since the worst-case running time of sparse blossom is polynomial in the number of nodes n (at most $O(n^4)$, see Appendix B), the expected running time to decode a cluster region (if any) at a given node is at most $\sum_{n_c=1}^{\infty} Ae^{-bn_c} n_c^4 = O(1)$, i.e. constant. Therefore, the running time is linear in the number of nodes. Here we have assumed that the probability of observing a cluster region at a node decays exponentially in its size. However, in [Fow15] it was shown that this is indeed the case at very low error rates. Furthermore, we provide empirical evidence for this exponential decay for error rates of practical interest in Figure 9, where we plot the distribution of the sizes of alternating trees and blossoms observed when using sparse blossom to decode surface codes with 0.3% circuit-level noise. Our benchmarks in Figure 10 and Figure 11 are also consistent with a running time that is roughly linear in the number of nodes, and even above threshold (Figure 12) the observed complexity of $O(n^{1.32})$ is only slightly worse than linear.

Note that here we have ignored the fact that our radix heap priority queue is shared by all clusters. This does not impact the overall theoretical complexity, since the insert and extract-min operations of the radix heap have an amortized time complexity that is independent of the number of items in the queue. In particular, inserting an element takes $O(1)$ time, and extract-min takes amortized $O(B)$ time, where B is the number of bits used to store the priority (for us $B = 32$).

Nevertheless, our use of the timeline concept and a multiple tree fixed δ approach (relying on the priority queue) does result in more cache misses for very large problem instances, empirically resulting in an increase in the processing time per detection event. This is because events that are local in time (and therefore processed soon after one another) are in general not local in memory, since they can require inspecting regions or edges that are very far from one another in the detector graph. In contrast we would expect a single tree approach to have better memory locality for very large problem instances. However, an advantage of the multiple tree approach we have taken is that it "touches" less of the detector graph. For example, consider a simple problem of two isolated detection events in a uniform detector graph, separated by distance d. In sparse blossom, since we use a multiple tree approach, these two regions will grow at the same rate and will have explored a region of radius $d/2$ when they collide. In a 3D graph, let's assume that a region of radius r touches $\approx kr^3$ edges for some constant k. In contrast, in a single tree approach (such as used by Refs. [FWH12a; FWH12b; Fow15]), one region will grow to radius d and collide with the region of the other detection event, which will still have radius 0. Therefore, the multiple tree approach has touched $\approx 2k(d/2)^3$ edges, $\approx 4\times$ fewer than the kd^3 edges touched by the single tree approach. This is essentially the same advantage that a bidirectional Dijkstra search has over a regular Dijkstra search.

6 Computational results

We benchmarked the running time of our implementation of sparse blossom (PyMatching 2) for decoding surface code memory experiments (see Figure 10, Figure 11 and Figure 12). For 0.1% circuit-level depolarising noise, sparse blossom processes both X and Z bases of distance-17 surface code circuits in less than one microsecond per round of syndrome extraction on a single core, which matches the rate at which syndrome data is generated by superconducting quantum computers.

At low physical error rates (e.g. Figure 10), the roughly linear scaling of PyMatching v2 is a quadratic improvement over the empirical scaling of an implementation that constructs the path

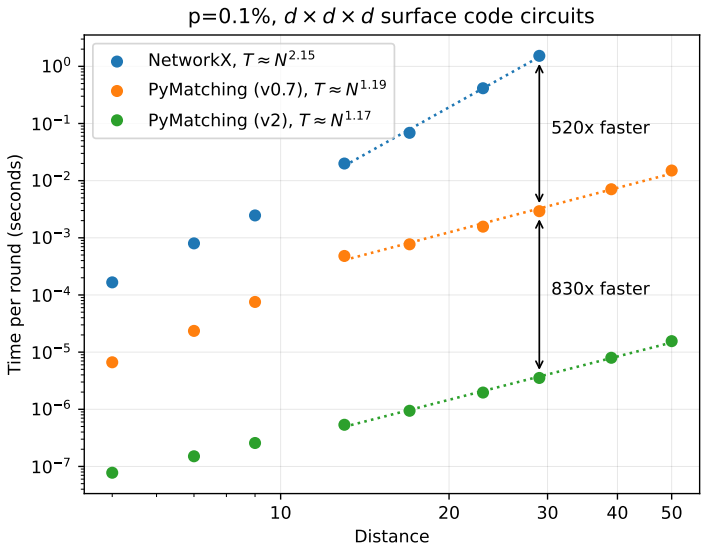


Figure 10: Decoding time per round for PyMatching v2 (our implementation of sparse blossom), compared to PyMatching v0.7 and a NetworkX implementation. For distance d, we find the time to decode d rounds of a distance d surface code circuit and divide this time by d to obtain the time per round. We use a circuit-level depolarising noise model where the probability $p = 0.1\%$ sets the strength of two-qubit depolarising noise after each CNOT gate, the probability that each reset or measurement fails, as well as the strength of single-qubit depolarising noise applied before each round. The threshold for this noise model is around 0.71%. PyMatching v0.7 uses a C++ implementation of the local matching algorithm described in [Hig22]. The pure Python NetworkX implementation first constructs a complete graph on the detection events, where each edge (u, v) represents a shortest path between u and v, and then uses the standard blossom algorithm on this graph to decode. All three decoders use a single core of an M1 Max processor.

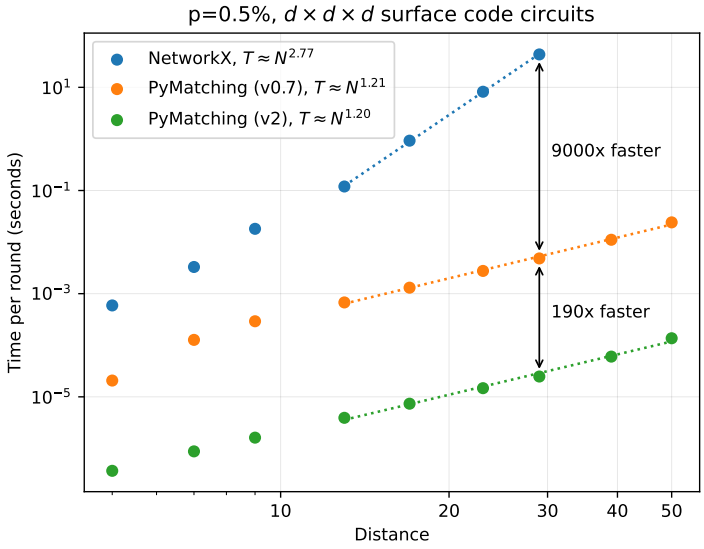


Figure 11: Decoding time per round for PyMatching v2 (our implementation of sparse blossom), compared to PyMatching v0.7 and a NetworkX implementation. The only difference with Figure 10 is that here we set $p = 0.5\%$.

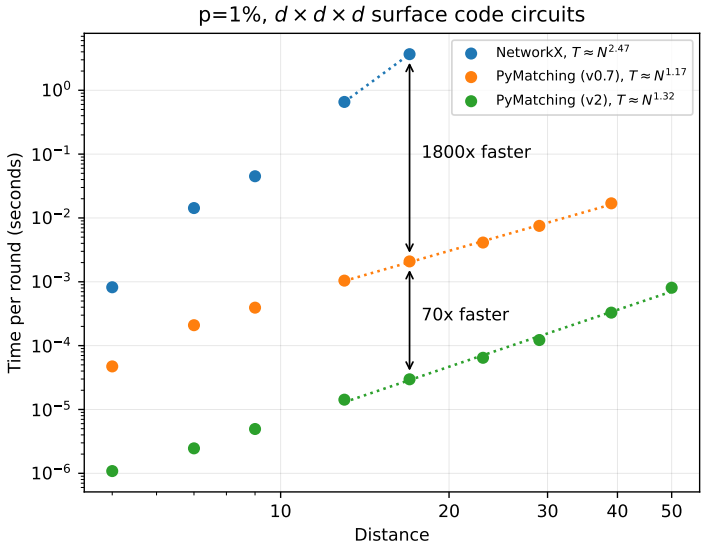


Figure 12: Decoding time per round for PyMatching v2 (our implementation of sparse blossom), compared to PyMatching v0.7 and a NetworkX implementation. The only difference with Figure 10 is that here we set $p = 1\%$.

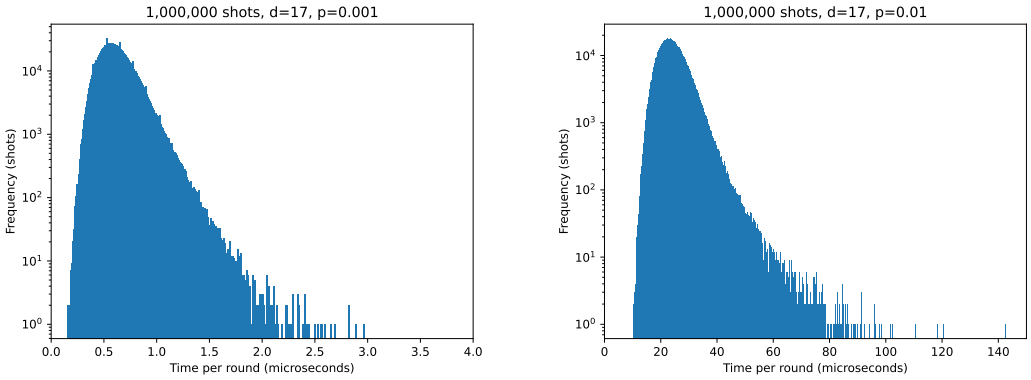
graph explicitly and solves the traditional MWPM problem as a separate subroutine. For 0.1%-1% physical error rates and distance 29 and larger, PyMatching v2 is $> 100,000\times$ faster than a pure Python implementation that uses the exact reduction to MWPM. Compared to the local matching approximation of the MWPM decoder used in [Hig22], PyMatching v2 has a similar empirical scaling but is around $100\times$ faster near threshold (error rates of 0.5% to 1%) and almost $1000\times$ faster below threshold ($p = 0.1\%$). Much of this speedup relative to local matching is due to the fact that local matching still explicitly constructs much more of the path graph than is ultimately used by the blossom algorithm. This is because the truncation of the path graph in local matching is not done adaptively during the blossom algorithm but instead in a separate step before the blossom algorithm begins. Furthermore, local matching is an approximation of the MWPM decoder, in contrast to Sparse Blossom (which is exact).

We analysed the distribution of the running time per shot of PyMatching v2 for simulated surface code data, see Figure 13 and Figure 14. For example, for distance-17 surface code circuits with $p = 0.1\%$ circuit-level noise, we observe a mean running time of 0.62 microseconds per round and find that 97.4% of the million shots were decoded with a running time below 1 microsecond per round. We also plot the relative standard deviation σ/μ of the running time per shot in Figure 14 and find that σ/μ decreases as either the distance or error rate is increased.

We also compared the speed of PyMatching v0.7 with that of PyMatching v2 on experimental data, by running both decoders on the full dataset of Google's recent experiment demonstrating the suppression of quantum errors by scaling a surface code logical qubit from distance 3 to distance 5 [AI23; Tea22]. On an M2 chip, PyMatching v0.7 took 3 hours and 43 minutes to decode all 7 million shots in the dataset, whereas PyMatching v2 took 71 seconds.

7 Conclusion

In this work, we have introduced a variant of the blossom algorithm, which we call sparse blossom, that directly solves the minimum-weight perfect matching decoding problem relevant to error correction. Our approach avoids the computationally expensive all-to-all Dijkstra searches often used in implementations of the MWPM decoder, where a reduction to the traditional blossom algorithm is used. Our implementation, available in version 2 of the open-source PyMatching Python package, can process around a million errors per second on a single core. For a distance-17



(a) (b)

Figure 13: Histograms showing the distribution of running times of sparse blossom using 17-round, distance-17 surface code circuits and a standard circuit-level depolarising noise model. In (a) we use $p = 0.1\%$ and a histogram bin width of 0.01 microseconds. 97.4% of the shots have a running time per round below 1 microsecond. In (b) we instead use $p = 1\%$ and a histogram bin width of 0.2 microseconds.

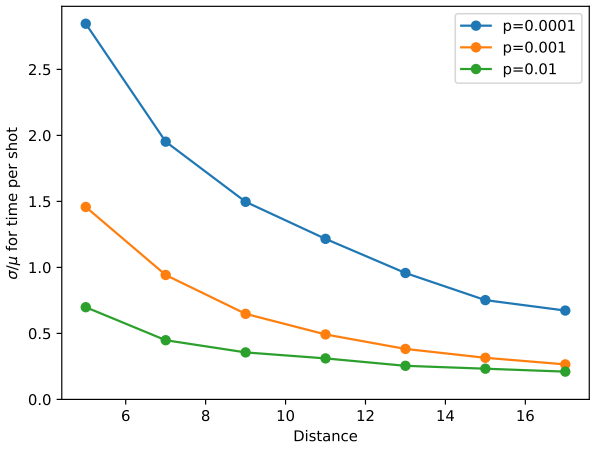


Figure 14: Relative standard deviation σ/μ of the time per shot for distance-d surface code circuits (with d rounds) and standard circuit-level depolarising noise. Here σ and μ are the standard deviation and mean, respectively, of the time per shot, sampling 1 million shots for each data point.

surface code, it can decode both X and Z bases in under 1µs per round of error correction, which matches the rate at which syndrome data is generated on a superconducting quantum computer.

Some of the techniques we have introduced can be directly applied to improve the performance of other decoders. For example, we introduced *compressed tracking*, which exploits the fact that the decoder only needs to predict which *logical observables* were flipped, rather than the physical errors themselves. This allowed us to use a sparse representation of paths in the detector graph, storing only the endpoints of a path, along with the logical observables it flips (as a bitmask). We showed that compressed tracking can be used to significantly simplify the union-find decoder (see Appendix A), leading to a compressed representation of the disjoint-set data structure and eliminating the need to construct a spanning tree in the peeling step of the algorithm.

When used for error correction simulations, our implementation can be trivially parallelised across batches of shots. However, achieving the throughput necessary for real-time decoding at scale motivates the development of a parallelised implementation of sparse blossom. For example, for the practically relevant task of decoding a distance-30 surface code at 0.1% circuit-level noise, the throughput of sparse blossom is around 3.5× slower than the $<$ 1µs per round throughput desired for a superconducting quantum computer. It would therefore be interesting to investigate whether a multi-core CPU or FPGA-based implementation could achieve the throughput necessary for real-time decoding at scale by adapting techniques in [WZ23] for sparse blossom. Finally, it would be interesting to adapt sparse blossom to exploit correlated errors that arise in realistic noise models, for example by using sparse blossom as a subroutine in an optimised implementation of a correlated matching [Fow13] or belief-matching [Hig+23] decoder.

Contributions

Both authors worked together on the design of the sparse blossom algorithm. Oscar Higgott wrote the paper and the majority of the code. Craig Gidney guided the project and implemented some final optimisations such as the radix heap.

Acknowledgements

We would like to thank Nikolas Breuckmann, Dan Browne, Austin Fowler, Noah Shutty and Barbara Terhal for helpful feedback that improved the manuscript.

References

[Ahu+90] Ravindra K Ahuja, Kurt Mehlhorn, James Orlin, and Robert E Tarjan. “Faster algorithms for the shortest path problem”. In: *Journal of the ACM (JACM)* 37.2 (1990), pp. 213–223. DOI: 10.1145/77600.77615.

[AI23] Google Quantum AI. “Suppressing quantum errors by scaling a surface code logical qubit”. In: *Nature* 614.7949 (Feb. 2023), pp. 676–681. ISSN: 1476-4687. DOI: 10.1038/s41586-022-05434-1.

[Bac06] Dave Bacon. “Operator quantum error-correcting subsystems for self-correcting quantum memories”. In: *Phys. Rev. A* 73 (1 Jan. 2006), p. 012340. DOI: 10.1103/PhysRevA.73.012340.

[Bar82] F Barahona. “On the computational complexity of Ising spin glass models”. In: *Journal of Physics A: Mathematical and General* 15.10 (Oct. 1982), p. 3241. DOI: 10.1088/0305-4470/15/10/028.

[BDL22] Burak Boyacı, Thu Huong Dang, and Adam N. Letchford. “On matchings, T-joins, and arc routing in road networks”. In: *Networks* 79.1 (2022), pp. 20–31. DOI: 10.1002/net.22033.

[Ber+99] Piotr Berman, Andrew B. Kahng, Devendra Vidhani, and Alexander Zelikovsky. “The T-join Problem in Sparse Graphs: Applications to Phase Assignment Problem in VLSI Mask Layout”. In: *Algorithms and Data Structures*. Ed. by Frank Dehne, Jörg-Rüdiger Sack, Arvind Gupta, and Roberto Tamassia. Berlin, Heidelberg: Springer Berlin Heidelberg, 1999, pp. 25–36. ISBN: 978-3-540-48447-9. DOI: 10.1007/3-540-48447-7˙3.

[Ber57] Claude Berge. “Two theorems in graph theory”. In: *Proceedings of the National Academy of Sciences* 43.9 (1957), pp. 842–844. DOI: 10.1073/pnas.43.9.842.

[BM06] H. Bombin and M. A. Martin-Delgado. “Topological Quantum Distillation”. In: *Phys. Rev. Lett.* 97 (18 Oct. 2006), p. 180501. DOI: 10.1103/PhysRevLett.97.180501.

[Bra+13] Sergey Bravyi, Guillaume Duclos-Cianci, David Poulin, and Martin Suchara. “Subsystem surface codes with three-qubit check operators”. In: *Quantum Info. Comput.* 13.11–12 (Nov. 2013), pp. 963–985. ISSN: 1533-7146. DOI: 10.26421/QIC13.11-12-4.

[Bro22] Benjamin J. Brown. “Conservation Laws and Quantum Error Correction: Toward a Generalized Matching Decoder”. In: *IEEE BITS the Information Theory Magazine* 2.3 (2022), pp. 5–19. DOI: 10.1109/MBITS.2023.3246025.

[BSV14] Sergey Bravyi, Martin Suchara, and Alexander Vargo. “Efficient algorithms for maximum likelihood decoding in the surface code”. In: *Phys. Rev. A* 90 (3 Sept. 2014), p. 032326. DOI: 10.1103/PhysRevA.90.032326.

[BT16] Nikolas P. Breuckmann and Barbara M. Terhal. “Constructions and Noise Threshold of Hyperbolic Surface Codes”. In: *IEEE Transactions on Information Theory* 62.6 (2016), pp. 3731–3744. DOI: 10.1109/TIT.2016.2555700.

[Cha+20] Rui Chao, Michael E. Beverland, Nicolas Delfosse, and Jeongwan Haah. “Optimization of the surface code design for Majorana-based qubits”. In: *Quantum* 4 (Oct. 2020), p. 352. ISSN: 2521-327X. DOI: 10.22331/q-2020-10-28-352.

[Das+22] Poulami Das, Christopher A. Pattison, Srilatha Manne, Douglas M. Carmean, Krysta M. Svore, Moinuddin Qureshi, and Nicolas Delfosse. “AFS: Accurate, Fast, and Scalable Error-Decoding for Fault-Tolerant Quantum Computers”. In: *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 2022, pp. 259–273. DOI: 10.1109/HPCA53966.2022.00027.

[Den+02] Eric Dennis, Alexei Kitaev, Andrew Landahl, and John Preskill. “Topological quantum memory”. In: *Journal of Mathematical Physics* 43.9 (2002), pp. 4452–4505. DOI: 10.1063/1.1499754.

[Der+24] Peter-Jan H. S. Derks, Alex Townsend-Teague, Ansgar G. Burchards, and Jens Eisert. “Designing fault-tolerant circuits using detector error models”. In: (2024). arXiv: 2407.13826 [quant-ph]. URL: https://arxiv.org/abs/2407.13826.

[DN21] Nicolas Delfosse and Naomi H. Nickerson. “Almost-linear time decoding algorithm for topological codes”. In: *Quantum* 5 (Dec. 2021), p. 595. ISSN: 2521-327X. DOI: 10.22331/q-2021-12-02-595.

[DZ20] Nicolas Delfosse and Gilles Zémor. “Linear-time maximum likelihood decoding of surface codes over the quantum erasure channel”. In: *Phys. Rev. Res.* 2 (3 July 2020), p. 033042. DOI: 10.1103/PhysRevResearch.2.033042.

[Edm65a] Jack Edmonds. “Maximum matching and a polyhedron with 0,1-vertices”. In: *Journal of Research of the National Bureau of Standards Section B Mathematics and Mathematical Physics* (1965), p. 125. DOI: 10.6028/jres.069b.013. URL: https://api.semanticscholar.org/CorpusID:15379135.

[Edm65b] Jack Edmonds. “Paths, Trees, and Flowers”. In: *Canadian Journal of Mathematics* 17 (1965), pp. 449–467. DOI: 10.4153/CJM-1965-045-4.

[EJ73] Jack Edmonds and Ellis L. Johnson. “Matching, Euler tours and the Chinese postman”. In: *Mathematical Programming* 5.1 (Dec. 1973), pp. 88–124. ISSN: 1436-4646. DOI: 10.1007/BF01580113.

[Fow13] Austin G Fowler. “Optimal complexity correction of correlated errors in the surface code”. In: *arXiv preprint arXiv:1310.0863* (2013).

[Fow15] Austin G. Fowler. “Minimum weight perfect matching of fault-tolerant topological quantum error correction in average O(1) parallel time”. In: *Quantum Info. Comput.* 15.1–2 (Jan. 2015), pp. 145–158. ISSN: 1533-7146. DOI: 10.26421/QIC15.1-2-9.

[FWH12a] Austin G. Fowler, Adam C. Whiteside, and Lloyd C. L. Hollenberg. “Towards Practical Classical Processing for the Surface Code”. In: *Phys. Rev. Lett.* 108 (18 May 2012), p. 180501. DOI: 10.1103/PhysRevLett.108.180501.

[FWH12b] Austin G. Fowler, Adam C. Whiteside, and Lloyd C. L. Hollenberg. “Towards practical classical processing for the surface code: Timing analysis”. In: *Phys. Rev. A* 86 (4 Oct. 2012), p. 042313. DOI: 10.1103/PhysRevA.86.042313. URL: https://link.aps.org/doi/10.1103/PhysRevA.86.042313.

[Gal83] Zvi Galil. “Efficient algorithms for finding maximal matching in graphs”. In: *CAAP’83*. Ed. by Giorgio Ausiello and Marco Protasi. Berlin, Heidelberg: Springer Berlin Heidelberg, 1983, pp. 90–113. ISBN: 978-3-540-38714-5. DOI: 10.1007/3-540-12727-5˙4.

[Gid+21] Craig Gidney, Michael Newman, Austin Fowler, and Michael Broughton. “A Fault-Tolerant Honeycomb Memory”. In: *Quantum* 5 (Dec. 2021), p. 605. ISSN: 2521-327X. DOI: 10.22331/q-2021-12-20-605.

[Gid20] Craig Gidney. *Decorrelated Depolarization*. https://algassert.com/post/2001. Accessed: 2021-10-04. 2020.

[Gid21] Craig Gidney. “Stim: a fast stabilizer circuit simulator”. In: *Quantum* 5 (July 2021), p. 497. ISSN: 2521-327X. DOI: 10.22331/q-2021-07-06-497.

[GJ23] Craig Gidney and Cody Jones. “New circuits and an open source decoder for the color code”. In: (2023). arXiv: 2312.08813 [quant-ph]. URL: https://arxiv.org/abs/2312.08813.

[HB21] Oscar Higgott and Nikolas P. Breuckmann. “Subsystem Codes with High Thresholds by Gauge Fixing and Reduced Qubit Overhead”. In: *Phys. Rev. X* 11 (3 Aug. 2021), p. 031039. DOI: 10.1103/PhysRevX.11.031039.

[HH21] Matthew B. Hastings and Jeongwan Haah. “Dynamically Generated Logical Qubits”. In: *Quantum* 5 (Oct. 2021), p. 564. ISSN: 2521-327X. DOI: 10.22331/q-2021-10-19-564.

[Hig+23] Oscar Higgott, Thomas C. Bohdanowicz, Aleksander Kubica, Steven T. Flammia, and Earl T. Campbell. “Improved Decoding of Circuit Noise and Fragile Boundaries of Tailored Surface Codes”. In: *Phys. Rev. X* 13 (3 July 2023), p. 031007. DOI: 10.1103/PhysRevX.13.031007.

[Hig22] Oscar Higgott. “PyMatching: A Python Package for Decoding Quantum Codes with Minimum-Weight Perfect Matching”. In: *ACM Transactions on Quantum Computing* 3.3 (June 2022). DOI: 10.1145/3505637.

[HNB20] Shilin Huang, Michael Newman, and Kenneth R. Brown. “Fault-tolerant weighted union-find decoding on the toric code”. In: *Phys. Rev. A* 102 (1 July 2020), p. 012419. DOI: 10.1103/PhysRevA.102.012419.

[KD23] Aleksander Kubica and Nicolas Delfosse. “Efficient color code decoders in $d \geq 2$ dimensions from toric code decoders”. In: *Quantum* 7 (Feb. 2023), p. 929. ISSN: 2521-327X. DOI: 10.22331/q-2023-02-21-929.

[Kol09] Vladimir Kolmogorov. “Blossom V: a new implementation of a minimum cost perfect matching algorithm”. In: *Mathematical Programming Computation* 1.1 (July 2009), pp. 43–67. ISSN: 1867-2957. DOI: 10.1007/s12532-009-0002-8.

[KV18] Bernhard H Korte and Jens Vygen. *Combinatorial optimization*. Springer, 2018. ISBN: 978-3-662-56039-6. DOI: 10.1007/978-3-662-56039-6.

[Liy+23] Namitha Liyanage, Yue Wu, Alexander Deters, and Lin Zhong. “Scalable Quantum Error Correction for Surface Codes using FPGA”. In: *2023 IEEE 31st Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. 2023, pp. 217–217. DOI: 10.1109/FCCM57271.2023.00045.

[MBG23] Matt McEwen, Dave Bacon, and Craig Gidney. “Relaxing Hardware Requirements for Surface Code Circuits using Time-dynamics”. In: *Quantum* 7 (Nov. 2023), p. 1172. ISSN: 2521-327X. DOI: 10.22331/q-2023-11-07-1172.

[Men86] Mikhail V Menshikov. “Coincidence of critical points in percolation problems”. In: *Soviet Mathematics Doklady*. Vol. 33. 1986, pp. 856–859.

[MG07] Jiri Matoušek and Bernd Gärtner. *Understanding and using linear programming*. Springer, 2007. ISBN: 978-3-540-30697-9. DOI: 10.1007/978-3-540-30717-4.

[MPT22] Kai Meinerz, Chae-Yeun Park, and Simon Trebst. “Scalable Neural Decoder for Topological Surface Codes”. In: *Phys. Rev. Lett.* 128 (8 Feb. 2022), p. 080505. DOI: 10.1103/PhysRevLett.128.080505.

[SBT11] Martin Suchara, Sergey Bravyi, and Barbara Terhal. “Constructions and noise threshold of topological subsystem codes”. In: *Journal of Physics A: Mathematical and Theoretical* 44.15 (Mar. 2011), p. 155301. DOI: 10.1088/1751-8113/44/15/155301.

[Sko+23] Luka Skoric, Dan E. Browne, Kenton M. Barnes, Neil I. Gillespie, and Earl T. Campbell. “Parallel window decoding enables scalable fault tolerant quantum computation”. In: *Nature Communications* 14.1 (Nov. 2023), p. 7040. ISSN: 2041-1723. DOI: 10.1038/s41467-023-42482-1.

[Sun+23] Neereja Sundaresan, Theodore J. Yoder, Youngseok Kim, Muyuan Li, Edward H. Chen, Grace Harper, Ted Thorbeck, Andrew W. Cross, Antonio D. Córcoles, and Maika Takita. “Demonstrating multi-round subsystem quantum error correction using matching and maximum likelihood decoders”. In: *Nature Communications* 14.1 (May 2023), p. 2852. ISSN: 2041-1723. DOI: 10.1038/s41467-023-38247-5.

[Tan+23] Xinyu Tan, Fang Zhang, Rui Chao, Yaoyun Shi, and Jianxin Chen. “Scalable Surface-Code Decoders with Parallelization in Time”. In: *PRX Quantum* 4 (4 Dec. 2023), p. 040344. DOI: 10.1103/PRXQuantum.4.040344.

[Tea22] Google Quantum AI Team. *Data for "Suppressing quantum errors by scaling a surface code logical qubit"*. Zenodo, July 2022. DOI: 10.5281/zenodo.6804040. URL: `https://doi.org/10.5281/zenodo.6804040`.

[Ter15] Barbara M. Terhal. “Quantum error correction for quantum memories”. In: *Rev. Mod. Phys.* 87 (2 Apr. 2015), pp. 307–346. DOI: 10.1103/RevModPhys.87.307.

[TM17] Giacomo Torlai and Roger G. Melko. “Neural Decoder for Topological Codes”. In: *Phys. Rev. Lett.* 119 (3 July 2017), p. 030501. DOI: 10.1103/PhysRevLett.119.030501.

[Tuc20] David Kingsley Tuckett. “Tailoring surface codes: Improvements in quantum error correction with biased noise”. (qecsim: `https://github.com/qecsim/qecsim`). PhD thesis. University of Sydney, 2020. DOI: 10.25910/x8xw-9077.

[Tut47] W. T. Tutte. “The Factorization of Linear Graphs”. In: *Journal of the London Mathematical Society* s1-22.2 (1947), pp. 107–111. DOI: 10.1112/jlms/s1-22.2.107.

[WLZ22] Yue Wu, Namitha Liyanage, and Lin Zhong. “An interpretation of Union-Find Decoder on Weighted Graphs”. In: (2022). arXiv: `2211.03288 [quant-ph]`.

[Wu22] Yue Wu. *Fusion Blossom*. `https://github.com/yuewuo/fusion-blossom`. 2022.

[WZ23] Yue Wu and Lin Zhong. “Fusion Blossom: Fast MWPM Decoders for QEC”. In: *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*. Vol. 01. 2023, pp. 928–938. DOI: 10.1109/QCE57702.2023.00107.

A Compressed tracking in Union-Find

Compressed tracking can be naturally adapted to the Union-Find decoder [DN21; HNB20], as shown in Figure 15. Each detection event is initialised with a region, which we refer to as a cluster, to be consistent with Ref. [DN21]. Using the same approach as for the compressed tracking in sparse blossom, we can track the detection event that each detector node has been reached from, as well as the observables that have been crossed to reach it. More explicitly, let $\mathcal{S}(u)$ again denote

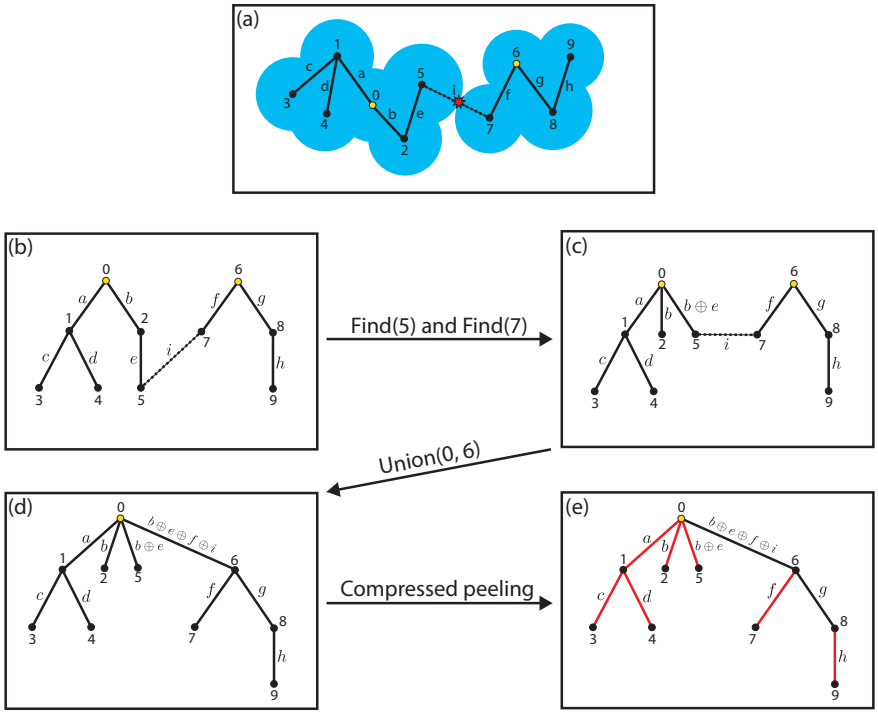


Figure 15: Compressed tracking in Union-Find. (a) Two clusters collide. Each node in the cluster tree denotes a detection event, and each edge is a compressed edge representing a path between the two detection events in the detector graph (not necessarily a shortest path). Each edge is labelled with a letter denoting the bitmask of the observables crossed along the path it represents. This differs from traditional Union-Find implementations, where every detector node in a cluster is a node in the cluster tree. When two clusters collide, we store a compressed edge for the path between detection events along which the collision occurred (the collision edge). (b) The two cluster trees, along with the collision edge (dashed line). (c) After finding the source detection events (5 and 7) involved in the collision, we call Find(5) and Find(7). When using path compression (which here connects node 5 to the root node), we ensure the observable bitmask is kept up-to-date by taking the sum (modulo 2) of bitmasks along the path to the root node. (d) Union(0, 6) adds the smaller cluster as a subtree of the root node of the larger cluster (node 6 becomes a child of node 0). We store the observable bitmask along edge (0, 6), by taking the sum (modulo 2) of the observable bitmasks on edges (0, 5), (6, 7) and the collision edge (5, 7). (e) The combined cluster now has even parity. We use compressed peeling to highlight a set of edges (shown in red) such that each node is incident to an odd number of edges. We take the sum (modulo 2) of the observable bitmasks on these highlighted edges to find the predicted logical observable.

the *source detection event* of a detector node u, and we initialise $\mathcal{S}(x) = x$ for each detection event x at the start of the algorithm. We denote the set of logical observables crossed an odd number of times from a node's source detection event by $l(u)$, and we initialise $l(x)$ as the empty set for each detection event x. We grow a cluster C by adding a node v from an edge $e \coloneqq (u, v)$ on the boundary of C (i.e. an edge such that $u \in C$ and $v \notin C$). As we add v to C, we set $\mathcal{S}(v) = \mathcal{S}(u)$ and $l(v) = l(u) \oplus l(e)$. Recall that $l(e)$ is the set of logical observables flipped by edge e and $\oplus$ denotes the symmetric difference of sets. Since we store $l(e_i)$ as a bitmask for each edge $e_i \in \mathcal{E}$, the symmetric difference of two edges $l(e_i) \oplus l(e_j)$ can be implemented particularly efficiently using a bitwise XOR.

We represent each cluster as a *compressed cluster tree*. Each node in the compressed cluster tree corresponds to a detection event, in contrast to the cluster tree introduced in [DN21], where each node is a detector node. Each edge $c \coloneqq (x, y)$ in the cluster tree is a compressed edge, representing a path P through the detector graph between detection events x and y. In contrast to compressed edges in sparse blossom, this path is in general not the minimum-length path. The compressed edge c is assigned the logical observables crossed an odd number of times by P, denoted $l(c)$ or $l(x, y)$. We check which cluster a detector node u belongs to by calling Find($\mathcal{S}(u)$).

We modify the path compression step of the Find operation such that whenever a path B (consisting of compressed edges) through the cluster tree between two detection events f and g is replaced by a compressed edge $c \coloneqq (f, g)$, the set of logical observables $l(c)$ of the new compressed edge is calculated $l(c) \coloneqq \bigoplus_{c_i \in B} l(c_i)$. A similar modification can be made if path splitting or path halving is used instead of path compression for the Find operation.

The Union operation is adapted in a similar manner. Suppose a cluster C_i collides with a cluster C_j along an edge (u, v). In other words, (u, v) is an edge in the detector graph such that $u \in C_i$ and $v \in C_j$. We construct the collision edge $(\mathcal{S}(u), \mathcal{S}(v))$ from local information at the point of collision, and assign it the set of logical observables $l(\mathcal{S}(u), \mathcal{S}(v)) \coloneqq l(u) \oplus l(v) \oplus l(u, v)$. When we merge cluster C_i with cluster C_j using the Union operation, we add the root node $\mathcal{R}(C_i)$ of the smaller cluster tree (say C_i) as a child of the root node $\mathcal{R}(C_j)$ of the larger cluster tree C_j by adding a compressed edge $c_{ij} \coloneqq (\mathcal{R}(C_i), \mathcal{R}(C_j))$ to the tree. We assign its set of logical observables $l(c_{ij})$ to be $l(c_{ij}) \coloneqq \bigoplus_{c_k \in P_{ij}} l(c_k)$, where P_{ij} is the path through the tree between $\mathcal{R}(C_i)$ and $\mathcal{R}(C_j)$.

Finally, once all clusters have even parity (an even number of detection events, or connected to the boundary), we can apply the peeling algorithm [DZ20] to the compressed cluster trees, which returns a set of compressed edges $\mathcal{P}$. We say the compressed edges in $\mathcal{P}$ are *highlighted* edges in the cluster tree. The set of logical observables that the decoder predicts to have been flippsed is then $\bigoplus_{c_i \in \mathcal{P}} l(c_i)$. This stage is much more efficient than the traditional peeling step of UF, as we do not need to construct a spanning tree in each cluster. Instead, we only run peeling on our compressed representation of the cluster trees. Compressed peeling is linear in the number of compressed edges in the cluster tree. For completeness, we give pseudocode for compressed peeling in Algorithm 1, however this is simply a recursive definition of the peeling algorithm of [DZ20] applied to the case of a compressed cluster tree comprised of a graph of compressed edges joining detection events, rather than to a spanning tree subgraph of a conventional union-find cluster in the detector graph (comprised of detector nodes and detector graph edges). The procedure is recursive and takes as input a node x in the compressed cluster tree, returning p_x and l_x. Here, l_x is the set of logical observables flipped by flipping the highlighted compressed edges that are descendants of x in the cluster tree (a descendant edge of x is an edge in the subtree rooted at x). The auxiliary variable p_x (used in the recursion) is the parity of the number of highlighted child edges of x in the cluster (where a child edge is an edge connecting x to one of its children in the cluster tree). Initially, we assume no node in the cluster tree is connected to the boundary, in which case we initialise $p_x^{init} = even$ for each node x. We run compressed peeling on the root node r to find the set of logical observables l_r flipped by all highlighted edges in the cluster tree. Note that p_r should always be odd for the root node if there is no boundary.

Recall that peeling should find a set of highlighted edges (edges in $\mathcal{P}$) such that each node in the tree is incident to an odd number of highlighted edges. We will first show that l_x (returned by compressed peeling) is indeed the set of logical observables flipped by highlighted edges that are descendants of x, and that p_x is the parity of the number of highlighted child edges of x. Consider the base case that x is a leaf node, in which case it has no child edges or descendants. In this case, compressed peeling correctly sets p_x to $even$ (since we initialise $p_x^{init} = even$) and l_x to the empty set. Now consider the inductive step, where x has children $\mathcal{C}(x)$ in the cluster tree. For each $y \in \mathcal{C}(x)$ we highlight the edge (x, y) if p_y is even, and p_x is set to the parity of the number of these highlighted child edges of x, as required. The main loop of the algorithm sets l_x to

$$l_x = \left(\bigoplus_{y \in \mathcal{C}(x)} l_y \right) \oplus \left(\bigoplus_{(x,w): w \in \mathcal{C}(x), (x,w) \in \mathcal{P}} l(x, w) \right)$$

and if we assume l_y is the set of logical observables flipped by highlighted descendant edges of y then clearly l_x is the set of logical observables flipped by highlighted descendant edges of x. Finally, note that since we apply compressed peeling to a tree, the function is called on each node x exactly once, and we highlight an edge (x, y) to a child y of x if and only if p_y is even. Therefore, each node becomes incident to an odd number of highlighted edges, as required.

We haven't yet considered the boundary. If there is a compressed edge (x, b) in a cluster tree C connecting a detection event x to the boundary b (there can be at most one such edge since a cluster becomes neutral once it hits the boundary), we first add $l(x, b)$ to the solution and remove

(x, b) from C, then we set $p_x^{init} = odd$ before applying compressed peeling to the root node r of the remaining cluster tree, adding the resulting l_r to the solution.

Algorithm 1 Compressed Peeling

```
procedure CompressedPeeling(x)
    p_x ← p_x^init                          ▷ Parity of the number of highlighted child edges of x
    l_x ← {}                                ▷ Observables flipped by highlighted descendant edges of x
    for each child y of x do
        p_y, l_y ← CompressedPeeling(y)
        l_x ← l_x ⊕ l_y
        if p_y is even then
            l_x ← l_x ⊕ l(x, y)             ▷ Compressed edge (x, y) becomes highlighted
            flip p_x
        end if
    end for
    return p_x, l_x
end procedure
```

B Worst-case running time

In this section, we will find a worst-case upper bound of the running time of sparse blossom. Note that this upper bound is likely loose, and furthermore differs greatly from the expected running time of sparse blossom for typical QEC problems, which we believe to be linear in the size of the graph. Let us denote the number of detector nodes by n, the number of edges in the detector graph by m and the number of detection events by q. First, note that each alternating tree always contains exactly one unmatched detection event, in the sense that only a single growing region needs to become matched for the whole tree to shatter into matched regions. Therefore, the alternating tree events that grow the alternating tree, form blossoms or shatter blossoms (events of type a, c, d or e in Figure 4) do not change the number of detection events that remain to be matched. On the other hand, when a tree hits another tree (type b), the number of unmatched detection events reduces by two, and when a tree hits the boundary (type f), or a boundary match (type g), then the number of unmatched detection events reduces by one. We refer to an alternating tree event that reduces the number of unmatched detection events as an *augmentation*, and refer to a portion of the algorithm between consecutive augmentations as a *stage*. Clearly, there can be at most q augmentations and at most q stages.

We now bound the complexity of each augmentation, and of each stage. In each stage, there are at most $O(q)$ blossoms formed or shattered, and at most $O(q)$ matches added to trees, since the same blossom cannot form and then shatter within a stage. To explain more concretely, first note that there are at most $O(q)$ blossoms or trivial regions at any moment in the algorithm. Within a stage, the only alternating tree events that are allowed are of type a, c, d and e (since b, f and g events are augmentations). Once a blossom is growing, the same blossom cannot become a shrinking region and shatter until after an augmentation, since the blossom must first become a match (through an augmentation) and then become an inner node in a type-a event. Therefore at most $O(q)$ blossoms can form in each stage. Finally, within each stage the only blossoms that can shatter, or be added to a tree as part of a match, are those blossoms (or trivial regions) that were already formed at the beginning of the stage, and there at most $O(q)$ of these.

We now consider the worst-case complexity of each of these possible operations within a stage. When a blossom forms, each node it owns updates its cached topmost blossom ancestor and wrapped radius, with cost proportional to the depth of the blossom tree, and this step has complexity $O(nq)$. Additionally, every node owned by the blossom is rescheduled, with $O(m)$ cost. Updating the blossom structure and alternating tree structure (e.g. the compressed edges) is at most $O(q)$. In total, forming a blossom has a running time of at most $O(nq + m)$, and the same upper bound applies for shattering a blossom. When a match is added to an alternating tree, the complexity is $O(m)$ from rescheduling the nodes, which exceeds the $O(q)$ cost of updating the

alternating tree structure. Finally, there is the cost of growing and shrinking regions. In each stage, a node can only be involved in $O(1)$ ARRIVE or LEAVE flooder events: once a region is growing, it (or its topmost blossom ancestor) continues to grow until the next augmentation. The cost of rescheduling nodes due to ARRIVE or LEAVE events in each stage is therefore $O(m)$. We also remind the reader that the tracker has constant-time complexity per operation (since the radix heap has constant-time insert and extract-min operations), so it does not contribute to the overall complexity. Therefore, in each stage the worst-case running time is dominated by the $O(nq^2 + mq)$ cost associated with up to q blossoms forming or shattering. There are q stages, leading to a $O(nq^3 + mq^2)$ worst-case complexity.

C Handling negative edge weights

Recall that an edge weight $\mathbf{w}[i] = \log((1 - \mathbf{p}[i])/\mathbf{p}[i])$ can become negative since we can have $\mathbf{p}[i] > 0.5$. It is therefore necessary to to handle negative edge weights to decode correctly for these error models. For example, consider the distance three repetition code check matrix

$$H = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{pmatrix} \tag{16}$$

with prior distribution $\mathbf{p} = (0.9, 0.9, 0.9)$ and an error $\mathbf{e} = (0, 1, 1)$ with syndrome $\mathbf{s} = H\mathbf{e} = (1, 0, 1)$. The two errors consistent with the syndrome are $(0, 1, 1)$, which has prior probability $0.1 \times 0.9^2 = 0.081$, and $(1, 0, 0)$, which has prior probability $0.9 \times 0.1^2 = 0.009$. Recall that MWPM decoding uses a weights vector $\mathbf{w} \in \mathbb{R}^3$ and finds a correction $\mathbf{c} \in \mathbb{F}_2^3$ satisfying $H\mathbf{c} = \mathbf{s}$ of minimal weight $\sum_i \mathbf{w}[i]\mathbf{c}[i]$. Therefore, it is important that the edge weights $\mathbf{w}[i] = \log((1 - \mathbf{p}[i])/\mathbf{p}[i])$ are indeed negative here, as this leads the decoder to predict the more probable (albeit higher *hamming* weight) $\mathbf{c} = (0, 1, 1)$ instead of the incorrect $\mathbf{c} = (1, 0, 0)$.

We now show how negative edge weights can be handled for the MWPM decoding problem for some check matrix $H \in \mathbb{F}_2^{n \times m}$ with weights vector $\mathbf{w} \in \mathbb{R}^m$ and an error $\mathbf{e} \in \mathbb{F}_2^m$ with syndrome $\mathbf{s} = H\mathbf{e} \in \mathbb{F}_2^n$. Even though sparse blossom only handles non-negative edge weights, we can still perform MWPM decoding when there are negative edge weights using the following procedure, which uses sparse blossom as a subroutine:

1. Define $\mathbf{b} \in \mathbb{F}_2^m$ such that $\mathbf{b}[i] = 1$ if $\mathbf{w}[i] < 0$ and $\mathbf{b}[i] = 0$ otherwise.
2. From $\mathbf{b}$, define adjusted edge weights $\mathbf{v} \in \mathbb{R}^m$ where $\mathbf{v}[i] \coloneqq (1 - 2\mathbf{b}[i])\mathbf{w}[i]$, as well as the adjusted syndrome $\mathbf{s}' = \mathbf{s} + H\mathbf{b}$.
3. Use sparse blossom to find a correction $\mathbf{c}'$ satisfying $H\mathbf{c}' = \mathbf{s}'$ of minimal adjusted weight $\sum_i \mathbf{v}[i]\mathbf{c}'[i]$. Note that the definition of $\mathbf{v}$ guarantees that every element $\mathbf{v}[i]$ is non-negative.
4. Return the correction $\mathbf{c} \coloneqq \mathbf{c}' + \mathbf{b}$, which is guaranteed to satisfy $H\mathbf{c} = \mathbf{s}$ with minimal total weight $\sum_i \mathbf{w}[i]\mathbf{c}[i]$.

We can verify the correctness of this procedure as follows. Firstly, note that $\mathbf{c}'$ satisfies $H\mathbf{c}' = \mathbf{s}'$ if and only if $\mathbf{c}$ satisfies $H\mathbf{c} = \mathbf{s}$. Secondly, note that

$$\sum_i \mathbf{w}[i]\mathbf{c}[i] = \sum_i (\mathbf{w}[i]\mathbf{c}'[i] + \mathbf{w}[i]\mathbf{b}[i] - 2\mathbf{w}[i]\mathbf{b}[i]\mathbf{c}'[i]) = \sum_i \mathbf{v}[i]\mathbf{c}'[i] + \sum_i \mathbf{w}[i]\mathbf{b}[i]. \tag{17}$$

Here, the first equality uses $\mathbf{c} \coloneqq \mathbf{c}' + \mathbf{b}$ and the term $-2\mathbf{w}[i]\mathbf{b}[i]\mathbf{c}'[i]$ accounts for the fact that the sum $\mathbf{c}' + \mathbf{b}$ is taken over the binary field $\mathbb{F}_2$, so any bit set in both $\mathbf{c}'$ and $\mathbf{b}$ should not have its corresponding weight $\mathbf{w}[i]$ contribute to the sum. Therefore, if we find a $\mathbf{c}'$ of minimal adjusted weight $\sum_i \mathbf{v}[i]\mathbf{c}'[i]$ satisfying $H\mathbf{c}' = \mathbf{s}'$, it is guaranteed that the correction $\mathbf{c} \coloneqq \mathbf{c}' + \mathbf{b}$ has minimal weight $\sum_i \mathbf{w}[i]\mathbf{c}[i]$ satisfying $H\mathbf{c} = \mathbf{s}$. Intuitively, wherever we have an error mechanism with high error probability ($> 50\%$), we are re-framing the decoding problem to instead predict if the error mechanism *didn't* occur. The handling of negative edge weights was also discussed in [Hig22], as well as in Corollary 12.12 of [KV18], where what Korte and Vygen refer to as a minimum-weight T-join is equivalent to what we call a MWEM.

D Example of detectors and observables for a repetition code

In Figure 16 we show the detectors, observables, matrices H and L and the detector graph $\mathcal{G}$ for the circuit corresponding to a memory experiment using a [[2,1,2]] bit-flip repetition code with two rounds of syndrome extraction.

Note that for a surface code memory experiment, the circuit and detector graph are instead three-dimensional. In this case, the sensitivity region corresponding to a logical Z observable measurement forms a 2D sheet in spacetime. We denote this logical observable measurement L_Z. This observable L_Z is included in the set $l(e)$ of an edge $e \in \mathcal{E}$ if the error mechanism associated with e flips L_Z. For this to happen, the edge e must pierce the 2D sheet (sensitivity region) and have Z-type detectors (detectors that are parities of Z measurements) at its endpoints.

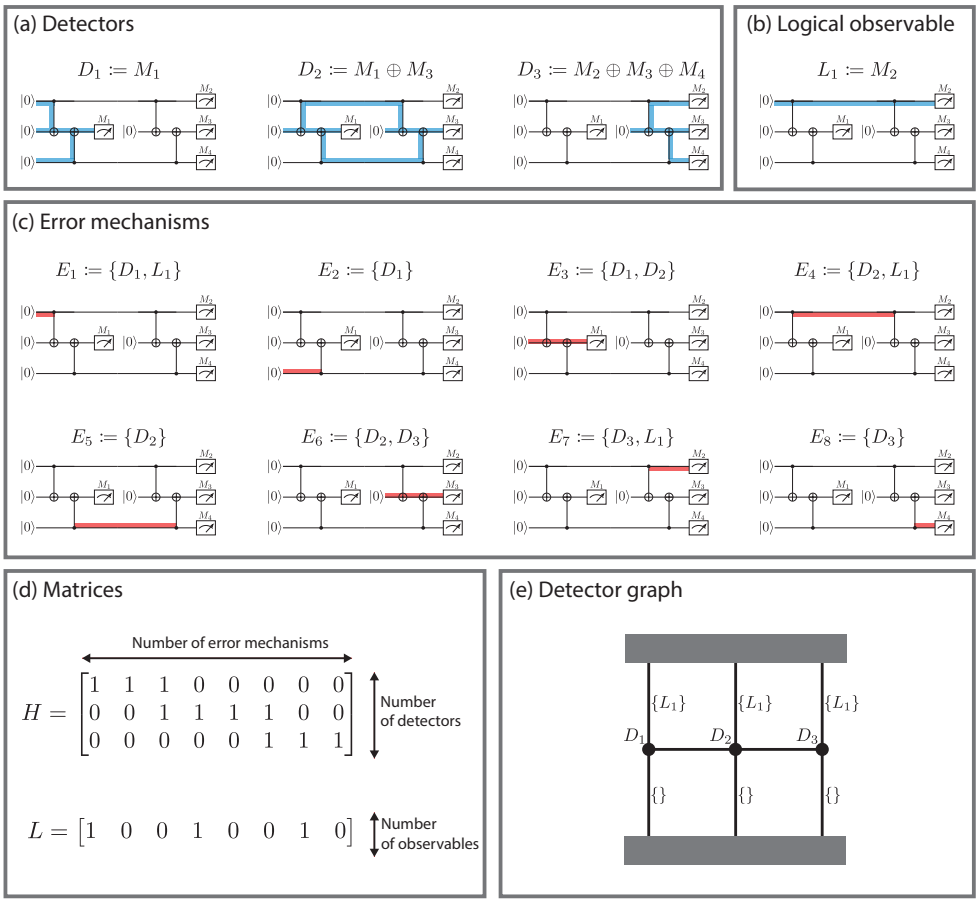


Figure 16: Representations of detectors, logical observables and error mechanisms for the circuit corresponding to a [[2,1,2]] repetition code memory experiment with two rounds of syndrome extraction. We use a bit-flip code (with stabilizer group $\langle ZZ \rangle$) and implement transversal initialisation and measurement in the Z basis. (a) The three detectors in the circuit. The blue highlighted regions are the corresponding Z-type detecting regions [MBG23] (an error within this region which anti-commutes with its type will cause the corresponding detector to flip). (b) A logical Z observable. Here, the blue highlighted region is the Z-type sensitivity region corresponding to the logical Z observable - errors that anti-commute with Z in this region will flip the outcome of the corresponding logical measurement L_1. (c) Given a stochastic Pauli noise model in the circuit, we can characterise errors based on the set of detectors and logical observables they flip. For a standard circuit-level depolarising noise model, there are eight different classes of error mechanism in this circuit, when classified this way. For each error mechanism, we highlight in red a region of the circuit where a single-qubit X error would flip the same detectors and observables. Note that these single-qubit X errors are just *examples* of Pauli errors contributing to the error mechanisms; for example, another Pauli error contributing to E_1 would be a two-qubit YX error after the first CNOT. (d) It is sometimes convenient to describe the detectors, observables and error mechanisms in terms of a detector check matrix H and observable matrix L. Each column in H or L corresponds to an error mechanism. Each row in H corresponds to a detector, with non-zero elements denoting the error mechanisms that flip the detector. Similarly, each row in L corresponds to a logical observable, with non-zero elements denoting the error mechanisms that flip the observable. (e) If H has column weight at most two, we can represent it with a detector graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. Each node $u \in \mathcal{V}$ corresponds to a detector. Each edge $(u, v) \in \mathcal{E}$ corresponds to an error mechanism that flips u and v, each half-edge $(u,) \in \mathcal{E}$ corresponds to an error mechanism that flips just u, denoted here by an edge connected to the boundary. Each edge $e \in \mathcal{E}$ is assigned the set of logical observables $l(e)$ it flips, as well as a weight $w(e) \coloneqq \log((1 - p_e)/p_e)$ (the weight is not shown in the diagram) where p_e is the probability that the corresponding error mechanism occurs in the noise model. Note that here there are two parallel half-edges adjacent to each node; this is a symptom of the fact that the code has distance 2, and therefore has distinct error mechanisms that flip the same set of detectors but different sets of logical observables. In this instance, we can just keep the edge with the lower weight (since our implementation of sparse blossom does not directly handle parallel edges).

Optimal ancilla-free Clifford+T approximation of z-rotations

Neil J. Ross and Peter Selinger
Department of Mathematics and Statistics
Dalhousie University

arXiv:1403.2975v3 [quant-ph] 15 Jun 2016

Abstract

We consider the problem of approximating arbitrary single-qubit z-rotations by ancilla-free Clifford+T circuits, up to given epsilon. We present a fast new probabilistic algorithm for solving this problem optimally, i.e., for finding the shortest possible circuit whatsoever for the given problem instance. The algorithm requires a factoring oracle (such as a quantum computer). Even in the absence of a factoring oracle, the algorithm is still near-optimal under a mild number-theoretic hypothesis. In this case, the algorithm finds a solution of T-count $m + O(\log(\log(1/\varepsilon)))$, where m is the T-count of the second-to-optimal solution. In the typical case, this yields circuit approximations of T-count $3\log_2(1/\varepsilon) + O(\log(\log(1/\varepsilon)))$. Our algorithm is efficient in practice, and provably efficient under the above-mentioned number-theoretic hypothesis, in the sense that its expected runtime is $O(\text{polylog}(1/\varepsilon))$.

1 Introduction

Practical quantum computing requires the fault-tolerant implementation of a universal gate set. The decomposition of arbitrary unitary operators into gates from this fixed set is then an important problem. Most of the common error correction schemes, including most stabilizer codes and surface codes, permit a relatively inexpensive fault-tolerant implementation of gates from the Clifford group. However, since Clifford gates are not universal for quantum computation, at least one non-Clifford gate must be added to the basic gate set to achieve universality. A common choice for this additional gate is the T-gate or $\pi/8$-gate. The T-gate is not the only possible extension of the Clifford group but it is considered to be the most practical one. This is due to the availability of fault-tolerant implementations of the T-gate. For this reason, the Clifford+T gate set is often considered as the most promising candidate for practical quantum computing.

In this paper, we consider the problem of approximating arbitrary single-qubit z-rotations by Clifford+T circuits up to given ε. Until about two years ago, the state-of-the-art algorithm for this problem was the Solovay-Kitaev algorithm, which yields circuits of size $O(\log^c(1/\varepsilon))$, where $c > 3$. At the other end of the spectrum are algorithms based on exhaustive search. While such algorithms achieve optimal circuit sizes, they have exponential runtimes. For example, the algorithm of [5] is feasible up to $\varepsilon \approx 10^{-4}$. Even the improved algorithm of [11], which combines search-based and other methods and achieves optimal T-counts, still has exponential runtime which makes it feasible only for precisions up to $\varepsilon \approx 10^{-17}$.

Within the last two years, a new generation of efficient number-theoretic algorithms have been proposed for the approximate synthesis problem, achieving circuit sizes of $O(\log(1/\varepsilon))$ with polynomial runtime. Unlike the Solovay-Kitaev algorithm, which is based on a geometric method of successive approximations, these new algorithms are based on solving Diophantine equations. The first such algorithm was due to Kliuchnikov, Maslov, and Mosca [9]. It uses a small number of ancilla qubits to approximate a given single-qubit operator. An improved algorithm was given in [14], which uses no ancillas and achieves T-counts of $K + 4\log_2(1/\varepsilon)$ for approximating arbitrary z-rotations. This compares to the information-theoretic lower bound of $K + 3\log_2(1/\varepsilon)$.

In this paper, we present a fast new probabilistic algorithm for solving the single-qubit approximate synthesis problem. Our algorithm is optimal in an absolute sense, i.e., it finds the shortest possible circuit whatsoever for any given problem instance. To achieve this optimality, the algorithm requires an oracle for integer factoring. Of course, a quantum computer can serve as such an oracle by using Shor's algorithm [15]. But even in the absence of a factoring oracle, our algorithm is still nearly optimal: In this case, under a mild number-theoretic assumption, we can prove that the algorithm finds a solution of T-count $m + O(\log(\log(1/\varepsilon)))$, where m is the T-count of the second-to-optimal solution. In the typical case, m is given by the information-theoretic lower bound $3\log_2(1/\varepsilon)$. Therefore our algorithm, in the absence of a factoring oracle, yields circuit approximations of T-count $3\log_2(1/\varepsilon) + O(\log(\log(1/\varepsilon)))$ in the typical case.

We note that our algorithm is optimal only for the *specific* problem of approximating a given z-rotation by a linear sequence of single-qubit Clifford+T gates. It is already known that even smaller gate counts and/or circuit depths are achievable using additional techniques, such as ancillas, measurements, or state distillation [4, 16, 3, 2]. In particular, the methods of [3] produce so-called repeat-until-success circuits whose expected T-count is below the information theoretic lower bound for deterministic circuits.

It is likely that in the future, the existence of an efficient approximate synthesis algorithm will be considered an essential requirement for any universal gate set proposed for practical quantum computing, of similar importance, say, as the existence of a fault-tolerant implementation. Our algorithm is specialized to the Clifford+T gate set which is arguably the most relevant gate set to practical quantum computing. However, similar number-theoretic methods are also applicable to certain other universal gate sets. For example, Bocharov et al. [1] gave an efficient synthesis algorithm for the Clifford+V gate set that achieves gate counts linear in $\log(1/\varepsilon)$, and Kliuchnikov et al. [8] did the same for the $\langle \mathcal{F}, \mathcal{T} \rangle$ gate set. One may reasonably expect these gate sets to be amenable to the same kind of optimal synthesis that we provide here for the Clifford+T gate set.

2 Overview

Recall that the single-qubit Clifford group is generated by the Hadamard gate H, the phase gate S, and the scalar $\omega = e^{i\pi/4}$. By adding the non-Clifford operator T, one obtains a universal gate set for quantum computing.

$$\omega = e^{i\pi/4}, \quad H = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}, \quad T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}.$$

Our goal is to approximate an arbitrary z-rotation

$$R_z(\theta) = e^{-i\theta Z/2} = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}$$

by a Clifford+T operator up to given $\varepsilon > 0$. By a result of Kliuchnikov, Maslov, and Mosca [10], a unitary 2×2-operator can be exactly written as a product of Clifford+T operators if and only if all of its matrix entries belong to the ring $\mathbb{D}[\omega] = \mathbb{Z}[\frac{1}{\sqrt{2}}, i]$. Our strategy is therefore to approximate $R_z(\theta)$ by a unitary operator of the form

$$U = \begin{pmatrix} u & -t^\dagger \\ t & u^\dagger \end{pmatrix},$$

where $u, t \in \mathbb{D}[\omega]$. This problem can be solved in two stages:

(1) find a suitable candidate $u \in \mathbb{D}[\omega]$ that is a good approximation of $e^{-i\theta/2}$;

(2) solve the Diophantine equation $u^\dagger u + t^\dagger t = 1$, to ensure that U is unitary.

Problem (2) can be solved by standard number-theoretic methods. In the interest of self-containedness, we summarize these methods in Section 6 and Appendix C. In general, solving the Diophantine equation in (2) requires the ability to factor large integers. However, if no efficient factoring method is available, then (modulo a mild number-theoretic assumption), the Diophantine equation can still be solved with large enough probability to ensure that at most $O(\log(1/\varepsilon))$ candidates need to be tried.

The main new technical innovation of this paper is a new and optimal solution to problem (1). It turns out that the "suitability" of a candidate u can be expressed as a problem of the form $u \in A$ and $u^\bullet \in B$, where A and B are fixed convex subsets of the complex plane depending only on θ and ε, and $(-)^\bullet$ is the automorphism of the ring $\mathbb{D}[\omega]$ obtained by mapping $\sqrt{2}$ to $-\sqrt{2}$. We call such a problem a *two-dimensional grid problem*. In Sections 4 and 5, we formulate a general algorithm for solving one- and two-dimensional grid problems efficiently. Our algorithm proceeds by transforming the sets A and B using invertible linear operators that preserve solutions to the given problem, which we call *grid operators*. We show that for any two convex sets A and B one can construct a grid operator G such that the action of G on A and B normalizes the grid problem in an appropriate sense. The main technical ingredient that makes our solution efficient is an iterative process for constructing G, which is detailed in Appendix A.

The rest of this paper is organized as follows. In Section 3, we review some basic notions from algebra. We discuss one- and two-dimensional grid problems in Sections 4 and 5, respectively. In Section 6, we show how to solve the relevant Diophantine equation. A detailed description of the main synthesis algorithm is given in Section 7. In Section 8, we analyze the algorithm's correctness, optimality, and complexity. Some experimental results are given in Section 10. For better readability of the main body of the paper, certain technical results are proved in the appendices.

3 Some algebra

We introduce some notation and algebraic prerequisites. The set of natural numbers, including 0, is denoted by $\mathbb{N}$, the ring of integers is denoted by $\mathbb{Z}$, and we let $\omega = e^{i\pi/4} = (1+i)/\sqrt{2}$.

Definition 3.1. *(Extensions of $\mathbb{Z}$)* We are interested in the following rings of algebraic integers:

- $\mathbb{Z}[\sqrt{2}] = \{a + b\sqrt{2} \mid a, b \in \mathbb{Z}\}$, the ring of *quadratic integers with radicand 2*;
- $\mathbb{Z}[\omega] = \{a\omega^3 + b\omega^2 + c\omega + d \mid a, b, c, d \in \mathbb{Z}\}$, the ring of *cyclotomic integers of degree 8*;
- $\mathbb{D} = \mathbb{Z}[\frac{1}{2}] = \{\frac{a}{2^k} \mid a \in \mathbb{Z}, k \in \mathbb{N}\}$, the ring of *dyadic fractions*;
- $\mathbb{D}[\sqrt{2}] = \mathbb{Z}[\frac{1}{\sqrt{2}}] = \{a + b\sqrt{2} \mid a, b \in \mathbb{D}\}$; and
- $\mathbb{D}[\omega] = \mathbb{Z}[\frac{1}{\sqrt{2}}, i] = \{a\omega^3 + b\omega^2 + c\omega + d \mid a, b, c, d \in \mathbb{D}\}$.

We have the inclusions $\mathbb{Z} \subseteq \mathbb{Z}[\sqrt{2}] \subseteq \mathbb{Z}[\omega]$ and $\mathbb{D} \subseteq \mathbb{D}[\sqrt{2}] \subseteq \mathbb{D}[\omega]$. Moreover, $\mathbb{Z} \subseteq \mathbb{D}$, $\mathbb{Z}[\sqrt{2}] \subseteq \mathbb{D}[\sqrt{2}]$, and $\mathbb{Z}[\omega] \subseteq \mathbb{D}[\omega]$. Finally, $\mathbb{Z}[\sqrt{2}]$ and $\mathbb{Z}[\omega]$ are dense in $\mathbb{R}$ and $\mathbb{C}$, respectively.

Definition 3.2. *(Automorphisms)* The following maps are automorphisms of $\mathbb{D}[\omega]$:

- *Complex conjugation*, which we denote $(-)^\dagger$, acts on an arbitrary element of $\mathbb{D}[\omega]$ or $\mathbb{Z}[\omega]$ as follows:
$$(a\omega^3 + b\omega^2 + c\omega + d)^\dagger = -c\omega^3 - b\omega^2 - a\omega + d.$$
- $\sqrt{2}$*-conjugation*, which we denote $(-)^\bullet$, acts on an arbitrary element of $\mathbb{D}[\omega]$ or $\mathbb{Z}[\omega]$ as follows:
$$(a\omega^3 + b\omega^2 + c\omega + d)^\bullet = -a\omega^3 + b\omega^2 - c\omega + d$$

The action of $(-)^\bullet$ on an element of $\mathbb{D}[\sqrt{2}]$ or $\mathbb{Z}[\sqrt{2}]$ is given by $(a + b\sqrt{2})^\bullet = a - b\sqrt{2}$. In particular, this implies that if $t = a + b\sqrt{2}$ is an element of $\mathbb{D}[\sqrt{2}]$ (resp. $\mathbb{Z}[\sqrt{2}]$), then $t^\bullet t = a^2 - 2b^2$ is an element of $\mathbb{D}$ (resp. $\mathbb{Z}$).

Remark 3.3. If α and β are two distinct elements of $\mathbb{Z}[\sqrt{2}]$, then the following inequality holds:

$$|\alpha - \beta| \cdot |\alpha^\bullet - \beta^\bullet| \geqslant 1, \tag{1}$$

This follows from the fact that for $t \in \mathbb{Z}[\sqrt{2}]$, $t^\bullet t$ is an integer and $t^\bullet t = 0$ if and only if $t = 0$.

Definition 3.4. Let $t \in \mathbb{D}[\omega]$ and $k \in \mathbb{N}$. If $\sqrt{2}^k t \in \mathbb{Z}[\omega]$, then we say that k is a *denominator exponent* of t. The smallest such $k \geqslant 0$ is the *least denominator exponent* of t. For $k \in \mathbb{N}$, the elements of $\mathbb{D}[\sqrt{2}]$ (resp. $\mathbb{D}[\omega]$) having k as a denominator exponent form a ring, denoted $\frac{1}{\sqrt{2}^k}\mathbb{Z}[\sqrt{2}]$ (resp. $\frac{1}{\sqrt{2}^k}\mathbb{Z}[\omega]$).

Definition 3.5. We frequently refer to the following elements of $\mathbb{Z}[\sqrt{2}]$ and $\mathbb{Z}[\omega]$:

- $\lambda = 1 + \sqrt{2} \in \mathbb{Z}[\sqrt{2}]$ and
- $\delta = 1 + \omega \in \mathbb{Z}[\omega]$.

Remark 3.6. The element λ is invertible in the ring $\mathbb{Z}[\sqrt{2}]$, with inverse $\lambda^{-1} = -1 + \sqrt{2} = -\lambda^\bullet$. The element δ of the ring $\mathbb{Z}[\omega]$ satisfies $\delta^2 = \lambda\omega\sqrt{2}$ and $\delta^\dagger\delta = \lambda\sqrt{2}$.

4 One-dimensional grid problems

Definition 4.1. Let B be a set of real numbers. The *(real) grid* for B is the set

$$\mathrm{grid}(B) = \{\alpha \in \mathbb{Z}[\sqrt{2}] \mid \alpha^\bullet \in B\}. \tag{2}$$

When B is clear from the context, we refer to the elements of this set as *grid points*.

In the following, we will only be interested in the case where B is a closed interval $[y_0, y_1]$ with $y_0 < y_1$. In this case, the grid is discrete and infinite. It is discrete because the distance between grid points is bounded below by (1). And it is infinite by the density of $\mathbb{Z}[\sqrt{2}]$: there are infinitely many points $\beta \in B \cap \mathbb{Z}[\sqrt{2}]$, and for each of them, $\beta^\bullet$ is a grid point.

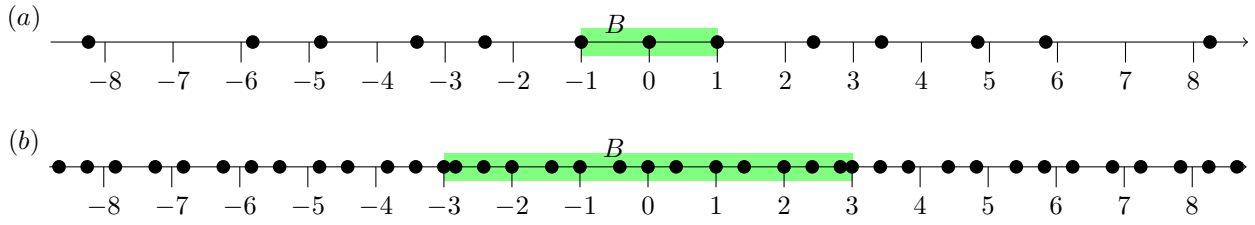


Figure 1: The real grid for two different intervals B. In both cases, the interval B is shown in green, and grid points are shown as black dots.

Example 4.2. Figure 1 illustrates the grids for the intervals $[-1,1]$ and $[-3,3]$, respectively. For example, the first few non-negative points in $\mathrm{grid}([-1,1])$ are 0, 1, $1+\sqrt{2}$, $2+\sqrt{2}$, $2+2\sqrt{2}$, $3+2\sqrt{2}$, and $4+3\sqrt{2}$. As one would expect, the grid for $[-3,3]$ is about three times denser than that for $[-1,1]$. We also note that $B \subseteq B'$ implies $\mathrm{grid}(B) \subseteq \mathrm{grid}(B')$.

Definition 4.3. Let A and B be sets of real numbers. The *one-dimensional grid problem* for A and B is the following:

$$\textbf{One-dimensional grid problem: } \text{Find } \alpha \in \mathbb{Z}[\sqrt{2}] \text{ satisfying } \alpha \in A \text{ and } \alpha^\bullet \in B. \tag{3}$$

Note that (3) can be equivalently written as $\alpha \in A \cap \mathrm{grid}(B)$. In other words, the grid problem is to find points in some given set A that belong to the grid for B. We also refer to the conditions $\alpha \in A$ and $\alpha^\bullet \in B$ as *grid constraints*.

In the case where A and B are finite intervals, the grid problem is guaranteed to have a finite number of solutions. We recall the following facts from [14]:

Lemma 4.4. *Let $A = [x_0, x_1]$ and $B = [y_0, y_1]$ be closed real intervals, such that $x_1 - x_0 = \delta$ and $y_1 - y_0 = \Delta$. If $\delta\Delta < 1$, then the grid problem* (3) *has at most one solution. If $\delta\Delta \geqslant (1+\sqrt{2})^2$, then the grid problem* (3) *has at least one solution.*

Proof. Lemmas 16 and 17 of [14]. □

Proposition 4.5. *There is an algorithm for enumerating all solutions of the one-dimensional grid problem for closed intervals $A = [x_0, x_1]$ and $B = [y_0, y_1]$. Moreover, the algorithm is efficient in the sense that it only requires a constant number of arithmetic operations per solution produced.*

Proof. It was already noted in [14, Lemma 17] that there is an efficient algorithm for computing one solution. To see that we can efficiently enumerate all solutions, let $\delta = x_1 - x_0$ and $\Delta = y_1 - y_0$ as before. Recall that $\lambda = 1+\sqrt{2}$ and that $\lambda^{-1} = -\lambda^\bullet$. The grid problem for the sets A and B is equivalent to the grid problem for $\lambda^{-1}A$ and $-\lambda B$, because $\alpha \in A$ and $\alpha^\bullet \in B$ hold if and only if $\lambda^{-1}\alpha \in \lambda^{-1}A$ and $(\lambda^{-1}\alpha)^\bullet \in -\lambda B$. Using such rescaling, we may without loss of generality assume that $\lambda^{-1} \leqslant \delta < 1$.

Now consider any solution $\alpha = a + b\sqrt{2} \in \mathbb{Z}[\sqrt{2}]$. From $\alpha \in [x_0, x_1]$, we know that $x_0 - b\sqrt{2} \leqslant a \leqslant x_1 - b\sqrt{2}$. But since $x_1 - x_0 < 1$, it follows that for any $b \in \mathbb{Z}$, there is at most one $a \in \mathbb{Z}$ yielding a solution. Moreover, we note that $b = (\alpha - \alpha^\bullet)/\sqrt{2}^3$, so that any solution satisfies $(x_0 - y_1)/\sqrt{2}^3 \leqslant b \leqslant (x_1 - y_0)/\sqrt{2}^3$. The algorithm then proceeds by enumerating all the integers b in the interval $[(x_0 - y_1)/\sqrt{2}^3, (x_1 - y_0)/\sqrt{2}^3]$. For each such b, find the unique integer a (if any) in the interval $[x_0 - b\sqrt{2}, x_1 - b\sqrt{2}]$. Finally, check if $a + b\sqrt{2}$ is a solution. The runtime is governed by the number of $b \in \mathbb{Z}$ that need to be checked, of which there are at most $O(y_1 - y_0) = O(\delta\Delta)$. As a consequence of Lemma 4.4, the total number of solutions is at least $\Omega(\delta\Delta)$, and so the algorithm is efficient. □

Remark 4.6. For the purposes of this paper, by an *arithmetic operation* we mean addition, subtraction, multiplication, division, exponentiation, and logarithm.

Remark 4.7. Since the inputs to the algorithm are real intervals, if we were to give a rigorous complexity-theoretic account, we should also clarify how these intervals are specified (for example, with rational endpoints, endpoints as computable real numbers, etc.). For our purposes, the manner in which an interval $A = [x_0, x_1]$ is specified as an input to the algorithm does not matter very much; it would be sufficient, for example, to assume that we are given rational bounds a, b with $a < x_0 < x_1 < b$, such that $b - a$ exceeds $x_1 - x_0$ by at most a fixed constant factor, as well as a procedure for deciding whether any given point of $\mathbb{D}[\sqrt{2}]$ is in A or not.

5 Two-dimensional grid problems

Recall that $\mathbb{Z}[\omega]$ is a subset of the complex numbers. In what follows, it is often convenient to identify the complex numbers with the Euclidean plane $\mathbb{R}^2$, so we will often interchangeably regard $\mathbb{Z}[\omega]$ as a subset of $\mathbb{C}$ and of $\mathbb{R}^2$.

Definition 5.1. Let B be a subset of $\mathbb{R}^2$. The *(complex) grid* for B is the set

$$\mathrm{Grid}(B) = \{u \in \mathbb{Z}[\omega] \mid u^\bullet \in B\}. \tag{4}$$

We will only be interested in the case where B is a bounded convex set with non-empty interior. In this case, the grid is discrete and infinite, just as in the one-dimensional case.

Example 5.2. Figure 2 illustrates the complex grids for several different convex sets B. Note that the grid has a 90-degree symmetry in (a), a 45-degree symmetry in (b), and a 180-degree symmetry in (c).

Definition 5.3. Let A and B be subsets of $\mathbb{R}^2$. The *two-dimensional grid problem* for A and B is the following:

$$\textbf{Two-dimensional grid problem: } \text{Find } u \in \mathbb{Z}[\omega] \text{ satisfying } u \in A \text{ and } u^\bullet \in B. \tag{5}$$

As in the one-dimensional case, the grid problem can be understood as looking for points in the intersection of the set A and the grid for B. Our goal will be to prove a two-dimensional analogue of Proposition 4.5, namely, that there is an efficient algorithm which, given two bounded convex subsets A and B of $\mathbb{R}^2$ with non-empty interior, enumerates all solutions of the two-dimensional grid problem for A and B.

However, the proof is more complicated than in the one-dimensional case. We will consider several special cases before solving the general problem in Section 5.6.

Remark 5.4. By analogy with Remark 4.7, we should indicate what it means for a bounded convex subset A of $\mathbb{R}^2$ to be "given" as the input to an algorithm. Again, the details of this do not matter much. For our purposes, it will suffice to assume that a convex set is given along with the following information:

- a convex polygon enclosing A, say with rational vertices, and such that the area of the polygon exceeds that of A by at most a fixed constant factor;
- a method to decide, for any given point of $\mathbb{D}[\omega]$, whether it is in A or not; and
- a method to compute the intersection of A with any straight line in $\mathbb{D}[\omega]$. More precisely, given any straight line parameterized as $L(t) = p + tq$, with $p, q \in \mathbb{D}[\omega]$, we can effectively determine the interval $\{t \mid L(t) \in A\}$ in the sense of Remark 4.7.

5.1 Upright rectangles

A special case of the two-dimensional grid problem arises when both A and B are *upright rectangles*, by which we mean sets of the form $[x_0, x_1] \times [y_0, y_1]$. If A and B are upright rectangles, then the two-dimensional grid problem can easily be reduced to the one-dimensional case. We start with a lemma characterizing $\mathbb{Z}[\omega]$.

Lemma 5.5. *A complex number u is in $\mathbb{Z}[\omega]$ if and only it can be written of the form $u = \alpha + \beta i$ or of the form $u = \alpha + \beta i + \omega$, where $\alpha, \beta \in \mathbb{Z}[\sqrt{2}]$.*

Proof. The right-to-left implication is trivial. For the left-to-right implication, let $u = a\omega^3 + b\omega^2 + c\omega + d$, where $a, b, c, d \in \mathbb{Z}$. Noting that $\omega = \frac{1+i}{\sqrt{2}}$, we have

$$u = (d + \frac{c-a}{2}\sqrt{2}) + (b + \frac{c+a}{2}\sqrt{2})i.$$

If $c - a$ (and therefore $c + a$) is even, then u is of the first form, with $\alpha = d + \frac{c-a}{2}\sqrt{2}$ and $\beta = b + \frac{c+a}{2}\sqrt{2}$. If $c - a$ (and therefore $c + a$) is odd, then u is of the second form, with $\alpha = d + \frac{c-a-1}{2}\sqrt{2}$ and $\beta = b + \frac{c+a-1}{2}\sqrt{2}$. □

Lemma 5.6. *There is an algorithm which, given a pair of upright rectangles A and B, enumerates all solutions of the two-dimensional grid problem for A and B. Moreover, the algorithm requires only a constant number of arithmetic operations per solution produced.*

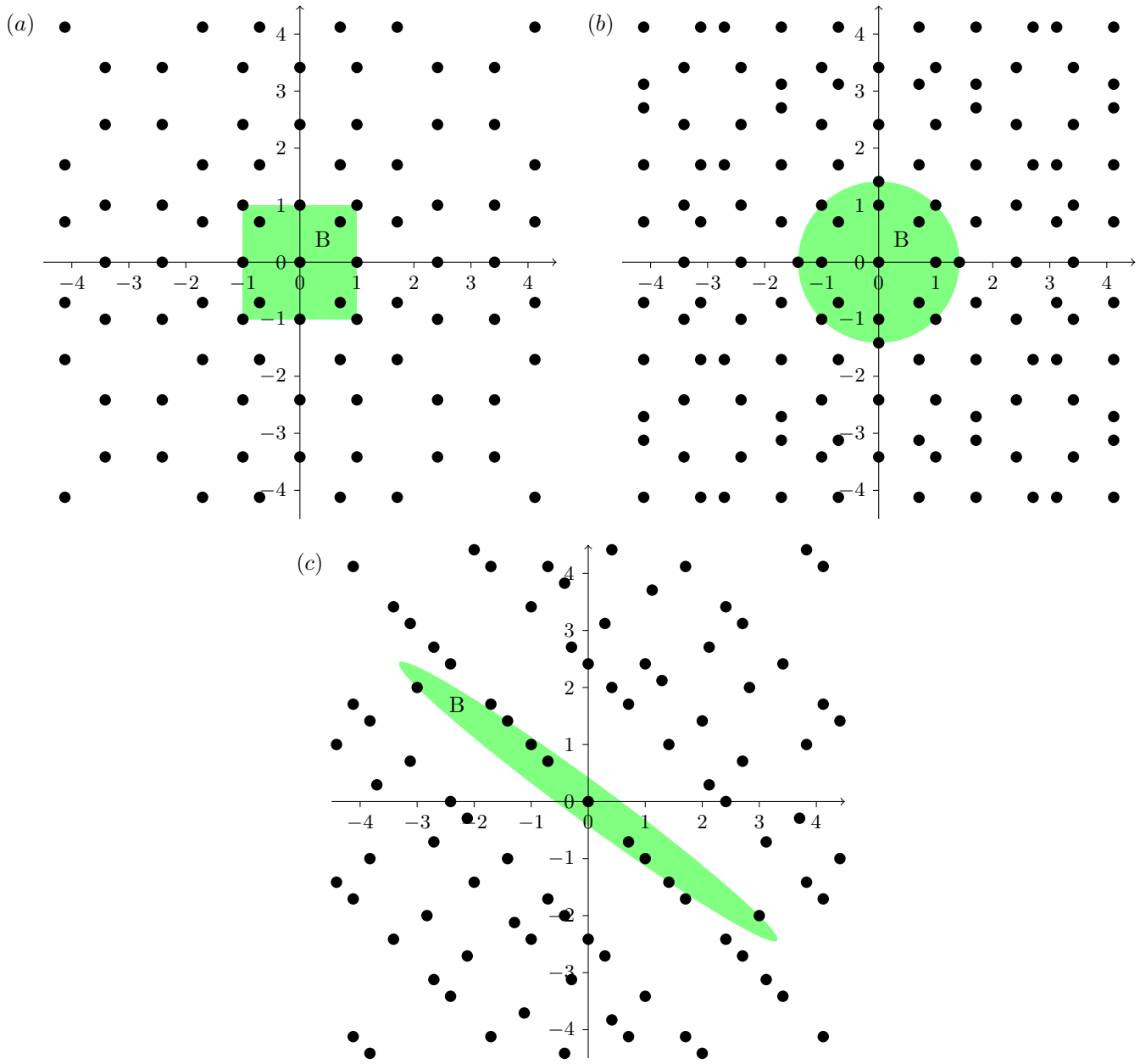


Figure 2: The complex grid for three different convex sets B. In each case, the set B is shown in green, and grid points are shown as black dots. (a) $B = [-1, 1]^2$. (b) $B = \{(x, y) \mid x^2 + y^2 \leqslant 2\}$. (c) $B = \{(x, y) \mid 6x^2 + 16xy + 11y^2 \leqslant 2\}$.

Proof. By assumption, $A = A_x \times A_y$ and $B = B_x \times B_y$, where A_x, A_y, B_x, and B_y are closed intervals. By Lemma 5.5, any potential solution is of the form $u = \alpha + \beta i$ or $u = \alpha + \beta i + \omega$, where $\alpha, \beta \in \mathbb{Z}[\sqrt{2}]$. When $u = \alpha + \beta i$, then $u^\bullet = \alpha^\bullet + \beta^\bullet i$. Therefore, the two-dimensional grid constraints $u \in A$ and $u^\bullet \in B$ are equivalent to the one-dimensional constraints $\alpha \in A_x$, $\alpha^\bullet \in B_x$ and $\beta \in A_y$, $\beta^\bullet \in B_y$. On the other hand, when $u = \alpha + \beta i + \omega$, let $v = u - \omega = \alpha + \beta i$. Then $v^\bullet = u^\bullet + \omega$, and the constraints $u \in A$ and $u^\bullet \in B$ are equivalent to $v \in A - \omega$ and $v^\bullet \in B + \omega$, which reduces to the one-dimensional constraints $\alpha \in A_x - \frac{1}{\sqrt{2}}$, $\alpha^\bullet \in B_x + \frac{1}{\sqrt{2}}$ and $\beta \in A_y - \frac{1}{\sqrt{2}}$, $\beta^\bullet \in B_y + \frac{1}{\sqrt{2}}$. In both cases, the solutions to the one-dimensional constraints can be efficiently enumerated by Proposition 4.5. □

5.2 Upright sets

We can generalize the method of Section 5.1 to convex sets that are *close* to upright rectangles in a suitable sense.

Definition 5.7. Let A be a bounded convex subset of $\mathbb{R}^2$. The *bounding box* of A, denoted $\mathrm{BBox}(A)$, is the smallest set of the form $[x_0, x_1] \times [y_0, y_1]$ that contains A. The *uprightness* of A, denoted $\mathrm{up}(A)$, is defined to be the ratio of the area of A to the area of its bounding box:

$$\mathrm{up}(A) = \frac{\mathrm{area}(A)}{\mathrm{area}(\mathrm{BBox}(A))}. \tag{6}$$

Therefore, the uprightness is a real number between 0 and 1. We say that A is *M-upright* if $\mathrm{up}(A) \geqslant M$.

Lemma 5.8. *There exists an algorithm which, given a pair A, B of convex M-upright sets, enumerates all solutions of the two-dimensional grid problem for A and B. Moreover, the algorithm requires $O(1/M^2)$ arithmetic operations per solution produced. In particular, when $M > 0$ is fixed, it requires only a constant number of operations per solution.*

Proof. By Lemma 5.6, we can efficiently enumerate the solutions of the grid problem for $\mathrm{BBox}(A)$ and $\mathrm{BBox}(B)$. Moreover, as shown in the proof of Lemma 5.6, the solutions are arranged in rows and columns. For each such candidate solution u, we only need to check whether u is also a solution for A and B. To establish the efficiency of the algorithm, we need to ensure that the total number of solutions is not too small in relation to the total number of candidates produced. To see this, note that, with the exception of trivial cases, when the number of rows or columns is very small, M-uprightness and convexity ensure that the proportion of candidates u that are solutions for A and B is approximately $M^2 : 1$. Therefore, the runtime per solution differs from that of Lemma 5.6 by at most a factor of $O(1/M^2)$. □

Example 5.9. Figure 3 shows three different examples of grid problems. Each example uses the same set $B = [-1, 1] \times [-1, 1]$, shown in green. The grid for B is shown as black dots. The sets A_i are shown in red, for $i = 1, 2, 3$, and their bounding boxes are shown in outline. The typical case of an upright set is A_1. Here, a fixed proportion of grid points from the bounding box of A_1 are elements of A_1. The exceptional case of an upright set is A_2. Its bounding box spans only two columns of the grid. Therefore, although the bounding box contains many grid points, A_2 does not. However, this case is easily dealt with, by solving a one-dimensional grid problem for each of the grid columns separately. Finally, the set A_3 is not upright. In this case, Lemma 5.8 is not helpful, and a priori, it could be a difficult problem to find grid points in A_3.

5.3 Grid operators

The method of Section 5.2 can be further generalized by using certain linear transformations to turn non-upright sets into upright sets. The linear transformations that are useful for this purpose are *special grid operators*:

Definition 5.10. As before, we regard $\mathbb{Z}[\omega]$ as a subset of $\mathbb{R}^2$. A real linear operator $G : \mathbb{R}^2 \to \mathbb{R}^2$ is called a *grid operator* if $G(\mathbb{Z}[\omega]) \subseteq \mathbb{Z}[\omega]$. Moreover, a grid operator G is called *special* if it has determinant ± 1.

Grid operators are characterized by the following lemma.

Lemma 5.11. *Let $G : \mathbb{R}^2 \to \mathbb{R}^2$ be a linear operator, which we can identify with a real 2×2-matrix with real entries. Then G is a grid operator if and only if it is of the form*

$$G = \begin{bmatrix} a + \frac{a'}{\sqrt{2}} & b + \frac{b'}{\sqrt{2}} \\ c + \frac{c'}{\sqrt{2}} & d + \frac{d'}{\sqrt{2}} \end{bmatrix}, \tag{7}$$

where $a, b, c, d, a', b', c', d'$ are integers satisfying $a + b + c + d \equiv 0 \pmod 2$ and $a' \equiv b' \equiv c' \equiv d' \pmod 2$.

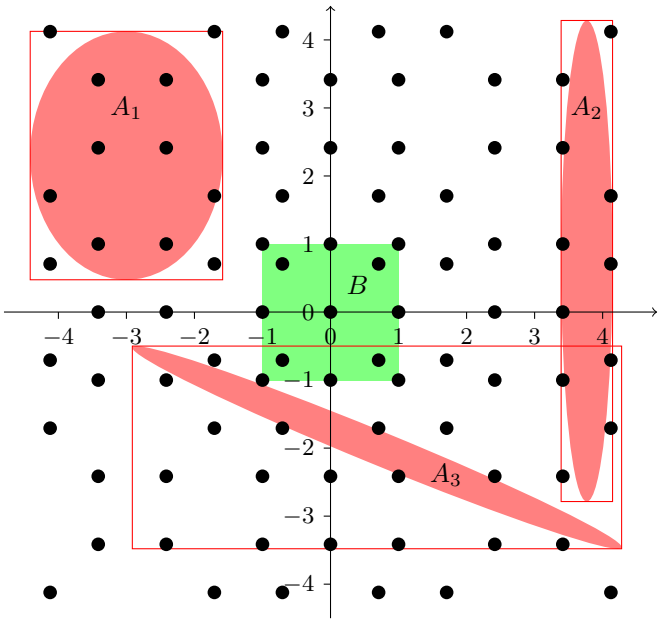


Figure 3: Grid problems for upright and non-upright sets

Proof. By Lemma 5.5, we know that a vector $u \in \mathbb{R}^2$ is in $\mathbb{Z}[\omega]$ if and only if it can be written of the form

$$u = \begin{bmatrix} x_1 + \frac{x_2}{\sqrt{2}} \\ y_1 + \frac{y_2}{\sqrt{2}} \end{bmatrix}, \tag{8}$$

where x_1, x_2, y_1, y_2 are integers and $x_2 \equiv y_2 \pmod 2$. A simple computation then shows that every operator of the form (7) is a grid operator. For the converse, consider an arbitrary grid operator G. We prove the claim by applying G to the three particular points $\left[\begin{smallmatrix}1\\0\end{smallmatrix}\right]$, $\left[\begin{smallmatrix}0\\1\end{smallmatrix}\right]$, and $\frac{1}{\sqrt{2}}\left[\begin{smallmatrix}1\\1\end{smallmatrix}\right] \in \mathbb{Z}[\omega]$. From $G\left[\begin{smallmatrix}1\\0\end{smallmatrix}\right] \in \mathbb{Z}[\omega]$ and $G\left[\begin{smallmatrix}0\\1\end{smallmatrix}\right] \in \mathbb{Z}[\omega]$, it follows that the columns of G are of the form (8), so that G is of the form (7), with integers $a, b, c, d, a', b', c', d'$ satisfying $a' \equiv c' \pmod 2$ and $b' \equiv d' \pmod 2$. Moreover, we have

$$G\begin{bmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{bmatrix} = \begin{bmatrix} \frac{a'+b'}{2} + \frac{a+b}{\sqrt{2}} \\ \frac{c'+d'}{2} + \frac{c+d}{\sqrt{2}} \end{bmatrix} \in \mathbb{Z}[\omega],$$

which implies $a+b \equiv c+d \pmod 2$ and $a'+b' \equiv c'+d' \equiv 0 \pmod 2$. Together, these conditions imply $a+b+c+d \equiv 0 \pmod 2$ and $a' \equiv b' \equiv c' \equiv d' \pmod 2$, as claimed. □

Remark 5.12. The composition of two (special) grid operators is again a (special) grid operator. If G is a special grid operator, then G is invertible and G^{-1} is a special grid operator. If G is a (special) grid operator, then $G^\bullet$ is a (special) grid operator, defined by applying $(-)^\bullet$ separately to each matrix entry, and satisfying $G^\bullet u^\bullet = (Gu)^\bullet$.

The interest of special grid operators lies in the following fact:

Proposition 5.13. *Let G be a special grid operator, and let A and B be subsets of $\mathbb{R}^2$. Define*

$$G(A) = \{Gu \mid u \in A\},$$
$$G^\bullet(B) = \{G^\bullet u \mid u \in B\}.$$

Then $u \in \mathbb{Z}[\omega]$ is a solution to the two-dimensional grid problem for A and B if and only if Gu is a solution to the two-dimensional grid problem for $G(A)$ and $G^\bullet(B)$. In particular, the two-dimensional grid problem for A and B is computationally equivalent to that for $G(A)$ and $G^\bullet(B)$.

Proof. Let $u \in \mathbb{Z}[\omega]$. Then u is a solution to the grid problem for A and B if and only if $u \in A$ and $u^\bullet \in B$, if and only if $Gu \in G(A)$ and $G^\bullet u^\bullet = (Gu)^\bullet \in G^\bullet(B)$, if and only if Gu is a solution to the grid problem for $G(A)$ and $G^\bullet(B)$. □

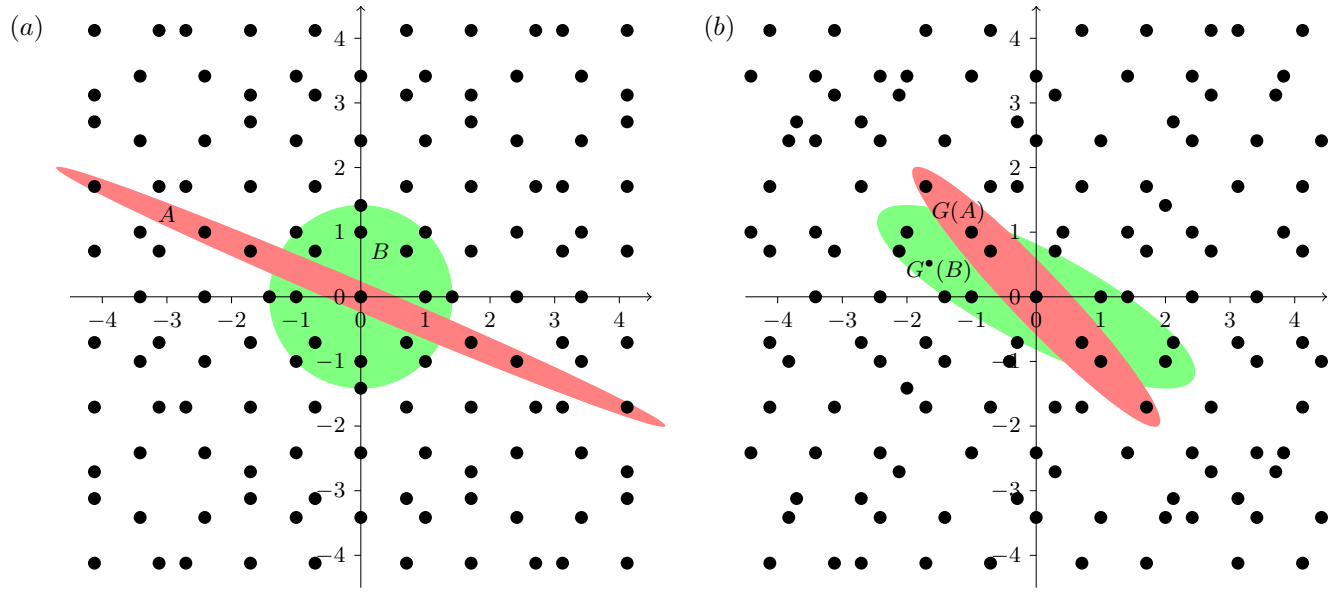


Figure 4: (a) The grid problem for two sets A and B. (b) The grid problem with $G(A)$ and $G^\bullet(B)$. Note that the solutions of (a), which are the grid points in the set A, are in one-to-one correspondence with the solutions of (b), which are the grid points in the set $G(A)$.

Example 5.14. Figure 4(a) illustrates the grid problem for a pair of sets A and B. As before, the set B is shown in green, and $\mathrm{Grid}(B)$ is shown as black dots. The set A is shown in red, and the solutions to the grid problem are the seven grid points that lie in A. Figure 4(b) shows the grid problem for the sets $G(A)$ and $G^\bullet(B)$, where G is the special grid operator

$$G = \begin{bmatrix} 1 & \sqrt{2} \\ 0 & 1 \end{bmatrix}.$$

Note that, as predicted by Proposition 5.13, the solutions of the transformed grid problem are in one-to-one correspondence with those of the original problem; namely, in each case, there are seven solutions.

5.4 Ellipses

Combining the results of Sections 5.2 and 5.3, we know that the grid problem for convex sets A and B can be solved efficiently, provided that we can find a grid operator G such that $G(A)$ and $G^\bullet(B)$ are sufficiently upright. Our key technical result is that in case A and B are ellipses, this is always the case.

Definition 5.15. Let D be a positive definite real 2×2-matrix with non-zero determinant, and let $p \in \mathbb{R}^2$ be a point. The *ellipse* defined by D and centered at p is the set

$$E = \{u \in \mathbb{R}^2 \mid (u-p)^\dagger D(u-p) \leqslant 1\}.$$

Theorem 5.16. *Suppose $A, B \subseteq \mathbb{R}^2$ are ellipses. Then there exists a grid operator G such that $G(A)$ and $G^\bullet(B)$ are $1/6$-upright. Moreover, if A and B are M-upright, then G can be efficiently computed in $O(\log(1/M))$ arithmetic operations.*

Since the proof is long and technical, we give it in Appendix A.

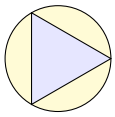
5.5 The enclosing ellipse of a bounded convex set

Our final step in the solution of the two-dimensional grid problem is to generalize Theorem 5.16 from ellipses to arbitrary bounded convex sets with non-empty interior. This can be done because every such set A can be inscribed in an ellipse whose area is not much greater than that of A, as stated in the following proposition.

Proposition 5.17. *Let A be a bounded convex subset of $\mathbb{R}^2$ with non-empty interior. Then there exists an ellipse E such that $A \subseteq E$, and such that*

$$\operatorname{area}(E) \leqslant \frac{4\pi}{3\sqrt{3}} \operatorname{area}(A).$$

The proof is in Appendix B. Note that $\frac{4\pi}{3\sqrt{3}} \approx 2.4184$. We remark that the bound in Proposition 5.17 is sharp; the bound is attained in case A is an equilateral triangle. In this case, the enclosing ellipse is a circle, and the ratio of the areas is exactly $\frac{4\pi}{3\sqrt{3}}$.



5.6 General solution of the two-dimensional grid problem

We are finally in a position to solve the two-dimensional grid problem for arbitrary bounded convex sets of non-empty interior.

Theorem 5.18. *There is an algorithm which, given two bounded convex subset A and B of $\mathbb{R}^2$ with non-empty interior, enumerates all solutions of the two-dimensional grid problem for A and B. Moreover, if A and B are M-upright, then the algorithm requires $O(\log(1/M))$ arithmetic operations overall, plus a constant number of arithmetic operations per solution produced.*

Proof. Given two such sets A and B, we can first find ellipses A' and B' containing A and B, respectively, and whose areas exceed those of A and B by at most a fixed constant factor N. Such ellipses exist by Proposition 5.17; moreover, it is not hard to see that they can be found efficiently if A and B are polygons with rational vertices. Since we assume that each given convex set is equipped with such an enclosing rational polygon (Remark 5.4), A' and B' can be found efficiently for arbitrary given A and B.

Next, by Theorem 5.16, we can use $O(\log(1/M))$ arithmetic operations to find a grid operator G such that $G(A')$ and $G^\bullet(B')$ are 1/6-upright. It follows that $G(A)$ and $G^\bullet(B)$ are N/6-upright. By Lemma 5.8, we can efficiently enumerate all solutions u of the grid problem for $G(A)$ and $G^\bullet(B)$. By Proposition 5.13, $G^{-1}u$ then enumerates the solutions to the grid problem for A and B. □

Remark 5.19. Note that the complexity of $O(\log(1/M))$ overall operations in Theorem 5.18 is exponentially better than the complexity of $O(1/M^2)$ per candidate we obtained in Lemma 5.8. This improvement is entirely due to the use of grid operators in Theorem 5.16.

5.7 Scaled grid problems

Sometimes we want to find solutions to a grid problem where the points are taken in $\mathbb{D}[\omega]$ instead of $\mathbb{Z}[\omega]$. There are two variants of this problem: we may either enumerate the solutions for a *fixed* denominator exponent, or enumerate all solutions in order of *increasing* least denominator exponent.

Definition 5.20. Let A and B be subsets of $\mathbb{R}^2$. The *two-dimensional scaled grid problem for fixed $k \geqslant 0$* is to find $u \in \frac{1}{\sqrt{2}^k}\mathbb{Z}[\omega]$ satisfying $u \in A$ and $u^\bullet \in B$. The *two-dimensional scaled grid problem for arbitrary $k \geqslant 0$* is to find $u \in \mathbb{D}[\omega]$ satisfying $u \in A$ and $u^\bullet \in B$.

Proposition 5.21. *There is an algorithm which, given two bounded convex subsets A and B of $\mathbb{R}^2$ with non-empty interior and an integer $k \geqslant 0$, enumerates all solutions of the two-dimensional scaled grid problem for A, B, and k. Moreover, if A and B are M-upright, then the algorithm requires $O(\log(1/M))$ arithmetic operations overall, plus a constant number of arithmetic operations per solution produced.*

Proof. Note that $u = \frac{1}{\sqrt{2}^k}v$ is a solution to the scaled grid problem for A, B, and k if and only if v is a solution to the (unscaled) grid problem for $\sqrt{2}^k A$ and $(-\sqrt{2})^k B$. The claim then immediately follows from Proposition 5.18. □

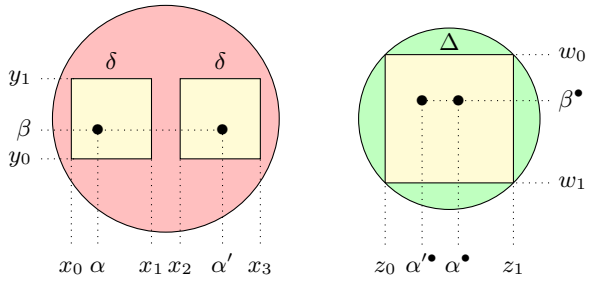
Proposition 5.22. *There is an algorithm which, given two bounded convex subsets A and B of $\mathbb{R}^2$ with non-empty interior, enumerates (the infinite sequence of) all solutions u of the two-dimensional scaled grid problem for A, B, and arbitrary $k \geqslant 0$, in order of increasing k. Moreover, if A and B are M-upright, then the algorithm requires $O(\log(1/M))$ arithmetic operations overall, plus a constant number of arithmetic operations per solution produced.*

Proof. This can be done by applying Lemma 5.21 to each $k = 0, 1, 2, \ldots$, in increasing order. In principle, this method enumerates each solution multiple times, since each solution for k is also a solution for $k+1$. As a slight optimization, such duplicate enumeration can be avoided by noting that for $k > 0$, $u = \frac{1}{\sqrt{2}^k}(a\omega^3 + b\omega^2 + c\omega + d)$ is an element of $\mathbb{Z}[\omega]/\sqrt{2}^k - \mathbb{Z}[\omega]/\sqrt{2}^{k-1}$ if and only if $a - c$ or $b - d$ (or both) are odd. Finally, we note that, because uprightness is invariant under scaling, the grid operator G in the proof of Proposition 5.18 only needs to be computed once, rather than once for every k. □

We end this section with some lower bounds on the number of solutions to two-dimensional scaled grid problems.

Lemma 5.23. *Let A and B be convex subsets of $\mathbb{R}^2$, and let $k \geqslant 0$. Assume A contains a circle of radius r and B contains a circle of radius R, such that $rR \geqslant \frac{1}{2^k}(1+\sqrt{2})^2$. Then the scaled grid problem for k has at least 2 solutions.*

Proof. By scaling the problem by a factor of $\sqrt{2}^k$, we can assume without loss of generality that $k = 0$. Let $\delta = r/\sqrt{2}$ and $\Delta = R\sqrt{2}$, and inscribe two squares of size $\delta \times \delta$ in the first circle, and one square of size $\Delta \times \Delta$ in the second circle, as shown here:



Since $\delta\Delta = rR \geqslant (1+\sqrt{2})^2$, by Lemma 4.4, we can find $\alpha, \alpha', \beta \in \mathbb{Z}[\sqrt{2}]$ such that $\alpha \in [x_0, x_1]$, $\alpha^\bullet \in [z_0, z_1]$, $\alpha' \in [x_2, x_3]$, $\alpha'^\bullet \in [z_0, z_1]$, $\beta \in [y_0, y_1]$, and $\beta^\bullet \in [w_0, w_1]$. Then $u = \alpha + i\beta$ and $v = \alpha' + i\beta$ are two different solutions to the two-dimensional grid problem as claimed. □

Lemma 5.24. *Let A and B be convex subsets of $\mathbb{R}^2$, and assume that the two-dimensional scaled grid problem for k has at least two distinct solutions. Then for all $\ell \geqslant 0$, the scaled grid problem for $k + 2\ell$ has at least $2^\ell + 1$ solutions.*

Proof. Let $u \neq v$ be solutions of the scaled grid problem for k. For each $j = 0, 1, \ldots, 2^\ell$, let $\phi = \frac{j}{2^\ell}$, and consider $u_j = \phi u + (1-\phi)v$. Then u_j has denominator exponent $k+2\ell$. Also, u_j is a convex combination of u and v; moreover, since $\phi^\bullet = \phi$, we also know that $u_j^\bullet = \phi u^\bullet + (1-\phi)v^\bullet$ is a convex combination of $u^\bullet$ and $v^\bullet$. Since A and B are convex, it follows that u_j is a solution of the scaled grid problem for $k + 2\ell$, yielding $2^\ell + 1$ distinct solutions. □

Remark 5.25. The bound in Lemma 5.24 is sufficient for our purposes, but it is not tight. In fact, the number of solutions grows as $O(4^k)$.

6 Solving a Diophantine equation

We will be interested in solving equations of the following form: given $\xi \in \mathbb{D}[\sqrt{2}]$, find $t \in \mathbb{D}[\omega]$ such that

$$t^\dagger t = \xi. \tag{9}$$

The following necessary condition is immediate:

Lemma 6.1 (Necessary condition)**.** *The equation* (9) *has a solution only if $\xi \geqslant 0$ and $\xi^\bullet \geqslant 0$.*

Proof. Assume $t^\dagger t = \xi$. Since t is a complex number, we have $\xi = t^\dagger t \geqslant 0$. Similarly, since $t^\bullet$ is a complex number, we have $\xi^\bullet = (t^\bullet)^\dagger (t^\bullet) \geqslant 0$. □

The following theorem states that the problem of solving the equation (9) can be reduced to the prime factorization problem for integers.

Theorem 6.2. *Let $\xi \in \mathbb{D}[\sqrt{2}]$. Note that $\xi^\bullet \xi \in \mathbb{D}$, so we can write $\xi^\bullet \xi = \frac{n}{2^\ell}$ for some $n \in \mathbb{Z}$ and $\ell \in \mathbb{N}$. There exists a probabilistic algorithm which, given ξ and, in case $n \neq 0$, a prime factorization of n, determines whether or not the equation* (9) *has a solution, and finds a solution if there is one. Moreover, the expected runtime of this algorithm is polynomial in the size of n.*

Theorem 6.2 is a well-known result in computational algebraic number theory. For the benefit readers who are not experts in number theory, we give an elementary and more or less self-contained proof in Appendix C.

7 The approximate synthesis algorithm

7.1 The approximate synthesis problem

Recall that the z-rotation by angle θ is the unitary operator

$$R_z(\theta) = e^{-i\theta Z/2} = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}.$$

Definition 7.1. Given θ and a precision $\varepsilon > 0$, the *approximate synthesis problem for z-rotations* is to find an operator U expressible in the single-qubit Clifford+T gate set, such that

$$\|R_z(\theta) - U\| \leqslant \varepsilon. \tag{10}$$

Moreover, we want the T-count of the operator U to be as small as possible; here, the T-count of a Clifford+T circuit is the number of T-gates appearing in it. The norm in (10) is the operator norm.

It is known from [10] that a single-qubit operator can be exactly represented over the Clifford+T gate set if and only if it can be written of the form

$$U = \begin{pmatrix} u & -t^\dagger \omega^\ell \\ t & u^\dagger \omega^\ell \end{pmatrix}, \tag{11}$$

where $u, t \in \mathbb{D}[\omega]$ and ℓ is an integer. The following lemma shows that, for the purposes of approximate synthesis, we may assume without loss of generality that $\ell = 0$.

Lemma 7.2. *If $\varepsilon < |1 - e^{i\pi/8}|$, then all solutions of the approximate synthesis problem have the form*

$$U = \begin{pmatrix} u & -t^\dagger \\ t & u^\dagger \end{pmatrix}. \tag{12}$$

If $\varepsilon \geqslant |1 - e^{i\pi/8}|$, then there exists a solution of T-count 0 (i.e., a Clifford operator), and it is also of the form (12).

Proof. To prove the first claim, assume $\varepsilon < |1 - e^{i\pi/8}|$. Let U be of the form (11), satisfying (10). Let $e^{i\phi_1}$ and $e^{i\phi_2}$ be the eigenvalues of $UR_z(\theta)^{-1}$, with $\phi_1, \phi_2 \in [-\pi, \pi]$. Using (10), we have $\|I - UR_z(\theta)^{-1}\| \leqslant \varepsilon < |1 - e^{i\pi/8}|$. On the other hand, $\|I - UR_z(\theta)^{-1}\| = \max\{|1 - e^{i\phi_1}|, |1 - e^{i\phi_2}|\}$. It follows that $|1 - e^{i\phi_j}| < |1 - e^{i\pi/8}|$ for $j = 1, 2$, hence $-\pi/8 < \phi_j < \pi/8$, hence $-\pi/4 < \phi_1 + \phi_2 < \pi/4$, so $|1 - e^{i(\phi_1+\phi_2)}| < |1 - e^{i\pi/4}| = |1 - \omega|$. On other hand, we have $e^{i(\phi_1+\phi_2)} = \det(UR_z(\theta)^{-1}) = \omega^\ell$, hence $|1 - \omega^\ell| < |1 - \omega|$, which implies $\omega^\ell = 1$. Therefore, U is of the form (12). To prove the second claim, assume $\varepsilon \geqslant |1 - e^{i\pi/8}|$. Let j be the integer closest to $\frac{2\theta}{\pi}$, so that $|j - \frac{2\theta}{\pi}| \leqslant \frac{1}{2}$, or equivalently, $|j\frac{\pi}{4} - \frac{\theta}{2}| \leqslant \frac{\pi}{8}$. Let U be as in (12), with $u = \omega^{-j}$ and $t = 0$. Then $\|R_z(\theta) - U\| = |e^{i\theta/2} - u^\dagger| = |1 - u^\dagger e^{-i\theta/2}| = |1 - e^{i(j\frac{\pi}{4} - \frac{\theta}{2})}| \leqslant |1 - e^{i\frac{\pi}{8}}| \leqslant \varepsilon$. So (10) holds. But $U = S^j \omega^{-j}$ is a Clifford operator, so has T-count 0. □

Our strategy is therefore to approximate $R_z(\theta)$ by an operator U of the form (12), with $u, t \in \mathbb{D}[\omega]$, and then use the exact synthesis algorithm of [10] to synthesize U into a sequence of Clifford+T gates with minimal T-count. The following lemma relates the T-count of the resulting circuit to the least denominator exponent k of u.

Lemma 7.3. *Let U be a unitary operator as in* (12)*, where $u, t \in \mathbb{D}[\omega]$, and let k be the least denominator exponent of u. Then the T-count of U is either $2k - 2$ or $2k$. Moreover, if $k > 0$ and U has T-count $2k$, then $U' = TUT^\dagger$ has T-count $2k - 2$. We further note that $\|R_z(\theta) - U'\| = \|R_z(\theta) - U\|$, so for the purpose of solving* (10)*, it does not matter whether U or U' is used. Hence, without loss of generality, we may assume that U as in* (12) *always has T-count exactly $2k - 2$ when $k > 0$, and 0 when $k = 0$.*

Proof. Because U is unitary, we have $t^\dagger t + u^\dagger u = 1$. We first claim that t and u have the same least denominator exponent. Indeed, in the ring $\mathbb{Z}[\omega]$, an element s is divisible by $\sqrt{2}$ if and only if $s^\dagger s$ is divisible by 2. The left-to-right implication is obvious, and the right-to-left implication follows, e.g., from Lemma 2 of [6]. Then for any $k \geqslant 0$, we have $\sqrt{2}^k u \in \mathbb{Z}[\omega]$ iff $2^k u^\dagger u \in \mathbb{Z}[\omega]$ iff $2^k(1 - t^\dagger t) \in \mathbb{Z}[\omega]$ iff $2^k t^\dagger t \in \mathbb{Z}[\omega]$ iff $\sqrt{2}^k t \in \mathbb{Z}[\omega]$. This proves that u and t have the same denominator exponents, and in particular, the same least denominator exponent.

The claims about the T-counts of U and U' follow by inspection of Figure 2 of [7]. Using the terminology of Definitions 7.4 and 7.6 of [7], this figure shows every possible k-residue of a Clifford+T operator, modulo a right action of the group $\langle S, X, \omega \rangle$. Because U is of the form (12), only a subset of the k-residues is actually possible, and the figure shows that for this subset, the T-count is $2k$ or $2k-2$. Moreover, in each of the possible cases where $k > 0$ and U has T-count $2k$, the figure also shows that $U' = TUT^\dagger$ has T-count $2k - 2$.

For the final claim, we have $\|R_z(\theta) - U\| = \|TR_z(\theta)T^\dagger - TUT^\dagger\| = \|R_z(\theta) - U'\|$ because $R_z(\theta)$ and T commute. □

So our task is to find an operator U of the form (12), satisfying (10), and such that the denominator exponent of u is as small as possible. It is useful to first re-express (10) as a property of u. Let $z = e^{-i\theta/2}$. Using $u^\dagger u + t^\dagger t = 1$ and $z^\dagger z = 1$, we have

$$\|R_z(\theta) - U\|^2 = \|u - z\|^2 + \|t\|^2 = (u-z)^\dagger(u-z) + t^\dagger t = u^\dagger u + t^\dagger t - z^\dagger u - u^\dagger z + z^\dagger z = 2 - 2\,\mathrm{Re}(z^\dagger u).$$

So (10) is equivalent to $2 - 2\,\mathrm{Re}(z^\dagger u) \leqslant \varepsilon^2$, or equivalently, $\mathrm{Re}(z^\dagger u) \geqslant 1 - \frac{\varepsilon^2}{2}$. If we identify the complex numbers $z = x + yi$ and $u = a + bi$ with 2-dimensional real vectors $\vec{z} = (x, y)^T$ and $\vec{u} = (a, b)^T$, then $\mathrm{Re}(z^\dagger u)$ is just their inner product $\vec{z} \cdot \vec{u}$, and therefore (10) is equivalent to

$$\vec{z} \cdot \vec{u} \geqslant 1 - \frac{\varepsilon^2}{2}. \tag{13}$$

In summary, the approximate synthesis problem reduces to the following:

Problem 7.4. Given an angle θ and a precision $\varepsilon > 0$, find $u, t \in \mathbb{D}[\omega]$ such that

(a) $t^\dagger t + u^\dagger u = 1$,

(b) $\vec{z} \cdot \vec{u} \geqslant 1 - \varepsilon^2/2$, where $z = e^{-i\theta/2}$, with notation as above,

(c) and such that u has the smallest denominator exponent we can find.

7.2 Reduction to a grid problem and a Diophantine equation

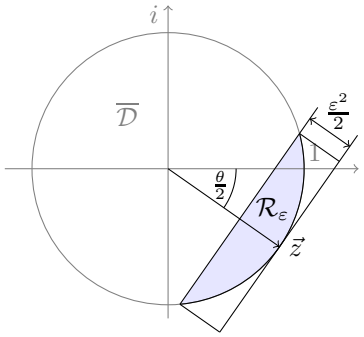
As we will now show, Problem 7.4 can be reduced to a scaled grid problem and a Diophantine equation, and therefore it can be efficiently solved by the methods of Sections 5 and 6. Let $\overline{\mathcal{D}}$ be the closed unit disk, regarded either as a subset of $\mathbb{C}$ or of $\mathbb{R}^2$.

Lemma 7.5. *If $u, t \in \mathbb{D}[\omega]$ and $t^\dagger t + u^\dagger u = 1$, then $u \in \overline{\mathcal{D}}$ and $u^\bullet \in \overline{\mathcal{D}}$.*

Proof. Note that $u^\dagger u = 1 - t^\dagger t \leqslant 1$, so $u \in \overline{\mathcal{D}}$. Similarly, $(u^\bullet)^\dagger(u^\bullet) = 1 - (t^\bullet)^\dagger(t^\bullet) \leqslant 1$, so $u^\bullet \in \overline{\mathcal{D}}$. □

The condition $\vec{z} \cdot \vec{u} \geqslant 1 - \varepsilon^2/2$ from Problem 7.4(b) defines a certain subset of the unit disk, which we call the *ε-region* for θ:

$$\mathcal{R}_\varepsilon = \{\vec{u} \in \overline{\mathcal{D}} \mid \vec{u} \cdot \vec{z} \geqslant 1 - \frac{\varepsilon^2}{2}\}. \tag{14}$$



By Lemma 7.5 and Problem 7.4(b), a *necessary* condition for a solution to Problem 7.4 is that $u \in \mathcal{R}_\varepsilon$ and $u^\bullet \in \overline{\mathcal{D}}$. This is an instance of a scaled two-dimensional grid problem; note that both the ε-region and the unit disk are convex, and are effectively given in the sense of Remark 5.4. Therefore, by Proposition 5.22, there exists an efficient algorithm that enumerates all such u in increasing order of least denominator exponent.

For each $u \in \mathbb{D}[\omega]$ satisfying the grid problem, it remains to check whether the equation $t^\dagger t + u^\dagger u = 1$ has a solution $t \in \mathbb{D}[\omega]$. This is equivalent to solving the Diophantine equation

$$t^\dagger t = 1 - u^\dagger u,$$

which is of the form (9). Let $\xi = 1 - u^\dagger u \in \mathbb{D}[\sqrt{2}]$, and write $\xi^\bullet \xi = \frac{n}{2^\ell}$, where $n \in \mathbb{Z}$ and $\ell \in \mathbb{N}$. By Theorem 6.2, there is an efficient algorithm that can solve this equation (or determine that no solution exists), given a prime factorization of n.

7.3 The main algorithm

Putting together the results of Sections 7.1 and 7.2, we obtain the following algorithm for solving the approximate synthesis problem:

Algorithm 7.6. Given θ and ε, let $A = \mathcal{R}_\varepsilon$ be the ε-region, and let $B = \overline{\mathcal{D}}$ be the unit disk.

1. Use Proposition 5.22 to enumerate the infinite sequence of solutions to the scaled grid problem $u \in A$ and $u^\bullet \in B$, where $u \in \mathbb{D}[\omega]$, in the order of increasing least denominator exponent k.
2. For each such solution u:
   (a) Let $\xi = 1 - u^\dagger u \in \mathbb{D}[\sqrt{2}]$, and write $\xi^\bullet \xi = \frac{n}{2^\ell}$, where $n \in \mathbb{Z}$ and $\ell \geqslant 0$ is minimal.
   (b) Attempt to find a prime factorization of n. If $n \neq 0$ but no prime factorization is found, skip step 2(c) and continue with the next u.
   (c) Use the algorithm of Theorem 6.2 to solve the equation $t^\dagger t = \xi$. If a solution t exists, go to step 3; otherwise, continue with the next u.
3. Define U as in equation (12), let $U' = TUT^\dagger$, and use the exact synthesis algorithm of [10] to find a Clifford+T circuit implementing either U or U', whichever has smaller T-count. Output this circuit and stop.

8 Analysis of the algorithm

8.1 Correctness

Proposition 8.1 (Correctness)**.** *If Algorithm 7.6 terminates, then it yields a valid solution to the approximate synthesis problem, i.e., it yields a Clifford+T circuit approximating $R_z(\theta)$ up to ε.*

Proof. By construction. By steps 2(a) and 2(c) of the algorithm, we have $t^\dagger t + u^\dagger u = 1$, so U is unitary. By step 1 of the algorithm, u belongs to the ε-region, so (13) holds. This implies that U satisfies (10). Moreover, as noted in Lemma 7.3, U' also satisfies (10), so whichever of these operators is returned approximates $R_z(\theta)$ up to ε. □

8.2 Optimality

The optimality of the algorithm hinges on step 2(b), "attempt to find a prime factorization of n". In the presence of a factoring oracle (such as a quantum computer), this can always be done. In this case, Algorithm 7.6 is guaranteed to find an optimal solution to the approximate synthesis problem. In the absence of a factoring oracle, we must potentially discard some candidate solutions u, until we find one for which n can be factored. We analyze these two situations in Propositions 8.2 and 8.8.

Proposition 8.2 (Optimality in the presence of a factoring oracle)**.** *In the presence of an oracle for integer factoring, the circuit returned by Algorithm 7.6 has the smallest T-count of any single-qubit Clifford+T circuit approximating $R_z(\theta)$ up to ε.*

Proof. By construction, step 1 of the algorithm enumerates all solutions u of the scaled grid problem in order of increasing denominator exponent k. Step 2(a) always succeeds, and step 2(b) always succeeds by using the factoring oracle. By Theorem 6.2, step 2(c) succeeds if and only if the equation $t^\dagger t + u^\dagger u = 1$ has a solution. Therefore, when step 2(c) succeeds, the algorithm has found a solution of Problem 7.4 for which u has the lowest possible denominator exponent k. Let m be the T-count of the final solution. By Lemma 7.3, we have $m = 2k - 2$, except when $k = 0$, in which case $m = 0$.

To show that this T-count is minimal, let $\bar{U}$ be any solution of the approximate synthesis problem with T-count $\bar{m}$. By Lemma 7.2, we may assume without loss of generality that

$$\bar{U} = \begin{pmatrix} \bar{u} & -\bar{t}^\dagger \\ \bar{t} & \bar{u}^\dagger \end{pmatrix}.$$

Let $\bar{k}$ be the denominator exponent of $\bar{u}$. By minimality of k, we have $k \leqslant \bar{k}$, hence $m \leqslant \bar{m}$. □

We emphasize that the optimality in Proposition 8.2 is *absolute*, i.e., not merely asymptotic or up to an additive constant. Of all the Clifford+T operators approximating $R_z(\theta)$ to within ε, the algorithm finds one with the lowest T-count.

To analyze the algorithm in the absence of a factoring oracle, we must address the question of how many candidates must be generated before steps 2(b) and 2(c) of the algorithm succeed. In this case, the algorithm may still use any classical algorithm to try to factor the number n in step 2(b), but the amount of effort extended on any particular n must be capped. In our complexity analysis for this case, in Proposition 8.8 below, we conservatively assume that the only n that the algorithm can successfully factor are those n that are already prime. In reality the algorithm might do a little better. In order to complete the analysis, we must rely on a number-theoretic assumption about the distribution of prime numbers.

Hypothesis 8.3. Each number n produced in step 2(a) of Algorithm 7.6 is asymptotically as likely to be prime as a randomly chosen odd number of comparable size. Moreover, the primality of each n can be modelled as an independent random event.

Lemma 8.4. *Each of the numbers n produced in step 2(a) of Algorithm 7.6 satisfies $n \geqslant 0$, and either $n = 0$ or $n \equiv 1 \,(\mathrm{mod}\, 8)$.*

Proof. See Appendix D. □

Lemma 8.5. *Let u be a candidate produced in step 1 of Algorithm 7.6, let k be its least denominator exponent, and let n be the integer computed in step 2(a). Then $n \leqslant 4^k$.*

Proof. By assumption, we can write $u = v/\sqrt{2}^k$, where $v \in \mathbb{Z}[\omega]$. From step 2(a) of the algorithm, we have $\xi = 1 - u^\dagger u = \frac{1}{2^k}(2^k - v^\dagger v) = \frac{\alpha}{2^k}$, where $\alpha \in \mathbb{Z}[\sqrt{2}]$. Therefore $\xi^\bullet \xi = \frac{\alpha^\bullet \alpha}{2^{2k}} = \frac{n}{2^\ell}$. Since $\alpha^\bullet \alpha$ is an integer and ℓ is minimal, we have $\ell \leqslant 2k$. Also, by assumption, both u and $u^\bullet$ are in the unit disk, so $u^\dagger u \leqslant 1$ and $(u^\bullet)^\dagger (u^\bullet) \leqslant 1$. It follows that $0 \leqslant \xi, \xi^\bullet \leqslant 1$, hence $\xi^\bullet \xi \leqslant 1$. Therefore, $\frac{n}{2^\ell} \leqslant 1$, which implies $n \leqslant 2^\ell \leqslant 4^k$ as claimed. □

Lemma 8.6. *Let $b > 0$ be an arbitrary fixed constant. Then for $a \geqslant 1$,*

$$\sum_{x=1}^{\infty} \left(1 - \frac{1}{a + b \ln x}\right)^x = O(a).$$

Proof. See Appendix E. □

Definition 8.7. Let U', U'' be two solutions of the approximate synthesis problem of the form

$$U' = \begin{pmatrix} u' & -t'^\dagger \\ t' & u'^\dagger \end{pmatrix}, \quad U'' = \begin{pmatrix} u'' & -t''^\dagger \\ t'' & u''^\dagger \end{pmatrix}. \tag{15}$$

We say that U' and U'' are *equivalent solutions* if $u' = u''$.

Proposition 8.8 (Near-optimality in the absence of a factoring oracle)**.** *Let m be the T-count of the solution of the approximate synthesis problem found by Algorithm 7.6 in the absence of an oracle for integer factoring. Then*

(a) The approximate synthesis problem has an expected number of at most $O(\log(1/\varepsilon))$ non-equivalent solutions with T-count less than m.

(b) The expected value of m is $m'' + O(\log(\log(1/\varepsilon)))$, where m' and m'' are the T-counts of the optimal and second-to-optimal solutions of the approximate synthesis problem (up to equivalence), and $m' \leqslant m''$.

Proof. If $\varepsilon \geqslant |1 - e^{i\pi/8}|$, then by Lemma 7.2, there is a solution with T-count 0, and the algorithm easily finds it. In this case, there is nothing to show. So assume without loss of generality that $\varepsilon < |1 - e^{i\pi/8}|$. Then by Lemma 7.2, all solutions are of the form (12).

(a) Consider the list $u_1, u_2, \ldots$ of candidates generated in step 1 of the algorithm. Let $k_1, k_2, \ldots$ be their respective least denominator exponents, and let $n_1, n_2, \ldots$ be the corresponding integers calculated in step 2(a). By Lemma 8.5, we have $n_j \leqslant 4^{k_j}$ for all j. By Hypothesis 8.3, the probability that n_j is prime is asymptotically no smaller than that of a randomly chosen odd integer less than 4^{k_j}, which, by the well-known prime number theorem, is greater than

$$p_j := \frac{2}{\ln(4^{k_j})} = \frac{1}{k_j \ln 2}. \tag{16}$$

Note that u_1 and u_2 are two distinct solutions to the scaled grid problem of step 1 of the algorithm. Since the candidates are enumerated in order of increasing denominator exponent, k_2 is a denominator exponent for both u_1 and u_2. It follows by Lemma 5.24 that there are at least $2^\ell + 1$ distinct candidates of denominator exponent $k_2 + 2\ell$, for all $\ell \geqslant 0$. In other words, for all j, if $j \leqslant 2^\ell + 1$, we have $k_j \leqslant k_2 + 2\ell$. In particular, this holds for $\ell = \lfloor 1 + \log_2 j \rfloor$, and therefore,

$$k_j \leqslant k_2 + 2(1 + \log_2 j). \tag{17}$$

Combining (17) with (16), we have

$$p_j \geqslant \frac{1}{(k_2 + 2(1 + \log_2 j)) \ln 2} = \frac{1}{(k_2 + 2) \ln 2 + 2 \ln j} \tag{18}$$

Let j_0 be the smallest index such that n_{j_0} is prime. By Hypothesis 8.3, we can treat the primality of each n_j as an independent random event. Therefore,

$$\begin{aligned} P(j_0 > j) &= P(n_1, \ldots, n_j \text{ are not prime}) \\ &\leqslant (1 - p_1)(1 - p_2) \cdots (1 - p_j) \\ &\leqslant (1 - p_j)^j \\ &\leqslant \left(1 - \frac{1}{(k_2 + 2) \ln 2 + 2 \ln j}\right)^j. \end{aligned}$$

The expected value of j_0 is

$$E(j_0) \;=\; \sum_{j=0}^{\infty} P(j_0 > j) \;\leqslant\; 1 + \sum_{j=1}^{\infty} \left(1 - \frac{1}{(k_2 + 2) \ln 2 + 2 \ln j}\right)^j \;=\; O(k_2), \tag{19}$$

where we have used Lemma 8.6 to estimate the sum.

Next, we will estimate k_2. The width of the ε-region $\mathcal{R}_\varepsilon$, as shown in (14), is $\varepsilon^2/2$ at the widest point, and we can inscribe a disk of radius $r = \varepsilon^2/4$ in it. Also, the closed unit disk $\overline{\mathcal{D}}$ has radius $R = 1$. It follows by Lemma 5.23 that the scaled grid problem for $\mathcal{R}_\varepsilon$ and $\overline{\mathcal{D}}$, as in step 1 of the algorithm, has at least two solutions, provided that

$$rR = \frac{\varepsilon^2}{4} \geqslant \frac{(1 + \sqrt{2})^2}{2^k}, \tag{20}$$

or equivalently, provided that

$$k \geqslant 2 + 2 \log_2(1 + \sqrt{2}) + 2 \log_2(1/\varepsilon). \tag{21}$$

We therefore have $k_2 \leqslant k$ for *all* k satisfying (21). It follows that

$$k_2 = O(\log(1/\varepsilon)), \tag{22}$$

and therefore, using (19), also

$$E(j_0) = O(\log(1/\varepsilon)). \tag{23}$$

To finish the proof of part (a), recall that j_0 was defined to be the smallest index such that n_{j_0} is prime. This ensures that step 2(b) of the algorithm succeeds for the candidate u_{j_0}. Furthermore, we have $n \equiv 1 \pmod 8$ by Lemma 8.4, and therefore the equation $t^\dagger t = \xi$ has a solution by Proposition C.26. Hence the remaining steps of the algorithm also succeed for u_{j_0}.

Now let r be the number of non-equivalent solutions of the approximate synthesis problem of T-count strictly less than m. As noted above, any such solution U is of the form (12). Then the least denominator exponent of u is strictly smaller than k_{j_0}, so that $u = u_j$ for some $j < j_0$. In this way, each of the r non-equivalent solutions is mapped to a different index $j < j_0$. It follows that $r < j_0$, and hence $E(r) \leqslant E(j_0) = O(\log(1/\varepsilon))$, as was to be shown.

(b) Let U' be an optimal solution of the approximate synthesis problem, and let U'' be optimal among the solutions that are not equivalent to U'. Let u' and u'' be as in (15), and let k', k'' be the least denominator exponents of u' and u'', respectively, with $k' \leqslant k''$. Let k_2 and j_0 be as in the proof of part (a). Note that, by definition, $k_2 \leqslant k''$. Let k be the least denominator exponent of the solution of the approximate synthesis problem found by the algorithm. Then $k \leqslant k_{j_0}$. Using (17), we have

$$k \leqslant k_{j_0} \leqslant k_2 + 2(1 + \log_2 j_0) \leqslant k'' + 2(1 + \log_2 j_0).$$

Recall from Lemma 7.3 that $2k - 2 \leqslant m \leqslant 2k$, and similarly $2k'' - 2 \leqslant m'' \leqslant 2k''$. Hence $m \leqslant 2k \leqslant 2k'' + 4(1 + \log_2 j_0) \leqslant m'' + 6 + 4\log_2 j_0$. These calculations apply to any one run of the algorithm. Taking expected values over many randomized runs, we therefore have

$$E(m) \leqslant m'' + 6 + 4E(\log_2 j_0) \leqslant m'' + 6 + 4\log_2 E(j_0). \tag{24}$$

Note that we have used the law $E(\log j_0) \leqslant \log(E(j_0))$, which holds because log is a concave function. Combining (24) with (23), we therefore have the desired result:

$$E(m) = m'' + O(\log(\log(1/\varepsilon))). \tag{25}$$

□

Remark 8.9. In the near-optimal case of Proposition 8.8, our algorithm can additionally be used to compute a firm lower bound for the T-count of any solution of the approximate synthesis problem for the given θ and ε. Namely, the algorithm can consider the first candidate u_j for which the Diophantine equation solver does not fail — i.e., either it solves the equation or it times out. If k_j is the least denominator exponent of u_j, then a lower bound for the T-count is $2k_j - 2$ (or 0 when $k_j = 0$). Note that this is not the usual information-theoretic lower bound that applies in the average case, but an actual lower bound for each particular problem instance.

8.3 Worst-case behavior

In [14], an approximate synthesis algorithms for z-rotations was given that always returns a solution of T-count at most $K + 4\log_2(1/\varepsilon)$, where K is a constant approximately equal to 10. We note that Algorithm 7.6 enumerates all the solutions of the grid problem for the ε-region, whereas the algorithm of [14] only enumerates a subset of the solutions. Also, Algorithm 7.6 can solve the Diophantine equation in all the cases in which the algorithm of [14] can solve it. It follows that in all cases, the solution returned by Algorithm 7.6 is at least as good as that returned by the algorithm of [14]. In particular, Algorithm 7.6 always returns a solution of T-count at most $K + 4\log_2(1/\varepsilon)$. Moreover, it is known from [14, Section 9], that there are certain combinations of θ and ε for which this T-count is actually optimal to within a constant number of gates. Thus our algorithm's performance is $K + 4\log_2(1/\varepsilon)$ in the worst case, but this worst case behavior is only achieved in those cases where it is actually optimal.

One may ask whether a more precise statement can be made about the angles θ for which this worst-case behavior occurs. We believe that this is indeed possible. Let $\mathbb{Q}(\sqrt{2})$ be the field of rational numbers extended with $\sqrt{2}$.

Conjecture 8.10. *Fix an angle θ, and consider the T-count as $\varepsilon \to 0$. Then the T-count behaves as $K + 4\log_2(1/\varepsilon)$ when $\tan\frac{\theta}{2} \in \mathbb{Q}(\sqrt{2})$, and as $K + 3\log_2(1/\varepsilon)$ otherwise.*

Giving a rigorous proof of this conjecture is beyond the scope of this paper and probably difficult, but we will sketch a plausibility argument. We believe that the first part, showing that the T-count behaves no better than $K + 4\log_2(1/\varepsilon)$ when $\tan\frac{\theta}{2} \in \mathbb{Q}(\sqrt{2})$, could be made rigorous without too much difficulty. In addition, we also offer experimental evidence supporting Conjecture 8.10 in Section 10.

The essential point is this. Consider the grid associated to a convex bounded set B as in Figure 5. Note that the slope r of any line passing through two grid points satisfies $r \in \mathbb{Q}(\sqrt{2})$ (unless the line is vertical, in which case the slope is infinite). Conversely, fix some real number r. Let us call a line of slope r a *grid line* if it passes through at least one grid point. We first wish to show that if $r \in \mathbb{Q}(\sqrt{2})$, then the set of grid lines is discrete, i.e., there is a minimum distance between each pair of grid lines. To see why this is so, note that $r \in \frac{1}{n}\mathbb{Z}[\sqrt{2}]$ for some $n \in \mathbb{Z}$. Each grid line is uniquely determined by its y-intercept, i.e., its point of intersection with the y-axis. Consider a grid line with y-intercept b, and passing through the grid point $u \in \mathbb{Z}[\omega]$. Using Lemma 5.5, we can assume without loss of generality that $u = \alpha + \beta i$, where $\alpha, \beta \in \mathbb{Z}[\sqrt{2}]$; the case where $u = \alpha + \beta i + \omega$ is analogous. Then we have $b = \beta - r\alpha$, and therefore $b \in \frac{1}{n}\mathbb{Z}[\sqrt{2}]$. Moreover, the line with y-intercept $b^\bullet$ and slope $r^\bullet$ contains $u^\bullet$, and therefore intersects B. Since B is bounded, it follows that $b^\bullet$ lies in a certain finite interval B' of real numbers determined only by B and $r^\bullet$. The two relations $b \in \frac{1}{n}\mathbb{Z}[\sqrt{2}]$ and $b^\bullet \in B'$ determine a one-dimensional grid problem, and therefore the set of solutions is discrete. This shows that the set of grid lines is discrete when $r \in \mathbb{Q}(\sqrt{2})$.

Recall from (14) that for given θ and ε, the ε-region is bounded by a circular arc and a straight line. The slope of the straight line is $r = -1/\tan\frac{\theta}{2}$, and is independent of ε. If $r \in \mathbb{Q}(\sqrt{2})$, then the ε-region is "parallel" to the grid lines of slope r, as shown in Figure 5(a). In this case, the number of solutions to the grid problem is not dominated by the *area*, but by the *width* of the ε-region. More specifically, for the scaled grid problem of Algorithm 7.6 to have a solution for a given denominator exponent k, it is a necessary condition that the ε-region intersects some grid line. The distance between the grid lines scales as $\frac{1}{2^k}$, and the width of the ε-region is $\frac{\varepsilon^2}{2}$, and therefore, we expect the

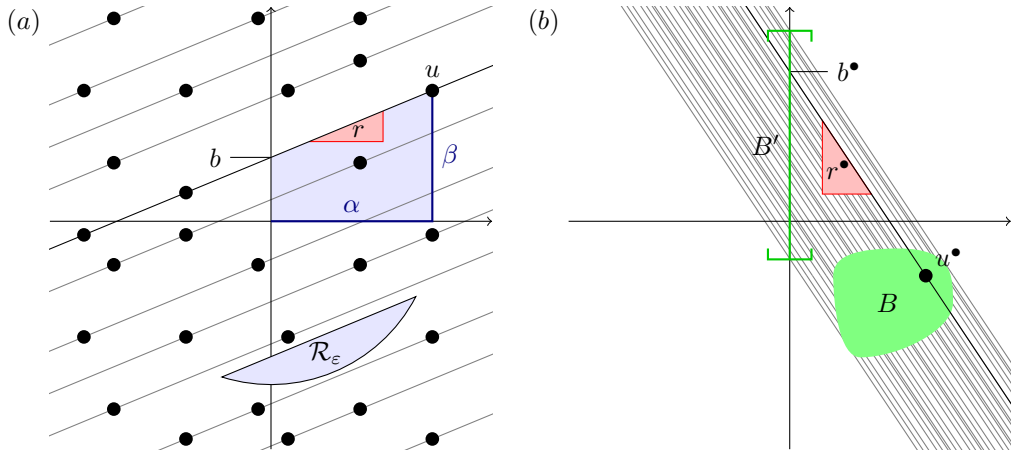


Figure 5: Part (a) shows the grid points and grid lines (of slope $r \in \mathbb{Q}(\sqrt{2})$) for a convex set B. Part (b) shows the set B and the dual grid lines (of slope $r^\bullet$), all of whose y-intercepts lie in an interval B'.

ε-region to intersect at least one grid line when $\frac{1}{2^k} \leqslant c\varepsilon^2$, for some constant c. Equivalently $k \geqslant K + 2\log\frac{1}{\varepsilon}$. By Lemmas 7.3 and 9.7, the T-count scales as $2k$, i.e., $K + 4\log\frac{1}{\varepsilon}$.

On the other hand, if $r \notin \mathbb{Q}(\sqrt{2})$, then the set of grid lines is dense, and in this case, it appears that the solution to the grid problem is dominated by the *area* of the ε-region. In this case, the number of grid points per area scales as 4^k, and the area of the ε-region is proportional to ε^3; hence we expect a grid point to fall in the ε-region when $\frac{1}{4^k} \leqslant c\varepsilon^3$, or equivalently, $2k \geqslant K + 3\log\frac{1}{\varepsilon}$, yielding the typical T-count of $K + 3\log\frac{1}{\varepsilon}$ in that case.

8.4 Time complexity of the algorithm

Proposition 8.11 (Complexity)**.** *Algorithm 7.6 runs in expected time* $O(\text{polylog}(1/\varepsilon))$*. This is true whether or not a factorization oracle is used.*

Proof. Let M be the uprightness of the ε-region. Let j_0 be the average number of candidates tried in steps 2(a)–(c) of the algorithm, and let k_{j_0} be the least denominator exponent of the final candidate. Let n be the largest integer that appears in step 2(a) of the algorithm.

By Proposition 5.22, step 1 of the algorithm requires $O(\log(1/M))$ arithmetic operations, plus a constant number per candidate. For each of the j_0 candidates, step 2(a) requires $O(1)$ arithmetic operations. Step 2(b) also requires $O(1)$ arithmetic operations, either due to the use of a factoring oracle, or else, because we can put an arbitrary fixed bound on the amount of effort invested in factoring any given integer. At minimum, this will succeed when the integer in question is prime, which is sufficient for the estimates of Proposition 8.8. Step 2(c) requires $O(\text{polylog}(n))$ operations by Theorem 6.2. Finally, step 3 requires $O(k_{j_0})$ arithmetic operations; see, e.g., Theorem 5.1 of [7]. So the total expected number of arithmetic operations is

$$O(\log(1/M)) + j_0 \cdot O(\text{polylog}(n)) + O(k_{j_0}). \tag{26}$$

Recall that the ε-region $\mathcal{R}_\varepsilon$, shown in (14), contains a disk of radius $\varepsilon^2/4$. Hence, $\text{area}(\mathcal{R}_\varepsilon) \geqslant \frac{\pi}{16}\varepsilon^4$. On the other hand, the square $[-1,1] \times [-1,1]$ is a (not very tight) bounding box for $\mathcal{R}_\varepsilon$. It follows that

$$M = \text{up}(\mathcal{R}_\varepsilon) = \frac{\text{area}(\mathcal{R}_\varepsilon)}{\text{area}(\text{BBox}(\mathcal{R}_\varepsilon))} = \Omega(\varepsilon^4),$$

hence $\log(1/M) = O(\log(1/\varepsilon))$. From (23), the expected value of j_0 is $O(\log(1/\varepsilon))$. Combining (17) with (22), we therefore have

$$k_{j_0} \leqslant k_2 + 2(1 + \log_2 j_0) = O(\log(1/\varepsilon)) + O(\log(\log(1/\varepsilon))) = O(\log(1/\varepsilon)).$$

From Lemma 8.5, and the fact that candidates are enumerated in order of increasing denominator exponent, we have $n \leqslant 4^{k_{j_0}}$, hence

$$\text{polylog}(n) = O(\text{poly}(k_{j_0})) = O(\text{polylog}(1/\varepsilon)).$$

Combining all of these estimates with (26), the expected number of arithmetic operations for the algorithm is $O(\text{polylog}(1/\varepsilon))$. Moreover, each individual arithmetic operation can be performed with precision $O(\log(1/\varepsilon))$,

taking time $O(\text{polylog}(1/\varepsilon))$. Therefore the total expected time complexity of the algorithm is $O(\text{polylog}(1/\varepsilon))$, as desired. □

9 Approximation up to a phase

So far, we have considered the problem of approximate synthesis "on the nose", i.e., the operator U in Definition 7.1 was literally required to approximate $R_z(\theta)$ in the operator norm. However, it is well-known that global phases have no observable effect in quantum mechanics, so in quantum computing, it is also common to consider the problem of approximate synthesis "up to a phase". This is made precise in the following definition.

Definition 9.1. Given θ and some $\varepsilon > 0$, the *approximate synthesis problem for z-rotations up to a phase* is to find an operator U expressible in the single-qubit Clifford+T gate set, and a unit scalar λ, such that

$$\|R_z(\theta) - \lambda U\| \leqslant \varepsilon. \tag{27}$$

Moreover, it is desirable to find U of smallest possible T-count. As before, the norm in (27) is the operator norm.

In this section, we will give a version of Algorithm 7.6 that optimally solves the approximate synthesis problem up to a phase. The central insight is that it is in fact sufficient to restrict λ to only two possible phases, namely $\lambda = 1$ and $\lambda = \sqrt{\omega} = e^{i\pi/8}$. A similar technique was also used in [11].

First, note that if W is a unitary 2×2-matrix and $\det W = 1$, then $\operatorname{tr} W$ is real. This is obvious, because $\det W = 1$ ensures that the two eigenvalues of W are each other's complex conjugates.

Lemma 9.2. *Let W be a unitary 2×2-matrix, and assume that $\det W = 1$ and $\operatorname{tr} W \geqslant 0$. Then for all unit scalars λ, we have*

$$\|I - W\| \leqslant \|I - \lambda W\|.$$

Proof. We may assume without loss of generality that W is diagonal. Since $\det W = 1$, we can write

$$W = \begin{pmatrix} e^{i\phi} & 0 \\ 0 & e^{-i\phi} \end{pmatrix}$$

for some ϕ. By symmetry, we can assume without loss of generality that $0 \leqslant \phi \leqslant \pi$. Since $\operatorname{tr} W \geqslant 0$, we have $\phi \leqslant \pi/2$. Now consider a unit scalar $\lambda = e^{i\psi}$, where $-\pi \leqslant \psi \leqslant \pi$. Then $\|I - \lambda W\| = \max\{|1 - e^{i(\psi+\phi)}|, |1 - e^{i(\psi-\phi)}|\}$ and $\|I - W\| = |1 - e^{i\phi}|$. If $\psi \geqslant 0$, then $|1 - e^{i\phi}| \leqslant |1 - e^{i(\psi+\phi)}|$. Similarly, if $\psi \leqslant 0$, then $|1 - e^{i\phi}| \leqslant |1 - e^{i(\psi-\phi)}|$. In either case, we have $\|I - W\| \leqslant \|I - \lambda W\|$, as claimed. □

Lemma 9.3. *Fix ε, a unitary operator R with $\det R = 1$, and a Clifford+T operator U. The following are equivalent:*

(1) There exists a unit scalar λ such that

$$\|R - \lambda U\| \leqslant \varepsilon;$$

(2) There exists $n \in \mathbb{Z}$ such that

$$\|R - e^{in\pi/8} U\| \leqslant \varepsilon.$$

Proof. It is obvious that (2) implies (1). For the opposite implication, first note that, because U is a Clifford+T operator, we have $\det U = \omega^k$ for some $k \in \mathbb{Z}$, and therefore $\det(R^{-1}U) = \omega^k$. Let $V = e^{-ik\pi/8} R^{-1} U$, so that $\det V = 1$. If $\operatorname{tr} V \geqslant 0$, let $W = V$; otherwise, let $W = -V$. Either way, we have $W = e^{in\pi/8} R^{-1} U$, where $n \in \mathbb{Z}$, and $\det W = 1$, $\operatorname{tr} W \geqslant 0$. Let $\lambda' = e^{-in\pi/8}\lambda$. By Lemma 9.2, we have

$$\begin{aligned} & \|I - W\| \leqslant \|I - \lambda' W\| \\ \implies & \|I - e^{in\pi/8} R^{-1} U\| \leqslant \|I - \lambda' e^{in\pi/8} R^{-1} U\| \\ \implies & \|R - e^{in\pi/8} U\| \leqslant \|R - \lambda' e^{in\pi/8} U\|, \\ \implies & \|R - e^{in\pi/8} U\| \leqslant \|R - \lambda U\|, \end{aligned}$$

which implies the desired claim. □

Remark 9.4. A version of Lemma 9.3 also applies to gate sets other than Clifford+T, as long as the gate set has discrete determinants.

Corollary 9.5. *In Definition 9.1, it suffices without loss of generality to consider only the two scalars $\lambda = 1$ and $\lambda = e^{i\pi/8}$.*

Proof. Let U be a Clifford+T operator satisfying (27) for some unit scalar λ. By Lemma 9.3, there exists a λ of the form $e^{in\pi/8}$ also satisfying (27). Then we can write $\lambda = \omega^k \lambda'$, where $k \in \mathbb{Z}$ and $\lambda' \in \{1, e^{i\pi/8}\}$. Letting $U' = \omega^k U$, we have $\lambda' U' = \lambda U$, and therefore

$$\|R_z(\theta) - \lambda' U'\| \leqslant \varepsilon,$$

as claimed. Moreover, since $\omega = e^{i\pi/4}$ is a Clifford operator, U and U' have the same T-count. □

To solve the approximate synthesis problem up to a phase, we therefore need an algorithm for finding optimal solutions of (27) in case $\lambda = 1$ and $\lambda = e^{i\pi/8}$ For $\lambda = 1$, this is of course just Algorithm 7.6. So all that remains to do is to find an algorithm for solving

$$\|R_z(\theta) - e^{i\pi/8} U\| \leqslant \varepsilon. \tag{28}$$

We use a sequence of steps very similar to those of Section 7.1 to reduce this to a grid problem and a Diophantine equation. We first consider the form of U.

Lemma 9.6. *If $\varepsilon < |1 - e^{i\pi/8}|$, then all solutions of (28) have the form*

$$U = \begin{pmatrix} u & -t^\dagger \omega^{-1} \\ t & u^\dagger \omega^{-1} \end{pmatrix}. \tag{29}$$

Proof. This is completely analogous to the proof of Lemma 7.2, using $e^{i\pi/8}U$ in place of U. □

Recall that $\delta = 1 + \omega$, and note that $\frac{\delta}{|\delta|} = e^{i\pi/8}$. Also note that $\delta\omega^{-1} = \delta^\dagger$, and that the element δ is invertible in $\mathbb{D}[\omega]$ with inverse $\delta^{-1} = (\omega - i)/\sqrt{2}$. Suppose that U is of the form (29). Let $u' = \delta u$ and $t' = \delta t$. We have:

$$\begin{aligned} \|R_z(\theta) - e^{i\pi/8} U\| &= \left\| R_z(\theta) - \frac{\delta}{|\delta|} \begin{pmatrix} u & -t^\dagger \omega^{-1} \\ t & u^\dagger \omega^{-1} \end{pmatrix} \right\| \\ &= \left\| R_z(\theta) - \frac{1}{|\delta|} \begin{pmatrix} \delta u & -\delta^\dagger t^\dagger \\ \delta t & \delta^\dagger u^\dagger \end{pmatrix} \right\| \\ &= \left\| R_z(\theta) - \frac{1}{|\delta|} \begin{pmatrix} u' & -t'^\dagger \\ t' & u'^\dagger \end{pmatrix} \right\|. \end{aligned}$$

Recall the definition of the ε-region $\mathcal{R}_\varepsilon$ from (14). Using exactly the same argument as in Section 7, it follows that (28) holds if and only if $\frac{u'}{|\delta|} \in \mathcal{R}_\varepsilon$, i.e., $u' \in |\delta| \mathcal{R}_\varepsilon$.

As before, in order for U to be unitary, of course it must satisfy $u^\dagger u + t^\dagger t = 1$, and a necessary condition for this is $u, u^\bullet \in \overline{\mathcal{D}}$. The latter condition can be equivalently re-expressed in terms of u' by requiring $u' \in |\delta| \overline{\mathcal{D}}$ and $u'^\bullet \in |\delta^\bullet| \overline{\mathcal{D}}$. Therefore, finding solutions to (28) of the form (29) reduces to the two-dimensional grid problem $u' \in |\delta| \mathcal{R}_\varepsilon$ and $u'^\bullet \in |\delta^\bullet| \overline{\mathcal{D}}$, together with the usual Diophantine equation $u^\dagger u + t^\dagger t = 1$. The last remaining piece of the puzzle is to compute the T-count of U, and in particular, to ensure that potential solutions are found in order of increasing T-count.

Lemma 9.7. *Let U be a Clifford+T operator of the form (29), and let k be the least denominator exponent of $u' = \delta u$. Then the T-count of U is either $2k - 1$ or $2k + 1$. Moreover, if $k > 0$ and U has T-count $2k + 1$, then $U' = TUT^\dagger$ has T-count $2k - 1$.*

Proof. This can be proved by a tedious but easy induction, analogous to Lemma 7.3. □

We therefore arrive at the following algorithm for solving (28). Here we assume $\varepsilon < |1 - e^{i\pi/8}|$, so that Lemma 9.6 applies.

Algorithm 9.8. Given θ and ε, let $A = |\delta| \mathcal{R}_\varepsilon$, and let $B = |\delta^\bullet| \overline{\mathcal{D}}$.

1. Use Proposition 5.22 to enumerate the infinite sequence of solutions to the scaled grid problem $u' \in A$ and $u'^\bullet \in B$, where $u' \in \mathbb{D}[\omega]$, in the order of increasing least denominator exponent k.
2. For each such solution u':
   (a) Let $u = u'/\delta$, let $\xi = 1 - u^\dagger u \in \mathbb{D}[\sqrt{2}]$, and write $\xi^\bullet \xi = \frac{n}{2^\ell}$, where $n \in \mathbb{Z}$ and $\ell \geqslant 0$ is minimal.
   (b) Attempt to find a prime factorization of n. If $n \neq 0$ but no prime factorization is found, skip step 2(c) and continue with the next u'.

(c) Use the algorithm of Theorem 6.2 to solve the equation $t^\dagger t = \xi$. If a solution t exists, go to step 3; otherwise, continue with the next u'.

3. Define U as in equation (29), let $U' = TUT^\dagger$, and use the exact synthesis algorithm of [10] to find a Clifford+T circuit implementing either U or U', whichever has smaller T-count. Output this circuit and stop.

Algorithm 9.8 is optimal in the presence of a factoring oracle, and near-optimal in the absence of a factoring oracle, in the same sense as Algorithm 7.6. Its expected time complexity is $O(\text{polylog}(1/\varepsilon))$. The proofs are completely analogous to those of Section 8. We then arrive at the following composite algorithm for the approximate synthesis problem for z-rotations up to a phase:

Algorithm 9.9 (Approximate synthesis up to a phase)**.** Given θ and ε, run both Algorithms 7.6 and 9.8, and return whichever circuit has the smaller T-count.

Proposition 9.10 (Correctness, time complexity, and optimality)**.** *Algorithm 9.9 yields a valid solution to the approximate synthesis problem up to a phase. Its expected running time is* $O(\text{polylog}(1/\varepsilon))$*. In the presence of a factoring oracle, the algorithm is optimal, i.e., the returned circuit has the smallest T-count of any single-qubit Clifford+T circuit approximating $R_z(\theta)$ up to ε and up to a phase. Moreover, in the absence of a factoring oracle, the algorithm is near-optimal in the following sense. Let m be the T-count of the solution found. Then:*

(a) The approximate synthesis problem has an expected number of at most $O(\log(1/\varepsilon))$ *non-equivalent solutions with T-count less than m.*

(b) The expected value of m is $m''' + O(\log(\log(1/\varepsilon)))$, where m''' is the T-count of the third-to-optimal solution (up to equivalence) of the approximate synthesis problem up to a phase.

Proof. The correctness and time complexity of Algorithm 9.9 follows from that of Algorithms 7.6 and 9.8. The optimality results follow from those of Algorithms 7.6 and 9.8, keeping in mind that Algorithm 7.6 finds an optimal (or near-optimal) solution for the phase $\lambda = 1$, Algorithm 9.8 finds an optimal (or near-optimal) solution for the phase $\lambda = e^{i\pi/8}$, and by Corollary 9.5, these are the only two phases that need to be considered.

The only subtlety that must be pointed out is that in part (b) of the near-optimality, we use the T-count of the *third*-to-optimal solution, rather than the second-to-optimal one as in Proposition 8.8. This is because the optimal and second-to-optimal solutions may belong to Algorithms 7.6 and 9.8, respectively, so that it may not be until the third-to-optimal solution that the near-optimality result of either Algorithm 7.6 or Algorithm 9.8 can be invoked. □

Remark 9.11. Algorithms 7.6 and 9.8 share the same ε-region up to scaling, and therefore the uprightness computation of Theorem 5.16 only needs to be done once.

Remark 9.12. By Lemmas 7.3 and 9.7, Algorithm 7.6 always produces circuits with even T-count, and Algorithm 9.8 always produces circuits with odd T-count. Instead of running both algorithms to completion, it is possible to interleave the two algorithms, so that all potential solutions are considered in order of increasing T-count. This is a slight optimization which does not, however, affect the asymptotic time complexity.

10 Experimental results

We implemented Algorithm 7.6 in the programming language Haskell; the implementation is available from [13]. Running on a single core of a 3.4GHz Intel i5-3570 CPU, we approximated the operator $R_z(\pi/128)$ up to various ε. The results are summarized in the leftmost four columns of Table 1. For example, here is the approximation of $R_z(\pi/128)$ up to $\varepsilon = 10^{-10}$:

$$\begin{aligned} u &= \frac{1}{\sqrt{2}^{52}}(-26687414\omega^3 + 10541729\omega^2 + 10614512\omega + 40727366) \\ t &= \frac{1}{\sqrt{2}^{52}}(30805761\omega^3 - 23432014\omega^2 + 2332111\omega + 20133911) \\ U &= \texttt{HTSHTSHTSHTHTHTHTSHTHTSHTSHTSHTHTHTSHTSHTHTHTSHTHTSHTHTHTHTHTHTSHTSHT} \\ &\quad \texttt{SHTHTSHTHTSHTHTHTHTSHTHTHTSHTHTSHTHTHTHTSHTSHTSHTHTHTSHTSHTSHTSHTHTSHTS} \\ &\quad \texttt{HTSHTSHTHTSHTHTSHTSHTHTHTHTHTSHTHTHTHTSHTSHTSHTHTSHTSHTHTHTSHTHTHTHTHTS} \\ &\quad \texttt{HTSHTHTHTHTHTSHTHTHTHTSHTHTHTHTHTHTH}\omega^7 \end{aligned}$$

In addition to ε, the T-count of the computed operator, and the actual error, we have also reported the lower bound on the T-count, which was computed according to Remark 8.9. It can be seen that the actual T-count achieved by

ε	T-count	T-bound	Actual error	Runtime	Candidates	Time/Candidate
10^{-10}	102	$\geqslant 102$	$0.91180 \cdot 10^{-10}$	$0.0190s$	3.0	$0.0064s$
10^{-20}	200	$\geqslant 198$	$0.87670 \cdot 10^{-20}$	$0.0433s$	7.0	$0.0061s$
10^{-30}	298	$\geqslant 298$	$0.99836 \cdot 10^{-30}$	$0.0600s$	7.0	$0.0085s$
10^{-40}	402	$\geqslant 400$	$0.77378 \cdot 10^{-40}$	$0.0976s$	11.7	$0.0084s$
10^{-50}	500	$\geqslant 500$	$0.82008 \cdot 10^{-50}$	$0.1353s$	20.3	$0.0067s$
10^{-60}	602	$\geqslant 596$	$0.61151 \cdot 10^{-60}$	$0.1548s$	16.0	$0.0097s$
10^{-70}	702	$\geqslant 698$	$0.40936 \cdot 10^{-70}$	$0.1931s$	20.9	$0.0093s$
10^{-80}	804	$\geqslant 794$	$0.92372 \cdot 10^{-80}$	$0.2402s$	27.2	$0.0088s$
10^{-90}	898	$\geqslant 898$	$0.96607 \cdot 10^{-90}$	$0.2696s$	22.2	$0.0121s$
10^{-100}	1000	$\geqslant 998$	$0.78879 \cdot 10^{-100}$	$0.3443s$	31.2	$0.0110s$
10^{-200}	1998	$\geqslant 1994$	$0.73266 \cdot 10^{-200}$	$1.1423s$	62.3	$0.0183s$
10^{-500}	4990	$\geqslant 4986$	$0.67156 \cdot 10^{-500}$	$8.6509s$	170.4	$0.0508s$
10^{-1000}	9974	$\geqslant 9966$	$0.80457 \cdot 10^{-1000}$	$47.9300s$	270.4	$0.1773s$
10^{-2000}	19942	$\geqslant 19934$	$0.88272 \cdot 10^{-2000}$	$383.1024s$	556.7	$0.6881s$

Table 1: Experimental results. The first four columns report the T-count, computed lower bound on the T-count, and the actual error for approximating the operator $R_z(\pi/128)$ up to various ε. The remaining columns report the runtime for each ε, averaged over 50 independent runs of Algorithm 7.6 with random angles θ. The runtime is further broken down into average number of candidates tried per run of the algorithm, and time spent per candidate.

	$\varepsilon = 10^{-100}$	$\varepsilon = 10^{-200}$	$\varepsilon = 10^{-300}$	$\varepsilon = 10^{-400}$	$\varepsilon = 10^{-500}$
$\theta = 2\tan^{-1}(2+3\sqrt{2})$	1320	2646	3976	5306	6636
$\theta = 2\tan^{-1}(5/3)$	1314	2646	3966	5306	6636
$\theta = 2\tan^{-1}((2+7\sqrt{2})/5)$	1308	2642	3968	5294	6630
$\theta = 2\tan^{-1}(2+3\sqrt{3})$	1002	1996	2994	3992	4990
$\theta = 2\tan^{-1}(\sqrt{5})$	998	1998	2996	3990	4990
$\theta = 2\tan^{-1}(1+11\sqrt{7})$	1000	1998	2996	3992	4990
$4\log_2(1/\varepsilon)$	1329	2658	3986	5315	6644
$3\log_2(1/\varepsilon)$	997	1993	2990	3986	4983

Table 2: Worst-case and typical T-counts. The top part of the table shows the T-counts for various combinations of θ and ε. The bottom part of the table shows the actual values of $4\log_2(1/\varepsilon)$ and $3\log_2(1/\varepsilon)$ for each ε, rounded to the closest integer. As predicted by Conjecture 8.10, the T-counts are close to $4\log_2(1/\varepsilon)$ when $\theta = 2\tan^{-1}(r)$, for $r \in \mathbb{Q}(\sqrt{2})$, and close to $3\log_2(1/\varepsilon)$ otherwise.

the algorithm exceeds this lower bound by at most a very small amount, which is consistent with $O(\log(\log(1/\varepsilon)))$ as predicted by Proposition 8.8(b). This excess could be further reduced by increasing the amount of effort spent on factoring, and will become zero in the presence of a factoring oracle.

Table 1 also shows the runtime as a function of ε. Since the enumeration of solutions to the grid problem in the algorithm is deterministic, the algorithm tends to find the same one or two solutions each time it is run with the same parameters. For this reason, we have averaged the runtimes in Table 1 over 50 runs of the algorithm with random angles θ, for each ε. As shown in Table 1, we were able to achieve approximations up to $\varepsilon = 10^{-1000}$ with a T-count of under 10000 in less than 50 seconds on average. This compares to a T-count of 13300 and an average runtime of 504.8 seconds reported in [14] on the same hardware. We also note that the experimental runtimes in Table 1 are consistent with the polynomial runtime predicted by Proposition 8.11, and appear to be $O(\log^3(1/\varepsilon))$.

Table 2 shows T-counts as a function of θ and ε. As predicted by Conjecture 8.10, the T-counts are close to $4\log_2(1/\varepsilon)$ when $\theta = 2\tan^{-1}(r)$, for $r \in \mathbb{Q}(\sqrt{2})$, and close to $3\log_2(1/\varepsilon)$ otherwise.

11 Conclusion

We have presented a fast new probabilistic algorithm for approximating arbitrary single-qubit z-rotations by Clifford+T circuits. Our algorithm is optimal in the presence of a factoring oracle, i.e., it finds the shortest possible circuit whatsoever for the given problem instance. In the absence of a factoring oracle, our algorithm is still nearly optimal. In particular, no quantum computer is required to run our algorithm. The main technical innovation of this paper is a new efficient algorithm for solving two-dimensional grid problems, such as the ones that arise in candidate selection for approximate synthesis. We solved this problem by an iterative method that successively increases the uprightness of a pair of convex sets until the problem is in a form where it can be solved directly.

It is an interesting question whether a similar algorithm can be found for giving optimal or near-optimal approximations of arbitrary single-qubit operators. In its current form, our method only applies to z-rotations. Since any arbitrary single-qubit operator can be decomposed into three z-rotations using Euler angles, our algorithm can currently achieve a T-count of $9\log_2(1/\varepsilon) + O(\log(\log(1/\varepsilon)))$. (Interestingly, the average case gate complexity can always be achieved in this situation, because if the operator to be approximated happens to exhibit worst-case behavior, we can always change it by multiplying it by a small number of random Clifford+T gates). However, the information-theoretic lower bound in this situation remains $K+3\log_2(1/\varepsilon)$, so there is still potential for improvement.

12 Acknowledgements

This research was supported by the Natural Sciences and Engineering Research Council of Canada (NSERC). This research was supported by the Intelligence Advanced Research Projects Activity (IARPA) via Department of Interior National Business Center contract number D12PC00527. The U.S. Government is authorized to reproduce and distribute reprints for Governmental purposes notwithstanding any copyright annotation thereon. Disclaimer: The views and conclusions contained herein are those of the authors and should not be interpreted as necessarily representing the official policies or endorsements, either expressed or implied, of IARPA, DoI/NBC, or the U.S. Government.

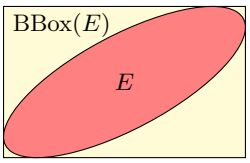
A Proof of Theorem 5.16

This appendix contains a proof of Theorem 5.16. We start by reformulating the problem in more convenient terms. Recall that the notion of uprightness introduced in Section 5.2 was defined for an arbitrary bounded convex subset of $\mathbb{R}^2$. If the set in question is an ellipse, we can expand the definition of uprightness into an explicit expression. Recall from Definition 5.15 that an ellipse centered at p can be written as $E = \{u \in \mathbb{R}^2 \mid (u-p)^\dagger D(u-p) \leqslant 1\}$, where D is a positive definite matrix whose entries are, e.g., as follows:

$$D = \begin{bmatrix} a & b \\ b & d \end{bmatrix}.$$

We can compute the area of E and the area of its bounding box using D. Indeed, we have $\text{area}(E) = \pi/\sqrt{\det(D)}$ and $\text{area}(\text{BBox}(E)) = 4\sqrt{ad}/\det(D)$. Substituting these in Definition 5.7 yields the desired expression for uprightness:

$$\text{up}(E) = \frac{\text{area}(E)}{\text{area}(\text{BBox}(E))} = \frac{\pi}{4}\sqrt{\frac{\det(D)}{ad}}. \tag{30}$$



It follows that the uprightness of E is invariant under translation and scalar multiplication.

Recall that $\lambda = \sqrt{2}+1$. The matrix D corresponding to an ellipse E has determinant 1 if and only if it can be written in the form

$$D = \begin{bmatrix} e\lambda^{-z} & b \\ b & e\lambda^{z} \end{bmatrix} \tag{31}$$

for some $b, e, z \in \mathbb{R}$ with $e > 0$ and $e^2 = b^2 + 1$. In this case, the definition of uprightness (30) simplifies to

$$\text{up}(E) = \frac{\pi}{4e^2} = \frac{\pi}{4\sqrt{b^2+1}}. \tag{32}$$

Equivalently, if $\mathrm{up}(E) = M$, then

$$b^2 = \frac{\pi^2}{16M^2} - 1. \tag{33}$$

Since Theorem 5.16 deals with pairs of ellipses, it is convenient to introduce the following terminology for discussing pairs of matrices.

Definition A.1. A *state* is a pair of real symmetric positive definite matrices of determinant 1. Given a state (D, Δ) with

$$D = \begin{bmatrix} e\lambda^{-z} & b \\ b & e\lambda^{z} \end{bmatrix} \qquad \Delta = \begin{bmatrix} \varepsilon\lambda^{-\zeta} & \beta \\ \beta & \varepsilon\lambda^{\zeta} \end{bmatrix} \tag{34}$$

we define its *skew* as $\texttt{Skew}(D, \Delta) = b^2 + \beta^2$ and its *bias* as $\texttt{Bias}(D, \Delta) = \zeta - z$.

Note that the skew of a state is small if and only if both b^2 and β^2 are small, which happens, by (32), if and only if the ellipses corresponding to D and Δ both have large uprightness. So our strategy for increasing the uprightness will be to reduce the skew. In what follows, we use (D, Δ) to denote an arbitrary state and always assume that the entries of D and Δ are given as in (34). For future reference, we record here another useful property of states.

Remark A.2. If (D, Δ) is a state with $b \geqslant 0$, then $-be \leqslant -b^2$. Indeed:

$$e^2 = b^2 + 1 \implies e^2 \geqslant b^2 \implies e \geqslant b \implies -be \leqslant -b^2.$$

Similarly, if $b \leqslant 0$, then $be \leqslant -b^2$. Analogous inequalities also hold for β and ε.

The action of a grid operator on an ellipse can be adapted to states in a natural way, provided that the operator is special.

Definition A.3. The action of special grid operators on states is defined as follows. Here, $G^\dagger$ denotes the transpose of G, and $G^\bullet$ is defined by applying $(-)^\bullet$ separately to each matrix entry, as in Remark 5.12.

$$(D, \Delta) \cdot G = (G^\dagger D G, G^{\bullet\dagger} \Delta G^\bullet).$$

Lemma A.4. *Let (D, Δ) be a state, and let A and B be the ellipses centered at the origin that are defined by D and Δ, respectively. Then the ellipses $G(A)$ and $G^\bullet(B)$ are defined by the matrices D' and Δ', where*

$$(D', \Delta') = (D, \Delta) \cdot G^{-1}$$

Proof. We have

$$\begin{aligned} G(A) &= \{G(u) \in \mathbb{R}^2 \mid u^\dagger D u \leqslant 1\} \\ &= \{v \in \mathbb{R}^2 \mid (G^{-1}v)^\dagger D (G^{-1}v) \leqslant 1\} \\ &= \{v \in \mathbb{R}^2 \mid v^\dagger (G^{-1})^\dagger D G^{-1} v \leqslant 1\}, \end{aligned}$$

so the ellipse $G(A)$ is defined by the positive operator $D' = (G^{-1})^\dagger D G^{-1}$. The proof for $G^\bullet(B)$ is similar. □

The main ingredient in the proof of Theorem 5.16 is the following Step Lemma.

Lemma A.5 (Step Lemma)**.** *For any state (D, Δ), if $\texttt{Skew}(D, \Delta) \geqslant 15$, then there exists a special grid operator G such that $\texttt{Skew}((D, \Delta) \cdot G) \leqslant 0.9\ \texttt{Skew}(D, \Delta)$. Moreover, G can be computed using a constant number of arithmetic operations.*

Before proving the Step Lemma, we show how it can be used to derive Theorem 5.16, whose statement we reproduce here.

Theorem. *Suppose $A, B \subseteq \mathbb{R}^2$ are ellipses. Then there exists a grid operator G such that $G(A)$ and $G^\bullet(B)$ are $1/6$-upright. Moreover, if A and B are M-upright, then G can be efficiently computed in $O(\log(1/M))$ arithmetic operations.*

Proof. Let D and Δ be the matrices defining A and B respectively, in the sense of Definition 5.15. Since uprightness is invariant under translations and scaling, we may without loss of generality assume that both ellipses are centered at the origin, and that $\det D = \det \Delta = 1$.

The pair (D, Δ) is a state. By applying Lemma A.5 repeatedly, we get grid operators $G_1, \ldots, G_n$ such that:

$$\texttt{Skew}((D, \Delta) \cdot G_1 \ldots G_n) \leqslant 15. \tag{35}$$

$$R = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} \quad A = \begin{bmatrix} 1 & -2 \\ 0 & 1 \end{bmatrix} \quad B = \begin{bmatrix} 1 & \sqrt{2} \\ 0 & 1 \end{bmatrix}$$
$$K = \frac{1}{\sqrt{2}}\begin{bmatrix} -\lambda^{-1} & -1 \\ \lambda & 1 \end{bmatrix} \quad X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \quad Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Figure 6: List of useful grid operators.

Now let $(D', \Delta') = (D, \Delta) \cdot G_1 \dots G_n$ and set $G = (G_1 \cdots G_n)^{-1}$. By Lemma A.4, the ellipses $G(A)$ and $G^\bullet(B)$ are defined by the matrices D' and Δ', respectively. Let b and β be the anti-diagonal entries of the matrices D' and Δ', respectively. We have:

$$b^2 + \beta^2 = \texttt{Skew}(D', \Delta') = \texttt{Skew}((D, \Delta) \cdot G^{-1}) = \texttt{Skew}((D, \Delta) \cdot G_1 \dots G_n) \leqslant 15,$$

hence $b^2 \leqslant 15$ and $\beta^2 \leqslant 15$. Using (32), we get

$$\text{up}(G(A)) = \frac{\pi}{4\sqrt{b^2+1}} \geqslant \frac{\pi}{4\sqrt{16}} \geqslant 1/6 \quad \text{and} \quad \text{up}(G^\bullet(B)) = \frac{\pi}{4\sqrt{\beta^2+1}} \geqslant \frac{\pi}{4\sqrt{16}} \geqslant 1/6,$$

as desired.

To bound the number of operations, note that each application of G_j reduces the skew by at least 10 percent. Therefore, the number n in (35) satisfies $n \leqslant \log_{0.9}(15/\texttt{Skew}(D, \Delta)) = O(\log(\texttt{Skew}(D, \Delta)))$. Using (33), we have

$$\log(\texttt{Skew}(D, \Delta)) = \log(b^2 + \beta^2) \leqslant \log((\frac{\pi^2}{16M^2} - 1) + (\frac{\pi^2}{16M^2} - 1)) = O(\log(1/M)).$$

It follows that the computation of G requires $O(\log(1/M))$ applications of the Step Lemma, each of which requires a constant number of arithmetic operations, proving the final claim of the theorem. □

The remainder of this appendix is devoted to proving the Step Lemma. To each state, we associate the pair (z, ζ). The proof of the Step Lemma is essentially a case distinction on the location of the pair (z, ζ) in the plane. We find coverings of the plane with the property that if the point (z, ζ) belongs to some region $\mathcal{O}$ of our covering, then it is easy to compute a special grid operator G such that $\texttt{Skew}((D, \Delta) \cdot G) \leqslant 0.9\ \texttt{Skew}(D, \Delta)$. The relevant grid operators are given in Figure 6. Each one of the next 5 subsections is dedicated to a particular region of the plane. We prove the Step Lemma in Section A.6.

A.1 The Shift Lemma

In this section, we consider states (D, Δ) such that $|\texttt{Bias}(D, \Delta)| > 1$. Any such state can be "shifted" to a state (D', Δ') of equal skew but with $|\texttt{Bias}(D', \Delta')| \leqslant 1$.

Definition A.6. The *shift operators* σ and τ are defined by:

$$\sigma = \sqrt{\lambda^{-1}}\begin{bmatrix} \lambda & 0 \\ 0 & 1 \end{bmatrix}, \tau = \sqrt{\lambda^{-1}}\begin{bmatrix} 1 & 0 \\ 0 & -\lambda \end{bmatrix}$$

Even though σ and τ are not grid operators, we can use them to define an operation on states called a *shift by* k. By abuse of notation, we write this operation as an action.

Definition A.7. Given a state (D, Δ) and $k \in \mathbb{Z}$, the *k-shift of* (D, Δ) is defined as:

$$(D, \Delta) \cdot \text{Shift}^k = (\sigma^k D \sigma^k, \tau^k \Delta \tau^k).$$

The notation $(D, \Delta) \cdot \text{Shift}^k$ is justified by the following lemma.

Lemma A.8. *The shift of a state is a state and moreover:*

$$\texttt{Skew}((D, \Delta) \cdot \textit{Shift}^k) = \texttt{Skew}(D, \Delta) \quad \textit{and} \quad \texttt{Bias}((D, \Delta) \cdot \textit{Shift}^k) = \texttt{Bias}(D, \Delta) + 2k$$

Proof. Compute $(D,\Delta)\cdot \text{Shift}^k$:

$$\begin{aligned}(D,\Delta)\cdot\text{Shift}^k &= (\sigma^k D\sigma^k, \tau^k\Delta\tau^k)\\ &= (\sigma^k \begin{bmatrix} e\lambda^{-z} & b \\ b & e\lambda^{z}\end{bmatrix}\sigma^k, \tau^k\begin{bmatrix}\varepsilon\lambda^{-\zeta} & \beta\\ \beta & \varepsilon\lambda^{\zeta}\end{bmatrix}\tau^k)\\ &= (\begin{bmatrix} e\lambda^{-z+k} & b \\ b & e\lambda^{z-k}\end{bmatrix}, \begin{bmatrix}\varepsilon\lambda^{-\zeta-k} & (-1)^k\beta\\ (-1)^k\beta & \varepsilon\lambda^{\zeta+k}\end{bmatrix})\end{aligned}$$

The resulting matrices are clearly symmetric and positive definite. Moreover, since σ^k and τ^k have determinant ± 1, both $\sigma^k D\sigma^k$ and $\tau^k\Delta\tau^k$ have determinant 1. Finally:

- $\texttt{Skew}((D,\Delta)\cdot\text{Shift}^k) = b^2 + ((-1)^k\beta)^2 = b^2+\beta^2 = \texttt{Skew}(D,\Delta)$ and
- $\texttt{Bias}((D,\Delta)\cdot\text{Shift}^k) = (\zeta+k)-(z-k) = \texttt{Bias}(D,\Delta)+2k$,

which completes the proof. □

For every special grid operator G, there is a special grid operator G' whose action on a state corresponds to shifting the state by k, applying G and then shifting the state by $-k$.

Lemma A.9. *If G is a special grid operator and $k\in\mathbb{Z}$, then $G' = \sigma^k G\sigma^k$ is a special grid operator and moreover $G'^\bullet = (-\tau)^k G^\bullet \tau^k$.*

Proof. It suffices to show this for $k=1$. Suppose $G = \begin{bmatrix} w & x\\ y & z\end{bmatrix}$ is a special grid operator and note that:

$$G' = \sigma G\sigma = \begin{bmatrix}\lambda w & x\\ y & \lambda^{-1}z\end{bmatrix} = \begin{bmatrix}\lambda^{-1} & 0\\ 0 & \lambda^{-1}\end{bmatrix}\begin{bmatrix}\lambda & 0\\ 0 & 1\end{bmatrix} G \begin{bmatrix}\lambda & 0\\ 0 & 1\end{bmatrix}.$$

Since all the factors in the above product are grid operators, the result is also a grid operator. Moreover $\det(\sigma G\sigma) = \det(G) = 1$ so that $\sigma G\sigma$ is special. Finally:

$$G'^\bullet = (\sigma G\sigma)^\bullet = \begin{bmatrix}\lambda^\bullet w^\bullet & x^\bullet\\ y^\bullet & (\lambda^{-1})^\bullet z^\bullet\end{bmatrix} = \begin{bmatrix}-\lambda^{-1}w^\bullet & x^\bullet\\ y^\bullet & -\lambda z^\bullet\end{bmatrix} = -\tau G^\bullet\tau.$$

□

Lemma A.10. *If G is a grid operator, then:*

$$(((D,\Delta)\cdot \textit{Shift}^k)\cdot G)\cdot \textit{Shift}^k = (D,\Delta)\cdot(\sigma^k G\sigma^k).$$

Proof. Write $G' = \sigma^k G\sigma^k$. Simple computation then yields the result:

$$\begin{aligned}(((D,\Delta)\cdot\text{Shift}^k)\cdot G)\cdot\text{Shift}^k &= ((\sigma^k D\sigma^k,\, \tau^k\Delta\tau^k)\cdot G)\cdot\text{Shift}^k\\ &= (G^\dagger\sigma^k D\sigma^k G,\, G^{\bullet\dagger}\tau^k\Delta\tau^k G^\bullet)\cdot\text{Shift}^k\\ &= (\sigma^k G^\dagger\sigma^k D\sigma^k G\sigma^k,\, \tau^k G^{\bullet\dagger}\tau^k\Delta\tau^k G^\bullet\tau^k)\\ &= (\sigma^k G^\dagger\sigma^k D\sigma^k G\sigma^k,\, ((-\tau)^k G^{\bullet\dagger}\tau^k)\Delta((-\tau)^k G^\bullet\tau^k))\\ &= (G'^\dagger D G',\, G'^{\bullet\dagger}\Delta G'^\bullet)\\ &= (D,\Delta)\cdot G'\\ &= (D,\Delta)\cdot(\sigma^k G\sigma^k).\end{aligned}$$

□

Shifts allow us to consider only states (D,Δ) with $\texttt{Bias}(D,\Delta)\in[-1,1]$ in the proof of the Step Lemma.

Lemma A.11. *If the Step Lemma holds for all states (D,Δ) with $\texttt{Bias}(D,\Delta)\in[-1,1]$, then it holds for all states.*

Proof. Let (D,Δ) be some state with $\texttt{Skew}(D,\Delta)\geqslant 15$. Let $x = \texttt{Bias}(D,\Delta)$ and set $k = \lfloor\frac{1-x}{2}\rfloor$. Then by Lemma A.8, we have $\texttt{Skew}((D,\Delta)\cdot\text{Shift}^k) = \texttt{Skew}(D,\Delta)$ and $\texttt{Bias}((D,\Delta)\cdot\text{Shift}^k)\in[-1,1]$. Then by assumption, there exists a special grid operator G such that $\texttt{Skew}(((D,\Delta)\cdot\text{Shift}^k)\cdot G)\leqslant 0.9\,\texttt{Skew}((D,\Delta)\cdot\text{Shift}^k)$. Now by Lemma A.9 we know that $G' = \sigma^k\, G\, \sigma^k$ is a special grid operator. Moreover, by Lemma A.10 and A.8, we have:

$$\begin{aligned}\texttt{Skew}((D,\Delta)\cdot G') &= \texttt{Skew}((((D,\Delta)\cdot\text{Shift}^k)\cdot G)\cdot\text{Shift}^k)\\ &= \texttt{Skew}(((D,\Delta)\cdot\text{Shift}^k)\cdot G)\\ &\leqslant 0.9\,\texttt{Skew}((D,\Delta)\cdot\text{Shift}^k)\\ &= 0.9\,\texttt{Skew}(D,\Delta),\end{aligned}$$

which completes the proof. □

A.2 The R Lemma

Definition A.12. The *hyperbolic sine in base* λ is defined as:

$$\sinh_\lambda(x) = \frac{\lambda^x - \lambda^{-x}}{2}.$$

Lemma A.13. *Recall the operator R from Figure 6. If (D, Δ) is such that $\texttt{Skew}(D, \Delta) \geqslant 15$, and such that $-0.8 \leqslant z \leqslant 0.8$ and $-0.8 \leqslant \zeta \leqslant 0.8$, then:*

$$\texttt{Skew}((D, \Delta) \cdot R) \leqslant 0.9\ \texttt{Skew}(D, \Delta).$$

Proof. Compute the action of R on (D, Δ):

$$\begin{aligned}
R^\dagger D R &= \frac{1}{2}\begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix}\begin{bmatrix} e\lambda^{-z} & b \\ b & e\lambda^{z} \end{bmatrix}\begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} \\
&= \begin{bmatrix} \dots & \frac{e(\lambda^z - \lambda^{-z})}{2} \\ \frac{e(\lambda^z - \lambda^{-z})}{2} & \dots \end{bmatrix} = \begin{bmatrix} \dots & e\sinh_\lambda(z) \\ e\sinh_\lambda(z) & \dots \end{bmatrix}, \\
R^{\bullet\dagger} \Delta R^\bullet &= \frac{1}{2}\begin{bmatrix} -1 & -1 \\ 1 & -1 \end{bmatrix}\begin{bmatrix} \varepsilon\lambda^{-\zeta} & \beta \\ \beta & \varepsilon\lambda^{\zeta} \end{bmatrix}\begin{bmatrix} -1 & 1 \\ -1 & -1 \end{bmatrix} \\
&= \begin{bmatrix} \dots & \frac{\varepsilon(\lambda^\zeta - \lambda^{-\zeta})}{2} \\ \frac{\varepsilon(\lambda^\zeta - \lambda^{-\zeta})}{2} & \dots \end{bmatrix} = \begin{bmatrix} \dots & \varepsilon\sinh_\lambda(\zeta) \\ \varepsilon\sinh_\lambda(\zeta) & \dots \end{bmatrix}.
\end{aligned}$$

Therefore $\texttt{Skew}((D, \Delta) \cdot R) = e^2 \sinh^2_\lambda(z) + \varepsilon^2 \sinh^2_\lambda(\zeta)$. But recall that $e^2 = b^2 + 1$ and $\varepsilon^2 = \beta^2 + 1$, so that in fact:

$$\texttt{Skew}((D, \Delta) \cdot R) = (b^2 + 1)\sinh^2_\lambda(z) + (\beta^2 + 1)\sinh^2_\lambda(\zeta).$$

We assumed $-0.8 \leqslant z, \zeta \leqslant 0.8$ and this implies that $\sinh^2_\lambda(\zeta), \sinh^2_\lambda(z) \leqslant \sinh^2_\lambda(0.8)$. Writing $y = \sinh^2_\lambda(0.8)$ for brevity, and using the assumption that $\texttt{Skew}(D, \Delta) \geqslant 15$, we get:

$$\begin{aligned}
\texttt{Skew}((D, \Delta) \cdot R) &= (b^2 + 1)\sinh^2_\lambda(z) + (\beta^2 + 1)\sinh^2_\lambda(\zeta) \\
&\leqslant (b^2 + 1)y + (\beta^2 + 1)y \\
&= (b^2 + \beta^2 + 2)y \\
&\leqslant \texttt{Skew}(D, \Delta)(1 + \tfrac{2}{15})y.
\end{aligned}$$

This completes the proof, since $(1 + \frac{2}{15})y = (1 + \frac{2}{15})\sinh^2_\lambda(0.8) \approx 0.663 \leqslant 0.9$. □

A.3 The K Lemma

Definition A.14. The *hyperbolic cosine in base* λ is defined as:

$$\cosh_\lambda(x) = \frac{\lambda^x + \lambda^{-x}}{2}.$$

Lemma A.15. *Recall the operator K from Figure 6. If (D, Δ) is such that $\texttt{Bias}(D, \Delta) \in [-1, 1]$, $\texttt{Skew}(D, \Delta) \geqslant 15$, and such that $b, \beta \geqslant 0$, $z \leqslant 0.3$, and $0.8 \leqslant \zeta$, then:*

$$\texttt{Skew}((D, \Delta) \cdot K) \leqslant 0.9\ \texttt{Skew}(D, \Delta).$$

Proof. Compute the action of K on (D, Δ):

$$\begin{aligned}
&K^\dagger D K \\
&= \frac{1}{2}\begin{bmatrix} -\lambda^{-1} & \lambda \\ -1 & 1 \end{bmatrix}\begin{bmatrix} e\lambda^{-z} & b \\ b & e\lambda^{z} \end{bmatrix}\begin{bmatrix} -\lambda^{-1} & -1 \\ \lambda & 1 \end{bmatrix} \\
&= \frac{1}{2}\begin{bmatrix} \dots & e(\lambda^{z+1} + \lambda^{-z-1}) - 2\sqrt{2}b \\ e(\lambda^{z+1} + \lambda^{-z-1}) - 2\sqrt{2}b & \dots \end{bmatrix} \\
&= \begin{bmatrix} \dots & e\cosh_\lambda(z+1) - \sqrt{2}b \\ e\cosh_\lambda(z+1) - \sqrt{2}b & \dots \end{bmatrix},
\end{aligned}$$

$$
\begin{aligned}
&K^{\bullet\dagger}\Delta K^{\bullet}\\
&= \frac{1}{2}\begin{bmatrix} \lambda & -\lambda^{-1} \\ -1 & 1 \end{bmatrix}\begin{bmatrix} \varepsilon\lambda^{-\zeta} & \beta \\ \beta & \varepsilon\lambda^{\zeta} \end{bmatrix}\begin{bmatrix} \lambda & -1 \\ -\lambda^{-1} & 1 \end{bmatrix}\\
&= \frac{1}{2}\begin{bmatrix} \dots & -\varepsilon(\lambda^{\zeta-1}+\lambda^{-\zeta+1})+2\sqrt{2}\beta \\ -\varepsilon(\lambda^{\zeta-1}+\lambda^{-\zeta+1})+2\sqrt{2}\beta & \dots \end{bmatrix}\\
&= \begin{bmatrix} \dots & \sqrt{2}\beta-\varepsilon\cosh_\lambda(\zeta-1) \\ \sqrt{2}\beta-\varepsilon\cosh_\lambda(\zeta-1) & \dots \end{bmatrix}.
\end{aligned}
$$

Therefore:

$$\texttt{Skew}((D,\Delta)\cdot K) = (\sqrt{2}b - e\cosh_\lambda(z+1))^2 + (\sqrt{2}\beta - \varepsilon\cosh_\lambda(\zeta-1))^2. \tag{36}$$

But recall that $e^2 = b^2+1$, and from Remark A.2 that $b \geqslant 0$ implies $-be \leqslant -b^2$, so:

$$
\begin{aligned}
&(\sqrt{2}b - e\cosh_\lambda(z+1))^2\\
&\quad= 2b^2 - 2\sqrt{2}\,be\cosh_\lambda(z+1) + e^2\cosh^2_\lambda(z+1)\\
&\quad\leqslant 2b^2 - 2\sqrt{2}\,b^2\cosh_\lambda(z+1) + (b^2+1)\cosh^2_\lambda(z+1)\\
&\quad= b^2(2 - 2\sqrt{2}\cosh_\lambda(z+1) + \cosh^2_\lambda(z+1)) + \cosh^2_\lambda(z+1)\\
&\quad= b^2(\sqrt{2} - \cosh_\lambda(z+1))^2 + \cosh^2_\lambda(z+1).
\end{aligned}\tag{37}
$$

Reasoning analogously, we also have

$$(\sqrt{2}\beta - \varepsilon\cosh_\lambda(\zeta-1))^2 \;\leqslant\; \beta^2(\sqrt{2} - \cosh_\lambda(\zeta-1))^2 + \cosh^2_\lambda(\zeta-1). \tag{38}$$

By assumption, $\texttt{Bias}(D,\Delta) \in [-1,1]$, thus $\zeta \leqslant z+1$. This, together with the assumptions $0.8 \leqslant \zeta$ and $z \leqslant 0.3$, implies that both $z+1$ and $\zeta-1$ are in the interval $[-0.2, 1.3]$. On this interval, the function $\cosh^2_\lambda(x)$ assumes its maximum at $x = 1.3$, and the function $f(x) = (\sqrt{2} - \cosh_\lambda(x))^2$ assumes its maximum at $x = 0$. Therefore,

$$b^2(\sqrt{2} - \cosh_\lambda(z+1))^2 + \cosh^2_\lambda(z+1) \;\leqslant\; b^2(\sqrt{2} - \cosh_\lambda(0))^2 + \cosh^2_\lambda(1.3). \tag{39}$$

and

$$\beta^2(\sqrt{2} - \cosh_\lambda(\zeta-1))^2 + \cosh^2_\lambda(\zeta-1) \;\leqslant\; \beta^2(\sqrt{2} - \cosh_\lambda(0))^2 + \cosh^2_\lambda(1.3). \tag{40}$$

Combining (36)–(40), together with the assumption that $\texttt{Skew}(D,\Delta) \geqslant 15$, yields:

$$
\begin{aligned}
\texttt{Skew}((D,\Delta)\cdot K) &= (\sqrt{2}b - e\cosh_\lambda(z+1))^2 + (\sqrt{2}\beta - \varepsilon\cosh_\lambda(\zeta-1))^2\\
&\leqslant (b^2+\beta^2)(\sqrt{2} - \cosh_\lambda(0))^2 + 2\cosh^2_\lambda(1.3)\\
&= \texttt{Skew}(D,\Delta)(\sqrt{2} - \cosh_\lambda(0))^2 + 2\cosh^2_\lambda(1.3)\\
&\leqslant \texttt{Skew}(D,\Delta)((\sqrt{2} - \cosh_\lambda(0))^2 + \frac{2}{15}\cosh^2_\lambda(1.3))
\end{aligned}
$$

This completes the proof since $(\sqrt{2} - \cosh_\lambda(0))^2 + \frac{2}{15}\cosh^2_\lambda(1.3) \approx 0.571 \leqslant 0.9$. □

A.4 The A Lemma

Definition A.16. Let $g(x) = (1-2x)^2$.

Lemma A.17. *Recall the operator A from Figure 6. If (D,Δ) is such that $\texttt{Bias}(D,\Delta) \in [-1,1]$, $\texttt{Skew}(D,\Delta) \geqslant 15$, and such that $b, \beta \geqslant 0$ and $0.3 \leqslant z, \zeta$, then there exists $n \in \mathbb{Z}$ such that:*

$$\texttt{Skew}((D,\Delta)\cdot A^n) \leqslant 0.9\ \texttt{Skew}(D,\Delta).$$

Proof. Let $c = \min\{z,\zeta\}$ and $n = \max\{1, \lfloor\frac{\lambda^c}{2}\rfloor\}$. Compute the action of A^n on (D,Δ):

$$
\begin{aligned}
A^{n\dagger}DA^n &= \begin{bmatrix} 1 & 0 \\ -2n & 1 \end{bmatrix}\begin{bmatrix} e\lambda^{-z} & b \\ b & e\lambda^{z} \end{bmatrix}\begin{bmatrix} 1 & -2n \\ 0 & 1 \end{bmatrix}\\
&= \begin{bmatrix} \dots & b - 2ne\lambda^{-z} \\ b - 2ne\lambda^{-z} & \dots \end{bmatrix},
\end{aligned}
$$

$$
\begin{aligned}
A^{n\bullet\dagger}\Delta A^{n\bullet} &= A^{n\dagger}\Delta A^n\\
&= \begin{bmatrix} \dots & \beta - 2n\varepsilon\lambda^{-\zeta} \\ \beta - 2n\varepsilon\lambda^{-\zeta} & \dots \end{bmatrix}.
\end{aligned}
$$

Therefore:

$$\texttt{Skew}((D,\Delta)\cdot A^n) = (b - 2ne\lambda^{-z})^2 + (\beta - 2n\varepsilon\lambda^{-\zeta})^2$$

But recall that $e^2 = b^2 + 1$ and $\varepsilon^2 = \beta^2 + 1$, and from Remark A.2 that $b, \beta \geqslant 0$ implies $-be \leqslant -b^2$ and $-\varepsilon\beta \leqslant -\beta^2$. Using these facts, we can expand the above formula as follows:

$$\begin{array}{rcl}
\texttt{Skew}((D,\Delta)\cdot A^n) & & \\
& = & (b - 2ne\lambda^{-z})^2 + (\beta - 2n\varepsilon\lambda^{-\zeta})^2 \\
& = & b^2 - 4nbe\lambda^{-z} + 4n^2e^2\lambda^{-2z} + \beta^2 - 4n\beta\varepsilon\lambda^{-\zeta} + 4n^2\varepsilon^2\lambda^{-2\zeta} \\
& \leqslant & b^2 - 4nb^2\lambda^{-z} + 4n^2(b^2+1)\lambda^{-2z} + \beta^2 - 4n\beta^2\lambda^{-\zeta} + 4n^2(\beta^2+1)\lambda^{-2\zeta} \\
& = & b^2(1 - 4n\lambda^{-z} + 4n^2\lambda^{-2z}) + \beta^2(1 - 4n\lambda^{-\zeta} + 4n^2\lambda^{-2\zeta}) + 4n^2(\lambda^{-2z} + \lambda^{-2\zeta}) \\
& = & b^2(1 - 2n\lambda^{-z})^2 + \beta^2(1 - 2n\lambda^{-\zeta})^2 + 4n^2(\lambda^{-2z} + \lambda^{-2\zeta}) \\
& = & b^2 g(n\lambda^{-z}) + \beta^2 g(n\lambda^{-\zeta}) + 4n^2(\lambda^{-2z} + \lambda^{-2\zeta}).
\end{array}$$

Writing $y = \max\{g(n\lambda^{-z}), g(n\lambda^{-\zeta})\}$ for brevity, and using the assumption that $\texttt{Skew}(D,\Delta) \geqslant 15$ together with the fact that $c \leqslant z, \zeta$, we get:

$$\begin{array}{rcl}
\texttt{Skew}((D,\Delta)\cdot A^n) & \leqslant & b^2 y + \beta^2 y + 8n^2\lambda^{-2c} \\
& = & \texttt{Skew}(D,\Delta)y + 8n^2\lambda^{-2c} \\
& \leqslant & \texttt{Skew}(D,\Delta)(y + \frac{8}{15}n^2\lambda^{-2c}).
\end{array}$$

To finish the proof, it remains to show that $y + \frac{8}{15}n^2\lambda^{-2c} \leqslant 0.9$. There are two cases:

- If $\lfloor \frac{\lambda^c}{2} \rfloor \geqslant 1$, then $\frac{\lambda^c}{4} \leqslant n \leqslant \frac{\lambda^c}{2}$. From $n \leqslant \frac{\lambda^c}{2}$, we have $2n\lambda^{-c} \leqslant 1$, and so $\frac{8}{15}n^2\lambda^{-2c} \leqslant \frac{2}{15}$. Moreover, because $\texttt{Bias}(D,\Delta) \in [-1,1]$, we have $c \leqslant z, \zeta \leqslant c+1$. Hence $\frac{1}{4\lambda} = \frac{\lambda^c}{4}\lambda^{-c-1} \leqslant n\lambda^{-c-1} \leqslant n\lambda^{-z}, n\lambda^{-\zeta} \leqslant n\lambda^{-c} \leqslant \frac{1}{2}$. On the interval $[\frac{1}{4\lambda}, \frac{1}{2}]$, the function $g(x)$ assumes its maximum at $x = \frac{1}{4\lambda}$. This implies that $y \leqslant g(\frac{1}{4\lambda})$. This completes the present case since we get:

$$y + \frac{8}{15}n^2\lambda^{-2c} \leqslant g(\frac{1}{4\lambda}) + \frac{2}{15} \approx 0.762 \leqslant 0.9.$$

- If $\lfloor \frac{\lambda^c}{2} \rfloor < 1$, then $n = 1$ and $\lambda^c < 2$. From $0.3 \leqslant c$, we have $\frac{8}{15}n^2\lambda^{-2c} \leqslant \frac{8}{15}\lambda^{-0.6}$. Moreover, because $\texttt{Bias}(D,\Delta) \in [-1,1]$, we have $0.3 \leqslant c \leqslant z, \zeta \leqslant c+1$. With $\lambda^c \leqslant 2$, this implies that $\frac{1}{2\lambda} \leqslant \lambda^{-c-1} \leqslant \lambda^{-z}, \lambda^{-\zeta} \leqslant \lambda^{-0.3}$. Therefore both λ^{-z} and $\lambda^{-\zeta}$ are in the interval $[\frac{1}{2\lambda}, \lambda^{-0.3}]$. On this interval, the function $g(x)$ assumes its maximum at $x = \frac{1}{2\lambda}$, and therefore $y \leqslant g(\frac{1}{2\lambda})$. This completes the proof since:

$$y + \frac{8}{15}n^2\lambda^{-2c} \leqslant g(\frac{1}{2\lambda}) + \frac{8}{15}\lambda^{-0.6} \approx 0.657 \leqslant 0.9.$$

□

A.5 The B Lemma

Definition A.18. Let $h(x) = (1 - \sqrt{2}x)^2$.

Lemma A.19. *Recall the operator B from Figure 6. If (D,Δ) is such that $\texttt{Bias}(D,\Delta) \in [-1,1]$, $\texttt{Skew}(D,\Delta) \geqslant 15$, and such that $b \leqslant 0 \leqslant \beta$ and $-0.2 \leqslant z, \zeta$, then there exists $n \in \mathbb{Z}$ such that:*

$$\texttt{Skew}((D,\Delta)\cdot B^n) \leqslant 0.9\ \texttt{Skew}(D,\Delta).$$

Proof. Let $c = \min\{z,\zeta\}$, $n = \max\{1, \lfloor \frac{\lambda^c}{\sqrt{2}} \rfloor\}$ and compute the action of B^n on (D,Δ):

$$\begin{array}{rcl}
B^{n\dagger}DB^n & = & \begin{bmatrix} 1 & 0 \\ \sqrt{2}n & 1 \end{bmatrix} \begin{bmatrix} e\lambda^{-z} & b \\ b & e\lambda^{z} \end{bmatrix} \begin{bmatrix} 1 & \sqrt{2}n \\ 0 & 1 \end{bmatrix} \\
& = & \begin{bmatrix} \cdots & b + \sqrt{2}ne\lambda^{-z} \\ b + \sqrt{2}ne\lambda^{-z} & \cdots \end{bmatrix},
\end{array}$$

$$\begin{array}{rcl}
B^{n\bullet\dagger}DB^{n\bullet} & = & \begin{bmatrix} 1 & 0 \\ -\sqrt{2}n & 1 \end{bmatrix} \begin{bmatrix} \varepsilon\lambda^{-\zeta} & \beta \\ \beta & \varepsilon\lambda^{\zeta} \end{bmatrix} \begin{bmatrix} 1 & -\sqrt{2}n \\ 0 & 1 \end{bmatrix} \\
& = & \begin{bmatrix} \cdots & \beta - \sqrt{2}n\varepsilon\lambda^{-\zeta} \\ \beta - \sqrt{2}n\varepsilon\lambda^{-\zeta} & \cdots \end{bmatrix}.
\end{array}$$

Therefore:

$$\texttt{Skew}((D,\Delta)\cdot B^n) = (b+\sqrt{2}\,ne\lambda^{-z})^2 + (\beta-\sqrt{2}\,n\varepsilon\lambda^{-\zeta})^2.$$

But recall that $e^2 = b^2+1$, that $\varepsilon^2 = \beta^2+1$, and from Remark A.2 that $b \leqslant 0 \leqslant \beta$ implies $be \leqslant -b^2$ and $-\beta\varepsilon \leqslant -\beta^2$. Using these facts, we can expand the above formula as follows:

$$\begin{array}{rcl}
\multicolumn{3}{l}{\texttt{Skew}((D,\Delta)\cdot B^n)} \\
&=& (b+\sqrt{2}\,ne\lambda^{-z})^2 + (\beta-\sqrt{2}\,n\varepsilon\lambda^{-\zeta})^2 \\
&=& b^2 + 2\sqrt{2}\,nbe\lambda^{-z} + 2n^2e^2\lambda^{-2z} + \beta^2 - 2\sqrt{2}\,n\beta\varepsilon\lambda^{-\zeta} + 2n^2\varepsilon^2\lambda^{-2\zeta} \\
&\leqslant& b^2 - 2\sqrt{2}\,nb^2\lambda^{-z} + 2n^2(b^2+1)\lambda^{-2z} + \beta^2 - 2\sqrt{2}\,n\beta^2\lambda^{-\zeta} + 2n^2(\beta^2+1)\lambda^{-2\zeta} \\
&=& b^2(1-2\sqrt{2}\,n\lambda^{-z}+2n^2\lambda^{-2z}) + \beta^2(1-2\sqrt{2}\,n\lambda^{-\zeta}+2n^2\lambda^{-2\zeta}) + 2n^2(\lambda^{-2z}+\lambda^{-2\zeta}) \\
&=& b^2(1-\sqrt{2}\,n\lambda^{-z})^2 + \beta^2(1-\sqrt{2}\,n\lambda^{-\zeta})^2 + 2n^2(\lambda^{-2z}+\lambda^{-2\zeta}). \\
&=& b^2h(n\lambda^{-z}) + \beta^2h(n\lambda^{-\zeta}) + 2n^2(\lambda^{-2z}+\lambda^{-2\zeta}).
\end{array}$$

Writing $y = \max\{h(n\lambda^{-z}), h(n\lambda^{-\zeta})\}$ for brevity, and using the assumption that $\texttt{Skew}(D,\Delta) \geqslant 15$, together with the fact that $c \leqslant z,\zeta$, we get:

$$\begin{array}{rcl}
\texttt{Skew}((D,\Delta)\cdot B^n) &\leqslant& b^2y + \beta^2y + 4n^2\lambda^{-2c} \\
&=& \texttt{Skew}(D,\Delta)y + 4n^2\lambda^{-2c} \\
&\leqslant& \texttt{Skew}(D,\Delta)(y + \frac{4}{15}n^2\lambda^{-2c}).
\end{array}$$

To finish the proof, it remains to show that $y + \frac{4}{15}n^2\lambda^{-2c} \leqslant 0.9$. There are two cases:

- If $\lfloor \frac{\lambda^c}{\sqrt{2}} \rfloor \geqslant 1$, then $\frac{\lambda^c}{2\sqrt{2}} \leqslant n \leqslant \frac{\lambda^c}{\sqrt{2}}$. From $n \leqslant \frac{\lambda^c}{\sqrt{2}}$, we have $2n^2\lambda^{-2c} \leqslant 1$, and so $\frac{4n^2\lambda^{-2c}}{15} \leqslant \frac{2}{15}$. Moreover, because $\texttt{Bias}(D,\Delta) \in [-1,1]$, we have $c \leqslant z,\zeta \leqslant c+1$. Hence $\frac{1}{2\sqrt{2}\,\lambda} = \frac{\lambda^c}{2\sqrt{2}}\lambda^{-c-1} \leqslant n\lambda^{-c-1} \leqslant n\lambda^{-z}, n\lambda^{-\zeta} \leqslant n\lambda^{-c} \leqslant \frac{1}{\sqrt{2}}$. On the interval $[\frac{1}{2\sqrt{2}\,\lambda}, \frac{1}{\sqrt{2}}]$, the function $h(x)$ assumes its maximum at $x = \frac{1}{2\sqrt{2}\,\lambda}$. This implies that $y \leqslant h(\frac{1}{2\sqrt{2}\,\lambda})$. This completes the present case since we get:

$$y + \frac{4}{15}n^2\lambda^{-2c} \leqslant h(\frac{1}{2\sqrt{2}\,\lambda}) + \frac{2}{15} \approx 0.762 \leqslant 0.9.$$

- If $\lfloor \frac{\lambda^c}{\sqrt{2}} \rfloor < 1$, then $n = 1$ and $\lambda^c < \sqrt{2}$. From $-0.2 \leqslant c$, we have $\frac{4}{15}n^2\lambda^{-2c} \leqslant \frac{4}{15}\lambda^{0.4}$. Moreover, because $\texttt{Bias}(D,\Delta) \in [-1,1]$, we have $-0.2 \leqslant c \leqslant z,\zeta \leqslant c+1$. With $\lambda^c \leqslant \sqrt{2}$, this implies that $\frac{1}{\sqrt{2}\,\lambda} \leqslant \lambda^{-c-1} \leqslant \lambda^{-z}, \lambda^{-\zeta} \leqslant \lambda^{0.2}$. Therefore both λ^{-z} and $\lambda^{-\zeta}$ are in the interval $[\frac{1}{\sqrt{2}\,\lambda}, \lambda^{0.2}]$. On this interval, the function $h(x)$ assumes its maximum at $x = \lambda^{0.2}$, and therefore $y \leqslant h(\lambda^{0.2})$. This completes the proof since:

$$y + \frac{4}{15}n^2\lambda^{-2c} \leqslant h(\lambda^{0.2}) + \frac{4}{15}\lambda^{0.4} \approx 0.851 \leqslant 0.9.$$

□

A.6 Proof of the Step Lemma

The proof of the Step Lemma is now basically a case distinction, using the cases enumerated in Sections A.1–A.5, as well as some additional symmetric cases. In particular, the following remark will allow us to use the grid operators X and Z to reduce the number of cases to consider.

Remark A.20. The grid operator Z negates the anti-diagonal entries of a state while the operator X swaps the diagonal entries of a state. This follows by simple computation:

$$(D,\Delta)\cdot Z = \left(\begin{bmatrix} e\lambda^{-z} & -b \\ -b & e\lambda^{z} \end{bmatrix}, \begin{bmatrix} \varepsilon\lambda^{-\zeta} & -\beta \\ -\beta & \varepsilon\lambda^{\zeta} \end{bmatrix}\right), \quad (D,\Delta)\cdot X = \left(\begin{bmatrix} e\lambda^{z} & b \\ b & e\lambda^{-z} \end{bmatrix}, \begin{bmatrix} \varepsilon\lambda^{\zeta} & \beta \\ \beta & \varepsilon\lambda^{-\zeta} \end{bmatrix}\right).$$

Moreover, $\texttt{Bias}((D,\Delta)\cdot Z) = \texttt{Bias}(D,\Delta)$ and $\texttt{Bias}((D,\Delta)\cdot X) = -\texttt{Bias}(D,\Delta)$.

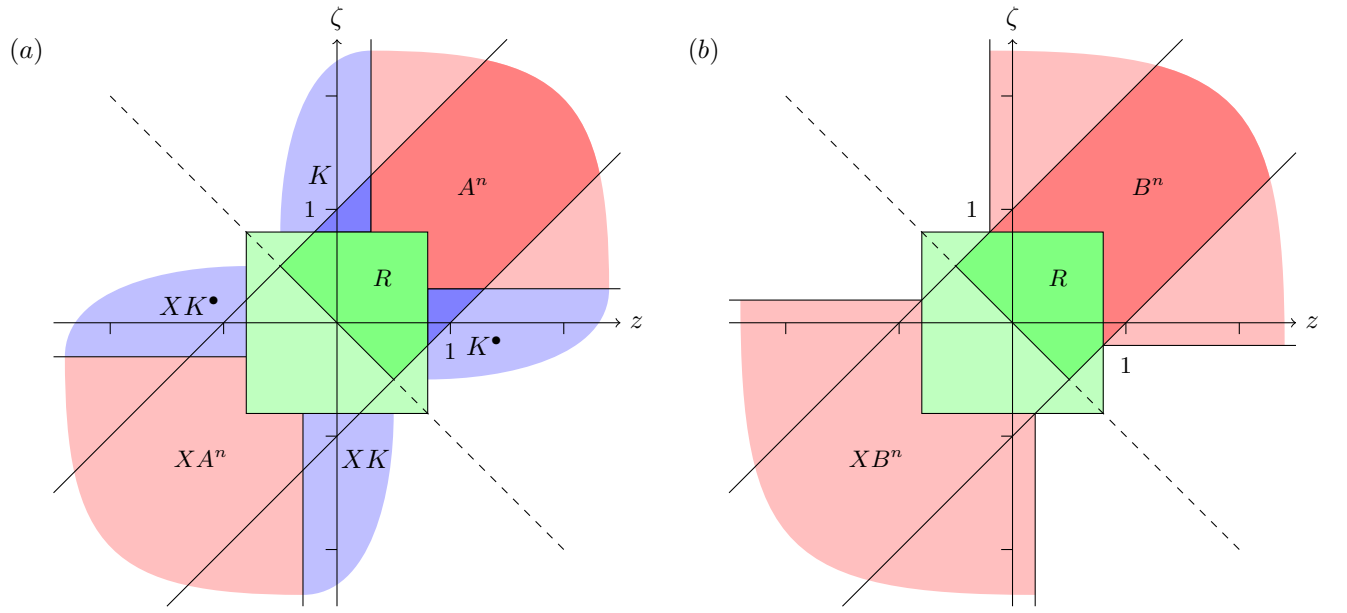


Figure 7: (a) A covering of the region $z-\zeta \in [-1,1]$ and $z+\zeta \geqslant 0$ for the case $b \geqslant 0$. (b) A covering of the region $z-\zeta \in [-1,1]$ and $z+\zeta \geqslant 0$ for the case $b \leqslant 0$.

Lemma (Step Lemma)**.** *For any state (D,Δ), if $\texttt{Skew}(D,\Delta) \geqslant 15$, then there exists a special grid operator G such that $\texttt{Skew}((D,\Delta)\cdot G) \leqslant 0.9\ \texttt{Skew}(D,\Delta)$. Moreover, G can be computed using a constant number of arithmetic operations.*

Proof. Let (D,Δ) be a state such that $\texttt{Skew}(D,\Delta) \geqslant 15$. By Lemma A.11 we can assume w.l.o.g. that $\texttt{Bias}(D,\Delta) \in [-1,1]$. Moreover, by Remark A.20, we can also assume that $\beta \geqslant 0$ and $z+\zeta \geqslant 0$. Note that the application of the grid operators X and/or Z in Remark A.20 preserves the fact that $\texttt{Bias}(D,\Delta) \in [-1,1]$. We now treat in turn the cases $b \geqslant 0$ and $b \leqslant 0$.

Case 1 $b \geqslant 0$. A covering of the strip defined by $z-\zeta \in [-1,1]$ and $z+\zeta \geqslant 0$ is depicted in Figure 7(a). The R region (in green) and the A region (in red) are defined as the intersection of this space with $\{(z,\zeta) \mid -0.8 \leqslant z,\zeta \leqslant 0.8\}$ and $\{(z,\zeta) \mid z \leqslant 0.3 \text{ and } 0.8 \leqslant \zeta\}$ respectively. The K and $K^\bullet$ regions (both in blue) fill the remaining space. We now consider in turn the possible locations of the pair (z,ζ) in this covering.

1. If $-0.8 \leqslant z,\zeta \leqslant 0.8$, then $\texttt{Skew}((D,\Delta)\cdot R) \leqslant 0.9\ \texttt{Skew}(D,\Delta)$ by Lemma A.13.
2. If $z \leqslant 0.3$ and $0.8 \leqslant \zeta$, then $\texttt{Skew}((D,\Delta)\cdot K) \leqslant 0.9\ \texttt{Skew}(D,\Delta)$ by Lemma A.15.
3. If $0.3 \leqslant z,\zeta$, then there exists $n \in \mathbb{Z}$ such that $\texttt{Skew}((D,\Delta)\cdot A^n) \leqslant 0.9\ \texttt{Skew}(D,\Delta)$ by Lemma A.17.
4. If $0.8 \leqslant z$ and $\zeta \leqslant 0.3$, then note that $(D,\Delta)\cdot K^\bullet = (\Delta,D)\cdot K$, and therefore by Lemma A.15:

$$\texttt{Skew}((D,\Delta)\cdot K^\bullet) = \texttt{Skew}((\Delta,D)\cdot K) \leqslant 0.9\ \texttt{Skew}(\Delta,D) = 0.9\ \texttt{Skew}(D,\Delta).$$

Case 2 $b \leqslant 0$. As above, we use a covering of the strip defined by $z-\zeta \in [-1,1]$ and $z+\zeta \geqslant 0$ and consider the possible locations of (z,ζ) in this space. The relevant covering is depicted in Figure 7(b), where the R region (in green) is defined as above and the B region (in red) is defined as the intersection of the strip with $\{(z,\zeta) \mid z,\zeta \geqslant -0.2\}$.

1. If $-0.8 \leqslant z,\zeta \leqslant 0.8$, then $\texttt{Skew}((D,\Delta)\cdot R) \leqslant 0.9\ \texttt{Skew}(D,\Delta)$ by Lemma A.13.
2. If $z,\zeta \geqslant -0.2$ then there exists $n \in \mathbb{Z}$ such that $\texttt{Skew}((D,\Delta)\cdot B^n) \leqslant 0.9\ \texttt{Skew}(D,\Delta)$ by Lemma A.19.

Finally, note that only a constant number of calculations are required to decide which of the above cases applies. Moreover, each case only requires a constant number of operations. Specifically, the computation of k and σ^k in Lemma A.11, of n and A^n in Lemma A.17, and of n and B^n in Lemma A.19 each require just a fixed number of operations, and each of the remaining cases produces a fixed grid operator. □

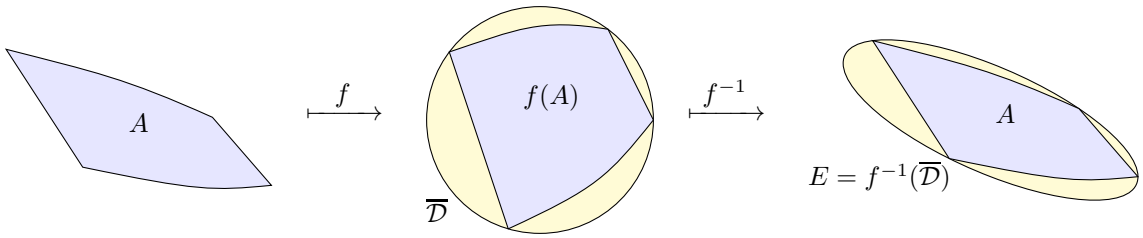
B Proof of Proposition 5.17

We prove Proposition 5.17, whose statement we reproduce here.

Proposition. *Let A be a bounded convex subset of $\mathbb{R}^2$ with non-empty interior. Then there exists an ellipse E such that $A \subseteq E$, and such that*

$$\operatorname{area}(E) \leqslant \frac{4\pi}{3\sqrt{3}} \operatorname{area}(A). \tag{41}$$

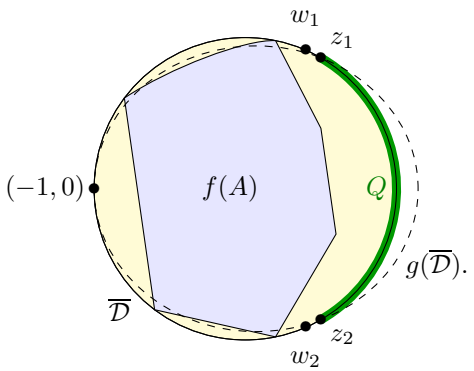
Proof. We may assume without loss of generality that A is compact, for it it is not, we can replace A by its closure, which has the same area as A because A is convex. Let $\overline{\mathcal{D}}$ be the closed unit disk, and consider the collection $\operatorname{Aff}(A, \overline{\mathcal{D}})$ of all affine transformations $f : \mathbb{R}^2 \to \mathbb{R}^2$ satisfying $f(A) \subseteq \overline{\mathcal{D}}$. Then $\operatorname{Aff}(A, \overline{\mathcal{D}})$, with the natural topology, is a compact set. Therefore, there exists some $f \in \operatorname{Aff}(A, \overline{\mathcal{D}})$ maximizing the area of $f(A)$. We claim that f is invertible. Indeed, since A is bounded, there exists some $\lambda > 0$ such that $\lambda A \subseteq \overline{\mathcal{D}}$. Since λA has non-zero area, and multiplication by λ is an affine map, it follows that $f(A)$ has non-zero area as well, and so f is invertible. Let $E = f^{-1}(\overline{\mathcal{D}})$. We claim that E is the desired ellipse.



We have $A \subseteq E$ by construction. We must show (41). Because affine transformations preserve ratios of areas, we may equivalently show

$$\operatorname{area}(\overline{\mathcal{D}}) \leqslant \frac{4\pi}{3\sqrt{3}} \operatorname{area}(f(A)).$$

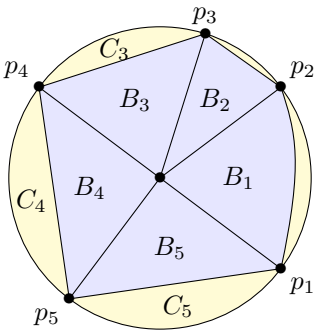
Let $\partial\overline{\mathcal{D}}$ be the boundary of $\overline{\mathcal{D}}$, and consider points p where $f(A)$ "touches" the boundary, i.e., points $p \in f(A) \cap \partial\overline{\mathcal{D}}$. We claim that any arc segment of $\partial\overline{\mathcal{D}}$ of length $2\pi/3$ radians (120 degrees) contains at least one such point. To prove this, assume, for the sake of contradiction, that there is such an arc segment Q not containing any point of $f(A)$. By rotational symmetry, we may without loss of generality assume that Q is the arc from $-\pi/3$ to $\pi/3$ radians on the unit circle. Let z_1 and z_2 be the endpoints of Q, as shown here:



Since both Q and $f(A)$ are compact, there exists some $d > 0$ such that the distance between any point of A and any point of Q is at least d. Let w_1 and w_2 be the two points on the unit circle whose distance from Q is $d/2$. Now consider the affine transformation g that fixes $(-1, 0)$ and maps w_1 to z_1 and w_2 to z_2. Then $g(\overline{\mathcal{D}})$ is an ellipse whose boundary passes through the points $(-1, 0)$, z_1, and z_2, shown as a dashed line in the above illustration. It is therefore bisected by Q. Since the map g moves no point of the unit disk by more than distance d, the set $g(f(A))$ does not intersect Q. It follows that $g(f(A))$ is contained in $\overline{\mathcal{D}}$. On the other hand, a calculation shows that the area of $g(f(A))$ is slightly greater than that of $f(A)$, contradicting the assumption that the area of $f(A)$ was maximal.

We have proved that every arc segment of length $2\pi/3$ radians on the boundary of $\overline{\mathcal{D}}$ contains a point of $f(A)$. It follows that there is some finite cyclic sequence of points $p_1, \ldots, p_n \in f(A) \cap \partial\overline{\mathcal{D}}$ such that consecutive points are no more than $2\pi/3$ radians apart. By connecting each p_i to the center, we partition each of the sets $f(A)$ and $\overline{\mathcal{D}}$ into n

pieces $B_1, \ldots, B_n$ and $C_1, \ldots, C_n$, respectively.



The fact that the inner angles are less than $2\pi/3$ immediately implies that $\text{area}(C_i) \leqslant \frac{4\pi}{3\sqrt{3}} \text{area}(B_i)$ for all i; hence also $\text{area}(\overline{\mathcal{D}}) \leqslant \frac{4\pi}{3\sqrt{3}} \text{area}(f(A))$. This finishes the proof of the proposition. □

C Proof of Theorem 6.2

Consider the rings $\mathbb{Z}$ and $\mathbb{D}$, together with their respective extensions $\mathbb{Z}[\sqrt{2}]$, $\mathbb{Z}[\omega]$ and $\mathbb{D}[\sqrt{2}]$, $\mathbb{D}[\omega]$, as introduced in Section 3. We wish to give an efficient method for solving equations of the form $t^\dagger t = \xi$, for given $\xi \in \mathbb{D}[\sqrt{2}]$ and unknown $t \in \mathbb{D}[\omega]$. To do this, we first classify the primes in $\mathbb{Z}[\sqrt{2}]$ and $\mathbb{Z}[\omega]$, then show how to solve the equation $t^\dagger t = \xi$ in the case where $\xi \in \mathbb{Z}[\sqrt{2}]$ and $t \in \mathbb{Z}[\omega]$, which we finally extend to the general case. None of the results in this section are original; they are well-known from the theory of cyclotomic fields, which goes back to the 19th century work of Gauss and Kummer. Nevertheless, we hope that the following detailed treatment may be useful to readers who are not experts in algebraic number theory. Moreover, we hope to give sufficient details to enable an interested reader, in principle, to implement the algorithm. This will also aid in our complexity analysis.

A fundamental property of the rings $\mathbb{Z}$, $\mathbb{Z}[\sqrt{2}]$, and $\mathbb{Z}[\omega]$ is that they are *Euclidean domains*; this implies that the notions of divisibility, greatest common divisor, and unique prime factorization all make sense in these rings. Recall that in a ring, a *unit* is an invertible element. In a Euclidean domain, we write $x \,|\, y$ if x is a divisor of y, and $x \sim y$ if $x \,|\, y$ and $y \,|\, x$; equivalently, $x \sim y$ iff there exists a unit u that $xu = y$. An element x is *prime* if x is not a unit, and $x = ab$ implies that either a or b is a unit. Note that if x is prime and $x \,|\, ab$, then $x \,|\, a$ or $x \,|\, b$; this follows from Euclid's algorithm.

C.1 Units in $\mathbb{Z}[\sqrt{2}]$

Definition C.1. We say that $\xi \in \mathbb{Z}[\sqrt{2}]$ is *doubly positive* if $\xi \geqslant 0$ and $\xi^\bullet \geqslant 0$.

Lemma C.2. *The units of $\mathbb{Z}[\sqrt{2}]$ are of the form $u = (-1)^n \lambda^m$, where $\lambda = 1 + \sqrt{2}$. Moreover, a unit u is doubly positive if and only if u is a square in $\mathbb{Z}[\sqrt{2}]$.*

Proof. Lemma 10 of [14]. □

Lemma C.3. *Let $\xi \in \mathbb{Z}[\sqrt{2}]$, and consider $n = \xi^\bullet \xi$. If n is a unit in $\mathbb{Z}$, then ξ is a unit in $\mathbb{Z}[\sqrt{2}]$.*

Proof. If n is a unit, then there exists $m \in \mathbb{Z}$ such that $nm = 1$, hence $\xi \xi^\bullet m = 1$, hence ξ is invertible in $\mathbb{Z}[\sqrt{2}]$. □

C.2 Primes in $\mathbb{Z}[\sqrt{2}]$

Lemma C.4. *Let $\xi \in \mathbb{Z}[\sqrt{2}]$, and consider $n = \xi^\bullet \xi$. If n is prime in $\mathbb{Z}$, then ξ is prime in $\mathbb{Z}[\sqrt{2}]$.*

Proof. Suppose $\xi = \alpha\beta$ in $\mathbb{Z}[\sqrt{2}]$, and consider $n = \xi^\bullet \xi = \alpha^\bullet \alpha \beta^\bullet \beta$. Since $\alpha^\bullet \alpha$ and $\beta^\bullet \beta$ are integers and n is prime, we must have that either $\alpha^\bullet \alpha$ or $\beta^\bullet \beta$ is a unit in $\mathbb{Z}$; hence α or β is a unit in $\mathbb{Z}[\sqrt{2}]$. So ξ is prime. □

Lemma C.5. *For every prime ξ of $\mathbb{Z}[\sqrt{2}]$, there exists a unique (up to a unit) prime p of $\mathbb{Z}$ such that $\xi \,|\, p$.*

Proof. To show existence, consider $n = \xi^\bullet \xi$. Note that $\xi \neq 0$, hence $n \neq 0$. Let $n = p_1 p_2 \cdots p_k$ be a prime factorization of n. Since $\xi \,|\, p_1 p_2 \cdots p_k$ and ξ is prime in $\mathbb{Z}[\sqrt{2}]$, there exists some i such that $\xi \,|\, p_i$.

To show uniqueness, assume $\xi \,|\, p$ and $\xi \,|\, q$, where $p \not\sim q$. Then $\gcd(p, q) = 1$, hence by Euclid's algorithm, we can write $1 = np + mq$, for integers n and m. Then $\xi \,|\, np + mq = 1$, which is absurd since ξ is prime. □

Lemma C.6. *Let ξ be a prime of $\mathbb{Z}[\sqrt{2}]$, and let p be the unique (up to a unit) prime of $\mathbb{Z}$ such that $\xi \mid p$. Then exactly one of the following holds: $\xi \sim p$ or $\xi^\bullet \xi \sim p$.*

Proof. First note that at most one of these properties can hold, because otherwise $\xi \sim \xi^\bullet \xi$, which implies that $\xi^\bullet$ is a unit, which is absurd since it is prime.

We now show that at least one of the properties holds. Since $\xi \mid p$ and p is an integer, we have $\xi^\bullet \mid p$, hence $\xi^\bullet \xi \mid p^2$. Also, $\xi^\bullet \xi$ is an integer, so either $\xi^\bullet \xi \sim 1$, $\xi^\bullet \xi \sim p$, or $\xi^\bullet \xi \sim p^2$. The first of these cases is absurd, since ξ would then be a unit. In the second case, we have $\xi^\bullet \xi \sim p$, which is to be shown. In the third case, since $\xi \mid p$, there is $\alpha \in \mathbb{Z}[\sqrt{2}]$ such that $\xi\alpha = p$. Then we have $\xi^\bullet \xi \alpha^\bullet \alpha = p^2 \sim \xi^\bullet \xi$, which implies that α is a unit, hence $\xi \sim p$. □

Lemma C.7. *Let p be a prime of $\mathbb{Z}$. Then the prime factorization of p in $\mathbb{Z}[\sqrt{2}]$ consists of one or two factors.*

Proof. Let ξ be some prime factor of p in $\mathbb{Z}[\sqrt{2}]$. By Lemma C.6, either $\xi \sim p$ or $\xi^\bullet \xi \sim p$. Either way, this gives a prime factorization of p in $\mathbb{Z}[\sqrt{2}]$. □

We now determine the prime factorization in $\mathbb{Z}[\sqrt{2}]$ of every prime p of $\mathbb{Z}$. It turns out that there are 5 cases, depending whether p is even, or $p \equiv 1, 3, 5, 7 \pmod 8$.

Lemma C.8. *The prime factorization of $p = 2$ in $\mathbb{Z}[\sqrt{2}]$ is $p = \sqrt{2} \cdot \sqrt{2}$. The prime factorization of $p = -2$ in $\mathbb{Z}[\sqrt{2}]$ is $p = -\sqrt{2} \cdot \sqrt{2}$.*

Proof. We only need to show that $\sqrt{2}$ is prime in $\mathbb{Z}[\sqrt{2}]$, but this follows from Lemma C.4. □

Lemma C.9. *Let p be a prime of $\mathbb{Z}$ such that $p \equiv 3 \pmod 8$ or $p \equiv 5 \pmod 8$. Then p is prime in $\mathbb{Z}[\sqrt{2}]$.*

Proof. Let ξ be a prime factor of p in $\mathbb{Z}[\sqrt{2}]$. By Lemma C.6, we know that $\xi \sim p$ or $\xi^\bullet \xi \sim p$. In the former case, p is prime and we are done. In the latter case, writing $\xi = a + b\sqrt{2}$, we have $\xi^\bullet \xi = a^2 - 2b^2 = \pm p$, hence $a^2 - 2b^2 \equiv \pm 3 \pmod 8$. By easy case distinction, we see that a^2 can only be 0, 1, or 4 modulo 8, and $2b^2$ can only be 0 or 2 modulo 8, so $a^2 - 2b^2 \equiv \pm 3 \pmod 8$ is plainly impossible. □

Lemma C.10. *Let $\xi \in \mathbb{Z}[\sqrt{2}]$ such that $\xi \sim \xi^\bullet$. Then either $\xi \sim n$, or $\xi \sim n\sqrt{2}$, for some $n \in \mathbb{Z}$.*

Proof. By assumption, $\xi \sim \xi^\bullet$, so there exists a unit $u \in \mathbb{Z}[\sqrt{2}]$ such that $\xi^\bullet = u\xi$. By Lemma C.2, the units of $\mathbb{Z}[\sqrt{2}]$ are exactly of the form $u = (-1)^n \lambda^m$, where $\lambda = 1 + \sqrt{2}$. Applying the automorphism to $\xi^\bullet = u\xi$, we get $\xi = u^\bullet \xi^\bullet$, hence $\xi^\bullet \xi = u^\bullet u \xi^\bullet \xi$, hence $u^\bullet u = 1$, hence $(\lambda^\bullet \lambda)^m = 1$. But $\lambda^\bullet \lambda = -1$, so m is even; say $m = 2k$. Let $\xi' = \lambda^k \xi$. Note that $\xi' \sim \xi$. Then we have

$$\xi'^\bullet = (\lambda^\bullet)^k \xi^\bullet = (-\lambda)^{-k} u\xi = (-1)^{-k} \lambda^{-k} (-1)^n \lambda^m \xi = \pm \lambda^k \xi = \pm \xi'. \tag{42}$$

We can write $\xi' = a + b\sqrt{2}$ for some $a, b \in \mathbb{Z}$. From (42), we have either $a - b\sqrt{2} = a + b\sqrt{2}$, or $a - b\sqrt{2} = -(a + b\sqrt{2})$. In the first case, $b = 0$, and therefore $\xi \sim \xi' = a$. In the second case, $a = 0$, and therefore $\xi \sim \xi' = b\sqrt{2}$. □

Lemma C.11. *Let p be a prime of $\mathbb{Z}$ such that $p \equiv 1 \pmod 8$ or $p \equiv 7 \pmod 8$. Then p has a prime factorization of the form $p \sim \xi^\bullet \xi$ in $\mathbb{Z}[\sqrt{2}]$; moreover, $\xi \not\sim \xi^\bullet$, so that the two prime factors are distinct.*

Proof. It is well-known (by quadratic reciprocity) that if p is a prime with $p \equiv \pm 1 \pmod 8$, then 2 is a quadratic residue modulo p. Therefore, the equation $x^2 \equiv 2 \pmod p$ has an integer solution; let x be such a solution. Let $\alpha = x + \sqrt{2}$. Then $\alpha^\bullet \alpha = x^2 - 2 \equiv 0 \pmod p$, and hence $p \mid \alpha^\bullet \alpha$. On the other hand, clearly $p \nmid \alpha$ (since $\alpha / p \notin \mathbb{Z}[\sqrt{2}]$), and similarly $p \nmid \alpha^\bullet$, which shows that p is not a prime in $\mathbb{Z}[\sqrt{2}]$. By Lemma C.6, $p \sim \xi^\bullet \xi$ for some prime ξ of $\mathbb{Z}[\sqrt{2}]$.

The final claim follows from Lemma C.10, because if $\xi \sim \xi^\bullet$, then either $\xi \sim n$ or $\xi \sim n\sqrt{2}$. In the first case, $n^2 \mid \xi^\bullet \xi \sim p$, but p is prime, so that $n = \pm 1$, contradicting that ξ is prime. In the second case, $2n^2 \mid \xi^\bullet \xi \sim p$, contradicting the assumption that p is odd. □

Lemma C.12. *Let p be a prime of $\mathbb{Z}$. A prime factorization of p in $\mathbb{Z}[\sqrt{2}]$ can be computed in probabilistic polynomial time.*

Proof. Assume without loss of generality that $p > 0$. If $p = 2$, then $p = \sqrt{2}^2$ is the desired prime factorization by Lemma C.8. If $p \equiv 3, 5 \pmod 8$, then p is already prime in $\mathbb{Z}[\sqrt{2}]$ by Lemma C.9. The remaining case is when $p \equiv 1, 7 \pmod 8$. In this case the prime factorization is of the form $p \sim \xi^\bullet \xi$ by Lemma C.11. Moreover, the proof of Lemma C.11 indicates how such ξ can be computed, namely as $\xi = \gcd(p, x + \sqrt{2})$, where x is a solution of $x^2 \equiv 2 \pmod p$. The equation $x^2 \equiv 2 \pmod p$ can be solved in probabilistic polynomial time by a well-known algorithm, see [12]. □

C.3 Primes in $\mathbb{Z}[\omega]$

Lemma C.13. *Let ξ be a prime in $\mathbb{Z}[\sqrt{2}]$. Then either ξ is prime in $\mathbb{Z}[\omega]$, or else $\xi \sim t^\dagger t$ where t is some prime of $\mathbb{Z}[\omega]$. In particular, the prime factorization of ξ in $\mathbb{Z}[\omega]$ consists of either one or two factors.*

Proof. This is similar to the proof of Lemma C.7. Let t be some prime factor of ξ in $\mathbb{Z}[\omega]$. Then $t \,|\, \xi$, therefore $t^\dagger \,|\, \xi^\dagger = \xi$, therefore $t^\dagger t \,|\, \xi^2$ in $\mathbb{Z}[\omega]$. But both $t^\dagger t$ and ξ are elements of $\mathbb{Z}[\sqrt{2}]$, so $t^\dagger t \,|\, \xi^2$ in $\mathbb{Z}[\sqrt{2}]$. Since ξ is prime in $\mathbb{Z}[\sqrt{2}]$, there are only three possibilities: $t^\dagger t \sim 1$, $t^\dagger t \sim \xi$, and $t^\dagger t \sim \xi^2$. In the first case, t is a unit, contradicting the fact that it is prime in $\mathbb{Z}[\omega]$. In the second case, we are done. In the third case, since $t \,|\, \xi$, there is $a \in \mathbb{Z}[\omega]$ such that $at = \xi$. Then $a^\dagger a t^\dagger t = \xi^\dagger \xi = \xi^2 = t^\dagger t$, which implies $a^\dagger a = 1$. Therefore a is a unit, and $t \sim \xi$. Therefore ξ is prime in $\mathbb{Z}[\omega]$ and we are done. □

C.4 The Diophantine equation $t^\dagger t = \xi$

We are interested in solving equations of the form

$$t^\dagger t = \xi, \tag{43}$$

where $\xi \in \mathbb{D}[\sqrt{2}]$ is given, and $t \in \mathbb{D}[\omega]$ is unknown.

Definition C.14. Recall that for elements $\xi, \xi' \in \mathbb{Z}[\sqrt{2}]$, the notation $\xi \sim \xi'$ means that ξ, ξ' differ by a unit, i.e., there exists a unit $u \in \mathbb{Z}[\sqrt{2}]$ such that $\xi = u\xi'$. We extend this notation also to the ring $\mathbb{D}[\sqrt{2}]$: for $\xi, \xi' \in \mathbb{D}[\sqrt{2}]$, we write $\xi \sim \xi'$ iff there exists a unit $u \in \mathbb{Z}[\sqrt{2}]$ such that $\xi = u\xi'$. Note that we have taken u to be a unit of the ring $\mathbb{Z}[\sqrt{2}]$, not of $\mathbb{D}[\sqrt{2}]$.

It will often be convenient to replace (43) by the following weaker condition.

Definition C.15. We say that $\xi \in \mathbb{D}[\sqrt{2}]$ is †-*decomposable* if the equation

$$t^\dagger t \sim \xi \tag{44}$$

has a solution $t \in \mathbb{D}[\omega]$.

Solutions to (43) can be recovered from solutions to (44) by using the following observation:

Lemma C.16. *Let $\xi \in \mathbb{D}[\sqrt{2}]$. Then equation (43) has a solution if and only if ξ is doubly positive and †-decomposable.*

Proof. If ξ is a solution to (43), then it is obviously †-decomposable. It is also doubly positive by Lemma 6.1. Conversely, assume $t^\dagger t \sim \xi$. Then there exists a unit u of $\mathbb{Z}[\sqrt{2}]$ such that $\xi = ut^\dagger t$. Since both ξ and $t^\dagger t$ are doubly positive, it follows that u is doubly positive, and hence u is a square of the ring $\mathbb{Z}[\sqrt{2}]$ by Lemma C.2; say $u = v^2$. Since $v \in \mathbb{Z}[\sqrt{2}]$, we have $v = v^\dagger$. Setting $t' = vt$, we have $\xi = v^\dagger v t^\dagger t = t'^\dagger t'$, which finishes the proof. □

C.5 The case $\xi \in \mathbb{Z}[\sqrt{2}]$

Lemma C.17. *Suppose $t^\dagger t \sim \xi$ for $t \in \mathbb{D}[\omega]$ and $\xi \in \mathbb{Z}[\sqrt{2}]$. Then $t \in \mathbb{Z}[\omega]$.*

Proof. Note that, in $\mathbb{Z}[\omega]$, we have $\sqrt{2}^k \,|\, t$ if and only if $2^k \,|\, t^\dagger t$. Choose $t' \in \mathbb{Z}[\omega]$ and $k \geqslant 0$ such that $t = t'/\sqrt{2}^k$. Then $t'^\dagger t' = 2^k \xi$, hence $2^k \,|\, t'^\dagger t'$, hence $\sqrt{2}^k \,|\, t'$, hence $t \in \mathbb{Z}[\omega]$. □

Lemma C.18. *If x, y, z are three elements of a Euclidean domain, then*

$$\gcd(xy, z) \,|\, \gcd(x, z) \cdot \gcd(y, z).$$

Proof. By considering the prime factorization of z. □

Lemma C.19. *Suppose $\xi = \alpha\beta$, where $\alpha, \beta \in \mathbb{Z}[\sqrt{2}]$ and $\gcd(\alpha, \beta) = 1$. Then ξ is †-decomposable iff α and β are †-decomposable.*

Proof. For the right-to-left implication, assume $\alpha \sim s^\dagger s$ and $\beta \sim r^\dagger r$. Clearly $\xi = \alpha\beta \sim (sr)^\dagger sr$. For the left-to-right implication, assume $\xi \sim t^\dagger t$. Note that $t \in \mathbb{Z}[\omega]$ by Lemma C.17. To show that α is †-decomposable, let $s = \gcd(t, \alpha)$. We claim that $s^\dagger s \sim \alpha$. Clearly, since $s \,|\, \alpha$ and $s^\dagger \,|\, \alpha$, we have $s^\dagger s \,|\, \alpha^2$. One the other hand, we know that $s^\dagger s \,|\, t^\dagger t \sim \xi = \alpha\beta$. Since α and β are relatively prime, it follows that $s^\dagger s \,|\, \alpha$. Conversely, note that by Lemma C.18, $\alpha = \gcd(t^\dagger t, \alpha) \,|\, \gcd(t^\dagger, \alpha) \cdot \gcd(t, \alpha) = s^\dagger s$. Therefore $s^\dagger s \sim \alpha$ and α is †-decomposable. The argument for β is similar. □

Lemma C.20. *Suppose that $\xi \in \mathbb{Z}[\sqrt{2}]$ is prime. Let $p > 0$ be the unique positive prime in $\mathbb{Z}$ such that $\xi \mid p$. Then ξ is $\dagger$-decomposable if and only if $p = 2$ or $p \equiv 1, 3, 5 \pmod 8$.*

Proof. We consider each case in turn. If $p = 2$, then $\xi \sim \sqrt{2}$. Let $\delta = 1 + \omega$; a simple calculation shows that $\delta^\dagger \delta = \lambda\sqrt{2} \sim \xi$. So ξ is $\dagger$-decomposable.

If $p \equiv 1 \pmod 4$, then by quadratic reciprocity, there exists some integer u such that $u^2 \equiv -1 \pmod p$. Therefore $\xi \mid p \mid u^2 + 1 = (u + i)(u - i)$. Let $t = \gcd(\xi, u + i)$. We claim that $\xi \sim t^\dagger t$. Note that $t \mid \xi$, hence $t^\dagger \mid \xi^\dagger = \xi$, hence $t^\dagger t \mid \xi^2$. Since ξ is prime in $\mathbb{Z}[\sqrt{2}]$, there are 3 possibilities: $t^\dagger t \sim 1$, $t^\dagger t \sim \xi$, or $t^\dagger t \sim \xi^2$. The first case is not possible, because in this case, t would be a unit, so that ξ is relatively prime to $u + i$, hence to $u - i$, hence to $u^2 + 1$, contradicting $\xi \mid u^2 + 1$. In the second case, we have $t^\dagger t \sim \xi$, which was to be shown. In the third case, we have $t^\dagger t \sim \xi^2$. Since $t \mid \xi$, there exists some $s \in \mathbb{Z}[\omega]$ such that $ts = \xi$. But then $t^\dagger t s^\dagger s = \xi^\dagger \xi = \xi^2 \sim t^\dagger t$, so that s is a unit. In this case, we have $t \sim \xi$, therefore $\xi \mid u + i$, therefore also $\xi = \xi^\dagger \mid u - i$, hence $\xi \mid (u + i) - (u - i) \sim 2$, contradicting the fact that p is the only prime integer divisible by ξ.

The case $p \equiv 3 \pmod 8$ is very similar. In this case, by quadratic reciprocity, -2 is a square modulo p, so that there exists some integer u such that $u^2 \equiv -2 \pmod p$. Therefore $\xi \mid p \mid u^2 + 2 = (u + i\sqrt{2})(u - i\sqrt{2})$. Let $t = \gcd(\xi, u + i\sqrt{2})$. We claim that $\xi \sim t^\dagger t$. Note that $t \mid \xi$, hence $t^\dagger \mid \xi^\dagger = \xi$, so $t^\dagger t \mid \xi^2$. So again we have the three possibilities $t^\dagger t \sim 1$, $t^\dagger t \sim \xi$, or $t^\dagger t \sim \xi^2$. The first case is not possible, because in this case, t would be a unit, so that ξ is relatively prime to $u + i\sqrt{2}$, hence to $u - i\sqrt{2}$, hence to $u^2 + 2$, contradicting $\xi \mid u^2 + 2$. In the second case, we have $t^\dagger t \sim \xi$, which was to be shown. In the third case, we have $t^\dagger t \sim \xi^2$. Since $t \mid \xi$, there exists some $s \in \mathbb{Z}[\omega]$ such that $ts = \xi$. But then $t^\dagger t s^\dagger s = \xi^\dagger \xi = \xi^2 \sim t^\dagger t$, so that s is a unit. In this case, we have $t \sim \xi$, therefore $\xi \mid u + i\sqrt{2}$, therefore also $\xi = \xi^\dagger \mid u - i\sqrt{2}$, hence $\xi \mid i(u - i\sqrt{2}) - i(u + i\sqrt{2}) = 2\sqrt{2}$. On the other hand, $\xi \mid p$, therefore $\xi \mid \gcd(p, 2\sqrt{2}) = 1$, contradicting the fact that ξ is not a unit.

Finally, if $p \equiv 7 \pmod 8$, then $p \sim \xi^\bullet \xi$ by Lemma C.11. Assume that ξ is $\dagger$-decomposable as $\xi \sim t^\dagger t$. Then

$$(t^\dagger t)^\bullet (t^\dagger t) \sim \xi^\bullet \xi \sim p.$$

Note that $t^\bullet t \in \mathbb{Z}[i]$, so we can write $t^\bullet t = a + bi$ for some $a, b \in \mathbb{Z}$. But then we have

$$p \sim (t^\bullet t)(t^\bullet t)^\dagger = (a + bi)(a - bi) = a^2 + b^2.$$

But a^2 and b^2 can only be congruent to 0, 1, or 4 modulo 8, contradicting $p \equiv 7 \pmod 8$. Therefore ξ is not $\dagger$-decomposable, which is what was claimed. □

Lemma C.21. *Suppose $\xi \in \mathbb{Z}[\sqrt{2}]$ is prime. Let $p > 0$ be the unique positive prime in $\mathbb{Z}$ such that $\xi \mid p$. Let m be a positive integer. Then ξ^m is $\dagger$-decomposable if and only if m is even or $p \equiv 1, 2, 3, 5 \pmod 8$.*

Proof. If m is even, then note that $\xi \in \mathbb{Z}[\sqrt{2}]$, so $\xi = \xi^\dagger$; therefore, $t = \xi^{m/2}$ will be a solution. If $p \equiv 1, 2, 3, 5 \pmod 8$, then by Lemma C.20, there exists $s \in \mathbb{Z}[\omega]$ with $s^\dagger s \sim \xi$; therefore $t = s^m$ is a solution of $t^\dagger t \sim \xi^m$. The only remaining case is then m is odd and $p \equiv 7 \pmod 8$. In this case, there can be no solution. For assume on the contrary that $t^\dagger t = \xi^m$. Then as in the proof of Lemma C.20, we can write $t^\bullet t = a + bi$ for some $a, b \in \mathbb{Z}$, and we get

$$p^m \sim (t^\bullet t)(t^\bullet t)^\dagger = (a + bi)(a - bi) = a^2 + b^2,$$

contradicting $p^m \equiv 7 \pmod 8$. □

Remark C.22. The proofs of Lemmas C.20 and C.21 are constructive, and immediately yield efficient algorithms for determining t, provided that we have an efficient method of solving $u^2 \equiv -1 \pmod p$ when p is a prime congruent to $1 \pmod 4$ and of solving $u^2 \equiv -2 \pmod p$ when p is a prime congruent to $3 \pmod 8$. These last problems can be solved in probabilistic polynomial time by a well-known algorithm, see [12].

Lemma C.23. *Given $\xi \in \mathbb{Z}[\sqrt{2}]$, together with its prime factorization in $\mathbb{Z}[\sqrt{2}]$, there exists an algorithm that determines, in probabilistic polynomial time, whether the equation $t^\dagger t \sim \xi$ has a solution or not, and finds a solution if there is one.*

Proof. Let $\xi \sim \xi_1^{m_1} \xi_2^{m_2} \cdots \xi_k^{m_k}$ be a prime factorization of ξ in $\mathbb{Z}[\sqrt{2}]$, where $\xi_1, \ldots, \xi_k$ are distinct (i.e., pairwise non-associate) primes. By Lemma C.19, $t^\dagger t \sim \xi$ has a solution if and only if $t^\dagger t \sim \xi_i^{m_i}$ has a solution for all i. By Lemma C.21, $t^\dagger t \sim \xi_i^{m_i}$ has a solution if and only if m_i is even or $p \equiv 1, 2, 3, 5 \pmod 8$. Since these conditions are easy to check, we can therefore determine the existence of a solution efficiently, i.e., in polynomial time.

Moreover, once a solution has been shown to exist, the actual solution can be efficiently computed. Namely, for each i, find t_i such that $t_i^\dagger t_i \sim \xi_i^{m_i}$ as in Lemma C.21. Then $t \sim t_1 t_2 \cdots t_k$ satisfies $t^\dagger t \sim \xi_1^{m_1} \xi_2^{m_2} \cdots \xi_k^{m_k} \sim \xi$. By Remark C.22, all of this can be computed in probabilistic polynomial time. □

Proposition C.24. *Let $\xi \in \mathbb{Z}[\sqrt{2}]$, and let $n = \xi^\bullet \xi$. Note that n is an integer. Given the prime factorization of n, there exists an algorithm that determines, in probabilistic polynomial time, whether the equation $t^\dagger t \sim \xi$ has a solution or not, and finds a solution if there is one.*

Proof. Suppose that $n = \pm p_1^{m_1} \cdots p_k^{m_k}$ is the prime factorization of n, where $p_1, \ldots, p_k$ are distinct positive primes. Each p_i can be efficiently factored into primes of $\mathbb{Z}[\sqrt{2}]$ by Lemma C.12; this yields the prime factorization of $n = \xi^\bullet \xi$ in $\mathbb{Z}[\sqrt{2}]$. From this, it is easy to obtain a prime factorization of ξ. The rest of the claim then follows from Lemma C.23. □

C.6 The case $\xi \in \mathbb{D}[\sqrt{2}]$

The following lemma can be used to reduce the problem of †-decomposability in $\mathbb{D}[\sqrt{2}]$ to †-decomposability in $\mathbb{Z}[\sqrt{2}]$.

Lemma C.25. *Consider $\xi \in \mathbb{D}[\sqrt{2}]$. Then ξ is †-decomposable if and only if $\sqrt{2}\,\xi$ is †-decomposable.*

Proof. Recall that $\delta = 1 + \omega$ satisfies $\delta^\dagger \delta = \lambda\sqrt{2} \sim \sqrt{2}$. Also note that δ is invertible in $\mathbb{D}[\omega]$; specifically, $\delta^{-1} = \delta\lambda^{-1}\omega^{-1}/\sqrt{2} \in \mathbb{D}[\omega]$. Assume that ξ is †-decomposable, so that $t^\dagger t \sim \xi$. Letting $t' = \delta t$, we have $t'^\dagger t' = \delta^\dagger \delta t^\dagger t \sim \sqrt{2} t^\dagger t \sim \sqrt{2}\,\xi$, so that $\sqrt{2}\,\xi$ is †-decomposable. The converse is proved similarly, using δ^{-1} instead of δ. □

We can now prove Theorem 6.2, whose statement we reproduce here.

Theorem. *Let $\xi \in \mathbb{D}[\sqrt{2}]$. Note that $\xi^\bullet \xi \in \mathbb{D}$, so we can write $\xi^\bullet \xi = \frac{n}{2^\ell}$ for some $n \in \mathbb{Z}$ and $\ell \in \mathbb{N}$. There exists a probabilistic algorithm which, given ξ and, in case $n \neq 0$, a prime factorization of n, determines whether or not the equation $t^\dagger t = \xi$ has a solution, and finds a solution if there is one. Moreover, the expected runtime of this algorithm is polynomial in the size of n.*

Proof. Let ξ be an element of $\mathbb{D}[\sqrt{2}]$ with $\xi^\bullet \xi = \frac{n}{2^\ell}$ for some $n \in \mathbb{Z}$. First, the algorithm can easily check whether ξ is doubly positive, and if it is not, there is no solution. Next, if $n = 0$, then $\xi = 0$, and $t = 0$ is obviously a solution, so there is nothing to do. Otherwise, let $\xi' = \sqrt{2}^\ell \xi$. Since $\xi' \in \mathbb{Z}[\sqrt{2}]$ and $\xi'^\bullet \xi' = n$, and a factorization of n is given, we can use Proposition C.24 to efficiently determine whether $s^\dagger s \sim \xi'$ has a solution. By Lemma C.25 this is the case if and only if the equation $t^\dagger t \sim \xi$ also has a solution, in which case it is given by $t = \delta^{-\ell} s$. Finally, since ξ is doubly positive, we can solve $t'^\dagger t' = \xi$ by Lemma C.16. □

We conclude this appendix with a useful fact: the algorithm of Theorem 6.2 always succeeds in case n is a prime that is congruent to 1 modulo 8.

Proposition C.26. *Let $\xi = \mathbb{D}[\sqrt{2}]$ be doubly positive, with $\xi^\bullet \xi = \frac{n}{2^\ell}$ for some $n \in \mathbb{Z}$. If n is prime and $n \equiv 1 \pmod 8$, then the equation $t^\dagger t = \xi$ has a solution.*

Proof. Let $\xi' = \sqrt{2}^\ell \xi$. Then $\xi' \in \mathbb{Z}[\sqrt{2}]$ and $\xi'^\bullet \xi' = n$. Then ξ' is prime by Lemma C.4 and †-decomposable by Lemma C.20. By Lemma C.25, ξ is †-decomposable, so that the equation $t^\dagger t = \xi$ can be solved by Lemma C.16. □

D Proof of Lemma 8.4

Definition D.1. Recall that $\delta = 1 + \omega$. Every element $u \in \mathbb{D}[\omega]$ can be written in the form

$$u = \frac{1}{\delta^k}(a\omega^3 + b\omega^2 + c\omega + d), \tag{45}$$

where $a, b, c, d \in \mathbb{Z}$ and $k \geqslant 0$. The smallest k such that u can be written in this form is called the *least δ-exponent* of u.

Remark D.2. A calculation shows that $\frac{1}{\delta}(a\omega^3 + b\omega^2 + c\omega + d)$ is equal to

$$\frac{1}{2}\left[\,(a - b + c - d)\omega^3 + (a + b - c + d)\omega^2 + (-a + b + c - d)\omega + (a - b + c + d)\,\right].$$

It follows that an element $a\omega^3 + b\omega^2 + c\omega + d \in \mathbb{Z}[\omega]$ is divisible by δ if and only if $a + b + c + d$ is even.

We can now prove Lemma 8.4, whose statement we reproduce here.

Lemma. *Each of the numbers n produced in step 2(a) of Algorithm 7.6 satisfies $n \geqslant 0$, and either $n = 0$ or $n \equiv 1 \pmod 8$.*

Proof. Recall that in step 2(a) of Algorithm 7.6, we are given $u \in \mathbb{D}[\omega]$ such that $u \in \overline{\mathcal{D}}$ and $u^\bullet \in \overline{\mathcal{D}}$. We let $\xi = 1 - u^\dagger u$ and write $\xi^\bullet \xi = \frac{n}{2^\ell}$, where $\ell \geqslant 0$ is minimal. We must show that $n \geqslant 0$, and that either $n = 0$ or $n \equiv 1 \pmod 8$.

The first claim is trivial, since by assumption $u^\dagger u \leqslant 1$ and $(u^\dagger u)^\bullet \leqslant 1$, and therefore $\xi, \xi^\bullet \geqslant 0$, which implies $n \geqslant 0$. For the second claim, write u in the form (45), with least δ-exponent k.

- Case 1: $k \leqslant 1$. In this case, one can show by a direct calculation (for example with the help of the algorithm from Proposition 5.21) that the scaled two-dimensional grid problem only has the 9 solutions $u = 0$ and $u = \omega^j$, where $j = 0, \ldots, 7$. In the first case, $n = 1$, and in the remaining 8 cases, $n = 0$, so the claim follows.

- Case 2: $k \geqslant 2$. We calculate
$$u^\dagger u = \frac{1}{(\delta^\dagger \delta)^k}(A + B\sqrt{2}),$$
where $A = a^2+b^2+c^2+d^2$ and $B = cd+bc+ab-da$. Note that, since k is the least δ-exponent of u, Remark D.2 implies that $a + b + c + d$ is odd. It easily follows that A is odd; moreover, since $B \equiv (a + c)(b + d) \pmod 2$, it also follows that B is even. Also note that $(\delta^\dagger \delta)^k = (\lambda\sqrt{2})^k$ is an element of $\mathbb{Z}[\sqrt{2}]$, and is divisible by 2 since $k \geqslant 2$. Therefore, we have $(\delta^\dagger \delta)^k = C + D\sqrt{2}$ for some $C, D \in \mathbb{Z}$ where C, D are both even. We further calculate
$$\xi = 1 - u^\dagger u = \frac{1}{(\delta^\dagger \delta)^k}(C + D\sqrt{2} - A - B\sqrt{2}) = \frac{1}{(\delta^\dagger \delta)^k}(x + y\sqrt{2}),$$
where $x = C - A$ is odd and $y = D - B$ is even. Noting that $(\delta^\dagger \delta)^\bullet (\delta^\dagger \delta) = 2$ and that $x^2 - 2y^2 \equiv 1 \pmod 8$, we therefore have
$$\xi^\bullet \xi = \frac{1}{((\delta^\dagger \delta)^\bullet (\delta^\dagger \delta))^k}(x^2 - 2y^2) = \frac{1}{2^k}(x^2 - 2y^2).$$
It follows that $\ell = k$ and $n = x^2 - 2y^2$, and therefore $n \equiv 1 \pmod 8$, which was to be shown. □

E Proof of Lemma 8.6

Lemma E.1. *(a)* $\sqrt{x} \geqslant \ln x$ *for all* $x > 0$.

(b) $3\sqrt{x} \geqslant (\ln x)^2$ *for all* $x \geqslant 1$.

(c) $(1 - \frac{a}{x})^x \leqslant e^{-a}$, *for all* $x \geqslant a > 0$.

Proof. By elementary calculus. For (a) and (b), note that the functions $f(x) = \ln x/\sqrt{x}$ and $g(x) = (\ln x)^2/3\sqrt{x}$, on the given domains, take their maxima at $x = e^2$ and $x = e^4$, respectively, and in both cases, the maximum is less than 1. For (c), first note that for all $z \in \mathbb{R}$, $z + 1 \leqslant e^z$; the claim follows by letting $z = -\frac{a}{x}$. □

We can now prove Lemma 8.6, whose statement we reproduce here.

Lemma. *Let* $b > 0$ *be an arbitrary fixed constant. Then for* $a \geqslant 1$,
$$\sum_{x=1}^{\infty} \left(1 - \frac{1}{a + b \ln x}\right)^x = O(a).$$

Proof. Let
$$x_0 = 16a^2 + 144b^2. \tag{46}$$
We claim that for all $x \geqslant x_0$,
$$\left(1 - \frac{1}{a + b \ln x}\right)^x \leqslant \frac{1}{x^2}. \tag{47}$$
Indeed, from $x \geqslant 16a^2$, we get
$$\frac{\sqrt{x}}{2} \geqslant 2a. \tag{48}$$
From $x \geqslant 144b^2$, we get
$$\frac{\sqrt{x}}{6} \geqslant 2b. \tag{49}$$

Combining (48) and (49) with Lemma E.1(a) and (b), we have

$$x = \frac{\sqrt{x}}{2} \cdot \sqrt{x} + \frac{\sqrt{x}}{6} \cdot 3\sqrt{x} \geqslant 2a \ln x + 2b(\ln x)^2 = 2 \ln x(a + b \ln x),$$

hence

$$\frac{1}{a + b \ln x} \geqslant \frac{2 \ln x}{x},$$

hence

$$\left(1 - \frac{1}{a + b \ln x}\right)^x \leqslant \left(1 - \frac{2 \ln x}{x}\right)^x \leqslant e^{-2 \ln x} = \frac{1}{x^2}$$

where the final inequality uses Lemma E.1(c). This finishes the proof of (47). The lemma now immediately follows, because we have

$$\begin{aligned}
\sum_{x=1}^{\infty} \left(1 - \frac{1}{a + b \ln x}\right)^x &= \sum_{x=1}^{\lfloor x_0 \rfloor} \left(1 - \frac{1}{a + b \ln x}\right)^x + \sum_{x=\lfloor x_0 \rfloor + 1}^{\infty} \left(1 - \frac{1}{a + b \ln x}\right)^x \\
&\leqslant \sum_{x=1}^{\lfloor x_0 \rfloor} \left(1 - \frac{1}{a + b \ln x_0}\right)^x + \sum_{x=\lfloor x_0 \rfloor + 1}^{\infty} \frac{1}{x^2} \\
&\leqslant \sum_{x=0}^{\infty} \left(1 - \frac{1}{a + b \ln x_0}\right)^x + \sum_{x=1}^{\infty} \frac{1}{x^2} \\
&= a + b \ln x_0 + \frac{\pi^2}{6} \;=\; a + b \ln(16a^2 + 144b^2) + \frac{\pi^2}{6} \;=\; O(a).
\end{aligned}$$

□

References

[1] A. Bocharov, Y. Gurevich, and K. M. Svore. Efficient decomposition of single-qubit gates into V basis circuits. *Phys. Rev. A*, 88:012313 (13 pages), 2013. Also available from `arXiv:1303.1411`.

[2] A. Bocharov, M. Roetteler, and K. M. Svore. Efficient synthesis of probabilistic quantum circuits with fallback. *Physical Review A*, 91:052317, May 2015. Also available from `arXiv:1409.3552`.

[3] A. Bocharov, M. Roetteler, and K. M. Svore. Efficient synthesis of universal repeat-until-success quantum circuits. *Phys. Rev. Lett.*, 114:080502, Feb. 2015. Also available from `arXiv:1404.5320`.

[4] G. Duclos-Cianci and K. M. Svore. Distillation of nonstabilizer states for universal quantum computation. *Phys. Rev. A*, 88:042325 (7 pages), 2013. Also available from `arXiv:1210.1980`.

[5] A. G. Fowler. Constructing arbitrary Steane code single logical qubit fault-tolerant gates. *Quantum Information and Computation*, 11(9–10):867–873, 2011. Also available from `arXiv:quant-ph/0411206`.

[6] B. Giles and P. Selinger. Exact synthesis of multiqubit Clifford+T circuits. *Physical Review A*, 87:032332, 2013. Also available from `arXiv:1212.0506`.

[7] B. Giles and P. Selinger. Remarks on Matsumoto and Amano's normal form for single-qubit Clifford+T operators. `arXiv:1312.6584`, Dec. 2013.

[8] V. Kliuchnikov, A. Bocharov, and K. M. Svore. Asymptotically optimal topological quantum compiling. *Phys. Rev. Lett.*, 112:140504, Apr 2014. Also available from `arXiv:1310.4150`.

[9] V. Kliuchnikov, D. Maslov, and M. Mosca. Asymptotically optimal approximation of single qubit unitaries by Clifford and T circuits using a constant number of ancillary qubits. *Phys. Rev. Lett.*, 110:190502 (5 pages), 2013. Also available from `arXiv:1212.0822v2`.

[10] V. Kliuchnikov, D. Maslov, and M. Mosca. Fast and efficient exact synthesis of single qubit unitaries generated by Clifford and T gates. *Quantum Information and Computation*, 13(7–8):607–630, 2013. Also available from `arXiv:1206.5236v4`.

[11] V. Kliuchnikov, D. Maslov, and M. Mosca. Practical approximation of single-qubit unitaries by single-qubit quantum Clifford and T circuits. *IEEE Transactions on Computers*, 65(1):161–172, Jan 2016. Also available from `arXiv:1212.6964v2`.

[12] M. O. Rabin. Probabilistic algorithms in finite fields. *SIAM Journal on Computing*, 9(2):273–280, 1980.

[13] N. J. Ross and P. Selinger. Exact and approximate synthesis of quantum circuits, version 0.3.0.1. Software implementation, `http://www.mathstat.dal.ca/~selinger/newsynth/`, 2015.

[14] P. Selinger. Efficient Clifford+T approximation of single-qubit operators. *Quantum Information and Computation*, 15(1-2):159–180, Jan. 2015. Also available from `arXiv:1212.6253`.

[15] P. W. Shor. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings of the 35th Annual Symposium on Foundations of Computer Science*, pages 124–134, 1994. Also available from `arXiv:quant-ph/9508027`.

[16] N. Wiebe and V. Kliuchnikov. Floating point representations in quantum circuit synthesis. *New Journal of Physics*, 15:093041 (24 pages), 2013. Also available from `arXiv:1305.5528`.

MIT-CTP/4610

A Quantum Approximate Optimization Algorithm

Edward Farhi and Jeffrey Goldstone

Center for Theoretical Physics

Massachusetts Institute of Technology

Cambridge, MA 02139

Sam Gutmann

arXiv:1411.4028v1 [quant-ph] 14 Nov 2014

Abstract

We introduce a quantum algorithm that produces approximate solutions for combinatorial optimization problems. The algorithm depends on an integer $p \geq 1$ and the quality of the approximation improves as p is increased. The quantum circuit that implements the algorithm consists of unitary gates whose locality is at most the locality of the objective function whose optimum is sought. The depth of the circuit grows linearly with p times (at worst) the number of constraints. If p is fixed, that is, independent of the input size, the algorithm makes use of efficient classical preprocessing. If p grows with the input size a different strategy is proposed. We study the algorithm as applied to MaxCut on regular graphs and analyze its performance on 2-regular and 3-regular graphs for fixed p. For $p = 1$, on 3-regular graphs the quantum algorithm always finds a cut that is at least 0.6924 times the size of the optimal cut.

I. INTRODUCTION

Combinatorial optimization problems are specified by n bits and m clauses. Each clause is a constraint on a subset of the bits which is satisfied for certain assignments of those bits and unsatisfied for the other assignments. The objective function, defined on n bit strings, is the number of satisfied clauses,

$$C(z) = \sum_{\alpha=1}^{m} C_\alpha(z) \tag{1}$$

where $z = z_1 z_2 \dots z_n$ is the bit string and $C_\alpha(z) = 1$ if z satisfies clause α and 0 otherwise. Typically C_α depends on only a few of the n bits. Satisfiability asks if there is a string that satisfies every clause. MaxSat asks for a string that maximizes the objective function. Approximate optimization asks for a string z for which $C(z)$ is close to the maximum of C. In this paper we present a general quantum algorithm for approximate optimization. We study its performance in special cases of MaxCut and also propose an alternate form of the algorithm geared toward finding a large independent set of vertices of a graph.

The quantum computer works in a 2^n dimensional Hilbert space with computational basis vectors $|z\rangle$, and we view (1) as an operator which is diagonal in the computational basis. Define a unitary operator $U(C, \gamma)$ which depends on an angle γ,

$$U(C, \gamma) = e^{-i\gamma C} = \prod_{\alpha=1}^{m} e^{-i\gamma C_\alpha} \ . \tag{2}$$

All of the terms in this product commute because they are diagonal in the computational basis and each term's locality is the locality of the clause α. Because C has integer eigenvalues we can restrict γ to lie between 0 and 2π. Define the operator B which is the sum of all single bit σ^x operators,

$$B = \sum_{j=1}^{n} \sigma_j^x. \tag{3}$$

Now define the β dependent product of commuting one bit operators

$$U(B, \beta) = e^{-i\beta B} = \prod_{j=1}^{n} e^{-i\beta\sigma_j^x} \tag{4}$$

where β runs from 0 to π. The initial state $|s\rangle$ will be the uniform superposition over computational basis states:

$$|s\rangle = \frac{1}{\sqrt{2^n}} \ \sum_z |z\rangle \, . \tag{5}$$

For any integer $p \geq 1$ and $2p$ angles $\gamma_1 \ldots \gamma_p \equiv \boldsymbol{\gamma}$ and $\beta_1 \ldots \beta_p \equiv \boldsymbol{\beta}$ we define the angle dependent quantum state:

$$|\boldsymbol{\gamma}, \boldsymbol{\beta}\rangle = U(B, \beta_p)\, U(C, \gamma_p) \cdots U(B, \beta_1)\, U(C, \gamma_1)\ |s\rangle\,. \tag{6}$$

Even without taking advantage of the structure of the instance, this state can be produced by a quantum circuit of depth at most $mp + p$. Let F_p be the expectation of C in this state

$$F_p(\boldsymbol{\gamma}, \boldsymbol{\beta}) = \langle \boldsymbol{\gamma}, \boldsymbol{\beta}|\, C\, |\boldsymbol{\gamma}, \boldsymbol{\beta}\rangle\,. \tag{7}$$

and let M_p be the maximum of F_p over the angles,

$$M_p = \max_{\boldsymbol{\gamma}, \boldsymbol{\beta}}\ F_p\,(\boldsymbol{\gamma}, \boldsymbol{\beta}). \tag{8}$$

Note that the maximization at $p-1$ can be viewed as a constrained maximization at p so

$$M_p \geq M_{p-1}. \tag{9}$$

Furthermore we will later show that

$$\lim_{p \to \infty} M_p = \max_z\ C(z). \tag{10}$$

These results suggest a way to design an algorithm. Pick a p and start with a set of angles $(\boldsymbol{\gamma}, \boldsymbol{\beta})$ that somehow make F_p as large as possible. Use the quantum computer to get the state $|\boldsymbol{\gamma}, \boldsymbol{\beta}\rangle$. Measure in the computational basis to get a string z and evaluate $C(z)$. Repeat with the same angles. Enough repetitions will produce a string z with $C(z)$ very near or greater than $F_p(\boldsymbol{\gamma}, \boldsymbol{\beta})$. The rub is that it is not obvious in advance how to pick good angles.

If p doesn't grow with n, one possibility is to run the quantum computer with angles $(\boldsymbol{\gamma}, \boldsymbol{\beta})$ chosen from a fine grid on the compact set $[0, 2\pi]^p \times [0, \pi]^p$, moving through the grid to find the maximum of F_p. Since the partial derivatives of $F_p(\boldsymbol{\gamma}, \boldsymbol{\beta})$ in (7) are bounded by $\mathcal{O}(m^2 + mn)$ this search will efficiently produce a string z for which $C(z)$ is close to M_p or larger. However we show in the next section that if p does not grow with n and each bit is involved in no more than a fixed number of clauses, then there is an efficient classical calculation that determines the angles that maximize F_p. These angles are then used to run the quantum computer to produce the state $|\boldsymbol{\gamma}, \boldsymbol{\beta}\rangle$ which is measured in the computational basis to get a string z. The mean of $C(z)$ for strings obtained in this way is M_p.

II. FIXED p ALGORITHM

We now explain how for fixed p we can do classical preprocessing and determine the angles $\boldsymbol{\gamma}$ and $\boldsymbol{\beta}$ that maximize $F_p(\boldsymbol{\gamma}, \boldsymbol{\beta})$. This approach will work more generally but we illustrate it for a specific problem, MaxCut for graphs with bounded degree. The input is a graph with n vertices and an edge set $\{\langle jk \rangle\}$ of size m. The goal is to find a string z that makes

$$C = \sum_{\langle jk \rangle} C_{\langle jk \rangle}, \tag{11}$$

where

$$C_{\langle jk \rangle} = \frac{1}{2} \left(-\sigma_j^z \sigma_k^z + 1 \right), \tag{12}$$

as large as possible. Now

$$F_p(\boldsymbol{\gamma}, \boldsymbol{\beta}) = \sum_{\langle jk \rangle} \langle s | \, U^\dagger(C, \gamma_1) \cdots U^\dagger(B, \beta_p) \, C_{\langle jk \rangle} U(B, \beta_p) \cdots U(C, \gamma_1) \, | s \rangle \, . \tag{13}$$

Consider the operator associated with edge $\langle jk \rangle$

$$U^\dagger(C, \gamma_1) \cdots U^\dagger(B, \beta_p) C_{\langle jk \rangle} U(B, \beta_p) \cdots U(C, \gamma_1). \tag{14}$$

This operator only involves qubits j and k and those qubits whose distance on the graph from j or k is less than or equal to p. To see this consider $p = 1$ where the previous expression is

$$U^\dagger(C, \gamma_1) \, U^\dagger(B, \beta_1) C_{\langle jk \rangle} U(B, \beta_1) \, U(C, \gamma_1). \tag{15}$$

The factors in the operator $U(B, \beta_1)$ which do not involve qubits j or k commute through $C_{\langle jk \rangle}$ and we get

$$U^\dagger(C, \gamma_1) \, e^{i\beta_1(\sigma_j^x + \sigma_k^x)} C_{\langle jk \rangle} e^{-i\beta_1(\sigma_j^x + \sigma_k^x)} \, U(C, \gamma_1). \tag{16}$$

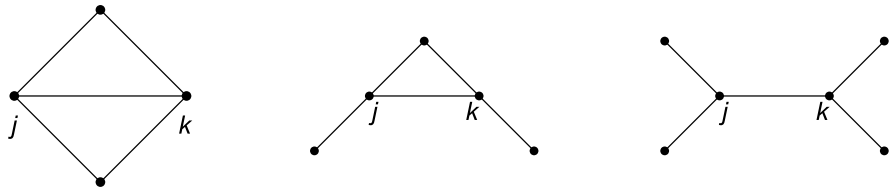
Any factors in the operator $U(C, \gamma_1)$ which do not involve qubits j or k will commute through and cancel out. So the operator in equation (16) only involves the edge $\langle jk \rangle$ and edges adjacent to $\langle jk \rangle$, and qubits on those edges. For any p we see that the operator in (14) only involves edges at most p steps away from $\langle jk \rangle$ and qubits on those edges.

Return to equation (13) and note that the state $|s\rangle$ is the product of σ^x eigenstates

$$|s\rangle = |+\rangle_1 \, |+\rangle_2 \ldots |+\rangle_n \tag{17}$$

so each term in equation (13) depends only on the subgraph involving qubits j and k and those at a distance no more than p away. These subgraphs each contain a number of qubits that is independent of n (because the degree is bounded) and this allows us to evaluate F_p in terms of quantum subsystems whose sizes are independent of n.

As an illustration consider MaxCut restricted to input graphs of fixed degree 3. For $p = 1$, there are only these possible subgraphs for the edge $\langle jk \rangle$:



(18)

We will return to this case later.

For any subgraph G define the operator C_G which is C restricted to G,

$$C_G = \sum_{\langle \ell \ell' \rangle \epsilon G} C_{\langle \ell \ell' \rangle}, \tag{19}$$

and the associated operator

$$U(C_G, \gamma) = e^{-i\gamma C_G}. \tag{20}$$

Also define

$$B_G = \sum_{j \epsilon G} \sigma_j^x \tag{21}$$

and

$$U(B_G, \beta) = e^{-i\beta B_G}. \tag{22}$$

Let the state $|s, G\rangle$ be

$$|s, G\rangle = \prod_{\ell \epsilon G} |+\rangle_\ell .$$

Return to equation (13). Each edge $\langle j, k \rangle$ in the sum is associated with a subgraph $g(j, k)$ and makes a contribution to F_p of

$$\langle s, g(j,k)| U^\dagger \left(C_{g(j,k)}, \gamma_p\right) \cdots U^\dagger (B_{g(j,k)}, \beta_1) C_{\langle jk \rangle} U \left(B_{g(j,k)}, \beta_1\right) \cdots U \left(C_{g(j,k)}, \gamma_p\right) |s, g(j,k)\rangle \tag{23}$$

The sum in (13) is over all edges, but if two edges $\langle jk \rangle$ and $\langle j'k' \rangle$ give rise to isomorphic subgraphs, then the corresponding functions of $(\boldsymbol{\gamma}, \boldsymbol{\beta})$ are the same. Therefore we can view

the sum in (13) as a sum over subgraph types. Define

$$f_g(\boldsymbol{\gamma},\boldsymbol{\beta}) = \langle s, g(j,k)|\, U^\dagger(C_{g(j,k)},\gamma_1)\cdots U^\dagger(B_{g(j,k)},\beta_p) C_{\langle jk\rangle} U(B_{g(j,x)}\beta_p)\cdots \\ U(C_{g(j,k)},\gamma_1)\,|s,g(j,k)\rangle\,, \quad (24)$$

where $g(j,k)$ is a subgraph of type g. F_p is then

$$F_p(\boldsymbol{\gamma},\boldsymbol{\beta}) = \sum_g w_g\, f_g(\boldsymbol{\gamma},\boldsymbol{\beta}) \quad (25)$$

where w_g is the number of occurrences of the subgraph g in the original edge sum. The functions f_g do not depend on n and m. The only dependence on n and m comes through the weights w_g and these are just read off the original graph. Note that the expectation in (24) only involves the qubits in subgraph type g. The maximum number of qubits that can appear in (23) comes when the subgraph is a tree. For a graph with maximum degree v, the numbers of qubits in this tree is

$$q_{\text{tree}} = 2\left[\frac{(v-1)^{p+1}-1}{(v-1)-1}\right], \quad (26)$$

(or $2p+2$ if $v=2$), which is n and m independent. For each p there are only finitely many subgraph types.

Using (24), $F_p(\boldsymbol{\gamma},\boldsymbol{\beta})$ in (25) can be evaluated on a classical computer whose resources are not growing with n. Each f_g involves operators and states in a Hilbert space whose dimension is at most $2^{q_{\text{tree}}}$. Admittedly for large p this may be beyond current classical technology, but the resource requirements do not grow with n.

To run the quantum algorithm we first find the $(\boldsymbol{\gamma},\boldsymbol{\beta})$ that maximize F_p. The only dependence on n and m is in the weights w_g and these are easily evaluated. Given the best $(\boldsymbol{\gamma},\boldsymbol{\beta})$ we turn to the quantum computer and produce the state $|\boldsymbol{\gamma},\boldsymbol{\beta}\rangle$ given in equation (6). We then measure in the computational basis and get a string z and evaluate $C(z)$. Repeating gives a sample of values of $C(z)$ between 0 and $+m$ whose mean is $F_p(\boldsymbol{\gamma},\boldsymbol{\beta})$. An outcome of at least $F_p(\boldsymbol{\gamma},\boldsymbol{\beta})-1$ will be obtained with probability $1-1/m$ with order $m\log m$ repetitions.

III. CONCENTRATION

Still using MaxCut on regular graphs as our example, it is useful to get information about the spread of C measured in the state $|\boldsymbol{\gamma},\boldsymbol{\beta}\rangle$. If v is fixed and p is fixed (or grows

slowly with n) the distribution of $C(z)$ is actually concentrated near its mean. To see this, calculate

$$\langle \boldsymbol{\gamma}, \boldsymbol{\beta} | \, C^2 \, | \boldsymbol{\gamma}, \boldsymbol{\beta} \rangle - \langle \boldsymbol{\gamma}, \boldsymbol{\beta} | \, C \, | \boldsymbol{\gamma}, \boldsymbol{\beta} \rangle^2 \tag{27}$$

$$\begin{aligned} = \sum_{\substack{\langle jk \rangle \\ \langle j'k' \rangle}} \Big[& \langle s | \, U^\dagger(C, \gamma_1) \cdots U^\dagger(B, \beta_p) \, C_{\langle jk \rangle} \, C_{\langle j'k' \rangle} \, U(B, \beta_p) \cdots U(C, \gamma_1) \, | s \rangle \\ & - \langle s | \, U^\dagger(C, \gamma_1) \cdots U^\dagger(B, \beta_p) \, C_{\langle jk \rangle} \, U(B, \beta_p) \cdots U(C, \gamma_1) \, | s \rangle \\ & \cdot \langle s | \, U^\dagger(C, \gamma_1) \cdots U^\dagger(B, \beta_p) \, C_{\langle j'k' \rangle} \, U(B, \beta_p) \cdots U(C, \gamma_1) \, | s \rangle \Big]. \end{aligned} \tag{28}$$

If the subgraphs $g(j,k)$ and $g(j',k')$ do not involve any common qubits, the summand in (28) will be 0. The subgraphs $g(j,k)$ and $g(j',k')$ will have no common qubits as long as there is no path in the instance graph from $\langle jk \rangle$ to $\langle j'k' \rangle$ of length $2p+1$ or shorter. From (26) with p replaced by $2p+1$ we see that for each $\langle jk \rangle$ there are at most

$$2 \left[\frac{(v-1)^{2p+2} - 1}{(v-1) - 1} \right] \tag{29}$$

edges $\langle j'k' \rangle$ which could contribute to the sum in (28) (or $4p+4$ if $v = 2$) and therefore

$$\langle \boldsymbol{\gamma}, \boldsymbol{\beta} | \, C^2 \, | \boldsymbol{\gamma}, \boldsymbol{\beta} \rangle - \langle \boldsymbol{\gamma}, \boldsymbol{\beta} | \, C \, | \boldsymbol{\gamma}, \boldsymbol{\beta} \rangle^2 \leqslant 2 \left[\frac{(v-1)^{2p+2} - 1}{(v-1) - 1} \right] \cdot m \tag{30}$$

since each summand is at most 1 in norm. For v and p fixed we see that the standard deviation of $C(z)$ is at most of order $\sqrt{m}$. This implies that the sample mean of order m^2 values of $C(z)$ will be within 1 of $F_p(\boldsymbol{\gamma}, \boldsymbol{\beta})$ with probability $1 - \frac{1}{m}$. The concentration of the distribution of $C(z)$ also means that there is only a small probability that the algorithm will produce strings with $C(z)$ much bigger than $F_p(\boldsymbol{\gamma}, \boldsymbol{\beta})$.

IV. THE RING OF DISAGREES

We now analyze the performance of the quantum algorithm for MaxCut on 2-regular graphs. Regular of degree 2 (and connected) means that the graph is a ring. The objective operator is again given by equation (11) and its maximum is n or $n-1$ depending on whether n is even or odd. We will analyze the algorithm for all p.

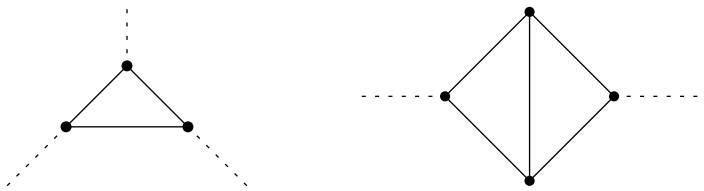
For any p (less than $n/2$), for each edge in the ring, the subgraph of vertices within p of the edge is a segment of $2p+2$ connected vertices with the given edge in the middle. So for

each p there is only one type of subgraph, a line segment of $2p+2$ qubits and the weight for this subgraph type is n. We numerically maximize the function given in (24) and we find that for $p = 1, 2, 3, 4, 5$ and 6 the maxima are 3/4, 5/6, 7/8, 9/10, 11/12, and 13/14 to 13 decimal places from which we conclude that $M_p = n(2p+1)/(2p+2)$ for all p. So the quantum algorithm will find a cut of size $n(2p+1)/(2p+2) - 1$ or bigger. Since the best cut is n, we see that our quantum algorithm can produce an approximation ratio that can be made arbitrarily close to 1 by making p large enough, independent of n. For each p the circuit depth can be made $3p$ by breaking the edge sum in C into two sums over $\langle j, j+1\rangle$ with j even and j odd. So this algorithm has a circuit depth independent of n.

V. MAXCUT ON 3-REGULAR GRAPHS

We now look at how the Quantum Approximate Optimization Algorithm, the QAOA, performs on MaxCut on (connected) 3-regular graphs. The approximation ratio is $C(z)$, where z is the output of the quantum algorithm, divided by the maximum of C. We first show that for $p = 1$, the worst case approximation ratio that the quantum algorithm produces is 0.6924.

Suppose a 3-regular graph with n vertices (and accordingly $3n/2$ edges) contains T "isolated triangles" and S "crossed squares", which are subgraphs of the form,



. (31)

The dotted lines indicate edges that leave the isolated triangle and the crossed square. To say that the triangle is isolated is to say that the 3 edges that leave the triangle end on distinct vertices. If the two edges that leave the crossed square are in fact the same edge, then we have a 4 vertex disconnected 3-regular graph. For this special case (the only case where the analysis below does not apply) the approximation ratio is actually higher than 0.6924. In general, $3T + 4S \leq n$ because no isolated triangle and crossed square can share a vertex.

Return to the edge sum in $F_1(\gamma, \beta)$ of equation (13). For each crossed square there is

one edge $\langle jk \rangle$ for which $g(j,k)$ is the first type displayed in (18). Call this subgraph type g_4 because it has 4 vertices. In each crossed square there are 4 edges that give rise to subgraphs of the second type displayed in (18). We call this subgraph type g_5 because it has 5 vertices. All 3 of the edges in any isolated triangle have subgraph type g_5, so there are $4S+3T$ edges with subgraph type g_5. The remaining edges in the graph all have a subgraph type like the third one displayed in (18) and we call this subgraph type g_6. There are $(3n/2 - 5S - 3T)$ of these so we have

$$F_1(\gamma,\beta) = S f_{g_4}(\gamma,\beta) + (4S+3T)\, f_{g_5}(\gamma,\beta) + \left(\frac{3n}{2} - 5S - 3T\right) f_{g_6}(\gamma,\beta) \tag{32}$$

The maximum of F_1 is a function of n, S, and T,

$$M_1(n,S,T) = \max_{\gamma,\beta} F_1(\gamma,\beta). \tag{33}$$

Given any 3 regular graph it is straightforward to count S and T. Then using a classical computer it is straightforward to calculate $M_1(n,S,T)$. Running a quantum computer with the maximizing angles γ and β will produce the state $|\gamma,\beta\rangle$ which is then measured in the computational basis. With order $n \log n$ repetitions a string will be found whose cut value is very near or larger than $M_1(n,S,T)$.

To get the approximation ratio we need to know the best cut that can be obtained for the input graph. This is not just a function of S and T. However a graph with S crossed squares and T isolated triangles must have at least one unsatisfied edge per crossed square and one unsatisfied edge per isolated triangle so the number of satisfied edges is $\leq (3n/2 - S - T)$. This means that for any graph characterized by n, S and T the quantum algorithm will produce an approximation ratio that is at least

$$\frac{M_1(n,S,T)}{\left(\frac{3n}{2} - S - T\right)}. \tag{34}$$

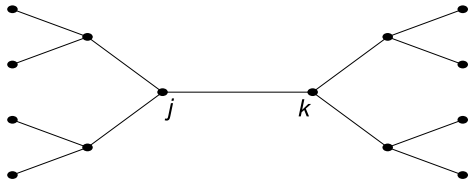
It is convenient to scale out n from the top and bottom of (34). Note that M_1/n which comes from F_1/n depends only on $S/n \equiv s$ and $T/n \equiv t$. So we can write (34) as

$$\frac{M_1(1,s,t)}{\left(\frac{3}{2} - s - t\right)} \tag{35}$$

where $s,t \geq 0$ and $4s+3t \leq 1$. It is straightforward to numerically evaluate (35) and we find that it achieves its minimum value at $s = t = 0$ and the value is 0.6924. So we know that on any 3-regular graph, the QAOA will always produce a cut whose size is at least 0.6924

times the size of the optimal cut. This $p = 1$ result on 3-regular graphs is not as good as known classical algorithms [1].

It is possible to analyze the performance of the QAOA for $p = 2$ on 3-regular graphs. However it is more complicated then the $p = 1$ case and we will just show partial results. The subgraph type with the most qubits is this tree with 14 vertices:



(36)

Numerically maximizing (24) with g given by (36) yields 0.7559. Consider a 3-regular graph on n vertices with $o(n)$ pentagons, squares and triangles. Then all but $o(n)$ edges have (36) as their subgraph type. The QAOA at $p = 2$ cannot detect whether the graph is bipartite, that is, completely satisfiable, or contains many odd loops of length 7 or longer. If the graph is bipartite the approximation ratio is 0.7559 in the limit of large n. If the graph contains many odd loops (length 7 or more), the approximation ratio will be higher.

VI. RELATION TO THE QUANTUM ADIABATIC ALGORITHM

We are focused on finding a good approximate solution to an optimization problem whereas the Quantum Adiabatic Algorithm, QAA [2], is designed to find the optimal solution and will do so if the run time is long enough. Consider the time dependent Hamiltonian $H(t) = (1 - t/T)B + (t/T)C$. Note that the state $|s\rangle$ is the highest energy eigenstate of B and we are seeking a high energy eigenstate of C. Starting in $|s\rangle$ we could run the quantum adiabatic algorithm and if the run time T were long enough we would find the highest energy eigenstate of C. Because B has only non-negative off-diagonal elements, the Perron-Frobenius theorem implies that the difference in energies between the top state and the one below is greater than 0 for all $t < T$, so for sufficiently large T success is assured. A Trotterized approximation to the evolution consists of an alternation of the operators $U(C, \gamma)$ and $U(B, \beta)$ where the sum of the angles is the total run time. For a good approximation we want each γ and β to be small and for success we want a long run time so together these

force p to be large. In other words, we can always find a p and a set of angles $\boldsymbol{\gamma}, \boldsymbol{\beta}$ that make $F_p(\boldsymbol{\gamma}, \boldsymbol{\beta})$ as close to M_p as desired. With (9), this proves the assertion of (10).

The previous discussion shows that we can get a good approximate solution to an optimization problem by making p sufficiently large, perhaps exponentially large in n. But the QAA works by producing a state with a large overlap with the optimal string. In this sense (10), although correct, may be misleading. In fact on the ring of disagrees the state produced at $p = 1$, which gives a 3/4 approximation ratio, has an exponentially small overlap with the optimal strings.

We also know an example where the QAA fails and the QAOA succeeds. In this example (actually a minimization) the objective function is symmetric in the n bits and therefore depends only on the Hamming weight. The objective function is plotted in figure 1 of reference [3]. Since the beginning Hamiltonian is also symmetric the evolution takes place in a subspace of dimension $n+1$ with a basis of states $|w\rangle$ indexed by the Hamming weight. The example can be simulated and analyzed for large n. For subexponential run times, the QAA is trapped in a false minimum at $w = n$. The QAOA can be similarly simulated and analyzed. For large n, even with $p = 1$, there are values of γ_1 and β_1 such that the final state is concentrated near the true minimum at $w = 0$.

The Quantum Approximate Optimization Algorithm has the key feature that as p increases the approximation improves. We contrast this to the performance of the QAA. For realizations of the QAA there is a total run time T that also appears in the instantaneous Hamiltonian, $H(t) = \widetilde{H}(t/T)$. We start in the ground state of $\widetilde{H}(0)$ seeking the ground state of $\widetilde{H}(1)$. As T goes to infinity the overlap of the evolved state with the desired state goes to 1. However the success probability is generally not a monotonic function of T. See figure 2 of reference [4] for an extreme example where the success probability is plotted as a function of T for a particular 20 qubit instance of Max2Sat. The probability rises and then drops dramatically, and the ultimate rise for large T is not seen for times that can be reasonably simulated. It may well be advantageous in designing strategies for the QAOA to use the fact that the approximation improves as p increases.

VII. A VARIANT OF THE ALGORITHM

We are now going to give a variant of the basic algorithm which is suited to situations where the search space is a complicated subset of the n bit strings. We work with an example that illustrates the basic idea. Consider the problem of finding a large independent set in a given graph of n vertices. An independent set is a subset of the vertices with the property that no two vertices in the subset have an edge between them. With the vertices labeled 1 to n, a subset of the vertices corresponds to the string $z = z_1 z_2 \dots z_n$ with each bit being 1 if the corresponding vertex is in the subset and the bit is 0 if the vertex is not. We restrict to strings which correspond to independent sets in the graph. The size of the independent set is the Hamming weight of the string z which we denote by $C(z)$,

$$C(z) = \sum_{j=1}^{n} z_j, \tag{37}$$

and the goal is to find a string z that makes $C(z)$ large.

The Hilbert space for our quantum algorithm has an orthonormal basis $|z\rangle$ where z is any string corresponding to an independent set. In cases of interest, the Hilbert space dimension is exponentially large in n, though not as big as 2^n. The Hilbert space is not a simple tensor product of qubits. The operator C is associated with the γ dependent unitary

$$U(C, \gamma) = e^{-i\gamma C} \tag{38}$$

where γ lies between 0 and 2π because C has integer eigenvalues. We define the quantum operator B that connects the basis states:

$$\langle z|B|z'\rangle = \begin{cases} 1 : z \text{ and } z' \text{ differ in one bit} \\ 0 : \text{otherwise} \end{cases} . \tag{39}$$

Note that B is the adjacency matrix of the hypercube restricted to the legal strings, that is, those that correspond to independent sets in the given graph. Now, in general, B does not have integer eigenvalues so we define

$$U(B, b) = e^{-ibB} \tag{40}$$

where b is a real number.

For the starting state of our algorithm we take the easy to construct state $|z=0\rangle$ corresponding to the empty independent set which has the minimum value of C. For $p \geq 1$, we have p real numbers $b_1, b_2 \ldots b_p \equiv \mathbf{b}$ and $p-1$ angles $\gamma_1, \gamma_2, .\gamma_{p-1} \equiv \boldsymbol{\gamma}$. The quantum state

$$|\mathbf{b}, \boldsymbol{\gamma}\rangle = U(B, b_p)U(C, \gamma_{p-1}) \cdots U(B, b_1)|z=0\rangle \tag{41}$$

is what we get after the application of an alternation of the operators associated with B and C. Now we can define

$$F_p(\mathbf{b}, \boldsymbol{\gamma}) = \langle \mathbf{b}, \boldsymbol{\gamma}|C|\mathbf{b}, \boldsymbol{\gamma}\rangle \tag{42}$$

as the expectation of C in the state $|\mathbf{b}, \boldsymbol{\gamma}\rangle$. And finally we define the maximum,

$$M_p = \max_{\mathbf{b}, \boldsymbol{\gamma}} F_p(\mathbf{b}, \boldsymbol{\gamma}). \tag{43}$$

The maximization at $p-1$ is the maximization at p with $b_p = 0$ and $\gamma_{p-1} = 0$ so we have

$$M_p \geq M_{p-1}. \tag{44}$$

Furthermore,

$$\lim_{p \to \infty} M_p = \max_{z \text{ legal}} C(z). \tag{45}$$

To see why (45) is true note that the initial state is the ground state of C, which we view as the state with the maximum eigenvalue of $-C$. We are trying to reach a state which is an eigenstate of $+C$ with maximum eigenvalue. There is an adiabatic path (which stays at the top of the spectrum throughout) with run time T that achieves this as T goes to infinity. This path has two parts. In the first we interpolate between the beginning Hamiltonian $-C$ and the Hamiltonian B,

$$H(t) = \left(1 - \frac{2t}{T}\right)(-C) + \frac{2t}{T}B \quad , \quad 0 \leq t \leq \frac{T}{2} \tag{46}$$

We evolve the initial state with this Hamiltonian for time $T/2$ ending arbitrarily close to the top state of B. Next we interpolate between the Hamiltonian B and the Hamiltonian $+C$,

$$H(t) = \left(2 - \frac{2t}{T}\right)B + \left(\frac{2t}{T} - 1\right)C \quad , \quad \frac{T}{2} \leq t \leq T \tag{47}$$

evolving the quantum state just produced from time $t = T/2$ to $t = T$. As in section VI, using the Perron-Frobenius Theorem, the Adiabatic Theorem and Trotterization we get the result given in (45).

Together (41) through (45) suggest a quantum subroutine for an independent set algorithm. For a given p and a given $(\mathbf{b}, \boldsymbol{\gamma})$ produce the quantum state $|\mathbf{b}, \boldsymbol{\gamma}\rangle$ of (41). Measure in the computational basis to get a string z which labels an independent set whose size is the Hamming weight of z. Repeat with the same $(\mathbf{b}, \boldsymbol{\gamma})$ to get an estimate of $F_p(\mathbf{b}, \boldsymbol{\gamma})$ in (42). This subroutine can be called by a program whose goal is to get close to M_p given by (43). This enveloping program can be designed using either the methods outlined in this paper or novel techniques.

For $p = 1$, the subroutine can be thought of as evolving the initial state $|z = 0\rangle$ with the Hamlitonian B for a time b. B is the adjacency matrix of a big graph whose vertices correspond to the independent sets of the input graph and whose edges can be read off (39). We view this as a continuous time quantum walk entering the big graph at a particular vertex [5]. In the extreme case where the input graph has no edges, all strings of length n represent independent sets so the Hilbert space dimension is 2^n. In this case B is the adjacency matrix of the hypercube, realizable as in (3). Setting $b = \pi/2$, the state (41) (with $p = 1$ there is only one unitary) is $|z = 11\ldots11\rangle$ which maximizes the objective function. In the more general case we can view (41) as a succession of quantum walks punctuated by applications of C dependent unitaries which aid the walk in achieving its objective. The algorithm of the previous sections can also be viewed this way although the starting state is not a single vertex.

VIII. CONCLUSION

We introduced a quantum algorithm for approximate combinatorial optimization that depends on an integer parameter p. The input is an n bit instance with an objective function C that is the sum of m local terms. The goal is to find a string z for which $C(z)$ is close to C's global maximum. In the basic algorithm, each call to the quantum computer uses a set of $2p$ angles $(\boldsymbol{\gamma}, \boldsymbol{\beta})$ and produces the state

$$|\boldsymbol{\gamma}, \boldsymbol{\beta}\rangle = U(B, \beta_p)\, U(C, \gamma_p) \cdots U(B, \beta_1)\, U(C, \gamma_1)\ |s\rangle\,. \tag{48}$$

This is followed by a measurement in the computational basis yielding a string z with an associated value $C(z)$. Repeated calls to the quantum computer will yield a good estimate

of

$$F_p(\boldsymbol{\gamma}, \boldsymbol{\beta}) = \langle \boldsymbol{\gamma}, \boldsymbol{\beta}| \, C \, |\boldsymbol{\gamma}, \boldsymbol{\beta}\rangle \, . \tag{49}$$

Running the algorithm requires a strategy for a picking a sequence of sets of angles with the goal of making F_p as big as possible. We give several possible strategies for finding a good set of angles.

In section II we focused on fixed p and the case where each bit is in no more than a fixed number of clauses. In this case there is an efficient classical algorithm that determines the best set of angles which is then fed to the quantum computer. Here the quantum computer is run with only the best set of angles. Note that the "efficient" classical algorithm which evaluates (25) using (24) could require space doubly exponential in p.

An alternative to using a classical preprocessor to find the best angles is to make repeated calls to the quantum computer with different sets of angles. One strategy, when p does not grow with n is to put a fine grid on the compact set $[0, 2\pi]^p \times [0, \pi]^p$ where the number of points is only polynomial in n and m. This works because the function F_p does not have peaks that are so narrow that they are not seen by the grid.

The QAOA can be run on a quantum computer with p growing with n as long as there is a strategy for choosing sets of angles. Perhaps for some combinatorial optimization problem, good angles can be discovered in advance. Or the quantum computer can be called to evaluate $F_p(\boldsymbol{\gamma}, \boldsymbol{\beta})$, the expectation of C in the state $|\boldsymbol{\gamma}, \boldsymbol{\beta}\rangle$. This call can be used as a subroutine by a classical algorithm that seeks the maximum of the smooth function $F_p(\boldsymbol{\gamma}, \boldsymbol{\beta})$. We hope that either p fixed or growing slowly with n will be enough to have this quantum algorithm be of use in finding solutions to combinatorial search problems beyond what classical algorithms can achieve.

IX. ACKNOWLEDGEMENTS

This work was supported by the US Army Research Laboratory's Army Research Office through grant number W911NF-12-1-0486, and the National Science Foundation through grant number CCF-121-8176. The authors thank Elizabeth Crosson for discussion and help in preparing the manuscript. We also thank Cedric Lin and Han-Hsuan Lin for their help. EF would like to thank the Google Quantum Artificial Intelligence Lab for discussion and

support.

---

[1] Eran Halperin, Dror Livnat, Uri Zwick.
MAX CUT in cubic graphs, 2004.
Journal of Algorithms, Volume 53 Issue 2, Pages 169-185.

[2] Edward Farhi, Jeffrey Goldstone, Sam Gutmann, Michael Sipser.
Quantum computation by adiabatic evolution, 2000.
arXiv:quant-ph/0001106.

[3] Edward Farhi, Jeffrey Goldstone, Sam Gutmann.
Quantum Adiabatic Evolution Algorithms versus Simulated Annealing, 2002.
arXiv:quant-ph/0201031.

[4] Elizabeth Crosson, Edward Farhi, Cedric Yen-Yu Lin, Han-Hsuan Lin, Peter Shor.
Different strategies for optimization with the quantum adiabatic algorithm, 2014.
arXiv:1401.7320 [quant-ph].

[5] Edward Farhi, Sam Gutmann.
Quantum Computation and Decision Trees, 1997.
Phys. Rev. A 58, 915 arXiv:quant-ph/9706062.

arXiv:1811.03975v1 [quant-ph] 9 Nov 2018

Quantum computational finance: quantum algorithm for portfolio optimization

Patrick Rebentrost[1, 2, *] and Seth Lloyd[3, 2, †]

[1] *Centre for Quantum Technologies, National University of Singapore, 3 Science Drive 2, Singapore 117543*

[2] *Research Laboratory of Electronics, Massachusetts Institute of Technology, Cambridge, MA 02139*

[3] *Department of Mechanical Engineering, Massachusetts Institute of Technology, Cambridge, MA 02139*

We present a quantum algorithm for portfolio optimization. We discuss the market data input, the processing of such data via quantum operations, and the output of financially relevant results. Given quantum access to the historical record of returns, the algorithm determines the optimal risk-return tradeoff curve and allows one to sample from the optimal portfolio. The algorithm can in principle attain a run time of $\mathrm{poly}(\log(N))$, where N is the size of the historical return dataset. Direct classical algorithms for determining the risk-return curve and other properties of the optimal portfolio take time $\mathrm{poly}(N)$ and we discuss potential quantum speedups in light of the recent works on efficient classical sampling approaches.

PACS numbers:

I. INTRODUCTION

Quantum computing promises advantages for certain problems, including prime factoring, hidden subgroup problems, and unstructured search [1]. Quantum computers perform favorably the task of processing vectors in large-dimensional Hilbert spaces. For n quantum bits (qubits), the corresponding Hilbert space is 2^n dimensional and a normalized quantum state over n qubits is fully described by $2^n - 1$ complex numbers. Many basic quantum algorithms, such as the quantum Fourier transform, operate with run time $\mathcal{O}\left(\mathrm{poly}(n)\right)$ while manipulating all the coefficients classically would take $\mathcal{O}\left(2^n\right)$ operations. Quantum computing also provides speedups for sparse matrix inversion via the Harrow-Hassidim-Lloyd (HHL) algorithm [2, 3], an algorithm that finds application, e.g., in data fitting [4] and machine learning [5]. Because algorithms such as the quantum matrix inversion algorithm provide the solution in the form of a quantum state, care must be taken to insure that the algorithm can reveal the desired output in time $\mathcal{O}\left(\mathrm{poly}(n)\right)$ [6].

Modern finance employs large amounts of computational resources for a variety of tasks. Computers are used for example for the analysis of historical data, high-frequency trading, pricing of exotic financial derivatives, portfolio optimization and risk management [7–10]. The complexity often arises from the

*Electronic address: cqtfpr@nus.edu.sg,pr@patrickre.com

†Electronic address: slloyd@mit.edu

analysis of time series of asset prices over long periods involving many assets, motivating the use of quantum computing for financial problems. Quantum mechanics in combination with finance has been considered before. One avenue is the application of techniques from quantum physics, such as path integrals, to options pricing [11, 12]. With the advent of intermediate-scale quantum computers, employing their power in finance is becoming more and more viable. Recently, the quantum amplitude estimation algorithm was shown to allow up to quadratic speedups for derivative pricing [13] and risk management [14]. Several financial optimization problems have been considered by 1Qbit [15] in the context of the D-Wave machine and the quantum annealing paradigm. In the same context, portfolio optimization was discussed in [16, 17]. To this point, no gate model quantum algorithm for portfolio optimization has been exhibited.

Here, we present quantum algorithms for portfolio management, specifically portfolio optimization [18] and risk analysis. Portfolio optimization can be phrased as an equality-constraint quadratic programming problem: one finds the portfolio that maximizes expected return for fixed standard deviation (risk), or, equivalently, that minimizes the standard deviation of the return for fixed expected return. The graph of maximal returns for fixed risk/minimum risks for fixed returns is called the risk-return curve, see [19] for a figure. Determining the risk-return curve and identifying the portfolio that maximizes return for a given risk is a central task of classical portfolio management theory. Quadratic optimization is a problem that is known to be susceptible to speedup via quantum computation: here, we apply the quantum linear systems algorithm and its variations [2, 3] to obtain the the risk-return curve and to identify the minimal-risk portfolio for given desired return. The optimal portfolio is presented as a quantum state which can then be sampled to give a sparse portfolio that is close to optimal.

The quantum portfolio optimization algorithm can exhibit a run time that is logarithmic in the number of assets and the number of time steps. Suppose that there are N possible investments and T time instances, and our data set consists of T N-dimensional vectors of the historic returns for those investments stored in random access memory (RAM). Direct classical algorithms for portfolio optimization proceed by constructing the $N \times N$ covariance matrix for the returns, using $\mathcal{O}\left(TN^2\right)$ operations/calls to RAM, and implementing its pseudo-inverse ($\mathcal{O}\left(N^2\right)$ operations/memory calls) to construct the optimal risk-return curve and to find the portfolio that maximizes return for a fixed value of risk. The quantum algorithms for portfolio management presented here can find the risk-return curve and reveal the quantum state corresponding to the optimal portfolio in time $\mathcal{O}\left(\mathrm{poly}\log(TN)\right)$. If the relevant data is stored in quantum random access memory (qRAM) [20–23], so that the quantum computer can access it in quantum parallel, then a quantum computer can implement the HHL linear systems algorithm and its variants [2, 3], quantum walk Hamiltonian simulation methods, and the quantum state exponentiation method [24, 25] to implement the matrix pseudo-inverse and to solve the quadratic optimization problem. The quantum algorithm for performing the

pseudoinverse can take $\mathcal{O}\left(\text{poly}\log(TN)\right)$ operations/memory calls. Of course, loading the data set into the qRAM to begin with takes time $\mathcal{O}\left(TN\right)$: the logarithmic run time refers to the analysis of the data once it has been loaded into memory. The output of the quantum algorithm is the optimal risk-return curve, together with quantum representations of the optimal portfolios that attain the maximum return for each value of the risk. The algorithm returns the optimal portfolio as a quantum state vector (dimension N). To determine the exact composition of the optimal portfolio would take time $\mathcal{O}\left(N\right)$ to determine each component of that state vector. We can, however, sample from the optimal portfolio: the probability of returning a particular stock is proportional to (the square of) its weight in the portfolio. By standard results on Monte Carlo sampling, a portfolio constructed from a sample of M elements taken from the optimal portfolio has an expected standard deviation within $\mathcal{O}\left(1/\sqrt{M}\right)$ of the minimum standard deviation. Moreover, we can make quantum measurements on the optimal portfolio state to determine its weight in different sectors, e.g., energy, telecommunications, technology, etc. We can also compare the expected return and variance of any specific portfolio with that of the optimal one, to determine the degree of sub-optimality. Finally, we can map out the efficient frontier, that is the optimal risk-return trade-off curve, in a time $\mathcal{O}\left(\log TN\right)$, where T is the time horizon (number of time steps) and N is the number of assets in the portfolio. We discuss methods for loading the financial data via quantum random access memory and the preparation of the necessary input data structures, which are the expected return vector and the covariance matrix. Finally, we discuss the quantum algorithms given here in the context of recent quantum-inspired classical algorithms that also give a logarithmic run time, for example for recommendation systems, principal component analysis, and supervised clustering [26–28].

II. MARKET DATA

A. Prices and returns

The sheer size of the financial market means that a vast amount of data is processed by the electronic exchanges. For each asset, such as stocks, bonds, options, and so on, the prices are moving on ever-shortening time scales, as short as milliseconds or below. For managing a diversified portfolio or for performing a quantitative analysis of the economy [29], a large number of high-dimensional vectors have to be processed. Each high-dimensional vector represents the time series of a particular asset. Concretely, we have N asset prices $\Pi_s(t)$ over a discretized time interval $T' \in \mathbb{Z}_+$, taken to be a positive integer, where $s = [N]$ and $t = [T']$ and by definition $[x] := 1, \ldots, x$. Here, the time difference is normalized to 1 which depending on the desired application can be below milliseconds.

In addition, a "return" of a financial asset is a percentage profit or loss over a defined time period. Let this time period be a positive integer $\Delta t \in \mathbb{Z}_+$. The return for an asset s at time t is here defined as

$$y_s(t) := \frac{\Pi_s(t) - \Pi_s(t - \Delta t)}{\Pi_s(t - \Delta t)}. \tag{1}$$

The time label for the return is $t = \Delta t + 1, \ldots, T'$, as the prices $\Pi_s(t)$ are by assumption only given for $[T']$. We can redefine the time label for the returns as $t = [T]$, where $T = T' - \Delta t$.

B. Expected return and covariance

The important data structures in this work are the vector of expected returns and the covariance matrix. Arrange the historical returns for the different assets in a vector $\vec{y}(t)$. We denote the corresponding random variable of the returns at a future time Δt from today by $\vec{Y}$. The expected return is defined as

$$\vec{R}^{\mathrm{id}} := \mathbb{E}\left[\vec{Y}\right]. \tag{2}$$

Based on the historical prices, we obtain an estimate of this expected return via

$$\vec{R} = \frac{1}{T}\sum_{t=1}^{T} \vec{y}(t). \tag{3}$$

Assuming σ_s is the variance of the returns of asset s, the standard deviation of the sampled $\vec{R}_s$ is $\mathcal{O}\left(\sigma_s/\sqrt{T}\right)$. Similarly, the covariance matrix is defined via the mean-normalized returns as

$$\Sigma^{\mathrm{id}} = \mathbb{V}\left[\left(\vec{Y} - \vec{R}^{\mathrm{id}}\right)\left(\vec{Y} - \vec{R}^{\mathrm{id}}\right)^T\right], \tag{4}$$

and an estimate can be obtained from the samples as

$$\Sigma = \frac{1}{T-1}\sum_{t=1}^{T}(\vec{y}(t) - \vec{R})(\vec{y}(t) - \vec{R})^T. \tag{5}$$

III. QUANTUM MARKET DATA

A. Quantum data access

For the data input into the quantum computation, we follow closely the discussion in [20–23]. Let $\tilde{d}_i$ be given data with $i = [N_{\mathrm{qRAM}}]$. Take d_i to be the m-bit approximation for these data and assume that the implied error is smaller than other errors in the problem and the quantum algorithm. We assume access to the following query operation

$$\sum_{i=1}^{N_{\mathrm{qRAM}}} \alpha_i |i\rangle|0^{\otimes m}\rangle \to \sum_{i=1}^{N_{\mathrm{qRAM}}} \alpha_i |i\rangle|d_i\rangle, \tag{6}$$

where $\alpha_i \in \mathbb{C}$ are arbitrary amplitudes. One way to implement such an operation is via quantum random access memory (qRAM) [21, 22]. Assume that the data are stored in a classical information storage medium such as a CD or classical RAM. The value d_i of the data is stored in the i'th cell of the memory. Classical RAM can be turned into qRAM by a set of quantum switches allowing the memory cells to be accessed. The quantum switches are arranged as a branching tree of with depth $\mathcal{O}\left(\log N_{\mathrm{qRAM}}\right)$ and width N_{qRAM}. In addition, we need a quantum register with $\log N_{\mathrm{qRAM}}$ qubits, whose contents label the memory cell to be accessed. Quantum RAM allows a quantum-parallel access of the data. As shown in [21, 22], qRAMs have the surprising feature that even though all $\mathcal{O}\left(N_{\mathrm{qRAM}}\right)$ switches in the array participate in a memory call in quantum parallel, the expected number of switches that are actually switched or activated is $\log N_{\mathrm{qRAM}}$: consequently, the energy requirements and amount of decoherence induced by a memory call is proportional to the logarithm of the memory size. When we compare the performance of a quantum algorithm to a classical algorithm, we assume that data can be accessed via classical RAM or quantum RAM, respectively.

There are various potential ways of the financial data being presented as an input to the quantum computation.

B. Expected return and covariance

We start with a setting where the prices are stored and accessed via quantum RAM.

Data Input 1 (Prices)**.** *Assume the qRAM availability of the prices $\Pi_s(t)$ to m-bit approximation, where $s = [N]$ and $t = [T]$, and m is such that the inaccuracies are negligible compared to other errors.*

Based on this data input, we show how to create the expected return and the covariance matrix for the quantum finance algorithms. Input (1) provides an operation

$$|s\rangle|t\rangle|0^{\otimes m}\rangle \rightarrow |s\rangle|t\rangle|\Pi_s(t)\rangle. \tag{7}$$

From this operation the single-step returns can be computed and stored in a m-bit encoding by a reversible operation on the qubit registers

$$|\Pi_s(t)\rangle|\Pi_s(t-\Delta t)\rangle|0^{\otimes m}\rangle \;\rightarrow\; |\Pi_s(t)\rangle|\Pi_s(t-\Delta t)\rangle|y_s(t)\rangle. \tag{8}$$

Here, $|y_s(t)\rangle$ is a multi-qubit state storing $y_s(t)$ in binary representation. Thus, with small polynomial overhead, we are able to construct the operation

$$|s\rangle|t\rangle|0^{\otimes m}\rangle \;\rightarrow\; |s\rangle|t\rangle|y_s(t)\rangle, \tag{9}$$

for $t = [T]$. To obtain a quantum state related to the return vector $\vec{y}$, we perform a controlled rotation of an ancilla qubit as

$$|y_s(t)\rangle|0\rangle \to |y_s(t)\rangle \left(\sqrt{1-\delta^2 y_s(t)^2}|0\rangle + \delta y_s(t)|1\rangle\right), \tag{10}$$

where we choose $0 < \delta$ such that $|y_s(t)|\delta \leq 1$ for all s, t. We apply the sequence of price oracle call, return calculation, and controlled rotation to the superposition $\frac{1}{\sqrt{TN}} \sum_{t=1}^{T} \sum_{s=1}^{N} |t\rangle|s\rangle$. Uncomputing the data registers and measuring the ancilla in state $|1\rangle$ in such a superposition then gives the state

$$\frac{1}{|y|} \sum_{t=1}^{T} \sum_{s=1}^{N} y_s(t)|t\rangle|s\rangle = \frac{1}{|y|} \sum_{t=1}^{T} |\vec{y}(t)||t\rangle|\vec{y}(t)\rangle =: |\chi\rangle, \tag{11}$$

with $|y|^2 = \sum_{t,s} y_s^2(t) = \sum_t |\vec{y}(t)|^2$. Here, we obtain the quantum states encoding the returns for the times t in amplitude encoding

$$|\vec{y}(t)\rangle := \frac{1}{|\vec{y}(t)|} \sum_{s=1}^{N} y_s(t)|s\rangle. \tag{12}$$

The success probability of the ancilla measurement is $P_\chi = \delta^2 \sum_{s,t} y_s(t)^2/TN$. When we assume most of the returns are $y_s(t) = \Theta(1)$, then $P_\chi = \Omega(1)$.

We now turn to the preparation of the vector of expected returns. We generate the vector of expected returns by first applying a Hadamard operation to all qubits of the time register of $|\chi\rangle$ and then measuring the computational zero state

$$\begin{aligned} \frac{1}{\sqrt{T}} \sum_{t=1}^{T} \langle t|\chi\rangle \;&\to\; \frac{1}{|y'|} \sum_{s=1}^{N} \left(\sum_{t=1}^{T} y_s(t) \right) |s\rangle \\ &\equiv\; |\vec{R}\rangle. \end{aligned} \tag{13}$$

Here, $|y'|^2 = \sum_s (\sum_t y_s(t))^2$. The success probability of this measurement is given by $P_R = |y'|^2/(T|y|^2)$. Again, if $y_s(t) = \Theta(1)$, then the success probablity $P_R = \Omega(1)$. A simple quantum finance algorithm can be obtained by sampling of the quantum state $|\vec{R}\rangle$. A particular stock $|s\rangle$ is measured with the probability $\left(\frac{\sum_{t=1}^{T} y_s(t)}{|y'|}\right)^2$. Thus, sampling can give an indication of the assets with the largest returns, especially if certain assets dominate the return vector.

In addition, we can prepare a quantum density matrix proportional to the covariance matrix of the returns. In Appendix A, we show how to prepare a state $|\tilde{\chi}\rangle = \frac{1}{|\tilde{y}|} \sum_{t,s} |t\rangle|s\rangle \left(y_s(t) - \frac{1}{T} \sum_{t'} y_s(t')\right)$, with $|\tilde{y}|^2 = \sum_{t,s} \left(y_s(t) - \frac{1}{T} \sum_{t'} y_s(t')\right)^2 \equiv (T-1)\mathrm{tr}\Sigma$, that amplitude-encodes the mean-normalized returns, in contrast to $|\chi\rangle$ above. If we take a partial trace over the time register of this state $|\tilde{\chi}\rangle$, we obtain the

(normalized) covariance matrix in the subspace of the asset label

$$\begin{aligned}\mathrm{Tr}_1\{|\tilde{\chi}\rangle\langle\tilde{\chi}|\} &= \frac{1}{|\tilde{y}|^2}\sum_{s,s'=1}^{N}\sum_{t=1}^{T}\left(y_s(t)-\frac{1}{T}\sum_{t'}y_s(t')\right)\left(y_{s'}(t)-\frac{1}{T}\sum_{t'}y_s(t')\right)|s\rangle\langle s'| \\ &= \frac{\Sigma}{\mathrm{tr}\Sigma}. \end{aligned} \tag{14}$$

The average return Eq. (13) and the covariance matrix Eq. (14) are the main quantities to be used for the portfolio optimization. Another simple quantum algorithm follows. By measuring this covariance matrix one can obtain samples of the assets with the largest variances and pairs of assets with the largest covariances.

C. Enhanced data input for prices and expected returns

For the quantum portfolio optimization algorithm, it will be advantageous to assume a particular data structure embedded in qRAM. In [26], Kerenidis and Prakash (KP) use a specific binary-tree data structure to store vectors. Each node of the tree stores the subnorms, i.e., the sums of squares of the entries of the vector corresponding to the nodes below. The lowest nodes store the squares of the vector elements and their signs. A similar idea of storing subnorms in quantum RAM was previously discussed in [20]. Such a natural data structure can be embedded in quantum RAM itself and has the advantage that it can be updated efficiently and continuously and allows for a deterministic state preparation analogous to Grover-Rudolph [30] including signs.

Data Input 2 (KP for prices)**.** *Assume a KP data structure embedded in qRAM for present day's asset prices* Π_s, $s = [N]$.

This data structure allows the preparation without measurement of the state

$$|0^{\otimes\lceil\log N\rceil}\rangle \to \frac{1}{|\Pi|}\sum_{s=1}^{N}\Pi_s|s\rangle =: |\Pi\rangle. \tag{15}$$

with $|\Pi|^2 = \sum_{s=1}^{N}\Pi_s^2$. The next data input assumption is stronger, since we assume that the average returns are already precomputed and stored.

Data Input 3 (KP for average returns)**.** *Assume a KP data structure embedded in qRAM for the average returns* R_s, $s = [N]$.

Using this data input, we can perform deterministically the operation $|0^{\otimes\lceil\log N\rceil}\rangle \to |\vec{R}\rangle$. No measurement is required in contrast to Eq. (13).

D. Precomputed covariance matrix

In Sec. III B, we prepared the covariance matrix as a density matrix. Alternatively, in some cases the covariance matrix is already precomputed and stored. Assuming the corresponding quantum access to the matrix elements allows the use of well-studied quantum walk methods for employing Σ in a quantum algorithm.

Data Input 4 (Covariance matrix)**.** *Assume access to the elements Σ_{jk} by querying an oracle as follows:*

$$|jk\rangle|0^{\otimes m}\rangle \to |jk\rangle|\Sigma_{jk}\rangle. \tag{16}$$

In addition, for sparse matrices, we assume the following data input.

Data Input 5 (Sparsity)**.** *Let $i = [N]$ and $l = [s_\Sigma]$, where s_Σ is the sparsity of Σ. Assume access to the operation $|i, l\rangle \to |i, g_\Sigma(i, l)\rangle$, where the efficiently computable function $g_\Sigma(i, l)$ gives the column index of the l-th nonzero element of row i of matrix Σ.*

The use of these operations will be discussed in Section IV A.

IV. PORTFOLIO OPTIMIZATION

H. Markowitz is recognized for introducing the modern version of portfolio theory [18]. An optimal investment strategy achieves a certain desired return while risk is minimized, which is the setting we consider here. Equivalently, the problem can be posed as attaining a certain desired risk while maximizing the return. This simple idea of risk-return optimization leads to the notion of portfolio diversification, that is, the optimal portfolio is likely one investing in many relatively uncorrelated assets. In contrast, when the chosen strategy is to only optimize returns, one obtains that the optimal portfolio selects only the single asset (or a few assets) with the highest expected return, irrespective of the risk involved.

The assets are specified by a vector of today's asset prices given by $\vec{\Pi}$, the historical expected return vector of the assets $\vec{R}$, and the historical correlations in the assets given by Σ. Note the sampling errors when using $\vec{R}$ versus $\vec{R}^{\mathrm{id}}$ and Σ versus Σ^{id}, which carry over to the quantum algorithm. Let the total wealth of an investor be given by ξ. This wealth shall be allocated in the assets, specified by a portfolio allocation vector $\vec{w}$ which gives the amount invested in each asset. Here, we allow for short-selling, which means that the entries of $\vec{w}$ can be negative. Based on the total wealth, we obtain the constraint for the portfolio given by $\xi = \vec{\Pi}^T\vec{w}$. Based on the historical returns, the portfolio has an expected return given by $\vec{R}^T\vec{w}$. In addition, the portfolio risk can by specified as $\vec{w}^T\Sigma\vec{w}$. The portfolio optimization problem considered here

is that the investor would like to achieve an expected return $\mu = \vec{R}^T \vec{w}$, while minimizing the portfolio risk $\vec{w}^T \Sigma \vec{w}$. Thus, the problem is an equality-constrained quadratic program as follows:

$$\min_{\vec{w}} \ \vec{w}^T \Sigma \vec{w} \tag{17}$$

$$\text{s.t.} \quad \vec{R}^T \vec{w} = \mu, \tag{18}$$

$$\vec{\Pi}^T \vec{w} = \xi. \tag{19}$$

We may also redefine the portfolio vector such that the last equality becomes $\vec{1}^T \vec{w} = \xi$, where $\vec{1}$ is the vector of all 1's. Introducing the Lagrange multipliers θ and η for the equality constraints Eqs. (18) and (19), the linear equation system to solve for the optimization problem is given by $M\vec{x} = \vec{b}$, which is defined as

$$\begin{pmatrix} 0 & 0 & \vec{R}^T \\ 0 & 0 & \vec{\Pi}^T \\ \vec{R} & \vec{\Pi} & \Sigma \end{pmatrix} \begin{pmatrix} \eta \\ \theta \\ \vec{w} \end{pmatrix} = \begin{pmatrix} \mu \\ \xi \\ \vec{0} \end{pmatrix}. \tag{20}$$

Quantum mechanically, we solve the corresponding linear system $\hat{M}|\eta, \theta, \vec{w}\rangle = |\mu, \xi, \vec{0}\rangle$ via the HHL algorithm and its variants [2, 3]. The eigenvalues of the matrix $\hat{M} = M/\mathrm{Tr}M$ are denoted by λ_j and the eigenvectors by $|u_j\rangle$. The right-hand side of Eq. (20) becomes the normalized quantum state $|\mu, \xi, \vec{0}\rangle$. Such a state is easy to prepare since it only consists of the two quantities μ and ξ. The solution the HHL algorithm provides is the normalized quantum state:

$$|\eta, \theta, \vec{w}\rangle = \frac{1}{|v|} \sum_{j:\lambda_j \geq 1/\kappa} \frac{\beta_j}{\lambda_j} |u_j\rangle, \tag{21}$$

with the norm $|v| = \sqrt{\sum_{j:\lambda_j \geq 1/\kappa} (\beta_j/\lambda_j)^2}$ and $\beta_j := \langle u_j | \mu, \xi, \vec{0}\rangle$. Here, κ is chosen to be a constant. The HHL algorithm projects onto the well-conditioned subspace with eigenvalues greater than $1/\kappa$. In this way, the algorithm finds the pseudoinverse of $\hat{M}$ in such a way that only eigenvalues $\lambda_j \geq 1/\kappa$ are taken into account. Let us denote this pseudoinverse by $\hat{M}_\kappa^{-1}$. The procedure is equal to the actual pseudoinverse $\hat{M}^{-1}$ whenever $1/\kappa$ is smaller than the smallest eigenvalue $|\lambda_{\min}|$ of $\hat{M}$. Otherwise, $\hat{M}_\kappa^{-1}|\vec{0}, \mu, \xi\rangle$ approximates $\hat{M}^{-1}|\vec{0}, \mu, \xi\rangle$ to an error

$$\epsilon_\kappa := \left| \hat{M}_\kappa^{-1} |\vec{0}, \mu, \xi\rangle - \hat{M}^{-1} |\vec{0}, \mu, \xi\rangle \right|_2. \tag{22}$$

Thus, efficient quantum portfolio optimization requires $\kappa = O\left(\mathrm{poly}\left(\log d\right)\right)$, but also the requirement that $\hat{M}$ to be such that either (1) $|\lambda_{\min}| \geq 1/\kappa$, with no additional errors in finding the pseudoinverse, or (2) $|\lambda_{\min}| < 1/\kappa$ but with $\epsilon_\kappa = \mathcal{O}(\epsilon)$ so that the error ϵ_κ accumulates in accordance with an overall desired error $\mathcal{O}(\epsilon)$.

The success probability of preparing $|\eta, \theta, \vec{w}\rangle$ is

$$P_w = C^2 \sum_{j:\lambda_j \geq 1/\kappa} \left| \frac{\beta_j}{\lambda_j} \right|^2, \tag{23}$$

with a user-specified $C = \mathcal{O}(1/\kappa)$. We can determine P_w itself from multiple runs of the linear systems algorithm. To relate the quantum state $|\eta, \theta, \vec{w}\rangle$ to the actual solution of (20), we multiply $|\eta, \theta, \vec{w}\rangle$ by a factor

$$\sqrt{\frac{P_w(\mu^2 + \xi^2)}{C^2}}\, \mathrm{tr}\Sigma, \tag{24}$$

which involves only known quantities. The trace $\mathrm{tr}\Sigma$ is estimated via Appendix A, Eq. (A15). When measuring properties of the portfolio we always multiply the result by this factor to obtain the actual desired value. To obtain a quantum state $|\vec{w}\rangle$ we project the state $|\eta, \theta, \vec{w}\rangle$ onto the desired part.

A central step in the HHL algorithm is the controlled matrix exponentiation of $\hat{M}$, i.e., the quantum simulation of the unitary $e^{-i\hat{M}\Delta t}$ for the use in phase estimation. The matrix $M = H_\Sigma + H_{\vec{R}} + H_{\vec{\Pi}}$ consists of three parts, (i) the covariance matrix part H_Σ, (ii) the expected return vector part $H_{\vec{R}}$, and (iii) the price vector $H_{\vec{\Pi}}$. The simulation of these parts is discussed in the following subsections. The matrix sum can be then simulated via the Lie product formula a small time step Δt such that $\exp(-iMt) \approx \exp(-iH_\Sigma \Delta t) \exp(-iH_{\vec{R}} \Delta t) \exp(-iH_{\vec{\Pi}} \Delta t)$ with error $\mathcal{O}\left(\Delta t^2\right)$.

A. Simulation of the covariance matrix

The simulation of the covariance matrix depends on the data input model and market conditions. First, if we assume Input (1), we prepare the covariance matrix as a quantum density matrix, see Eq. (14). With this density matrix, we can use quantum state exponentiation techniques [24, 31], which require the covariance matrix to be low rank. Such low-rank situations frequently occur in the financial context. Often market conditions exists where a relatively small number of underlying *factors* are driving the asset prices [29]. Such factors could be for example the interest rate policies of the Federal Reserve or the geopolitical climate. Some macroeconomic factors drive predominantly certain subsectors of the stock market. For example, the automobile or aviation sectors depend on consumer behavior, or the energy sector depends on the availability of fossil fuels, and so forth.

The strategy presented in reference [24] can be used to enact the matrix exponential of the covariance matrix. Let σ be a quantum state in an N-dimensional Hilbert space and $\rho = \Sigma/\mathrm{tr}\Sigma$ in another N-dimensional Hilbert space. The task is to perform $e^{-i\rho t}\sigma e^{i\rho t}$, i.e., to use ρ as a Hamiltonian. With the

$N^2 \times N^2$ dimensional swap matrix S we can generate

$$\mathrm{tr}_1\{e^{-iS\Delta t}\rho \otimes \sigma e^{iS\Delta t}\} = \sigma - i\Delta t[\rho, \sigma] + \mathcal{O}\left(\Delta t^2\right), \tag{25}$$

where the trace is over the ρ Hilbert space. To simulate $e^{-i\rho t}$ for a total time t and to accuracy ϵ, one needs $\mathcal{O}\left(t^2/\epsilon\right)$ copies of ρ. The rank of the effectively simulated density matrix is given by $r = \mathcal{O}\left(t^2\right)$ [31]. A controlled simulation required for phase estimation can be implemented [24, 25].

In addition, we can also efficiently simulate the covariance matrix in other regimes and different forms of data input. Let Inputs (4) and (5) be given. The elements of the covariance matrix have been measured and stored and the effective covariance matrix is sparse. In some cases, there are market conditions which correspond to such high-rank, sparse covariance matrices. An example of this case is when disjoint small subsets of stocks are driven by independent factors each. An example situation would be if pairs of stocks follow similar trajectories (one of the pairs for example being the two leading sugary beverage companies.) These situations correspond to effectively sparse, high-rank covariance matrices.

In this case, we can apply quantum simulation techniques for sparse Hamiltonians [32–34], which employ Inputs (4) and (5) and quantum walks. These techniques exhibit a query complexity of $\mathcal{O}\left(\frac{(s_\Sigma \tau)^{1+o(1)}}{\epsilon^{o(1)}}\right)$, where s_Σ is the sparsity of Σ, $\tau = \|\Sigma\|_{\max} t$ with t the simulation time and $\|\cdot\|_{\max}$ the maximum element norm, ϵ the accuracy in operator norm, and $o(1)$ denotes a quantity strictly dominated by a constant. The gate complexity of these techniques scales as $\tilde{\mathcal{O}}\left(\log N\right)$ in the dimension of the matrix.

B. Simulation of the matrices for returns and prices

The portfolio optimization involves the equality constraints, encoded in the $N + 2$ dimensional symmetric matrices containing a row and column with the N dimensional return vector and the vector of the prices. The simulation is relatively straightforward [35]. Both correspond to a "star" graph, where one site is connected to N of others with given weights. For the return matrix, the center of the star graph is site $|1\rangle$. The two nonzero eigenvalues of the embedded return matrix $H_{\vec{R}}$ are $\lambda_\pm^R = \pm\sqrt{\sum_{s=1}^N R_s^2}$ and the corresponding eigenstates are $|\lambda_\pm^R\rangle = \frac{1}{\sqrt{2}}\left(|1\rangle + \frac{1}{\lambda_\pm^R}\sum_{s=1}^N R_s|s+2\rangle\right)$. We note again that the matrix $H_{\vec{R}}$ is $N + 2$ dimensional. To enact the matrix exponential, one transforms into the eigenbasis of the weighted star, exponentiates the eigenvalues and transforms back into the computational basis [35]. The transformation into the eigenbasis is accomplished by first performing the transformation $|1\rangle \to 1/\sqrt{2}(|1\rangle + |3\rangle)$, $|3\rangle \to 1/\sqrt{2}(|1\rangle - |3\rangle)$, and the identity on the other states, corresponding to a single Hadamard operation between states $|1\rangle$ and $|3\rangle$. Then perform any transformation satisfying $|1\rangle \to |1\rangle$ and $|3\rangle \to |\vec{R}\rangle$. Thus we require the Input (3) which allows a deterministic procedure of preparing $|\vec{R}\rangle$. For the price matrix $H_{\vec{\Pi}}$,

we proceed analogously, with the center of the star graph being $|2\rangle$ instead of $|1\rangle$. We require Input (2) for performing a deterministic preparation of $|\Pi\rangle$. This step may be simplified by redefine the portfolio vector such that $\vec{1}^T\vec{w} = \xi$, which implies that only the uniform superposition has to be prepared instead of $|\Pi\rangle$.

V. WHAT TO DO WITH THE RESULTING QUANTUM STATE

Once we have prepared the optimal portfolio state $|w\rangle$, we would like to obtain classically relevant information from it. However, we would like to avoid measuring the full state, which would take $\mathcal{O}(N)$ copies and would eliminate any possibility for a $\mathcal{O}(\log N)$ performance. In the following, we describe different ways of potentially using the quantum portfolio state.

First, on the optimal portfolio state we can measure the risk $\xi = \langle w|\Sigma|w\rangle$ of the optimal portfolio. Given the density matrix Eq. (14), we can perform a swap test [20, 36] between that density matrix and $|w\rangle \equiv |w(\mu,\xi)\rangle$. This allows to map out the risk-return trade-off curve in the two-dimensional (μ,ξ) space. The accuracy is ϵ at a gate complexity of $\tilde{\mathcal{O}}\left(\log N/\epsilon^2\right)$. The swap test can also be used to compare the optimal portfolio obtained by the quantum algorithm against another portfolio vector prepared as a quantum state. Consider portfolio state $|\tilde{w}\rangle$ offered by a counterparty, say a hedge fund. We would like to make a decision to invest into this portfolio. The distance from the optimal portfolio $|w\rangle$ can be computed again via a swap test with gate complexity as above to a given accuracy ϵ. If the overlap between the portfolios is sufficiently high, one can consider the investment $|\tilde{w}\rangle$ to be sufficiently rational.

Another simple way of using the portfolio quantum state is by measuring projectors. We can measure $\langle P_{\text{sector}}|w\rangle$ with $|P_{\text{sector}}\rangle$ being a quantum state that projects into a certain subset of assets. This allows to obtain the weight of the optimal portfolio in a certain sector of the economy.

We can also take a sampling approach with respect to the portfolio state $|w\rangle$. Assume for the present discussion, we have Input (3) for the average returns R_j. In addition, assume that the portfolio state that is the output of the portfolio optimization is sparse, i.e., only $s_w = \mathcal{O}(\log N)$ elements are non-zero. Multiple samples, say $M = \mathcal{O}(\log N)$, effectively obtain another sparse portfolio $|w'\rangle$. One sample gives an index $j \in [N]$ with probability $|w_j|^2$. Multiple samples obtain the numbers M_j of occurrences of index j. The best approximation of the actual portfolio position based on the obtained information is $|w'_j| := \sqrt{M_j/M}$, where we lose the sign of the entry. The sign can be recovered by the following reasoning, which we call *long/short assumption*. If the corresponding average return R_j is positive (negative) then also the position in the optimal portfolio is positive (negative). The optimal portfolio goes long on the stocks that have a positive average return and goes short on the stocks that have a negative average return. Since we have query access to R_j including the signs we can find the sparse portfolio $w'_j = \text{sgn}(R_j)\sqrt{M_j/M}$. The estimators $|w'_j|^2$

have expectation value $|w_j|^2$ under the long/short assumption and variance $\sigma_j^2 = \frac{|w_j|^2(1-|w_j|^2)}{M}$.

Now compare the return and risk of this sampled portfolio to the optimal portfolio. Define a random variable Z to be R_j/w'_j depending on the sample outcome j, where we assume again we have query access to R_j. The difference to Tang [27] is that we do not have query access to w_j but rather have only the estimate w'_j. The expectation value is given by

$$\mathbb{E}[Z] = \sum_j |w_j|^2 \frac{R_j}{w'_j} = \sum_j |w_j|^2 |R_j| \sqrt{\frac{M}{M_j}} \xrightarrow{M_j/M \to |w_j|^2} \sum_j |w_j||R_j|. \tag{26}$$

Thus, given many samples and the correctness of the long/short assumption, the expectation value converges to the desired expected return of the optimal portfolio. As we only have a finite number of samples, we perform an error analysis. Start with the second moment

$$\mathbb{E}[Z^2] = \sum_j |w_j|^2 \frac{R_j^2}{w'^2_j} = \sum_j |w_j|^2 R_j^2 \frac{M}{M_j} \xrightarrow{M_j/M \to |w_j|^2} \sum_j R_j^2. \tag{27}$$

The variance converges to the one expected from infinite samples [27]. Using $\sum'_j$ to indicate that only j's are summed over that have $|w_j|^2 > 0$, we can analyse the finite-sample behavior as

$$\mathbb{E}[Z^2] = \sum_j |w_j|^2 R_j^2 \left| \frac{M}{M_j} - \frac{1}{|w_j|^2} + \frac{1}{|w_j|^2} \right| \leq \sum_j R_j^2 + \sum'_j |w_j|^2 R_j^2 \left| \frac{M}{M_j} - \frac{1}{|w_j|^2} \right| \tag{28}$$

$$= \sum_j R_j^2 + \mathcal{O}\left(\sum'_j R_j^2 \frac{\sigma_j}{|w_j|^2} \right). \tag{29}$$

Thus, we have an additional variance of the return proportional to $\sum'_j R_j^2 \sqrt{\frac{(1-|w_j|^2)}{M|w_j|^2}}$. This variance behaves as expected considering that $|w_j|^2 = \mathcal{O}(1/s_w)$ because of the sparsity assumption. We have used that if $|x - x'| \leq \epsilon$ then $|\frac{1}{x} - \frac{1}{x'}| = \mathcal{O}\left(\frac{\epsilon}{x^2}\right)$.

Now consider the risk of the sampled portfolio. First, note that the covariance matrix Σ is positive semi-definite and the minimized risk $\langle w|\Sigma|w\rangle$ is always smaller than the sampled one $\langle w'|\Sigma|w'\rangle$. In addition, we show the following simple bound. Let $||w\rangle - |w'\rangle| \leq \epsilon_w$. Then, $|\langle w|\Sigma|w\rangle - \langle w'|\Sigma|w'\rangle| = \mathcal{O}(\epsilon_w)$. This follows from

$$|\langle w|\Sigma|w\rangle - \langle w'|\Sigma|w'\rangle| = |\langle w|\Sigma|w\rangle - \langle w|\Sigma|w'\rangle + \langle w|\Sigma|w'\rangle - \langle w'|\Sigma|w'\rangle| \tag{30}$$

$$\leq |\langle w|\Sigma|w\rangle - \langle w|\Sigma|w'\rangle| + |\langle w|\Sigma|w'\rangle - \langle w'|\Sigma|w'\rangle| \leq 2\|\Sigma\|\epsilon_w. \tag{31}$$

Moreover, we know that the variance of M_j/M is $|w_j|^2(1-|w_j|^2)/M$, thus the l_2 norm of the difference $||w\rangle - |w'\rangle|$ is bounded as $\epsilon_w = \mathcal{O}\left(\sqrt{s_w |w_{\max}|^2 (1-|w_{\max}|^2)/M}\right)$. If the probabilities are evenly distributed, $|w_j|^2 = \mathcal{O}(1/s_w)$, then $\epsilon_w = \mathcal{O}\left(1/\sqrt{M}\right)$. Thus, the risk of the sampled portfolio has two contributions: the original risk of the optimal portfolio and the additional risk from the finite sampling. The finite sampling risk goes as $\mathcal{O}\left(1/\sqrt{M}\right)$.

VI. DISCUSSION AND CONCLUSION

Quantum matrix inversion schematically prepares a state $|x\rangle = A^{-1}|b\rangle$ on which measurements $\langle x|O|x\rangle$ can be performed. Measuring all the elements of $|x\rangle$ itself would remove the possibility of an exponential speedup. It is thus crucial to find good measurement operators O which give results that are hard to obtain otherwise and at the same time that give new insights on the state $|x\rangle$. Measuring the risk of an optimal portfolio and the binary decision to invest in an offered portfolio provide classical information that relies on global properties of the state. The full set of interesting measurements on the portfolio vector is a topic for further research.

In principle, the algorithms presented here can be efficient in the sense that they have a run time proportional to the number of qubits involved in the computations, i.e., the run time is $\mathcal{O}(\log N)$. Hardware requirements are $\mathcal{O}(N)$ to store and access the data. The conditions for such a logarithmic run time is that the input can be prepared efficiently and the criteria of quantum matrix inversion are satisfied [2, 37]. Classical Monte Carlo methods can be efficient, often closing potential exponential separations between quantum algorithms and naive classical methods. In particular, classical methods prove powerful when certain assumptions can be made on the preprocessing of the input data, especially for low-rank situations [27, 28, 38], where the singular value decomposition can be constructed from a polylogarithmic number of samples. Such classical methods allow the performance of matrix completion algorithms [27] and principal component analysis [28] in polylogarithmic time, and could be applied to the low-rank matrix pseudo-inverse as well. Exponential separations between a quantum algorithm and sampling algorithms have been shown for example for oracle promise problems involving the evaluation of inner products of vectors processed via Fourier transforms [39]. Sparse matrix inversion also shows the potential for exponential speedups, evidenced indirectly via encoding a universal quantum computation into a matrix inversion [2]. Classical sampling is hard, i.e, $\Omega(N)$, for example for obtaining a low-rank matrix approximation when no prior information is given about the matrix [40].

While exponential speedups are the most compelling, quantum computing can provide polynomial speedups in rather generic settings, for example for search [41], Monte Carlo [42, 43], semi-definite programming [44, 45], and generalized convex optimization [46, 47]. The results of [27, 28] also allow for polynomial speedups for the discussed algorithms. It is thus believable that such speedups can be obtained for financial applications such as derivative pricing [13] or the present work of portfolio optimization. The financial context is particularly sensitive to speedups in terms of the potential rewards. Small improvements in speed for particular financial calculations can have a large impact in terms of financial reward and thus motivate further study of the intersection of quantum computing and finance.

Acknowledgments

P. R. acknowledges support from Singapores Ministry of Education and National Research Foundation. S. L. was supported by ARO.

Appendix A: Preparing the covariance matrix

The covariance matrix is defined over the mean-adjusted returns. Given the oracle in Eq. (6), we show here how to construct a quantum state that contains the mean-adjusted returns in amplitude encoding. The notation omits ancilla qubits in the $|0\rangle$ state. We proceed as

$$|0\dots0\rangle \;\rightarrow\; \frac{1}{\sqrt{2TN}}\sum_{t,s}|t\rangle|s\rangle(|0\rangle_a+|1\rangle_a) \tag{A1}$$

$$\text{(controlled Hadamards)} \;\rightarrow\; \frac{1}{\sqrt{2TN}}\sum_{t,s}|t\rangle|s\rangle\left(|0\rangle_a|0\rangle_b+\frac{1}{\sqrt{T}}|1\rangle_a\sum_{t'}|t'\rangle_b\right) \tag{A2}$$

$$\text{(oracle query)} \;\rightarrow\; \frac{1}{\sqrt{2TN}}\sum_{t,s}|t\rangle|s\rangle\left(|0\rangle_a|0\rangle_b|0\rangle_c+\frac{1}{\sqrt{T}}|1\rangle_a\sum_{t'}|t'\rangle_b|y_s(t')\rangle_c\right) \tag{A3}$$

$$\text{(controlled rotation)} \;\rightarrow\; \frac{1}{\sqrt{2TN}}\sum_{t,s}|t\rangle|s\rangle\,(|0\rangle_a|0\rangle_b|0\rangle_c|1\rangle_d+ \tag{A4}$$

$$\frac{1}{\sqrt{T}}|1\rangle_a\sum_{t'}|t'\rangle_b|y_s(t')\rangle_c\left(\sqrt{1-\delta^2y_s(t')^2}|0\rangle_d+\delta y_s(t')|1\rangle_d\right)\Bigg) \tag{A5}$$

$$\text{(uncompute)} \;\rightarrow\; \frac{1}{\sqrt{2TN}}\sum_{t,s}|t\rangle|s\rangle\,(|0\rangle_a|0\rangle_b|1\rangle_d+ \tag{A6}$$

$$\frac{1}{\sqrt{T}}|1\rangle_a\sum_{t'}|t'\rangle_b\left(\sqrt{1-\delta^2y_s(t')^2}|0\rangle_d+\delta y_s(t')|1\rangle_d\right)\Bigg) \tag{A7}$$

$$\text{(controlled Hadamards)} \;\rightarrow\; \frac{1}{\sqrt{2TN}}\sum_{t,s}|t\rangle|s\rangle\,(|0\rangle_a|0\rangle_b|1\rangle_d+ \tag{A8}$$

$$\frac{1}{T}|1\rangle_a\sum_{t't''}(-1)^{t'\cdot t''}|t''\rangle_b\left(\sqrt{1-\delta^2y_s(t')^2}|0\rangle_d+\delta y_s(t')|1\rangle_d\right)\Bigg). \tag{A9}$$

Rotate another ancilla, with $0<\delta\le\max_{s,t}|y_s(t)|$,

$$\rightarrow\; \frac{1}{\sqrt{2TN}}\sum_{t,s}|t\rangle|s\rangle\left(|0\rangle_a|0\rangle_b|1\rangle_d\left(\sqrt{1-\delta^2y_s(t)^2}|0\rangle_e+\delta y_s(t)|1\rangle_e\right)+\right. \tag{A10}$$

$$\left.\frac{1}{T}|1\rangle_a\sum_{t't''}(-1)^{t'\cdot t''}|t''\rangle_b\left(\sqrt{1-\delta^2y_s(t')^2}|0\rangle_d+\delta y_s(t')|1\rangle_d\right)|1\rangle_e\right). \tag{A11}$$

Postselect on the state

$$\frac{1}{\sqrt{2}}\left(|0\rangle_a + |1\rangle_a\right)|0\rangle_b|1\rangle_d|1\rangle_e. \tag{A12}$$

The postselection obtains

$$\rightarrow \frac{1}{|\tilde{y}|}\sum_{t,s}|t\rangle|s\rangle\left(y_s(t) - \frac{1}{T}\sum_{t'}y_s(t')\right) := |\tilde{\chi}\rangle. \tag{A13}$$

with $|\tilde{y}|^2 = \sum_{t,s}\left(y_s(t) - \frac{1}{T}\sum_{t'}y_s(t')\right)^2$. The success probability is

$$P_{\tilde{\chi}} = \frac{\delta^2|\tilde{y}|^2}{4TN}. \tag{A14}$$

If most of the returns are sufficiently large, $y_s(t) = \Theta(1)$, and also most $y_s(t) - \frac{1}{T}\sum_{t'}y_s(t') = \Theta(1)$, then the success probability is $P_{\tilde{\chi}} = \Omega(1)$. Note that $P_{\tilde{\chi}}$ encodes the trace of the sample covariance matrix Eq. (5) via

$$P_{\tilde{\chi}} = \frac{\delta^2(T-1)\mathrm{tr}\Sigma}{4TN}. \tag{A15}$$

By repeating the above state preparation and postselection, this success probability can be determined and the trace $\mathrm{tr}\Sigma$ can be estimated.

---

[1] M. S. Nielsen and I. Chuang, *Quantum computation and quantum information* (Cambridge University Press, 2000).

[2] A. W. Harrow, A. Hassidim, and S. Lloyd, Phys. Rev. Lett. **103**, 150502 (2009).

[3] A. Childs, R. Kothari, and R. Somma, SIAM Journal on Computing **46**, 1920 (2017).

[4] N. Wiebe, D. Braun, and S. Lloyd, Phys. Rev. Lett. **109**, 050505 (2012).

[5] P. Rebentrost, M. Mohseni, and S. Lloyd, Physical Review Letters **113**, 130503 (2014).

[6] S. Aaronson, Nature Physics **11**, 291 (2015).

[7] P. Glasserman, *Monte Carlo Methods in Financial Engineering* (Springer-Verlag, 2003).

[8] H. Föllmer and A. Schied, *Stochastic Finance: An Introduction in Discrete Time* (Walter de Gruyter, 2004).

[9] J. C. Hull, *Options, futures, and other derivatives* (Prentice Hall, 2012).

[10] A. Green, *XVA: Credit, Funding and Capital Valuation Adjustments* (John Wiley & Sons, 2015).

[11] B. E. Baaquie, *Quantum finance* (Cambridge University Press, 2004).

[12] E. E. Haven, Physica A: Statistical Mechanics and its Applications **304**, 507 (2002).

[13] P. Rebentrost, B. Gupt, and T. R. Bromley, Phys. Rev. A **98**, 022321 (2018).

[14] S. Woerner and D. J. Egger, arXiv:1806.06893 (2018).

[15] G. Rosenberg, *Finding optimal arbitrage opportunities using a quantum annealer* (2016), 1QBit white paper: `https://1qbit.com/whitepaper/arbitrage/`.

[16] M. Marzec, *Portfolio Optimization: Applications in Quantum Computing* (Wiley-Blackwell, 2016), chap. 4, pp. 73–106.

[17] D. Venturelli and A. Kondratyev, arXiv:1810.08584 (2018).

[18] H. Markovitz, The Journal of Finance **7**, 77 (1952).

[19] Wikipedia, *Efficient Frontier* (2018), `https://en.wikipedia.org/wiki/Efficient_frontier`.

[20] S. Lloyd, M. Mohseni, and P. Rebentrost, arXiv:1307.0411 (2013).

[21] V. Giovannetti, S. Lloyd, and L. Maccone, Phys. Rev. Lett. **100**, 160501 (2008).

[22] V. Giovannetti, S. Lloyd, and L. Maccone, Phys. Rev. A **78**, 052310 (2008).

[23] F. D. Martini, V. Giovannetti, S. Lloyd, L. Maccone, E. Nagali, L. Sansoni, and F. Sciarrino, Phys. Rev. A **80**, 010302 (2009).

[24] S. Lloyd, M. Mohseni, and P. Rebentrost, Nature Physics **10**, 631 (2014).

[25] S. Kimmel, C. Y.-Y. Lin, G. H. Low, M. Ozols, and T. J. Yoder, npj Quantum Information **3**, 13 (2017).

[26] I. Kerenidis and A. Prakash, in *8th Innovations in Theoretical Computer Science Conference (ITCS 2017)*, edited by C. H. Papadimitriou (Schloss Dagstuhl, Dagstuhl, Germany, 2017), vol. 67 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pp. 49:1–49:21.

[27] E. Tang, Electronic Colloquium on Computational Complexity **128** (2018).

[28] E. Tang, arXiv:1811.00414 (2018).

[29] J. Bai, Econometrica **71**, 135 (2003).

[30] L. Grover and T. Rudolph, arXiv:quant-ph/0208112 (2002).

[31] P. Rebentrost, A. Steffens, I. Marvian, and S. Lloyd, Physical Review A **97**, 012327 (2018).

[32] D. W. Berry and A. M. Childs, Quantum Info. Comput. **12**, 29 (2012), ISSN 1533-7146.

[33] D. W. Berry, A. M. Childs, and R. Kothari, Foundations of Computer Science (FOCS), 2015 IEEE 56th Annual Symposium on pp. 792–809 (2015).

[34] G. H. Low and I. L. Chuang, Physical Review Letters **118**, 010501 (2017).

[35] A. Childs, Comm. Math. Phys. **294**, 581 (2010).

[36] H. Buhrman, R. Cleve, J. Watrous, and R. de Wolf, Phys. Rev. Lett. **87**, 167902 (2001).

[37] S. Aaronson, Nature Physics **11**, 291 (2015).

[38] A. Frieze, R. Kannan, and S. Vempala, J. ACM **51**, 1025 (2004).

[39] S. Aaronson, arXiv:0910.4698 (2009).

[40] Z. Bar-Yossef, in *Proc. 35th Annual ACM Symp. on Theory of Computing* (2003).

[41] L. K. Grover, in *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing* (ACM, 1996), pp. 212–219.

[42] M. Szegedy, in *FOCS 04 Proceedings of the 45th Annual IEEE Symposium on Foundations of Computer Science* (IEEE Computer Soc., Washington, D.C., 2004), pp. 32–41.

[43] A. Montanaro, Proc. R. Soc. A **471**, 0301 (2015).

[44] F. G. S. L. Brandao and K. Svore, in *FOCS 17 Proceedings of the 45th Annual IEEE Symposium on Foundations of Computer Science* (IEEE Computer Soc., Washington, D.C., 2017).

[45] J. van Apeldoorn, A. Gilyén, S. Gribling, and R. de Wolf, arXiv:1705.01843 (2017).

[46] J. van Apeldoorn, A. Gilyén, S. Gribling, and R. de Wolf, arXiv:1809.00643 (2018).

[47] S. Chakrabarti, A. M. Childs, T. Li, and X. Wu, arXiv:1809.01731 (2018).

Low overhead quantum computation using lattice surgery

Austin G. Fowler, Craig Gidney
Google Inc., Santa Barbara, California 93117, USA
(Dated: September 4, 2019)

When calculating the overhead of a quantum algorithm made fault-tolerant using the surface code, many previous works have used defects and braids for logical qubit storage and state distillation. In this work, we show that lattice surgery reduces the storage overhead by over a factor of 4, and the distillation overhead by nearly a factor of 5, making it possible to run algorithms with 10^8 T gates using only 3.7×10^5 physical qubits capable of executing gates with error $p \sim 10^{-3}$. These numbers strongly suggest that defects and braids in the surface code should be deprecated in favor of lattice surgery.

arXiv:1808.06709v4 [quant-ph] 30 Aug 2019

I. INTRODUCTION

In [1], we contributed overhead calculations based on the surface code [2, 3] using defects and braiding [4–6] showing that non-trivial post-classical computations could be performed in hours using of order a million superconducting qubits. Here, we shall focus on lattice surgery, systematically building up the techniques required to implement an arbitrary algorithm and calculate the precise time and space overhead. We find that general algorithms can be implemented with nearly a factor of 5 fewer qubits and comparable time using lattice surgery. The results here were prepared in parallel with and are consistent with the recent work [7].

II. STORAGE

A double-defect logical qubit (Fig. 1) consists of two holes in a large non-rotated surface code such that each hole's circumference is no less than the code distance d, and the holes are also separated by at least d. In a simple regular packing, the kind of packing that has been used in many resource estimates, $12.5d^2$ physical qubits are required per logical qubit.

Lattice surgery [8], makes use of rotated logical qubits (Fig. 2a) consisting of d^2 data qubits and $d^2 - 1$ measure qubits. In this work, we shall arrange the rotated logical qubits being used in an algorithm in patterns similar to Fig. 2b. Such arrangements enable logical qubits to be both acted on locally and easily moved to a shared workspace if required for multi-logical-qubit operations. To leading order, $3d^2$ physical qubits are required per logical qubit, better than a factor of 4 overhead saving.

III. LOGICAL INITIALIZATION AND MEASUREMENT

Logical $|0\rangle/|+\rangle$, namely $|0_L\rangle/|+_L\rangle$, can be prepared as shown in Fig. 3. After preparation, logical operators can be moved around using products of stabilizers, for example as in Fig. 4. Logical Z or X measurements can be achieved by measuring each data qubit in the Z or X basis and, after correction, taking the product of these ± 1 values along the indicated lines, with final modification by any sign accumulated through operator movement or manipulation to obtain the actual result.

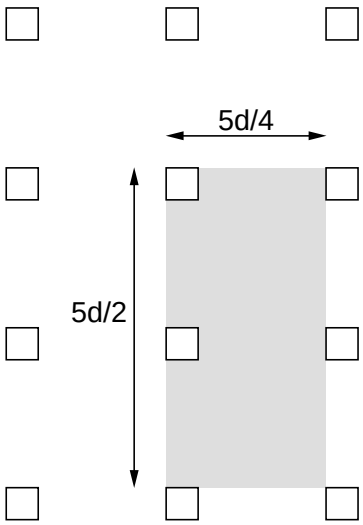


FIG. 1. In terms of the code distance d, a double-defect logical qubit occupies $25d^2/8$ space to leading order. Each unit of d represents two qubits, a data and measure qubit, so to leading order $12.5d^2$ physical qubits are required.

IV. XX AND ZZ LOGICAL MEASUREMENT

Procedures for measuring logical XX and ZZ given qubits of equal size are shown in Figs. 5–6. All described actions with all stabilizer and individual qubit measurements occur only post error correction. It is also useful to know how to measure such operators given qubits of unequal size, and an example of an unequal size ZZ logical measurement can be found in Fig. 7.

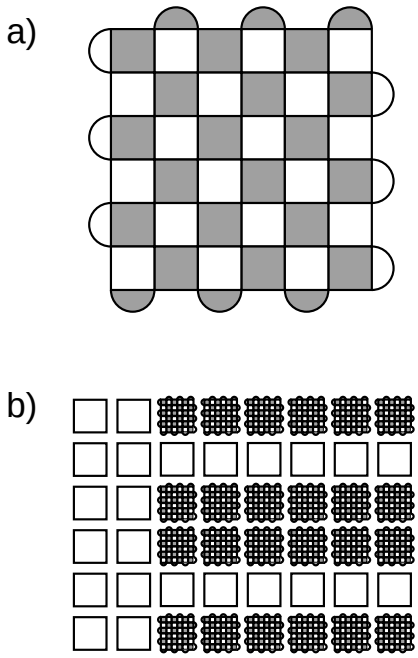


FIG. 2. a) Distance $d = 7$ rotated logical qubit. Dark regions represent X stabilizers, light regions represent Z stabilizers. Each region is associated with a measurement qubit. A data qubit is located at each intersection point of dark lines. b) Proposed layout of rotated logical qubits permitting local operations in parallel and easy movement of collections of logical qubits to the workspace on the left where multi-logical-qubit operations can take place.

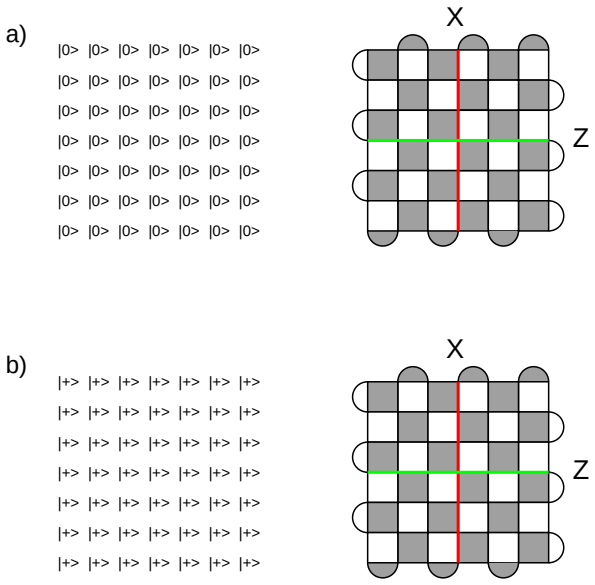


FIG. 3. a) Preparation of $|0_L\rangle$. All data qubits are initialized to $|0\rangle$ then all stabilizers measured the code distance d times. b) Preparation of $|+_L\rangle$. All data qubits are initialized to $|+\rangle$ then all stabilizers measured the code distance d times.

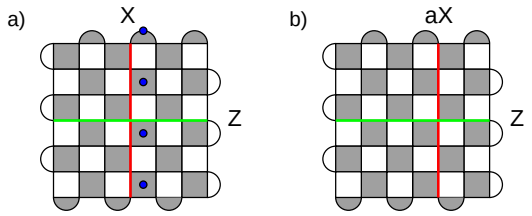


FIG. 4. a) Before movement. The product of the ± 1 measurement results of the stabilizers marked by blue circles shall be denoted a. b) After movement, the new logical operator is related to the old by a.

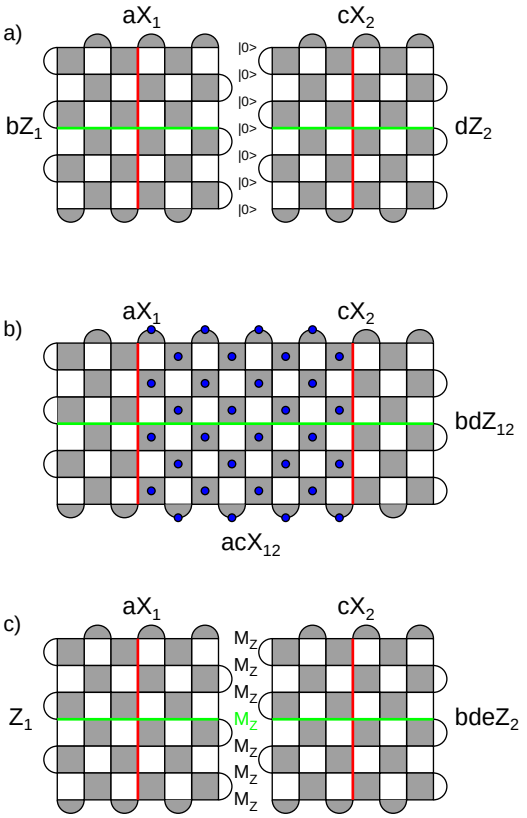


FIG. 5. a) Getting ready to measure the logical operator $X_{12} = X_1X_2$. b) The product of the stabilizers marked by blue circles gives us the eigenvalue of the tensor product of X operators along the red lines. This must then be modified by the current signs of tensor products of operators along these lines to give the actual desired result. This pattern of stabilizers is measured d times. c) After splitting, the eigenvalue of the measurement in green is denoted by e, and this and the sign of the Z_{12} operator can be associated with either Z_1 or Z_2.

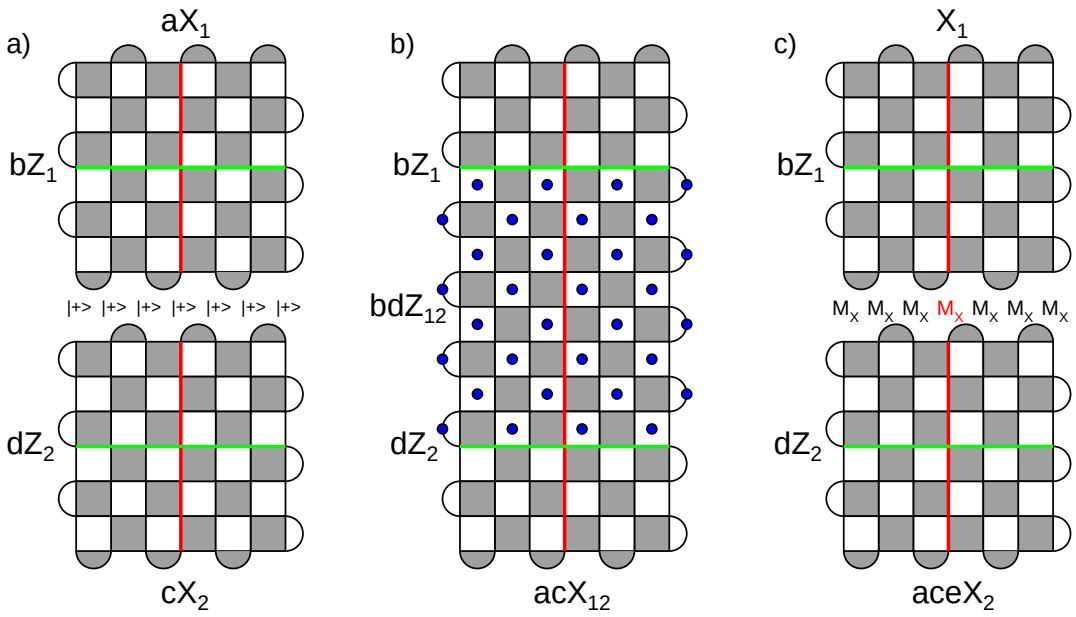


FIG. 6. a) Getting ready to measure the logical operator $Z_{12} = Z_1 Z_2$. b) The product of the stabilizers marked by blue circles gives us the eigenvalue of the tensor product of Z operators along the green lines. This must then be modified by the current signs of tensor products of operators along these lines to give the actual desired result. This pattern of stabilizers is measured d times. c) After splitting, the eigenvalue of the measurement in red is denoted by e, and this and the sign of the X_{12} operator can be associated with either X_1 or X_2.

V. MULTI-BODY OPERATOR MEASUREMENTS

A powerful feature of lattice surgery is the ability to compactly measure multi-body X and Z operators. The explicit steps to perform this for a multi-body X measurement are shown in Fig. 8. Fig. 9 shows the procedure for logical XXZ. When measuring multi-body mixed X and Z operators, it can be necessary to rotate logical qubits.

VI. STATE INJECTION

The output probability of error p_o of 15-1 state distillation [9, 10] depends on the injected state error probability p_i as $p_o = 35p_i^3$. Using the state injection techniques of [11], it is seems possible with approximately 50% heralded chance of success to create a distance 7 rotated logical qubit in an arbitrary state with probability of error on acceptance p_i equal to the physical 2-qubit gate error rate p_2, however note that the simulations in [11] were performed using un-rotated square logical qubits. We are interested in injecting the state $|T\rangle = (|0\rangle + e^{i\pi/4}|1\rangle)/\sqrt{2}$. The injection process itself takes two rounds of surface code error detection. We assume it takes an amount of time equal to one further round to determine whether the injection was successful. We shall base our construction on clusters of 20 injection attempts (Fig. 10), ensuring a heralded probability of failure of each cluster of order 10^{-6}. When a state is injected successfully, it is expanded to distance 15 (Fig. 11).

VII. THE S/S† GATE

We will choose to implement these gates using the method shown in Fig. 7 of [12]. We will assume that given a $d = 15$ surface code it is possible in 15 error detection cycles to implement S/S† with distance no less that 7. Note that implementing this gate in this manner typically involves measuring some stabilizers of extended size, irregular shape, and increased weight.

VIII. THE T/T† GATE

Fig. 12 shows how this can be build out of circuit elements already described.

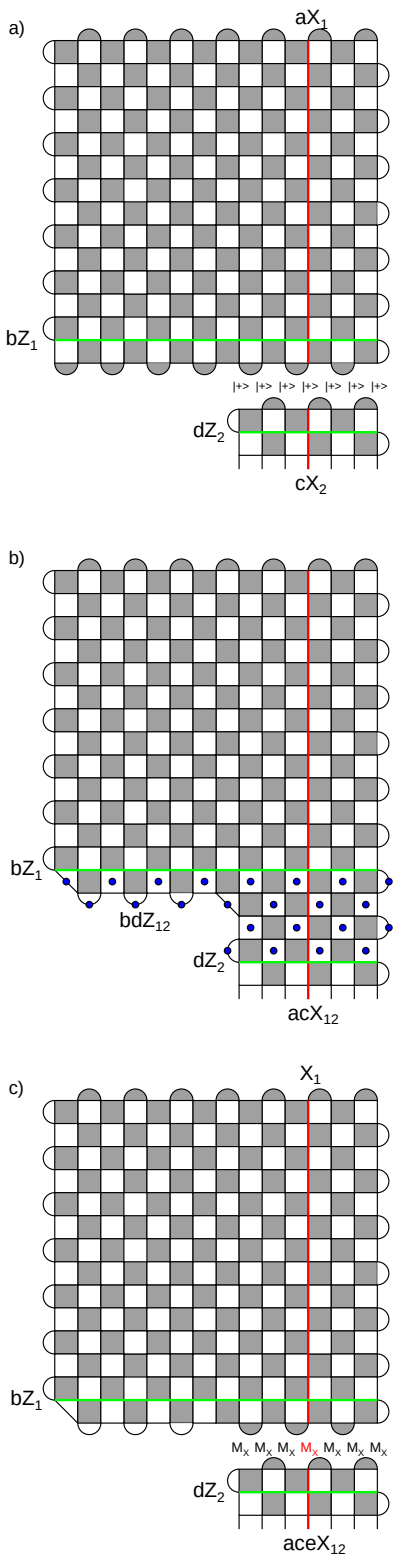


FIG. 7. Analogue of Fig. 6, namely logical ZZ measurement, given qubits of unequal size.

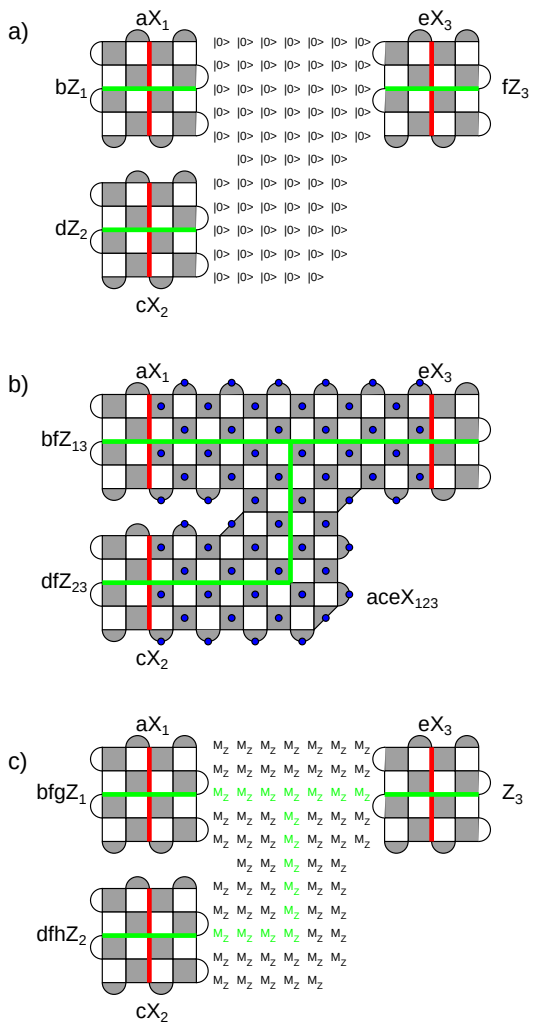


FIG. 8. a) Getting ready to measure the logical operator $X_{123} = X_1X_2X_3$. b) The product of the stabilizers marked by blue circles gives us the eigenvalue of the tensor product of X operators along the red lines. This must then be modified by the current signs of tensor products of operators along these lines to give the actual desired result. This pattern of stabilizers is measured d times. c) After splitting, the eigenvalue of the chain of measurements in green connecting Z_1 to Z_3 is denoted by g, and the chain connecting Z_2 to Z_3 by h. The signs of the Z_{13} and Z_{23} operators are most conveniently associated with Z_1 and Z_2.

IX. HALF DISTANCE ROTATION

It is frequently useful to be able to rotate square logical qubits by 90°. When space is limited and this rotation must be done in place, the code distance will in general be reduced. In Figs. 13–14, we describe a simple rotation method that halves the code distance but will be sufficient for our purposes.

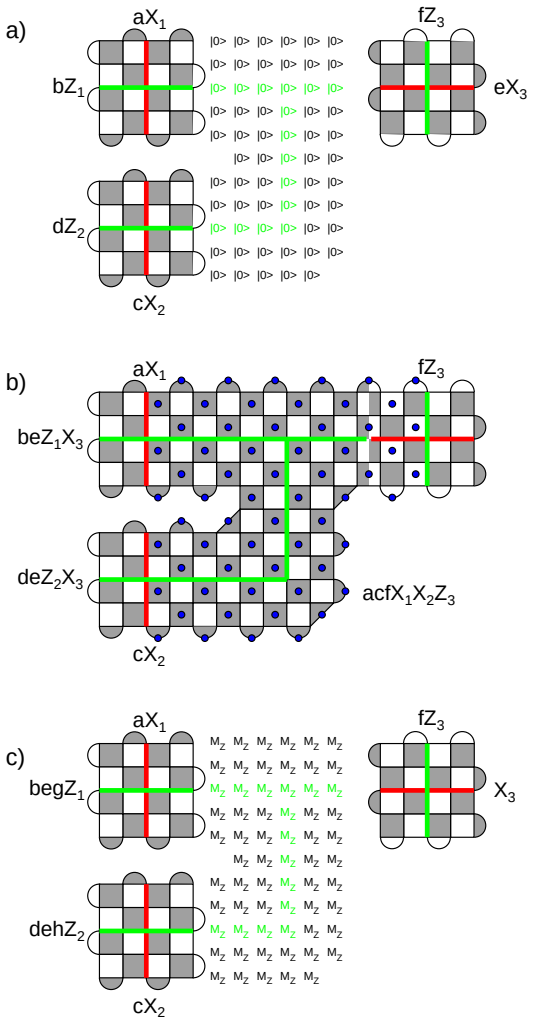


FIG. 9. a) Getting ready to measure the logical operator $X_1X_2Z_3$. b) The product of the stabilizers marked by blue circles gives us the eigenvalue of the tensor product $X_1X_2Z_3$. This must then be modified by the current signs of these indivdul operators to get the actual desired result. This pattern of stabilizers is measured d times. c) After splitting, the eigenvalue of the chain of measurements in green connecting Z_1 to X_3 is denoted by g, and the chain connecting Z_2 to X_3 by h. The signs of the Z_1X_3 and Z_2X_3 operators are most conveniently associated with Z_1 and Z_2.

X. DISTILLATION

Consider Fig. 15a. The first 4 columns of CNOTs prepare a superposition of computational basis states, each of which has either 8 or no 1s in it. Looked at another way, each column of CNOTs modifies the initial state such that it has an additional stabilizer generator corresponding to X on the control and every target.

These 8-body X stabilizer generators are constructed such that the various patterns of Xs that touch the top

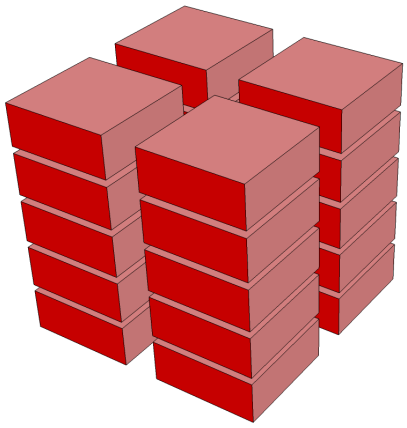


FIG. 10. In this figure, and all subsequent 3D figures, time runs vertically. Each red cuboid is an attempt to inject the state $|T\rangle = (|0\rangle + e^{i\pi/4}|1\rangle)/\sqrt{2}$ into a distance 7 code. In 15 rounds of a distance 15 code, there is space for 20 injection attempts. When an injection is successful, which occurs 50% of the time [11], no further injection attempts are made and the injected logical qubit is expanded to distance 15 (Fig. 11) for more reliable storage.

15 qubits uniquely correspond to every binary number from 1 to 15. This enables a single Z error after state preparation to be located, and arbitrary pairs of Z errors to be detected. This state can be considered logical $|0\rangle$ of a distance 3 code.

The 5th and final column of CNOTs corresponds to the controlled application of logical X, meaning the superposition $(|0_L\rangle + |1_L\rangle)/\sqrt{2}$ is prepared. To be more precise, given the 16th qubit, $(|0_L\rangle|0\rangle + |1_L\rangle|1\rangle)/\sqrt{2}$ is prepared. States in $|1_L\rangle$ contain 15 or 7 1s.

The number of 1s in $|0_L\rangle$ and $|1_L\rangle$ implies that applying transversal $T^\dagger$ will result in $(|0_L\rangle|0\rangle + e^{i\pi/4}|1_L\rangle|1\rangle)/\sqrt{2}$. Z errors during the $T^\dagger$ gates can then be detected by measuring each qubit in the X basis, as the eigenvalues of the 4 X stabilizer generators can be reconstructed by taking the appropriate product of X measurements. If any generator has a negative eigenvalue, the 16th qubit is discarded.

If the probability of a Z error during $T^\dagger$ is p, the probability of rejection is approximately $15p$. The collective effect of the X measurements is a logical X measurement, which cuts the output down to $(|T\rangle = (|0\rangle + e^{i\pi/4}|1\rangle)/\sqrt{2}$ up to a possible Z correction. 35 combinations of 3 Z errors are undetectable and lead to an erroneous output of $Z|T\rangle$, hence the input-output error relationship $p_o = 35p_i^3$ already mentioned.

The 5th and final column of CNOTs is structurally identical to the first 4 columns, meaning it can be considered the 5th stabilizer generator of a 16 qubit code. This suggests an efficient method of preparing this state using lattice surgery, namely the direct measurement of the 5 stabilizer generators. Fig. 16 explicitly illustrates

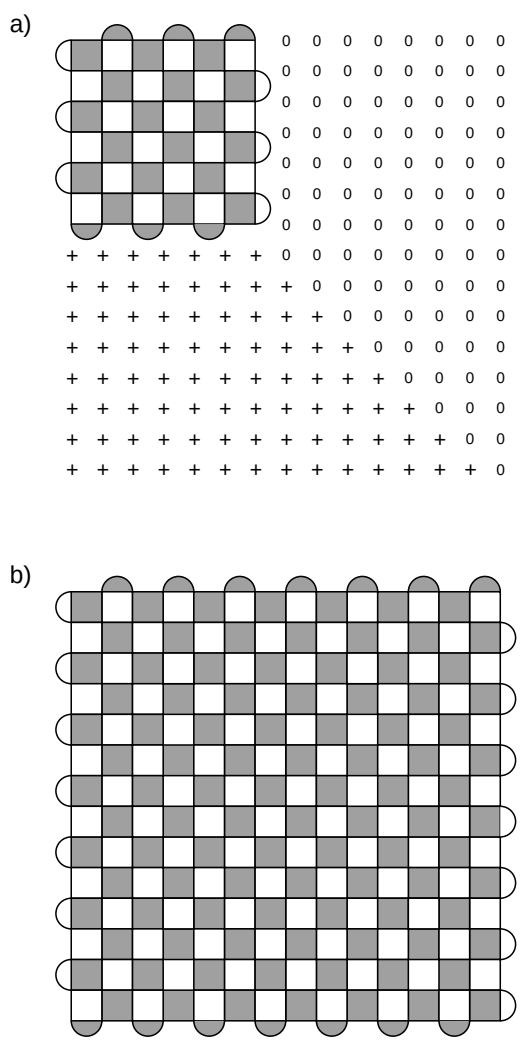


FIG. 11. a) Getting ready to expand a distance 7 logical qubit to distance 15. Initialization to $|0\rangle$ and $|+\rangle$ is denoted by 0 and + for visual simplicity. b) After initialization, the complete pattern of stabilizers is measured. The full benefit of the increased code size will not be reached until the complete pattern has been measured 15 times ensuring that the new stabilizers are known with distance 15 confidence, however the single-round logical error rate will be strictly lower even during the first complete error detection round.

such measurements.

Putting it all together, after preparing the 16 qubit state, 15 of these qubits get half-distance rotated and ZZ measured with the expanded output of multi-attempt state injection as shown in Fig. 17.

If we assume that the probability of a Z error during the complete $T^\dagger$ structure is $p_i = 10^{-3}$, the output probability of error will be $p_o = 3.5 \times 10^{-8}$. This might be sufficient for medium-term algorithms, however further error suppression is likely necessary for long-term algorithms.

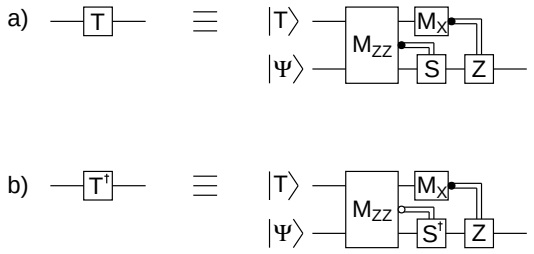


FIG. 12. a) T gate and b) $T^\dagger$ built out of lattice-surgery structures. Solid dots represent classically controlled execution if the associated measurement is 1 (-1 eigenstate), the open dot is for 0 (+1 eigenstate) triggered execution.

If we wish to repeat state distillation, we need to attempt to prepare at least 16 first-level $|T\rangle$ states to reliably get 15 inputs given the $15p_i$ failure rate. 8 packed level one distillations are shown in Fig. 18. The effective height of each layer of the structure is $6.5d$, where d is explicitly fixed at 15.

For the second level of distillation, we will use the two circuits of Fig.12b)–c). These circuits have been designed such that the last round of CNOTs of b) slots into the first round of CNOTs of c), and the last round of CNOTS of c) slots into the first round of CNOTs of b). This property ensures that the lattice surgery versions of these circuits (Fig. 19) fit snuggly together, reducing the overall height and therefore time of execution (Fig. 20).

XI. HADAMARD

The logical Hadamard gate, being a Clifford gate, can in principle be performed entirely in classical processing, by simply relabeling the logical X and Z operators [13]. When using lattice surgery, this comes at the cost of occasionally needing to double the size of the logical qubit, and being able to measure logical Y operators, which doubles the size again. Furthermore, physical-level stabilizer measurements of weight greater than 4 and structure that may or may not be suitable for the underlying hardware are required.

We are therefore motivated to present a method of performing Hadamard that does not change the definitions of the logical operators and uses simple underlying circuits. Our proposal can be found in Figs. 21–22.

XII. CNOT

CNOT is an extremely common gate in many quantum algorithms. Regular CNOT and single-control multiple-target CNOT can be implemented in similar manners (Fig. 23). Note that the total time required is $2d$ as

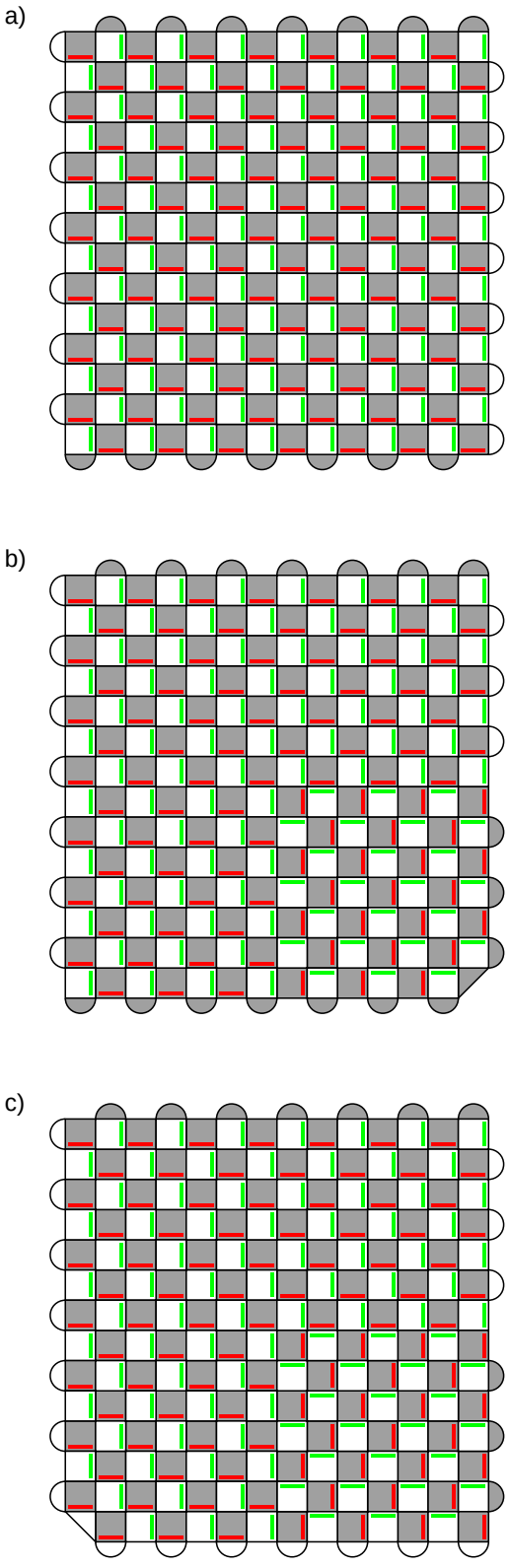


FIG. 13. a) Initial distance 15 surface code marked up to show the last 2 qubits interacted with when measuring each stabilizer. b) Modification of the boundary stabilizers and interaction pattern, maintained for 7 rounds. c) Further boundary modification leading to logical Z being supported on the lower half of the right boundary, maintained for 7 rounds. This is then followed by 7 rounds of b), then back to the a) pattern.

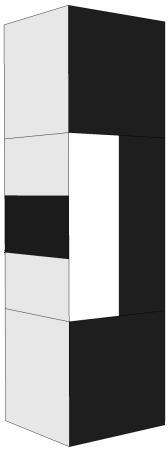


FIG. 14. 3D version of Fig. 13 showing an additional 15 rounds of standard distance 15 both before and after the half rotation. Light shading represents where logical Z is supported, dark shading represents where logical X is supported.

the ancilla initialization and measurement can both be performed transversely.

XIII. CZ

The CNOT of the previous section can be modified to create a CZ gate simply by changing one of the operators being measured (Fig. 24).

XIV. SWAP

Logical qubits can be moved around one another using a series of single logical qubit moves (Fig. 25).

XV. ALGORITHM OVERHEAD

In keeping with [1], we shall focus on the situation where you only distill one $|T\rangle$ state at a time (Fig. 26). To ensure a high chance of keeping pace with a superconducting quantum computer, we shall process surface code detection events using the simple techniques of [14] rather than the more computationally expensive and better logical error suppressing techniques of [15]. This leads to an approximate logical error rate per round of error detection of $p_L = 0.1(100p)^{(d+1)/2}$.

Calculating the overhead of a Clifford+T algorithm is then straightforward, if you can approximate the algorithm as being execution-time dominated by the preparation of $|T\rangle$ states, meaning few sections of many Clifford gates, meaning the Clifford gates in the algorithm can be performed in parallel with the $|T\rangle$ preparations.

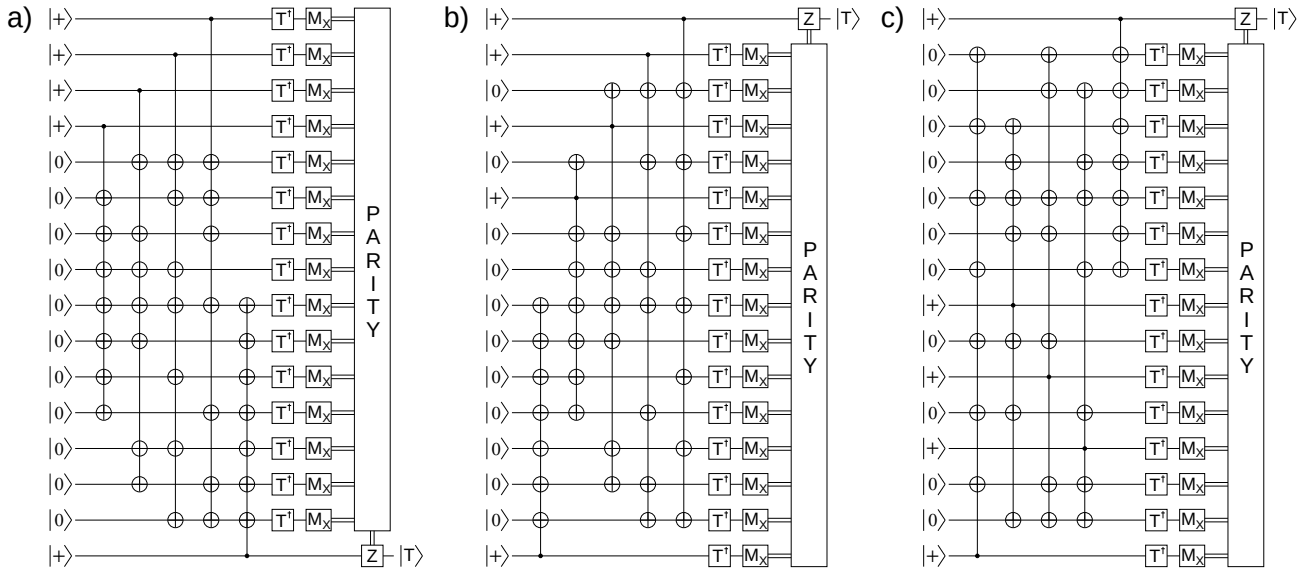


FIG. 15. Three versions of the state distillation circuit, identical up to qubit permutation.

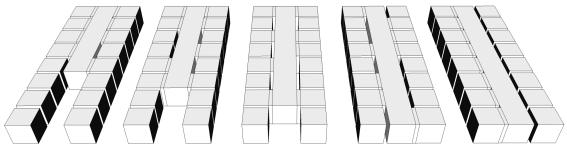


FIG. 16. 5 structures corresponding to the 5 columns of CNOTs in Fig. 15a. Instead of implementing CNOTs, the same state can be prepared with multi-body X measurements. In each structure the left 8 blocks correspond to the lower 8 qubits, and the right 8 blocks to the upper 8 qubits (in reverse order so the top qubit is the bottom right block). The element in the center of each structure performs the appropriate multi-body X measurement.

We include a spreadsheet both describing in detail and performing the necessary calculations.

XVI. CONCLUSION

Using the defect and braiding overhead estimation techniques of [1] an algorithm involving 10^8 T gates and 100 logical qubits, on hardware with a characteristic gate error rate of 10^{-3} and surface code error detection circuit time of 1μs takes 4.5 hours and requires 1.8×10^6 physical qubits. By contrast, using lattice surgery techniques as described in this paper and the spreadsheet, the same algorithm would run in 5.4 hours, so comparable, but require just 3.7×10^5 physical qubits. This near factor of 5 qubit saving while maintaining comparable runtime implies defects and braiding should be deprecated in favor of lattice surgery.

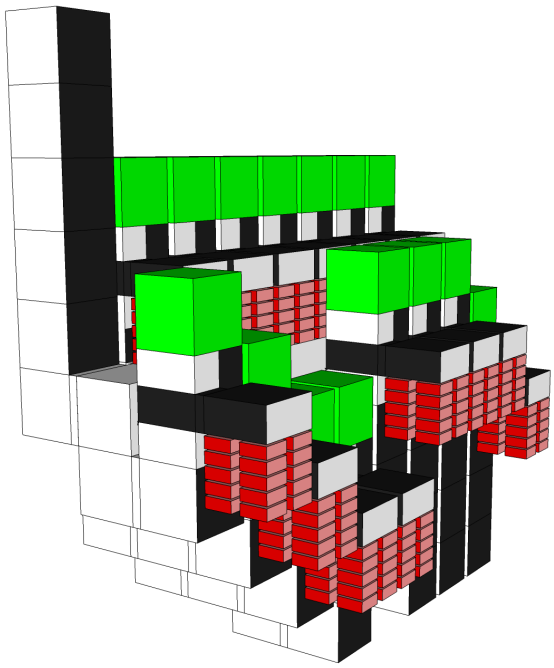


FIG. 17. The first round of state distillation begins with 5 X stabilizer generator measurements taken from Fig. 16 and stacked one atop another, with unnecessary blocks deleted both from top and bottom (no need to create a logical qubit before its first interaction, or hold onto it after its last). As soon as possible, appropriate outputs are half-distance rotated and unequal-size ZZ measurement interacted with the expanded output of multi-attempt state injection. The green box capping this is an $S^\dagger$ gate that is included or not based on whether the ZZ measurement yields 0 or 1 (Fig. 12b).

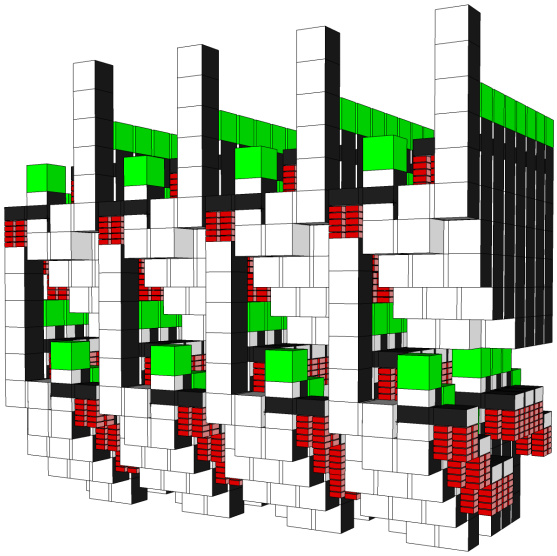


FIG. 18. 8 attempts to produce a $|T\rangle$ state. Note that the 2nd layer of distillation is mirror reflected from the first.

XVII. ACKNOWLEDGMENTS

We thank Daniel Litinski for discussions leading to the simplification of stabilizers measured during mixed-operator multi-body logical measurements.

[1] R. Babbush, C. Gidney, D. W. Berry, N. Wiebe, J. McClean, A. Paler, A. Fowler, and H. Neven, arXiv:1805.03662 (2018).
[2] S. B. Bravyi and A. Y. Kitaev, quant-ph/9811052 (1998).
[3] E. Dennis, A. Kitaev, A. Landahl, and J. Preskill, J. Math. Phys. **43**, 4452 (2002), quant-ph/0110143.
[4] R. Raussendorf and J. Harrington, Phys. Rev. Lett. **98**, 190504 (2007), quant-ph/0610082.
[5] R. Raussendorf, J. Harrington, and K. Goyal, New J. Phys. **9**, 199 (2007), quant-ph/0703143.
[6] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, Phys. Rev. A **86**, 032324 (2012), arXiv:1208.0928.
[7] D. Litinski, arXiv:1808.02892 (2018).
[8] C. Horsman, A. G. Fowler, S. Devitt, and R. Van Meter, New J. Phys. **14**, 123011 (2012), arXiv:1111.4022.
[9] S. Bravyi and A. Kitaev, Phys. Rev. A **71**, 022316 (2005), quant-ph/0403025.
[10] B. W. Reichardt, Quant. Info. Proc. **4**, 251 (2005), quant-ph/0411036.
[11] Y. Li, New J. Phys. **17**, 023037 (2015), arXiv:1410.7808.
[12] B. J. Brown, K. Laubscher, M. S. Kesselring, and J. R. Wootton, Phys. Rev. X **7**, 021029 (2017), arXiv:1609.04673.
[13] D. Litinski and F. von Oppen, Quantum **2**, 62 (2018), arXiv:1709.02318.
[14] A. G. Fowler, D. S. Wang, and L. C. L. Hollenberg, Quant. Info. Comput. **11**, 8 (2011), arXiv:1004.0255.
[15] A. G. Fowler, arXiv:1310.0863 (2013).

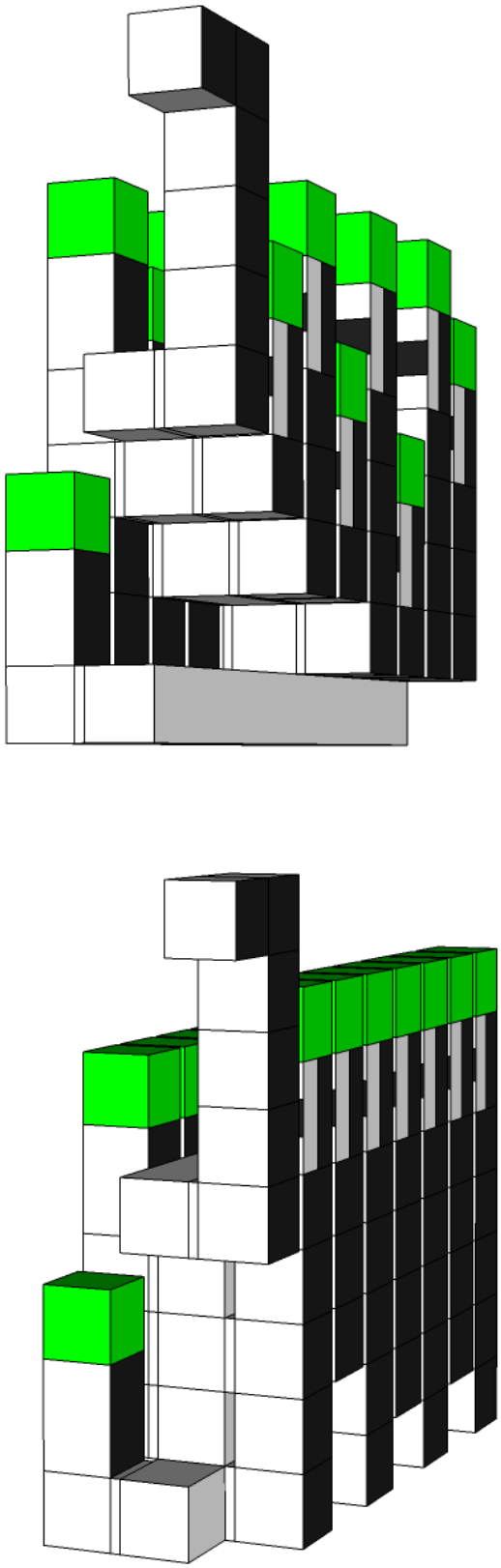


FIG. 19. Top: lattice surgery version of Fig.12b). Bottom: lattice surgery version of Fig.12c). Block position convention as in Fig. 17.

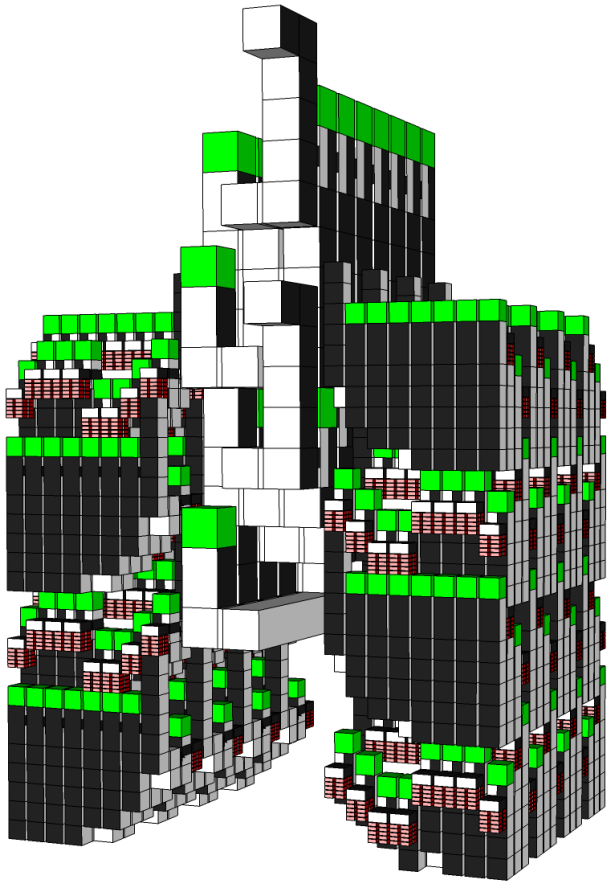


FIG. 20. Two rounds of two levels of state distillation.

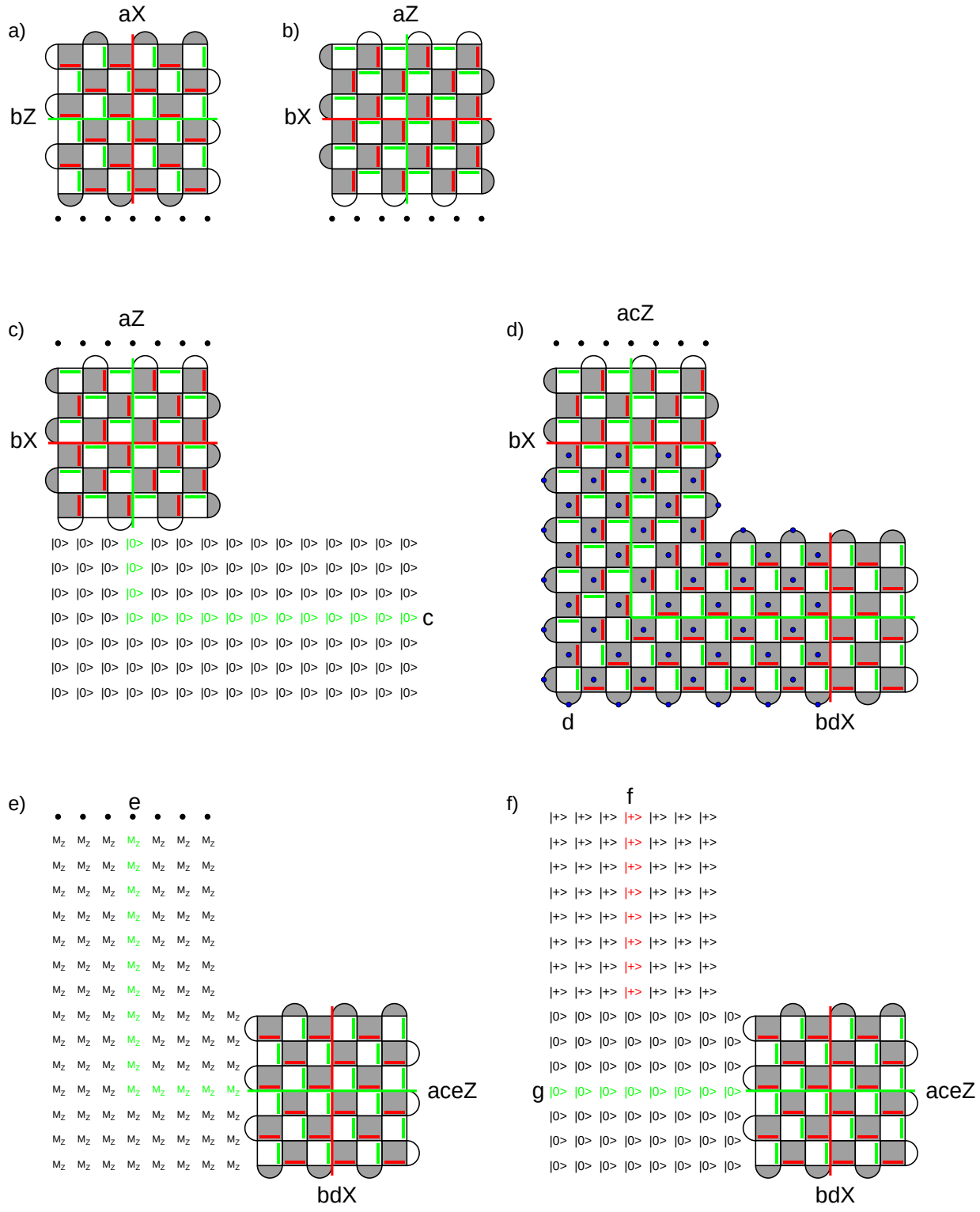


FIG. 21. a) Initial state. b) State after transversal Hadamard. c) State after swap downwards and preparing to expand the logical qubit. d) Expanded logical qubit. Note the orientation of circuits around the corner to ensure the full code distance is preserved. Blue dots show how to move the logical X operator using the product d of stabilizers. e) Contraction of the logical qubit. f) Initialization of the way back to the original position. This and subsequent steps could be omitted if a return to the original position is not required.

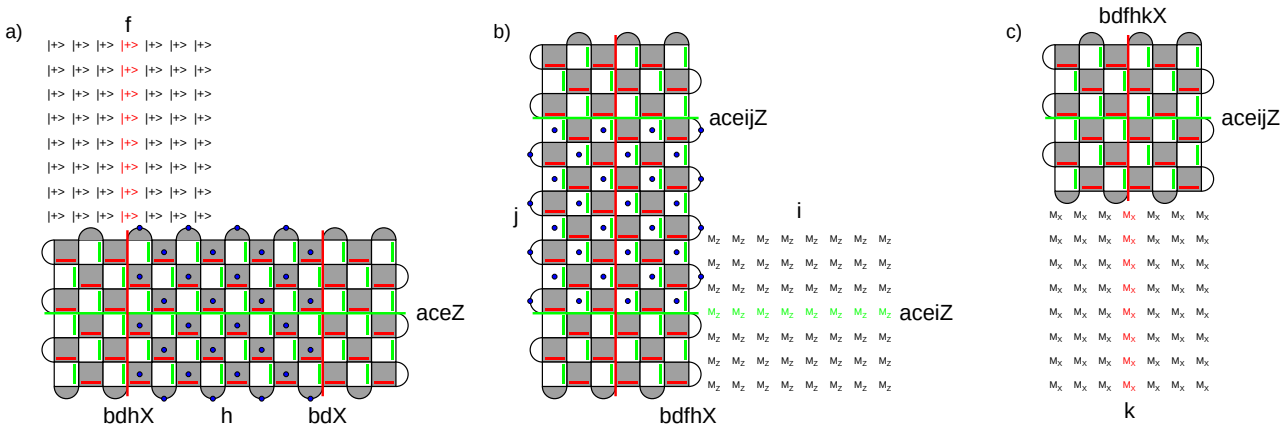


FIG. 22. a) Stabilizer pattern to move logical qubit back to original position. Movement of logical X operator. b) Trimming and movement of logical Z operator. c) Trimming and movement of logical X operator.

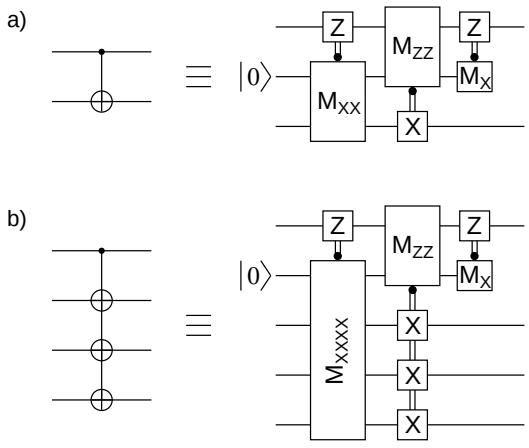


FIG. 23. a) Lattice surgery implementation of CNOT. b) Extension to single-control multiple-target CNOT.

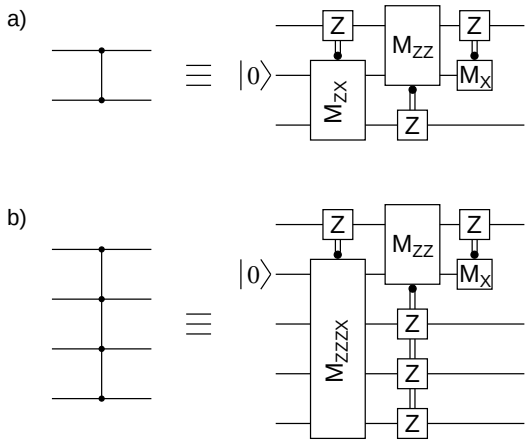


FIG. 24. a) Lattice surgery implementation of CZ. b) Extension to single-control multiple-target CZ.

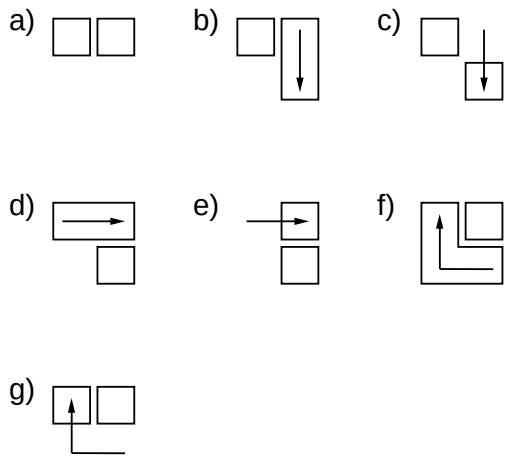


FIG. 25. a) Initial configuration. b)–c) Downward movement, d error detection rounds. d)–e) Rightward movement, d error detection rounds. f)–g) L-shaped return movement using the techniques in Fig. 22, a final d error detection rounds for a total of $3d$.

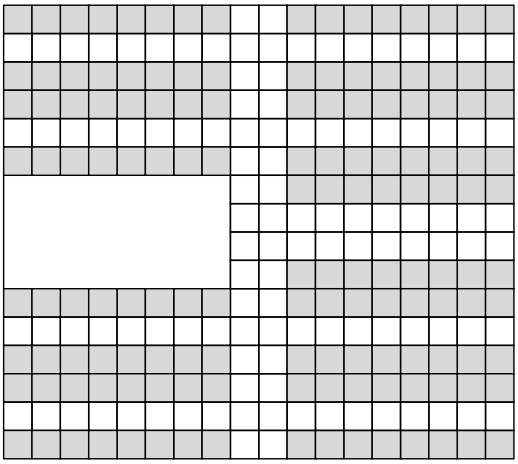


FIG. 26. Example layout of an algorithm showing a single level of distillation (large open rectangle), data logical qubits (shaded squares), and ancilla logical qubits for communication and interaction (white squares).

A Survey of Quantum Computing for Finance

Dylan A. Herman*[1], Cody Googin*[2], Xiaoyuan Liu*[3], Alexey Galda[4], Ilya Safro[3], Yue Sun[1], Marco Pistoia[1], and Yuri Alexeev[5]

[1]JPMorgan Chase Bank, N.A., New York, NY, USA — {dylan.a.herman,yue.sun,marco.pistoia}@jpmorgan.com
[2]University of Chicago, Chicago, IL, USA — codygoogin@uchicago.edu
[3]University of Delaware, Newark, DE, USA — {joeyxliu,isafro}@udel.edu
[4]Menten AI, San Francisco, CA, USA — alexey.galda@menten.ai
[5]Argonne National Laboratory, Lemont, IL, USA — yuri@anl.gov

arXiv:2201.02773v4 [quant-ph] 27 Jun 2022

Abstract. Quantum computers are expected to surpass the computational capabilities of classical computers during this decade and have transformative impact on numerous industry sectors, particularly finance. In fact, finance is estimated to be the first industry sector to benefit from quantum computing, not only in the medium and long terms, but even in the short term. This survey paper presents a comprehensive summary of the state of the art of quantum computing for financial applications, with particular emphasis on stochastic modeling, optimization, and machine learning, describing how these solutions, adapted to work on a quantum computer, can potentially help to solve financial problems, such as derivative pricing, risk modeling, portfolio optimization, natural language processing, and fraud detection, more efficiently and accurately. We also discuss the feasibility of these algorithms on near-term quantum computers with various hardware implementations and demonstrate how they relate to a wide range of use cases in finance. We hope this article will not only serve as a reference for academic researchers and industry practitioners but also inspire new ideas for future research.

*These authors contributed equally to this work.

Contents

1 Introduction **1**

2 Applicability of Quantum Computing to Finance **2**
2.1 Macroeconomic Challenges for Financial Institutions . . . 2
2.1.1 Keeping Up with Regulations . . . 2
2.1.2 Addressing Customer Expectations and Big-Data Requirements . . . 2
2.1.3 Ensuring Data Security . . . 3
2.2 Financial Use Cases for Quantum Computing . . . 3

3 Quantum Computing Concepts **4**
3.1 Fundamentals of Quantum Information . . . 5
3.2 Models of Quantum Computation . . . 6
3.2.1 Gate-Based Quantum Computing . . . 6
3.2.2 Adiabatic Quantum Computing . . . 7
3.3 Quantum Hardware Challenges . . . 7
3.3.1 Noise . . . 8
3.3.2 Connectivity . . . 8
3.3.3 Gate Speed . . . 8
3.3.4 Native Gates . . . 8

4 Foundational Quantum Algorithms **8**
4.1 Quantum Unstructured Search . . . 9
4.2 Quantum Amplitude Amplification . . . 9
4.3 Quantum Phase Estimation . . . 10
4.4 Quantum Amplitude Estimation . . . 10
4.5 Quantum Linear System Algorithms . . . 10
4.6 Quantum Walks . . . 11
4.7 Variational Quantum Algorithms . . . 12
4.8 Adiabatic Quantum Optimization . . . 12

5 Stochastic Modeling **12**
5.1 Monte Carlo Integration . . . 12
5.2 Numerical Solutions of Differential Equations . . . 14
5.2.1 Quantum-linear-system-based algorithms . . . 15
5.2.2 Variational algorithms . . . 16
5.3 Financial Applications . . . 16
5.3.1 Derivative Pricing . . . 16
Options. . . . 17
Collateralized Debt Obligations. . . . 18
5.3.2 Risk Modeling . . . 19
Value at Risk. . . . 19
Economic Capital Requirement. . . . 19
The Greeks. . . . 19
Credit Value Adjustments. . . . 20

6 Optimization **20**
6.1 Combinatorial Optimization . . . 21
6.1.1 Quantum Annealing . . . 21
6.1.2 Quantum Approximate Optimization Algorithm . . . 22
6.1.3 Variational Quantum Eigensolver . . . 23
6.1.4 Variational Quantum Imaginary Time Evolution (VarQITE) . . . 23
6.1.5 Optimization by Quantum Unstructured Search . . . 24
6.2 Convex Optimization . . . 24
6.3 Large-Scale Optimization . . . 24
6.4 Financial Applications . . . 24
6.4.1 Portfolio Optimization . . . 24
Combinatorial Formulations. . . . 25
Convex Formulations. . . . 26
6.4.2 Swap Netting . . . 26

6.4.3 Optimal Arbitrage . . . 27
6.4.4 Identifying Creditworthiness . . . 27
6.4.5 Financial Crashes . . . 27

7 Machine Learning **28**
7.1 Regression . . . 28
7.2 Classification . . . 29
7.2.1 Quantum Support Vector Machine . . . 29
7.2.2 Quantum Nearest-Neighbors Algorithm . . . 30
7.3 Clustering . . . 30
7.3.1 Quantum k-Means Clustering . . . 30
7.3.2 Quantum Spectral Clustering . . . 30
7.4 Dimensionality Reduction . . . 31
7.5 Generative Models . . . 31
7.5.1 Quantum Circuit Born Machine . . . 31
7.5.2 Quantum Bayesian Networks . . . 31
7.5.3 Quantum Boltzmann Machines . . . 32
7.5.4 Quantum Generative Adversarial Networks . . . 32
7.6 Quantum Neural Networks . . . 32
7.6.1 Quantum Feedforward Neural Network . . . 32
7.6.2 Quantum Convolutional Neural Network . . . 32
7.6.3 Quantum Graph Neural Networks . . . 33
7.7 Quantum Reinforcement Learning . . . 33
7.8 Natural Language Modeling . . . 34
7.9 Financial Applications . . . 34
7.9.1 Anomaly Detection . . . 34
7.9.2 Asset Pricing . . . 34
7.9.3 Implied Volatility . . . 35

8 Hardware Implementations of Use Cases **35**
8.1 Quantum Risk Analysis on a Transmon Device . . . 35
8.2 Portfolio Optimization with Reverse Quantum Annealing . . . 35
8.3 Portfolio Optimization on a Trapped-Ion Device . . . 36

9 Conclusion and Outlook **38**

1 Introduction

Quantum computation relies on a fundamentally different means of processing and storing information than today's classical computers use. The reason is that the information does not obey the laws of classical mechanics but those of quantum mechanics. Usually, quantum-mechanical effects become apparent only at very small scales, when quantum systems are properly isolated from the surrounding environments. These conditions, however, make the realization of a quantum computer a challenging task. Even so, according to a McKinsey & Co. report [1], finance is estimated to be the first industry sector to benefit from quantum computing (Section 2), largely because of the potential for many financial use cases to be formulated as problems that can be solved by quantum algorithms suitable for near-term quantum computers. This is important because current quantum computers are small-scale and noisy, yet the hope is that we can still find use cases for them. In addition, a variety of quantum algorithms will be more applicable when large-scale, robust quantum computers become available, which will significantly speed up computations used in finance.

The diversity in the potential hardware platforms or physical realizations of quantum computation is completely unlike any of the classical computational devices available today. There exist proposals based on vastly different physical systems: superconductors, trapped ions, neutral atoms, photons, and others (Section 3.3), yet no clear winner has emerged. A large number of companies are competing to be the first to develop a quantum computer capable of running applications useful in a production environment. In theory, any computation that runs on a quantum computer may also be executed on a classical computer—the benefit that quantum computing may bring is the potential reduction in *time* and memory *space* with which the computational tasks are performed [2], which in turn may lead to unprecedented scalability and accuracy of the computations. In addition, any classical algorithm can be modified (in a potentially nontrivial way) such that it can be executed on a universal quantum computer, but without any speedup. Obtaining quantum speedup requires developing new algorithms that specifically take advantage of quantum-mechanical properties. Thus, classical and quantum computers will need to work together. Moreover, in order to solve a real-world problem, a classical computer should be able to efficiently insert *and* obtain the necessary data into and from a quantum computer.

This promised efficiency of quantum computers enables certain computations that are otherwise infeasible for current classical computers to complete in any reasonable amount of time (Section 3). In general, however, the speedup for each task can vary greatly or may even be currently unknown (Section 4). While these speedups, if found, can have a tremendous impact in practice, they are typically difficult to obtain. And even if they are discovered, the underlying quantum hardware must be powerful enough to minimize errors without introducing an overhead that counteracts the algorithmic speedup. We do not know exactly when such robust hardware will exist. Thus, the goal of quantum computing research is to develop quantum algorithms (Section 4) that solve useful problems faster and to build robust hardware platforms to run them on. The industry needs to understand the problems that can best benefit from quantum computing and the extent of these benefits, in order to make full use of its revolutionary power when production-grade quantum devices are available.

To this end, we offer a comprehensive overview of the applicability of quantum computing to finance. Additionally, we provide insight into the nature of quantum computation, focusing particularly on specific financial problems for which quantum computing can provide potential speedups compared with classical computing. The sections in this article can be grouped into two parts. The *first part* contains introductory material: Section 2 introduces computationally intensive financial problems that potentially benefit from quantum computing, whereas Sections 3 and 4 introduce the core concepts of quantum computation and quantum algorithms. The *second part*—the main focus of the article—reviews research performed by the scientific community to develop quantum-enhanced versions of three major problem-solving techniques used in finance: stochastic modeling (Section 5), optimization (Section 6), and machine learning (Section 7). The connections between financial problems and these problem-solving techniques are outlined in Table 1. Section 8 covers experiments on current quantum hardware that have been performed by researchers to solve financial use cases.

Overview of Earlier Surveys

Although the field of quantum computing is one of the most dynamic today and some scientific conclusions, approaches, and technologies become obsolete relatively quickly, we reiterate the importance of getting information from different perspectives. Here we emphasize the difference between our survey and several existing surveys that have recently been published or appeared in open domain. One of the key differences is that in our survey we not only summarize the recent achievements but also discuss the limitations of quantum devices and algorithmic approaches that are currently subject to massive abuse in the media.

In [3], Egger et al. focus on covering the hardware and algorithmic work done by IBM. We take a much broader view and survey the entire landscape of quantum technologies and their applicability in the finance domain. For example, we believe it is beneficial to also discuss the quantum annealing-based approaches and other gate-based devices. Also, with regard to optimization applications, we discuss a much wider variety of financial applications that make use of optimization and may take advantage of quantum hardware development.

The article by Bouland et al. [4] focuses on works done by the QC Ware team with a central emphasis on quantum Monte Carlo integration. Some more recent work has been done in that area and is included in our survey. Also Bouland et al. focus on portfolio optimization. While this is an important financial problem that we also cover, we have tried to include other financial applications that use optimization as well. Moreover, we tried to delve deeper into the various quantum machine learning approaches for generative modeling and neural networks.

The survey by Orus et al. [5] does an excellent job at highlighting financial applications that make use of quantum annealers. However, we believe the quantitative finance and quantum computing communities would also benefit from hearing about other quantum optimization approaches and devices. We also tried to delve a little bit deeper into how quantum annealing works and discuss universal adiabatic computation. Given that the field of quantum computation is so dynamic, we believe the community can benefit from seeing more recent work.

A recent survey by Pistoia et al. [6] covers a variety of quantum machine learning algorithms applicable to finance. In our review, we discuss a broader array of applications outside the realm of machine learning, such as financial applications that make use of stochastic modeling and optimization.

There are also several problem-specific surveys such as the derivative pricing [7], supply chain finance [8] and high-frequency trading [9] that tackle different goals than that of our survey.

2 Applicability of Quantum Computing to Finance

Numerous financial use cases require the ability to assess a wide range of potential outcomes. To this extent, financial institutions employ statistical models and algorithms to predict future outcomes. Such techniques are fairly effective but not infallible. In a world where huge amounts of data are generated daily, computers that can perform predictive computations accurately are becoming a predominant need. For this reason, several financial institutions are turning to quantum computing, given its promise to analyze vast amounts of data and compute results faster and more accurately than any classical computer has ever been able to do. Financial institutions believe that once they are capable of leveraging quantum computing, they are likely to see important benefits. This section introduces financial use cases that are projected to lend themselves to a quantum computing approach.

2.1 Macroeconomic Challenges for Financial Institutions

The financial services industry is expected to reach a market cap of $28.53 trillion by 2025, with a compound annual growth rate of 6% [10]. While this industry is often considered incredibly stable and insulated from change, shifting government regulations, new technologies, and customer expectations present challenges that require the industry to adapt. We discuss here three key macroeconomic financial trends that can be impacted by quantum computing: keeping up with regulations, addressing customer expectations and big data requirements, and ensuring data security.

2.1.1 Keeping Up with Regulations

The Third Basel Accord, commonly known as Basel III, is an international regulatory framework set up in response to the 2008 financial crisis. Basel III sets requirements on capital, risk coverage, leverage, risk management and supervision, market discipline, liquidity, and supervisory monitoring [11]. These regulations require computing numerous risk metrics, such as value at risk (VaR) and conditional value at risk (CVaR). Because of the stochastic nature of the parameters involved in these computations, closed-form analytical solutions are generally not available, and hence numerical methods have to be employed. One of the most widely used numerical methods is Monte Carlo integration, since it is generic and scalable to high-dimensional problems [12]. Quantum Monte Carlo integration (Section 5.1) has been shown to offer up to a quadratic speedup over the performance of its classical counterpart. Engaging in ways to determine risk metrics with higher levels of accuracy and efficiency will enable financial institutions to make better-informed loan-related decisions and financial projections.

2.1.2 Addressing Customer Expectations and Big-Data Requirements

Customer personalization has become increasingly important in finance. For example, financial technology (FinTech), in contrast to traditional finance, thrives on providing personalized services. A Goldman Sachs report has highlighted the necessity for financial institutions to provide new big-data-driven services to align with changing consumer behavior [13]. According to a Salesforce 2020 publication [14], 52% of customers expect companies to always personalize offers, while only 27% of customers feel that the financial services industry provides great service and support [14]. Financial institutions must meet this expectation in order to continue to retain their customers. The current classical techniques for analyzing data have large computational demands and require

significant time to train algorithms, whereas quantum algorithms may be able to speed up these computationally intensive and time-consuming components. As financial institutions continue to generate data, they must be able to employ that data in a functional way to improve their business strategy. Additionally, data organization can allow financial institutions to engage with customers' finances more specifically and effectively, supporting their customer service and keeping customers engaged despite other options such as FinTech. Much of this data analysis is done by dealing with large matrices, which can be computationally demanding for classical computers.

2.1.3 Ensuring Data Security

Ensuring data security is one of the most pressing concerns for the financial services industry, according to an article by McKinsey [15] discussing rising cybersecurity issues. An increase in digitization has occurred due to the rise of FinTech. Additionally, as a result of the COVID-19 pandemic, substantially more financial activity is performed online. According to the McKinsey article, 95% of board committees discuss cyber and tech risks four or more times a year. Financial institutions must stay up to date with online security and remain vigilant against attackers. Quantum computing data modeling and machine learning techniques could allow financial institutions to identify potential risks with higher accuracy. Furthermore, even if one does not plan to use quantum computation, one must be knowledgeable about it because of its ability to break current public-key cryptographic standards [2, 16, 17], such as RSA and Diffie–Hellmann, including the common variants based on elliptic curves [18]. This is due to quantum computing's ability to efficiently solve the Abelian hidden subgroup problem [19]. Although current quantum computers cannot achieve this task, financial institutions need to become ready by investigating how to utilize quantum-safe protocols [20] and, potentially, quantum-enhanced cryptography [21], each of which has its own potential usages in the layers of a network. While the situation is by no means as serious for cryptographically-secure hash functions or symmetric-key ciphers, it is important to be aware of the potential quantum-enhanced attacks [22–25]. Here, however, we are going to focus on quantum algorithms that have direct applications in finance, so protocols for quantum-enhanced cryptographic attacks, quantum-safe cryptography, and quantum-enhanced cryptography are all out of the scope of this survey.

2.2 Financial Use Cases for Quantum Computing

As discussed earlier, finance is one of the first industry sectors expected to benefit from quantum computing, largely because financial problems lend themselves to be solved on near-term quantum computers. Since many of the financial problems are based on stochastic variables as inputs, they are, more often than not, relatively more tolerant to imprecisions in the solutions compared with problems in chemistry and physics. Arguably, certain problems in chemistry and physics also can benefit from quantum computing. In this survey, however, we aim to review common financial problems that can benefit from quantum computing and to discuss the associated quantum algorithms applicable to solving them. We group these problems and quantum algorithms into three broad categories: stochastic modeling, optimization, and machine learning.

Stochastic modeling is concerned with the study of the dynamics and statistical characteristics of stochastic processes. In finance, one of the most commonly seen problems that involves stochastic modeling, using numerical techniques, is estimating the prices of financial assets and their associated risks, whose values may depend on certain stochastic processes. In Section 5, we will discuss two of the most used techniques for modeling stochastic processes in finance—Monte Carlo methods and numerical solutions of differential equations, and the corresponding quantum approaches for them, namely, quantum Monte Carlo integration (QMCI) and quantum partial differential equation (PDE) solvers. We will also demonstrate the applicability of the two quantum approaches to financial problems through examples of derivative pricing (Section 5.3.1) and risk modeling (Section 5.3.2). Stochastic modeling with quantum machine learning (QML) techniques will be discussed in Sections 7.9.2 and 7.9.3.

Optimization involves finding optimal inputs to a real-valued function so as to minimize or maximize the value of the function. In Section 6 we will review prevailing quantum techniques for combinatorial and convex optimization problems. We will cover adiabatic and variational quantum algorithms, as well as other quantum-classical hybrid approaches. A large variety of financial problems that involve optimization could potentially benefit from these quantum techniques, in particular, portfolio optimization (Section 6.4.1), hedging and swap netting (Section 6.4.2), optimal arbitrage (Section 6.4.3), credit scoring (Section 6.4.4), and financial crash prediction (Section 6.4.5).

Machine learning has become an essential aspect in modern business and research across various industries. It has revolutionized data processing and decision-making, empowering organizations large and small with unprecedented ability to leverage the ever-growing amount of easily accessible information. Using quantum machine learning techniques can potentially speed up the training of the algorithm or certain parts of the whole process. We will explore these techniques in Section 7. In finance, some of the most common use cases for machine learning are anomaly detection (Section 7.9.1), natural language modeling (Section 7.8), asset pricing (Section 7.9.2), and

implied volatility calculation (Section 7.9.3). Applications of quantum machine learning in these use cases will be discussed in the respective sections.

In Table 1 we present a summary of the quantum techniques and their applicable financial use cases covered in this survey.

Problem Category	**Example Use Cases**	**Classical Solutions**	**Quantum Solutions**
Stochastic Modeling	Derivative Pricing (Section 5.3.1), Risk Analysis (Section 5.3.2)	Monte Carlo Integration, Numerical PDE Solver, Machine Learning	Quantum Monte Carlo Integration (Section 5.1), Quantum PDE Solver (Section 5.2), Quantum Machine Learning (Section 7)
Optimization	Portfolio Optimization (Section 6.4.1), Hedging, Swap Netting (Section 6.4.2), Optimal Arbitrage (Section 6.4.3), Credit Scoring (Section 6.4.4), and Financial Crash Prediction (Section 6.4.5)	Branch-and-Bound (with cutting -planes, heuristics, etc.) for non-convex cases [26] and Interior-Point Methods for certain convex cases [27]	Quantum Optimization (Section 6)
Machine Learning	Anomaly Detection (Section 7.9.1), Natural Language Modeling (Virtual Agents, Analyzing Financial Documents, Section 7.8), Risk Clustering (Section 7.3)	Deep Learning, Cluster Analysis	Quantum Machine Learning (Section 7), Quantum Cluster Analysis (Sections 7.3.1 and 7.3.2)

Table 1: Financial Use Cases with Corresponding Classical and Quantum Solutions

3 Quantum Computing Concepts

Quantum computing is an emerging and promising field of science [28]. Global venture capital funds, public and private companies, and governments have invested millions of dollars in this technology. Quantum computing is driven by the need to solve computationally hard problems efficiently and accurately. For decades, the advancement of classical computing power has been remarkably following the well-known Moore's law. Initially introduced by Gordon Moore in 1965, Moore's law forecasts that the number of transistors in a classical computer chip will double every two years, resulting in lower prices for manufacturers and consumers. Today, transistors are already at the size at which quantum-mechanical effects become apparent and problematic [29]. These effects will only become more difficult to manage as classical chips decrease in size. This problem is partially circumvented by alternative forms of computing that do not use conventional transistors, such as photonic computing, neuromorphic computing, biocomputing, and special-purpose chips. However, these techniques still use classical algorithms and are subject to the scaling of them. Thus, it is imperative to investigate quantum computing, which provides scaling advantages no classical architectures can achieve.

Google has demonstrated an enormous computational speedup using quantum computing for the task of simulating the output of pseudo-random quantum circuits [30]. Specifically, Google claimed that what takes its quantum device, Sycamore, 200 seconds to accomplish would take Summit, one of the most powerful supercomputers, approximately 10,000 years. Thus, *quantum supremacy* has been achieved. As defined by John Preskill [31], quantum supremacy is the promise that certain computational (not necessarily useful) tasks can be executed exponentially faster on a quantum processor than on a classical one. The decisive demonstration

of quantum devices to solve more useful problems faster and/or more accurately than classical computers, such as large-scale financial problems (Section 2.2), is called *quantum advantage*. Quantum advantage is more elusive and, arguably, has not been demonstrated yet. The main challenge is that existing quantum hardware does not yet seem capable of running algorithms on large enough problem instances. Current quantum hardware (Section 3.3) is in the *noisy intermediate-scale quantum* (NISQ) technology era [32], meaning that the current quantum devices are underpowered and suffer from multiple issues. The *fault-tolerant* era refers to a yet unknown period in the future in which large-scale quantum devices that are robust against errors will be present. In the next two subsections we briefly discuss quantum information (Section 3.1) and models for quantum computation (Section 3.2). In the last subsection (Section 3.3) we give an overview of the current state of quantum hardware and the challenges that must be overcome.

3.1 Fundamentals of Quantum Information

All computing systems rely on the fundamental ability to store and manipulate information. Today's classical computers operate on individual bits, which store information as binary 0 or 1 states. In contrast, quantum computers use the physical laws of quantum mechanics to manipulate information. At this fundamental level, the unit of information is represented by a *quantum bit*, or *qubit*. Physically, a qubit is any two-level quantum system [2, 33]. Mathematically, the state space of a single qubit can be associated with the complex projective line, denoted $\mathbb{C}\mathrm{P}^1$ [34].[1] However, one commonly considers qubit states as elements ψ, called *state vectors*, of a two-dimensional complex-vector space but restrict consideration to those that satisfy $\|\psi\|_2 = 1$ and allow for ψ and $e^{i\theta}\psi$ to be used interchangeably (i.e., consider specific elements of the equivalence classes in $\mathbb{C}\mathrm{P}^1$). A state vector is usually denoted by using Dirac's "bra-ket" notation: ψ is represented by the "ket" $|\psi\rangle$. Examples of two single-qubit kets are the states $|0\rangle$ and $|1\rangle$, which are analogous to the classical bits 0 and 1.

Multiqubit state spaces are expressed through the use of the vector space tensor product [35], which results in a 2^n-dimensional complex vector space for n qubits. The tensor product of two state vectors is denoted as $|\psi_1\rangle \otimes |\psi_2\rangle$, $|\psi_1\rangle\,|\psi_2\rangle$, or $|\psi_1\psi_2\rangle$ and extends naturally to multiple qubits. *Entanglement* is a quantum phenomenon that can be present in multiqubit systems in which the states of the subsystems *cannot* be described independently. This results in correlated observations (measurement results) even when physical communication between qubits, which would correlate them in a classical sense, is impossible [2]. Entangled states are mathematically expressed as tensors that are not factorable over the involved subsystems, in other words, non-simple tensors. A quantum state is a *product state* of certain subsystems if those subsystems *can* be described independently and it is a simple tensor with respect to those subsystems. Qubits are in a *superposition* state with respect to a given basis for the state space if their state vector can be expressed as a nontrivial linear combination of basis states. Thus, a qubit can be in a state that is an arbitrary superposition of $|0\rangle$ and $|1\rangle$, whereas a classical bit can be in only one of the analogous states 0 or 1. The coefficients of the superposition are complex numbers called *amplitudes*. Although multiplying a state by a phase $e^{i\theta}$, called a *global phase*, has no physical significance, the *relative phase* between the amplitudes of two basis states in a superposition does.

A *measurement* in quantum mechanics consists of probing a system to obtain a numerical result. Measurement of a quantum system is probabilistic. A *projective measurement* of a system with respect to a *Hermitian* operator A, called an *observable*, results in the state vector of the system being orthogonally projected onto an eigenspace, with orthogonal projector Π_λ, and the observable quantity is the associated eigenvalue, λ. A potential projective measurement result λ is observed with probability equal to $\|\Pi_\lambda\,|\psi\rangle\|_2^2$. The *expected value* of the measurement is equal to $\langle\psi|A|\psi\rangle$, where $\langle\psi|$, called a "bra," is the Hermitian adjoint. In physics, the quantum *Hamiltonian* is the observable for a system's energy. A *ground state* of a quantum Hamiltonian is a state vector in an eigenspace associated with the smallest eigenvalue and thus has the lowest energy. Any physical transformation of a quantum system can be represented by a completely positive non-trace increasing linear operator. Two important special cases of such operations are *unitary* operators and measurements, which are not unitary. The dynamics of a closed quantum system follow the Schrödinger equation, $i\hbar\,|\dot{\psi}(t)\rangle = H\,|\psi(t)\rangle$, where $\hbar$ is the reduced Planck constant and H is the system's quantum Hamiltonian. The unitary operator $e^{-iH\Delta/\hbar}$, which transforms a solution $|\psi(t)\rangle$ to the Schrödinger equation at one point in time t into another one at a different point in time $t+\Delta$, is called the *time-evolution* operator or propagator. There can also be a *time-dependent Hamiltonian*, $H(t)$, where the operator H changes over time. However, the propagator is computed by the product integral [36] to take into account that $H(t_i)$ and $H(t_j)$ may not commute for all t_i, t_j in the specified evolution time interval [33]. The simulation of time evolution on a quantum computer is called *Hamiltonian simulation*. For quantum algorithms, the Hamiltonian can be an arbitrary observable with no connection to a physical system, and $\hbar := 1$ is often assumed. Three important observables on a single qubit are the Pauli operators: $\mathsf{X} := |+\rangle\langle+| - |-\rangle\langle-|$, $\mathsf{Y} := |i\rangle\langle i| - |-i\rangle\langle -i|$, and $\mathsf{Z} := |0\rangle\langle 0| - |1\rangle\langle 1|$, where $|\pm\rangle := \frac{|0\rangle \pm |1\rangle}{\sqrt{2}}$, and $|\pm i\rangle := \frac{|0\rangle \pm i|1\rangle}{\sqrt{2}}$, where $|\psi\rangle\langle\psi|$ denotes the

[1] Generally, quantum states live in a projective Hilbert space, which can be infinite-dimensional [33].

vector outer product.[2] For n qubits, the set $\{|k\rangle \mid k \in \{0,1\}^n\}$ forms the *computational basis*. *Measuring in the computational basis* probabilistically projects the quantum state onto a computational basis state. A measurement in the context of quantum computation, without further clarification, usually refers to a measurement in the computational basis.

A positive semi-definite operator ρ on the quantum state space with $\mathrm{Tr}(\rho) = 1$ is called a *density operator*, where $\mathrm{Tr}(\cdot)$ is the trace operator. The classical probabilistic mixture of *pure states* $|\psi_i\rangle$, in other words, state vectors, with probabilities p_i has density operator $\rho := \sum_i p_i |\psi_i\rangle\langle\psi_i|$ and is called a *mixed state*. In general, a density operator ρ represents a mixed state if $\mathrm{Tr}(\rho^2) < 1$ and a pure state if $\mathrm{Tr}(\rho^2) = 1$. *Fidelity* is one of the distance metrics for quantum states ρ and σ, defined as $F(\rho,\sigma) = \mathrm{Tr}(\sqrt{\rho^{1/2}\sigma\rho^{1/2}})$ [2]. The *fidelity of an operation* refers to the computed fidelity between the quantum state that resulted after the operation was performed and the expected state. *Decoherence* is the loss of information about a quantum state to its environment, due to interactions with it. Decoherence constitutes an important source of error for near-term quantum computers.

For further details, the works of Nielsen and Chuang [2] and Kitaev et al. [37] are the standard references on the subject of quantum computation. The former covers a variety of topics from quantum information theory, while the latter focuses on quantum computational complexity theory.

3.2 Models of Quantum Computation

A qubit-based[3] *model of quantum computation* is, simply, a methodology for performing computations on qubits. A model can realize *universal* quantum computation if it can be used to approximate any n-qubit unitary, for all n, with arbitrarily small error [2, 40]. Two universal models are polynomial-time equivalent if they can simulate each other with only a polynomially-bounded overhead in computational resources [40, 41]. We briefly discuss two polynomial-time-equivalent models that can be used to realize universal quantum computation: gate-based quantum computing and adiabatic quantum computing. There are others based on qubits [40] that we will not review since current quantum devices do not implement them, even in a non-universal fashion.

When running an algorithm on quantum hardware, one must contend with errors that can occur during computation. In the near term, *quantum error mitigation* techniques are commonly used to reduce (mitigate) errors, rather than eliminate them. However, error correction will be required in the long term. Techniques for *quantum error correction* [42] or fault tolerance have been developed to construct *logical* qubits and operations that tolerate errors. This strategy has been extensively studied for gate-based quantum computing [43].

3.2.1 Gate-Based Quantum Computing

Gate-based quantum computing, also known as the quantum circuit model, composes a countable sequence of unitary operators, called *gates*, into a *circuit* to manipulate qubits that start in a fixed state [2, 37]. A circuit can be viewed as a directed acyclic graph tracking the operations applied, which qubits they operate on, and the time step at which they are applied. Typically, the circuit ends with measuring in the computational basis. However, measurements of subsets of qubits before the end of the circuit and operations conditioned on the results of measurements are also possible [44]. Sequences of gates that act on separate sets of qubits can, if hardware supports it, be applied in parallel. The *circuit depth* is the longest sequence of gates in the circuit, from initial state until measurement, that cannot be made shorter through further parallelization. The quantum circuit model is analogous to the Boolean circuit model of classical computation [37]. An important difference with quantum computation is that because of unitarity all gates are invertible, i.e., computation involving only quantum gates is reversible. A *shot* is a single execution of a quantum circuit followed by measurements. Multiple shots are required if the goal is to obtain statistics.

This model can be utilized to realize universal quantum computation by considering circuits with n qubits built by using any type of single-qubit gate and only one type of two-qubit gate [2]. Any n-qubit unitary can be approximated in this manner to arbitrarily small error. The two-qubit gates can be used to mediate entanglement. A discrete set of gates that can be used for universal quantum computation is called a *basis gate set*. The basis gate set realized by a quantum device is called its *native gate set*. The native gate set typically consists of a finite number of (parameterized and non-parameterized) single-qubit gates and one two-qubit entangling operation. An arbitrary single-qubit gate can be approximately decomposed into the native single-qubit gates.

Some common single-qubit gates are the Hadamard gate (H), the Phase (S) and $\frac{\pi}{8}$-Phase (T) gates, the Pauli gates (X, Y, Z), and the rotation gates generated by Pauli operators ($\mathsf{R_X}(\theta), \mathsf{R_Y}(\theta), \mathsf{R_Z}(\theta)$). Examples of two-qubit gates are the controlled-NOT gate denoted CX or CNOT and the SWAP gate. The H, S, and CX gates generate,

[2] The Pauli operators are also sometimes denoted as σ_x, σ_y, and σ_z, respectively.

[3] There are other models of quantum computation that are not based on qubits, such as n-level quantum systems, for $n > 2$, called *qudits* [38], and continuous-variable, infinite-dimensional systems [39]. This article discusses only quantum models and algorithms based on qubits.

under the operation of composition, the *Clifford group*, which is the normalizer of the group of Pauli gates [2]. The *time complexity* of an algorithm is measured in terms of the depth of the circuit implementing it. This is in terms of native gates when running the circuit on hardware. The majority of commercially available quantum computers implement the quantum circuit model.

We note that quantum circuits generated for gate-based quantum computers can also be simulated on classical hardware. It is generally a computationally inefficient process, however, with enormous memory requirements scaling exponentially with the number of qubits [45–48]. But for certain types of computations such as computing the expectation values of observables, these requirements can be dramatically reduced by using tensor network simulators [49–51]. Classical simulation is commonly used for the development of algorithms, verification, and benchmarking, to name a few applications.

3.2.2 Adiabatic Quantum Computing

Adiabatic quantum computing (AQC) is a model for quantum computation that relies on the quantum *adiabatic theorem*. This theorem states that as long as the evolution of a time-dependent Hamiltonian, $H(t)$, is sufficiently slow (defined in references) from an initial nth eigenstate of the time-dependent Hamiltonian at $t = 0$, it will remain in an instantaneous nth eigenstate throughout the evolution for all t [52]. A unitary operation can be applied by following an adiabatic trajectory from an initial Hamiltonian H_i whose ground state is easy to prepare (e.g., a product state of qubits) to the final Hamiltonian H_f whose ground state is the result that would be obtained after applying the unitary. A computation consists of evolving a time-dependent Hamiltonian $H(s)$ for time t_f according to a schedule $s(t) : [0, t_f] \mapsto [0, 1]$, which interpolates between the two (non-commuting) Hamiltonians: $H(s) = (1-s)H_i + sH_f$ [52]. The time required for an adiabatic algorithm to produce the result to some specified error is related to the minimum *spectral gap* among all instantaneous Hamiltonians. The spectral gap of the Hamiltonian at time t is the difference between the nth eigenvalue and the next closest one (e.g., difference between the ground-state energy and the first excited level). This quantity helps define how slow the evolution needs to be in order to remain on the adiabatic trajectory—although it is notoriously difficult to bound in practice [52]. One example of where this has been done is for the adiabatic version of Grover's algorithm (Section 4.1) [53]. There exist classes of time-dependent Hamiltonians that through adiabatic evolution can approximate arbitrary unitary operators applied to qubits [52, 54]. Thus AQC can realize universal quantum computation. As of this writing, there does not exist a commercially available device for universal AQC. By polynomial equivalence to the circuit model, however, gate-based devices can efficiently simulate it. There are commercially available devices that implement a specific AQC algorithm for combinatorial optimization, called *quantum annealing*, discussed in Section 4.8. It is believed to be unlikely that the transverse-field Ising (stoquastic [52]) Hamiltonian used for quantum annealing is universal [54].

3.3 Quantum Hardware Challenges

As mentioned, the quantum computers available today are called NISQ devices. Multiple physical realizations of qubit-based quantum computation have been developed over the past decade. The majority of efforts to develop quantum technologies at this time are driven by industry and, to a lesser degree, by academia. Superconducting-based quantum computers [55] are manufactured by IBM, Google, D-Wave, and Rigetti, among others. Quantum computers using trapped atomic ions [56] have been developed, for example, by IonQ, Quantinuum, and AQT. These two technologies are currently the most widely available and thus, have been frequently utilized for research. However, other promising technologies also are under development, including photonic systems by PsiQuantum and Xanadu; neutral-atoms by ColdQuanta, QuEra Computing, Atom Computing and Pasqal, spins in silicon by Silicon Quantum Computing; quantum dots; molecular qubits; and topological quantum systems. As mentioned in Section 3.2, most of the current quantum devices and those that are in development, with the exception of the D-Wave quantum annealers, follow the quantum circuit model. All of the mentioned quantum devices have system-level attributes, which need to be considered in multiqubit systems. Significant scientific and engineering efforts are needed to improve and optimize these devices. The most important attributes are discussed in the remainder of this section.

The technical challenges of NISQ devices discussed below have a major impact on the current state of quantum algorithms. The algorithms can be roughly split into two camps: (1) near-term algorithms designed to run on NISQ devices and (2) algorithms that have a theoretically proven advantage for when hardware advances enough but require a large number of logical qubits with extremely high-fidelity quantum gates. In this work we cover both types of algorithms. One should keep in mind that arguably none of the discussed algorithms implemented on NISQ devices provide a decisive advantage over classical algorithms yet.

3.3.1 Noise

Qubits lose their desired quantum state or decohere (Section 3.1) over time. The decoherence times of each of the qubits are important attributes of a quantum device. Various mechanisms of qubit decoherence exist, such as quantum amplitude and phase relaxation processes and depolarizing quantum noise [2]. One potentially serious challenge for superconducting systems is cosmic rays [57]. While single-qubit decoherence is relatively well understood, the multiqubit decoherence processes, generally called *crosstalk*, pose more serious challenges [58]. Even two-qubit operations have an order of magnitude higher error rate than do single-qubit operations. This makes it difficult to entangle a large number of qubits without errors. Various error-mitigation techniques [59, 60] have been developed for the near term. In the long term, quantum error correction using logical operations, briefly mentioned in Section 3.2, will be necessary [43, 61]. However, this requires multiple orders of magnitude more physical qubits than available on today's NISQ systems, and native operations must have sufficiently low error rates. Thus, algorithms executed on current hardware must contend with the presence of noise and are typically limited to low-depth circuits. In addition, current quantum error correction techniques theoretically combat local errors [43, 58, 61]. The robustness of these techniques to non-local errors is still being investigated [62].

3.3.2 Connectivity

Quantum circuits need to be optimally mapped to the topology of a quantum device in order to minimize errors and total run time. With current quantum hardware, qubits can interact only with neighboring subsets of other qubits. Existing superconducting and trapped-ion processors have a fixed topology that defines the allowed connectivity. However, trapped-ion devices can change the ordering of the ions in the trap to allow for, originally, non-neighboring qubits to interact [63]. Since this approach utilizes a different degree of freedom from that used to store the quantum information for computation, technically the device can be viewed as having an all-to-all connected topology. For existing superconducting processors, the positioning of the qubits cannot change. As a result, two-qubit quantum operations between remote qubits have to be mediated by a chain of additional two-qubit SWAP gates via the qubits connecting them. Moreover, the two-qubit gate error of current NISQ devices is high. Therefore, quantum circuit optimizers and quantum hardware must be developed with these limitations in mind. Connectivity is also a problem with current quantum annealers, which is discussed in Section 6.1.1.

3.3.3 Gate Speed

Having fast quantum gates is important for achieving quantum supremacy and quantum advantage with NISQ devices in the quantum circuit model. However, some types of quantum devices are particularly slow, for example trapped-ion quantum processors, compared with superconducting devices, although these devices typically have lower gate error rates. There is a well-known trade-off between space, speed, and fidelity. Execution time is particularly relevant for algorithms that require a large number of circuit execution repetitions, such as variational algorithms (Section 4.7) and sampling circuits. Error rates typically increase sharply if the gate time is reduced below a certain duration. Thus, one must find the right balance, which often requires tedious calibrations and fine-tuning.

3.3.4 Native Gates

Another important attribute, specific to gate-based quantum devices, is the set of available quantum gates that can be executed natively, namely, those that map to physical operations on the system (Section 3.2.1). The existence of a diverse universal set of native gates is crucial for designing short-depth high-fidelity quantum circuits. Developing advanced quantum compilers is, therefore, critical for efficiently mapping general quantum gates to the native gates of a particular device.

4 Foundational Quantum Algorithms

In this section we discuss foundational quantum algorithms that are the building blocks for more sophisticated algorithms. These algorithms have been extended to address problems in different domains, particularly finance, as described in Sections 5, 6, and 7. We briefly review the concepts from computational complexity theory that are relevant to the discussions in this survey. We refer the reader to the references for more rigorous discussions on the topic.

Computational complexity, the efficiency in time or space with which computational problems can be solved, is presented by using the standard asymptotic "Big-O" notation [64]: the set $\mathcal{O}(f(n))$ for an upper bound on complexity (worst case) and the set $\Omega(g(n))$ for a lower bound on complexity (best case). Also, $h(n) \in \Theta(f(n))$

if and only if $h \in \mathcal{O}(f(n))$ and $h \in \Omega(f(n))$.[4] Computational problems, which are represented as functions, are divided into *complexity classes*. The most common classes are defined based on *decision problems*, which can be represented as Boolean functions. P denotes the complexity class of decision problems that can be solved in *polynomial time* (i.e., the time required is upper bounded, in "Big-O" terms, by a polynomial in the input size) by a deterministic Turing machine (TM), in other words, a traditional classical computer [2, 37]. An amount of time or space required for computation that is upper bounded by a polynomial is called *asymptotically efficient*, whereas, for example, an amount upper bounded by an exponential function is not. However, asymptotic efficiency does not necessarily imply efficiency in practice; the coefficients or order of the polynomial can be large. NP denotes the class of decision problems for which proofs of correctness exist, called *witnesses*, that can be executed in polynomial time on a deterministic TM [37].

A problem is *hard* for a complexity class if any problem in that class can be reduced in polynomial time to it, and it is *complete* if it is both hard and contained in the class [65]. Problems in P can be solved efficiently on a classical or quantum device. While NP contains P, it also contains many problems not known to be efficiently solvable on either a quantum or classical device. #P is a set of counting problems. More specifically, it is the class of functions that count the number of witnesses that can be computed in polynomial time on a deterministic TM for NP problems [65]. BQP, which contains P, denotes the class of decision problems solvable in polynomial time on a quantum computer with bounded probability of error [2]. A common way to represent the complexity of quantum algorithms is by using *query complexity* [66]. Roughly speaking, the problem setup provides the algorithm access to functions that the algorithm considers as "black boxes." These are represented as unitary operators called *quantum oracles*. The asymptotic complexity is computed based on the number of calls, or *queries*, to the oracles.

The goal of quantum algorithms research is to develop quantum algorithms that provide *computational speedups*, a reduction in computational complexity. In practice, however, one commonly finds algorithms with improved efficiency without any proven reduction in complexity. These algorithms typically utilize *heuristics*. As mentioned in Section 3.3, a majority of algorithms for NISQ fall into this category. When one can theoretically show a reduction in asymptotic complexity, the algorithm has a *provable speedup* for the problem. In the discussions that follow, we emphasize the category into which each algorithm currently falls.

4.1 Quantum Unstructured Search

Grover's algorithm [67] is a quantum procedure for solving unstructured search with an asymptotic quadratic speedup, in terms of the number of oracle queries made, over the best known classical algorithm. The goal of unstructured search is as follows: Given an oracle $f : \mathcal{X} \mapsto \{0, 1\}$, find $w \in \mathcal{X}$ such that $f(w) = 1$. Sometimes w is also called a marked element. More specifically, the algorithm amplifies the probability of measuring the state $|w\rangle$ encoding w such that $f(w) = 1$. In Grover's algorithm, a marked state is identified by utilizing a phase oracle, $O_f|x\rangle = (-1)^{f(x)}|x\rangle$, which can be a composition of an oracle computing f into a register and a conditional phase gate. One can represent f by a unitary utilizing techniques for reversible computation [2]. Classically, in the worst case f will be evaluated on all items in the set, $\mathcal{O}(N)$ queries. Grover's algorithm makes $\mathcal{O}(\sqrt{N})$. The complexity stated for this algorithm is said to be in terms of quantum query complexity. In fact, this algorithm is optimal for unstructured search [68].

Along with an efficient unitary simulating f, the algorithm requires the ability to query uniform superpositions of states representing elements of $\mathcal{X}$. This criterion is typically referred to as *quantum access to classical data* [69]. The general case can be achieved by using quantum random access memory (qRAM) [70]. As of this writing, the problems associated with constructing a physical qRAM have not been solved. However, a uniform superposition over computational basis states (i.e., with Hadamard gates, Section 3.2.1) can be used for the simple case where $\mathcal{X} = \{0, \ldots, N-1\}$. Moreover, hybrid techniques have been developed to deal with larger classical data sets without qRAM [71]. Ambainis [72] developed a variable-time version of the algorithm that more efficiently handles cases when queries can take different times to complete. We note that loading data onto a quantum computer is generally exponentially expensive. It is believed that a quantum computer may help with computing small but complex datasets, which are computationally easy to load onto a quantum computer.

4.2 Quantum Amplitude Amplification

The quantum amplitude amplification (QAA) algorithm [73] is a generalization of Grover's algorithm (Section 4.1). Suppose an oracle O_ϕ exists that marks a quantum state $|\phi\rangle$ (e.g., O_ϕ only acts nontrivially on $|\phi\rangle$, say, by adding a phase shift by π). Also, assume there is a unitary U and $|\Phi\rangle := U|\psi\rangle$ for some input state $|\psi\rangle$. In

[4] Let $*$ signify $\mathcal{O}$, Ω, or Θ. It is common practice to abuse notation and use the symbol $=$ instead of $\in$ to signify $g(n)$ is in $*(f(n))$ [64]. Similarly, it is common to interchange the phrases "the complexity is $*(f(n))$" and "the complexity is in $*(f(n))$."

addition, if $|\Phi\rangle$ has nonzero overlap with the marked state $|\phi\rangle$, namely, $\langle\Phi|\,\phi\rangle \neq 0$, then without loss of generality $|\Phi\rangle = \sin(\theta_a)\,|\phi\rangle + \cos(\theta_a)|\phi^{\perp}\rangle$, where $\langle\phi|\,\phi^{\perp}\rangle = 0$ and $\sin(\theta_a) = \langle\Phi|\,\phi\rangle$. For example, in the case of Grover's algorithm, $|\Phi\rangle$ is a uniform superposition over states corresponding to the elements of $\mathcal{X}$, and $|\phi\rangle$ is the marked state. QAA will amplify the amplitude $\sin(\theta_a)$ and as a consequence the probability $\sin^2(\theta_a)$ of measuring $|\phi\rangle$ (using an observable with $|\phi\rangle$ as an eigenstate) to $\Omega(1)$[5] using $\mathcal{O}(\frac{1}{\sin(\theta_a)})$ queries to U and O_ϕ. This is done utilizing a series of interleaved reflection operators involving U, $|\psi\rangle$, and O_ϕ [73].

This algorithm can be understood as a quantum analogue to classical probabilistic boosting techniques and achieves a quadratic speedup [73]. Classically, $\mathcal{O}(\frac{1}{\sin^2(\theta_a)})$ samples would be required, in expectation, to measure $|\phi\rangle$ with $\Omega(1)$ probability. QAA is an indispensable technique for quantum computation because of the inherit randomness of quantum mechanics. Improvements to the algorithm have been made to handle issues that occur when one cannot perform the required reflection about $|\psi\rangle$ [74], the number of queries to make (which requires knowing $\sin(\theta_a)$) is not known [75, 76], and O_ϕ is imperfect with bounded error [77]. The last two improvements are also useful for Grover's algorithm (Section 4.1). A version also exists for quantum algorithms (i.e., a unitary U) that can be decomposed into multiple steps with different stopping times [78].

4.3 Quantum Phase Estimation

The quantum phase estimation (QPE) algorithm [79] serves as a critical subroutine in many quantum algorithms such as HHL (Section 4.5). Given a desired additive error of δ, QPE makes $\mathcal{O}(\frac{1}{\delta})$ queries to a unitary U to produce an estimate $\tilde{\theta}_j$ of the phase $\theta_j \in [0, 1)$ of one of the eigenvalues of U, $e^{i2\pi\theta_j}$, such that $|\theta_j - \tilde{\theta}_j| \leq \delta$. The probability of successfully measuring an estimate $\tilde{\theta}_j$ to the actual eigenvalue phase θ_j within error δ is $\Omega(1)$ and can be boosted to $1 - \epsilon$ by adding and then discarding $\mathcal{O}(\log(1/\epsilon))$ ancillary qubits [2, 80]. Alternatively, it can be boosted to $1 - \frac{1}{\text{poly}(n)}$ by using $\mathcal{O}(\log(n))$ repetitions of QPE and taking the most frequent estimate, according to standard Chernoff bounds [81]. This brings the overall query complexity to $\mathcal{O}(\frac{1}{\delta\epsilon})$ and $\mathcal{O}(\frac{\log(n)}{\delta})$, respectively. In the cases where the actual eigenvalue phase θ_j, can be represented with finite precision, QPE can perform the transformation $|v_j\rangle\,|0\rangle \mapsto |v_j\rangle\,|\theta_j\rangle$, when $|v_j\rangle$ is an eigenvector of U. A more detailed explanation of the algorithm and what happens when the eigenvalue phases cannot be perfectly represented with the specified precision, δ, is provided by Nielsen and Chuang [2].

4.4 Quantum Amplitude Estimation

The quantum amplitude estimation (QAE) algorithm [73] estimates the total probability of measuring states marked by a quantum oracle. Consider the state $|\Phi\rangle$, oracle O_ϕ, unitary U, and amplitude $\sin(\theta_a)$ as defined in Section 4.2. QAE utilizes $\mathcal{O}(\frac{1}{\delta\sin(\theta_a)})$ queries to U and O_ϕ to estimate $\sin^2(\theta_a)$ to relative additive error $\delta\sin^2(\theta_a)$ [81]. The algorithm utilizes QPE (Section 4.3) as a subroutine, which scales as $\mathcal{O}(\frac{1}{\delta})$, for desired additive error δ. Some variants of QAE do not make use of QPE [82–84], including a variational (Section 4.7) one [85], and are potentially more feasible on near-term devices. An important application of QAE to finance is to perform Monte Carlo integration, discussed in Section 5.1. Other applications of QAE include estimating the probability of success of a quantum algorithm and counting.

4.5 Quantum Linear System Algorithms

The quantum linear systems problem (QLSP) is defined as follows Given an $\mathcal{O}(1)$–sparse invertible Hermitian matrix $A \in \mathbb{R}^{N\times N}$ and a vector $\vec{b} \in \mathbb{R}^N$, output a quantum state $|x\rangle$ that is the solution to $A\vec{x} = \vec{b}$ up to a normalization constant with bounded error probability [86]. While this definition requires the sparsity to be independent of the dimensions of the matrix, the quantum linear system algorithms (QLSAs) have a polynomial dependence on sparsity. As can be understood by viewing the time complexities of these algorithms, we can allow for $\mathcal{O}(\text{polylog}(N))$–sparse matrices, which is what is meant when we simply say "a sparse matrix" [87]. The $\mathcal{O}(1)$-sparsity specification comes from the decision problem version of QLSP that is BQP-complete [86, 88].

The Harrow–Hassidim–Lloyd (HHL) algorithm [88] is the first algorithm invented for solving the QLSP. It provides an exponential speedup in the system size N for well-conditioned matrices (for QLSP this implies a condition number in $\mathcal{O}(\text{polylog}(N))$) [89], over all known classical algorithms for a simulation of this problem. For matrix sparsity s and condition number κ, HHL runs with worst-case time complexity $\mathcal{O}(s^2\kappa^2\log(N)/\epsilon)$,

[5] $\Omega(1)$ probability means the probability that the event occurs is at least some constant. This is used to signify that the desired output of the algorithm occurs with a probability that is independent of the variables in the algorithm's time complexity or query complexity. In addition, this implies an asymptotically efficient amount of repetitions (classical probabilistic boosting) can be used to increase the probability of success close to 1 (e.g., see Section 4.3).

for desired error ϵ. This complexity was computed under the assumption that a procedure based on higher-order Suzuki–Trotter methods [90, 91] was used for sparse Hamiltonian simulation. However, a variety of more efficient techniques have been developed since the paper's inception [92, 93]. These potentially reduce the original HHL's quadratic dependence on sparsity. HHL's query complexity, which is independent of the complexity of Hamiltonian simulation, is $\mathcal{O}(\kappa^2/\epsilon)$ [94].

Alternative quantum linear systems solvers also exist that have better dependence on some of the parameters, such as almost linear in the condition number from Ambainis [78] and polylogarithmic dependence on $1/\epsilon$ for precision from Childs, Kothari, and Somma (CKS) [87]. In addition, Wossnig et al. [95] utilized the quantum singular value estimation (QSVE) algorithm of Kerenedis and Prakash [69] to implement a QLS solver for dense matrices. This algorithm has $\mathcal{O}(\sqrt{N}\text{polylog}(N))$ dependence on N and hence offers no exponential speedup. However, it still obtains a quadratic speedup over HHL for dense matrices. Following this, Kerenedis and Prakash generalized the QSVE-based linear systems solver to handle both sparse and dense matrices and introduced a technique for spectral norm estimation [96]. In addition, the quantum singular value transform (QSVT) framework provides methods for QLS that have the improved dependencies on κ and $1/\epsilon$ mentioned above [97, 98]. The QSVT framework also provides algorithms for a variety of other linear algebraic routines such as implementing singular-value-threshold projectors [69, 99] and matrix-vector multiplication [97]. As an alternative to QLS based on the QSVT, Costa et al. [94] devised a discrete-time adiabatic approach [100] to the QLSP that has optimal [88] query complexity: linear in κ and logarithmic in $1/\epsilon$. Since QLS algorithms invert Hermitian matrices, they can also be used to compute the Moore–Penrose pseudoinverse of an arbitrary matrix. This requires computing the Hermitian dilation of the matrix and filtering out singular values that are near zero [88, 99].

While solving the QLSP does not provide classical access to the solution vector, this can still be done by utilizing methods for vector-state tomography [101]. Applying tomography would no longer allow for an exponential speedup in the system size N. However, polynomial speedups using tomography for some linear-algebra-based algorithms are still possible (Section 6.2). In addition, without classical access to the solution, certain statistics, such as the expectation of an observable with respect to the solution state, can still be obtained without losing the exponential speedup in N. Providing quantum access to A and $\vec{b}$ is a difficult problem that can, in theory, be dealt with by using quantum models for data access. Two main models of data access for quantum linear algebra routines exist [97]: the sparse-data access model and the quantum-accessible data structure. The sparse-data access model provides efficient quantum access to the nonzero elements of a sparse matrix. The quantum-accessible data structure, suitable for fixed-sized, non-sparse inputs, is a classical data structure stored in a quantum memory (e.g., qRAM) and provides efficient quantum queries to its elements [97].

For problems that involve only low-rank matrices, classical Monte Carlo methods for numerical linear algebra (e.g., the FKV algorithm [102]) can be used. These techniques have been used to produce classical algorithms that have an asymptotic exponential speedup in dimension for various problems involving low-rank linear algebra (e.g., some machine learning problems) [103, 104]. As of this writing, these classical *dequantized* algorithms have impractical polynomial dependence on other parameters, making the quantum algorithms still useful. In addition, since these results do not apply to sparse linear systems, there is still the potential for provable speedups for problems involving sparse, high-rank matrices.

4.6 Quantum Walks

Formally, random walks are discrete-time stochastic processes formed through the summation of independent and identically distributed random variables [105] and play a fundamental role in finance [106]. In the limit, such discrete-time processes approach the continuous-time Wiener process [107]. One type of stochastic process whose logarithm follows a Wiener process is geometric Brownian motion (GBM), commonly used to model market uncertainty. Random walks have also been generalized to the vertices of graphs [108]. In this case they can be viewed as finite-state Markov chains over the set of vertices [109]. Quantum walks are a quantum mechanical analogue to the processes mentioned above [110]. Generally, quantum walks in discrete time can be viewed as a Markov chain over its *edges* (i.e., a quantum analogue of a classical bipartite walk) [111, 112] and consist of a series of interleaved reflection operators. This approach generalizes QAA (Section 4.2) and Grover's algorithm (Section 4.1) [113, 114]. Discrete-time quantum walks have be shown to provide polynomially faster mixing times [115–117] as well as polynomially faster hitting times for Markov-chain-based search algorithms [111, 112, 118–122]. Alternatively, there are quantum walks in continuous time [123, 124], which have certain provable advantages [125], using a different notion of hitting time. A similar result, using this notion, has also been shown for discrete-time quantum walks [126]. In addition, the various mixing operators used in the Quantum Approximate Optimization Algorithm (QAOA, Section 6.1.2) can be viewed as continuous-time quantum walks connecting feasible solutions [127]. There have been multiple proposed unifications of quantum walks in continuous and discrete time [128, 129], something that is true for the classical versions [107]. For a comprehensive review of quantum walks, see the survey by Venegas–Andraca [129]; and for an overview and a discussion of connections between the various quantum-walk-based search algorithms, see the work of Apers et al. [119].

4.7 Variational Quantum Algorithms

Variational quantum algorithms (VQAs) [130], also known as classical-quantum hybrid algorithms, are a class of algorithms, typically for gate-based devices, that modify the parameters of a quantum (unitary) procedure based on classical information obtained from running the procedure on a quantum device. This information is typically in the form of a cost function dependent on the expectations of observables with respect to the state that the quantum procedure produces. Generally, the training of quantum variational algorithms is NP-hard [131] which withholds us from reaching arbitrarily good approximate local minima. In addition, these methods are heuristic. The primordial quantum-circuit-based variational algorithm is called the variational quantum eigensolver (VQE) [132] and is used to compute the minimum eigenvalue of an observable (e.g., the ground-state energy of a physically realizable Hamiltonian). However, it is also applicable to optimization problems (Section 6.1.3) and both linear (e.g., quantum linear systems [133, 134]) and nonlinear problems (Section 5.2.2). VQE utilizes one of the quantum variational principles based on the Rayleigh quotient of a Hermitian operator (Section 6.1.3). The quantum procedure that provides the state, called the ansatz, is typically a parameterized quantum circuit (PQC) built from Pauli rotations and two-qubit gates (Section 3.2.1). For VQAs in general, finding the optimal set of parameters is a non-convex optimization problem [135]. A variation of this algorithm, specific to combinatorial optimization problems, known as the quantum approximate optimization algorithm (QAOA) [136], utilizes the alternating operator ansatz [137] and is typically used for observables that are diagonal in the computational basis. There are also variational algorithms for approximating certain dynamics, such as real-time and imaginary-time quantum evolution that utilize variational principles for dynamics [138] (e.g., McLachlan's principle [139]). Furthermore, variational algorithms utilizing PQCs and various cost functions have been applied to machine learning tasks (Section 7.6) [140]. Problems using VQAs are seen as one of the leading potential applications of near-term quantum computing.

4.8 Adiabatic Quantum Optimization

Adiabatic quantum optimization [141], commonly referred to as quantum annealing (QA), is an AQC (Section 3.2.2) algorithm for quadratic unconstrained binary optimization (QUBO). QUBO is an NP-hard combinatorial optimization problem. These problems are discussed in Section 6.1. The main reason behind the interest in QA is that it provides a nonclassical heuristic, called quantum tunneling, that is potentially helpful for escaping from local minima. Its closest classical analogue is simulated annealing [142], a Markov chain Monte Carlo method that was inspired by classical thermodynamics and uses temperature as a heuristic to escape from local minima.[6] So far QA has proven useful for solving QUBOs with tall yet narrow peaks in the cost function [143, 144]. The overall benefit of QA is still a topic of ongoing research [144, 145]. However, the current scale of quantum annealers allows for potential applicability in the near term [146].

5 Stochastic Modeling

Stochastic processes are commonly used to model phenomena in physical sciences, biology, epidemiology, and finance. In finance, stochastic modeling is often used to help make investment decisions, usually with a goal of maximizing return and minimizing risks. Quantities that are descriptive of the market condition, including stock prices, interest rates, and their volatilities, are often modeled by stochastic processes and represented by random variables. The evolution of such stochastic processes is governed by stochastic differential equations (SDEs), and stochastic modeling aims to solve the SDEs for the expectation value of a certain random variable of interest, such as the expected payoff of a financial derivative at a future time, which determines the price of the derivative.

Although analytical solutions for SDEs are available for a few simple cases, such as the well-known Black–Scholes equation for European options [106], a vast majority of financial models involve SDEs of more complex forms that have to resort to numerical approaches.

In the following subsections, we review two commonly used numerical methods for solving SDEs, namely, Monte Carlo integration (MCI, Section 5.1) and numerical solutions of differential equations (ODEs/PDEs, Section 5.2), and we discuss respective quantum solutions and potential advantages. Then, in Section 5.3, we show example financial applications for these quantum solutions.

5.1 Monte Carlo Integration

Monte Carlo methods utilize sampling to approximate the solutions to problems that are intractable to solve analytically or with numerical methods that scale poorly for high-dimensional problems. Classical Monte Carlo

[6] Somma et al. [120] produced a quantum algorithm with provable speedup, in terms of the spectral gap, for simulated annealing using quantum walks (Section 4.6).

methods have been used for inference [147], integration [148], and optimization [149]. Monte Carlo integration (MCI), the focus of this subsection, is critical to finance for pricing and risk predictions [12]. These tasks often require large amounts of computational power and time to achieve the desired precision in the solution. Therefore, MCI is another key technique, heavily used in finance, for which a quantum approach is appealing. Interestingly, there exists a quantum algorithm with a proven advantage over classical Monte Carlo methods for numerical integration.

In stochastic modeling, MCI typically is used to estimate the expectation of a quantity that is a function of other random variables. The methodology usually starts with a stochastic model (e.g. SDEs) for the underlying random variables from which samples can be taken and the corresponding values of the target quantity subsequently evaluated given the samples drawn. For example, consider the following sequence of random variables, $X_0, \ldots, X_T$. These random variables are often assumed to follow a diffusion process [107], where each X_t with $t \in \{0, 1, \ldots, T\}$ represents the value of the quantity of interest at time t, and the collective values of the random variables from the outcome of a single drawing are known as a sample path. Suppose the quantity whose expectation we want to compute is a function of this process at various time points: $g(X_0, \ldots, X_T)$. In order to estimate the expectation, many sample paths are drawn, and sample averages of g are computed. The estimation error decays as $\mathcal{O}(\frac{1}{\sqrt{N_s}})$ independent of the problem dimension, where N_s is the number of paths taken. This is in accordance with the law of large numbers and Chebyshev's inequality [148].

Quantum MCI (QMCI) [150–152] provides a quadratic speedup for Monte Carlo integration by making use of the QAE algorithm (Section 4.4).[7] With QAE, using the example from the previous paragraph, the error in the expectation computation decays as $\mathcal{O}(\frac{1}{N_q})$, where N_q is the number of queries made to a quantum oracle that computes $g(X_0, \ldots, X_T)$. This is in contrast to the complexity in terms of the number of *samples* mentioned earlier for MCI. Thus, if samples are considered as classical queries, QAE requires quadratically fewer queries than classical MCI requires in order to achieve the same desired error.

The general procedure for Monte Carlo integration utilizing QAE is outlined below [154]. Let Ω be the set of potential sample paths ω of a stochastic process that is distributed according to $p(\omega)$, and $f : \Omega \mapsto A$ is a real-valued function on Ω, where $A \subset \mathbb{R}$ is bounded. The task is to compute $\mathbb{E}[f(\omega)]$ [152], which can be achieved with QAE in three steps as follows.

1. Construct a unitary operator $\mathcal{P}_l$ to load a discretized and truncated version of $p(\omega)$. The probability value $p(\omega)$ translates to the amplitude of the quantum state $|\omega\rangle$ representing the discrete sample path ω. In mathematical form, it is
$$\mathcal{P}_l |0\rangle = \sum_{\omega \in \Omega} \sqrt{p(\omega)}\, |\omega\rangle . \tag{1}$$

2. Convert f into a normalized function $\tilde{f} : \Omega \mapsto [0, 1]$, and construct a unitary $\mathcal{P}_f$ that computes $\tilde{f}(\omega)$ and loads the value onto the amplitude of $|\omega\rangle$. [8] The resultant state after applying $\mathcal{P}_l$ and $\mathcal{P}_f$ is
$$\mathcal{P}_f \mathcal{P}_l |0\rangle = \sum_{\omega \in \Omega} \sqrt{(1 - \tilde{f}(\omega))p(\omega)}\, |\omega\rangle |0\rangle + \sqrt{\tilde{f}(\omega)p(\omega)}\, |\omega\rangle |1\rangle . \tag{2}$$

3. Using the notation from Section 4.4, perform quantum amplitude estimation with $U = \mathcal{P}_f \mathcal{P}_l$ and an oracle, O_ϕ, that marks states with the last qubit being $|1\rangle$. The result of QAE will be an approximation to $\sum_{\omega \in \Omega} \tilde{f}(\omega)p(\omega) = \mathbb{E}[\tilde{f}(\omega)]$. This value can be estimated to a desired error ϵ utilizing $\mathcal{O}(\frac{1}{\epsilon})$ evaluations of U and its inverse [152]. Then scale $\mathbb{E}[\tilde{f}(\omega)]$ to the original bounded range, A, of f to obtain $\mathbb{E}[f(\omega)]$.

Variants [152] of QMCI also exist that can be applied when $f(\omega)$ has bounded L^2 norm and bounded variance (both absolutely and relatively). The problem of estimating a multidimensional random variable is called multivariate Monte Carlo estimation. Quantum algorithms that obtain similar quadratic speedups in terms of error ϵ have been proposed by Cornelissen and Jerbi [156]. They solved this problem in the framework of Markov reward processes, and thus it has applicability to reinforcement learning (Section 7.7).

As mentioned earlier (Section 4.1), preparing an arbitrary quantum state is a difficult problem. In the case of QMCI, we need to prepare the state in (2). The Grover–Rudolph algorithm [157] is a promising technique for loading efficiently integrable distributions (e.g., log-concave distributions). It requires the ability to compute the probability mass over subregions of the sample space. However, it has been shown that when numerical

[7] To disambiguate, there are classical computational techniques called *quantum Monte Carlo* methods used in physics to classically simulate quantum systems [153]. These are not the topic of this discussion.

[8] In general, quantum arithmetic methods [155] can be used to load $\arcsin\left(\sqrt{\tilde{f}(\omega)}\right)$ into a register and perform controlled rotations to load $\sqrt{\tilde{f}(\omega)}$ onto the amplitudes [154].

methods such as classical MCI are used to integrate the distribution, QMCI does not provide a quadratic speedup when using this state preparation technique [158]. Additionally, variational methods (Section 4.7) have been developed to load distributions [159], such as normal distributions [154]. Geometric Brownian motion (GBM) is an example of a diffusion process whose solution is a series of log-normal random variables. These variables can be implemented by utilizing procedures for loading standard normal distributions, followed by quantum arithmetic [154, 155].

While GBM is common for modeling stock prices, it is often considered unrealistic [160] because it assumes a constant volatility. In addition, unlike realistic models, the SDE of GBM can be solved in closed form; that is, the distribution at every point in time is known, which avoids the need to use approximations involving the SDE to simulate the process. The local volatility (LV) model is one approach for accounting for a changing volatility by making it a function of the current price and time step. However, the corresponding SDE does not have a closed-form solution, and hence this means numerical methods such as Euler–Maruyama schemes [161] need to be used to simulate paths of the SDE with discretized time steps. Kaneko et al. [162] proposed quantum circuits implementing the required arithmetic for simulating the LV model, namely, the operation $\mathcal{P}_l$, based on improved techniques for simulating stochastic dynamics on a quantum device [163]. However, the discretized simulation of the SDE introduces additional error and complexities when computing expectations of functions of the stochastic process the SDE defines. Popular classical approaches to this problem are known as Multilevel Monte Carlo (MLMC) methods [164], which can be used to approximately recover the error scaling of classical single-level MCI. An et al. [165] proposed a quantum-enhanced version of MLMC that utilizes QMCI as subroutine to compute all expectations involved. This allowed them to approximately recover the error scaling of QMCI, and the approach benefits from being applicable to a wider variety of stochastic dynamics beyond GBM and LV. With regard to implementing payoff functions, Herbert [166] developed a near-term technique for functions that can be approximated well by truncated Fourier series. Furthermore, if fault tolerance (Section 3) is to be taken into account, advancements in the field of quantum error correction are required in order to realize quantum advantage [167].

Also worth noting are non-unitary methods for efficient state preparation [44, 168, 169]. These methods are usually incompatible with QAE, however, because of the non-invertibility of the operations involved. The methods would be more useful in a broader context where the inverse of the state preparation operation is not required; this topic is out of the scope of this survey.

5.2 Numerical Solutions of Differential Equations

While Monte Carlo methods are widely used for modeling stochastic processes, other numerical methods are also commonly used to calculate properties of stochastic systems. In fact, according to the Feynman–Kac formula [170, 171], the expectation of certain random variables, whose evolution is described by SDEs, can be formulated as the solution to parabolic partial differential equations (PDEs). This connection between SDEs and PDEs allows one to study stochastic processes using deterministic methods. Specifically, this enables an alternative to Monte Carlo methods for studying SDEs. A prime example of an alternative to MCI, applicable to certain problems, is the Black–Scholes PDE, which forms the keystone for pricing financial derivatives.

Classical numerical methods [9] for solving PDEs and ordinary differential equations (ODEs) usually require discretization of the domain on which solutions are sought. Specifically, the finite difference method (FDM) solves a PDE by approximating the solution and its derivatives on a predefined grid in both space and time and advancing it in time on the grid. Similarly, the finite volume method (FVM) employs a grid to divide the space into volumes, on which the function to be solved is integrated and the volume integrals are converted to surface integrals using the divergence theorem. These surface integrals are then connected by the original PDE. Another grid-based method, the finite element method (FEM), uses functions to form a basis of solutions on the subdomains divided by the grid. These basis functions are then assembled into an approximate for the solution in the entire domain.

In addition to grid based methods mentioned above, spectral methods may also be used to solve PDEs. Spectral methods [173] expand the solution into a linear combination of basis functions in the entire domain, whose spatial derivatives can be computed exactly. Although spectral methods do not require a grid when computing the spatial derivatives, numerical solution on a computer still requires evaluation of the functions on a grid. Moreover, spectral methods are often expensive to implement for PDEs with nonlinear terms. The reason is that spectral methods often require Fourier transforms, which have a computational cost of $O(N \log N)$, with N being the number of points on which the solution is evaluated. In contrast, grid-based spatial differentiation methods usually have a complexity of $O(N)$.

All these techniques can easily become computationally intractable for complex and high-dimensional problems, since they often require extremely large grid sizes to achieve desired accuracy and numerical stability in the

[9] For details on numerical methods for PDEs, see, for example, [172].

solution. Specifically, these algorithms have complexities that scale at least linearly with the number of points N on which the solution is to be evaluated and exponentially in the number of dimensions d of the spatial variable.

On the other hand, quantum algorithms often address the same problem by representing a vector of size N using $O(\log N)$ space, and operating on it in potentially $O(\text{poly}(\log N))$ time.

5.2.1 Quantum-linear-system-based algorithms

Since classical algorithms for numerical solutions of PDEs such as the ones mentioned above often resort to the solution of linear systems, a natural candidate for quantum algorithms for PDEs is to employ the quantum linear system algorithms as mentioned in Section 4.5.

Because of the linear nature of QLS, QLSAs have been used to solve linear PDEs and ODEs [174–178]. These algorithms often consider differential equations with the following form:

$$\mathcal{L}(u(\boldsymbol{x})) = f(\boldsymbol{x}),$$

where $\boldsymbol{x} \in \mathbb{C}^d$ is a d-dimensional vector, $f(\boldsymbol{x}) \in \mathbb{C}$ is a scalar function that accounts for the inhomogeneity in the PDE, $u(\boldsymbol{x}) \in \mathbb{C}$ is the scalar function that we are solving, and $\mathcal{L}$ is a linear differential operator. The differential equations are transformed into a set of linear equations by using the same approaches as the classical algorithms mentioned above for computing numerical derivatives given a discretization. The resultant linear system can then be solved by QLSAs. This approach will potentially give the PDE solver algorithm a complexity of $O(\text{poly}(d, \log(1/\epsilon)))$, where ϵ is the error tolerance, which is an exponential improvement in d compared with that of the best-known classical algorithms.

In particular, FDM-based quantum algorithms [174–177, 179] use quantum states $|a\rangle = \sum_{\boldsymbol{x}} u(\boldsymbol{x}) |\boldsymbol{x}\rangle$ and $|b\rangle = \sum_{\boldsymbol{x}} f(\boldsymbol{x}) |\boldsymbol{x}\rangle$ to encode the functions $u(\boldsymbol{x})$ and $f(\boldsymbol{x})$ (up to a normalization factor), with the N computational basis states representing the N discretized points in $\boldsymbol{x}$ and amplitudes proportional to the scalar values of the functions at the respective points. A finite-difference scheme that defines how the derivatives are approximated using values from neighboring grid points is then used to convert $\mathcal{L}$ into an $N \times N$ matrix operator A acting on $|a\rangle$, and hence the solution of the differential equations becomes a QLSP:

$$A |a\rangle = |b\rangle ,$$

where $|a\rangle$ is the solution we would like to obtain.

Similarly, quantum versions of FVM [180] and FEM [178] also use amplitude-encoded quantum states to represent the solution or the basis functions used to approximate the solution, and generate the matrix A based on the relation of the grid points as defined by differential equations and the respective methods. The same applies to quantum algorithms based on spectral methods, in which the discretization is done in the spectral space [177].

To extend the QLSA-based algorithms to the solution of nonlinear differential equations, linearization of the equations is usually required. One approach is Carleman linearization [181, 182], which approximates the nonlinear functions in an ODE with truncated Taylor expansions of them, so that the nonlinear ODE can be approximated by a system of linear ODEs of the different powers of the solution. Specifically, Liu et al. [183] utilize Carleman linearization to convert dissipative quardratic ODEs into linear ODEs, which are then solved by QLSA. The overall complexity achieved by this algorithm is $O(qT^2\text{poly}(\log T, \log d, \log 1/\epsilon)/\epsilon)$, where T is the evolution time requested for the solution, q measures the decay of the solution, and d and ϵ are the same as defined earlier.

Another approach for implementing nonlinearity is to build multiple copies of the quantum state representing the solution, apply linear transformations in the expanded space to simulate the nonlinearities in the differential equations, and then trace out the additional copies to achieve the effective nonlinear transformation on a single copy of the quantum state [184, 185]. In particular, Lloyd et al. [185] combined the methods for simulating the dynamics of the nonlinear Schrödinger equation with QLSA-based quantum linear differential equation solvers to solve nonlinear ODEs with an m-th order polynomial nonlinear term. It was shown that by using a sufficiently large number of copies, n, of the quantum state that scales quadratically with the evolution time T, the approximation error ϵ is suppressed by a factor of $1/n$.

While the aforementioned linearization approaches all involve approximations, the Koopman–von Neumann approach [186] is another linearization method that converts nonlinear ODEs to linear transport equations, and the conversion is exact. Specifically, Jin et al. [187] showed that a system of d nonlinear ODEs can be exactly converted into a $d + 1$ dimensional linear transport PDE, which can be solved with QLSA-based linear PDE solvers. Although this approach applies generally to all nonlinear ODEs, it was shown that the only possible quantum advantage one may get in the general case is when we have multiple ($M \gg 1$) initial data points on which an ensemble average of some physical variables are to be estimated.

In the same work, it is also demonstrated that for certain types of nonlinear PDEs such as the Hamilton–Jacobi and hyperbolic PDEs, an exact linearization exists using the level set formalism [188], which converts the

$d+1$-dimensional (d spatial dimensions and time) nonlinear PDE to a $2d+1$- and a $d+2$-dimensional PDE, respectively.

For a general nonlinear PDE with $d+1$ dimensions, one may discretize the nonlinear PDE and convert it to a system of nonlinear ODEs [187], and then use the linearization techniques mentioned above to further convert the nonlinear ODEs to linear PDEs or ODEs. Therefore, same as the approach for general nonlinear ODEs, this algorithm will have a quantum advantage only when M is large for general nonlinear PDEs.

We note that in all of the QLSA-based algorithms discussed above, the output will be in the form of a quantum state, which is generally difficult to be accessed classically without losing information or voiding the overall quantum speedup. Nevertheless, as discussed in Section 4.5, one can still efficiently obtain certain statistics of the solution without classical access to the quantum output state. This is usually the assumption of these QLSA-based algorithms.

5.2.2 Variational algorithms

In addition to QLSA-based PDE solvers, variational algorithms have been developed for solving linear and nonlinear PDEs. One approach is to transform the PDE to a Wick-rotated Schrödinger equation with an imaginary time variable and then use variational quantum imaginary time evolution (VarQITE; see Section 6.1.4) to solve the equation along the imaginary time axis. This approach has been demonstrated in solving linear PDEs such as the Black–Scholes equation [189], the Feynman–Kac equation, and the Ornstein–Uhlenbeck equation [190], and nonlinear PDEs such as the Hamilton-Jacobi and the Bellman equation [190].

In another approach proposed by Lubasch et al. [191], a variational circuit is built to construct the quantum state representing the solution of the PDE in an amplitude encoding. Nonlinearities are implemented by creating multiple copies of the variational circuit and hence the quantum state. The algorithm then employs tensor networks to efficiently calculate the linear operations acting on the solution functions, and the variational parameters are optimized based on cost functions built on the expectations of these linear operators on the quantum state, obtained by measurements at the end of the circuit.

While these two variational algorithms both encode the solution into a quantum state such that the computational basis states denote the discretized values of the spatial variable and the amplitudes are proportional to the corresponding values of the function, Kyriienko et al. [192, 193] proposed a different approach to encode the function $u(\boldsymbol{x})$. Quantum circuits in this approach start with a quantum feature map in which values of the function variables $\boldsymbol{x}$ are used as rotation angles in the circuit. The feature map is followed by a variational circuit parameterized by $\boldsymbol{\theta}$, which produces the quantum state $|u_{\boldsymbol{\theta}}(\boldsymbol{x})\rangle$. The encoded function value $u(\boldsymbol{x})$ is then given by the expectation value of a cost operator $\mathcal{C}$, namely,

$$u(\boldsymbol{x}) = \langle u_{\boldsymbol{\theta}}(\boldsymbol{x})| \, \mathcal{C} \, |u_{\boldsymbol{\theta}}(\boldsymbol{x})\rangle \, .$$

Derivatives of u may be approximated by shifting the $\boldsymbol{x}$ parameters following a finite-difference scheme or computed exactly by using quantum circuit differentiation with respect to $\boldsymbol{x}$. Using the outputs of the quantum circuits, a classical loss function is built to quantify how well the solution satisfies the PDE, and hence the best solution can be obtained by minimizing the loss function through adjusting $\boldsymbol{\theta}$ in the variational circuit.

Paine et al. [194] utilized the differentiable quantum circuit approach for simulating the quantile function of an SDE. The quantile function, whose evolution satisfies a PDE [195], is the inverse of the cumulative distribution function of the solution to the SDE and can be used for generating samples [196]. Kubo et al. [197] considered the trinomial-tree model [198] for SDEs. The evolution of the probability distribution can be represented as the solution to the Schrödinger equation with a non-Hermitian Hamiltonian and thus simulated with VarQITE. The authors also proposed methods for computing expectation values of functions of the SDE variables.

5.3 Financial Applications

Two main financial applications can be solved via the quantum algorithms presented for stochastic modeling: derivative pricing and risk modeling.

5.3.1 Derivative Pricing

The mathematical framework used for pricing financial assets uses techniques from measure-theoretic probability theory [199]. Specifically, the market is modeled as a filtered probability space $(\Omega, \mathfrak{F}, \mathbb{P})$, where the sample space Ω corresponds to potential states of the market, the filtration $\mathfrak{F} := \{\mathcal{F}_t\}_{t=0}^{\infty}$ corresponds to information and events that become observable over time (i.e., sub-σ-algebras), and $\mathbb{P}$ is the real-world market probability measure. The time t can be continuous or discrete. Nondeterministic financial quantities such as stock prices and interest rates are represented as $\mathfrak{F}$-adapted stochastic processes. Powerful mathematical properties can be derived when the market is assumed to be arbitrage free and complete. An arbitrage-free market is one in which there does not exist a riskless profit with no initial investment. The arbitrage-free assumption is equivalent to the existence of at

least one probability measure, equivalent to the real-world one $\mathbb{P}$, under which the discounted (stochastic) value process, $\{V_t\}_{t=0}^T$, of a portfolio of financial instruments is a martingale. Such a measure is called an equivalent martingale measure or *risk-neutral measure* $\mathbb{Q}$. The value process being a $\mathbb{Q}$ martingale, also called the fair value, implies that the current value of the portfolio is equal to the expected value at maturity:

$$V_t = \mathbb{E}_{\mathbb{Q}}[V_T|\mathcal{F}_t] = \mathbb{E}_{\mathbb{Q}}[C(t,T)|\mathcal{F}_t]. \tag{3}$$

The second equality follows since the definition of the fair value at maturity V_T, as seen from time t, is the discounted accumulated cash flows realized from time t until T, which from now on we denote $C(t,T)$. If the stronger assumption of a complete market is included, namely, all participants have access to all information that could be obtained about the market, then the risk-neutral measure is unique.

An important type of pricing problem in finance is the pricing of derivatives. A derivative is a contract that derives its value from another source (e.g., collection of assets, financial benchmarks) called the *underlying*, which is modeled by $\{X_t(\omega)\}_{t=0}^T$, where $\omega \in \Omega$. The value of a derivative is typically calculated by simulating the dynamics of the underlying and computing the payoff accordingly. The payoff is a sequence of Borel-measurable functions z_t, which map the evolution of the underlying to the (discounted) payoff process $Z_t := z_t(X_0, \ldots, X_t)$. The functions z_t are based on the contract. Thus, for derivatives, $C(t,T)$ is the accumulation of all Z_t occurring from t until maturity T. We can apply Equation (3) under a risk-neutral measure. Thus, the value of the derivative at t, V_t, is equal $\mathbb{E}[C(t,T)|\mathcal{F}_t]$. This justifies the usage of MCI for derivative pricing [12, 200], assuming sample paths follow the risk-neutral measure.

We next discuss the application of QMCI and quantum differential equation solvers to the pricing of financial derivatives. Specifically, we consider two types of derivatives: options and collateralized debt obligations (CDOs).

Options. An option gives the holder the right, but not the obligation, to purchase (call option) or sell (put option) an asset at a specified price (strike price) and within a given time frame (exercise window) in the future. EOne of the most well-known models for options is the Black–Scholes model [106]. This model assumes that the price of the underlying (risky) asset, $\{X_t\}_{t=0}^T$, evolves like a GBM with a constant volatility. The Black–Scholes PDE, which governs the evolution of the price of an option on the underlying asset, has a closed-form solution, called the Black–Scholes formula, for the simple European option. A European call option has a payoff that satisfies $z_t = 0, \forall t < T$, and $z_T(X_T) = e^{-rT}(X_T - K)^+$.[10] This payoff depends only on the state of the underlying at maturity; in other words, it is path independent. The interest rate r is the return of a risk-free asset, which is included in the Black–Scholes model. According to Equation (3), the fair value of the European option at time t is $V_t = e^{rt}\mathbb{E}_{\mathbb{Q}}[z_T(X_T)|X_t]$, and the Black–Scholes formula provides V_t in closed form for all t. However, many pricing tasks involve more complicated options that are often path dependent, and hence information about the asset at multiple time steps is required; that is, z_T depends on previous X_t. This is one of the reasons MCI-based approaches [201] are more widely used than those based on directly solving PDEs, since the path-dependent PDE can be complicated.

Stamatopoulos et al. [202] constructed quantum procedures for the uncertainty distribution loading, $\mathcal{P}_l$, and the payoff function, $\mathcal{P}_f$, for European options. The operator $\mathcal{P}_l$ loads a distribution over the computational basis states representing the price at maturity. The operator $\mathcal{P}_f$ approximately computes the rectified linear function, characteristic of the payoff for European options. QMCI has also been applied to path-dependent options such as barrier and Asian options [202, 203], and to multi-asset options [202]. For these options, prices at different points on a sample path or for different assets are represented by separate quantum registers, and the payoff operator $\mathcal{P}_f$ is applied to all of these registers. Pricing of more complicated derivatives with QAE can be found in the literature. For example, Chakrabarti et al. performed an in-depth resource estimation and error analysis for QAE applied to the pricing of autocallable options and target accrual redemption forwards (TARFs) [154].

Miyamoto and Kubo [204] utilized the FDM-based approach to solve the multi-asset Black–Scholes PDE accelerated by using QLSA, as mentioned in Section 5.2.1. However, since applying the quantum FDM method to the Black–Scholes PDE produces a quantum state encoding the value of the option at exponentially many different initial prices, the quantum speedup is lost if we need to read out a single amplitude. The authors avoided this issue by reading out the expected value of the option at a future time point under the probability distribution of the price of the underlying at the same time point, which is the current price of the option as a consequence of the value process being a martingale under the risk-neutral measure. This approach results in an exponential speedup in terms of the dimension of the PDE over the classical FDM-based approach. Linden et al. [205] compared various quantum and classical approaches for solving the heat equation, which the Black–Scholes PDE reduces to. The quantum approaches discussed make use of the QLSA-based PDE solvers and quantum walks (Section 4.6). Fontanela et al. [189] transformed the Black–Scholes PDE into the Schödinger equation with a non-Hermitian Hamiltonian. They utilized VarQITE to simulate the Wick-rotated Schödinger equation. Similarly, Alghassi et al. [190] applied imaginary time evolution to the more general Feynmann–Kac formula

[10] $(A)^+ := \max\{A, 0\}$

for linear PDEs. Alternatively, Gonzalez–Conde et al. [206] used unitary dilation, that is, unitary evolution in an expanded space, and postselection to simulate the nonunitary evolution associated with the non-Hermitian Hamiltonian that results from mapping the Black–Scholes PDE to the Schrödinger. They also mentioned the potential applicability beyond constant-volatility models.

Another prominent option is the American-style option [199]. While European, barrier, and Asian options allow the holder to exercise their right only at maturity, American options allow the holder to buy/sell the underlying asset at any point in time up to maturity, T. Thus, the buyer of the option needs to determine the optimal exercise time that maximizes the expected payoff. More generally, finding the optimal execution time is an optimal stopping problem [207]. In modern probability theory, a *stopping time* is an $\mathfrak{F}$-adapted stochastic process $\tau : \Omega \mapsto [0, \infty)$. The adaptedness of τ implies that making the decision to stop at time t only requires information obtained at time steps up to and including t. The corresponding stopped process at time t, using stopping time τ, is defined as $(Z_\tau)_t := Z_{\min(t,\tau(\omega))}$. This definition is used because the option payoff cannot change once the buyer has decided to exercise the option.

The American option pricing problem involves a sequence of stopping times. An optimal stopping time τ_t starting at time t consists of choosing to exercise at the current time when the payoff is greater than or equal to the expected payoff that could be obtained from continuing, that is, $\mathbb{E}[Z_{\tau_{t+1}}|X_t]$, and is called a continuation value. This recursive definition implies that finding the optimal stopping time can be formulated as solving a dynamic programming problem. The value of the American option is the maximum expected payoff; and since an optimal stopping time maximizes the expected payoff, this price is equal to $\mathbb{E}[Z_{\tau_0}|X_0]$, where the current time is 0. This is can be computed by using Monte Carlo integration with sampled payoffs, with discretized time steps, that have optimally stopped.[11] However, this also requires computing the continuation values. A popular approach to estimate continuation values is through least-squares regression using a fixed set of basis functions. The combined technique of regressing the conditional expectation values and sampling optimally-stopped payoffs to solve the dynamic programming problem is called the least-squares Monte Carlo method (LSM) [208].

Doriguello et al. [209] proposed a quantum version of LSM that makes use of QMCI. The authors presented unitary operators for computing the stopping time at time step t, τ_t, which was done by backtracking from time step T. The stopping time τ_{t-1} can be computed from the stopping time τ_t by using the definition of the dynamic program for the optimal stopping time. Note that although in the classical case, we can store all previously computed τ_t as we compute them backward in time, in the quantum case, we need to compute τ_t for each linear regression step by starting at the maturity date T. As a result, while the quantum LSM has an improved error dependence, it now depends quadratically on the maturity time T. Miyamoto [210] performed a similar quantization of LSM for Bermudan options, similar to American options but the exercise times are a discrete subset of time steps until maturity. Alghassi et al. [190] also mentioned the potential applicability of their VarQITE approach, mentioned in Section 5.2.2, to the pricing of American options and options with a stochastic volatility.

Collateralized Debt Obligations. A collaterialized debt obligation (CDO) is a derivative that is backed by a pool of loans and other assets that serve as collateral if the loan defaults. The tool is used to free up capital and shift risk. A typical CDO pool is divided into three tranches: equity, mezzanine, and senior. The equity tranche is the first to bear loss; the mezzanine tranche investors bear loss if the loss is greater than the first attachment point; and the senior tranche investors lose money if the loss is greater than the second attachment point. Although the senior tranche is protected from loss, other default events can cause the CDO to collapse, such as events that caused the 2008 financial crisis [211].

Conditional independence models [212] are often used for estimating the chances of parties defaulting on the credit in the CDO pool. In such models, given the systemic risk W, the default risks $B_1, \ldots, B_n$ of the assets involved are independent. The random variable W is then used as a latent variable to introduce correlations between the default risks. The distributions of the default risks and the systemic risk are usually either Gaussian [212] or normal-inverse Gaussian [213].

Tang et al. [211] presented quantum procedures for both the $\mathcal{P}_l$ and $\mathcal{P}_f$ operators used for QMCI. The goal was to utilize QAE to estimate the expected loss of a given tranche. $\mathcal{P}_l$ is a composition of rotations to load the uncorrelated probabilities of default (i.e., Bernoulli random variables). Then a procedure is used for loading the distribution of W, and controlled rotations are used to apply the correlations. $\mathcal{P}_f$ computes the loss under a given realization of the variables $B_1, \ldots, B_n$ and uses a quantum comparator to determine whether the loss falls within the specified tranche range. QAE is then used, instead of MCI, to compute the expectation of the total loss given that it falls within the tranche range.

[11] Technically, the discretization of time converts the American option into a Bermudan option.

5.3.2 Risk Modeling

Risk modeling is another crucial problem for financial institutions, since risk metrics can determine the probability and amount of loss under various financial scenarios. Risk modeling is also important in terms of compliance with regulations. Because of the Basel III regulations mentioned earlier, financial institutions need to calculate and meet requirements for their VaR, CVaR, and other metrics. However, the current classical approach of Monte Carlo methods [12] is computationally intensive. Similar to the pricing problem mentioned above, a quantum approach could also benefit this financial problem. In this subsection we explore the application of the presented quantum algorithms to computing risk metrics such as VaR and CVaR. We also review quantum approaches for sensitivity analysis and credit valuation adjustments.

Value at Risk. As first mentioned in Section 2.1.1, VaR and CVaR are common metrics used for risk analysis. Mathematically, VaR at confidence $\alpha \in [0, 1]$ is defined as $\mathrm{VaR}_\alpha[X] = \inf_{x \geq 0}\{x \mid \mathbb{P}[X \leq x] \geq \alpha\}$, usually computed by using Monte Carlo integration [214]. CVaR is defined as $\mathrm{CVaR}_\alpha[X] = \mathbb{E}[X \mid 0 \leq X \leq \mathrm{VaR}_\alpha[X]]$ [215]; α is generally around 99.9% for the finance industry. QAE can be used to evaluate VaR_α and CVaR_α faster than with classical MCI. The procedure to do so, by Woerner and Egger [215, 216], is as follows.

Similar to the CDO case above (Section 5.3.1), the risk can be modeled by using a Gaussian conditional independence model. Quantum algorithms for computing VaR_α and CVaR_α via QAE can utilize the same realization of $\mathcal{P}_l$ as for the CDO case [215]. Note that $\mathbb{P}[X \leq x] = \mathbb{E}[h(X)]$, where $h(X) = 1$ for $X \leq x$ and 0 otherwise. Thus $\mathcal{P}_f$ implements a quantum comparator similar to both of the previous applications discussed for derivatives. A bisection search can be used to identify the value of x such that $\mathbb{P}[X \leq x] \geq \alpha$ using multiple iterations of QAE. CVaR_α uses a comparator to check against the obtained value for VaR_α when computing the conditional expectation of X.

Economic Capital Requirement. Economic capital requirement (ECR) summarizes the amount of capital required to remain solvent at a given confidence level and time horizon [216]. Egger et al. [216] presented a quantum method using QMCI to compute ECR. Consider a portfolio of K assets with $L_1, \ldots, L_K$ denoting the losses associated with each asset. The expected value of the total loss, $\mathcal{L}$, is $\mathbb{E}[\mathcal{L}] = \sum_{m=1}^{K} \mathbb{E}[L_m]$. The losses can be modeled by using the same Gaussian conditional independence model discussed above. ECR at confidence level α is defined as

$$\mathrm{ECR}_\alpha[\mathcal{L}] = \mathrm{VaR}_\alpha[\mathcal{L}] - \mathbb{E}[\mathcal{L}],$$

where α is generally around 99.9% for the finance industry. The expected total loss $\mathbb{E}[\mathcal{L}]$ can be efficiently computed classically, so only the estimate of $\mathrm{VaR}_\alpha[\mathcal{L}]$ is computed by using QAE [216].

The Greeks. Sensitivity analysis is an important component of financial risk modeling. The stochastic model of the underlying assets of a financial derivative, which is the process X_t, typically contains multiple variables representing the market state. Some common examples are the spot prices of the underlying assets $S_t^{(i)}$, current volatilities $\sigma_t^{(i)}$, and the maturity times $T^{(i)}$. In mathematical finance, the partial derivatives, or sensitivities, of V_t with respect to these quantities are called *Greeks*. For example, $\frac{\partial V_t}{\partial S_t^{(i)}}$ are called *Deltas*, $\frac{\partial V_t}{\partial \sigma_t^{(i)}}$ are called *Vegas*, $\frac{\partial^2 V_t}{(\partial \sigma_t^{(i)})^2}$ are called *Gammas*, and $\frac{\partial V_t}{\partial T^{(i)}}$ are called *Thetas*. Classically, this can be computed utilizing finite-difference methods and MCI. The optimal classical method, which uses variance reduction techniques, requires $\mathcal{O}(k/\epsilon^2)$ samples to compute k Greeks. Stamatopoulos et al. [217] proposed using quantum finite-difference methods, which were first proposed by Jordan [218], to accelerate the computation of Greeks.

The quantum setting uses what is called a probability oracle. In the context of financial sensitivity, this makes use of the operation in (2) from the QMCI algorithm, namely, operators $\mathcal{P}_l$ and $\mathcal{P}_f$. For computing the k-dimensional numerical gradient, Jordan's original algorithm utilized a single call to a quantum binary oracle, which computes a fixed-point approximation to V_t into a register. The derivative of V_t is then encoded in the relative phases of quantum state utilizing phase kickback. In the case described above, however, we are provided an analog quantity encoded in the amplitude of a quantum state, and thus we would require an analog to digital, e.g. QAE, to analog conversion. The query complexity of QAE scales exponentially in the number of bits in the digital representation. Gilyén, Arunachalam, and Wiebe (GAW) [219] proposed a conversion technique that avoids the digitization step. Their approach takes as input a probability oracle and uses block-encoding-based Hamiltonian simulation [97, 220] to perform the phase encoding. In other words, it converts the probability oracle to a phase oracle. The GAW algorithm makes use of higher-order central difference methods [221]; and under certain smoothness conditions of the function whose derivative we are taking [222], the algorithm uses $\mathcal{O}(\sqrt{k}/\epsilon)$ queries to a probability oracle to compute the k-dimensional gradient. Stamatopoulos et al. showed that this can be used to produce a quadratic speedup for computing k Greeks over the classical MCI and finite-difference-based methods. They also proposed an alternative technique that does not attain the full quadratic speedup

but avoids the Hamiltonian simulation required for the probability to phase oracle conversion. As mentioned by Stamatopoulos et al., the number of Greeks, k, can be large in practice, which makes the quadratic speedup significant.

Credit Value Adjustments. Value adjustments [223], or XVAs, are a set of corrections to the value of a financial instrument due to factors that are not included in the pricing model. Specifically, credit value adjustments (CVAs) modify the price of a financial contract to take into account the risk of loss associated with a counterparty defaulting or failing to meet their obligation, namely, the counterparty-credit risk (CCR). The usual risk-neutral valuation does not take CCR into account. For example, from the perspective of a holder of an option, if the seller of the contract defaults at time t, the loss to the holder is the value of the option at t. For certain contracts, a proportion of the lost value can be recovered. This fraction is called the recovery rate, R. The value that is lost after recovery is called the loss given default (LGD). The LGD is computed as $1 - R$.

We consider a finite set of time points: $t_0, t_1, \ldots, t_N$, where t_N is the largest maturity date of all contracts in the portfolio. If we are performing a valuation of the CCR at time t_0, the expected loss if a default occurs at $t_i > t_0$ requires determining the fair value of the instrument at time t_i. Since t_i is in the future, however, the expected value lost if default occurs at time t_i, called the expected exposure profile, is $\mathbb{E}_{\mathbb{Q}}[(V_{t_i})^+|\mathcal{F}_{t_0}]$, where V_{t_i} is the fair price of a portfolio of contracts at t_i. Since default can occur before or at maturity probabilistically, the CVA at t_0 is defined to be

$$\text{CVA}_{t_0} := \mathbb{E}[(1-R)(V_\tau)^+|\mathcal{F}_{t_0}] = (1-R)\sum_{i=0}^{N} \mathbb{P}(t_i < \tau < t_{i+1})\mathbb{E}_{\mathbb{Q}}[(V_{t_i})^+|\mathcal{F}_{t_0}], \tag{4}$$

where measure $\mathbb{P}$ denotes the probability of defaulting in a given time period. The maximum is to ensure we consider only obligations of the counterparty, and τ is a random stopping time representing the time of default. The adjusted price is

$$V'_{t_0} := V_{t_0} - \text{CVA}_{t_0}. \tag{5}$$

As mentioned by Han and Rebentrost [224], if the joint distribution can be represented as a simple product, namely,

$$q_j^{(t_i)} := \mathbb{P}(t_i < \tau < t_{i+1})\mathbb{Q}(j), \tag{6}$$

where j is a series of economic events, for example a path of the underlying assets, leading to a price of $V_{t_i}^{(j)}$ at time t_i occurring with risk-neutral probability $\mathbb{Q}$, then the CVA computation can be viewed as the following inner product:

$$\text{CVA}_{t_0} = \sum_{i,j} q_j^{(t_i)} V_{t_i}^{(j)} := \vec{q} \cdot \vec{V}. \tag{7}$$

Han and Rebentrost presented a variety of quantum inner product estimation algorithms that make use of QMCI to obtain a quadratic speedup over the classical approach of sampling the entries of $\vec{V}$ according to $\vec{q}$ and taking sample averages. Their approach also applies to the problem of pricing a portfolio of derivatives. Along similar lines, Alcazar et al. [225] proposed a more near-term approach to solving this problem that makes use of a Bayesian approach to QAE [226, 227]. This Bayesian QAE approach has a complexity that varies according to the expected hardware noise and allows for interpolating between the complexities of MCI and QMCI. They also performed an end-to-end resource analysis of their approach.

6 Optimization

Solving optimization problems (e.g., portfolio optimization, arbitrage) presents the most promising commercially relevant applications on NISQ hardware (Section 3.3). In this section we discuss the potential of using quantum algorithms to help solve various optimization problems. We start by discussing quantum algorithms for the NP-hard[12] *combinatorial* optimization problems [26, 228] in Section 6.1, with particular focus on those with quadratic cost functions. Neither classical nor quantum algorithms are believed to be able to solve these NP-hard problems asymptotically efficiently (Section 4). However, the hope is that quantum computers can solve these problems faster than classical computers on realistic problems such that it has an impact in practice. Section 6.2 focuses on *convex* optimization. In its most general form convex optimization is NP-hard. However, in a variety of situations such problems can be solved efficiently [229] and have a lot of nice properties and a rich theory [27]. There exist quantum algorithms that utilize QLS (Section 4.5), and other techniques to provide polynomial speedups for certain convex, specifically conic, optimization problems. These can have a significant impact

[12]By definition (Section 4), optimization problems are not in NP, but they can be NP-hard. However, there is an analogous class for optimization problems called NPO [228].

in practice. Section 6.3 discusses *large-scale* optimization, which utilizes hybrid approaches that decompose large problems into smaller subtasks solvable on a near-term quantum device. The more general mixed-integer programming problems, in which there are both discrete and continuous variables, can potentially be handled by using quantum algorithms for optimization (both those mentioned in Section 6.1 and, in certain scenarios, Section 6.2) as subroutines in a classical-quantum hybrid approach (i.e., decomposing the problem in a similar manner to the ways mentioned in Section 6.3) [146, 230]. Many quantum algorithms for combinatorial optimization are NISQ friendly and heuristic (i.e., with no asymptotically proven benefits). The convex optimization algorithms are usually for the fault-tolerant era of quantum computation.

6.1 Combinatorial Optimization

In the general case, integer programming (IP) problems, which involve variables restricted to integers (including those with integer variable constraints), are NP-hard [26]. This section focuses on *combinatorial optimization*, sometimes used synonymously with discrete or integer optimization. In this paper, combinatorial optimization refers specifically to integer optimization problems that only consist of binary variables, namely, binary integer programs [26]. Problems that consist of selecting objects from a finite set to optimize some cost function, potentially including constraints, fit this formalism. However, integer programs can be reduced to binary programs by representing integers in binary or using one-hot encoding. Most of the financial optimization problems presented in this article can be reduced to combinatorial problems with quadratic cost functions called binary quadratic programming problems [26].

A binary quadratic program without constraints, which is also NP-hard, is called a quadratic unconstrained binary optimization problem [231] and lends itself naturally to many quantum algorithms. A QUBO problem on N binary variables can be expressed as

$$\min_{\vec{x}\in\mathbb{B}^N} \vec{x}^\mathsf{T} Q \vec{x} + \vec{x}^\mathsf{T} \vec{b}, \tag{8}$$

where $\vec{b} \in \mathbb{R}^N$, $Q \in \mathbb{R}^{N\times N}$, and $\mathbb{B} = \{0, 1\}$. Unconstrained integer quadratic programming (IQP) problems can be converted to QUBO in the same way that IP can be reduced to general binary integer programming [232]. Constraints can be accounted for by optimizing the Lagrangian function [27] of the constrained problem, where each dual variable is a hyperparameter that signifies a penalty for violating the constraint [233].

QUBO has a connection with finding the ground state of the generalized Ising Hamiltonian from statistical mechanics [233]. A simple example of an Ising model is a two-dimensional lattice under an external longitudinal magnetic field that is potentially site-dependent; in other words, the strength of the field at a site on the lattice is a function of location. At each site j, the direction of the magnetic moment of spin z_j may take on values in the set $\{-1, +1\}$. The value of z_j is influenced by both the moments of its neighboring sites and the strength of the external field. This model can be generalized to an arbitrary graph where the vertices on the graph represent the spin sites and weights on the edges signify the interaction strength between the sites, hence allowing for long-range interactions. The classical Hamiltonian for this generalized model is

$$\mathcal{H} = -\sum_{ij} J_{ij} z_i z_j - \sum_j h_j z_j,$$

where the magnitude of J_{ij} represents the amount of interaction between sites i and j and its sign is the desired relative orientation; h_j represents the external field strength and direction at site j [234]. Using a change of variables $z_j = 2x_j - 1$, a QUBO cost function as defined in Equation (8) can be transformed into a classical Ising Hamiltonian, with x_j being the jth element of $\vec{x}$. The classical Hamiltonian can then be "quantized" by replacing the classical variable z_j for the jth site with the Pauli-Z operator σ_j^z (Section 3.1). Therefore, finding the optimal solution for the QUBO is equivalent to finding the ground state of the corresponding Ising Hamiltonian.

Higher-order combinatorial problems can be modeled by including terms for multispin interactions. Some of the quantum algorithms presented in this section can handle such problems (e.g., QAOA in Section 6.1.2). From the quantum point of view, these higher-order versions are observables that are diagonal in the computational basis. The following subsections will more fully show why the quantum Ising or more generally the diagonal observable formulation results in quantum algorithms for solving combinatorial problems. In Section 6.4 we discuss a variety of financial problems that can be formulated as combinatorial optimization and solved by using the quantum methods described in this section.

6.1.1 Quantum Annealing

Quantum annealing, initially mentioned in Section 4.8, consists of adiabatically evolving according to the time-dependent transverse-field Ising Hamiltonian [234], which is mathematically defined as

$$H(t) = A(t)\left(-\sum_i \sigma_i^x\right) + B(t)\left(-\sum_{ij} J_{ij}\sigma_i^z\sigma_j^z - \sum_i h_i\sigma_i^z\right). \tag{9}$$

This formulation natively allows for solving QUBO. As discussed earlier, however, with proper encoding and additional variables, unconstrained IQP problems can be converted to QUBO. Techniques also exist for converting higher-order combinatorial problems into QUBO problems, at the cost of more variables [235, 236]. QUBO, as mentioned earlier, can, through a transformation of variables, be encoded in the site-dependent longitudinal magnetic field strengths $\{h_i\}$ and coupling terms $\{J_{ij}\}$ of an Ising Hamiltonian. The QA process initializes the system in a uniform superposition of all states, which is the ground state of $-\sum_i \sigma_i^x$. σ_i^x is the Pauli-X (Section 3.2) operator applied to the ith qubit. This initialization implies the presence of a large transverse-field component, $A(t)$ in Equation (9). The annealing schedule, controlled by tuning the strength of the transverse magnetic field, is defined by the functions $A(t)$ and $B(t)$ in Equation (9). If the evolution is slow enough, according to the adiabatic theorem (Section (3.2.2)), the system will remain in the instantaneous ground state for the entire duration and end in the ground state of the Ising Hamiltonian encoding the problem. This process is called *forward* annealing. There also exists *reverse* annealing [237], which starts in a user-provided classical state,[13] increases the transverse-field strength, pauses and then, assuming an adiabatic path, ends in a classical state. The pause is useful when reverse quantum annealing is used in combination with other classical optimizers.

The D-Wave devices are examples of commercially available quantum annealers [238]. These devices have limited connectivity resulting in extra qubits being needed to encode arbitrary QUBO problems (i.e., Ising models with interactions at arbitrary distances). Finding such an embedding, however, is in general a hard problem [239]. Thus heuristic algorithms [240] or precomputed templates [241] are typically used. Assuming the hardware topology is fixed, however, an embedding can potentially be efficiently found for QUBOs with certain structure [242]. Alternatively, the embedding problem associated with restricted connectivity can be avoided if the hardware supports three- and four-qubit interactions, by using the LHZ encoding [243] or its extension by Ender et al. [244]. The extension by Ender et al. even supports higher-order binary optimization problems [245, 246]. Moreover, the current devices for QA provide no guarantees on the adiabaticity of the trajectory that the system follows. Despite the mentioned limitations of today's annealers, a number of financial problems still remain that can be solved by using these devices (Section 6.4).

6.1.2 Quantum Approximate Optimization Algorithm

QAOA is a variational algorithm (Section 4.7) initially inspired by the adiabatic evolution of quantum annealing. The initial formulation of QAOA can be seen as a truncated or Trotterized [2] version of the QA evolution to a finite number of time steps.[14] QAOA follows the adiabatic trajectory in the limit of infinite time steps, which provides evidence of good convergence properties [136]. This algorithm caters to gate-based computers. However, it has been tested on an analog quantum simulator [247–249].

QAOA can be used to solve general unconstrained combinatorial problems. This capability allows QAOA to solve, without additional variables, a broader class of problems than QA can. As for QA, constraints can be encoded by using the Lagrangian of the optimization problem (i.e., penalty terms). Consider an unconstrained combinatorial maximization problem with N binary variables $\{x_i\}_{i=1}^N$ concatenated into the bit string $z := x_1 \dots x_N$ and cost function $f(z)$. The algorithm seeks a string z^* that maximizes f. This is done by first preparing a parameter-dependent N-qubit quantum state (realizable as a PQC, Section 4.7):

$$|\boldsymbol{\gamma}, \boldsymbol{\beta}\rangle = U(B, \beta_p)U(C, \gamma_p) \cdots U(B, \beta_1)U(C, \gamma_1) \, |s\rangle \, , \tag{10}$$

where $\boldsymbol{\gamma} = (\gamma_1, \dots, \gamma_p), \boldsymbol{\beta} = (\beta_1, \dots, \beta_p)$. The unitary $U(C, \gamma)$ is called the phase operator or phase separator and defined as $U(C, \gamma) = e^{-i\gamma C}$, where C is a diagonal observable that encodes the objective function $f(z)$. In addition, $U(B, \beta)$ is the mixing operator defined as $U(B, \beta) = e^{-i\beta B}$, where, in the initial QAOA formulation, $B = \sum_{i=1}^N \sigma_i^x$. The initial state, $|s\rangle$, is a uniform superposition, which is an excited state of B with maximum energy. However, other choices for B exist that allow QAOA to incorporate constraints without penalty terms [137]. Equation (10), where the choice of B can vary, is called the alternating operator ansatz [137]. Preparation of the state is then followed by a measurement in the computational basis (Section 3.1), indicating the assignments of the N binary variables. The parameters are updated such that the expectation of C, that is, $\langle \boldsymbol{\gamma}, \boldsymbol{\beta}| \, C \, |\boldsymbol{\gamma}, \boldsymbol{\beta}\rangle$, is maximized. Note that we can multiply the cost function by -1 to convert it to a minimization problem. The structure of the QAOA ansatz allows for finding good parameters purely classically for certain problem instances [136]. In general, however, finding such parameters is a challenging task. As mentioned earlier (Section 4.7), the training of quantum variational algorithms is NP-hard. This does in fact withhold us from reaching arbitrarily good approximate or local minima and serves as merely an indication of the complexity of finding the exact global

[13] The term "classical state" refers to any computational basis state. The quantum Ising Hamiltonian has an eigenbasis consisting of computational basis states.

[14] In the case of standard QAOA, however, the adiabatic trajectory would move between instantaneous eigenstates with the *highest* energy [136]. This still follows from the adiabatic theorem (Section 3.2.2).

minimum. However, a lot of recent research has been proposing practical approaches for effective parameter and required circuit depth estimation [250–255] as well as reducing the phase operator's complexity [256, 257] and mixer's [258] complexity. Recent demonstrations of QAOA on gate-based quantum hardware have revealed the capability of start-of-the-art quantum hardware technologies in handling unconstrained [259, 260] and constrained optimization problems [261].

6.1.3 Variational Quantum Eigensolver

The VQE algorithm, mentioned in Section 4.7, is a prominent algorithm in the NISQ era because it can be run on current hardware [262]. Essentially, the algorithm solves the problem of approximating the smallest eigenvalue of some observable and gives a prescription for reconstructing an approximation of a corresponding eigenstate. Given an observable, the expected value or equivalently the Rayleigh quotient (Section 4.7) with respect to any state is an upper bound of the smallest eigenvalue. There is equality only when the state is an eigenstate with the smallest eigenvalue. The variational procedure based on minimizing the Rayleigh quotient is known as the Rayleigh–Ritz method [138].

VQE utilizes a parameterized ansatz to represent a space of candidate wave functions. A classical optimizer modifies the parameters to minimize the expectation of the observable (i.e., the cost function) and thus tries to approach the value of the minimum eigenvalue. The hybrid system works in an iterative loop where the trial wave function is loaded onto the quantum processor and then the classical computer feeds the quantum processor new sets of parameters to improve the initial trial state based on the computed expected value. Like QAOA, VQE can also be used to find a minimum eigenvalue state of an observable, which is diagonal in the computational basis, used to encode a combinatorial optimization problem. VQE is more general than QAOA (Section 6.1.2). The reason is that QAOA specifically uses the alternating operator ansatz, while, VQE can utilize any PQC as an ansatz. However, certain ansätze are more suitable for particular problems because they take into account prior information about the ground-state wave function(s) [132]. Alternatively, there exist evolutionary, noise-resistant techniques for optimizing the structure of the ansatz and significantly increasing the space of candidate wave functions [263]. Amaro et al. [264] introduced quantum variational filtering, which can be used to increase the probability of sampling low-eigenvalue states of an observable when the filtering operator is applied to an arbitrary quantum state. Their algorithm, Filtering VQE, was applied to combinatorial optimization problems and outperformed both standard VQE and QAOA. A rigorous analysis of the convergence of VQE with an overparameterized ansatz has been performed by Xuchen et al. [265].

6.1.4 Variational Quantum Imaginary Time Evolution (VarQITE)

In relativistic terms, real-time evolution of a quantum state happens in Minkowski spacetime, which, unlike Euclidean space, has an indefinite metric tensor [266]. Imaginary-time evolution (ITE) [267] transforms the evolution to Euclidean space and is performed by making the replacement $t \mapsto it = \tau$, known as a *Wick rotation* [268]. The imaginary time dynamics follow the Wick-rotated Schrödinger equation

$$|\dot{\psi}(\tau)\rangle = -(H - E_\tau)\,|\psi(\tau)\rangle\,, \tag{11}$$

where H is a quantum Hamiltonian and $E_\tau = \langle\psi(\tau)|H\,|\psi(\tau)\rangle$. The time-evolved state (solution to Equation (11)) is $|\psi(\tau)\rangle = C(\tau)e^{-H\tau}\,|\psi(0)\rangle$, where $C(\tau)$ is a time-dependent normalization constant [269]. As $\tau \to \infty$, the smallest eigenvalue of the non-unitary operator $e^{-H\tau}$ dominates, and the state approaches a ground state of H. This is assuming the initial state has nonzero overlap with a ground state. Thus, ITE presents another method for finding the minimum eigenvalue of an observable [269, 270].

For variational quantum ITE (VarQITE) [138], the time-dependent state $|\psi(\tau)\rangle$ is represented by a parameterized ansatz (i.e., PQC), $|\phi(\boldsymbol{\theta}(\tau))\rangle$, with a set of time-varying parameters, $\boldsymbol{\theta}(\tau)$. The evolution is projected onto the circuit parameters, and the convergence is also dependent on the expressibility of the ansatz [138]. However, quantum ITE using McLachlan's principle is typically expensive to implement because of the required metric tensor computations and matrix inversion [138, 271]. Benedetti et al. developed an alternative method for VarQITE that avoids these computations and is gradient free [272]. Their variational time evolution approach is also applicable to real-time evolution. VarQITE can be used to solve a combinatorial optimization problem by letting H encode the combinatorial cost function, in a similar manner to QAOA and VQE, and is also not restricted to QUBO. In addition, VarQITE has been used to prepare states that do not result from unitary evolution in real time, such as the quantum Gibbs state [273].

We note that the variational principles for real-time evolution, mentioned in Section 4.7, can be used to approximate adiabatic trajectories [274]. This can potentially be used for combinatorial optimization as well.

6.1.5 Optimization by Quantum Unstructured Search

Dürr and Høyer [275] presented an algorithm that applies quantum unstructured search (Section 4.1) to the problem of finding the *global* [276] minimum of a black-box function. This approach makes use of the work of Boyer et al. [75] to apply Grover's search when the number of marked states is unknown. Unlike the metaheuristics of quantum annealing and gate-based variational approaches, the Dürr-Høyer algorithm has a provable quadratic speedup in query complexity. That is, given a search space $\mathcal{X}$ of size N, the algorithm requires $\mathcal{O}(\sqrt{N})$ queries to an oracle that evaluates the function $f : \mathcal{X} \mapsto \mathbb{R}$ to minimize. A classical program would require $\mathcal{O}(N)$ evaluations.

The requirements to achieve quadratic speedup are the same as for Grover's algorithm. The quantum oracle in this case is $O_{g_y}|x\rangle = (-1)^{g_y(x)}|x\rangle$; $g_y : \mathcal{X} \mapsto \{0, 1\}$ is a classical function mapping $g_y(x)$ to 1 if $f(x) \leq y$, otherwise 0; and y is the current smallest value of f evaluated so far [276]. The Grover adaptive search framework of Bulger et al. [277] generalizes and extends the Dürr–Høyer algorithm. Gilliam et al. [278] proposed a method for efficiently implementing the oracle O_{g_y} by converting to Fourier space. This applies particularly to binary polynomials, and thus this method can be used to solve arbitrary constrained combinatorial problems with a quadratic speedup over classical exhaustive search.

6.2 Convex Optimization

Quantum algorithms have been invented for common convex optimization problems [27] such as linear programming, second-order cone programming (SOCP), and semidefinite programming (SDP). These conic programs have a variety of financial applications, such as portfolio optimization, discussed in Section 6.4.1. The quantum linear systems algorithms from Section 4.5 have been used as subroutines to improve algorithms for linear programming such as the simplex method [279] in the work by Nannicini [280] and interior-point methods (IPMs) [229] in the work by Kerenedis and Prakash [101]. The quantum IPM utilizes QLS solvers with tomography to accelerate the computation of the Newton linear system. Kerenedis and Prakash [281] also applied quantum IPM to provide, under certain conditions, small polynomial speedups for the primal-dual IPM method for SOCP [282]. Furthermore, various approaches using quantum computing to solve SDPs have been presented. Brandão [283], Brandão et al. [284], and van Apeldoorn et al. [285] proposed quantum improvements to the multiplicative-weights method of Arora and Kale [286] for SDP. Alternatively, Kerenedis and Prakash applied their quantum IPM to SDP [101].

6.3 Large-Scale Optimization

Near-term quantum devices are expected to have a limited number of qubits in the foreseeable future. This is a major obstacle for real applications in finance that are large scale in many domains. The hybridization of quantum and classical computers using decomposition-based approaches is a prominent solution for this problem [287]. The idea behind this class of approaches is similar to what is done with numerical methods and high-performance computing. The main driving routine that is executed on a classical machine decomposes the problem into multiple parts. These subproblems are then each solved separately on a quantum device. The classical computer then combines solutions received from the quantum computer. For example, in quantum local search for modularity optimization [288] and an extension of it to a multilevel approach [289], the main driving routine identifies small subsets of variables and solves each subproblem on a quantum device. A similar decomposition approach for finding maximum graph cliques was developed by Chapuis et al. [290].

6.4 Financial Applications

In this section we explore multiple applications of the quantum optimization algorithms presented above to financial problems.

6.4.1 Portfolio Optimization

In finance, one of the most commonly seen optimization problems is *portfolio optimization*. Portfolio optimization is the process of selecting the best set of assets and their quantities, from a pool of assets being considered, according to some predefined objective. The objective can vary depending on the investor's preference regarding financial risk and expected return. Modern portfolio theory [291] focuses on the trade-offs between risk and return to produce what is known as an efficient portfolio, which maximizes the expected return given a certain amount of risk. This trade-off relationship is represented by a curve known as the efficient frontier. The expected return and risk of a financial portfolio can often be modeled respectively by looking at the mean and variance of static portfolio returns. The problem setup for portfolio optimization can be formulated as constrained utility maximization [291].

In portfolio optimization, each asset class, such as stocks, bonds, futures, and options, is assigned a weight. Additionally, all assets within the class are allocated in the portfolio depending on their respective risks, returns, time to maturity, and liquidity. The variables controlling how much the portfolio should invest in an asset can be continuous or discrete. However, the overall problem can contain variables of both types. Continuous variables are suitable for representing the proportion of the portfolio invested in the asset positions when non-discrete allocations are allowed. Discrete variables are used in situations where assets can be purchased or sold only in fixed amounts. The signs of these variables can also be used to indicate long or short positions. When risk is accounted for, the problem is typically a quadratic program, usually with constraints. Depending on the formulation, this problem can be solved with techniques for convex [291] or mixed-integer programming [292, 293]. Speeding up the execution of and improving the quality of portfolio optimization is particularly important when the number of variables is large.

This section first focuses on combinatorial formulations using the algorithms mentioned in Section 6.1. As mentioned there, the algorithms presented can be generalized to integer optimization problems. The second part of this section focuses on convex formulations of the problem solved by using algorithms from Section 6.2. The handling of mixed-integer programming was briefly mentioned at the beginning of Section 6.

Combinatorial Formulations. The first combinatorial formulation considered is risk minimization or hedging [294]. The risk term is quadratic and is derived from the covariance or correlation matrix computed by using historical pricing information. A simplified version of this problem can be formulated as the following QUBO:

$$\min_{\vec{x}\in\mathbb{B}^N} \vec{x}^\mathsf{T}\Sigma\vec{x}, \tag{12}$$

where $\Sigma \in \mathbb{R}^{N\times N}$ is the covariance or correlation matrix. This problem can be modified to include budget constraints. With regard to this problem, Kalra et al. used a time-indexed correlation graph such that the vertices represent assets and an edge between vertices represents the existence of a significant correlation between them [295]. The price changes are modeled by data collected daily. Instead of continuous correlation values, Kalra et al. utilized a threshold on the correlations to decide whether to create edges between assets. This was to create sparsity in the graph. One approach they took to solve risk minimization was formulating the problem as finding a maximum-independent set in the graph constructed. This is similar to Equation (12). They also presented a modified formulation suitable for the minimum graph coloring problem. Instead of only choosing whether to pick an asset or not, Rosenberg and Rounds [296] used binary variables to represent whether to hold long or short positions. This approach is known as long-short minimum risk parity optimization. The authors represented the problem by an Ising model, which is equivalent to a QUBO problem, as follows:

$$\min_{\vec{s}\in\{-1,1\}^N} \vec{s}^\mathsf{T}W^\mathsf{T}\Sigma W\vec{s},$$

where W is an $N\times N$ diagonal matrix with the entries of $\vec{w}\in[0,1]^N$, such that $\|\vec{w}\|_1 = 1$, on the diagonal; $\vec{w}$, contains a fixed weight for each asset j indicating the proportion of the portfolio that should hold a long or short position in j. In both cases, the authors utilized quantum annealing (Section 6.1.1).

Instead of just risk minimization, the problem can be reformulated to specify a desired fixed expected return, $\mu\in\mathbb{R}$,

$$\min_{\vec{x}\in\mathbb{B}^N} \vec{x}^\mathsf{T}\Sigma\vec{x} : \ \vec{r}^\mathsf{T}\vec{x} = \mu, \quad \vec{x}^\mathsf{T}\vec{1} = \beta \tag{13}$$

or to maximize the expected return,

$$\min_{\vec{x}\in\mathbb{B}^N} q\vec{x}^\mathsf{T}\Sigma\vec{x} - \vec{r}^\mathsf{T}\vec{x} \ : \ \vec{x}^\mathsf{T}\vec{1} = \beta, \tag{14}$$

where $\vec{x}$ is a vector of Boolean decision variables, $\vec{r}\in\mathbb{R}^N$ contains the expected returns of the assets, Σ is the same as above, $q>0$ is the risk level, and $\beta\in\mathbb{N}$ is the budget. In both cases, the Lagrangian of the problem, where the dual variables are used to encode user-defined penalties, can be mapped to a QUBO (Section 6.1). These formulations are known as mean-variance portfolio optimization problems [291]. More constraints can also be added to these problems.

Hodson et al. [297] and Slate et al. [298] both utilized QAOA to solve a problem similar to Equation (14). They allowed for three possible decisions: long position, short position, or neither. In addition, they utilized mixing operators (Section 6.1.2) that accounted for constraints. Alternatively, Gilliam et al. [278] utilized an extension of Grover adaptive search (Section 6.1.5) to solve Equation (14) with a quadratic speedup over classical unstructured search. Rosenberg et al. [299] and Mugel et al. [300] solved dynamic versions of Equation (14), where portfolio decisions are made for multiple time steps. Additionally, an analysis of benchmarking quantum annealers for portfolio optimization can be found in a paper by Grant et al. [301]. Furthermore, an approach using reverse quantum annealing (Section 6.1.1) was proposed by Venturelli and Kondratyev (Section 8.2). Note that any of the methods from Section 6.1 can be applied whenever quantum annealing can. Currently, however,

because quantum annealers possess more qubits than existing gate-based devices possess, more experimental results with larger problems have been obtained with annealers.

Convex Formulations. This part of the section focuses on formulations of the portfolio optimization problem that are convex. The original formulation of mean-variance portfolio optimization by Markowitz [291] was convex, and thus it did not contain integer constraints, as were included in the discussion above. Thus, if the binary variable constraints are relaxed, the optimization problem, represented by Equation (13), can be reformulated as a convex quadratic program with linear equality constraints with both a fixed desired return and budget (Equation 15):

$$\min_{\vec{w}\in\mathbb{R}^N} \vec{w}^\mathsf{T}\Sigma\vec{w} \;:\; \vec{p}^\mathsf{T}\vec{w} = \xi,\; \vec{r}^\mathsf{T}\vec{w} = \mu, \tag{15}$$

where $\vec{w} \in \mathbb{R}^N$ is the allocation vector. The ith entry of the vector $\vec{w}$ represents the proportion of the portfolio that should invest in asset i. This is in contrast to the combinatorial formulations where the result was a Boolean vector. The vectors $\vec{p} \in \mathbb{R}^N$ and $\vec{r} \in \mathbb{R}^N$ contain the assets' prices and returns, respectively; $\xi \in \mathbb{R}$ is the budget; and $\mu \in \mathbb{R}$ is desired expected return. In contrast to Equation (13), Equation (15) admits a closed-form solution [302]. However, more problem-specific conic constraints and cost terms (assuming these terms are convex) can be added, increasing the complexity of the problem, in which case it can potentially be solved with more sophisticated algorithms (e.g., interior-point methods). As mentioned, there exist polynomial speedups provided by quantum computing for common cone programs (Section 6.2). Equation (15) with additional positivity constraints on the allocation vector can be represented as an SOCP and solved with quantum IPMs [302].

Considering the exact formulation in Equation (15), however, we might be able to obtain superpolynomial speedup if we relax the problem even further. Rebentrost and Lloyd [303] presented a relaxed problem where the goals were to sample from the solution and/or compute statistics. Their approach was to apply the method of Lagrange multipliers to Equation (15) and obtain a linear system. This system can be formulated as a quantum linear systems problem and solved with QLS methods (Section 4.5). The result of solving the QLSP (Section 4.5) is a normalized quantum state corresponding to the solution found with time complexity that is polylogarithmic in the system size, N. Thus, if only partial information about the solution is required, QLS methods potentially provide an exponential speedup, in N, over all known classical algorithms for this scenario, although the sparsity and well-conditioned matrix assumptions mentioned in Section 4.5 have to be met to obtain this speedup. As mentioned in Section 4.5, however, in scenarios involving low-rank matrices randomized numerical linear algebra algorithms can be applied to obtain the same exponential speedup in dimension [304]. However, these methods typically have a high polynomial dependence on the rank and precision, which still allows for a potential polynomial quantum advantage. This dequantized approach was benchmarked in [305].

6.4.2 Swap Netting

A *swap* is a contract where two parties agree to exchange cash flows periodically for a specific term [306]. Common types of swaps are credit default swaps [307], foreign exchange currency swaps, and interest rate swaps [308]. The simplest example is the fixed-to-floating interest rate swap, where two parties exchange the responsibility of paying fixed-interest rate and floating-interest rate payments based on a principal amount called the notional value [309]. Examples of potential reasons to enter into such a contract could be to hedge or take advantage of the opposite party's comparative advantage [309]. Once an agreement between the parties has been made, a clearing house converts an agreement between two parties into two separate agreements with the clearing house. The clearing house potentially has multiple swaps with multiple parties. In the case that multiple cash flows cancel, the clearing house would like to *net* all of the contracts that cancel into a new one for only the net flow [310]. This is to reduce the risk exposure associated with having multiple contracts.

Rosenberg et al. [306] formulated the task of finding a set of "nettable" swaps associated with a single counterparty that maximizes the total notional value as a QUBO. This optimization problem can be solved in parallel for each subset of potentially nettable swaps. Since swaps can be netted for potentially many reasons, the set of combinations can grow significantly. The procedure proposed by the authors is as follows. First, partition the set of swaps associated with a counterparty into subsets that can be netted. Second, for a subset $\mathcal{M}$ in the partition, with $N := |\mathcal{M}|$, construct the following QUBO:

$$\max_{\vec{x}\in\mathbb{B}^N} \vec{x}^\mathsf{T}\vec{p} - \alpha\left(\sum_{i\in\mathcal{M}} \vec{x}_i\vec{d}_i\vec{p}_i\right)^2 - \beta\vec{x}^\mathsf{T}V\vec{x},$$

where $\vec{x} \in \mathbb{B}^N$ is a vector of decision variables indicating whether to include the swap in the netted subset, $\vec{p} \in \mathbb{R}^N$ is the vector of notional values associated with the swaps, and $\vec{d} \in \{+1, -1\}^N$ indicates the direction of the fixed-rate interest payments: $+1$ if the clearing house pays the counterparty and -1 if the opposite is true.

$V \in \mathbb{R}_{\geq 0}^{N \times N}$ signifies how incompatible two potentially "nettable" swaps are, and α and β weigh the importance of the second and third terms. The combined goal of the three terms is to maximize the total notional value netted (first term) such that the notional values of the selected swaps cancel (second term) and the swaps are compatible (third term). The QUBO problems were solved by utilizing quantum annealing (Section 6.1.1).

6.4.3 Optimal Arbitrage

An *arbitrage* opportunity occurs when one can profit from buying and selling similar financial assets in different markets because of price differences [311]. Arbitrage is due to the lack of market equilibrium; acting on arbitrage opportunities should shift the market back into equilibrium [312]. Thus, it is to one's advantage to be able to identify these opportunities as quickly and efficiently as possible. An example is currency arbitrage, where one can buy and sell currencies in foreign exchange (FX) markets such that when one ends up holding the initial currency again, one has profited because of the differences in FX rates (conversion rates) [311].

One way to formulate the problem of arbitrage detection is to view the different assets as the vertices of a graph with weighted edges between them indicating the negative logarithm of the conversion rate. The problem then reduces to identifying negative cycles [313], for which multiple classical algorithms exist [314]. However, the arbitrage opportunity detected is not necessarily the *optimal* one.

Soon and Ye [315] formulated the task of identifying the optimal currency arbitrage opportunity in the graph structure mentioned above as a binary linear programming problem. Thus this problem is NP-hard (Section 6.1). Rosenberg [313] reformulated this as a QUBO and solved it using quantum annealing. Alternatively, this can be solved by utilizing the other techniques for combinatorial optimization from Section 6.1. The QUBO resulted from converting the constrained binary linear program of Soon and Ye into an unconstrained problem. In addition, Rosenberg provided an alternative model that allows for arbitrage decisions that revisit the same asset (i.e., the same vertex). By definition, this is not allowed in a cycle. Solving this formulation of the problem effectively consists of selecting the assets to buy or sell and the point in the arbitrage loop at which the buying or selling occurs. However, this results in a problem with significantly more binary variables [313].

6.4.4 Identifying Creditworthiness

Creditworthiness identification is an important problem in finance. With large sums of money being loaned globally, it is important to be able to identify the key *independent* features that are *influential* in determining the creditworthiness of the requester. The task of identifying features that balance independence and influence can be expressed as a QUBO problem.

The problem setup for this *combinatorial* feature selection task is as follows. Assume that from an initial set of N features, we want to select K to use. The data is cleaned and set up in a matrix $U \in \mathbb{R}^{M \times N}$ where each column represents a feature and each row contains the values of the features for each of the M past credit recipients. The past decisions that were made regarding creditworthiness are represented in a column vector $\vec{v} \in \mathbb{B}^M$. The goal is to find a set of columns in U that are most correlated with $\vec{v}$ but not with each other. The next step is to construct a quadratic objective function that is a convex combination of terms evaluating the influence and independence of the features. Milne et al. [316] utilized quantum annealing to solve the QUBO. Thus the constraint of selecting K features can be accounted for by adding a penalty term to the QUBO. However, the problem can be solved by using any of the techniques mentioned in Section 6.1. Classical or quantum machine learning algorithms (Section 7) can then be used to classify new applicants based on the features selected [316].

6.4.5 Financial Crashes

A financial network can viewed as a collection of entities whose valuations are interconnected to the valuations of other members of the network. The analysis of financial networks is important for being able to predict crises. Elliot et al. [317] developed a problem formulation for counting the number of intuitions that fail in a financial network. Their proposed problem was found to be NP-hard [318]. If solved, however, it allows for analyzing how small changes in the network propagate through a web of complex interdependencies, something that is difficult [319] with alternative techniques for crash prediction [320]. By modeling a financial system, mapping the system to a QUBO problem, and perturbing the system to find the ground state it reaches, we can determine the effect of small perturbations on the market.

We briefly discuss the formulation of Elliott et al. [317]. First, suppose the financial network consists of N institutions and M assets, with prices $\vec{p} \in \mathbb{R}_{\geq 0}^M$, that the institutions can own. $D \in \mathbb{R}_{\geq 0}^{N \times M}$ contains the percentage of each asset owned by each institution. In addition, the institutions can have a stake, cross holdings, in each other represented by the matrix $C \in \mathbb{R}_{\geq 0}^{N \times N}$. The diagonal entries of C are set to 0, and the self-ownership is represented by the diagonal matrix $\tilde{C}$ such that $\tilde{C}_{jj} := I - \sum_i C_{ij}$. The equity valuations, $\vec{v}$, at equilibrium

satisfy

$$\vec{v} = \tilde{C}(I - C)^{-1}(D\vec{p} - \vec{b}(\vec{v}, \vec{p})), \tag{16}$$

where $\vec{b}(\vec{v}, \vec{p})$ is a term that decreases the equity value further if it falls past a certain threshold. This signifies a loss of investor confidence or the inability to pay operating costs, signifying failure [317].

Orús et al. [321] considered the problem of minimizing a quadratic function of the difference between the left and right sides of Equation (16). This allowed for a variational method to be used. Thus the minimum is attained when Equation (16) is satisfied. They then formulated this as a QUBO. Thus, after perturbing the system, the new equilibrium can be found by solving the QUBO. The next step is to count the number of institutions that go into failure as a function of the perturbation to assess the stability. The valuations $\vec{v}$ are continuous variables that can be approximated by using finite binary expansions. This method converts the initial problem into a binary optimization problem. Orús et al. solved the QUBO using quantum annealing, and Fellner et al. [246] solved it with QAOA. However, the right side of Equation (16) contains terms that are not quadratic. Thus, the resulting diagonal observable encoding it is not two-local (more specifically, not Ising). Orús et al. utilized the approach of Chancellor et al. [235] to convert the higher-order terms into two-body interaction terms, at the cost of extra variables. For the approach of Fellner et al. the problem Hamiltonian did not need to be mapped to an Ising Hamiltonian (Section 6.1.2).

7 Machine Learning

The field of machine learning has become a crucial part of various applications in the finance industry. Rich historical financial data and advances in machine learning make it possible, for example, to train sophisticated models to detect patterns in stock markets, find outliers and anomalies in financial transactions, automatically classify and categorize financial news, and optimize portfolios.

Quantum algorithms for machine learning can be further classified by whether they follow a fault-tolerant approach, a near-term approach, or a mixture of the two approaches. The fault-tolerant approach typically requires a fully error-corrected quantum computer, and the main goal is to find speedups compared with their classical counterparts, achieved mostly by applying the quantum algorithms introduced in Section 4.5 as subroutines. Thus the applicability of these algorithms may be limited since most of them begin with first loading the data into a quantum system, which can require exponential time [89, 322]. This issue can be addressed in theory by having access to qRAM (Section 4.1); as of today, however, no concrete implementation exists. On the other hand, with the development of more NISQ devices, near-term approaches aim to explore the capability of these small-scale quantum devices. Quantum algorithms such as VQAs (Section 4.7) consider another type of enhancement for machine learning: the quantum models might be able to generate correlations that are hard to represent classically [323, 324]. Such models face many challenges, however, such as trainability, accuracy, and efficiency, that need to be addressed in order to maintain the hope of achieving quantum advantage when scaling up these near-term quantum devices [130]. Substantial efforts are needed before we can answer the question of whether quantum machine learning algorithms can bring us practical and useful applications, and it might take decades. Nevertheless, quantum machine learning shows great promise to find improvements and speedups to elevate the current capabilities vastly.

7.1 Regression

Regression is the process of fitting a numeric function from the training data set. This process is often used to understand how the value changes when the attributes vary, and it is a key tool for economic forecasting. Typically, the problem is solved by applying least-squares fitting, where the goal is to find a continuous function that approximates a set of N data points $\{x_i, y_i\}$. The fit function is of the form

$$f(x, \vec{\lambda}) := \sum_{j=1}^{M} f_j(x)\lambda_j,$$

where $\vec{\lambda}$ is a vector of parameters and $f_j(x)$ is function of x and can be nonlinear. The optimal parameters can be found by minimizing the least-squares error:

$$\min_{\vec{\lambda}} \sum_{i=1}^{N} \left| f(x_i, \vec{\lambda}) - y_i \right|^2 = |F\vec{\lambda} - y|^2,$$

where the $N \times M$ matrix F is defined through $F_{ij} = f_j(x_i)$. Wiebe et al. [325] proposed a quantum algorithm to solve this problem, where they encoded the optimal parameters of the model into the amplitudes of a quantum

state and developed a quantum algorithm for estimating the quality of the least-squares fit by building on the HHL algorithm (Section 4.5). The algorithm consists of three subroutines: a quantum algorithm for performing the pseudoinverse of F, an algorithm that estimates the fitting quality, and an algorithm for learning the parameter $\vec{\lambda}$. Later, Wang [326] proposed using the CKS matrix-inversion algorithm (Section 4.5), which has better dependence on the precision in the output than HHL has, and used amplitude estimation (Section 4.4) to estimate the optimal parameters. Kerenedis and Prakash developed algorithms utilizing QSVE (Section 4.5) to perform a coherent gradient descent for normal and weighted least-squares regression [96]. Moreover, quantum annealing (Section 6.1.1) has been used to solve least-squares regression when formulated as a QUBO (Section 6.1) [327].

A Gaussian process (GP) is another widely used model for regression in supervised machine learning. It has been used, for example, in predicting the price behavior of commodities in financial markets. Given a training set of N data points $\{x_i, y_i\}$, the goal is to model a latent function $f(x)$ such that

$$y = f(x) + \epsilon_{\text{noise}},$$

where $\epsilon_{\text{noise}} \sim \mathcal{N}(0, \sigma_n^2)$ is independent and identically distributed Gaussian noise. A practical implementation of the Gaussian process regression (GPR) model needs to compute the Cholesky decomposition and therefore requires $O(N^3)$ running time. Zhao et al. [328] proposed using the HHL algorithm to speed up computations in GPR. By repeated sampling of the results of specific quantum measurements on the output states of the HHL algorithm, the mean predictor and the associated variance can be estimated with bounded error with potentially an exponential speedup over classical algorithms.

Another recent development is to use a feedforward quantum neural network with a quantum feature map, i.e., a unitary with configurable parameters used to encode the classical input data, and a Hamiltonian cost function evaluation for the purposes of continuous-variable regression in a technique called quantum circuit learning (QCL) [329]. This technique allows for a low-depth circuit and has important application opportunities in quantum algorithms for finance.

7.2 Classification

Classification is the process of placing objects into predefined groups. This type of process is also called pattern recognition. This area of machine learning can be used effectively in risk management and large data processing when the group information is of particular interest, for example, in creditworthiness identification and fraud detection.

7.2.1 Quantum Support Vector Machine

In machine learning, support vector machines (SVMs) are supervised learning models used to analyze data for classification and regression analysis. Given M training data elements of the form $\{(\vec{x}_j, y_j) : \vec{x}_j \in \mathbb{R}^N, y_j = \pm 1\}$, the task for the SVM is to classify a vector into one of two classes; it finds a maximum margin hyperplane with normal vector $\vec{w}$ that separates the two classes. In addition, the margin is given by the two hyperplanes that are parallel and separated by the maximum possible distance $2/\|\vec{w}\|_2$. Typically, we use the Lagrangian dual formulation to maximize the function

$$L(\alpha) = \sum_{j=1}^{M} y_j \alpha_j - \frac{1}{2} \sum_{j,k=1}^{M} \alpha_j K_{jk} \alpha_k,$$

where $\vec{\alpha} = (\alpha_1, \cdots, \alpha_M)^T$, subject to the constraints $\sum_{j=1}^{M} \alpha_j = 0$ and $y_j \alpha_j \geq 0$. Here, we have a key quantity for the supervised machine learning problem [330], the kernel matrix $K_{jk} = k(\vec{x}_j, \vec{x}_k) = \vec{x}_j \cdot \vec{x}_k$. Solving the dual form involves evaluating the dot products using the kernel matrix and then finding the optimal parameters α_j by quadratic programming, which takes $\mathcal{O}(M^3)$ in the non-sparse case. A quantum SVM was first proposed by Rebentrost et al. [331], who showed that a quantum SVM can be implemented with $\mathcal{O}(\log MN)$ runtime in both training and classification stages, under the assumption that there are oracles for the training data that return quantum states $|\vec{x}_j\rangle = \frac{1}{\|\vec{x}_j\|_2} \sum_{k=1}^{N} (\vec{x}_j)_k |k\rangle$, the norms $\|\vec{x}_j\|_2$ and the labels y_j are given, and the states are constructed by using qRAM [70, 332]. The core of this quantum classification algorithm is a non-sparse matrix exponentiation technique for efficiently performing a matrix inversion of the training data inner-product (kernel) matrix. Lloyd et al. [332] showed that, effectively, a low-rank approximation to kernel matrix is used due to the eigenvalue filtering procedure used by HHL [88]. As mentioned in Section 6.2, Kerenedis et al. proposed a quantum enhancement to second-order cone programming that was subsequently used to provide a small polynomial speedup to the training of the classical ℓ_1 SVM [281]. In addition, classical SVMs using quantum-enhanced feature spaces to construct quantum kernels have been proposed by Havlíček et al. [333]. One benefit of quantum kernels is that there exist metrics for testing for potential quantum advantage [324]. In addition, there have been techniques proposed to enable generalization [334, 335], which has been observed to be an issue with standard quantum-kernel methods [336].

7.2.2 Quantum Nearest-Neighbors Algorithm

The k-nearest neighbors technique is an algorithm that can be used for classification or regression. Given a data set, the algorithm assumes that data points closer together are more similar and uses distance calculations to group close points together and define a class based on the commonality of the nearby points. Classically, the algorithm works as follows: (1) determine the distance between the query example and each other data point in the set, (2) sort the data points in a list indexed from closest to the query example to farthest, and (3) choose the first k entries, and, if regression, return the mean of the k labels and, if classification, return the mode of k labels. The computationally expensive step is to compute the distance between elements in the data set. The quantum nearest-neighbor classification algorithm was proposed by Wiebe et al. [337], who used the Euclidean distance and the inner product as distance metrics. The distance between two points is encoded in the amplitude of a quantum state. They used amplitude amplification (Section 4.2) to amplify the probability of creating a state to store the distance estimate in a register, without measuring it. They then applied the Dürr–Høyer algorithm (Section 6.1.5). Later, Ruan et al. proposed using the Hamming distance as the metric [338]. More recently, Basheer et al. have proposed using a quantum k maxima-finding algorithm to find the k-nearest neighbors and use the fidelity and inner product as measures of similarity [339].

7.3 Clustering

Clustering, or cluster analysis, is an unsupervised machine learning task. It explores and discovers the grouping structure of the data. In finance, cluster analysis can be used to develop a trading approach that helps investors build a diversified portfolio. It can also be used to analyze different stocks such that the stocks with high correlations in returns fall into one basket.

7.3.1 Quantum k-Means Clustering

The k–means clustering algorithm (also known as Lloyd's algorithm) [340] clusters training examples into k clusters based on their distances to each of the k cluster centroids. The algorithm for k–means clustering is as follows: (1) choose initial values for the k centroids randomly or by some chosen method, (2) assign each vector to the closest centroid, and (3) recompute the centroids for each cluster; then repeat steps (2) and (3) until the clusters converge. The problem of optimally clustering data is NP–hard, and thus finding the optimal clustering using this algorithm can be computationally expensive. Quantum computing can be leveraged to accelerate a single step of k-means. For each iteration, the quantum Lloyd's algorithm [332] estimates the distance to the centroids in $\mathcal{O}(M\log(MN))$, while classical algorithms take $\mathcal{O}(M^2N)$, where M is the number of data points and N is the dimension of the vector. Wiebe et al. [337] showed that, with the same technique they used for the quantum nearest-neighbor algorithm described in Section 7.2, a step for k–means can be performed by using a number of queries that scales as $\mathcal{O}(M\sqrt{k}\log(k)/\epsilon)$. While a direct classical method requires $\mathcal{O}(kMN)$, the quantum solution is substantially better if $kN \gg M$. Kerenidis et al. [341] proposed q-means, which is the quantum equivalent of δ-k-means. The q-means algorithm has a running time that depends polylogarithmically on the number of data points. A NISQ version of k-means clustering using quantum computing has been proposed by Khan et al. [342]. Another version of k-means clustering has been proposed by Miyahara et al. [343] based on a quantum expectation-maximization algorithm for Gaussian mixture models.

7.3.2 Quantum Spectral Clustering

Spectral clustering methods [344] have had great successes in clustering tasks but suffer from a fast-growing running time of $\mathcal{O}(N^3)$, where N is the number of points in the data set. These methods obtain a solution for clustering by using the principal eigenvectors of the data matrix or the Laplacian matrix. Daskin [345] employed QAE (Section 4.4) and QPE (Section 4.3) for spectral clustering. Apers et al. [346] proposed quantum algorithms using the graph Laplacian for machine learning applications, including spectral k-means clustering. More recently, Kerenidis and Landman [347] proposed a quantum algorithm to perform spectral clustering. They first created a quantum state corresponding to the projected Laplacian matrix, which is efficient assuming qRAM access, and then applied the q-means algorithm [341]. Both steps depend polynomially on the number of clusters k and polylogarithmically on the dimension of the input vectors.

More discussions on unsupervised quantum machine learning techniques can be found in the works of Otterbach et al. [348] and Aïmeur et al. [349, 350]. Kerenidis et al. [351] discussed a quantum version of expectation-maximization, a common tool used in unsupervised machine learning. Kumar et al. described a technique for using quantum annealing for combinatorial clustering [352].

7.4 Dimensionality Reduction

The primary linear technique for dimensionality reduction, principal component analysis, performs a linear mapping of the data to a lower-dimensional space in such a way that the variance of the data in the low-dimensional representation is maximized. Generally, principal component analysis mathematically amounts to finding dominant eigenvalues and eigenvectors of a very large matrix. The standard context for PCA involves a data set with observations on M variables for each of N objects. The data defines an $N \times M$ data matrix X, in which the jth column is the vector $\vec{x}_j$ of observations on the jth variable. The aim is to find a linear combination of the columns of the matrix X with maximum variance. This process boils down to finding the largest eigenvalues and corresponding eigenvectors of the covariance matrix. Lloyd et al. [353] proposed quantum PCA. In addition, they showed that multiple copies of a quantum system with density matrix ρ can be used to construct the unitary transformation $e^{-i\rho t}$, which leads to revealing the eigenvectors corresponding to the large eigenvalues in quantum form for an unknown low-rank density matrix. He et al. introduced a low-complexity quantum principal component analysis algorithm [354]. Other quantum algorithms for PCA were proposed in [355–357]. Lastly, Li et al. [358] presented a quantum version of kernel PCA, Cong [359] presented quantum discriminant analysis, and Kerenidis and Luongo [360] developed quantum slow feature analysis.

For data sets that are high dimensional, incomplete, and noisy, the extraction of information is generally challenging. Topological data analysis (TDA) is an approach to study qualitative features and analyze and explore the complex topological and geometric structures of data sets. Persistent homology (PH) is a method used in TDA to study qualitative features of data that persist across multiple scales. Lloyd et al. [361] proposed a quantum algorithm to compute the Betti numbers, that is, the numbers of connected components, holes, and voids in the data set at varying scales. Essentially, the algorithm operates by finding the eigenvectors and eigenvalues of the combinatorial Laplacian and estimating Betti numbers to all orders and to accuracy δ in time $\mathcal{O}(n^5/\delta)$. The most significant speedup is for dense clique complexes [362]. An improved version as well as a NISQ version of the quantum algorithm for PH has been proposed by Ubaru et al. [363]. Further improvements for implementing the Dirac operator, required for PH, have been made by Kerenedis and Prakash [364].

7.5 Generative Models

Unsupervised generative modeling is at the forefront of deep learning research. The goal of generative modeling is to model the probability distribution of observed data and generate new samples accordingly. One of the most promising aspects of achieving potential quantum advantage lies in the sampling advantage, and many applications in finance require generating samples from complex distributions. Therefore, further investigations of generative quantum machine learning for finance are needed.

7.5.1 Quantum Circuit Born Machine

The quantum circuit Born machine (QCBM) [365] directly exploits the inherent probabilistic interpretation of quantum wave functions and represents a probability distribution using a quantum pure state instead of the thermal distribution. Liu et al. [366] developed an efficient gradient-based learning algorithm to train the QCBM. Numerical simulations suggest that in the task of learning the distribution, QCBM at least matches the performance of the restricted Boltzmann machine (Section 7.5.3) and demonstrates superior performance as the model scales [367]. QCBM was also used to model copulas [368], a family of multivariate distributions with uniform marginals. Recently, Kyriienko et al. [193] proposed separating the training and sampling stages. In the training stage, they build a model in the latent space, followed by a variational circuit. They showed how probability distributions can be trained and sampled efficiently, and how SDEs can act as differential constraints on such trainable quantum models.

7.5.2 Quantum Bayesian Networks

Bayesian networks are probabilistic graphical models [369] representing random variables and conditional dependencies via a directed acyclic graph, where each edge corresponds to a conditional dependency and each node corresponds to a unique random variable. Bayesian inference on this graph has many applications, such as prediction, anomaly detection, diagnostics, reasoning, and decision-making with uncertainty. Despite the large complications, exact inference is #P-hard, but a quadratic speedup in certain parameters can be obtained by using quantum technologies [370]. Mapping this problem to a quantum Bayesian network seems plausible since quantum mechanics naturally describes a probabilistic distribution. Tucci [371] introduced the quantum Bayesian network as an analogue to classical Bayesian networks, using quantum complex amplitudes to represent the conditional probabilities in a classical Bayesian network. Borujeni et al. [372] proposed a procedure to design a quantum circuit to represent a generic discrete Bayesian network: (1) map each node in the Bayesian network to one or more qubits, (2) map conditional probabilities of each node to the probability amplitudes associated with

qubit states, and (3) obtain the desired probability amplitudes through controlled rotation gates. In a later work they tested it on IBM quantum hardware [373]. The applications in finance include portfolio simulation [374] and decision-making modeling [375].

7.5.3 Quantum Boltzmann Machines

A Boltzmann machine is an undirected probabilistic graphical model (i.e., a Markov random field) [369] inspired by thermodynamics. Inference, classically, is usually performed by utilizing Markov chain Monte Carlo methods to sample from the model's equilibrium distribution (i.e., the Boltzmann distribution) [376]. Because of the intractability of the partition function in the general Boltzmann machine, the graph structure is typically bipartite, resulting in a restricted Boltzmann machine [377, 378]. These methods were more popular prior to the current neural-network-based deep learning revolution [376]. Quantum Boltzmann machines have been implemented by utilizing quantum annealing (Section 6.1.1) [379–381]. This is possible because of the annealer's ability to approximately sample ground states of Ising models. In addition, a gate-based variational approach using VarQITE (Section 6.1.4) has been designed by Zoufal et al. [273]. Since quantum states are always normalized, this approach allows for implementing Boltzmann machines with more complex graph structures.

7.5.4 Quantum Generative Adversarial Networks

Generative adversarial networks (GANs) represent a powerful tool for classical machine learning: a generator tries to create statistics for data that mimic those of the true data set, while a discriminator tries to discriminate between the true and fake data. Lloyd and Weedbrook [382] introduced the notion of quantum generative adversarial networks (QuGANs), where the data consists either of quantum states or of classical data and the generator and discriminator are equipped with quantum information processors. They showed that when the data consists of samples of measurements made on high-dimensional spaces, quantum adversarial networks may exhibit an exponential advantage over classical adversarial networks. QuGAN has been used to learn and load random distribution and can facilitate financial derivative pricing [159].

7.6 Quantum Neural Networks

Neural networks (NNs) have achieved many practical successes over the past decade, due to the novel computational power of GPUs. On the other side, the growing interest in quantum computing has led researchers to develop different variants of quantum neural networks (QNNs). However, many challenges arise in developing quantum analogues to the required components of NNs, such as the perceptron, an optimization algorithm, and a loss function. A few different approaches to QNNs have been developed. Most, however, follow the same steps [383]: initialize a network architecture; specify a learning task, implement a training algorithm, and simulate the learning task.

Current proposals for QNNs, discussed in the following subsections, involve a diverse collection of ideas with varying degrees of proximity to classical neural networks.

7.6.1 Quantum Feedforward Neural Network

Cao et al. [384] proposed a quantum neuron and demonstrated its applicability as a building block of quantum neural networks. Their approach uses repeat-until-success techniques for quantum gate synthesis. Kerenedis et al. developed a quantum analogue to the orthogonal feedforward neural network [385]; the orthogonality of the weight matrix is naturally preserved by using the reconfigurable beam splitter gate [386]. In addition, quantum algorithms can potentially speed up the operations used by classical feedforward neural networks [387]. Another quantum feedfoward NN has been developed by Beer et al. [383].

A different proposed path to designing quantum analogues to feedforward NNs on near-term devices is by using parameterized quantum circuits (Section 4.7) [333, 388–391]. Various attempts have been made to analyze the expressive power of the circuit-based QNN [324, 392–395]. Interestingly, the circuit-based QNN can be viewed as a kernel method with a quantum-enhanced feature map [324, 333, 394, 396]. Furthermore, analysis has been performed in the neural tangent kernel framework [397] to show the benefits of using a quantum-enhanced feature map [333] in conjunction with a classical NN [398].

7.6.2 Quantum Convolutional Neural Network

Convolutional neural networks (CNNs) were originally developed by LeCun et al. [399] in the 1980s. Recently, Kerenidis et al. [400] proposed a quantum convolutional neural network as a shallow circuit, reproducing completely the classical CNN, by allowing nonlinearities and pooling operations. They used a new quantum tomography algorithm with ℓ_∞ norm guarantees and new applications of probabilistic sampling in the context of

information processing. Quantum Fourier convolutional neural networks have been developed by Shen and Liu [401]. Cong et al. [402] developed circuit-based quantum analogues to CNNs.

7.6.3 Quantum Graph Neural Networks

Graph neural networks have recently attracted a lot of attention from the machine learning community. Effective neural networks are designed by modeling interactions (edges) between objects of interest (nodes) and assigning features to nodes and edges. The approach exploits real-world interconnectedness in finance. Graph neural networks are particularly helpful for training low-dimensional vector space representations of nodes that encode both complex graph structure and node/edge features. Verdon et al. [403] introduced quantum graph neural networks (QGNNs). This is a new type of quantum neural network ansatz that takes advantage of the graph structure of quantum processes and makes them effective on distributed quantum systems. In addition, quantum graph recurrent neural networks [404], quantum graph convolutional neural networks, and quantum spectral graph convolutional neural networks are introduced as special cases. A networked quantum system is described with an underlying graph $G = (V, E)$ in which each node v corresponds to a quantum subsystem with its Hilbert space $\mathcal{H}_v$ such that the entire system forms a Hilbert space $\mathcal{H} = \bigotimes_{v \in V} \mathcal{H}_v$. This can be extended to edges $\mathcal{H}_E$, and each node can represent more complicated quantum systems than a single qubit (e.g., several qubits or the entire part of a distributed quantum system). With this representation, the edges of a graph describe communication between vertex subspaces. The QGNN ansatz is a parameterized quantum circuit on a network, which consists of a sequence of different Hamiltonian evolutions, with the whole sequence repeated several times with variational parameterization. In this setting, the parameterized Hamiltonians have a graph topology of interactions.

7.7 Quantum Reinforcement Learning

Reinforcement learning is an area of machine learning that considers how agents ought to take actions in an environment in order to maximize their reward. It has been applied to many financial applications, including pricing and hedging of contingent claims, investment and portfolio allocation, buying and selling of a portfolio of securities subject to transaction costs, market making, asset liability management, and optimization of tax consequences.

The standard framework of reinforcement learning is based on discrete-time, finite-state Markov decision processes (MDPs). It comprises five components: $\{S, A_i, p_{ij}(a), r_{i,a}, V, i, j \in S, a \in A_i\}$, where S is the state space; A_i is the action space for state i; $p_{ij}(a)$ is the transition probability; r is the reward function defined as $r : \Gamma \to \{-\infty, +\infty\}$, where $\Gamma = \{(i, a) | i \in S, a \in A_i\}$; and V is a criterion function or objective function. The MDP has a history of successive states and decisions, defined as $h_n = (s_0, a_0, s_1, a_1, \cdots, s_{n-1}, a_{n-1}, s_n)$, and the policy π is a sequence $\pi = (\pi_0, \pi_1, \cdots)$. When the history at n is h_n, the agent will make a decision according to the probability distribution $\pi_n(\cdot|h_n)$ on A_{s_n}. At each step t, the agent observes the state of the environment s_t and then chooses an action a_t. The agent will then receive a reward r_{t+1}. The environment then will change to the next state s_{t+1} under action a_t, and the agent will again observe and act. The goal of reinforcement learning is to learn a mapping from states to action that maximize the expected sum of discounted reward or, formally, to learn a policy $\pi : S \times \bigcup_{i \in S} A_i \to [0, 1]$, such that

$$\begin{aligned} \max V_s^\pi &= \mathbb{E}[r_{t+1} + \gamma r_{t+2} + \cdots | s_t = s, \pi] \\ &= \sum_{a \in A_s} \pi(s, a)[r_s^a + \gamma \sum_{s'} p_{ss'} V_{s'}^\pi], \end{aligned}$$

where $\gamma \in [0, 1]$ is a discount factor and $\pi(s, a)$ is the probability of the agent to selecting action a at state s according to policy π.

Dong et al. [405] proposed representing the state set with a quantum superposition state; the eigenstate obtained by randomly observing the quantum state is the action. The probability of the eigen action is determined by the probability amplitude, which is updated in parallel according to the rewards. The probability of the "good" action is amplified by using iterations of Grover's algorithm (Section 4.1). In addition, they showed that the approach makes a good trade-off between exploration and exploitation using the probability amplitude and can speed up learning through quantum parallelism. Paparo et al. [406] showed that the computational complexity of a particular model, projective simulation, can be quadratically reduced. In addition, quantum neural networks (Section 7.6) and quantum Boltzmann machines (Section 7.5.3) have been used for approximate RL [407–411]. Cherret et al. described quantum reinforcement learning via policy iteration [412]. Additionally, Cornelissen explored how to apply quantum gradient estimation to quantum reinforcement learning [413].

7.8 Natural Language Modeling

Natural language processing (NLP) has flourished over the past couple of years as a result of advancements in deep learning [414]. One important component of NLP is building language models [415]. A language model is a statistical model used to predict what word will likely appear next in a sentence helpful. Today, deep neural networks using transformers [416] are utilized to construct language models [417–419] for a variety of tasks. Concerns remain, however, such as bias [420], controversial conclusions [421], energy consumption, and environmental impact [422]. Thus it makes sense to look to quantum computing for potentially better language models for NLP.

The DisCoCat framework, developed by Coecke et al. [423], combines both meaning and grammar, something not previously done, into a single language model utilizing category theory. For grammar, they utilized pregroups [424], sets with relaxed group axioms, to validate sentence grammar. Semantics is represented by a vector space model, similar to those commonly used in NLP [425]. However, DisCoCat also allows for higher-order tensors to distinguish different parts of speech.

Coecke et al. noted that pregroups and vector spaces are asymmetric and symmetric strict monoidal categories, respectively. Thus, they can be represented by utilizing the powerful diagrammatic framework associated with monoidal categories [426]. The revelation made by this group [427] was that this same diagrammatic framework is utilized by categorical quantum mechanics [428]. Because of the similarity between the two, the diagrams can be represented by quantum circuits [427] utilizing the algebra of ZX-calculus [429]. Thus, because of this connection and the use of tensor operations, the group argued that this natural language model is "quantum-native," meaning it is more natural to run the DisCoCat model on a quantum computer instead of a classical one [427]. Coecke et al. ran multiple experiments on NISQ devices for a variety of simple NLP tasks [430–432].

In addition, there are many ways in which QML models, such as QNNs (Section 7.6), could potentially enhance the capacity of existing natural language neural architectures [433–435].

7.9 Financial Applications

The field of quantum machine learning is still in its infancy, and many exciting developments are under way. We list here a few representative or potential financial applications. We also point readers to a recent survey on quantum machine learning for finance [6].

7.9.1 Anomaly Detection

Anomaly detection is a critical task for financial intuitions, from both a financial point of view and a technological one. In finance, fraud detection is an extremely important form of anomaly detection. Some examples are identifying fraudulent credit card transactions and financial documents. Also important is anomaly detection on communication networks used by the financial industry [436]. These tasks are typically performed, classically, by utilizing techniques from statistical analysis and machine learning [437]. The machine learning technique commonly used is unsupervised deep learning [438].

Researchers have also proposed quantum versions of these approaches, such as quantum GANs (Section 7.6). Herr et al. [439] used parameterized quantum circuits to model the generator used in AnoGan [440] for anomaly detection. Quantum amplitude estimation (Section 4.4) was utilized to enhance density-estimation-based anomaly detection [441]. Quantum variants of common clustering methods also exist, such as k-means (Section 7.3), that could be useful for anomaly detection. Furthermore, quantum-enhanced kernels (Section 7.2) could be utilized for kernel-based clustering methods [442]. Quantum Boltzmann machines (Section 7.5.3) have also been used for fraud detection [273].

7.9.2 Asset Pricing

The fundamental goal of asset pricing is to understand the behavior of risk premiums. Classically, machine learning has shown great potential to improve our empirical understanding of asset returns. For example, Gu et al. [443] studied the problem of predicting returns in cross-section and time series. They studied a collection of machine learning methods, including linear regression, generalized linear models, principal component regression, and neural networks. Chen et al. [444] proposed an asset pricing model for individual stock returns using deep neural networks.

The quantum version of these machine learning methods could potentially be applied to asset pricing, and investigations of the applicability are needed. Nagel [445] goes into detail about classical machine learning methods applied to asset pricing. Chen et al. [434] proposed a quantum version of the long short-term memory architecture, which is a variant of the recurrent neural network that has been demonstrated to be effective for temporal and sequence data modeling. Both make use of parameterized quantum circuits, and these QML models may offer an advantage over classical models in terms of training complexity and prediction performance.

7.9.3 Implied Volatility

Implied volatility is a metric that captures the market's forecast of a likely movement in the price of a security. It is one of the deciding factors in the pricing of options. Sakuma [446] investigated the use of the deep quantum neural network proposed by Beer et al. [383] in the context of learning implied volatility. Numerical results suggest that such a QML model is a promising candidate for developing powerful methods in finance [6].

8 Hardware Implementations of Use Cases

In this section we discuss experiments that were performed on quantum hardware to solve financial problems using some of the methods that have been presented. The hardware platforms include superconducting and trapped-ion universal quantum processors, as well as superconducting quantum annealers.

8.1 Quantum Risk Analysis on a Transmon Device

In this section we present the experimental results collected by Woerner and Egger [215], using their method for risk analysis mentioned in Section 5.3.2. They utilized the IBM transmon [55] quantum processors [215].

The authors tested the algorithm on two toy models: pricing a Treasury-bill, executed on quantum hardware, and determining the financial risk of a two-asset portfolio, performed in simulations that included modeling quantum noise on classical hardware. The QAE algorithm is used to estimate quantities related to a random variable represented as a quantum state, usually using n qubits to map it to the interval $\{0, ..., N-1\}$, with $N = 2^n$ being the number of possible realizations. The authors presented a method for efficiently applying an approximation to $f(X)$, where QAE computes $\mathbb{E}[f(X)]$, given that $f(x) = \sin^2(p(x))$ for some polynomial p. They used this approach to compute statistical moments, such as the mean. The $f(X)$ used for VaR, and similarly CVaR, requires using a quantum comparator, as mentioned in Section 5.3.2. In addition, for computing VaR, a bisection search is then performed utilizing multiple executions of QAE to find a threshold x such that $\mathbb{E}[f(X)] = \mathbb{P}[X \leq x] \geq \alpha$, for the chosen confidence α. For the comparator, the authors referenced the quantum ripple-carry adder of Cuccaro et al. [447]. For each of these methods, they discussed the trade-off between circuit depth, qubit count, and convergence rate. An example circuit for 3 evaluation qubits, corresponding to $2^3 = 8$ samples (i.e., quantum queries, Section 5.1), is shown in Figure 1 for computing the value of a T-Bill. Increasing the number of evaluation qubits above 3 yields smaller estimation errors than with classical Monte Carlo integration (Section 5.1), demonstrating the predicted quantum speedup; see Figure 2.

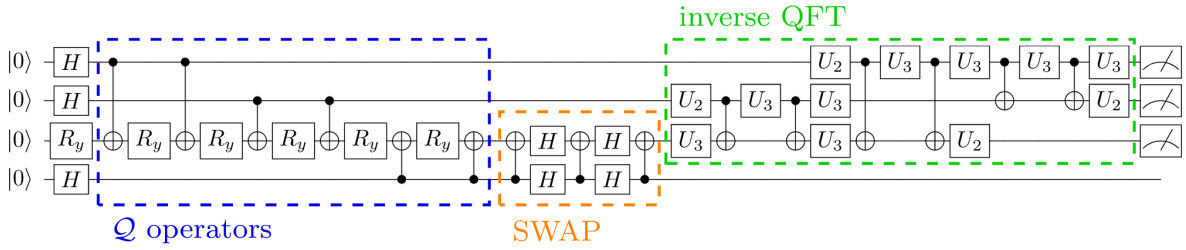


Figure 1: Quantum amplitude estimation circuit for approximating the expected value of a portfolio made up of one T-Bill. The algorithm uses 3 evaluation qubits, corresponding to 8 samples. (Image source: [215]; use permitted under the Creative Commons Attribution 4.0 International License.)

Despite the observed speedup of the QMCI-based algorithm, compared with classical Monte Carlo integration, practical quantum advantage cannot be achieved by using this algorithm on NISQ devices because of the relatively small qubit count, qubit connectivity and quality, and decoherence limitations present in modern quantum processors (Section 3.3). Additionally, Monte Carlo for numerical integration can be efficiently parallelized, imposing even stronger requirements on quantum algorithms in order to outperform standard classical methods.

8.2 Portfolio Optimization with Reverse Quantum Annealing

As explained in Section 6.4.1, quantum annealing (Section 6.1.1) is highly amenable to solving portfolio optimization problems. The commercially available D-Wave flux-qubit-based [55] quantum annealers have been successfully used to solve portfolio optimization instances. Determining the optimal portfolio composition is one of the most studied optimization problems in finance. A typical portfolio optimization problem can be cast into a QUBO or, equivalently, an Ising Hamiltonian suitable for a quantum annealer; see Section 6.1. The following presents the work of Venturelli and Kondratyev [448], who used a reverse-quantum-annealing-based approach.

Portfolio optimization was defined in Section 6.4.1. The pairwise relationships between the asset choices give rise to the quadratic form of the objective function to be optimized. The following technical challenges

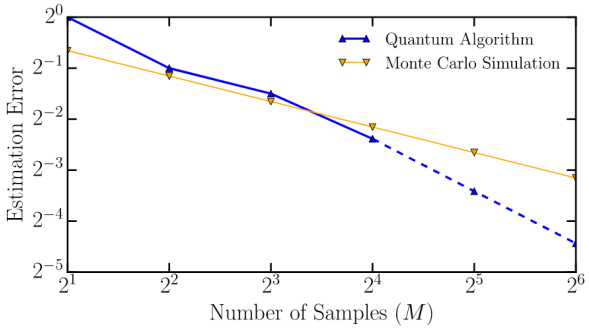


Figure 2: Results of executing the QAE algorithm on the IBMQ 5 Yorktown (ibmqx2) chip for a range of evaluation qubits, with 8,192 shots each. Because of the better convergence rate of the quantum algorithm (blue line), it achieves lower error rates than does Monte Carlo (orange line) for the version with 4 qubits, with the performance gap estimated to grow even more for larger instances on 5 and 6 qubits (blue dashed line). (Image source: [215]; use permitted under the Creative Commons Attribution 4.0 International License, http://creativecommons.org/licenses/by/4.0/)

need to be overcome in order to perform simulations on the D-Wave quantum annealer. First, the standard way of enforcing the number of selected assets by introducing a high-energy penalty term can be problematic with D-Wave devices because of precision issues and the physical limit on energy scales that can be programmed on the chip. The authors observed that this constraint can be enforced by artificially shifting the Sharpe ratios, used when computing the risk-adjusted returns of the assets, by a fixed amount. Second, embedding the Ising optimization problem in the Chimera topology of D-Wave quantum annealers presents a strong limitation on the problems that can be solved on the device (Section 6.1.1). To overcome this limitation, one can employ the minor-embedding compilation technique for fully connected graphs [242, 449], allowing the necessary embedding to be performed using an extended set of variables.

The novelty of this work by Venturelli and Kondratyev is the use of the reverse annealing protocol [237], which was found to be on average 100+ times faster than forward annealing, as measured by the time to solution (TTS) metric. Reverse quantum annealing, briefly mentioned in Section 6.1.1, is illustrated in Figure 3. It is a new approach enabled by the latest D-Wave architecture designed to help escape local minima through a combination of quantum and thermal fluctuations. The annealing schedule starts in some classical state provided by the user, as opposed to the quantum superposition state, as in the case of a forward annealing. The transverse field is then increased, driving backward annealing from the classical initial state to some intermediate quantum superposition state. The annealing schedule functions $A(t)$ and $B(t)$ (Section 6.1.1) are then kept constant for a fixed amount of time, allowing the system to perform the search for a global minimum. The reverse annealing schedule is completed by removing the transverse field, effectively performing the forward annealing protocol with the system evolving to the ground state of the problem Hamiltonian.

The DW2000Q device used in [448] allows embedding up to 64 logical binary variables on a fully-connected graph, and the authors considered portfolio optimization problems with 24–60 assets, with 30 randomly generated instances for each size, and calculating the TTS. The results for both forward and reverse quantum annealing were then compared with the industry-established genetic algorithm (GA) approach used as a classical benchmark heuristic. The TTS for reverse annealing with the shortest annealing and pause times was found to be several orders of magnitude smaller than for the forward annealing and GA approaches, demonstrating a promising step toward practical applications of NISQ devices.

The development of new quantum annealing protocols and approaches, such as the reverse quantum annealing considered in this section, is an active area of research.

8.3 Portfolio Optimization on a Trapped-Ion Device

As discussed in Section 6.4.1, the convex mean-variance portfolio optimization problem with linear equality constraints can be solved as a linear system of equations. The method of Lagrange multipliers turns the convex quadratic program (Equation (15)) into

$$\begin{bmatrix} 0 & 0 & \vec{r}^{\,\intercal} \\ 0 & 0 & \vec{p}^{\,\intercal} \\ \vec{r} & \vec{p} & \Sigma \end{bmatrix} \begin{bmatrix} \eta \\ \theta \\ \vec{w} \end{bmatrix} = \begin{bmatrix} \mu \\ \xi \\ \vec{0} \end{bmatrix}, \tag{17}$$

where η, θ are the dual variables. This lends itself to quantum linear systems algorithms (Section 4.5).

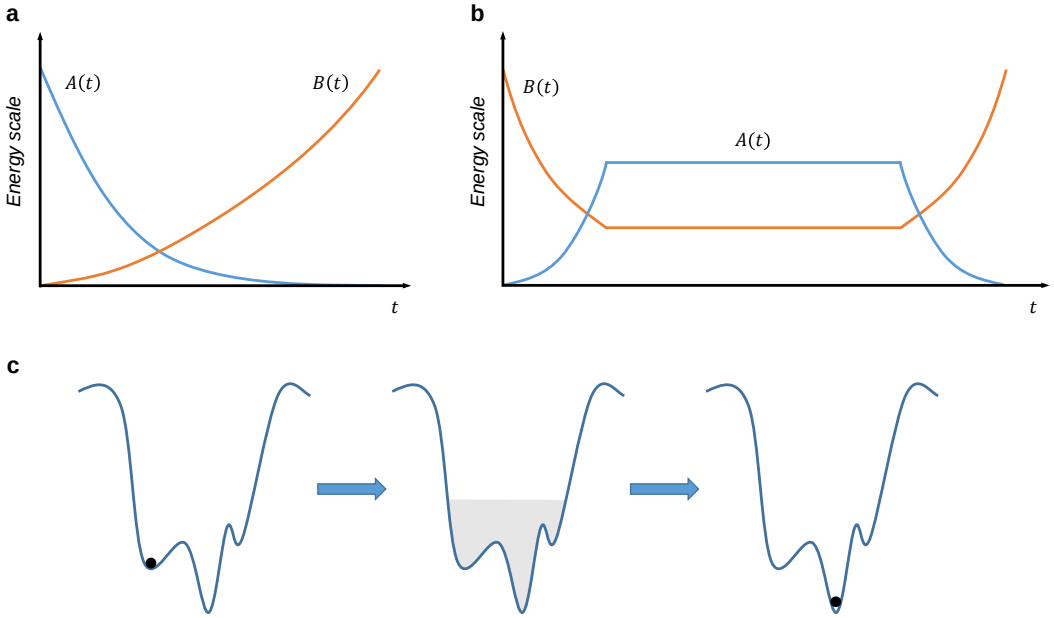


Figure 3: Forward (a) and reverse (b) quantum annealing protocols. During reverse quantum annealing, the system is initialized in a classical state (c, left), evolved following a "backward" annealing schedule to some quantum superposition state. At that point the evolution is interrupted for a fixed amount of time, allowing the system to perform a global search (c, middle). The system's evolution then is completed by the forward annealing schedule (c, right). Adapted from [237].

To reiterate, solving a QLSP consists of returning a quantum state, with bounded error, that is proportional to the solution of a linear system $A\vec{x} = \vec{b}$. Quantum algorithms for this problem on sparse and well-conditioned matrices potentially achieve an exponential speedup in the system size N. Although a QLS algorithm cannot provide classical access to amplitudes of the solution state while maintaining the exponential speedup in N, potentially useful financial statistics can be obtained from the state.

The QLS algorithm discussed in this section is HHL. This algorithm is typically viewed as one that is intended for the fault-tolerant era of quantum computation. Two of the main components of the algorithm that contribute to this constraint are quantum phase estimation (Section 4.3) and the loading of the inverse of the eigenvalue estimations onto the amplitudes of an ancilla, called eigenvalue inversion [88].[15] Yalovetzky et al. [450] developed an algorithm called NISQ-HHL with the goal of applying it to portfolio optimization problems on NISQ devices. While it does not retain the asymptotic speedup of HHL, the authors' approach can potentially provide benefits in practice. They applied their algorithm to small mean-variance portfolio optimization problems and executed them on the trapped-ion Quantinuum System Model H1 device [63].

QPE is typically difficult to run on near-term devices because of the large number of ancillary qubits and controlled operations required for the desired eigenvalue precision (depth scales as $O(\frac{1}{\delta})$ for precision δ). An alternative realization of QPE reduces the number of ancillary qubits to one and replaces all controlled-phase gates in the quantum Fourier transform component of QPE with classically controlled single-qubit gates [451]. This method requires mid-circuit measurements, ground-state resets, and quantum conditional logic (QCL). These features are available on the Quantinuum System Model H1 device. The paper calls this method QCL-QPE. The differences between these two QPE methods are highlighted in Figure 4.

With regard to eigenvalue inversion, implementing the controlled rotations on near-term devices typically requires a multicontrolled rotation for every possible eigenvalue that can be estimated by QPE, such as the uniformly controlled rotation gate [452]. This circuit is exponentially deep in the number of qubits. Fault-tolerant implementations can take advantage of quantum arithmetic methods to implement efficient algorithms for the required arcsin computation. An alternative approach for the near term is a hybrid version of HHL [453], which the developers of NISQ-HHL extended. This approach utilizes QPE to first estimate the required eigenvalues to perform the rotations for. While the number of controlled rotations is now reduced, this approach does not retain the asymptotic speedup provided by HHL because of the loss of coherence. A novel procedure

[15] Hamiltonian simulation also contributes to this constraint.

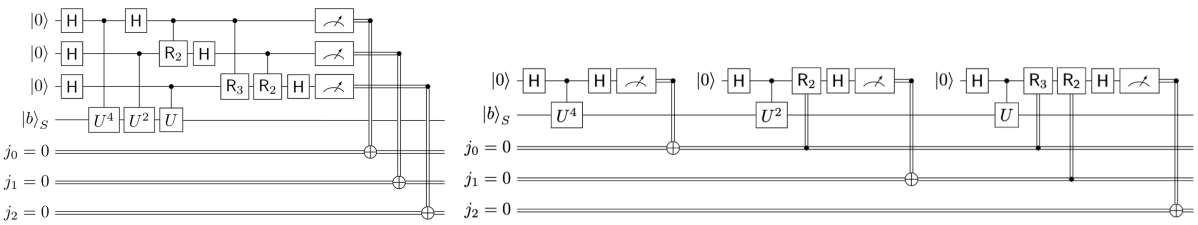


Figure 4: Circuits for estimating the eigenvalues of the unitary operator U to three bits using QPE (left circuit) or QCL-QPE (right circuit). S is the register that U is applied to, and j is a classical register; H refers to the Hadamard gate; and R_k, for $\mathsf{k} = 2, 3$, are the standard phase gates used by the quantum Fourier transform [2]. Adapted from [450].

was developed by the authors for scaling the matrix by a factor γ to make it easier to distinguish the eigenvalues in the output distribution of QPE and improve performance. Figure 5 displays experimental results collected from the Quantinuum H1 device to show the effect of the scaling procedure.[16]

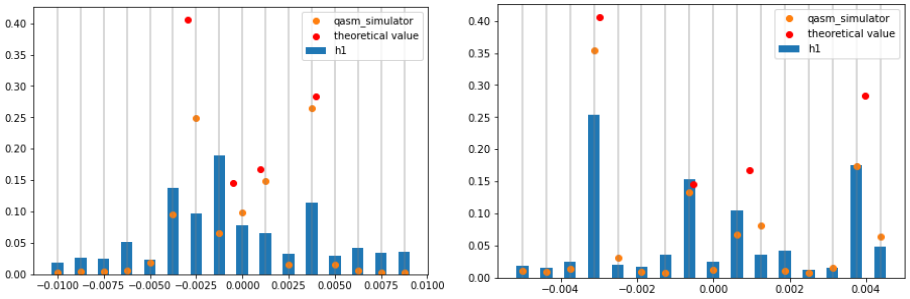


Figure 5: Probability distributions over the eigenvalue estimations from the QCL-QPE run using $\gamma = 50$ (left) and $\gamma = 100$ (right). The blue bars represent the experimental results on the H1 machine—1,000 shots (left) and 2,000 shots (right)—and we compare them with the theoretical eigenvalues (classically calculated using a numerical solver) represented with the red dots and the results from the Qiskit [454] QASM simulator represented with the orange dots. Adapted from [450].

These methods were executed on the Quantinuum H1 device for a few small portfolios (Table 2). A controlled-SWAP test [455] was performed between the state from HHL and the solution to the linear system obtained classically (loaded onto a quantum state) [450]. The number of H1 two-qubit ZZMax gates used is also displayed in Table 2.

	HHL	NISQ-HHL	Relative Change
Rotations	7	6	-14.29%
Eigenvalue Inversion Depth	138	120	-13.04%
Inner Product in Simulation	0.59	0.83	40.67%
Inner Product on H1-1	0.42	0.46	9.52%

Table 2: Comparison of the number of rotations in the eigenvalue inversion, ZZMax depth of the HHL plus the controlled-SWAP test circuits and the inner product calculated from the Qiskit state vector simulator and the Quantinuum H1 results with 3,000 shots. Adapted from [450].

Statistics such as risk can be computed efficiently from the quantum state. Sampling can also be performed if the solution state is sparse enough [303].

9 Conclusion and Outlook

Quantum computers are expected to surpass the computational capabilities of classical computers and achieve disruptive impact on numerous industry sectors, such as global energy and materials, pharmaceuticals and medical products, telecommunication, travel, logistics, and finance. In particular, the finance sector has a history of

[16] Let Π_λ denote the eigenspace projector for an eigenvalue λ of the Hermitian matrix in Equation (17), and let $|b\rangle$ be a quantum state proportional to the right side of Equation (17). To clarify, the y-axis value of the red dot corresponding to λ, on either of the histograms in Figure 5, is $\|\Pi_\lambda |b\rangle\|_2^2$.

creation and first adaptation of new technologies. This is true also when it comes to quantum computing. In fact, finance is estimated to be the first industry sector to benefit from quantum computing, not only in the medium and long terms, but even in the short term, because of the large number of financial use cases that lend themselves to quantum computing and their amenability to be solved effectively even in the presence of approximations.

A common misconception about quantum computing is that quantum computers are simply faster processors and hence speedups will be automatically achieved by porting a classical algorithm from a classical computer to a quantum one—just like moving a program to a system-upgraded hardware. In reality, quantum computers are fundamentally different from classical computers, so much so that the algorithmic process for solving an application has to be completely redesigned based on the architecture of the underlying quantum hardware.

Most of the use cases that characterize the finance industry sector have high computational complexity, and for this reason they lend themselves to quantum computing. However, the computational complexity of a problem per se is not a guarantee that quantum computing can make a difference. To assess commercial viability, we must first answer a couple questions. First, is there in fact any need for improved speed of calculations or computational-accuracy for the given application? Determining whether there is a gap in what the current technology can provide us is essential and is a question that financial players typically ask when doing proof of concept projects. Second, when adapting a classical finance application to quantum computing, can that application achieve quantum speedup and, if so, when will that happen based on the estimated evolution of the underlying quantum hardware?

Today's quantum computers are not yet capable of solving real-life-scale problems in the industry more efficiently and more accurately than classical computers can. Nevertheless, proofs of concept for quantum advantage and even quantum supremacy have already been provided (see Section 3). There is hope that the situation will change in the coming years with demonstrations of quantum advantage. It is crucial for enterprises, and particularly financial institutions, to use the current time to become quantum ready in order to avoid being left behind when quantum computers become operational in a production environment.

We are in the early stages of the quantum revolution, yet we are already observing a strong potential for quantum technology to transform the financial industry. So far, the community has developed potential quantum solutions for portfolio optimization, derivatives pricing, risk modeling, and several problems in the realm of artificial intelligence and machine learning, such as fraud detection and NLP.

In this paper we have provided a comprehensive review of quantum computing for finance, specifically focusing on quantum algorithms that can solve computationally challenging financial problems. We have described the current landscape of the state of the industry. We hope this work will be used by the scientific community, both in industry and in academia, not only as a reference, but also as a source of information to identify new opportunities and advance the state of the art.

Competing Interests

The authors declare that they have no competing interests.

Funding

C.N.G's work was supported in part by the U.S. Department of Energy, Office of Science, Office of Workforce Development for Teachers and Scientists (WDTS) under the Science Undergraduate Laboratory Internships Program (SULI). Y.A.'s work at Argonne National Laboratory was supported by the U.S. Department of Energy, Office of Science, under contract DE-AC02-06CH11357.

Disclaimer

This paper was prepared for information purposes by the teams of researchers from the various institutions identified above, including the Future Lab for Applied Research and Engineering (FLARE) group of JPMorgan Chase Bank, N.A.. This paper is not a product of the Research Department of JPMorgan Chase & Co. or its affiliates. Neither JPMorgan Chase & Co. nor any of its affiliates make any explicit or implied representation or warranty and none of them accept any liability in connection with this paper, including, but limited to, the completeness, accuracy, reliability of information contained herein and the potential legal, compliance, tax or accounting effects thereof. This document is not intended as investment research or investment advice, or a recommendation, offer or solicitation for the purchase or sale of any security, financial instrument, financial product or service, or to be used in any way for evaluating the merits of participating in any transaction.

Authors' Contributions

Dylan Herman, Cody Ning Googin, and Xiaoyuan Liu contributed equally to the manuscript. All authors contributed to the writing and reviewed and approved the final manuscript.

Acronyms

NISQ Noisy-Intermediate Scale Quantum
AQC Adiabatic Quantum Computing
qRAM Quantum Random Access Memory
QCL Quantum Conditional Logic
VQE Variational Quantum Eigensolver
QAOA Quantum Approximate Optimization Algorithm
QML Quantum Machine Learning
PCA Principal Component Analysis
TDA Topological Data Analysis
QNN Quantum Neural Network
PQC Parameterized Quantum Circuit
GAN Generative Adversarial Network
SVM Support Vector Machine
PH Persistent Homology
GP Gaussian Process
NP Non-Deterministic Polynomial
VQA Variational Quantum Algorithm
QPE Quantum Phase Estimation
NLP Natural Language Processing
RL Reinforcement Learning
QAA Quantum Amplitude Amplification
QAE Quantum Amplitude Estimation
QLSP Quantum Linear Systems Problem
QLS Quantum Linear System
QLSA Quantum Linear Systems Algorithm
HHL Harrow Hassidim Lloyd
QSVT Quantum Singular Value Transform
QSVE Quantum Singular Value Estimation
CKS Childs, Kothari, and Somma
QA Quantum Annealing
IP Integer Programming
IQP Integer Quadratic Programming
QUBO Quadratic Unconstrained Binary Optimization
QAOA Quantum Approximate Optimization Algorithm
ITE Imaginary-Time Evolution
VarQITE Variational Quantum Imaginary Time Evolution
MCI Monte Carlo Integration
QMCI Quantum Monte Carlo Integration
VaR Value at Risk
CVaR Conditional Value at Risk
CDO Collateralized Debt Obligation
ECR Economic Capital Requirement
CPU Central Processing Unit
GPU Graphics Processing Unit

References

[1] Alexandre Ménard, Ivan Ostojic, Mark Patel, and Daniel Volz. A game plan for quantum computing, 2020.

[2] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 2010.

[3] Daniel J Egger, Claudio Gambella, Jakub Marecek, Scott McFaddin, Martin Mevissen, Rudy Raymond, Andrea Simonetto, Stefan Woerner, and Elena Yndurain. Quantum computing for finance: State-of-the-art and future prospects. *IEEE Transactions on Quantum Engineering*, 1:1–24, 2020.

[4] Adam Bouland, Wim van Dam, Hamed Joorati, Iordanis Kerenidis, and Anupam Prakash. Prospects and challenges of quantum finance. *arXiv preprint arXiv:2011.06492*, 2020.

[5] Roman Orus, Samuel Mugel, and Enrique Lizaso. Quantum computing for finance: Overview and prospects. *Reviews in Physics*, 4:100028, 2019.

[6] Marco Pistoia, Syed Farhan Ahmad, Akshay Ajagekar, Alexander Buts, Shouvanik Chakrabarti, Dylan Herman, Shaohan Hu, Andrew Jena, Pierre Minssen, Pradeep Niroula, Arthur Rattew, Yue Sun, and Romina Yalovetzky. Quantum Machine Learning for Finance. *IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, November 2021. ICCAD Special Session Paper.

[7] Andrés Gómez, Álvaro Leitao, Alberto Manzano, Daniele Musso, María R Nogueiras, Gustavo Ordóñez, and Carlos Vázquez. A survey on quantum computational finance for derivatives pricing and VaR. *Archives of Computational Methods in Engineering*, pages 1–27, 2022.

[8] Paul Griffin and Ritesh Sampat. Quantum computing for supply chain finance. In *2021 IEEE International Conference on Services Computing (SCC)*, pages 456–459. IEEE, 2021.

[9] Apoorva Ganapathy. Quantum computing in high frequency trading and fraud detection. *Engineering International*, 9(2):61–72, 2021.

[10] Research and Markets. Financial Services Global Market Report 2021: COVID-19 Impact and Recovery to 2030, 2021.

[11] Basel Committee on Banking Supervision. High-level summary of Basel III reforms, 2017.

[12] Paul Glasserman. *Monte Carlo methods in financial engineering*, volume 53. Springer, 2004.

[13] Heath P. Terry, Debra Schwartz, and Tina Sun. The Future of Finance: Volume 3: The socialization of finance, 2015.

[14] Salesforce. Trends in Financial Services, 2020.

[15] Tucker Bailey, Soumya Banerjee, Christopher Feeney, and Heather Hogsett. Cybersecurity: Emerging challenges and solutions for the boards of financial-services companies, 2020.

[16] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, Oct 1997.

[17] Craig Gidney and Martin Ekerå. How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits. *Quantum*, 5:433, Apr 2021.

[18] Martin Roetteler, Michael Naehrig, Krysta M Svore, and Kristin Lauter. Quantum resource estimates for computing elliptic curve discrete logarithms. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 241–270, 2017.

[19] Andrew M Childs and Wim Van Dam. Quantum algorithms for algebraic problems. *Reviews of Modern Physics*, 82(1):1, 2010.

[20] D. J. Bernstein and T. Lange. Post-quantum cryptography. *Nature*, 2017.

[21] Nicolas Gisin, Grégoire Ribordy, Wolfgang Tittel, and Hugo Zbinden. Quantum cryptography. *Reviews of Modern Physics*, 74(1):145, 2002.

[22] Matthew Amy, Olivia Di Matteo, Vlad Gheorghiu, Michele Mosca, Alex Parent, and John Schanck. Estimating the cost of generic quantum pre-image attacks on SHA-2 and SHA-3. In *International Conference on Selected Areas in Cryptography*, pages 317–337. Springer, 2016.

[23] Akinori Hosoyamada and Yu Sasaki. Quantum collision attacks on reduced SHA-256 and SHA-512. *IACR Cryptol. ePrint Arch.*, 2021:292, 2021.

[24] Samuel Jaques, Michael Naehrig, Martin Roetteler, and Fernando Virdia. Implementing Grover oracles for quantum key search on AES and LowMC. *Advances in Cryptology–EUROCRYPT 2020*, 12106:280, 2020.

[25] Xavier Bonnetain, André Schrottenloher, and Ferdinand Sibleyras. Beyond quadratic speedups in quantum attacks on symmetric schemes. *IACR-EUROCRYPT-2022*, 10 2021.

[26] Laurence A. Wolsey and George L Nemhauser. *Integer and combinatorial optimization*, volume 55. John Wiley & Sons, 1999.

[27] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.

[28] Yuri Alexeev, Dave Bacon, Kenneth R. Brown, Robert Calderbank, Lincoln D. Carr, Frederic T. Chong, Brian DeMarco, Dirk Englund, Edward Farhi, Bill Fefferman, Alexey V. Gorshkov, Andrew Houck, Jungsang Kim, Shelby Kimmel, Michael Lange, Seth Lloyd, Mikhail D. Lukin, Dmitri Maslov, Peter Maunz, Christopher Monroe, John Preskill, Martin Roetteler, Martin J. Savage, and Jeff Thompson. Quantum computer systems for scientific discovery. *PRX Quantum*, 2(1), February 2021.

[29] Gagnesh Kumar and Sunil Agrawal. CMOS limitations and futuristic carbon allotropes. In *2017 8th IEEE Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON)*, pages 68–71. IEEE, 2017.
[30] Frank Arute, Kunal Arya, Ryan Babbush, Dave Bacon, Joseph C. Bardin, Rami Barends, Rupak Biswas, Sergio Boixo, Fernando G. S. L. Brandao, David A. Buell, et al. Quantum supremacy using a programmable superconducting processor. *Nature*, 574(7779):505–510, 2019.
[31] John Preskill. Quantum computing and the entanglement frontier. *arXiv preprint arXiv:1203.5813*, 2012.
[32] John Preskill. Quantum computing in the NISQ era and beyond. *Quantum*, 2:79, Aug 2018.
[33] Ramamurti Shankar. *Principles of quantum mechanics*. Springer Science & Business Media, 1994.
[34] Oswald Veblen and John Wesley Young. *Projective geometry*, volume 2. Ginn, 1918.
[35] Steven Roman. *Advanced linear algebra*. Springer, 2008.
[36] John D. Dollard and Charles N. Friedman. *Product Integration with Application to Differential Equations*. Encyclopedia of Mathematics and its Applications. London: Addison-Wesley, 1984.
[37] A. Yu. Kitaev, A. H. Shen, and M. N. Vyalyi. *Classical and Quantum Computation*. American Mathematical Society, USA, 2002.
[38] Yuchen Wang, Zixuan Hu, Barry C. Sanders, and Sabre Kais. Qudits and high-dimensional quantum computing. *Frontiers in Physics*, 8, Nov 2020.
[39] Samantha Buck, Robin Coleman, and Hayk Sargsyan. Continuous variable quantum algorithms: an introduction. *arXiv preprint arXiv:2107.02151*, 2021.
[40] Dong-Sheng Wang. A comparative study of universal quantum computing models: towards a physical unification a comparative study of universal quantum computing models: towards a physical unification. *Quantum Engineering*, page e85, 2021.
[41] Dorit Aharonov, Wim Van Dam, Julia Kempe, Zeph Landau, Seth Lloyd, and Oded Regev. Adiabatic quantum computation is equivalent to standard quantum computation. *SIAM Review*, 50(4):755–787, 2008.
[42] Daniel A. Lidar and Todd A. Brun. *Quantum Error Correction*. Cambridge University Press, 2013.
[43] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland. Surface codes: Towards practical large-scale quantum computation. *Physical Review A*, 86(3), Sep 2012.
[44] Arthur G Rattew, Yue Sun, Pierre Minssen, and Marco Pistoia. The efficient preparation of normal distributions in quantum registers. *Quantum*, 5:609, 2021.
[45] Xin-Chuan Wu, Sheng Di, Emma Maitreyee Dasgupta, Franck Cappello, Hal Finkel, Yuri Alexeev, and Frederic T. Chong. Full-state quantum circuit simulation by using data compression. *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, November 2019.
[46] Xin-Chuan Wu, Sheng Di, Emma Maitreyee Dasgupta, Franck Cappello, Hal Finkel, Yuri Alexeev, and Frederic T Chong. Full-state quantum circuit simulation by using data compression. In *Proceedings of the High Performance Computing, Networking, Storage and Analysis International Conference (SC19)*, Denver, CO, USA, 2019. IEEE Computer Society.
[47] Xin-Chuan Wu, Sheng Di, Franck Cappello, Hal Finkel, Yuri Alexeev, and Frederic Chong. Memory-efficient quantum circuit simulation by using lossy data compression. In *Proceedings of the 3rd International Workshop on Post-Moore Era Supercomputing (PMES) at SC18*, Denver, CO, USA, 2018.
[48] Xin-Chuan Wu, Sheng Di, Franck Cappello, Hal Finkel, Yuri Alexeev, and Frederic T Chong. Amplitude-aware lossy compression for quantum circuit simulation. In *Proceedings of 4th International Workshop on Data Reduction for Big Scientific Data (DRBSD-4) at SC18*, 2018.
[49] Danylo Lykov, Roman Schutski, Alexey Galda, Valerii Vinokur, and Yuri Alexeev. Tensor network quantum simulator with step-dependent parallelization. *arXiv preprint arXiv:2012.02430*, 2020.
[50] Danylo Lykov, Angela Chen, Huaxuan Chen, Kristopher Keipert, Zheng Zhang, Tom Gibbs, and Yuri Alexeev. Performance evaluation and acceleration of the QTensor quantum circuit simulator on GPUs. *2021 IEEE/ACM Second International Workshop on Quantum Computing Software (QCS)*, nov 2021.
[51] Danylo Lykov and Yuri Alexeev. Importance of diagonal gates in tensor network simulations. 2021.
[52] Tameem Albash and Daniel A. Lidar. Adiabatic quantum computation. *Reviews of Modern Physics*, 90(1), Jan 2018.
[53] Jérémie Roland and Nicolas J. Cerf. Quantum search by local adiabatic evolution. *Physical Review A*, 65(4), Mar 2002.
[54] Jacob D. Biamonte and Peter J. Love. Realizable Hamiltonians for universal adiabatic quantum computers. *Physical Review A*, 78(1), Jul 2008.
[55] P. Krantz, M. Kjaergaard, F. Yan, T. P. Orlando, S. Gustavsson, and W. D. Oliver. A quantum engineer's guide to superconducting qubits. *Applied Physics Reviews*, 6(2):021318, 2019.
[56] Colin D. Bruzewicz, John Chiaverini, Robert McConnell, and Jeremy M. Sage. Trapped-ion quantum computing: Progress and challenges. *Applied Physics Reviews*, 6(2):021314, 2019.

[57] Antti P. Vepsäläinen, Amir H. Karamlou, John L. Orrell, Akshunna S. Dogra, Ben Loer, Francisca Vasconcelos, David K. Kim, Alexander J. Melville, Bethany M. Niedzielski, Jonilyn L. Yoder, et al. Impact of ionizing radiation on superconducting qubit coherence. *Nature*, 584(7822):551–556, 2020.

[58] Mohan Sarovar, Timothy Proctor, Kenneth Rudinger, Kevin Young, Erik Nielsen, and Robin Blume-Kohout. Detecting crosstalk errors in quantum information processors. *Quantum*, 4:321, Sep 2020.

[59] Sergey Bravyi, Sarah Sheldon, Abhinav Kandala, David C. Mckay, and Jay M. Gambetta. Mitigating measurement errors in multiqubit experiments. *Phys. Rev. A*, 103(4), 2021.

[60] Jacob R. West, Daniel A. Lidar, Bryan H. Fong, and Mark F. Gyure. High fidelity quantum gates via dynamical decoupling. *Phys. Rev. Lett.*, 105(23), Dec 2010.

[61] A. Yu. Kitaev. Fault-tolerant quantum computation by anyons. *Annals of Physics*, 303(1):2–30, 2003.

[62] Austin G Fowler and John M Martinis. Quantifying the effects of local many-qubit errors and nonlocal two-qubit errors on the surface code. *Physical Review A*, 89(3):032316, 2014.

[63] J. Pino, Joan Dreiling, C. Figgatt, J. Gaebler, S. Moses, M. Allman, C. Baldwin, Michael Foss-Feig, D. Hayes, K. Mayer, C. Ryan-Anderson, and Brian Neyenhuis. Demonstration of the trapped-ion quantum CCD computer architecture. *Nature*, 2021.

[64] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to algorithms*. MIT press, 2009.

[65] Sanjeev Arora and Boaz Barak. *Computational complexity: a modern approach*. Cambridge University Press, 2009.

[66] Andris Ambainis. Understanding quantum algorithms via query complexity. In *Proceedings of the International Congress of Mathematicians: Rio de Janeiro 2018*, pages 3265–3285. World Scientific, 2018.

[67] Lov K Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of Computing*, pages 212–219, 1996.

[68] Charles H. Bennett, Ethan Bernstein, Gilles Brassard, and Umesh Vazirani. Strengths and weaknesses of quantum computing. *SIAM Journal on Computing*, 26(5):1510–1523, 1997.

[69] I. Kerenidis and A. Prakash. Quantum recommendation systems. *arXiv preprint arXiv:1603.08675*, 2016.

[70] Vittorio Giovannetti, Seth Lloyd, and Lorenzo Maccone. Quantum random access memory. *Physical Review Letters*, 100(16), April 2008.

[71] Aram W Harrow. Small quantum computers and large classical data sets. *arXiv preprint arXiv:2004.00026*, 2020.

[72] Andris Ambainis. Quantum search with variable times. *Theory of Computing Systems*, 47(3):786–807, 2010.

[73] Gilles Brassard, Peter Høyer, Michele Mosca, and Alain Tapp. Quantum amplitude amplification and estimation. *Quantum Computation and Information*, page 53–74, 2002.

[74] Dominic W Berry, Andrew M Childs, Richard Cleve, Robin Kothari, and Rolando D Somma. Exponential improvement in precision for simulating sparse Hamiltonians. In *Proceedings of the forty-sixth annual ACM symposium on Theory of Computing*, pages 283–292, 2014.

[75] M. Boyer, G. Brassard, P. Høyer, and A. Tapp. Tight bounds on quantum searching. *Fortschritte der Physik: Progress of Physics*, 46(4-5):493–505, 1998.

[76] Lov K. Grover. Fixed-point quantum search. *Phys. Rev. Lett.*, 2005.

[77] Peter Høyer, Michele Mosca, and Ronald de Wolf. Quantum search on bounded-error inputs. *Lecture Notes in Computer Science*, page 291–299, 2003.

[78] Andris Ambainis. Variable time amplitude amplification and quantum algorithms for linear algebra problems. In *STACS'12 (29th Symposium on Theoretical Aspects of Computer Science)*, volume 14, pages 636–647. LIPIcs, 2012.

[79] A. Yu. Kitaev. Quantum measurements and the Abelian stabilizer problem. *arXiv preprint quant-ph/9511026*, 1995.

[80] R. Cleve, A. Ekert, C. Macchiavello, and M. Mosca. Quantum algorithms revisited. *Proceedings of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences*, 454(1969):339—354, Jan 1998.

[81] Anupam Prakash. *Quantum algorithms for linear algebra and machine learning*. University of California, Berkeley, 2014.

[82] Yohichi Suzuki, Shumpei Uno, Rudy Raymond, Tomoki Tanaka, Tamiya Onodera, and Naoki Yamamoto. Amplitude estimation without phase estimation. *Quantum Information Processing*, 19(2), Jan 2020.

[83] Dmitry Grinko, Julien Gacon, Christa Zoufal, and Stefan Woerner. Iterative quantum amplitude estimation. *npj Quantum Information*, 7(1), Mar 2021.

[84] Tudor Giurgica-Tiron, Iordanis Kerenidis, Farrokh Labib, Anupam Prakash, and William Zeng. Low depth algorithms for quantum amplitude estimation. *arXiv preprint arXiv:2012.03348*, 2020.

[85] Kirill Plekhanov, Matthias Rosenkranz, Mattia Fiorentini, and Michael Lubasch. Variational quantum amplitude estimation. *arXiv preprint arXiv:2109.03687*, 2021.

[86] Danial Dervovic, Mark Herbster, Peter Mountney, Simone Severini, Naïri Usher, and Leonard Wossnig. Quantum linear systems algorithms: a primer. *arXiv preprint arXiv:1802.08227*, 2018.
[87] Andrew M. Childs, Robin Kothari, and Rolando D. Somma. Quantum algorithm for systems of linear equations with exponentially improved dependence on precision. *SIAM Journal on Computing*, 46(6):1920–1950, Jan 2017.
[88] Aram W. Harrow, Avinatan Hassidim, and Seth Lloyd. Quantum algorithm for linear systems of equations. *Phys. Rev. Lett.*, 103:150502, Oct 2009.
[89] Scott Aaronson. Read the Fine Print. *Nature Physics*, 11(4):291–293, 04 2015.
[90] Dominic W. Berry, Graeme Ahokas, Richard Cleve, and Barry C. Sanders. Efficient quantum algorithms for simulating sparse Hamiltonians. *Communications in Mathematical Physics*, 270(2):359—-371, Dec 2006.
[91] Andrew M. Childs, Yuan Su, Minh C. Tran, Nathan Wiebe, and Shuchen Zhu. Theory of Trotter error with commutator scaling. *Physical Review X*, 11(1), Feb 2021.
[92] Dominic W. Berry, Andrew M. Childs, and Robin Kothari. Hamiltonian simulation with nearly optimal dependence on all parameters. *2015 IEEE 56th Annual Symposium on Foundations of Computer Science*, Oct 2015.
[93] Guang Hao Low and Isaac L. Chuang. Optimal Hamiltonian simulation by quantum signal processing. *Phys. Rev. Lett.*, 118(1), Jan 2017.
[94] Pedro Costa, Dong An, Yuval R Sanders, Yuan Su, Ryan Babbush, and Dominic W Berry. Optimal scaling quantum linear systems solver via discrete adiabatic theorem. *arXiv preprint arXiv:2111.08152*, 2021.
[95] Leonard Wossnig, Zhikuan Zhao, and Anupam Prakash. Quantum linear system algorithm for dense matrices. *Physical Review Letters*, 120(5), Jan 2018.
[96] Iordanis Kerenidis and Anupam Prakash. Quantum gradient descent for linear systems and least squares. *Physical Review A*, 101(2), Feb 2020.
[97] Shantanav Chakraborty, András Gilyén, and Stacey Jeffery. The power of block-encoded matrix powers: improved regression techniques via faster Hamiltonian simulation. *arXiv preprint arXiv:1804.01973*, 2018.
[98] Sander Gribling, Iordanis Kerenidis, and Dániel Szilágyi. Improving quantum linear system solvers via a gradient descent perspective. *arXiv preprint arXiv:2109.04248*, 2021.
[99] András Gilyén, Yuan Su, Guang Hao Low, and Nathan Wiebe. Quantum singular value transformation and beyond: exponential improvements for quantum matrix arithmetics. *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing*, Junee 2019.
[100] Alexander Dranov, Johannes Kellendonk, and Rudolf Seiler. Discrete time adiabatic theorems for quantum mechanical systems. *Journal of Mathematical Physics*, 39(3):1340–1349, 1998.
[101] Iordanis Kerenidis and Anupam Prakash. A quantum interior point method for LPs and SDPs. *ACM Transactions on Quantum Computing*, 1(1), Oct 2020.
[102] Alan Frieze, Ravi Kannan, and Santosh Vempala. Fast Monte-Carlo algorithms for finding low-rank approximations. *Journal of the ACM (JACM)*, 51(6):1025–1041, 2004.
[103] Ewin Tang. A quantum-inspired classical algorithm for recommendation systems. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing*. Association for Computing Machinery, 2019.
[104] Nai-Hui Chia, András Gilyén, Tongyang Li, Han-Hsuan Lin, Ewin Tang, and Chunhao Wang. Sampling-based sublinear low-rank matrix arithmetic framework for dequantizing quantum machine learning. *Proceedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing*, June 2020.
[105] Gregory F. Lawler and Vlada Limic. *Random Walk: A Modern Introduction*. Cambridge Studies in Advanced Mathematics. Cambridge University Press, 2010.
[106] Fischer Black and Myron Scholes. The pricing of options and corporate liabilities. In *World Scientific Reference on Contingent Claims Analysis in Corporate Finance: Volume 1: Foundations of CCA and Equity Valuation*, pages 3–21. World Scientific, 2019.
[107] F. C. Klebaner. *Introduction to Stochastic Calculus with Applications*. Imperial College Press, 3rd edition, 2012.
[108] László Lovász. Random walks on graphs: A survey, combinatorics, Paul Erdos is Eighty. *Bolyai Soc. Math. Stud.*, 2, Jan 1993.
[109] J. R. Norris. *Markov Chains*. Cambridge Series in Statistical and Probabilistic Mathematics. Cambridge University Press, 1997.
[110] J. Kempe. Quantum random walks: An introductory overview. *Contemporary Physics*, 44(4):307–327, July 2003.
[111] Mario Szegedy. Quantum speed-up of Markov chain based algorithms. In *45th Annual IEEE symposium on foundations of computer science*, 2004.
[112] Frédéric Magniez, Ashwin Nayak, Jérémie Roland, and Miklos Santha. Search via quantum walk. *SIAM Journal on Computing*, 40(1):14–164, Jan 2011.
[113] Neil Shenvi, Julia Kempe, and K. Birgitta Whaley. Quantum random-walk search algorithm. *Physical Review A*, 67(5), May 2003.

[114] Andris Ambainis, Julia Kempe, and Alexander Rivosh. Coins make quantum walks faster. In *Proceedings of the Sixteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, SODA '05, page 1099–1108, USA, 2005. Society for Industrial and Applied Mathematics.

[115] Andris Ambainis, Eric Bach, Ashwin Nayak, Ashvin Vishwanath, and John Watrous. One-dimensional quantum walks. In *Proceedings of the thirty-third annual ACM symposium on Theory of Computing*, pages 37–49, 2001.

[116] D. Aharonov, A. Ambainis, J. Kempe, and U. Vazirani. Quantum walks on graphs. In *Proceedings of the thirty-third annual ACM symposium on Theory of Computing*, pages 50–59, 2001.

[117] Cristopher Moore and Alexander Russell. Quantum walks on the hypercube. In *Proceedings of the 6th International Workshop on Randomization and Approximation Techniques*, pages 164—178. Springer, 2002.

[118] Andris Ambainis. Quantum walk algorithm for element distinctness. *SIAM Journal on Computing*, 37(1):210–239, 2007.

[119] Simon Apers, András Gilyén, and Stacey Jeffery. A unified framework of quantum walk search. *arXiv preprint arXiv:1912.04233*, 2019.

[120] R. D. Somma, S. Boixo, H. Barnum, and E. Knill. Quantum simulations of classical annealing processes. *Phys. Rev. Lett.*, Sep 2008.

[121] Andrew M. Childs and Jason M. Eisenberg. Quantum algorithms for subset finding. *Quantum Info. Comput.*, 5(7):593–604, Nov 2005.

[122] Andris Ambainis, András Gilyén, Stacey Jeffery, and Martins Kokainis. Quadratic speedup for finding marked vertices by quantum walks. In *Proceedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing*, pages 412—424, New York, NY, USA, 2020. Association for Computing Machinery.

[123] Edward Farhi and Sam Gutmann. Quantum computation and decision trees. *Physical Review A*, 58(2):915–928, Aug 1998.

[124] Andrew M. Childs, Edward Farhi, and Sam Gutmann. An example of the difference between quantum and classical random walks. *Quantum Information Processing*, 2002.

[125] Andrew M. Childs, Richard Cleve, Enrico Deotto, Edward Farhi, Sam Gutmann, and Daniel A. Spielman. Exponential algorithmic speedup by a quantum walk. *Proceedings of the thirty-fifth ACM symposium on Theory of computing - STOC '03*, 2003.

[126] Julia Kempe. Quantum random walks hit exponentially faster. *arXiv preprint quant-ph/0205083*, 2002.

[127] Yue Ruan, Samuel Marsh, Xilin Xue, Xi Li, Zhihao Liu, and Jingbo Wang. Quantum approximate algorithm for NP optimization problems with constraints. *arXiv preprint arXiv:2002.00943*, 2020.

[128] Andrew M. Childs. On the relationship between continuous- and discrete-time quantum walk. *Communications in Mathematical Physics*, 294(2):581–603, Oct 2009.

[129] Salvador Elías Venegas-Andraca. Quantum walks: a comprehensive review. *Quantum Information Processing*, 11(5):101–1106, Jul 2012.

[130] M. Cerezo, Andrew Arrasmith, Ryan Babbush, Simon C. Benjamin, Suguru Endo, Keisuke Fujii, Jarrod R. McClean, Kosuke Mitarai, Xiao Yuan, Lukasz Cincio, and et al. Variational quantum algorithms. *Nature Reviews Physics*, 3(9):625—644, Aug 2021.

[131] Lennart Bittel and Martin Kliesch. Training variational quantum algorithms is NP-hard. *Phys. Rev. Lett.*, 127(12):120502, 2021.

[132] Alberto Peruzzo, Jarrod McClean, Peter Shadbolt, Man-Hong Yung, Xiao-Qi Zhou, Peter J. Love, Alán Aspuru-Guzik, and Jeremy L. O'Brien. A variational eigenvalue solver on a photonic quantum processor. *Nat. Commun.*, 5(1), July 2014.

[133] Carlos Bravo-Prieto, Ryan LaRose, Marco Cerezo, Yigit Subasi, Lukasz Cincio, and Patrick J Coles. Variational quantum linear solver. *arXiv preprint arXiv:1909.05820*, 2019.

[134] Hsin-Yuan Huang, Kishor Bharti, and Patrick Rebentrost. Near-term quantum algorithms for linear systems of equations with regression loss functions. *New Journal of Physics*, 23(11):113021, Nov 2021.

[135] Patrick Huembeli and Alexandre Dauphin. Characterizing the loss landscape of variational quantum circuits. *Quantum Science and Technology*, 6(2):025011, Feb 2021.

[136] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028*, 2014.

[137] Stuart Hadfield, Zhihui Wang, Bryan O'Gorman, Eleanor Rieffel, Davide Venturelli, and Rupak Biswas. From the quantum approximate optimization algorithm to a quantum alternating operator ansatz. *Algorithms*, 12(2):34, Feb 2019.

[138] Xiao Yuan, Suguru Endo, Qi Zhao, Ying Li, and Simon C. Benjamin. Theory of variational quantum simulation. *Quantum*, 3:191, Oct 2019.

[139] A.D. McLachlan. A variational solution of the time-dependent Schrödinger equation. *Molecular Physics*, 8(1):39–44, 1964.

[140] K. Mitarai, M. Negoro, M. Kitagawa, and K. Fujii. Quantum circuit learning. *Physical Review A*, 98(3), Sep 2018.

[141] Edward Farhi, Jeffrey Goldstone, Sam Gutmann, and Michael Sipser. Quantum computation by adiabatic evolution. *arXiv preprint quant-ph/0001106*, 2000.
[142] Peter JM Van Laarhoven and Emile HL Aarts. Simulated annealing. In *Simulated annealing: Theory and applications*, pages 7–15. Springer, 1987.
[143] Kostyantyn Kechedzhi and Vadim N. Smelyanskiy. Open-system quantum annealing in mean-field models with exponential degeneracy. *Physical Review X*, 6(2), May 2016.
[144] Vasil S. Denchev, Sergio Boixo, Sergei V. Isakov, Nan Ding, Ryan Babbush, Vadim Smelyanskiy, John Martinis, and Hartmut Neven. What is the computational value of finite-range tunneling? *Physical Review X*, 6(3), Aug 2016.
[145] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. Quantum adiabatic evolution algorithms versus simulated annealing. *arXiv preprint quant-ph/0201031*, 2002.
[146] Akshay Ajagekar, Travis Humble, and Fengqi You. Quantum computing based hybrid solution strategies for large-scale discrete-continuous optimization problems. *Computers & Chemical Engineering*, 132:106630, Jan 2020.
[147] Radford M Neal. *Probabilistic inference using Markov chain Monte Carlo methods*. Department of Computer Science, University of Toronto Toronto, ON, Canada, 1993.
[148] Christian P Robert, George Casella, and George Casella. *Monte Carlo statistical methods*, volume 2. Springer, 2004.
[149] Tito Homem-de-Mello and Güzin Bayraksan. Monte Carlo sampling-based methods for stochastic optimization. *Surveys in Operations Research and Management Science*, 19(1):56–85, 2014.
[150] Gilles Brassard, Frederic Dupuis, Sebastien Gambs, and Alain Tapp. An optimal quantum algorithm to approximate the mean and its application for approximating the median of a set of points over an arbitrary distance. *arXiv preprint arXiv:1106.4267*, 2011.
[151] S. Heinrich. Quantum summation with an application to integration. *Journal of Complexity*, 18(1):1–50, 2002.
[152] Ashley Montanaro. Quantum speedup of Monte Carlo methods. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 471(2181):20150301, Sep 2015.
[153] D. Ceperley and B. Alder. Quantum Monte Carlo. *Science*, 231(4738):555–560, 1986.
[154] Shouvanik Chakrabarti, Rajiv Krishnakumar, Guglielmo Mazzola, Nikitas Stamatopoulos, Stefan Woerner, and William J. Zeng. A threshold for quantum advantage in derivative pricing. *Quantum*, 5:463, June 2021.
[155] Thomas Häner, Martin Roetteler, and Krysta M Svore. Optimizing quantum circuits for arithmetic. *arXiv preprint arXiv:1805.12445*, 2018.
[156] Arjan Cornelissen and Sofiene Jerbi. Quantum algorithms for multivariate Monte Carlo estimation. 2021.
[157] Lov Grover and Terry Rudolph. Creating superpositions that correspond to efficiently integrable probability distributions. *arXiv preprint quant-ph/0208112*, 2002.
[158] Steven Herbert. No quantum speedup with grover-rudolph state preparation for quantum Monte Carlo integration. *Physical Review E*, 103(6), Jun 2021.
[159] Christa Zoufal, Aurélien Lucchi, and Stefan Woerner. Quantum generative adversarial networks for learning and loading random distributions. *npj Quantum Information*, 5(1), 2019.
[160] Bruno Dupire, The Black–scholes Model (see Black, and Gives Options. Pricing with a smile. *Risk Magazine*, pages 18–20, 1994.
[161] Gisiro Maruyama. On the transition probability functions of the Markov process. *Nat. Sci. Rep. Ochanomizu Univ*, 5:10–20, 1954.
[162] Kazuya Kaneko, Koichi Miyamoto, Naoyuki Takeda, and Kazuyoshi Yoshino. Quantum pricing with a smile: Implementation of local volatility model on quantum computer. 2020.
[163] Koichi Miyamoto and Kenji Shiohara. Reduction of qubits in a quantum algorithm for Monte Carlo simulation by a pseudo-random-number generator. *Physical Review A*, 102(2), aug 2020.
[164] Michael B Giles. Multilevel Monte Carlo path simulation. *Operations research*, 56(3):607–617, 2008.
[165] Dong An, Noah Linden, Jin-Peng Liu, Ashley Montanaro, Changpeng Shao, and Jiasu Wang. Quantum-accelerated multilevel monte carlo methods for stochastic differential equations in mathematical finance. *Quantum*, 5:481, June 2021.
[166] Steven Herbert. Quantum Monte-Carlo integration: The full advantage in minimal circuit depth. *arXiv preprint arXiv:2105.09100*, 2021.
[167] Ryan Babbush, Jarrod R. McClean, Michael Newman, Craig Gidney, Sergio Boixo, and Hartmut Neven. Focus beyond quadratic speedups for error-corrected quantum advantage. *PRX Quantum*, 2(1), March 2021.
[168] Niladri Gomes, Anirban Mukherjee, Feng Zhang, Thomas Iadecola, Cai-Zhuang Wang, Kai-Ming Ho, Peter P Orth, and Yong-Xin Yao. Adaptive variational quantum imaginary time evolution approach for ground state preparation. *Advanced Quantum Technologies*, page 2100114, 2021.
[169] Arthur G. Rattew and Bálint Koczor. Preparing arbitrary continuous functions in quantum registers with logarithmic complexity. 2022.

[170] Mark Kac. On distributions of certain Wiener functionals. *Transactions of the American Mathematical Society*, 65(1):1–13, 1949.
[171] Richard P Feynman. The principle of least action in quantum mechanics. In *Feynman's Thesis—A New Approach to Quantum Theory*, pages 1–69. World Scientific, 2005.
[172] Christian Grossmann, Hans-Görg Roos, and Martin Stynes. *Numerical treatment of partial differential equations*, volume 154. Springer, 2007.
[173] Jie Shen, Tao Tang, and Li-Lian Wang. *Spectral methods: algorithms, analysis and applications*, volume 41. Springer Science & Business Media, 2011.
[174] Yudong Cao, Anargyros Papageorgiou, Iasonas Petras, Joseph Traub, and Sabre Kais. Quantum algorithm and circuit design solving the Poisson equation. *New Journal of Physics*, 15(1):013021, 2013.
[175] Dominic W Berry. High-order quantum algorithm for solving linear differential equations. *Journal of Physics A: Mathematical and Theoretical*, 47(10):105301, 2014.
[176] Dominic W Berry, Andrew M Childs, Aaron Ostrander, and Guoming Wang. Quantum algorithm for linear differential equations with exponentially improved dependence on precision. *Communications in Mathematical Physics*, 356(3):1057–1081, 2017.
[177] Andrew M Childs, Jin-Peng Liu, and Aaron Ostrander. High-precision quantum algorithms for partial differential equations. *Quantum*, 5:574, 2021.
[178] Ashley Montanaro and Sam Pallister. Quantum algorithms and the finite element method. *Physical Review A*, 93(3):032324, 2016.
[179] Pedro CS Costa, Stephen Jordan, and Aaron Ostrander. Quantum algorithm for simulating the wave equation. *Physical Review A*, 99(1):012323, 2019.
[180] François Fillion-Gourdeau and Emmanuel Lorin. Simple digital quantum algorithm for symmetric first-order linear hyperbolic systems. *Numerical Algorithms*, 82(3):1009–1045, 2019.
[181] Torsten Carleman. Application de la théorie des équations intégrales linéaires aux systèmes d'équations différentielles non linéaires. *Acta Mathematica*, 59:63–87, 1932.
[182] Krzysztof Kowalski and W-H Steeb. *Nonlinear dynamical systems and Carleman linearization*. World Scientific, 1991.
[183] Jin-Peng Liu, Herman Øie Kolden, Hari K Krovi, Nuno F Loureiro, Konstantina Trivisa, and Andrew M Childs. Efficient quantum algorithm for dissipative nonlinear differential equations. *Proceedings of the National Academy of Sciences*, 118(35), 2021.
[184] Sarah K Leyton and Tobias J Osborne. A quantum algorithm to solve nonlinear differential equations. *arXiv preprint arXiv:0812.4423*, 2008.
[185] Seth Lloyd, Giacomo De Palma, Can Gokler, Bobak Kiani, Zi-Wen Liu, Milad Marvian, Felix Tennie, and Tim Palmer. Quantum algorithm for nonlinear differential equations. *arXiv preprint arXiv:2011.06571*, 2020.
[186] Ilon Joseph. Koopman–von Neumann approach to quantum simulation of nonlinear classical dynamics. *Physical Review Research*, 2(4):043102, 2020.
[187] Shi Jin and Nana Liu. Quantum algorithms for computing observables of nonlinear partial differential equations. *arXiv preprint arXiv:2202.07834*, 2022.
[188] Shi Jin and Xiantao Li. Multi-phase computations of the semiclassical limit of the Schrödinger equation and related problems: Whitham vs Wigner. *Physica D: Nonlinear Phenomena*, 182(1-2):46–85, 2003.
[189] Filipe Fontanela, Antoine Jacquier, and Mugad Oumgari. A quantum algorithm for linear PDEs arising in finance. *SIAM Journal on Financial Mathematics*, 12(4):SC98–SC114, 2021.
[190] Hedayat Alghassi, Amol Deshmukh, Noelle Ibrahim, Nicolas Robles, Stefan Woerner, and Christa Zoufal. A variational quantum algorithm for the Feynman–Kac formula. *arXiv preprint arXiv:2108.10846*, 2021.
[191] Michael Lubasch, Jaewoo Joo, Pierre Moinier, Martin Kiffner, and Dieter Jaksch. Variational quantum algorithms for nonlinear problems. *Physical Review A*, 101(1):010301, 2020.
[192] Oleksandr Kyriienko, Annie E Paine, and Vincent E Elfving. Solving nonlinear differential equations with differentiable quantum circuits. *Physical Review A*, 103(5):052416, 2021.
[193] Oleksandr Kyriienko, Annie E Paine, and Vincent E Elfving. Protocols for trainable and differentiable quantum generative modelling. *arXiv preprint arXiv:2202.08253*, 2022.
[194] Annie E. Paine, Vincent E. Elfving, and Oleksandr Kyriienko. Quantum quantile mechanics: Solving stochastic differential equations for generating time-series. 2021.
[195] Gyorgy Steinbrecher and William Shaw. Quantile mechanics. *European Journal of Applied Mathematics - EUR J APPL MATH*, 19, 08 2007.
[196] James E Gentle. *Random number generation and Monte Carlo methods*, volume 381. Springer, 2003.
[197] Kenji Kubo, Yuya O. Nakagawa, Suguru Endo, and Shota Nagayama. Variational quantum simulations of stochastic differential equations. *Physical Review A*, 103(5), May 2021.
[198] Phelim P. Boyle. Option valuation using a three jump process. 1986.
[199] Hans Föllmer and Alexander Schied. Stochastic finance. In *Stochastic Finance*. de Gruyter, 2016.
[200] Stefan Weinzierl. Introduction to Monte Carlo methods. *arXiv preprint hep-ph/0006269*, 2000.

[201] Phelim P Boyle. Options: A Monte Carlo approach. *Journal of Financial Economics*, 4(3):323–338, 1977.
[202] Nikitas Stamatopoulos, Daniel J. Egger, Yue Sun, Christa Zoufal, Raban Iten, Ning Shen, and Stefan Woerner. Option pricing using quantum computers. *Quantum*, 4:291, Jul 2020.
[203] Patrick Rebentrost, Brajesh Gupt, and Thomas R. Bromley. Quantum computational finance: Monte Carlo pricing of financial derivatives. *Physical Review A*, 98(2), Aug 2018.
[204] Koichi Miyamoto and Kenji Kubo. Pricing multi-asset derivatives by finite difference method on a quantum computer, 2021.
[205] Noah Linden, Ashley Montanaro, and Changpeng Shao. Quantum vs. classical algorithms for solving the heat equation. 2020.
[206] Javier Gonzalez-Conde, Ángel Rodríguez-Rozas, Enrique Solano, and Mikel Sanz. Simulating option price dynamics with exponential quantum speedup. 2021.
[207] Albert N Shiryaev. *Optimal stopping rules*, volume 8. Springer Science & Business Media, 2007.
[208] Francis A Longstaff and Eduardo S Schwartz. Valuing American options by simulation: a simple least-squares approach. *The Review of Financial Studies*, 14(1):113–147, 2001.
[209] João F. Doriguello, Alessandro Luongo, Jinge Bao, Patrick Rebentrost, and Miklos Santha. Quantum algorithm for stochastic optimal stopping problems with applications in finance, 2021.
[210] Koichi Miyamoto. Bermudan option pricing by quantum amplitude estimation and chebyshev interpolation, 2021.
[211] Hao Tang, Anurag Pal, Tian-Yu Wang, Lu-Feng Qiao, Jun Gao, and Xian-Min Jin. Quantum computation for pricing the collateralized debt obligations. *Quantum Engineering*, 3(4):e84, 2021.
[212] David X. Li. On default correlation: A copula function approach. *The Journal of Fixed Income*, 9(4):43–54, 2000.
[213] Ole E. Barndorff-Nielsen. Normal inverse Gaussian distributions and stochastic volatility modelling. *Scandinavian Journal of Statistics*, 24(1):1–13, 1997.
[214] L. Jeff Hong, Zhaolin Hu, and Guangwu Liu. Monte Carlo methods for value-at-risk and conditional value-at-risk: A review. *ACM Trans. Model. Comput. Simul.*, 24(4), Nov 2014.
[215] Stefan Woerner and Daniel J. Egger. Quantum risk analysis. *npj Quantum Information*, 5(1), Feb 2019.
[216] D. J. Egger, R. Garcia Gutierrez, J. Mestre, and S. Woerner. Credit risk analysis using quantum computers. *IEEE Transactions on Computers*, 70(12):2136–2145, Dec 2021.
[217] Nikitas Stamatopoulos, Guglielmo Mazzola, Stefan Woerner, and William J. Zeng. Towards quantum advantage in financial market risk using quantum gradient algorithms. 2021.
[218] Stephen P. Jordan. Fast quantum algorithm for numerical gradient estimation. *Phys. Rev. Lett.*, 95:050501, July 2005.
[219] András Gilyén, Srinivasan Arunachalam, and Nathan Wiebe. Optimizing quantum optimization algorithms via faster quantum gradient computation. In *Proceedings of the Thirtieth Annual ACM-SIAM Symposium on Discrete Algorithms*, SODA '19, page 1425–1444, USA, 2019. Society for Industrial and Applied Mathematics.
[220] Guang Hao Low and Isaac L Chuang. Hamiltonian simulation by qubitization. *Quantum*, 3:163, 2019.
[221] Jianping Li. General explicit difference formulas for numerical differentiation. *Journal of Computational and Applied Mathematics*, 183(1):29–52, 2005.
[222] Arjan Cornelissen. Quantum gradient estimation of gevrey functions. *arXiv preprint arXiv:1909.13528*, 2019.
[223] Andrew Green. *XVA: credit, funding and capital valuation adjustments*. John Wiley & Sons, 2015.
[224] Jeong Yu Han and Patrick Rebentrost. Quantum advantage for multi-option portfolio pricing and valuation adjustments. 2022.
[225] Javier Alcazar, Andrea Cadarso, Amara Katabarwa, Marta Mauri, Borja Peropadre, Guoming Wang, and Yudong Cao. Quantum algorithm for credit valuation adjustments. *New Journal of Physics*, 24(2):023036, feb 2022.
[226] Guoming Wang, Dax Enshan Koh, Peter D. Johnson, and Yudong Cao. Minimizing estimation runtime on noisy quantum computers. *PRX Quantum*, 2(1), March 2021.
[227] Dax Enshan Koh, Guoming Wang, Peter D. Johnson, and Yudong Cao. Foundations for Bayesian inference with engineered likelihood functions for robust amplitude estimation. *Journal of Mathematical Physics*, 63(5):052202, may 2022.
[228] J. Hromkovič. *Algorithmics for hard problems: introduction to combinatorial optimization, randomization, approximation, and heuristics*. Springer Science & Business Media, 2013.
[229] Yurii Nesterov and Arkadii Nemirovskii. *Interior-point polynomial algorithms in convex programming*. SIAM, 1994.
[230] Lee Braine, Daniel J. Egger, Jennifer Glick, and Stefan Woerner. Quantum algorithms for mixed binary optimization applied to transaction settlement. *IEEE Transactions on Quantum Engineering*, 2:1–8, 2021.

[231] Gary Kochenberger, Jin-Kao Hao, Fred Glover, Mark Lewis, Zhipeng Lü, Haibo Wang, and Yang Wang. The unconstrained binary quadratic programming problem: a survey. *Journal of combinatorial optimization*, 28(1):58–81, 2014.

[232] S. Okada, M. Ohzeki, and S. Taguchi. Efficient partition of integer optimization problems with one-hot encoding. *Scientific Reports*, 2019.

[233] Andrew Lucas. Ising formulations of many NP problems. *Frontiers in Physics*, 2:5, 2014.

[234] Tadashi Kadowaki and Hidetoshi Nishimori. Quantum annealing in the transverse Ising model. *Physical Review E*, 58(5):5355—5363, Nov 1998.

[235] Nicholas Chancellor, Stefan Zohren, and Paul A Warburton. Circuit design for multi-body interactions in superconducting quantum annealing systems with applications to a scalable architecture. *npj Quantum Information*, 3(1):1–7, 2017.

[236] Krzysztof Domino, Akash Kundu, Özlem Salehi, and Krzysztof Krawiec. Quadratic and higher-order unconstrained binary optimization of railway dispatching problem for quantum computing. *arXiv preprint arXiv:2107.03234*, 2021.

[237] James King, Masoud Mohseni, William Bernoudy, Alexandre Fréchette, Hossein Sadeghi, Sergei V Isakov, Hartmut Neven, and Mohammad H Amin. Quantum-assisted genetic algorithm. *arXiv preprint arXiv:1907.00707*, 2019.

[238] Masayuki Ohzeki. Breaking limitation of quantum annealer in solving optimization problems under constraints. *Scientific Reports*, 10, 02 2020.

[239] Elisabeth Lobe and Annette Lutz. Minor embedding in broken chimera and pegasus graphs is NP-complete. *arXiv preprint arXiv:2110.08325*, 2021.

[240] Jun Cai, William G Macready, and Aidan Roy. A practical heuristic for finding graph minors. *arXiv preprint arXiv:1406.2741*, 2014.

[241] Timothy Goodrich, Blair D. Sullivan, and T. Humble. Optimizing adiabatic quantum program compilation using a graph-theoretic framework. *Quantum Information Processing*, 17:1–26, 2018.

[242] Davide Venturelli, Salvatore Mandrà, Sergey Knysh, Bryan O'Gorman, Rupak Biswas, and Vadim Smelyanskiy. Quantum optimization of fully connected spin glasses. *Phys. Rev. X*, 5:031040, Sep 2015.

[243] Wolfgang Lechner, Philipp Hauke, and Peter Zoller. A quantum annealing architecture with all-to-all connectivity from local interactions. *Science Advances*, 1(9):e1500838, 2015.

[244] Kilian Ender, Roeland ter Hoeven, Benjamin E Niehoff, Maike Drieb-Schön, and Wolfgang Lechner. Parity quantum optimization: Compiler. *arXiv preprint arXiv:2105.06233*, 2021.

[245] Maike Drieb-Schön, Younes Javanmard, Kilian Ender, and Wolfgang Lechner. Parity quantum optimization: Encoding constraints. *arXiv preprint arXiv:2105.06235*, 2021.

[246] Michael Fellner, Kilian Ender, Roeland ter Hoeven, and Wolfgang Lechner. Parity quantum optimization: Benchmarks. *arXiv preprint arXiv:2105.06240*, 2021.

[247] Henrique Silvério, Sebastián Grijalva, Constantin Dalyac, Lucas Leclerc, Peter J Karalekas, Nathan Shammah, Mourad Beji, Louis-Paul Henry, and Loïc Henriet. Pulser: An open-source package for the design of pulse sequences in programmable neutral-atom arrays. *arXiv preprint arXiv:2104.15044*, 2021.

[248] Loïc Henriet, Lucas Beguin, Adrien Signoles, Thierry Lahaye, Antoine Browaeys, Georges-Olivier Reymond, and Christophe Jurczak. Quantum computing with neutral atoms. *Quantum*, 4:327, September 2020.

[249] S. Ebadi, A. Keesling, M. Cain, T. T. Wang, H. Levine, D. Bluvstein, G. Semeghini, A. Omran, J.-G. Liu, R. Samajdar, X.-Z. Luo, B. Nash, X. Gao, B. Barak, E. Farhi, S. Sachdev, N. Gemelke, L. Zhou, S. Choi, H. Pichler, S.-T. Wang, M. Greiner, V. Vuletić, and M. D. Lukin. Quantum optimization of maximum independent set using rydberg atom arrays. *Science*, 376(6598):1209–1215, 2022.

[250] Ruslan Shaydulin, Stuart Hadfield, Tad Hogg, and Ilya Safro. Classical symmetries and the quantum approximate optimization algorithm. *Quantum Information Processing*, 20(11):359, Oct 2021.

[251] Ruslan Shaydulin, Ilya Safro, and Jeffrey Larson. Multistart methods for quantum approximate optimization. In *2019 IEEE High Performance Extreme Computing Conference (HPEC)*, pages 1–8, 2019.

[252] Alexey Galda, Xiaoyuan Liu, Danylo Lykov, Yuri Alexeev, and Ilya Safro. Transferability of optimal QAOA parameters between random graphs. In *2021 IEEE International Conference on Quantum Computing and Engineering (QCE)*, pages 171–180. IEEE, 2021.

[253] Michael Streif and Martin Leib. Training the quantum approximate optimization algorithm without access to a quantum processing unit. *Quantum Science and Technology*, 5(3):034008, 2020.

[254] Sami Khairy, Ruslan Shaydulin, Lukasz Cincio, Yuri Alexeev, and Prasanna Balaprakash. Learning to optimize variational quantum circuits to solve combinatorial problems. In *Proceedings of the Thirty-Fourth AAAI Conference on Artificial Intelligence (AAAI)*, 2020.

[255] Ruslan Shaydulin and Yuri Alexeev. Evaluating quantum approximate optimization algorithm: A case study. In *Proceedings of the 2nd International Workshop on Quantum Computing for Sustainable Computing*, 2019.

[256] Zhen-Duo Wang, Pei-Lin Zheng, Biao Wu, and Yi Zhang. Quantum dropout for efficient quantum approximate optimization algorithm on combinatorial optimization problems. *arXiv preprint arXiv:2203.10101*, 2022.

[257] Xiaoyuan Liu, Ruslan Shaydulin, and Ilya Safro. Quantum approximate optimization algorithm with sparsified phase operator, 2022.

[258] Sahil Gulania, Bo Peng, Yuri Alexeev, and Niranjan Govind. Quantum time dynamics of 1D-Heisenberg models employing the Yang–Baxter equation for circuit compression. 2021.

[259] Matthew P Harrigan, Kevin J Sung, Matthew Neeley, Kevin J Satzinger, Frank Arute, Kunal Arya, Juan Atalaya, Joseph C Bardin, Rami Barends, Sergio Boixo, et al. Quantum approximate optimization of non-planar graph problems on a planar superconducting processor. *Nature Physics*, 17(3):332–336, 2021.

[260] Johannes S Otterbach, Riccardo Manenti, Nasser Alidoust, A Bestwick, M Block, B Bloom, S Caldwell, N Didier, E Schuyler Fried, S Hong, et al. Unsupervised machine learning on a hybrid quantum computer. *arXiv preprint arXiv:1712.05771*, 2017.

[261] Pradeep Niroula, Ruslan Shaydulin, Romina Yalovetzky, Pierre Minssen, Dylan Herman, Shaohan Hu, and Marco Pistoia. Constrained quantum optimization for extractive summarization on a trapped-ion quantum computer. 2022.

[262] Dmitry A Fedorov, Bo Peng, Niranjan Govind, and Yuri Alexeev. VQE method: A short survey and recent developments. *Materials Theory*, 6(1):1–21, 2022.

[263] Arthur G. Rattew, Shaohan Hu, Marco Pistoia, Richard Chen, and Steve Wood. A Domain-agnostic, Noise-resistant, Hardware-efficient Evolutionary Variational Quantum Eigensolver. *arXiv preprint arXiv:1910.09694*, 2020.

[264] David Amaro, Carlo Modica, Matthias Rosenkranz, Mattia Fiorentini, Marcello Benedetti, and Michael Lubasch. Filtering variational quantum algorithms for combinatorial optimization. *arXiv preprint arXiv:2106.10055*, 2021.

[265] Xuchen You, Shouvanik Chakrabarti, and Xiaodi Wu. A convergence theory for over-parameterized variational quantum eigensolvers. 2022.

[266] Gregory L Naber. *The Geometry of Minkowski Spacetime: An Introduction to the Mathematics of the Special Theory of Relativity*, volume 92. Springer Science & Business Media, 01 2012.

[267] Konrad Osterwalder and Robert Schrader. Axioms for Euclidean Green's functions. *Communications in mathematical physics*, 31(2):83–112, 1973.

[268] G. C. Wick. Properties of Bethe-Salpeter wave functions. *Phys. Rev.*, 96:1124–1134, Nov 1954.

[269] Sam McArdle, Tyson Jones, Suguru Endo, Ying Li, Simon C. Benjamin, and Xiao Yuan. Variational ansatz-based quantum simulation of imaginary time evolution. *npj Quantum Information*, 5(1), Sep 2019.

[270] Mario Motta, Chong Sun, Adrian T. K. Tan, Matthew J. O'Rourke, Erika Ye, Austin J. Minnich, Fernando G. S. L. Brandão, and Garnet Kin-Lic Chan. Determining eigenstates and thermal states on a quantum computer using quantum imaginary time evolution. *Nature Physics*, 16(2):205–210, Nov 2019.

[271] Kosuke Mitarai and Keisuke Fujii. Methodology for replacing indirect measurements with direct measurements. *Physical Review Research*, 1(1), Aug 2019.

[272] Marcello Benedetti, Mattia Fiorentini, and Michael Lubasch. Hardware-efficient variational quantum algorithms for time evolution. *Phys. Rev. Research*, 3:033083, Jul 2021.

[273] Christa Zoufal, Aurélien Lucchi, and Stefan Woerner. Variational quantum Boltzmann machines. *Quantum Machine Intelligence*, 3(1), Feb 2021.

[274] Ming-Cheng Chen, Ming Gong, Xiaosi Xu, Xiao Yuan, Jian-Wen Wang, Can Wang, Chong Ying, Jin Lin, Yu Xu, Yulin Wu, and et al. Demonstration of adiabatic variational quantum computing with a superconducting quantum coprocessor. *Physical Review Letters*, 125(18), Oct 2020.

[275] Christoph Dürr and Peter Høyer. A quantum algorithm for finding the minimum. *arXiv preprint quant-ph/9607014*, 1996.

[276] William P Baritompa, David W Bulger, and Graham R Wood. Grover's quantum algorithm applied to global optimization. *SIAM Journal on Optimization*, 15(4):1170–1184, 2005.

[277] David Bulger, William P. Baritompa, and Graham R. Wood. Implementing pure adaptive search with Grover's quantum algorithm. *Journal of optimization theory and applications*, 116(3):517–529, 2003.

[278] Austin Gilliam, Stefan Woerner, and Constantin Gonciulea. Grover adaptive search for constrained polynomial binary optimization. *Quantum*, 5:428, 2021.

[279] Dimitris Bertsimas and John N Tsitsiklis. *Introduction to linear optimization*, volume 6. Athena Scientific Belmont, MA, 1997.

[280] Giacomo Nannicini. Fast quantum subroutines for the simplex method. In *Integer Programming and Combinatorial Optimization - 22nd International Conference, IPCO 2021, Atlanta, GA, USA, May 19-21, 2021, Proceedings*, volume 12707 of *Lecture Notes in Computer Science*, pages 311–325. Springer, 2021.

[281] Iordanis Kerenidis, Anupam Prakash, and Dániel Szilágyi. Quantum algorithms for second-order cone programming and support vector machines. *Quantum*, 5:427, April 2021.

[282] Renato Monteiro and Takashi Tsuchiya. Polynomial convergence of primal-dual algorithms for the second-order cone program based on the MZ-family of directions. *Mathematical programming*, 88(1):61–83, 2000.
[283] Fernando G.S.L. Brandao and Krysta M. Svore. Quantum speed-ups for solving semidefinite programs. In *2017 IEEE 58th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 415–426, 2017.
[284] Fernando G. S. L. Brandão, Amir Kalev, Tongyang Li, Cedric Yen-Yu Lin, Krysta M. Svore, and Xiaodi Wu. Quantum SDP solvers: Large speed-ups, optimality, and applications to quantum learning. In *46th International Colloquium on Automata, Languages, and Programming (ICALP 2019)*, volume 132 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 27:1–27:14. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2019.
[285] Joran van Apeldoorn, András Gilyén, Sander Gribling, and Ronald de Wolf. Quantum SDP-solvers: Better upper and lower bounds. *Quantum*, 4:230, Feb 2020.
[286] Sanjeev Arora and Satyen Kale. A combinatorial, primal-dual approach to semidefinite programs. *J. ACM*, 63(2), may 2016.
[287] Ruslan Shaydulin, Hayato Ushijima-Mwesigwa, Christian FA Negre, Ilya Safro, Susan M Mniszewski, and Yuri Alexeev. A hybrid approach for solving optimization problems on small quantum computers. *Computer*, 52(6):18–26, 2019.
[288] Ruslan Shaydulin, Hayato Ushijima-Mwesigwa, Ilya Safro, Susan Mniszewski, and Yuri Alexeev. Network community detection on small quantum computers. *Advanced Quantum Technologies*, 2(9):1900029, 2019.
[289] Hayato Ushijima-Mwesigwa, Ruslan Shaydulin, Christian F. A. Negre, Susan M. Mniszewski, Yuri Alexeev, and Ilya Safro. Multilevel combinatorial optimization across quantum architectures. *ACM Transactions on Quantum Computing*, 2(1), feb 2021.
[290] Guillaume Chapuis, Hristo Djidjev, Georg Hahn, and Guillaume Rizk. Finding maximum cliques on the D-Wave quantum annealer. *Journal of Signal Processing Systems*, 91(3):363–377, 2019.
[291] Harry Markowitz. Portfolio selection. *The Journal of Finance*, 7(1):77–91, 1952.
[292] S. Benati and R. Rizzi. A mixed integer linear programming formulation of the optimal mean/value-at-risk portfolio problem. *European Journal of Operational Research*, 2007.
[293] Renata Mansini, Włodzimierz Ogryczak, and M. Grazia Speranza. *Linear and mixed integer programming for portfolio optimization*. Springer, 2015.
[294] Gerard Cornuejols, Marshall L Fisher, and George L Nemhauser. Exceptional paper—location of bank accounts to optimize float: An analytic study of exact and approximate algorithms. *Management Science*, 23(8):789–810, 1977.
[295] Angad Kalra, Faisal Qureshi, and Michael Tisi. Portfolio asset identification using graph algorithms on a quantum annealer. *Available at SSRN 3333537*, 2018.
[296] Gili Rosenberg and Maxwell Rounds. Long-short minimum risk parity optimization using a quantum or digital annealer. *White Paper 1Qbit*, 2018.
[297] Mark Hodson, Brendan Ruck, Hugh Ong, David Garvin, and Stefan Dulman. Portfolio rebalancing experiments using the quantum alternating operator ansatz. *arXiv preprint arXiv:1911.05296*, 2019.
[298] N. Slate, E. Matwiejew, S. Marsh, and J. B. Wang. Quantum walk-based portfolio optimisation. *Quantum*, 5:513, July 2021.
[299] Gili Rosenberg, Poya Haghnegahdar, Phil Goddard, Peter Carr, Kesheng Wu, and Marcos Lopez de Prado. Solving the optimal trading trajectory problem using a quantum annealer. *IEEE Journal of Selected Topics in Signal Processing*, 10(6):1053–1060, Sep 2016.
[300] Samuel Mugel, Carlos Kuchkovsky, Escolástico Sánchez, Samuel Fernández-Lorenzo, Jorge Luis-Hita, Enrique Lizaso, and Román Orús. Dynamic portfolio optimization with real datasets using quantum processors and quantum-inspired tensor networks. *Phys. Rev. Research*, 4:013006, Jan 2022.
[301] Erica Grant, Travis S. Humble, and Benjamin Stump. Benchmarking quantum annealing controls with portfolio optimization. *Physical Review Applied*, 15(1), Jan 2021.
[302] Iordanis Kerenidis, Anupam Prakash, and Dániel Szilágyi. Quantum algorithms for portfolio optimization. In *Proceedings of the 1st ACM Conference on Advances in Financial Technologies*, AFT '19, pages 147—-155, New York, NY, USA, 2019. Association for Computing Machinery.
[303] Patrick Rebentrost and Seth Lloyd. Quantum computational finance: quantum algorithm for portfolio optimization. *arXiv preprint arXiv:1811.03975*, 2018.
[304] András Gilyén, Seth Lloyd, and Ewin Tang. Quantum-inspired low-rank stochastic regression with logarithmic dependence on the dimension. 2018.
[305] Juan Miguel Arrazola, Alain Delgado, Bhaskar Roy Bardhan, and Seth Lloyd. Quantum-inspired algorithms in practice. *Quantum*, 4:307, aug 2020.
[306] G. Rosenberg, C. Adolphs, A. Milne, and A. Lee. Swap netting using a quantum annealer. *White Paper 1Qbit*, 2016.
[307] René M Stulz. Credit default swaps and the credit crisis. *Journal of Economic Perspectives*, 24(1):73–92, 2010.

[308] Henry TC Hu. Swaps, the modern process of financial innovation and the vulnerability of a regulatory paradigm. *U. Pa. L. Rev.*, 138:333, 1989.
[309] James Bicksler and Andrew H. Chen. An economic analysis of interest rate swaps. *The Journal of Finance*, 41(3):645–655, 1986.
[310] Darrell Duffie and Ming Huang. Swap rates and credit quality. *The Journal of Finance*, 51(3):921–949, 1996.
[311] Andrei Shleifer and Robert W. Vishny. The limits of arbitrage. *The Journal of Finance*, 52(1):35–55, 1997.
[312] Freddy Delbaen and Walter Schachermayer. *The mathematics of arbitrage*. Springer Science & Business Media, 2006.
[313] Gilli Rosenberg. Finding optimal arbitrage opportunities using a quantum annealer. *White Paper 1Qbit*, 2016.
[314] Boris V Cherkassky and Andrew V Goldberg. Negative-cycle detection algorithms. *Mathematical Programming*, 85(2), 1999.
[315] Wanmei Soon and Heng-Qing Ye. Currency arbitrage detection using a binary integer programming model. *International Journal of Mathematical Education in Science and Technology*, 42(3):369–376, 2011.
[316] Andrew Milne, Maxwell Rounds, and Phil Goddard. Optimal feature selection in credit scoring and classification using a quantum annealer. *White Paper 1Qbit*, 2017.
[317] Matthew Elliott, Benjamin Golub, and Matthew O. Jackson. Financial networks and contagion. *American Economic Review*, 104(10):3115–53, October 2014.
[318] Brett Hemenway and Sanjeev Khanna. Sensitivity and computational complexity in financial networks. *Algorithmic Finance*, 5(3-4):95–110, 2016.
[319] Ilene Grabel. Predicting financial crisis in developing economies: astronomy or astrology? *Eastern Economic Journal*, 29(2):243–258, 2003.
[320] Arturo Estrella and Frederic S. Mishkin. Predicting U.S. recessions: Financial variables as leading indicators. *The Review of Economics and Statistics*, 1998.
[321] Román Orús, Samuel Mugel, and Enrique Lizaso. Forecasting financial crashes with quantum computing. *Phys. Rev. A*, 99(6), Jun 2019.
[322] Martin Plesch and Časlav Brukner. Quantum-state preparation with universal gate decompositions. *Phys. Rev. A*, 83:032302, Mar 2011.
[323] Jarrod R McClean, Jonathan Romero, Ryan Babbush, and Alán Aspuru-Guzik. The theory of variational hybrid quantum-classical algorithms. *New Journal of Physics*, 18(2):023023, Feb 2016.
[324] Hsin-Yuan Huang, Michael Broughton, Masoud Mohseni, Ryan Babbush, Sergio Boixo, Hartmut Neven, and Jarrod R McClean. Power of data in quantum machine learning. *Nature Communications*, 12(1):1–9, 2021.
[325] Nathan Wiebe, Daniel Braun, and Seth Lloyd. Quantum algorithm for data fitting. *Phys. Rev. Lett.*, 109:050505, Aug 2012.
[326] Guoming Wang. Quantum algorithm for linear regression. *Phys. Rev. A*, 96:012335, Jul 2017.
[327] Prasanna Date and Thomas Potok. Adiabatic quantum linear regression. *Scientific reports*, 11(1):21905, 2021.
[328] Zhikuan Zhao, Jack K. Fitzsimons, and Joseph F. Fitzsimons. Quantum-assisted Gaussian process regression. *Physical Review A*, 99(5):052331, 2019.
[329] K. Mitarai, M. Negoro, M. Kitagawa, and K. Fujii. Quantum circuit learning. *Phys. Rev. A*, 98:032309, Sep 2018.
[330] K.-R. Muller, S. Mika, G. Ratsch, K. Tsuda, and B. Scholkopf. An introduction to kernel-based learning algorithms. *IEEE Transactions on Neural Networks*, 12(2):181–201, 2001.
[331] Patrick Rebentrost, Masoud Mohseni, and Seth Lloyd. Quantum support vector machine for big data classification. *Phys. Rev. Lett.*, 2014.
[332] Seth Lloyd, Masoud Mohseni, and Patrick Rebentrost. Quantum algorithms for supervised and unsupervised machine learning. *arXiv preprint arXiv:1307.0411*, 2013.
[333] Vojtěch Havlíček, Antonio D. Córcoles, Kristan Temme, Aram W. Harrow, Abhinav Kandala, Jerry M. Chow, and Jay M. Gambetta. Supervised learning with quantum-enhanced feature spaces. *Nature*, 567(7747):209–212, March 2019.
[334] Ruslan Shaydulin and Stefan M. Wild. Importance of kernel bandwidth in quantum machine learning. 2021.
[335] Abdulkadir Canatar, Evan Peters, Cengiz Pehlevan, Stefan M. Wild, and Ruslan Shaydulin. Bandwidth enables generalization in quantum kernel models. 2022.
[336] Jonas M. Kübler, Simon Buchholz, and Bernhard Schölkopf. The inductive bias of quantum kernels. 2021.
[337] N. Wiebe, A. Kapoor, and K. Svore. Quantum algorithms for nearest-neighbor methods for supervised and unsupervised learning. *Quantum Information & Computation*, 15, 2015.

[338] Yue Ruan, Xiling Xue, Heng Liu, Jianing Tan, and Xi Li. Quantum algorithm for k-nearest neighbors classification based on the metric of Hamming distance. *International Journal of Theoretical Physics*, 56(11):3496–3507, 2017.

[339] Afrad Basheer, A Afham, and Sandeep K Goyal. Quantum k-nearest neighbors algorithm. *arXiv preprint arXiv:2003.09187*, 2020.

[340] James MacQueen. Some methods for classification and analysis of multivariate observations. In *Proceedings of the fifth Berkeley symposium on mathematical statistics and probability*, volume 1, pages 281–297. Oakland, CA, USA, 1967.

[341] Iordanis Kerenidis, Jonas Landman, Alessandro Luongo, and Anupam Prakash. q-means: A quantum algorithm for unsupervised machine learning. *arXiv preprint arXiv:1812.03584*, 2018.

[342] Sumsam Ullah Khan, Ahsan Javed Awan, and Gemma Vall-Llosera. K-means clustering on noisy intermediate scale quantum computers. *arXiv preprint arXiv:1909.12183*, 2019.

[343] Hideyuki Miyahara, Kazuyuki Aihara, and Wolfgang Lechner. Quantum expectation-maximization algorithm. *Phys. Rev. A*, 101:012326, Jan 2020.

[344] Andrew Y. Ng, Michael I. Jordan, and Yair Weiss. On spectral clustering: Analysis and an algorithm. In *Proceedings of the 14th International Conference on Neural Information Processing Systems: Natural and Synthetic*, pages 849—-856, 2001.

[345] Ammar Daskin. Quantum spectral clustering through a biased phase estimation algorithm. *TWMS Journal of Applied and Engineering Mathematics*, 10(1):24–33, 2017.

[346] Simon Apers and Ronald de Wolf. Quantum speedup for graph sparsification, cut approximation and Laplacian solving. In *2020 IEEE 61st Annual Symposium on Foundations of Computer Science (FOCS)*, pages 637–648, 2020.

[347] I. Kerenidis and J. Landman. Quantum spectral clustering. *Physical Review A*, 103(4):042415, 2021.

[348] J. S. Otterbach, R. Manenti, N. Alidoust, A. Bestwick, M. Block, B. Bloom, S. Caldwell, N. Didier, E. Schuyler Fried, S. Hong, P. Karalekas, C. B. Osborn, A. Papageorge, E. C. Peterson, G. Prawiroatmodjo, N. Rubin, Colm A. Ryan, D. Scarabelli, M. Scheer, E. A. Sete, P. Sivarajah, Robert S. Smith, A. Staley, N. Tezak, W. J. Zeng, A. Hudson, Blake R. Johnson, M. Reagor, M. P. da Silva, and C. Rigetti. Unsupervised machine learning on a hybrid quantum computer, 2017.

[349] E. Aïmeur, G. Brassard, and S. Gambs. Quantum clustering algorithms.

[350] E. Aïmeur, G. Brassard, and S. Gambs. Quantum speed-up for unsupervised learning. *Mach Learn*, 20:261–287, 2013.

[351] Iordanis Kerenidis, Alessandro Luongo, and Anupam Prakash. Quantum expectation-maximization for Gaussian mixture models. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 5187–5197. PMLR, 13–18 Jul 2020.

[352] Vaibhaw Kumar, Gideon Bass, Casey Tomlin, and Joseph Dulny. Quantum annealing for combinatorial clustering. *Quantum Information Processing*, 17(2), jan 2018.

[353] Seth Lloyd, Masoud Mohseni, and Patrick Rebentrost. Quantum principal component analysis. *Nature Physics*, 10(9):631—633, July 2014.

[354] Chen He, Jiazhen Li, Weiqi Liu, and Z. Jane Wang. A low complexity quantum principal component analysis algorithm, 2020.

[355] CH. Yu, F. Gao, and S. et al Lin. Quantum data compression by principal component analysis. *Quantum Inf Process*, 18:249, 2019.

[356] Jie Lin, Wan-Su Bao, Shuo Zhang, Tan Li, and Xiang Wang. An improved quantum principal component analysis algorithm based on the quantum singular threshold method. *Physics Letters A*, 383(24):2862–2868, 2019.

[357] Armando Bellante, Alessandro Luongo, and Stefano Zanero. Quantum algorithms for data representation and analysis, 2021.

[358] YaoChong Li, Ri-Gui Zhou, RuiQing Xu, WenWen Hu, and Ping Fan. Quantum algorithm for the nonlinear dimensionality reduction with arbitrary kernel. *Quantum Science and Technology*, 6(1):014001, Nov 2020.

[359] Iris Cong and Luming Duan. Quantum discriminant analysis for dimensionality reduction and classification. *New Journal of Physics*, 18(7):073011, July 2016.

[360] Iordanis Kerenidis and Alessandro Luongo. Classification of the MNIST data set with quantum slow feature analysis. *Phys. Rev. A*, 101:062327, Jun 2020.

[361] Seth Lloyd, Silvano Garnerone, and Paolo Zanardi. Quantum algorithms for topological and geometric analysis of data. *Nat. Commun.*, 7(1):1–7, 2016.

[362] Casper Gyurik, Chris Cade, and Vedran Dunjko. Towards quantum advantage via topological data analysis. *arXiv preprint arXiv:2005.02607*, 2020.

[363] Shashanka Ubaru, Ismail Yunus Akhalwaya, Mark S Squillante, Kenneth L Clarkson, and Lior Horesh. Quantum topological data analysis with linear depth and exponential speedup. *arXiv preprint arXiv:2108.02811*, 2021.
[364] Iordanis Kerenidis and Anupam Prakash. Quantum machine learning with subspace states. 2022.
[365] Marcello Benedetti, Delfina Garcia-Pintos, Oscar Perdomo, Vicente Leyton-Ortega, Yunseong Nam, and Alejandro Perdomo-Ortiz. A generative modeling approach for benchmarking and training shallow quantum circuits. *npj Quantum Information*, 5(1), May 2019.
[366] Jin-Guo Liu and Lei Wang. Differentiable learning of quantum circuit Born machines. *Physical Review A*, 98(6), Dec 2018.
[367] Brian Coyle, Maxwell Henderson, Justin Chan Jin Le, Niraj Kumar, Marco Paini, and Elham Kashefi. Quantum versus classical generative modelling in finance. *Quantum Science and Technology*, 6(2):024013, 2021.
[368] Elton Yechao Zhu, Sonika Johri, Dave Bacon, Mert Esencan, Jungsang Kim, Mark Muir, Nikhil Murgai, Jason Nguyen, Neal Pisenti, Adam Schouela, et al. Generative quantum learning of joint probability distribution functions. *arXiv preprint arXiv:2109.06315*, 2021.
[369] Daphne Koller and Nir Friedman. *Probabilistic Graphical Models: Principles and Techniques*. The MIT Press, 2009.
[370] Guang Hao LLow, Theodore James Yoder, and Isaac L Chuang. Quantum inference on Bayesian networks. *Phys. Rev. A*, 89, June 2014.
[371] Robert R Tucci. Quantum Bayesian nets. *International Journal of Modern Physics B*, 09(03):295–337, 1995.
[372] S. E. Borujeni, S. Nannapaneni, N. H. Nguyen, E. C. Behrman, and J. E. Steck. Quantum circuit representation of Bayesian networks. *Expert Systems with Applications*, 2021.
[373] Sima E Borujeni, Nam H Nguyen, Saideep Nannapaneni, Elizabeth C Behrman, and James E Steck. Experimental evaluation of quantum bayesian networks on IBM QX hardware. In *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)*, pages 372–378. IEEE, 2020.
[374] G. Klepac. Chapter 12 – The Schrödinger equation as inspiration for a client portfolio simulation hybrid system based on dynamic Bayesian networks and the REFII model. In *Quantum Inspired Computational Intelligence*, pages 391–416. Morgan Kaufmann, Boston, 2017.
[375] Catarina Moreira and Andreas Wichert. Quantum-like Bayesian networks for modeling decision making. *Frontiers in Psychology*, 7:11, 2016.
[376] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep learning*. MIT Press, 2016.
[377] Geoffrey E Hinton. Training products of experts by minimizing contrastive divergence. *Neural computation*, 14(8):1771–1800, 2002.
[378] Ruslan Salakhutdinov and Geoffrey Hinton. Deep Boltzmann machines. In *Proceedings of the Twelth International Conference on Artificial Intelligence and Statistics*, volume 5, pages 448–455. PMLR, 2009.
[379] Marcello Benedetti, John Realpe-Gómez, Rupak Biswas, and Alejandro Perdomo-Ortiz. Estimation of effective temperatures in quantum annealers for sampling applications: A case study with possible applications in deep learning. *Physical Review A*, 94(2):022308, 2016.
[380] Vivek Dixit, Raja Selvarajan, Muhammad A Alam, Travis S Humble, and Sabre Kais. Training restricted Boltzmann machines with a d-wave quantum annealer. *Front. Phys.*, 9:589626, 2021.
[381] Mohammad H. Amin, Evgeny Andriyash, Jason Rolfe, Bohdan Kulchytskyy, and Roger Melko. Quantum Boltzmann machine. *Phys. Rev. X*, 8:021050, May 2018.
[382] Seth Lloyd and Christian Weedbrook. Quantum generative adversarial learning. *Phys. Rev. Lett.*, 121:040502, July 2018.
[383] Kerstin Beer, Dmytro Bondarenko, Terry Farrelly, Tobias J. Osborne, Robert Salzmann, Daniel Scheiermann, and Ramona Wolf. Training deep quantum neural networks. *Nature Communications*, 11(1), Feb 2020.
[384] Yudong Cao, Gian Giacomo Guerreschi, and Alán Aspuru-Guzik. Quantum neuron: an elementary building block for machine learning on quantum computers. *arXiv preprint arXiv:1711.11240*, 2017.
[385] Shuai Li, Kui Jia, Yuxin Wen, Tongliang Liu, and Dacheng Tao. Orthogonal deep neural networks. *IEEE transactions on pattern analysis and machine intelligence*, 43(4):1352—-1368, April 2021.
[386] Iordanis Kerenidis, Jonas Landman, and Natansh Mathur. Classical and quantum algorithms for orthogonal neural networks. *arXiv preprint arXiv:2106.07198*, 2021.
[387] Jonathan Allcock, Chang-Yu Hsieh, Iordanis Kerenidis, and Shengyu Zhang. Quantum algorithms for feedforward neural networks. *ACM Transactions on Quantum Computing*, 1(1):1–24, 2020.
[388] Edward Farhi and Hartmut Neven. Classification with quantum neural networks on near term processors. *arXiv preprint arXiv:1802.06002*, 2018.
[389] Maxwell Henderson, Samriddhi Shakya, Shashindra Pradhan, and Tristan Cook. Quanvolutional neural networks: powering image recognition with quantum circuits. *Quantum Machine Intelligence*, 2(1):1–9, 2020.

[390] Nathan Killoran, Thomas R. Bromley, Juan Miguel Arrazola, Maria Schuld, Nicolás Quesada, and Seth Lloyd. Continuous-variable quantum neural networks. *Phys. Rev. Research*, 1:033063, Oct 2019.
[391] Henry Liu, Junyu Liu, Rui Liu, Henry Makhanov, Danylo Lykov, Anuj Apte, and Yuri Alexeev. Embedding learning in hybrid quantum-classical neural networks, 2022.
[392] Maria Schuld, Ryan Sweke, and Johannes Jakob Meyer. Effect of data encoding on the expressive power of variational quantum-machine-learning models. *Physical Review A*, 103(3):032430, 2021.
[393] A. Abbas, D. Sutter, C. Zoufal, A. Lucchi, A. Figalli, and S. Woerner. The Power of Quantum Neural Networks. *Nature Computational Science*, 1(6):403–409, 2021.
[394] Maria Schuld. Supervised quantum machine learning models are kernel methods. *arXiv preprint arXiv:2101.11020*, 2021.
[395] Dylan Herman, Rudy Raymond, Muyuan Li, Nicolas Robles, Antonio Mezzacapo, and Marco Pistoia. Expressivity of variational quantum machine learning on the Boolean cube. 2022.
[396] Sofiene Jerbi, Lukas J. Fiderer, Hendrik Poulsen Nautrup, Jonas M. Kübler, Hans J. Briegel, and Vedran Dunjko. Quantum machine learning beyond kernel methods. 2021.
[397] Arthur Jacot, Franck Gabriel, and Clément Hongler. Neural tangent kernel: Convergence and generalization in neural networks. *arXiv preprint arXiv:1806.07572*, 2018.
[398] Kouhei Nakaji, Hiroyuki Tezuka, and Naoki Yamamoto. Quantum-enhanced neural networks in the neural tangent kernel framework. *arXiv preprint arXiv:2109.03786*, 2021.
[399] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
[400] Iordanis Kerenidis, Jonas Landman, and Anupam Prakash. Quantum algorithms for deep convolutional neural networks. *arXiv preprint arXiv:1911.01117*, 2019.
[401] Feihong Shen and Jun Liu. Qfcnn: Quantum fourier convolutional neural network. *arXiv preprint arXiv:2106.10421*, 2021.
[402] Iris Cong, Soonwon Choi, and Mikhail D. Lukin. Quantum convolutional neural networks. *Nature Physics*, 15(12):1273–1278, 2019.
[403] Guillaume Verdon, Trevor McCourt, Enxhell Luzhnica, Vikash Singh, Stefan Leichenauer, and Jack Hidary. Quantum graph neural networks. *arXiv preprint arXiv:1909.12264*, 2019.
[404] Jaeho Choi, Seunghyeok Oh, and Joongheon Kim. A tutorial on quantum graph recurrent neural network (QGRNN). In *2021 International Conference on Information Networking (ICOIN)*, pages 46–49, 2021.
[405] Daoyi Dong, Chunlin Chen, Hanxiong Li, and Tzyh-Jong Tarn. Quantum reinforcement learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, 38(5):1207–1220, 2008.
[406] G. D. Paparo, V. Dunjko, A. Makmal, M. A. Martin-Delgado, and H. J. Briegel. Quantum speedup for active learning agents. *Phys. Rev. X*, 2014.
[407] Samuel Yen-Chi Chen, Chao-Han Huck Yang, Jun Qi, Pin-Yu Chen, Xiaoli Ma, and Hsi-Sheng Goan. Variational quantum circuits for deep reinforcement learning. *IEEE Access*, 8:141007–141024, 2020.
[408] Samuel Yen-Chi Chen, Chih-Min Huang, Chia-Wei Hsing, Hsi-Sheng Goan, and Ying-Jer Kao. Variational quantum reinforcement learning via evolutionary optimization. *Machine Learning: Science and Technology*, 2021.
[409] Owen Lockwood and Mei Si. Reinforcement learning with quantum variational circuits. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, volume 16, pages 245–251. AAAI Press, 2020.
[410] Sofiene Jerbi, Lea M. Trenkwalder, Hendrik Poulsen Nautrup, Hans J. Briegel, and Vedran Dunjko. Quantum enhancements for deep reinforcement learning in large spaces. *PRX Quantum*, 2:010328, Feb 2021.
[411] Daniel Crawford, Anna Levit, Navid Ghadermarzy, Jaspreet S Oberoi, and Pooya Ronagh. Reinforcement learning using quantum Boltzmann machines. *arXiv preprint arXiv:1612.05695*, 2016.
[412] El Amine Cherrat, Iordanis Kerenidis, and Anupam Prakash. Quantum reinforcement learning via policy iteration, 2022.
[413] Arjan Cornelissen. Quantum gradient estimation and its application to quantum reinforcement learning. Master's thesis, Delft, Netherlands, 2018.
[414] Daniel W. Otter, Julian R. Medina, and Jugal K. Kalita. A survey of the usages of deep learning for natural language processing. *IEEE Transactions on Neural Networks and Learning Systems*, 32(2):604–624, 2021.
[415] Usman Naseem, Imran Razzak, Shah Khalid Khan, and Mukesh Prasad. A comprehensive survey on word representation models: From classical to state-of-the-art word representation language models. *ACM Trans. Asian Low-Resour. Lang. Inf. Process.*, 20(5), Jun 2021.
[416] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17, 2017.
[417] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American*

Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers). Association for Computational Linguistics, June 2019.
[418] Justin Sybrandt and Ilya Safro. CBAG: Conditional biomedical abstract generation. *Plos one*, 16(7):e0253905, 2021.
[419] Luciano Floridi and Massimo Chiriatti. GPT-3: Its nature, scope, limits, and consequences. *Minds and Machines*, 30:1–14, 2020.
[420] Su Lin Blodgett, Solon Barocas, Hal Daumé III, and Hanna Wallach. Language (technology) is power: A critical survey of "bias" in NLP. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, July 2020.
[421] Justin Sybrandt, Ilya Tyagin, Michael Shtutman, and Ilya Safro. Agatha: Automatic graph mining and transformer based hypothesis generation approach. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*, pages 2757–2764, 2020.
[422] Payal Dhar. The carbon impact of artificial intelligence. *Nature Machine Intelligence*, 2(8):423–425, 2020.
[423] Bob Coecke, Mehrnoosh Sadrzadeh, and Stephen Clark. Mathematical foundations for a compositional distributional model of meaning. *arXiv preprint arXiv:1003.4394*, 2010.
[424] Joachim Lambek. The mathematics of sentence structure. *Journal of Symbolic Logic*, 33(4):627–628, 1968.
[425] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. In *ICLR: Proceeding of the International Conference on Learning Representations Workshop*, 2013.
[426] Bob Coecke and Eric Oliver Paquette. Categories for the practising physicist. In *New structures for physics*, pages 173–286. Springer, 2010.
[427] Bob Coecke, Giovanni de Felice, Konstantinos Meichanetzidis, and Alexis Toumi. Foundations for near-term quantum natural language processing. *arXiv preprint arXiv:2012.03755*, 2020.
[428] Samson Abramsky and Bob Coecke. Categorical quantum mechanics. *Handbook of quantum logic and quantum structures*, 2:261–325, 2009.
[429] Bob Coecke and Ross Duncan. Interacting quantum observables: categorical algebra and diagrammatics. *New Journal of Physics*, 13(4):043016, Apr 2011.
[430] Giovanni de Felice, Alexis Toumi, and Bob Coecke. DisCoPy: Monoidal categories in Python. *Electronic Proceedings in Theoretical Computer Science*, 333:183–197, Feb 2021.
[431] R. Lorenz, A. Pearson, K. Meichanetzidis, D. Kartsaklis, and B. Coecke. QNLP in practice: Running compositional models of meaning on a quantum computer. *arXiv preprint arXiv:2102.12846*, 2021.
[432] Konstantinos Meichanetzidis, Alexis Toumi, Giovanni de Felice, and Bob Coecke. Grammar-aware question-answering on quantum computers. *arXiv preprint arXiv:2012.03756*, 2020.
[433] Johannes Bausch. Recurrent quantum neural networks. In H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 1368–1379. Curran Associates, Inc., 2020.
[434] Samuel Yen-Chi Chen, Shinjae Yoo, and Yao-Lung L Fang. Quantum long short-term memory. *arXiv preprint arXiv:2009.01783*, 2020.
[435] Yuto Takaki, Kosuke Mitarai, Makoto Negoro, Keisuke Fujii, and Masahiro Kitagawa. Learning temporal data with a variational quantum recurrent neural network. *Physical Review A*, 103(5), May 2021.
[436] Mohiuddin Ahmed, Abdun Naser Mahmood, and Jiankun Hu. A survey of network anomaly detection techniques. *Journal of Network and Computer Applications*, 60:19–31, 2016.
[437] Jarrod West and Maumita Bhattacharya. Intelligent financial fraud detection: A comprehensive review. *Computers & Security*, 57:47–66, 2016.
[438] Guansong Pang, Chunhua Shen, Longbing Cao, and Anton Van Den Hengel. Deep learning for anomaly detection: A review. *ACM Comput. Surv.*, 54(2), March 2021.
[439] Daniel Herr, Benjamin Obert, and Matthias Rosenkranz. Anomaly detection with variational quantum generative adversarial networks. *Quantum Science and Technology*, 6(4):045004, Jul 2021.
[440] T. Schlegl, P. Seeböck, S. M. Waldstein, U. Schmidt-Erfurth, and G. Langs. Unsupervised anomaly detection with generative adversarial networks to guide marker discovery. In *Information Processing in Medical Imaging*. Springer, 2017.
[441] Ming-Chao Guo, Hai-Ling Liu, Yong-Mei Li, Wen-Min Li, Su-Juan Qin, Qiao-Yan Wen, and Fei Gao. Quantum algorithms for anomaly detection using amplitude estimation. *arXiv preprint arXiv:2109.13820*, 2021.
[442] Dongkuan Xu and Yingjie Tian. A comprehensive survey of clustering algorithms. *Annals of Data Science*, 2(2):165–193, 2015.
[443] Shihao Gu, Bryan Kelly, and Dacheng Xiu. Empirical asset pricing via machine learning. *The Review of Financial Studies*, 33(5):2223–2273, 2020.
[444] L. Chen, M. Pelger, and J. Zhu. Deep learning in asset pricing. *SSRN*, 2020.
[445] Stefan Nagel. *Machine Learning in Asset Pricing*. Princeton University Press, 2021.

[446] T. Sakuma. Application of deep quantum neural networks to finance. *arXiv preprint arXiv:2011.07319*, 2020.

[447] Steven A Cuccaro, Thomas G Draper, Samuel A Kutin, and David Petrie Moulton. A new quantum ripple-carry addition circuit. *arXiv preprint quant-ph/0410184*, 2004.

[448] Davide Venturelli and Alexei Kondratyev. Reverse quantum annealing approach to portfolio optimization problems. *Quantum Machine Intelligence*, 1(1-2):17—30, Apr 2019.

[449] Tomas Boothby, Andrew D King, and Aidan Roy. Fast clique minor generation in Chimera qubit connectivity graphs. *Quantum Information Processing*, 15(1):495–508, 2016.

[450] Romina Yalovetzky, Pierre Minssen, Dylan Herman, and Marco Pistoia. NISQ-HHL: Portfolio optimization for near-term quantum hardware. *arXiv preprint arXiv:2110.15958*, 2021.

[451] Stephane Beauregard. Circuit for Shor's algorithm using 2n+3 qubits. *Quantum Info. Comput.*, 3(2):175—-185, Machr 2003.

[452] Mikko Möttönen, Juha J. Vartiainen, Ville Bergholm, and Martti M. Salomaa. Transformation of quantum states using uniformly controlled rotations. *Quantum Info. Comput.*, 5(6):467—473, Sep 2005.

[453] Yonghae Lee, Jaewoo Joo, and Soojoon Lee. Hybrid quantum linear equation algorithm and its experimental test on IBM quantum experience. *Scientific Reports*, 9, 03 2019.

[454] Gadi Aleksandrowicz, Thomas Alexander, Panagiotis Barkoutsos, Luciano Bello, Yael Ben-Haim, David Bucher, Francisco Jose Cabrera-Hernández, Jorge Carballo-Franquis, Adrian Chen, Chun-Fu Chen, et al. Qiskit: An open-source framework for quantum computing. 2019.

[455] R. G. Beausoleil, W. J. Munro, T. P. Spiller, and W. K. van Dam. Tests of quantum information. *US Patent 7,559,101 B2*, 2008.